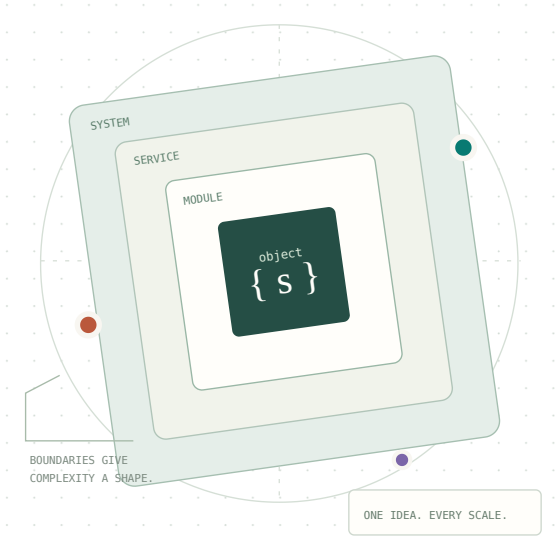
THE SYSTEM DESIGN ATLAS

From objects to *planet-scale systems.*

A system is more than its boxes and arrows. Learn the promises, mechanisms, and trade-offs that connect a single function to a distributed world.

Build the simple version. Find its limit. Understand the next design.



A CONCEPTUAL COVER STUDY · AUTHOR-CREATED

22	99	575	12
UNITS MAPPED	CHAPTERS TO READ	ORIGINAL FIGURES	COMPLETE CASE STUDIES

- **The complete first edition.** All 22 units, 99 chapters, and twelve complete case studies are ready, with worked mechanisms, original figures, exercises, and separate explained solutions. Inspect the coverage → · Download the complete PDF ↓

THE TERRITORY

Six parts. One connected story.

See the concept map ↗

PART 01

System design foundations

16 chapters · 4 units · worked mechanisms, failures and alternatives

Ready to read ↗

PART 02

Low-level design & software architecture

19 chapters · 4 units · worked mechanisms, failures and alternatives

Ready to read ↗

PART 03

Data, storage & scalability

20 chapters · 5 units · worked mechanisms, failures and alternatives

Ready to read ↗

PART 04

Distributed systems & reliability

16 chapters · 4 units · worked mechanisms, failures and alternatives

Ready to read ↗

PART 05

Security & production engineering

16 chapters · 4 units · worked mechanisms, failures and alternatives

Ready to read ↗

PART 06

Complete system design case studies

12 chapters · 1 units · twelve complete design reviews

Ready to read ↗

A PLACE TO BEGIN / UNIT 01

Before you scale a system, learn to *see one*.

Build a to-do application step by step. Save a task, organise its code, choose a design, and follow a request through the system.

Open the foundations unit ↗

1.1 From a program to a system ↗

1.2 Organising code into modules ↗

1.3 From requirements to design choices ↗

1.4 Drawing the system at different levels ↗

01 / THE PROBLEM

Begin where it breaks.

A concept earns its place by solving an observable problem.

02 / THE MECHANISM

Make execution visible.

Inspect the state, ownership, messages, and ordering.

03 / THE JUDGMENT

Keep the trade-off.

Understand what improves, what it costs, and when to choose differently.

PART 1 / UNIT 01

Introduction to System Design

How do we turn a small program into an application people can rely on? We will build a to-do application, one decision at a time.

Begin with a task kept in memory. Save it between runs, organise the code into modules, and work out what the application must do. Then draw the design and trace a request through it.

The running example

Design a basic to-do application. Begin with a single program and progressively identify modules, persistent storage, a web API, and deployment boundaries.

Unit 1 assessment → Explained solutions →

Concept coverage

From a program to a system

Prerequisites: Basic programming 4 original figures

Reading the figures: boxes name parts or states; arrow labels explain how they relate. Dashed arrows show a code dependency or a change in the level of detail. Each caption explains the example.

Imagine that you have written a small to-do program. You enter “Read the design notes,” and the program adds it to your list. You close the program. When you open it the next morning, the list is empty.

The program could add a task, but it could not keep one. To make it useful, we must decide where tasks are stored and what should happen when the program stops. These are our first system design questions.

In this chapter: We will identify the parts of this application, follow a change in its data, and decide when it is safe to display “Saved.”

1.1.1 · What makes a program part of a system?

A **software system** consists of parts that work together to perform a task. In our example, the program accepts a title, checks it, stores a task, and displays the list. Once we add permanent storage, the program and the stored data must work together.

It helps to state what we are designing. Our first application is for one person using one machine. The person can add tasks, list them, and mark them complete. A task reported as saved must still be present after the program restarts. For now, we assume that the machine’s disk remains intact.

We can draw a line around the application and its storage. This is the **system boundary**: it tells us which parts we are considering together. The person uses the system from outside this line.

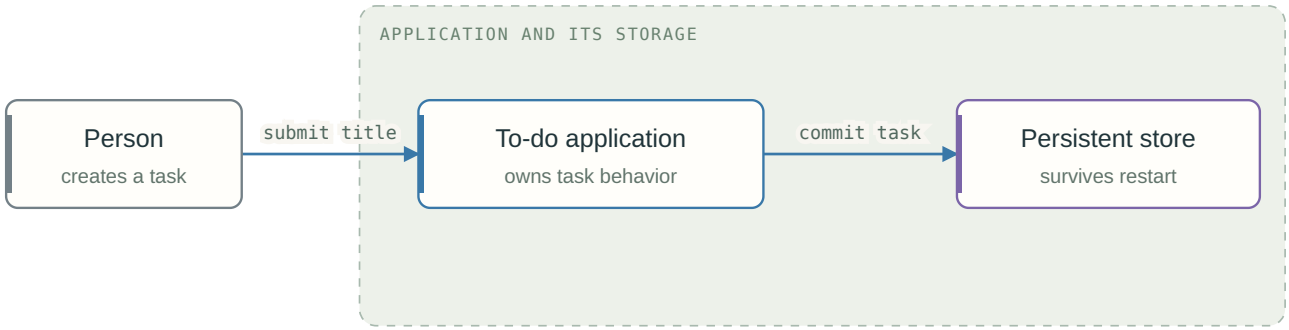


FIG. 1.1.1 **The application and storage work together**

The application needs storage to keep a task after restart. We include both inside the boundary when discussing that requirement.

Why this view: Include the parts needed to explain the behaviour you are studying. Author-created illustration.

The boundary depends on the question. To investigate a slow function, we may study only the running program. To investigate a missing task, we also need to examine storage. When drawing a boundary, state which question it helps answer.

System design is the work of deciding how these parts should be organised and how they should interact to meet the requirements. For our application, this includes choosing where to store tasks, which code checks their titles, and when to tell the user that a task has been saved.

> START WITH THE USER'S EXPECTATION
>
> If the screen says "Saved," what should the user be able to rely on?

1.1.2 · Follow one task through the program

Let us first keep tasks in memory. Memory holds the data used by a running program. In this example, the program creates a new, empty dictionary each time it starts.

The following Python code assumes that calls run one at a time, that the title is text, and that the caller supplies a positive integer identifier. It checks the title length and prevents two tasks from using the same identifier.

```
tasks = {}

def create(task_id, title):
    title = title.strip()
    if not 1 <= len(title) <= 200:
        raise ValueError("title must contain 1–200 characters")
    if task_id in tasks:
        raise ValueError("task id already exists")

    tasks[task_id] = {"title": title, "done": False}
    return {"id": task_id, **tasks[task_id]}

print(create(7, "  Read  "))
# {'id': 7, 'title': 'Read', 'done': False}
```

The call `create(7, " Read ")` first removes spaces from the ends of the title. It checks that the resulting title is between 1 and 200 characters long and that identifier 7 is unused. It then adds one task and returns its details.

Before the call, `tasks` is empty. After the call, it contains task 7. This is a **state change**. The state of a system is the information it currently holds that affects what it will do next. If we now ask for the task list, task 7 will appear because the state has changed.

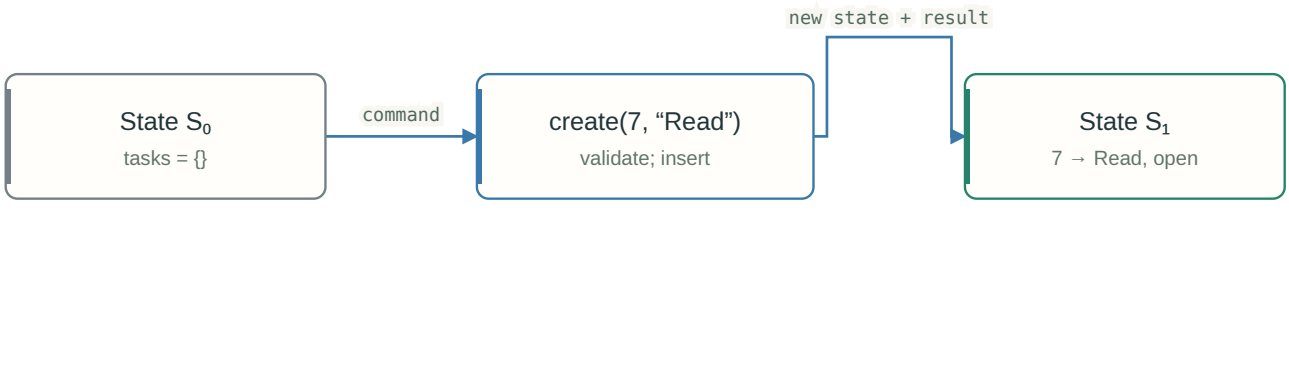


FIG. 1.1.2 **Creating a task changes the stored state**

Before creation, the collection is empty. After create(7, “Read”), it contains task 7, marked incomplete. The function returns the task details.

Why this view: Read the state before and after the operation to see exactly what changed. Author-created illustration.

We can use this one operation to distinguish four related ideas. The title “Read,” the number 7, and the value `False` are **data**. The collection of tasks currently stored is **state**. Trimming and checking the title are **computation**. Passing the title into the function and returning the result are **communication**. Later, communication may also involve files or a network.

Some rules must hold whenever the task collection is valid: each task must have a unique identifier, and each title must contain 1–200 characters after trimming. A rule of this kind is called an **invariant**. If validation fails, the function raises an error before changing the collection.

To complete task 7, we would check that it exists and set `tasks[7]["done"] = True`. Setting that flag to `True` again leaves the task completed. Creating another task is different: it adds another entry. We will return to this distinction when we discuss repeated requests.

<details>
<summary>▼ A closer look · Describing a state change with notation</summary>

We can write an operation as `T(S, c) → (S′, result)`. Here, `S` is the state before the operation, `c` is the command, and `S′` is the state afterward.

For our example, `S` is an empty task collection, `c` is “create task 7 with title Read,” and `S′` contains that task. If the title is empty, the result is an error and `S′ = S`. This notation describes the intended change. By itself, it says nothing about crashes or two callers running at the same time.
</details>

1.1.3 · Decide what “saved” means

The dictionary works while the program is running. When the program restarts, it creates a new dictionary and loses the old list. To meet our restart requirement, we need **persistent storage**: storage that keeps data after the program stops.

For this application, we will use SQLite, a database that can store its data in a local file. The application groups a write into a **transaction**. Completing the transaction successfully is called a **commit**. We will report success only after the database has committed the task.

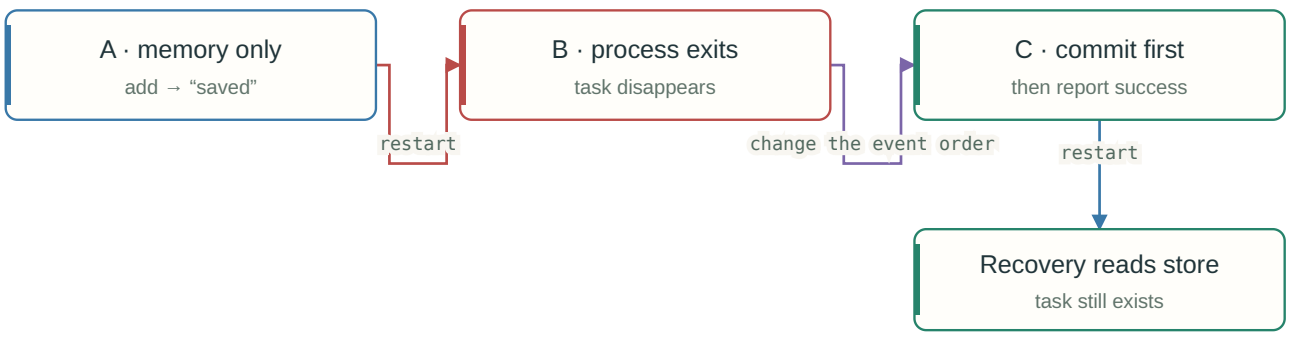


FIG. 1.1.3 **Save before reporting success**

Memory-only tasks disappear after restart. In the revised version, the application commits the task before reporting success. The task remains after a process restart if committed storage is intact.

Why this view: Compare the two versions by restarting the process. Author-created illustration.

The order matters. Suppose the application displays “Saved” and then tries to write the task. If it crashes between these events, the user has been told that a task was saved even though it is missing after restart. Waiting for commit avoids this particular failure.

This is a promise about restarting the program while its committed storage remains intact. If the only disk is destroyed, a local database cannot recover the data from that disk. The SQLite atomic-commit documentation explains how its commit mechanism works and which storage assumptions it relies on.

TRY IT · INTERACTIVE MODEL

What survives a restart?

Choose when the application reports success. Use “Next event” to follow the request, then restart the process and check whether the task remains.

Choice: C · commit before reporting success · 0 events completed

The caller has not received success.

APPLICATION MEMORY	SAVED IN STORAGE	WHAT THE CALLER SEES
Empty	Empty	No success received

Choose “Next event” to continue, or restart to see what remains.

This model follows one request. A commit either finishes completely or does not happen. Restart removes memory but preserves committed storage. No real files are written.

Try the policy that reports success before commit. Advance until the user receives “Saved,” then restart the process. Now repeat with commit before success. The different results come from the order of two events.

The model removes memory when the process restarts and keeps committed storage. It does not simulate a damaged disk or a power failure. The runnable Python example lets you create a task, close the real database connection, and read the task through a new connection.

1.1.4 · Give each part a clear contract

A **component** is a part of the system with a defined responsibility. We will soon separate the code that accepts input, the code that applies task rules, and the code that stores tasks. These components can all run inside the same program.

Components need a way to use one another. An **interface** describes the operations they offer, such as `create(title)` or `list_tasks()`. A **contract** explains how those operations behave.

For example, the name `create(title)` does not tell us whether blank titles are accepted or whether the result has already been saved. We must state those rules. The following contract describes the persistent version of our application, which will assign identifiers itself.

Table 1.1.1 · The contract for creating a task

Situation	Required behaviour	How to check it

Situation	Required behaviour	How to check it
The title has leading or trailing spaces.	Remove those spaces before checking and saving it.	Submit “ Read ” and check that the title is “Read.”
The trimmed title is empty or longer than 200 characters.	Reject it without adding a task.	Compare the task list before and after the attempt.
Creation succeeds.	Return the new identifier after commit.	Restart the program and find the task.
The database write fails.	Report the error instead of reporting success.	Make storage fail and inspect the result.
The task is committed, but its reply is lost.	The task remains saved; the caller may be unsure what happened.	Interrupt the reply after commit.

The last row introduces a problem we will examine in Chapter 1.4. Saving a task and delivering its reply are separate events. The application must account for both.

1.1.5 · Look at the same design at two levels

We can now ask two sets of questions. Where does the application store tasks, and how does it communicate with that storage? Within the application, which functions validate a title and perform the write?

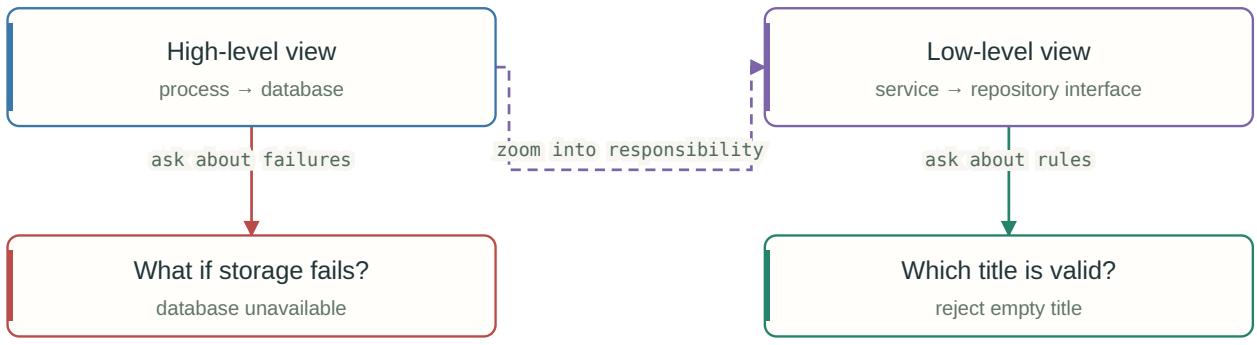


FIG. 1.1.4 **Two levels of detail in one design**

The high-level view shows the application and database. The low-level view shows the service, repository interface, and rules inside the application.

Why this view: Check that both views agree about saving a task before reporting success. Author-created illustration.

High-Level Design (HLD) usually describes the main parts of a system and how they work together. For our application, it shows the user, application, and persistent store. It also states that the application waits for commit before reporting success.

Low-Level Design (LLD) describes how a part is implemented: its functions, classes, interfaces, data structures, and algorithms. Here, it explains how the task service checks a title and calls the storage code.

Software architecture concerns the major choices that shape the system and the reasons for them. Choosing to keep tasks in a local database is one such choice. Different teams use HLD, LLD, and architecture somewhat differently; the useful distinction is the level of detail being discussed.

Both levels must agree. The high-level design may say “save before replying,” but the code must actually wait for the save to finish.

> CHECK YOUR UNDERSTANDING
>
> Putting a dictionary in a class does not make it persistent. Adding a database does not make a reply arrive. A successful restart test does not show what happens when the disk is destroyed. Each result depends on a different part of the design.

Remember: Start with what the application must do. Identify the parts and the data they hold. Then trace the events that must happen before the user receives success.

Before continuing: The application reports “Saved,” then crashes before writing to storage. Which event must move earlier to prevent the task from being lost?

Unit 1 exercises → · Unit 1 explained solutions →

Sources & further reading

These sources explain the ideas and mechanisms used in this chapter. The to-do application is our own teaching example. Links were reviewed on 21 October 2026.

1. Atomic Commit in SQLite ↗

   Official documentation · accessed 21 Oct 2026. Commit and operating-system assumptions.

   `Access reviewed · 21 October 2026`

2. RFC 9110: HTTP Semantics, June 2022 ↗

   Original standard. HTTP semantics; application commit policy is separate.

   Access reviewed · 21 October 2026

Organising code into modules

Prerequisites: 1.1 · From a program to a system 4 original figures

Reading the figures: boxes name parts or states; arrow labels explain how they relate. Dashed arrows show a code dependency or a change in the level of detail. Each caption explains the example.

Our to-do application can now save tasks between runs. We add a web form and a command-line interface. Both contain code that checks the title. Later, we change the title limit from 80 characters to 200, but update only the web form.

The same application now accepts a title through one entry point and rejects it through another. To fix the cause, we need to decide where the rule belongs.

In this chapter: We will divide the application into modules, examine the dependencies between them, and follow a change through the revised design.

1.2.1 · Why does one change require several edits?

A first version of the web handler may do everything in one place: read the request, check the title, write SQL, and build the response. For a small program, this is easy to follow. The difficulty appears when another feature needs some of the same behaviour.

Suppose the command-line interface copies the title check. A CSV exporter also reads the database table directly. We now have two copies of a rule and several places that know how tasks are stored. Changing either the rule or the table can affect several features.



FIG. 1.2.1 **Shared storage details make changes spread**

Several parts read or change the same task data directly. A change to a field or rule may therefore require edits in several places.

Why this view: Follow the arrows to find which parts know about the shared data. Author-created illustration.

Coupling describes the dependencies between parts of a system. In this example, the exporter is coupled to the database layout: it must know the column names. Rename a column, and its query may stop working.

A dependency is often necessary. The task service needs some way to store a task. The design question is how much it needs to know. Calling `repository.insert(title)` requires less knowledge of storage than constructing SQL and opening database files throughout the application.

Cohesion describes how closely the work inside a module belongs together. Checking task titles and completing tasks serve a shared purpose. Password handling, CSV formatting, and database connection setup have different purposes; placing them in a single “helpers” module gives that module little focus.

Coupling asks how one module depends on another. Cohesion asks whether the work within one module belongs together. These ideas help us decide where to put code.

1.2.2 · Separate input, task rules, and storage

Let us give the application three main responsibilities.

- The **input adapter** translates an incoming request into a call the application understands. A web adapter reads an HTTP request; a command-line adapter reads command-line input.
- The **task service** applies task rules. It checks the title and asks storage to create a task.
- The **repository** provides operations for storing and retrieving tasks. Our SQLite implementation contains the database queries and commits each change before returning.

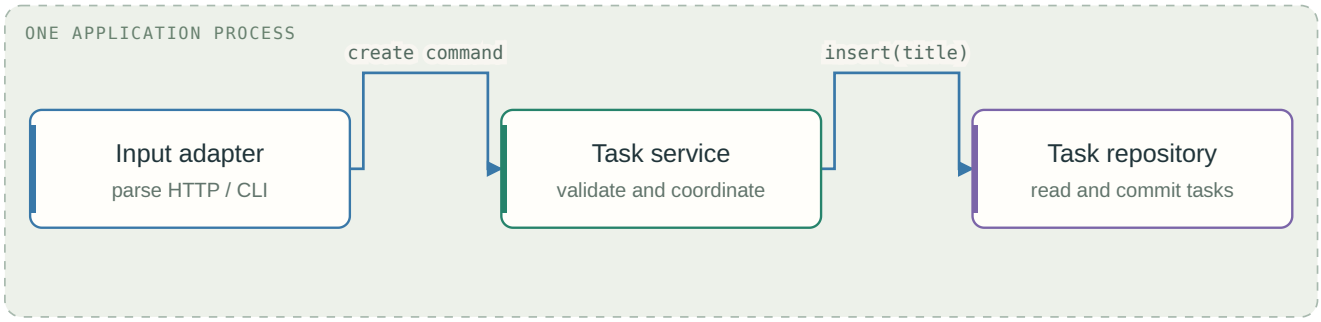


FIG. 1.2.2 **Separate responsibilities inside one process**

The adapter reads input, the service checks task rules, and the repository saves and reads tasks. These parts all run inside one application process.

Why this view: Each kind of work has a clear place in the code. Author-created illustration.

This division is an example of **separation of concerns**: giving different kinds of work their own place. The web adapter handles web details, the service handles task rules, and the repository handles storage details.

Here is the essential structure. This is pseudocode, so it shows the responsibilities without implementing a complete web server.

```
interface TaskRepository:
    insert(title) -> saved Task, or StorageError
    list() -> list of Task records
    set_done(id) -> saved Task, or NotFound, or StorageError

class TaskService:
    constructor(repository):
        self.repository = repository

    create(raw_title):
        title = trim(raw_title)
        require 1 <= length(title) <= 200
        return self.repository.insert(title)

handle_http(request, service):
    title = parse_json(request.body)["title"]
    task = service.create(title)
    return response(201, json(task))
```

Follow a request through this code. The web adapter reads the title from JSON. The service checks it. The repository inserts and commits the task. The saved task returns through the service to the adapter, which builds the response.

The command-line adapter can call the same `service.create(title)`. When the title limit changes, we update the shared rule. Each adapter still checks its own input format: for example, whether a web request contains valid JSON. Task rules and input-format rules have different jobs.

The real Python example implements the service and repository. It also checks that a title is text. A complete HTTP adapter would additionally handle missing fields, invalid JSON, and storage errors.

1.2.3 · Hide details that callers do not need

The service needs to request a saved task. It does not need to know the table name, SQL statement, or file path used to store it. The repository presents a simpler operation, `insert(title)`, while hiding those details. This is **abstraction**.

The contract still matters. In our design, `insert` must return only after commit. The service relies on that behaviour when it returns success. An abstraction should hide implementation details while explaining the behaviour callers depend on.

> ONE OPERATION, SEVERAL HIDDEN STEPS
>
> The service asks “store this task.” The repository handles the query and commit needed to carry out that request.

Encapsulation controls access to a component’s internal data. If every part of the application can change the repository’s records directly, it can bypass the rules we have just defined. Returning a separate task record, rather than a reference that callers can use to edit internal state, helps preserve that control.

Modularity is the organisation of a system into parts with clear responsibilities and interfaces. Separate files can help, but the division is useful only if it limits what each part must know about the others. Ten files that all edit the same internal data can still be difficult to change.

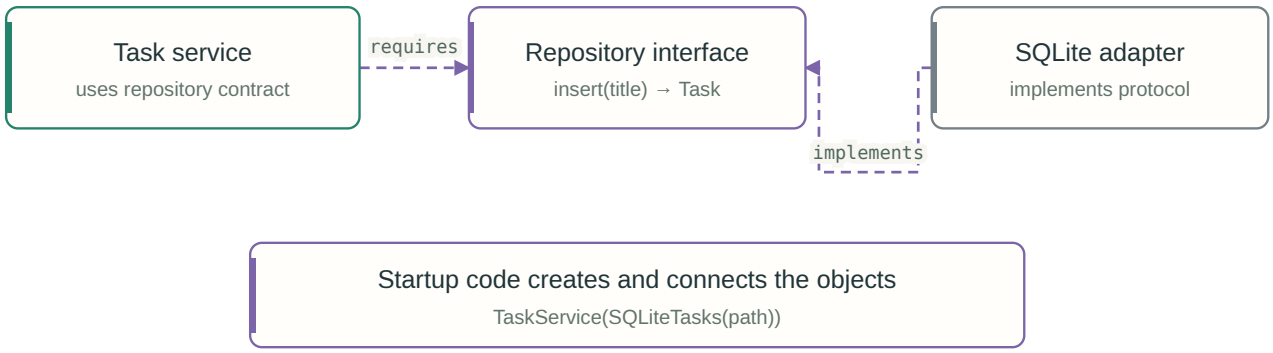


FIG. 1.2.3 **The service uses a repository contract**

The service uses the repository interface. The SQLite adapter implements that interface. Startup code creates the adapter and passes it to the service; during execution, the service calls it.

Why this view: The dashed arrows show code dependencies. These objects can run in the same process. Author-created illustration.

At application startup, we create the SQLite repository and pass it to the service: `TaskService(SQLiteTasks(path))`. Supplying a dependency this way is called **dependency injection**. The startup code that creates and connects these objects is sometimes called the **composition root**. Unit 6 examines these ideas in detail.

Read the dashed arrows in the figure as code dependencies: the service uses the repository contract, and the SQLite adapter implements it. During execution, the service calls that adapter. All of this can happen in one process, without a network call.

1.2.4 · Check the design by changing it

We can judge the division by asking where an edit would be needed. Consider four changes to the application.

Table 1.2.1 · Where should a change be made?

Change	Main place to edit	What else needs checking?
Accept a web form instead of JSON.	Web input adapter.	Does it pass the same title to the service and display errors correctly?

Change	Main place to edit	What else needs checking?
Increase the title limit.	Task service.	Can the database store the longer title? Update any matching database constraint.
Rename a database column.	SQLite repository and the procedure that updates existing data.	Can old records still be read after the change?
Recognise a repeated create request.	Input contract, service, and repository.	Can the application identify the request and recover its earlier result?

The last change affects several parts because it adds new behaviour. A caller must identify a repeated request, and storage must keep enough information to recognise it. Good modules help us see where agreement is needed; they cannot make every change fit inside one file.

TRY IT · INTERACTIVE MODEL

Where would you make the change?

Choose a change and compare the parts that need editing in the two designs.

Scenario: Replace JSON input with a form

Tangled design: Handler · Task logic · SQL inside the handler

Separated design: Input adapter

Update the adapter that reads input. It can pass the same title to the service, which saves it in the same way.

The results follow our example's division of responsibilities. They show where edits are needed, without estimating how long those edits take.

1.2.5 · Understand what a layer hides

We can also arrange the application as a series of **abstraction layers**. Each layer uses the operations offered by the one below it. A user asks to remember a task. The application turns that into a storage operation. The storage implementation uses the operating system to read and write files.

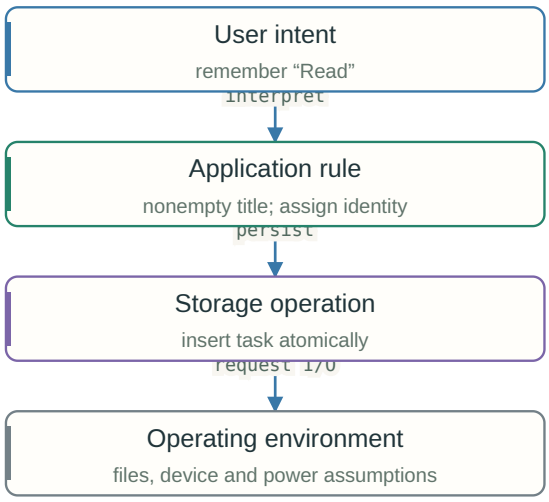


FIG. 1.2.4 **Each layer works at a different level of detail**

The user asks to remember a task. The application applies task rules and requests a storage write. Storage uses files and the underlying device to carry out the write.

Why this view: Follow one request through the layers to connect the user’s action to the work below it. Author-created illustration.

Each step lets us work at a different level of detail. When writing title validation, we do not need to reason about disk blocks. When investigating a damaged database file, those lower-level details may become important.

Some details must be visible in the contract. For example, suppose a repository returns results one at a time and keeps a database connection open until the caller finishes reading. A slow caller can then hold that connection for a long time. The caller needs to know when and how to close the result. Alternatively, the repository could return a bounded page of records and release the connection before returning.

▼ A closer look · Why a test substitute cannot check persistence

We can test title validation using a simple repository that stores tasks in memory. This makes the test quick and easy to control. But the substitute loses its tasks when it is discarded. It cannot demonstrate that SQLite saves a task across a restart.

For that check, use the real repository: create a task, close the connection, open a new connection to the same database, and read the task. The two tests answer different questions. Matching method names does not mean that two implementations have the same behaviour in every situation.

1.2.6 · Use the amount of structure the application needs

In this example, separating the title rule helps because both the web and command-line interfaces use it. Keeping SQL in a repository helps because several features need tasks without needing to know the table layout. These are concrete reasons for the boundaries.

Every added interface also takes work to understand and maintain. A short, temporary script may need only a few functions. As the program grows, introduce a boundary where it protects a rule, hides a storage detail, or allows a real change to be made safely.

The modules still run in one application. If that process crashes, all of them stop. Code organisation improves how we understand, change, and test the application; keeping it running through a machine failure requires further design work.

> COMMON MISTAKES
>
> Separate files do not necessarily have separate responsibilities. An interface does not automatically hide useful details. And fewer dependencies are not always better: the application needs well-defined dependencies to get its work done.

> FURTHER READING
>
> Parnas's 1972 paper examines how the same system can be divided into modules in different ways. It asks which design decisions each module should hide. Our storage example gives a small application of that question. Paper record and abstract ↗

Remember: Give related work a clear home. Let other modules use it through a contract. To check the division, follow a likely change and see which parts need editing.

Before continuing: Both the web form and command-line interface check the title limit. Where should the shared rule move, and which checks should remain in each adapter?

Unit 1 exercises → · Unit 1 explained solutions →

Sources & further reading

These sources explain the ideas and mechanisms used in this chapter. The to-do application is our own teaching example. Links were reviewed on 21 October 2026.

1. On the criteria to be used in decomposing systems into modules — D. L. Parnas, 1972 ↗

   Original research · ACM DOI. Module decomposition criteria; metadata and abstract verified.

   `Access reviewed · 21 October 2026`

2. Inversion of Control Containers and the Dependency Injection pattern — Martin Fowler, 2004 ↗

   Original engineering article. Dependency injection and wiring.

   `Access reviewed · 21 October 2026`

From requirements to design choices

Prerequisites: 1.2 · Organising code into modules 4 original figures

Reading the figures: boxes name parts or states; arrow labels explain how they relate. Dashed arrows show a code dependency or a change in the level of detail. Each caption explains the example.

The team asks us to make the to-do application “fast, safe, and easy to maintain.” These are reasonable goals, but they leave important questions unanswered. How fast should a task list appear? Which failures should saved tasks survive? What changes should be easy to make?

Before choosing a design, we need to turn these goals into requirements that people can understand and check.

In this chapter: We will write a small set of requirements, compare storage choices, and record why one choice fits our application.

1.3.1 · State what the application must do

A **requirement** states something the system must do or a condition it must satisfy. “A user can mark a task complete” describes behaviour. “A saved task remains after the application restarts” describes a condition that behaviour must meet.

Requirements about what the system does are called **functional requirements**. Requirements about qualities such as speed, security, and ease of maintenance are commonly called **nonfunctional requirements**. Both influence the design.

Consider a shared version of our application. For this example, assume a team of 20 people, one application machine, and a small budget. During normal use, the application receives at most 5 requests per second. These are teaching assumptions, rather than measurements of a real service.

The team needs to create, list, and complete tasks. Each person’s tasks are private. A successful save must survive an application restart, provided the committed storage remains intact. At least 95% of list requests should finish within 300 milliseconds under an agreed test workload.

The timing requirement gives us something to measure. If a test sends 1,000 list requests, at least 950 must finish within 300 milliseconds. To make results comparable, we must also specify where the test runs, how many tasks it lists, and how much traffic it sends. Unit 2 develops this measurement process.

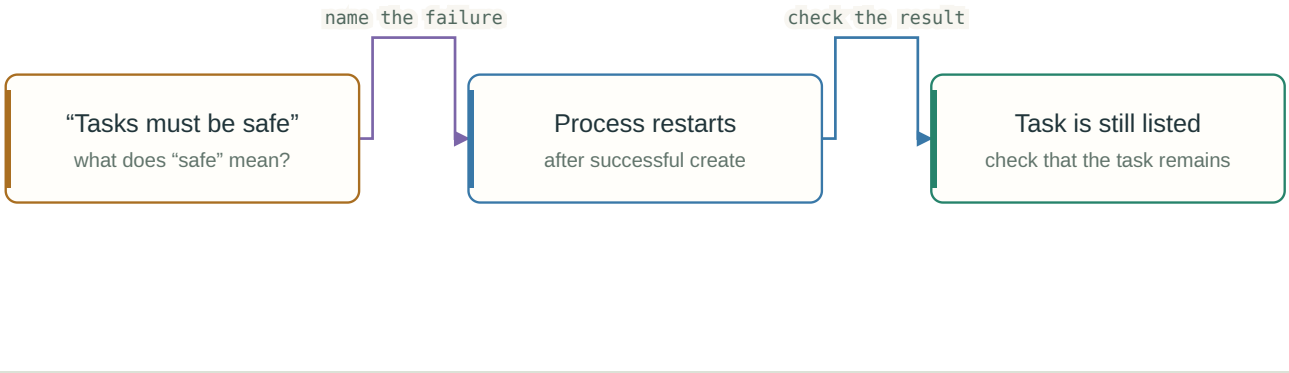


FIG. 1.3.1 **Turn “safe” into a checkable requirement**

Create a task successfully, restart the process, and check that the task remains. This requirement concerns a process restart with committed storage intact.

Why this view: A specific requirement tells us which test to run. Author-created illustration.

Table 1.3.1 · Making a requirement precise

Starting statement	Question to ask	A requirement we can check
“The app is fast.”	Which operation, under how much load?	At least 95% of list requests finish within 300 ms under the agreed workload.
“Tasks are safe.”	Which failure should they survive?	A task reported as saved can be read after the process restarts, with committed storage intact.
“Tasks are private.”	Who is allowed to do what?	Users cannot read or change another person’s tasks through any task endpoint.
“The code is easy to change.”	Which change should be easy?	Add a command-line interface that reuses the task service and repository.

Qualities such as performance, security, reliability, and maintainability are also called **quality attributes** or **architectural characteristics**. Their names help organise a review. A concrete scenario tells us what each one means for this application.

Some requirements fit more than one category. Checking whether a user may complete a task is application behaviour and part of its security. The classification is useful for organising work; the important step is to implement and check the requirement.

1.3.2 · Separate constraints, assumptions, and risks

Suppose the team says that the first version must run on an existing machine. That limits the choices available to us. Such a limit is a **constraint**. A fixed budget or a delivery deadline can also be a constraint.

We have assumed that the disk remains intact when the process restarts. An **assumption** is something we treat as true while making a decision. Writing it down makes clear where the reasoning applies.

Now suppose the only disk fails. We may lose the saved tasks. This is a **risk**: a possible event that could prevent the system from meeting its needs. The team must decide whether to accept it or change the design, for example by keeping recoverable copies elsewhere.

These terms answer different questions. A constraint asks what limits our choices. An assumption asks what our reasoning relies on. A risk asks what could go wrong. They often connect: relying on one intact disk exposes us to the risk of losing that disk.

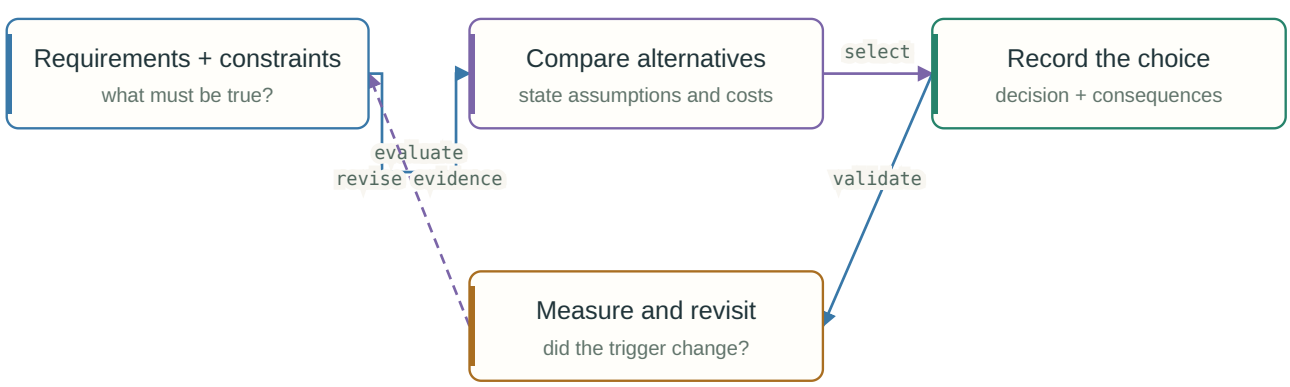


FIG. 1.3.2 **Choose, record, and review a design**

Write down the requirements, compare choices, and record the decision. Later, use measurements and changed requirements to decide whether the design still fits.

Why this view: Keep the reasons for a decision alongside the decision itself. Author-created illustration.

1.3.3 · Compare choices against the requirements

We have three storage candidates. We could keep tasks only in memory, use an embedded database such as SQLite, or run a separate database service. An **embedded database** runs as part of the application rather than as a separate server; here, it stores data in a local file.

First check the mandatory restart requirement. Memory alone fails it. However easy the code is to write, that option cannot meet the agreed need.

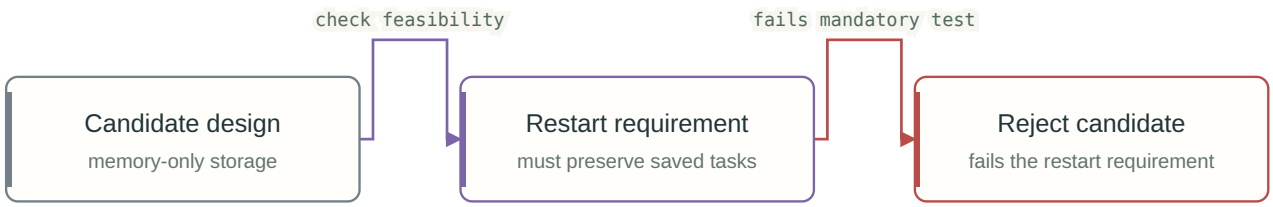


FIG. 1.3.3 **Check requirements before scoring preferences**

Memory-only storage loses tasks after restart, so we reject it for this application. We compare preference scores only for choices that meet the required behaviour.

Why this view: An easy-to-run choice is useful only if it meets the requirements. Author-created illustration.

The other two candidates can meet this requirement with suitable commit behaviour and storage settings. We can then compare their costs and benefits. Embedded storage keeps deployment small. A separate database can be accessed by multiple application machines, but also needs a network connection, access controls, and operating work.

A choice between competing benefits and costs is a **trade-off**. Which cost is worth paying depends on the requirements and on the people who will operate the system.

Table 1.3.2 · Example preference scores: 1 = poor fit, 5 = strong fit

Candidate	Survives process restart?	Simple to run here	Supports multiple app machines	Storage separate from app machine
Memory only	No; reject.	5	1	1

Candidate	Survives process restart?	Simple to run here	Supports multiple app machines	Storage separate from app machine
Embedded store	Yes, under our storage assumptions.	5	2	1
Separate database	Yes, with suitable commit settings.	2	4	4

The scores express judgments about fit for this example. They are not performance measurements. The low score for separating an embedded store from its application machine reflects our local-file setup, rather than a claim about every possible use of SQLite.

We can assign a **weight** to each preference. A higher weight means that preference matters more to the decision. A weighted score multiplies each preference score by its weight, adds the results, and divides by the sum of the weights.

If simplicity has weight 5 and the other two preferences have weight 1 each, the embedded store receives `(5×5 + 1×2 + 1×1) / 7 = 4.00`. The separate database receives `(5×2 + 1×4 + 1×4) / 7 ≈ 2.57`. With these priorities, the embedded store ranks higher.

TRY IT · INTERACTIVE MODEL

Which preferences matter most?

Weights (simplicity, multiple app machines, separate storage): 5, 1, 1.



Embedded store has the higher score with these weights. These are preference scores; they do not measure speed or availability.

Memory only: rejected because tasks disappear after restart.

The scores come from Table 1.3.2. A higher weight gives that preference more influence. Every acceptable choice must still preserve tasks across a process restart.

Try increasing the importance of multiple application machines and separate storage. The ranking may change. This is useful: it reveals how the decision depends on priorities. The memory-only option still fails

the restart requirement, regardless of its score.

1.3.4 · Record the decision and its reasons

A few months later, someone may ask why the team chose a local database. A short **Architecture Decision Record (ADR)** can preserve the answer. It records the situation, the decision, and the consequences. It should also say when the team needs to reconsider.

ADR 001 · ACCEPTED FOR THIS EXAMPLE

Use a local database and commit before reporting success

Situation

A small team uses one application machine. Tasks must survive a process restart. The current requirement does not include keeping the service available after that machine fails.

Decision

Use SQLite on the application machine. Keep input handling, task rules, and storage code in separate modules. Wait for commit before returning success.

Alternatives

Memory alone fails the restart requirement. A separate database is possible, but its extra operating work is not yet justified by the requirements.

Consequences

Deployment is small, and the application has a clear point at which a task is committed. Losing the machine stops the service. Losing its only disk can lose data. We still need backups, restore checks, access controls, and monitoring.

Reconsider when

The application must run on several machines, measured storage contention causes unacceptable delays, or tasks must survive loss of the machine or disk.

Checks to run

Create a task and restart the process. Simulate a storage error. Restore a backup. Measure request times with representative data and traffic.

An ADR need not be long. It needs enough detail for a later reader to understand why the choice made sense. Michael Nygard's article on documenting architecture decisions describes this practice.

If the requirements change, keep the old record and write a new one that replaces it. The earlier decision helps explain which assumption or need has changed.

1.3.5 · Distinguish the qualities we are checking

Some quality terms sound interchangeable. We need to separate them because they require different checks.

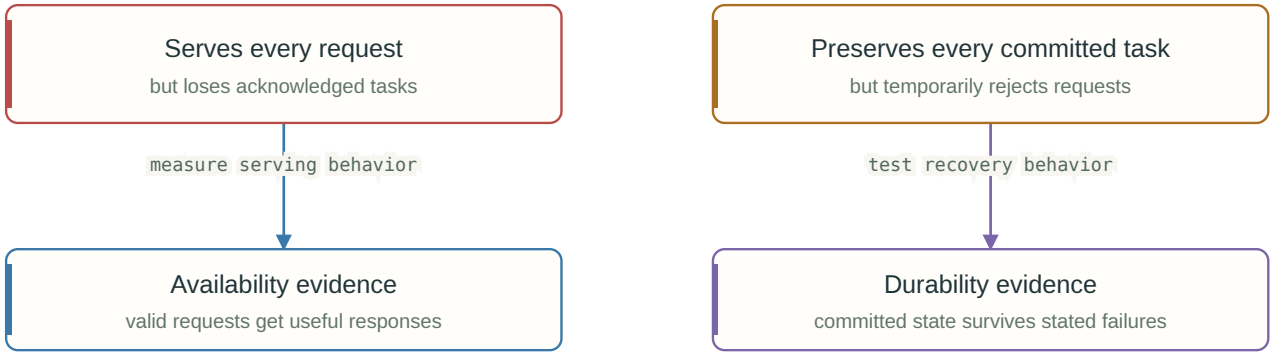


FIG. 1.3.4 **Availability and durability answer different questions**

A service may reply to requests yet lose tasks after restart. Another may keep every committed task but stop accepting requests while it recovers.

Why this view: Check both whether requests succeed and whether saved data survives the stated failure. Author-created illustration.

Availability concerns whether the system can provide the required service when needed. If users cannot list their tasks while the application restarts, it is unavailable for that operation during that interval. When measuring availability, define which requests count and what a useful response is.

Durability concerns whether committed data survives specified failures. Our restart test checks whether a saved task remains after the process stops and starts. It says nothing about losing the only disk.

Reliability usually refers more broadly to providing correct service over time. Teams sometimes use it differently, so a requirement should explain the failures and outcomes it covers.

Consistency concerns which observations of state the system allows. For example, after completing a task, should the user's next list request show it as complete? If the application later has several copies of the data, we must specify when each copy may show the change. Unit 14 examines precise consistency models.

These distinctions explain two possible outcomes. An application may keep replying to requests yet lose tasks it held only in memory. A database may preserve every committed task while refusing requests during recovery. The first has a data-loss problem; the second has a period of unavailability.

▼ A closer look · What makes a design simple enough?

Count the work needed to understand, change, operate, and recover the system. One large function may use few components yet be hard to maintain. A separate managed database may add a network connection while reducing backup work for a team.

The simplest sufficient design meets the agreed requirements with manageable work. Low traffic does not remove the need for access controls or correct saves. At the same time, an imagined future need does not automatically justify extra infrastructure today. Record likely changes and the conditions that would make them necessary.

COMMON MISTAKES

A high preference score cannot compensate for a failed requirement. More users do not, by themselves, tell us which part needs scaling. And a good uptime figure does not show whether saved data survives a failure.

Remember: Write requirements that can be checked. Reject choices that fail them. Compare the remaining choices, then record the reasons and the conditions for reconsidering.

Before continuing: The team still has only 20 users, but it now requires tasks to survive destruction of the only disk. Which part of ADR 001 must be reconsidered?

Unit 1 exercises → · Unit 1 explained solutions →

Sources & further reading

These sources explain the ideas and mechanisms used in this chapter. The to-do application is our own teaching example. Links were reviewed on 21 October 2026.

1. Documenting Architecture Decisions — Michael Nygard, 2011 ↗

   Original practice article. ADR structure and decision history.

   Access reviewed · 21 October 2026

2. AWS Well-Architected: the six pillars ↗

   Official guidance · accessed 21 Oct 2026. Review lenses, not correctness guarantees.

   Access reviewed · 21 October 2026

Drawing the system at different levels

Prerequisites: 1.3 · From requirements to design choices 4 original figures

Reading the figures: boxes name parts or states; arrow labels explain how they relate. Dashed arrows show a code dependency or a change in the level of detail. Each caption explains the example.

A diagram shows three boxes: input adapter, task service, and repository. A reviewer asks whether the application needs three servers. The author intended all three to be modules inside one program.

Both readers saw the same boxes but gave them different meanings. A useful system diagram must make its level of detail clear.

In this chapter: We will draw the to-do application from several viewpoints, trace a request, and use the drawings to find possible failures.

1.4.1 · Choose a view that answers a question

Suppose we want to explain the application to a new team member. We might begin with the person who uses it and the task system as a whole. A developer changing validation needs a closer view of the modules. Someone investigating a machine failure needs to see where the application and data run.

These are different **views** of the same system. Each selects the details needed for a particular question.

The C4 model names four levels for describing system structure: context, containers, components, and code. Moving through them is like zooming in on the application.

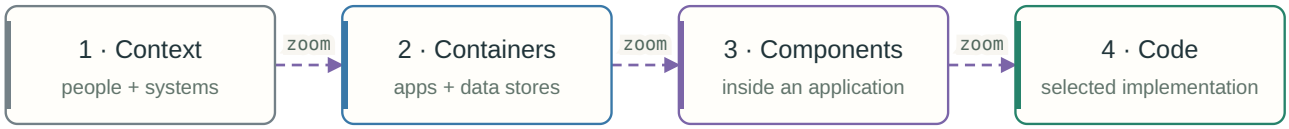


FIG. 1.4.1 **C4: zoom from the system into its code**

Each step shows more internal detail. The dashed arrows mean "zoom in." In C4, a container is an application or data store; it need not be a Docker container.

Why this view: Select the level that helps your reader answer a particular question. Author-created illustration.

Table 1.4.1 · Choosing a diagram

View	Question it answers	What it shows here
System context	Who uses the system, and which other systems interact with it?	A team member uses the task system.
Container	Which applications and data stores make up the system?	Browser client, server application, and task store.
Component	How is an application divided internally?	Input adapter, task service, and storage adapter inside the server application.
Code	How is a selected component implemented?	Task records, service methods, and the repository interface.
Sequence	In what order do events occur in one scenario?	The application validates a request, saves the task, commits, and replies.
Deployment	Where do the application and data run?	One machine runs the application and holds the database file.

A **container** has a particular meaning in C4: an application or data store. It can be a browser application, a server application, or a database. It does not have to be a Docker container. The C4 container definition explains this terminology.

Sequence and deployment views add information about order and location. They support the four structural levels rather than adding more levels of zoom. We can use whichever views help explain the design.

TRY IT · INTERACTIVE MODEL

Zoom in on the same application

Who uses the system?

Team member → uses → Task system

This view shows who uses the system. It leaves the internal parts for a closer view.

Select a level to see which parts it shows. Figure 1.4.1 and Table 1.4.1 explain all four levels without using these controls.

1.4.2 · Show where the program and data live

Our first version was an in-memory script. The second used modules and a local database. Now the team needs to access tasks from several computers, so we add a web interface. A browser sends requests to the server application, which uses the task service and repository.

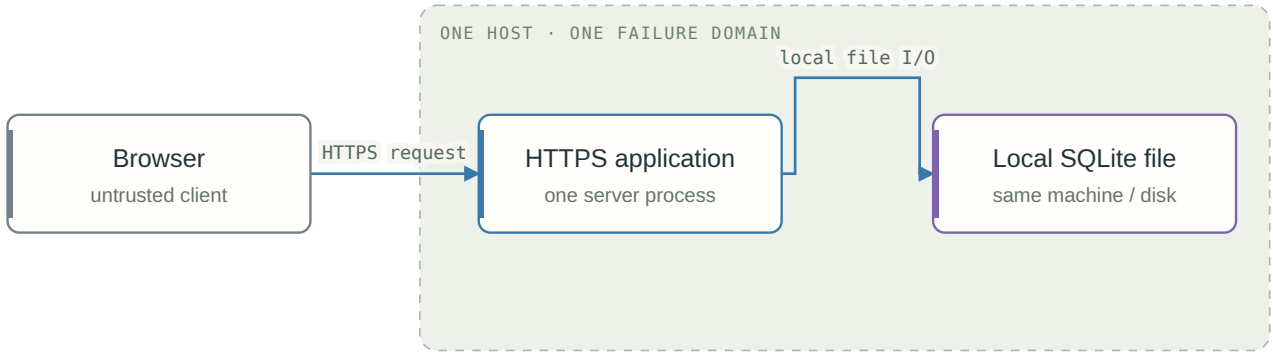


FIG. 1.4.2 **The application and database share one machine**

The browser sends requests to an application on one machine. The database file is on that same machine. If the machine fails, the application and access to the file are both affected.

Why this view: Show physical placement when asking what can fail together. Author-created illustration.

The deployment view shows an important detail: the server application and SQLite file are on the same machine. The browser is elsewhere. If the server machine fails, both the application and access to its database are affected.

Parts that can fail together because of one cause belong to a **failure domain**. Here, the machine is a shared failure domain. Drawing the service and repository as separate boxes does not give them independent machines.

The browser also introduces a **trust boundary**. The server cannot assume that a request is valid merely because it came from the application's usual web page. A user can construct requests directly.

The server must establish who the caller is; this is **authentication**. It must then check whether that caller may access the requested task; this is **authorization**. Every task operation needs the relevant check. Hiding another person's task from the web page does not prevent a direct request for it. Unit 18 develops these checks in detail.

1.4.3 · Trace a request from input to reply

Let us create a task through the web interface. The browser sends `POST /tasks` with a JSON body containing `{"title": "Read"}`. The following sequence describes the intended API; the local Python example is not an HTTP server.

```
1. Identify the caller from a validated session.
2. Read the title and check the input.
3. Begin a database transaction.
4. Insert the task with the caller as its owner.
5. Commit the transaction.
6. Return:
   201 Created
   Location: /tasks/7
   {"id": 7, "title": "Read", "done": false}
```

The `201 Created` response tells the client that creation succeeded. The `Location` header identifies the new resource. HTTP's specification defines this response. Waiting for the database commit is an additional rule of our application.

Now interrupt the sequence. If title validation fails, the application writes no task. If the transaction fails before commit, the application reports an error. If the process crashes after commit but before the reply, task 7 exists even though the browser has not received success.



FIG. 1.4.3 **The task is saved, but the reply is lost**

Task 7 is committed before its reply is lost. The client cannot tell what happened. A fresh create request may add task 8 unless the application recognises it as a retry.

Why this view: Trace the events in order to see why the client can be unsure even when a task exists. Author-created illustration.

What should the browser conclude after waiting too long for a reply? It cannot tell whether the task was saved. The request might never have reached the application, or the task might have been committed and the reply lost.

If the browser sends a fresh create request, the application may add task 8. The user intended one task but now has two. To recognise retries, the client and server need a stable identifier for the original operation. The server must store that identifier with its result so that a repeated attempt can retrieve the earlier result. Chapter 8.2 develops this mechanism, called request deduplication.

This is why event order belongs in the diagram. A drawing with only “request” and “response” can hide the commit that determines whether a task exists.

1.4.4 · Label arrows and copies of data

An arrow can mean a function call, a network request, a code dependency, or a step in a sequence. Label it with the actual relationship. For example, “HTTPS request,” “calls insert,” and “implements interface” describe different things.

Use a legend when colour or line style carries meaning. In this atlas, dashed arrows often show dependencies or zoom relationships, while solid arrows show the labelled action. Read the labels and caption to find the meaning in that particular view.

Also show where copies of data are held. The browser may display a task list it fetched a minute ago. The application may hold a task record in memory. The database contains the committed record. If another user

completes a task, the browser's old list does not update itself. We must decide when it fetches or receives new data.

A diagram that distinguishes these copies makes the freshness question visible. A single label saying "task data" does not tell us which copy a reader is seeing.

▼ A closer look · Four ways to review a diagram

1. **Stop a component.** What stops with it? If several modules share one process, a process crash affects all of them.
2. **Delay a call.** Who is waiting? Decide how long that caller can wait and what happens when the time runs out.
3. **Repeat a request.** Does it add another task? Point to the code and stored information that recognise a retry, if that behaviour is required.
4. **Change a data format.** Which readers and writers must change together? Check how old and new versions will work during the update.

These questions help uncover omissions. Tests are then needed to check the actual behaviour.

1.4.5 · Change the design when there is a reason

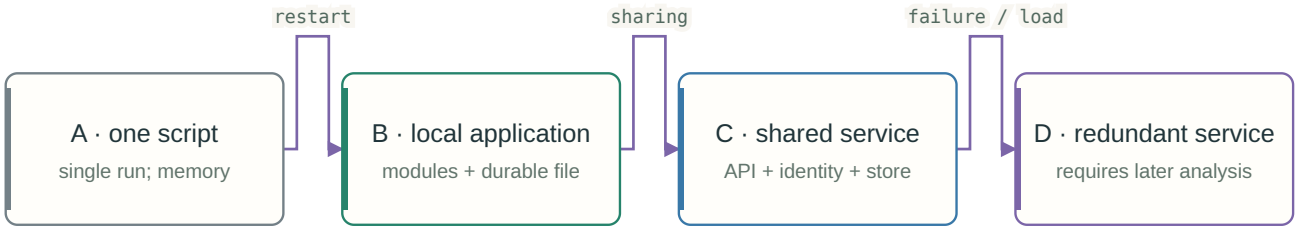


FIG. 1.4.4 **Each design change answers a new need**

Add storage when tasks must survive restart. Add a remote interface when people need shared access. Extra machines require further decisions about data, failures, and load.

Why this view: State the requirement that makes each change necessary. Author-created illustration.

Each change in our example answered a particular need. Persistent storage let tasks survive a process restart. Modules gave shared rules a clear home. A web interface let people use the application remotely. None of these changes made it survive failure of its only machine.

Suppose the application becomes slow. Before adding another server, measure where requests spend their time. A slow database query may need a better query or index. Adding application servers could send even more work to that same query. If computation in the application is the bottleneck and storage has spare capacity, additional workers may help.

Adding a second machine also changes our storage assumptions. If each machine has its own unrelated task database, a user may see different lists depending on which machine handles the request. We need a design for shared or coordinated data; a load balancer alone cannot supply it.

The drawings help us review these consequences together. A structure view assigns responsibilities. A sequence view shows event order. A deployment view shows where execution and data live. They should describe a consistent design.

> COMMON MISTAKES
>
> Three module boxes need not mean three servers. A timeout does not establish that a write failed. And another application machine does not automatically provide another usable copy of the data.

Remember: Choose a diagram for a specific question. Label what the boxes and arrows mean. Check that the code structure, request sequence, and deployment tell the same story.

Before continuing: Which view would help you find a reply sent before commit? Which view would show that the application and its only backup are on the same disk?

Unit 1 exercises → · Unit 1 explained solutions →

Sources & further reading

These sources explain the ideas and mechanisms used in this chapter. The to-do application is our own teaching example. Links were reviewed on 21 October 2026.

1. C4: diagrams ↗

   Original model documentation. Formal diagram levels and supporting views.

   Access reviewed · 21 October 2026

2. C4: container abstraction ↗

   Original model documentation. Meaning of container.

   Access reviewed · 21 October 2026

3. RFC 9110: HTTP Semantics, June 2022 ↗

   Original standard. HTTP semantics; application commit policy is separate.

   Access reviewed · 21 October 2026

PART 1 / UNIT 02

Requirements, Estimation, and Engineering Trade-offs

Opening a feed looks like one action, but it can fetch many images and update several records. Begin by finding out what people need to do and which work each journey causes.

The running example

A social platform has 10 million daily active users. Estimate requests per second, traffic at peak, storage growth, image bandwidth, and approximate infrastructure requirements.

Unit 2 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

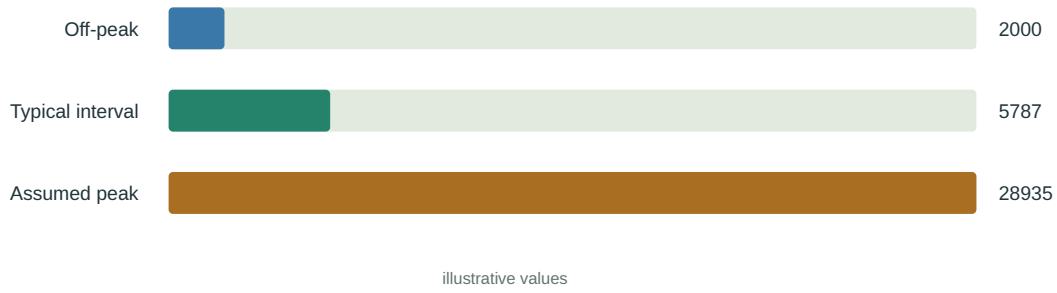


FIG. 2.0.1 **Request-volume timeline**

Request volume across illustrative intervals; these are assumed rates, not a measured daily traffic trace.

Author-created illustration. Numbers are illustrative unless stated otherwise.

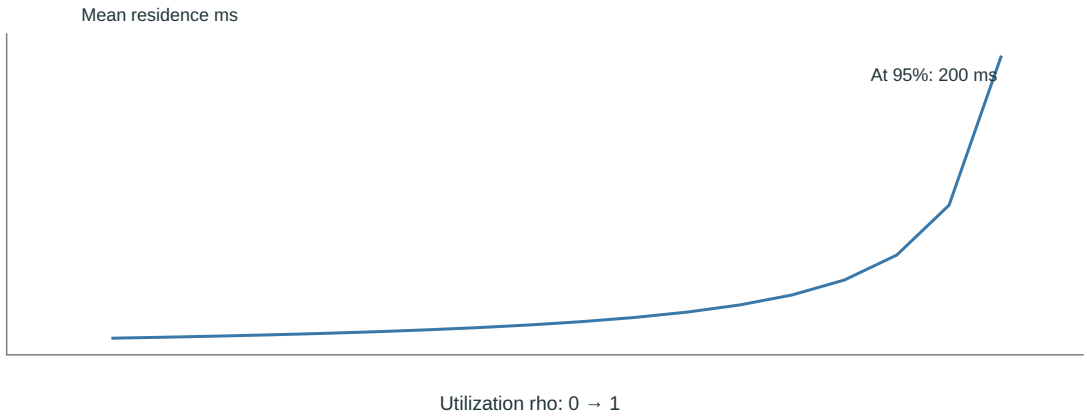


FIG. 2.0.2 **throughput-latency curve**

This single-server M/M/1 model assumes Poisson arrivals and independent exponential service times averaging 10 ms. Mean waiting plus service time rises as utilization approaches one; the curve does not predict p99.

Author-created illustration. Numbers are illustrative unless stated otherwise.

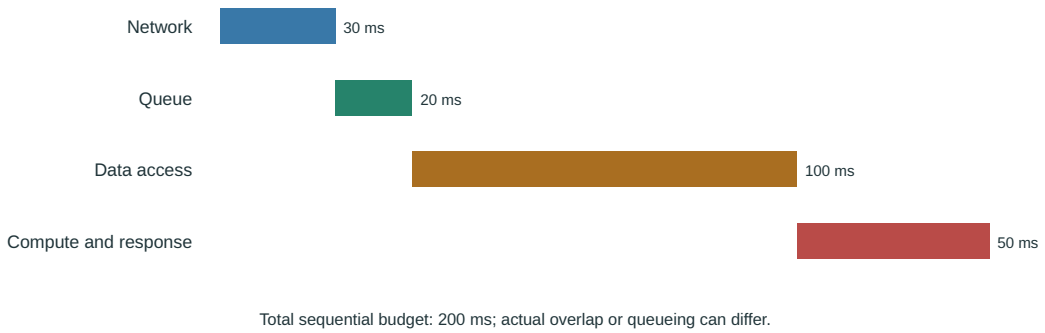


FIG. 2.0.3 **latency waterfall**

A 200 ms sequential budget allocates every stage; real overlap and tails require measurement.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Input	Calculation	Result
Daily API requests	10 million × 50	500 million
Average RPS	500 million / 86400	5787.04
Assumed peak	average × 5	28935.19
Post bytes/day	10 million × 2000	20 GB

FIG. 2.0.4 **capacity planning worksheet**

Explicit workload assumptions create a worksheet, not a benchmark.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Turn user journeys into workloads

Prerequisites: Introduction to System Design 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Opening a feed looks like one action, but it can fetch many images and update several records. Begin by finding out what people need to do and which work each journey causes.

2.1.1 · Gathering and prioritizing requirements

Suppose we are asked to build a social feed. Before estimating servers, we need to know what people will do: publish posts, follow accounts, read updates, or report harmful content. These journeys lead to different work inside the system.

Write the required outcome for each important journey. “Keep an accepted post and show it to followers within five seconds” is specific enough to discuss and test. Then separate essential behaviour from desirable improvements. Investigate requirements whose failure would cause the greatest harm or whose feasibility is uncertain. Record exclusions too: private messaging should not silently enter an estimate for a public feed.

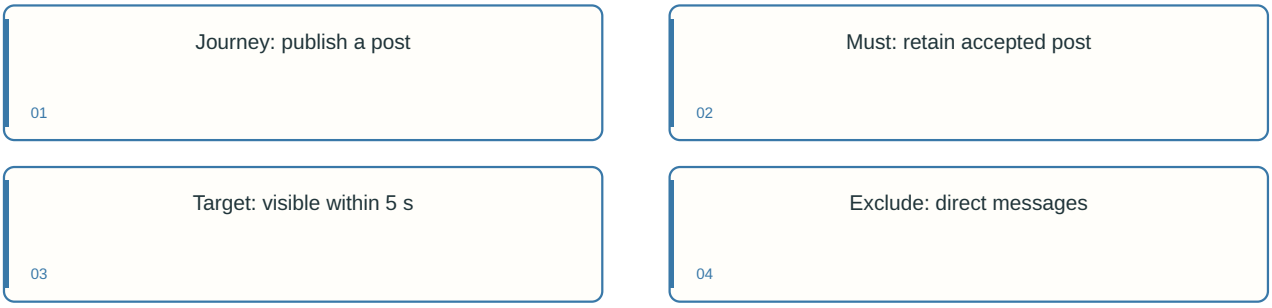


FIG. 2.1.1 **Gathering and prioritizing requirements · the example**

Requirements state necessary outcomes and constraints; prioritisation identifies what matters most and what still needs evidence.

Author-created illustration. Numbers are illustrative unless stated otherwise.

2.1.2 · User journeys and workload characteristics

Opening a feed appears to be one action. In an assumed session, however, the browser sends one feed request, fetches twenty images, and records one analytics event. These requests differ in size and in the work they cause.

A workload describes that mixture of operations and how often they arrive. For each operation, estimate its rate, payload size, and dependencies. A small response can still require expensive computation; an image can require little computation but substantial transfer. Keep these workloads separate when finding bottlenecks. The first estimates are assumptions. Replace them with measurements once the application is running.

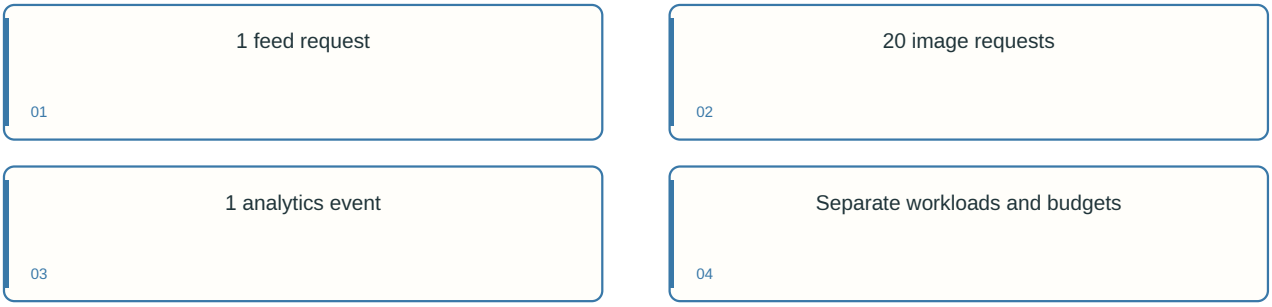


FIG. 2.1.2 **User journeys and workload characteristics · the example**

A workload describes the operations users cause, their frequency, payloads, and dependent work.

Author-created illustration. Numbers are illustrative unless stated otherwise.

2.1.3 · Read-heavy vs write-heavy systems

Assume users read ten feeds for every post they publish. At the public API, this looks read-heavy. But if publishing copies the new post into a thousand follower feeds, that one action causes a thousand additional storage writes.

A read/write ratio therefore needs a boundary: are we counting user actions, API requests, or database operations? Creating follower feeds in advance moves work to publication time. Building them when requested moves work to reading time. The choice depends on how often users publish and read. Read replicas can help some reads, but delayed replicas may not immediately show a user's new post.

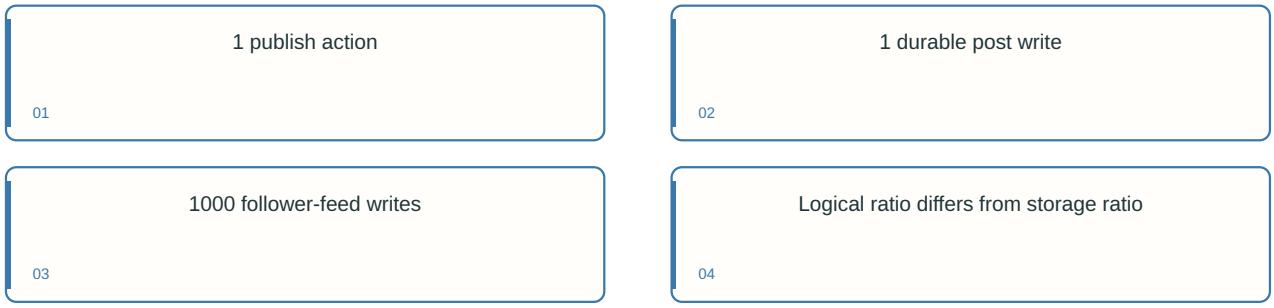


FIG. 2.1.3 **Read-heavy vs write-heavy systems · the example**

A read/write ratio counts work at a specified boundary; one user action can cause many physical operations.

Author-created illustration. Numbers are illustrative unless stated otherwise.

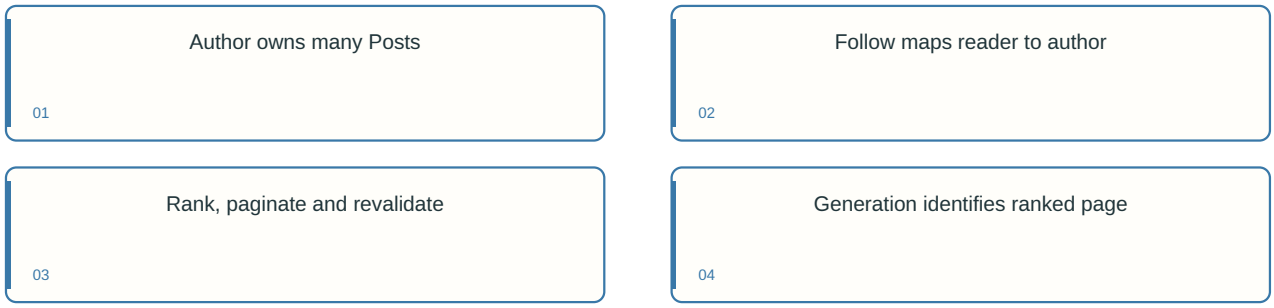


FIG. 2.1.4 **State and ownership in the related design**

The post and access relationships are authoritative; feed candidates are derived references.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · How would simultaneous retries change an estimate based on average user sessions?

Retries add attempts without adding new user intentions. Model their timing, maximum amplification, and the failing dependency separately from normal sessions. A correlated retry wave can exceed a peak factor based on independent arrivals. Bound attempts and admission, then test useful completion and recovery under that burst.

Remember: Count work at the boundary being designed, including operations caused by one user action.

Try applying it: Which requests and storage writes would a new image-upload feature add to the estimate?

Unit 2 exercises → · Unit 2 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Implementing SLOs — The SRE Workbook ↗

   Engineering book. Service indicators, objectives, and error budgets; not a source of the atlas workload numbers.

   `Access reviewed · 21 October 2026`

2. A Proof for the Queuing Formula: L = lambda W — John D. C. Little, 1961 ↗

   Original research · publisher record. Long-run queue accounting; publisher record inspected, full proof access may depend on access rights.

   `Access reviewed · 21 October 2026`

3. The Tail at Scale — Dean and Barroso, 2013 ↗

   Original research. Tail-latency considerations; atlas probabilities and latency examples are author-calculated.

   `Access reviewed · 21 October 2026`

Measure the experience

Prerequisites: 2.1 · Turn user journeys into workloads 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Averages can make a slow service look healthy. Follow offered requests, completed work, individual waits, and errors to choose measurements that reflect what users receive.

2.2.1 · Latency, throughput, QPS, RPS, TPS

Suppose a server receives 200 requests each second and completes 180. If the remaining requests stay queued, twenty more accumulate every second. Reporting only the completion rate would hide the growing wait.

Latency is the time one operation takes. Throughput is the amount of work completed per unit time. RPS usually means requests per second, QPS queries per second, and TPS transactions per second. State which work each number counts, since one request may cause several queries. Measure offered work, successful completions, errors, and rejected requests together. Otherwise, an apparently steady throughput can conceal users who are waiting or giving up.

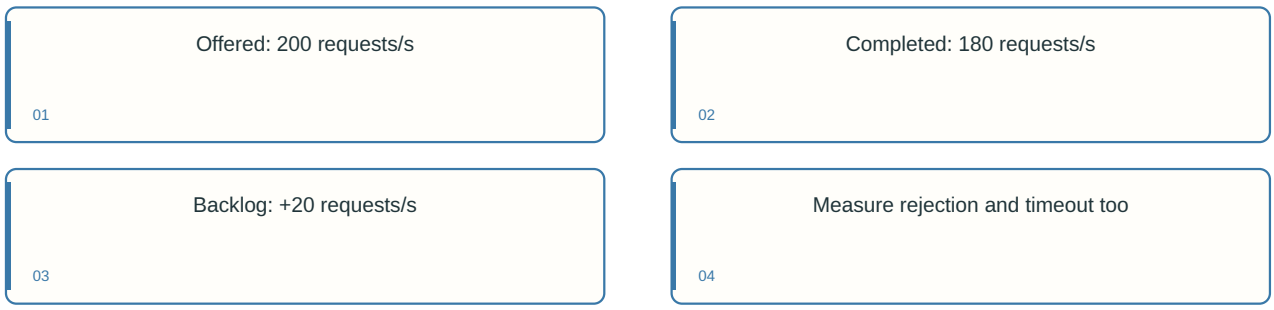


FIG. 2.2.1 **Latency, throughput, QPS, RPS, TPS · the example**

Latency is elapsed time per operation; throughput is completed work per interval. Define which queries, requests, or transactions are counted.

Author-created illustration. Numbers are illustrative unless stated otherwise.

2.2.2 · Average vs median vs p95/p99 latency

Consider 100 assumed request times: 99 requests take 10 milliseconds each, and one takes 1,000 milliseconds. The mean is (99×10 + 1,000)/100 = 19.9 milliseconds. That average tells us little about the user who waited a full second.

The median describes the middle of a sorted set. A percentile such as p95 or p99 describes a point near its upper end; the precise estimate depends on the percentile convention. Use the distribution to study slow requests. To find a fleet-wide p99, combine compatible observations or histograms. Averaging individual servers' p99 values does not produce the fleet p99. Account for timeouts rather than discarding them.

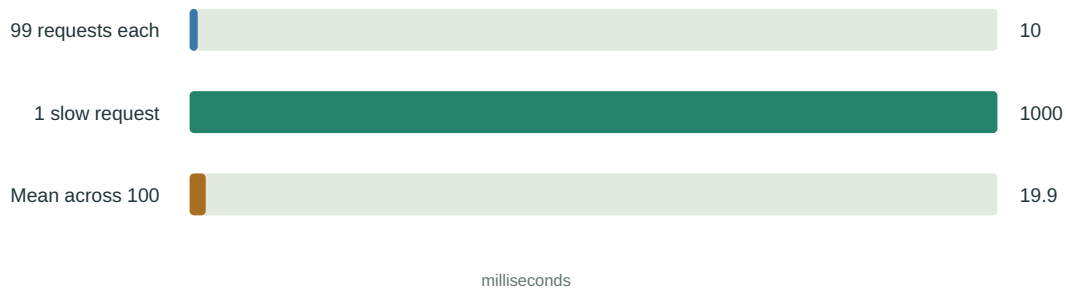


FIG. 2.2.2 **A small mean can hide one slow request**

The mean is (99×10+1000)/100 = 19.9 ms. This is a synthetic distribution, not a percentile convention.

Author-created illustration. Numbers are illustrative unless stated otherwise.

2.2.3 · Availability, reliability, durability, consistency

A payment status page can respond quickly while showing yesterday's information. Its ability to respond and the freshness of its answer are separate properties.

Availability concerns providing the required service when needed. Durability concerns whether committed data survives specified failures. Reliability usually concerns correct service over time; state how it is being measured. Consistency defines which observations of state are allowed. For example, must the next read show a completed payment? Choose the requirement for each operation. A popularity count may tolerate delayed updates, while selling the last ticket requires stronger coordination. A single label such as “reliable database” leaves these decisions unanswered.

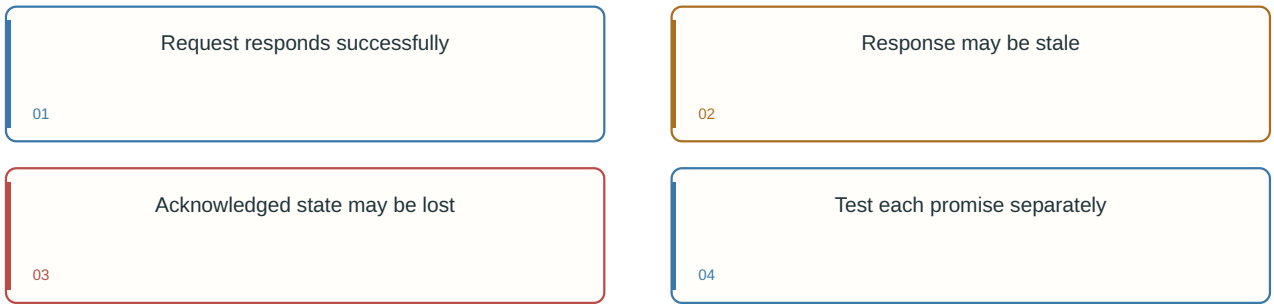


FIG. 2.2.3 **Availability, reliability, durability, consistency · the example**

Availability, durability, reliability, and consistency concern different serving, persistence, correctness, and observation requirements.

Author-created illustration. Numbers are illustrative unless stated otherwise.

2.2.4 · SLI, SLO, SLA, and error budgets

Suppose a service handles one million eligible requests in a month. A target of 99.9% successful requests allows 0.1%, or 1,000, to fail. This allowance is its event-based error budget for that population and period.

An SLI is the indicator we measure, such as the fraction of successful requests. An SLO is its target and measurement window. An SLA is an agreement with stated obligations and possibly consequences. Define eligible requests honestly; excluding failures merely to improve the number defeats the purpose. A time-based objective uses time as its denominator and gives a different budget. Use budget consumption to guide release and reliability work, while also considering the harm of a concentrated outage.

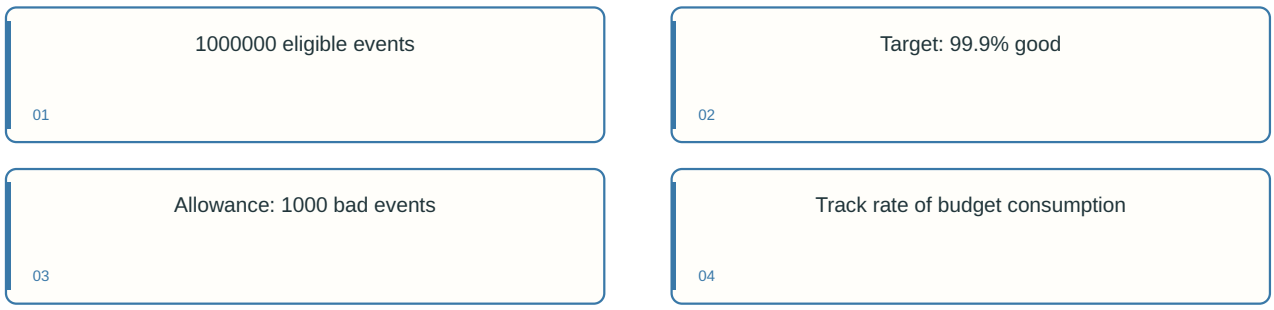


FIG. 2.2.4 **SLI, SLO, SLA, and error budgets · the example**

An SLI measures behaviour, an SLO targets it over a window, and an SLA states agreement obligations.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · How would simultaneous retries change an estimate based on average user sessions?

Retries add attempts without adding new user intentions. Model their timing, maximum amplification, and the failing dependency separately from normal sessions. A correlated retry wave can exceed a peak factor based on independent arrivals. Bound attempts and admission, then test useful completion and recovery under that burst.

Remember: Measure useful outcomes and their distribution, with an explicit population and observation window.

Try applying it: A service reports a low mean while some users wait a second. Which additional evidence would you request?

Unit 2 exercises → · Unit 2 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Implementing SLOs — The SRE Workbook ↗

   Engineering book. Service indicators, objectives, and error budgets; not a source of the atlas workload numbers.

   Access reviewed · 21 October 2026

2. A Proof for the Queuing Formula: L = lambda W — John D. C. Little, 1961 ↗

   Original research · publisher record. Long-run queue accounting; publisher record inspected, full proof access may depend on access rights.

   Access reviewed · 21 October 2026

3. The Tail at Scale — Dean and Barroso, 2013 ↗

   Original research. Tail-latency considerations; atlas probabilities and latency examples are author-calculated.

   Access reviewed · 21 October 2026

Estimate the resources

Prerequisites: 2.2 · Measure the experience 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Suppose ten million people use the service daily. We will turn explicit activity assumptions into requests, stored bytes, transfer, and headroom, keeping each resource's calculation separate.

2.3.1 · Capacity and growth planning

Suppose stored data grows by an assumed 10% each month. Starting with one unit, a year of this growth gives $1.1^{12} \approx 3.14$ units. That is a planning scenario, not evidence that request traffic and CPU usage will also triple.

Plan each important resource separately: stored history, frequently accessed data, background jobs, and spare capacity during failures. Include temporary space needed while old and new data layouts coexist during a migration. A useful capacity plan names the resource likely to fill first, the time needed to add capacity, and the observation that triggers action. Revisit the assumptions as measurements arrive.

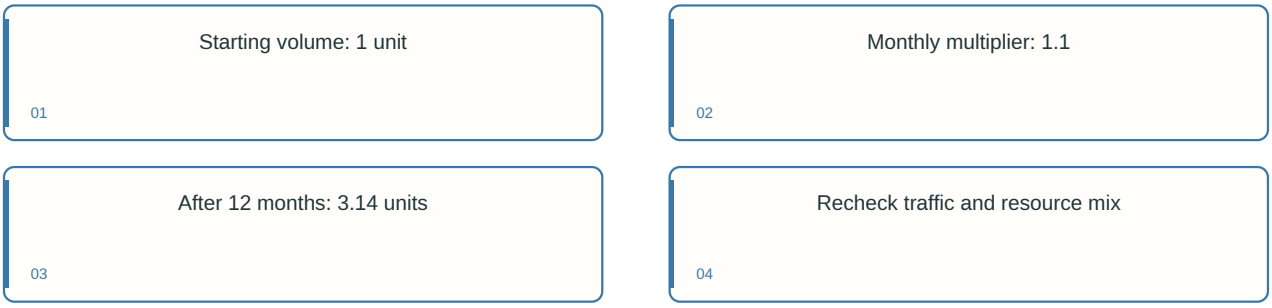


FIG. 2.3.1 **Capacity and growth planning · the example**

Capacity planning connects growth scenarios and resource limits to the time and evidence needed for expansion.

Author-created illustration. Numbers are illustrative unless stated otherwise.

2.3.2 · Storage, memory, bandwidth, and CPU estimates

Assume ten million daily users each make fifty API requests. The daily total is 10,000,000×50 = 500,000,000 requests. Divide by 86,400 seconds per day to obtain about 5,787 requests per second on average.

If each user publishes one 2 kB post daily, raw post growth is 20 GB/day. If each views ten 200 kB images, image delivery totals 20 TB/day. These calculations use decimal units and describe different resources. Add indexes, extra copies, retained history, and transfer overhead separately. Memory depends on the data kept active; CPU depends on the work per request. Neither can be inferred from the user count alone. These are assumed workloads, not company benchmarks.

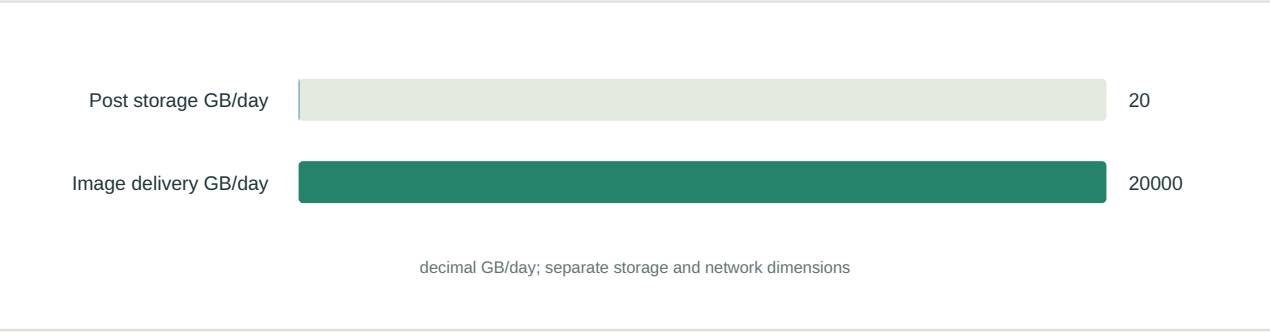


FIG. 2.3.2 **Different workload dimensions have different magnitudes**

Ten million assumed users each write one 2 kB post and view ten 200 kB images. Storage and delivered traffic are separate workloads.

Author-created illustration. Numbers are illustrative unless stated otherwise.

2.3.3 · Peak vs average load

A daily average spreads all requests evenly across 24 hours. Real users may arrive together after a notification or event. Suppose, for planning, that peak API traffic is five times the preceding average. The estimate becomes 5×5,787.04 ≈ 28,935 requests per second.

Also ask how long the peak lasts. Extra instances that take several minutes to start may arrive after a short spike has ended. Admission limits and existing spare capacity must handle the gap. Image traffic and posting traffic may peak differently, so a common multiplier is only an initial assumption. Retries during an outage can create another peak that ordinary user averages miss.

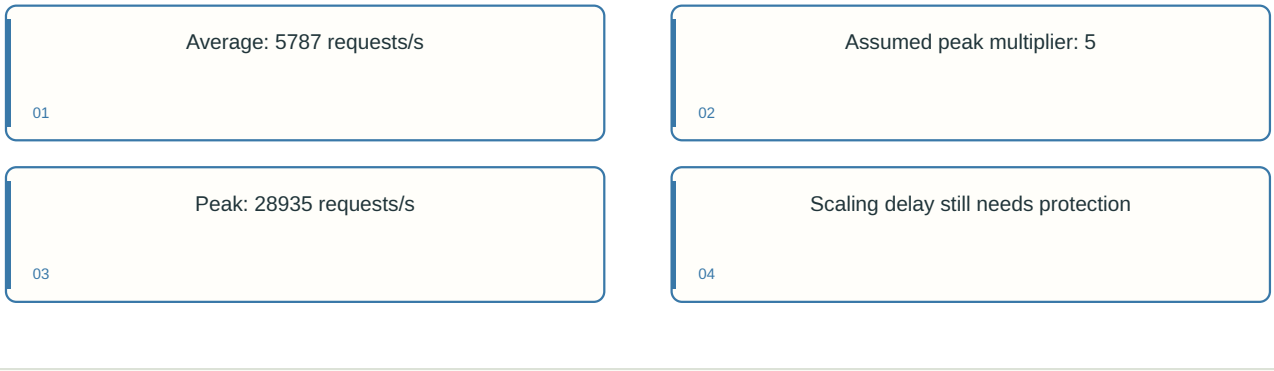


FIG. 2.3.3 **Peak vs average load · the example**

Peak load and its duration can differ substantially from an evenly distributed daily average.

Author-created illustration. Numbers are illustrative unless stated otherwise.

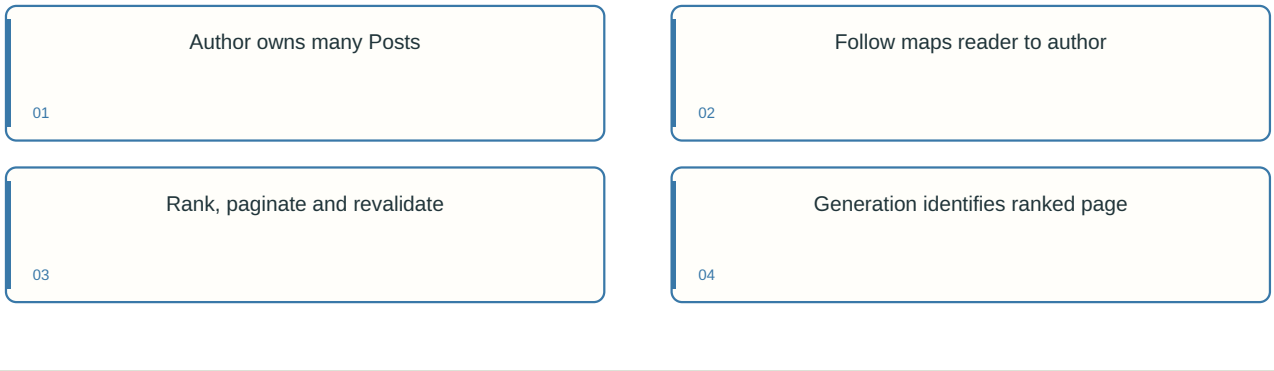


FIG. 2.3.4 **State and ownership in the related design**

The post and access relationships are authoritative; feed candidates are derived references.

Author-created illustration. Numbers are illustrative unless stated otherwise.

INFRASTRUCTURE WORKSHEET · ASSUMED CAPACITY

At 28,935.19 peak requests/s, a hypothetical tested instance capacity of 600 requests/s and a 60% utilization target imply ceil(28,935.19 / 360) = 81 active instances. Reserving capacity for one lost instance gives 82. This does not size the database, image delivery path, or failure domains. Validate each separately; the 600 figure is an assumed input, not a reported benchmark.

▼ Think further · How would simultaneous retries change an estimate based on average user sessions?

Retries add attempts without adding new user intentions. Model their timing, maximum amplification, and the failing dependency separately from normal sessions. A correlated retry wave can exceed a peak factor based on independent arrivals. Bound attempts and admission, then test useful completion and recovery under that burst.

Remember: Workload estimates are inputs to capacity testing; per-instance capacity and failure needs require their own evidence.

Try applying it: If users view twice as many images but publish the same number of posts, which estimates change?

Unit 2 exercises → · Unit 2 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Implementing SLOs — The SRE Workbook ↗

   Engineering book. Service indicators, objectives, and error budgets; not a source of the atlas workload numbers.

   Access reviewed · 21 October 2026

2. A Proof for the Queuing Formula: L = lambda W — John D. C. Little, 1961 ↗

   Original research · publisher record. Long-run queue accounting; publisher record inspected, full proof access may depend on access rights.

   Access reviewed · 21 October 2026

3. The Tail at Scale — Dean and Barroso, 2013 ↗

   Original research. Tail-latency considerations; atlas probabilities and latency examples are author-calculated.

   Access reviewed · 21 October 2026

Queues, bottlenecks & defensible decisions

Prerequisites: 2.3 · Estimate the resources 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A queue grows even though all workers are busy. We will relate waiting, parallel work, and dependent calls before comparing the designs that could meet the requirements.

2.4.1 · Little's Law and queueing intuition

Imagine a stable service completing 500 operations per second. Each operation spends an average of 40 milliseconds inside the boundary being measured. Little's Law gives average work in progress as L = λW: 500/s × 0.04 s = 20 operations.

Here L is the average number in the system, λ the long-run arrival/completion rate, and W the average time inside it. If that average time rises to 0.4 seconds while stable throughput stays 500/s, L becomes 200. Use consistent boundaries and units, including waiting time where applicable. This relation describes stable averages; it does not predict p99 or make an overloaded queue become stable.

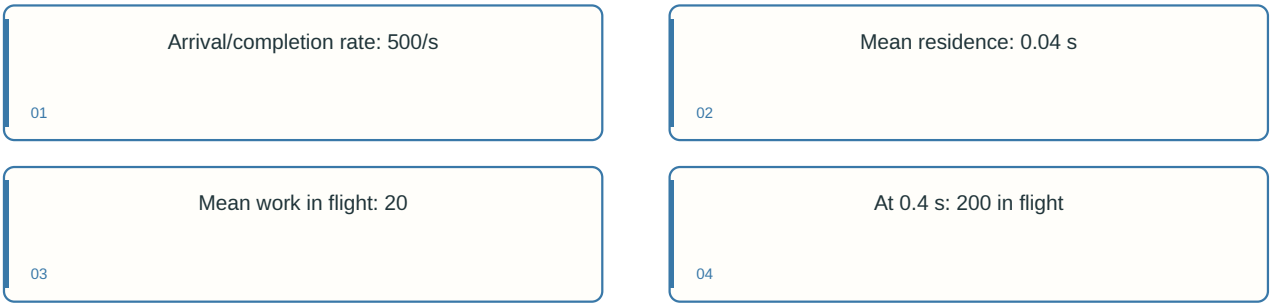


FIG. 2.4.1 **Little's Law and queueing intuition · the example**

Little's Law relates stable long-run averages: work in progress equals rate times residence time within one boundary.

Author-created illustration. Numbers are illustrative unless stated otherwise.

2.4.2 · Amdahl's Law and bottlenecks

Suppose a job spends 20% of its time in work that must run sequentially. The other 80% can be divided perfectly among workers. With four workers, the ideal time becomes 0.2 + 0.8/4 = 0.4 of the original. The speedup is therefore 1/0.4 = 2.5 times.

This is Amdahl's Law for a fixed amount of work. Even infinitely many workers cannot remove the sequential 20%, so the ideal ceiling is five times. Coordination can reduce the real benefit further. Find the stage that limits progress before adding workers. After improving it, measure again: another stage may become the new bottleneck.

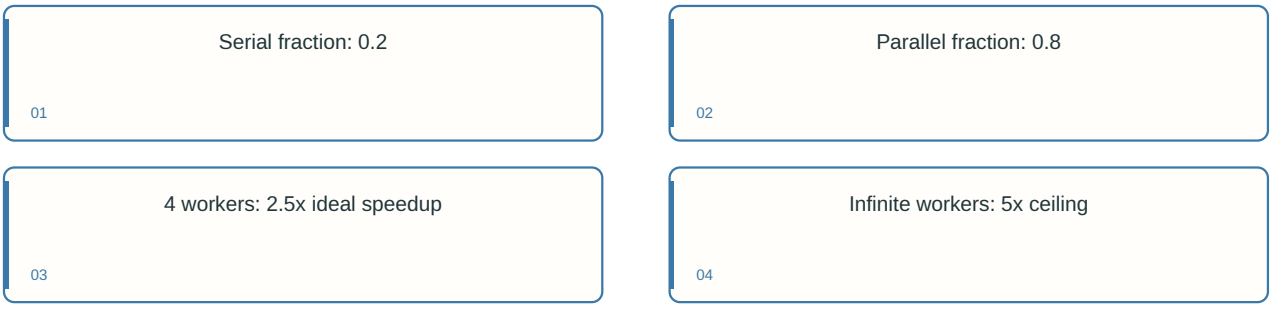


FIG. 2.4.2 **Amdahl's Law and bottlenecks · the example**

Amdahl's Law bounds fixed-work speedup when part of the job remains sequential.

Author-created illustration. Numbers are illustrative unless stated otherwise.

2.4.3 · Tail latency and latency budgets

A page request may start fifty calls in parallel and need all their results. Its completion then depends on the slowest required call. Assume each call independently has a 1% chance of exceeding a threshold. The chance that all are fast is 0.99^{50}; the chance that at least one is slow is $1-0.99^{50} \approx 39.5\%$.

This example shows how fan-out can worsen tail latency. Shared dependencies can also correlate delays, so independence must be justified. Divide the overall deadline among network travel, waiting, computation, and returning the reply. Sending duplicate calls to reduce waiting spends extra capacity and is appropriate only when repeated execution is safe.

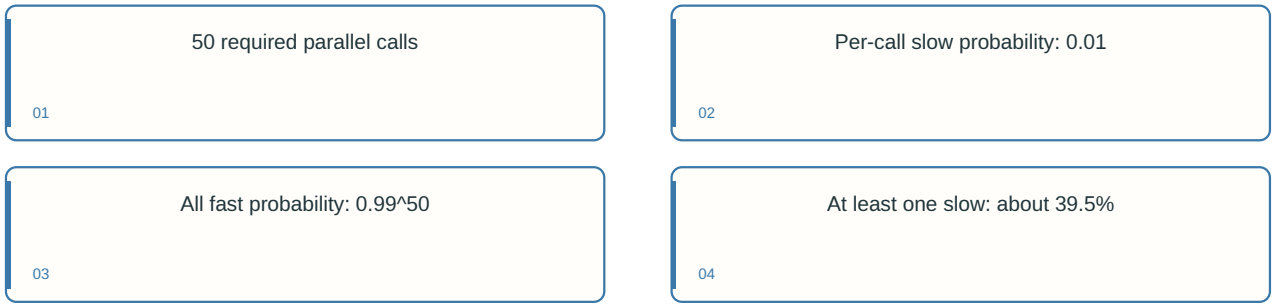


FIG. 2.4.3 **Tail latency and latency budgets · the example**

A request needing many results can be delayed by its slowest dependency; budget the complete path.

Author-created illustration. Numbers are illustrative unless stated otherwise.

2.4.4 · Cost, security, operational complexity, and maintainability as constraints

Suppose two designs both handle the expected traffic. One is cheaper but allows a tenant's cached result to reach another tenant. It fails a security requirement and should be rejected before comparing prices.

Other differences may be preferences: one design costs more money but takes less operating work. Include staffing, maintenance, migration, and recovery in the comparison. A team also needs the skills to run what it builds. Compare candidates against the same workload and failure requirements. A price advantage obtained by dropping backups or weakening isolation represents a different service, rather than a like-for-like saving.

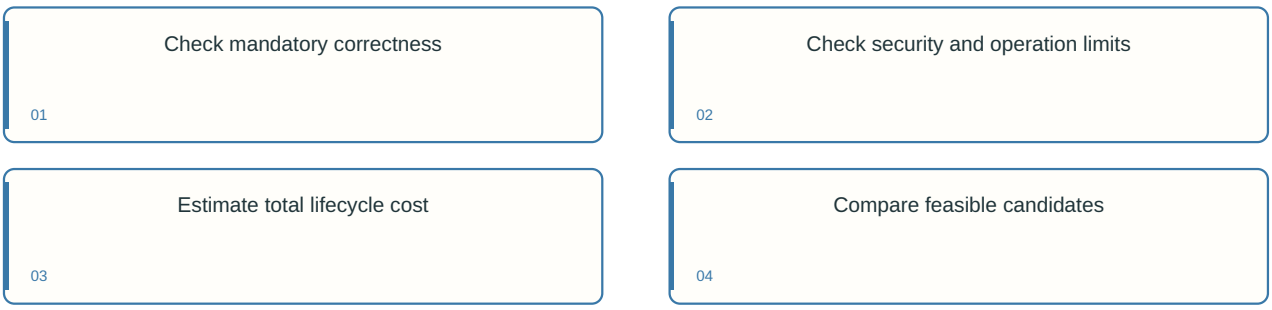


FIG. 2.4.4 **Cost, security, operational complexity, and maintainability as constraints · the example**

Feasible designs must satisfy mandatory security, cost, and operating constraints before preferences are ranked.

Author-created illustration. Numbers are illustrative unless stated otherwise.

2.4.5 · Decision matrices and trade-off analysis

After rejecting designs that fail mandatory requirements, we may still have several reasonable choices. A decision matrix lists the criteria, gives each candidate a score, and weights the criteria according to their importance. It makes the team's judgments visible.

Try changing uncertain scores and weights. If a small change reverses the result, the decision needs better evidence or a choice that is easy to reverse. Record the reason and the condition for reconsidering it in an ADR. The weighted total helps discussion; tests are still needed to establish performance and correctness. A high score cannot make an unsupported requirement true.

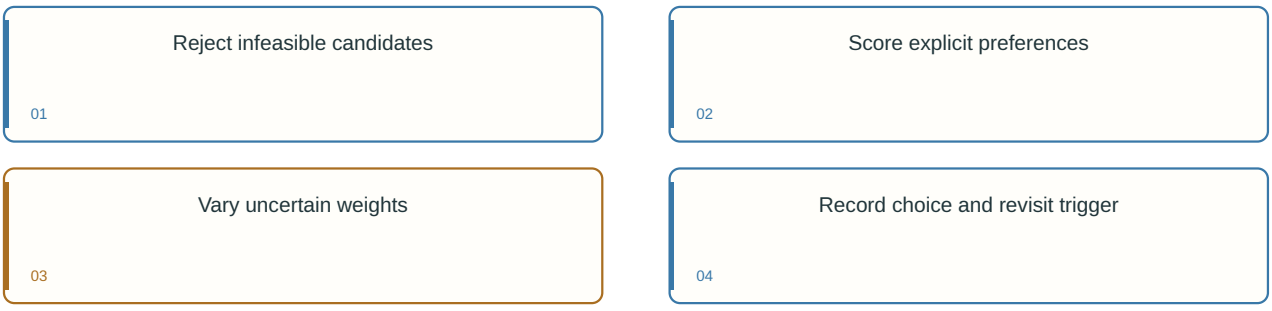


FIG. 2.4.5 **Decision matrices and trade-off analysis · the example**

A decision matrix makes weighted judgments explicit; it does not replace correctness evidence or measurements.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Try the mechanism in code

PYTHON · EXAMPLES/MECHANISMS.PY

```
from mechanisms import capacity, amdahl
print(capacity())
print(amdahl(.8, 4))
```

The defaults produce about 5,787.04 average requests/s and 28,935.19 at the assumed peak. Dividing by 600×0.6 and rounding up gives 81 active instances; allowing one to fail gives 82.

The worksheet sizes only its assumed request-serving resource. Database, image delivery, and the chosen failure placement still need their own capacity tests.

Download the executable mechanism collection ↓ · Follow the connected complete design →

THE WORKBENCH · TRY CHANGING THE INPUTS

Estimate capacity for the social application

Average API rate: **5,787.04 requests/s**; peak: **28,935.19 requests/s**.

Instances needed at the target utilization: **81**; instances if one may be lost: **82**.

Raw posts: 20 GB/day; image delivery: 20 TB/day under the fixed image assumptions.

Start with the stated daily workload, then change one input. Each user views ten 200 kB images and writes one 2 kB post per day. GB and TB use decimal units. The instance capacity is an assumption; database capacity and capacity after a larger outage need their own checks.

▼ Think further · How would simultaneous retries change an estimate based on average user sessions?

Retries add attempts without adding new user intentions. Model their timing, maximum amplification, and the failing dependency separately from normal sessions. A correlated retry wave can exceed a peak factor based on independent arrivals. Bound attempts and admission, then test useful completion and recovery under that burst.

Remember: More concurrency helps only where work and resources permit it; compare feasible choices with their assumptions visible.

Try applying it: When would adding workers increase waiting instead of useful completions?

Unit 2 exercises → · Unit 2 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Implementing SLOs — The SRE Workbook ↗

   Engineering book. Service indicators, objectives, and error budgets; not a source of the atlas workload numbers.

   `Access reviewed · 21 October 2026`

2. A Proof for the Queuing Formula: L = lambda W — John D. C. Little, 1961 ↗

   Original research · publisher record. Long-run queue accounting; publisher record inspected, full proof access may depend on access rights.

   `Access reviewed · 21 October 2026`

3. The Tail at Scale — Dean and Barroso, 2013 ↗

   Original research. Tail-latency considerations; atlas probabilities and latency examples are author-calculated.

   `Access reviewed · 21 October 2026`

PART 1 / UNIT 03

Networking Foundations for System Design

A browser has a website name, but packets need a destination and a transport. Trace how addresses, ports, sockets, and protocol behaviour connect the application to another machine.

The running example

Trace what happens when someone enters a website URL, from DNS lookup through server execution to browser rendering.

Unit 3 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

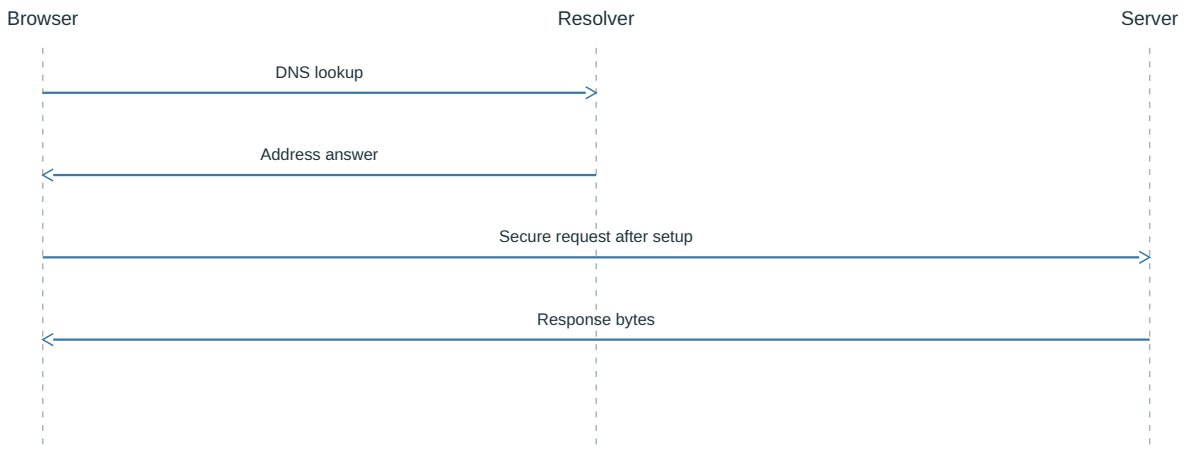


FIG. 3.0.1 **Complete HTTP request timeline**

The full request includes name resolution, connection/security setup, execution and rendering. This is a simplified ordered scenario.

Author-created illustration. Numbers are illustrative unless stated otherwise.

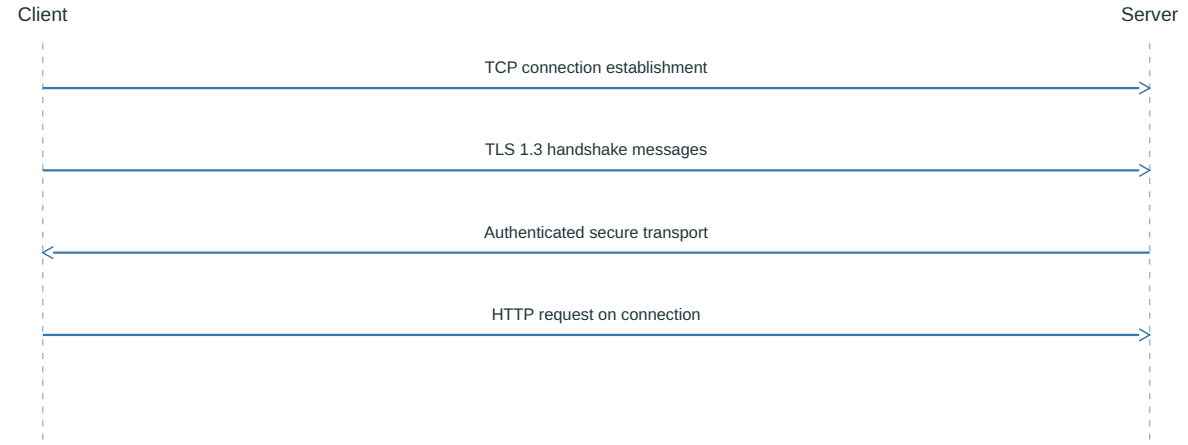
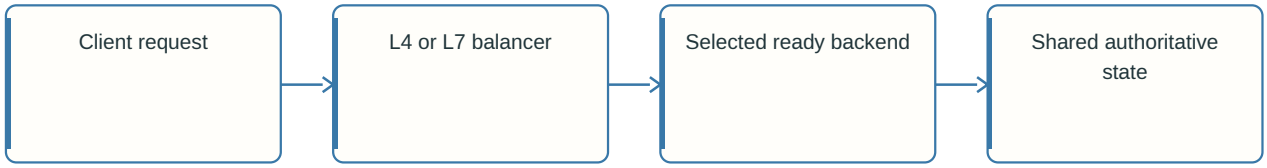


FIG. 3.0.2 **TCP/TLS interaction**

TCP/TLS interaction is shown at protocol-stage granularity; it is not a packet-exact capture or a universal RTT count.

Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 3.0.3 **load-balancing topology**

Balancing distributes suitable request paths, while state and failure ownership remain explicit.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Mode	Normal update path	Recovery obligation
Polling	Repeated client queries	Choose polling interval
SSE	Server streams HTTP events	Reconnect and replay policy
WebSocket	Bidirectional framed channel	Heartbeat, reconnect, history

FIG. 3.0.4 **WebSocket vs polling communication**

Transport patterns differ in connection and polling work; none automatically supplies durable business delivery.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Addresses, sockets & transports

Prerequisites: Introduction to System Design 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A browser has a website name, but packets need a destination and a transport. Trace how addresses, ports, sockets, and protocol behaviour connect the application to another machine.

3.1.1 · How the internet works from the application perspective

Type a website address into a browser. Before the page appears, the browser may resolve the name, establish a connection, negotiate security, send a request, and fetch additional resources. The server must also perform its application work, and the browser must render the result.

Some steps can reuse earlier work. A cached address avoids a lookup; an existing connection avoids setup. Follow the path used by the actual request before assigning a single “network time.” A fast server can still produce a slow page when images are large or connection setup takes several round trips. The request is a sequence of dependent activities.

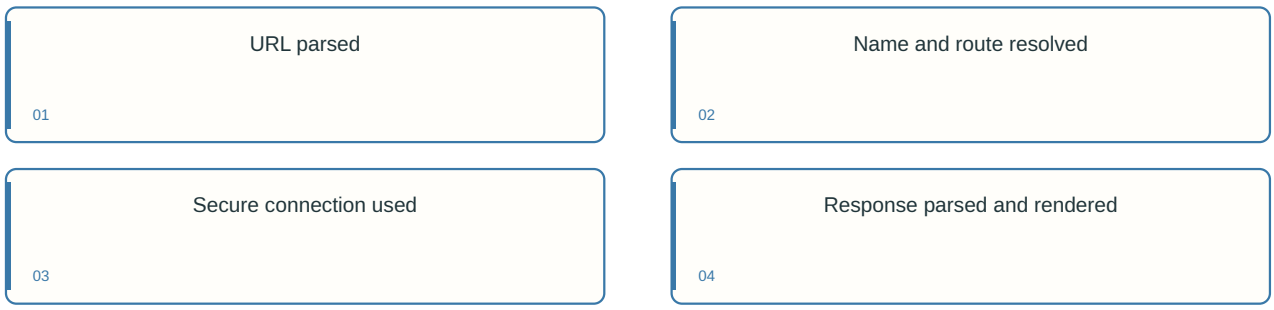


FIG. 3.1.1 **How the internet works from the application perspective · the example**

A page request can include name resolution, transport setup, security, server work, resource transfer, and rendering.

Author-created illustration. Numbers are illustrative unless stated otherwise.

3.1.2 · IP addresses, ports, sockets, routing

To contact a web server, a client needs an address and a transport port. Routing moves packets toward the destination address. The port helps the destination operating system direct transport traffic to the appropriate endpoint. A socket is an endpoint the operating system provides for communication.

An application might listen on port 443 but remain unreachable because it is bound to the wrong address, a firewall blocks the path, or replies cannot return. Investigate the whole route. An address or port also provides no proof of permission: reaching a service and being authorised to use it are different requirements. Private addressing alone does not establish that a caller is trusted.

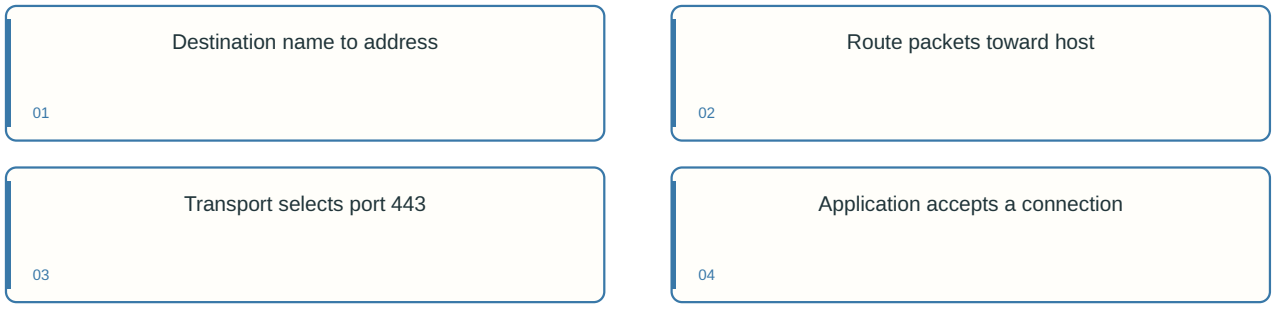


FIG. 3.1.2 **IP addresses, ports, sockets, routing · the example**

Addresses, ports, sockets, and routing connect application endpoints through actual network paths.

Author-created illustration. Numbers are illustrative unless stated otherwise.

3.1.3 · TCP vs UDP vs QUIC

Suppose an application sends two messages. With TCP, the receiver gets an ordered stream of bytes; it must use an application rule to discover where each message ends. UDP provides datagrams but does not supply TCP's ordered, reliable stream behaviour.

QUIC runs over UDP and provides its own secure transport with separate streams and connection identifiers. Loss affecting one stream need not block delivery on another as a missing TCP byte can. The streams still share congestion constraints and may have application dependencies. Choose a transport by the required behaviour and supported infrastructure. Using UDP does not by itself make an application faster, reliable, or secure.

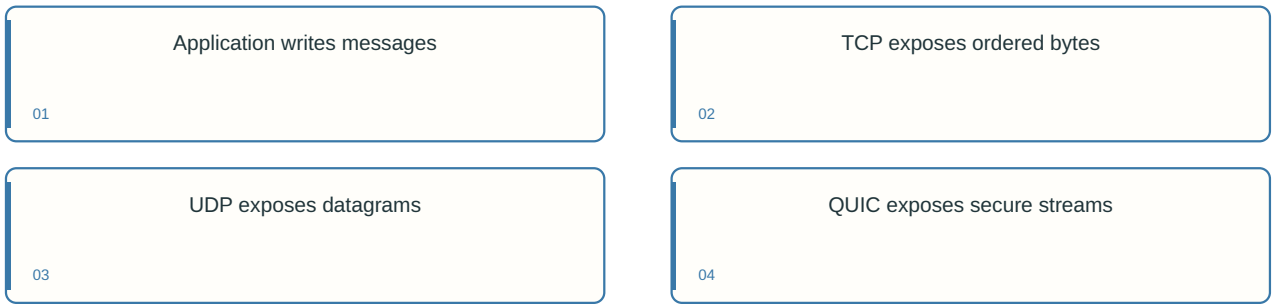


FIG. 3.1.3 **TCP vs UDP vs QUIC · the example**

TCP supplies a byte stream, UDP datagrams, and QUIC secure streams under their respective transport rules.

Author-created illustration. Numbers are illustrative unless stated otherwise.

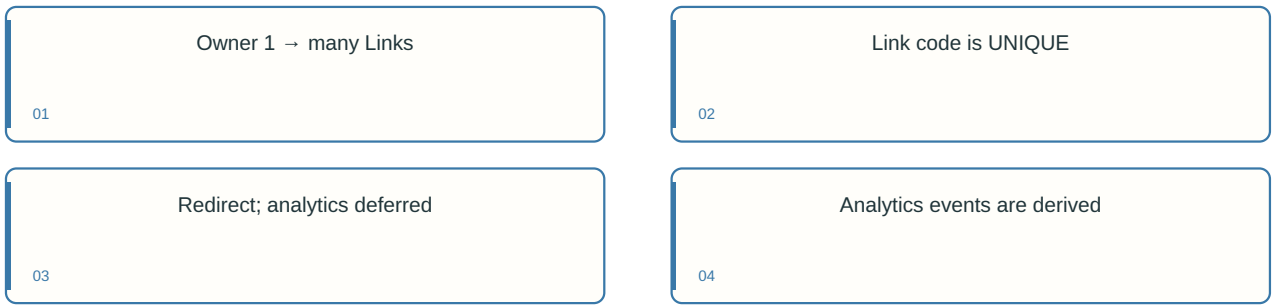


FIG. 3.1.4 **State and ownership in the related design**

A link has one owner and a unique code. The stored operation identifies its creation result.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · What changes when a local dependency becomes a cross-region call?

The call adds travel, connection, timeout, trust, and partial-failure behaviour. A local exception and an unknown remote outcome are different. Revisit deadlines, operation identity, encoding, credentials, and fallback, measuring the whole path. Preserve the business meaning while changing the communication arrangement.

Remember: Choose communication mechanisms by their actual delivery and ordering rules.

Try applying it: Where would you investigate if the application listens on its port but no remote client can reach it?

Unit 3 exercises → · Unit 3 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. RFC 9110: HTTP Semantics, June 2022 ↗

   Original standard. HTTP semantics; application commit policy is separate.

   `Access reviewed · 21 October 2026`

2. RFC 9000: QUIC Transport, 2021 ↗

   Original specification. QUIC streams and transport behavior; application walkthrough is author-created.

   `Access reviewed · 21 October 2026`

3. RFC 8446: TLS 1.3, 2018 ↗

   Original specification. Secure transport handshake and protocol scope.

   `Access reviewed · 21 October 2026`

A secure request, one round trip at a time

Prerequisites: 3.1 · Addresses, sockets & transports 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

The first secure request is slower than the next, even with identical server work. Follow name lookup, setup, encryption, and connection reuse to account for the difference.

3.2.1 · DNS resolution and DNS caching

A browser knows a name such as example.org, but it needs a usable network address. A DNS resolver obtains the relevant records, often using cached answers rather than asking authoritative servers each time. A record's time to live, or TTL, guides how long it may be cached.

Now change the address during failover. Some clients still hold the earlier answer; others already have open connections and do no new lookup. Negative answers may also be cached. The change therefore does not redirect every client immediately. Test DNS refresh and connection lifetimes together when designing failover. The authoritative record is only one part of how clients choose a destination.

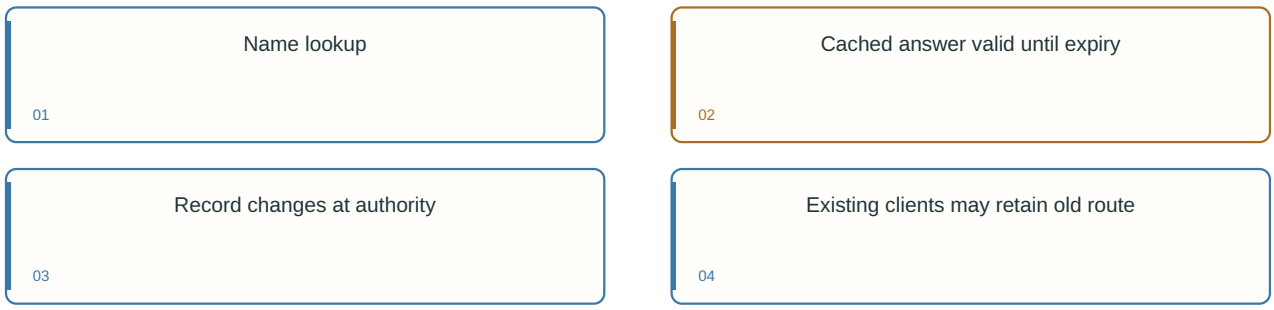


FIG. 3.2.1 **DNS resolution and DNS caching · the example**

DNS resolves names through records and caches; existing answers and connections delay the effect of a record change.

Author-created illustration. Numbers are illustrative unless stated otherwise.

3.2.2 · TLS and HTTPS

When a browser opens an HTTPS connection, it needs to establish that it is talking to the intended server and protect the traffic. TLS provides peer authentication under its trust configuration, along with confidentiality and integrity protection. The client must validate the certificate and its relationship to the requested hostname.

Trace where encryption ends. If a reverse proxy terminates TLS, the proxy-to-application connection needs its own policy. Protect the relevant keys as well. TLS protects the connection; the application must still check whether a signed-in user may read a particular task. A secure transport does not supply resource ownership rules.

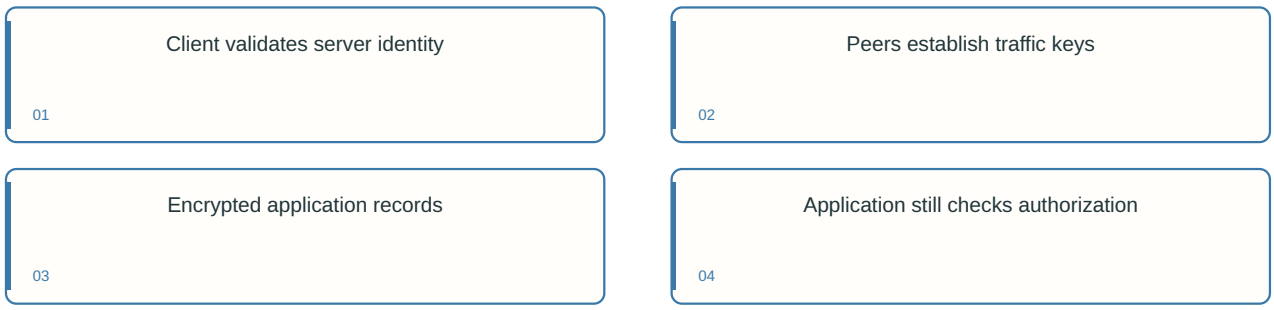


FIG. 3.2.2 **TLS and HTTPS · the example**

TLS protects and authenticates transport under its trust configuration; application resource authorisation remains necessary.

Author-created illustration. Numbers are illustrative unless stated otherwise.

3.2.3 · HTTP/1.1, HTTP/2, HTTP/3

A page often needs many resources. HTTP/1.1 clients may use multiple connections to avoid waiting behind other responses. HTTP/2 carries multiple streams over one TCP connection. If a TCP byte is lost, later bytes wait for recovery, which can delay several HTTP/2 streams.

HTTP/3 uses QUIC streams and changes that transport behaviour. The protocols retain HTTP's application meanings, such as methods and status codes, while carrying messages differently. Performance depends on loss, negotiation, proxies, and payloads. Upgrading the protocol can reduce some waiting, but the request still spends time in application queues and the database. Measure the complete path rather than attributing every delay to the HTTP version.

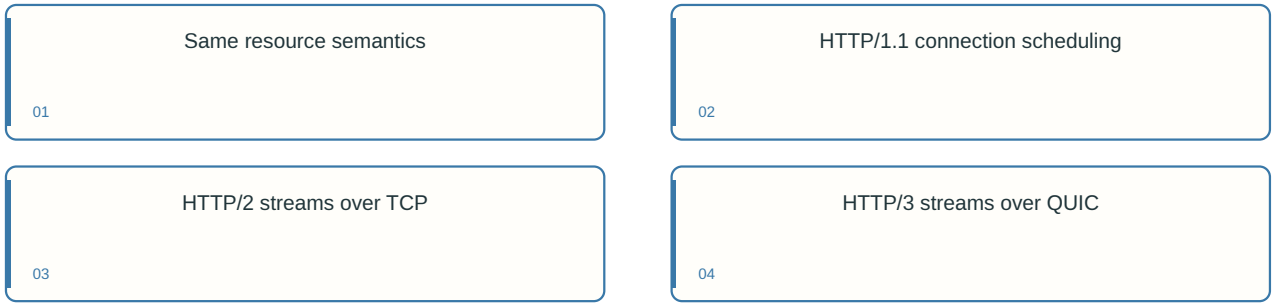


FIG. 3.2.3 **HTTP/1.1, HTTP/2, HTTP/3 · the example**

HTTP versions carry common application semantics using different connection and stream mechanisms.

Author-created illustration. Numbers are illustrative unless stated otherwise.

3.2.4 · Connection establishment and connection reuse

The first request to a server may need a new connection; a later request can reuse one. A new TCP connection normally performs a handshake, and secure setup adds exchanges whose number depends on the protocol and reuse options. Existing connections avoid some of this work.

Connection reuse needs limits. A connection left idle may be closed by an intermediary. A very long-lived connection may continue using an endpoint that discovery has replaced. Conversely, closing connections too often wastes setup work and CPU. Choose lifetimes and pool sizes that fit the infrastructure and workload, and handle replacement safely when an old connection no longer works.

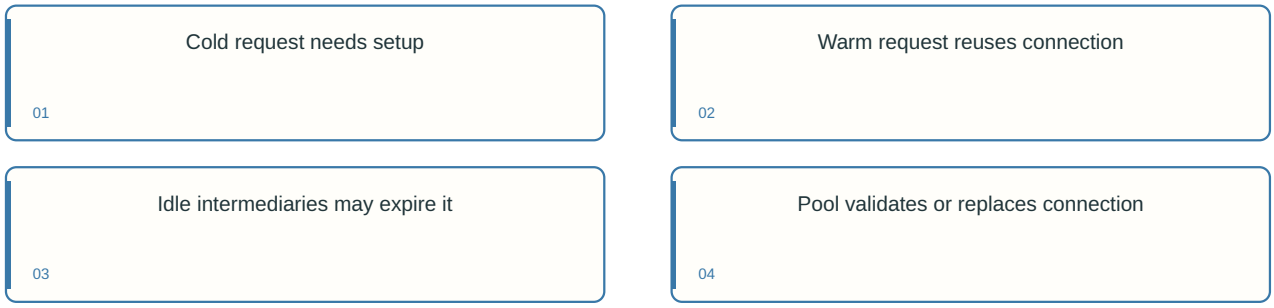


FIG. 3.2.4 **Connection establishment and connection reuse · the example**

Reusing a connection can avoid setup work while introducing lifetime, endpoint freshness, and replacement decisions.

Author-created illustration. Numbers are illustrative unless stated otherwise.

3.2.5 · Latency caused by network round trips

Assume a simplified cold request spends 20 ms on DNS, 40 ms on TCP setup, 40 ms on TLS setup, and 55 ms on the request, server work, and reply. In this sequential model, the total is 20+40+40+55 = 155 ms.

This is an assumed example, not a benchmark. Cached DNS, connection reuse, security resumption, and overlapping work change the calculation. It illustrates why a small amount of server computation can still accompany substantial delay. Moving a service closer can reduce network travel time. Reuse or batching can reduce the number of exchanges. First identify which exchanges the actual request needs.

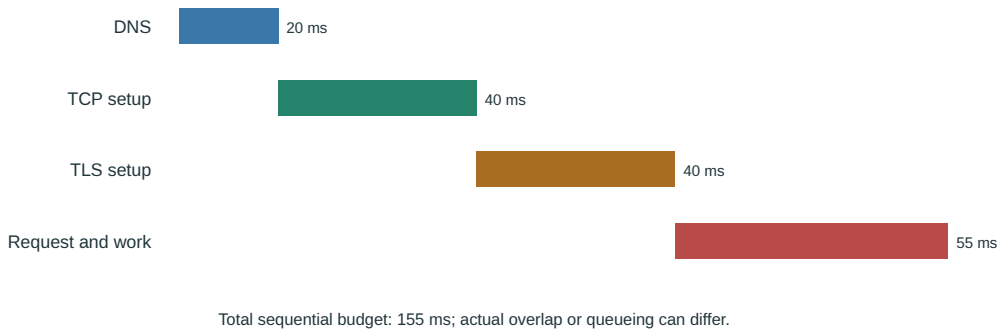


FIG. 3.2.5 **A cold request spends time before server work**

A sequential illustrative cold path totals 155 ms. Reuse, protocol, overlap, and actual distance change the result.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · What changes when a local dependency becomes a cross-region call?

The call adds travel, connection, timeout, trust, and partial-failure behaviour. A local exception and an unknown remote outcome are different. Revisit deadlines, operation identity, encoding, credentials, and fallback, measuring the whole path. Preserve the business meaning while changing the communication arrangement.

Remember: A request includes setup and travel as well as server execution; reused work changes the path.

Try applying it: Which parts of the 155 ms cold example can disappear with a usable warm connection?

Unit 3 exercises → · Unit 3 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. RFC 9110: HTTP Semantics, June 2022 ↗

   Original standard. HTTP semantics; application commit policy is separate.

   Access reviewed · 21 October 2026

2. RFC 9000: QUIC Transport, 2021 ↗

   Original specification. QUIC streams and transport behavior; application walkthrough is author-created.

   Access reviewed · 21 October 2026

3. RFC 8446: TLS 1.3, 2018 ↗

   Original specification. Secure transport handshake and protocol scope.

   Access reviewed · 21 October 2026

Intermediaries & persistent connections

Prerequisites: 3.2 · A secure request, one round trip at a time 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A proxy, gateway, and balancer can all sit between a browser and an application. Assign each a purpose, then compare the work involved in polling and streaming updates.

3.3.1 · Reverse proxies vs forward proxies

An organisation may send employees' outbound traffic through a forward proxy. A website may place a reverse proxy in front of its application servers. The first acts on the client side of the exchange; the second presents the servers to incoming clients.

A reverse proxy can terminate TLS, route requests, and apply limits, but it also introduces another component that can fail or time out. Decide which forwarded metadata the application may trust. For example, accepting a client-supplied identity header as if a trusted proxy had checked it can bypass authentication. The design must identify who creates the header and which callers can reach the receiving endpoint.

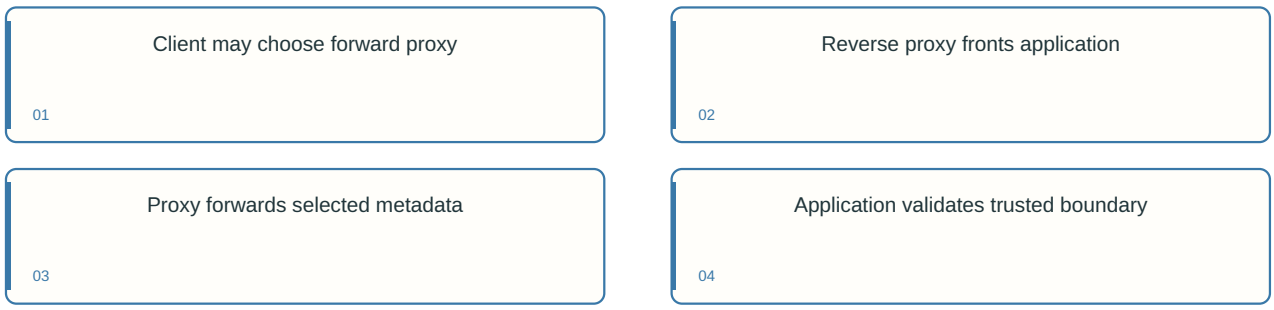


FIG. 3.3.1 **Reverse proxies vs forward proxies · the example**

A forward proxy acts for clients; a reverse proxy fronts servers, with explicit forwarding and trust rules.

Author-created illustration. Numbers are illustrative unless stated otherwise.

3.3.2 · Load balancers vs API gateways

Suppose incoming requests must be sent to one of several application instances. A load balancer chooses a backend. An API gateway often adds API-specific functions such as authentication integration, quotas, or request transformation. A product can perform both roles.

Assign responsibilities explicitly instead of relying on product names. A gateway can reject an oversized request or apply a coarse rate limit, while a domain service checks ownership of a payment. Moving every business rule into the gateway makes many teams depend on changes there. Also trace downstream capacity: distributing traffic across more application instances does not repair a database that is already overloaded.

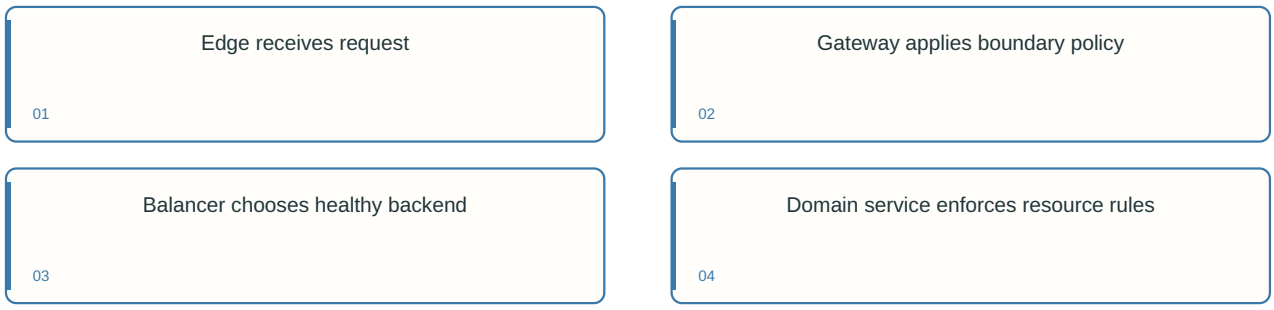


FIG. 3.3.2 **Load balancers vs API gateways · the example**

Balancers select backends; gateways commonly add API boundary policy. Products can combine those responsibilities.

Author-created illustration. Numbers are illustrative unless stated otherwise.

3.3.3 · L4 vs L7 load balancing

To route a connection by its destination port, a balancer can work with transport information: this is Layer-4 routing. To route an HTTP request by its host or path, it must inspect application messages: this is Layer-7 routing.

Application-aware routing supports richer decisions but adds parsing, processing, and trust requirements. The placement of encryption termination determines which component can see those messages. Also distinguish requests from connections. A long-lived connection can remain attached to one backend even while new connections go elsewhere. Choose the routing level by the information needed for the decision and the connection behaviour the application expects.

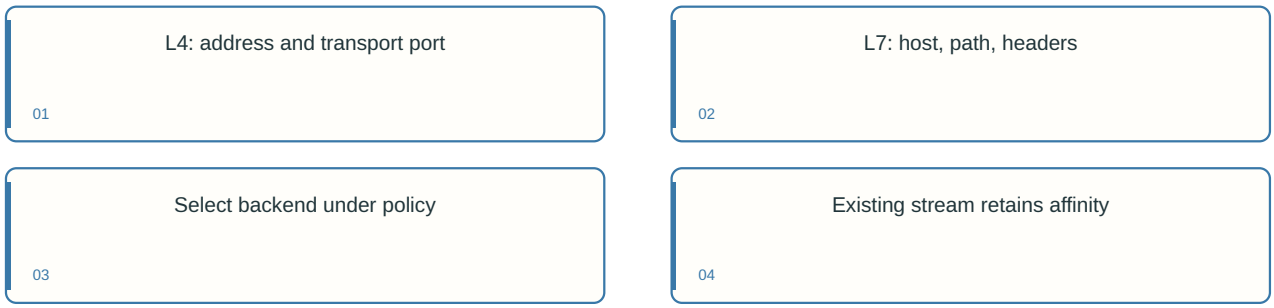


FIG. 3.3.3 **L4 vs L7 load balancing · the example**

L4 routing uses transport information; L7 routing can inspect application messages where encryption and policy permit it.

Author-created illustration. Numbers are illustrative unless stated otherwise.

3.3.4 · WebSockets, Server-Sent Events, and polling

Assume 1,000 clients ask for updates every five seconds. Even when nothing changes, polling creates 1,000/5 = 200 requests per second. Each response gives the client another snapshot.

Server-Sent Events lets a server stream events to a client over HTTP. WebSockets supports messaging in both directions after connection setup. Streaming avoids repeated empty polls but keeps connections open. Plan heartbeats, buffers, reconnects, and what happens to missed updates. A live connection is a delivery path, not a durable message history. If clients must recover updates after a disconnection, the application needs event identities and a replay or resynchronisation rule.

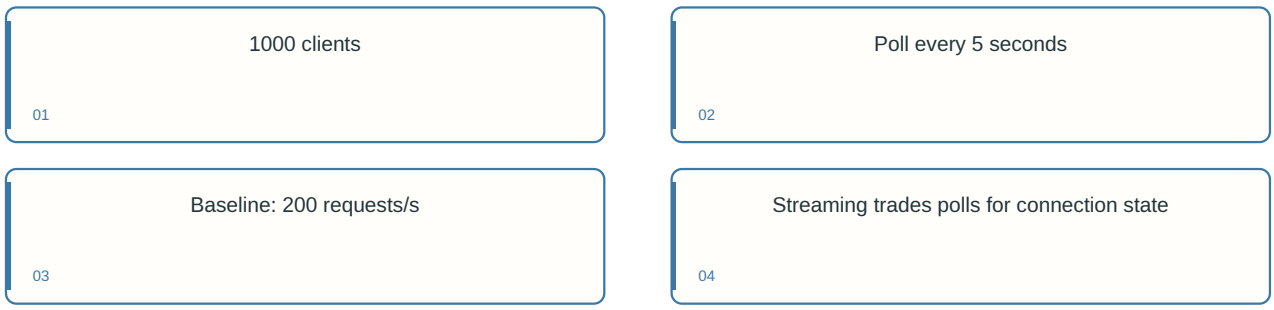


FIG. 3.3.4 **WebSockets, Server-Sent Events, and polling · the example**

Polling repeats requests; SSE streams server events; WebSockets supports two-way messages. Recovery remains an application concern.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · What changes when a local dependency becomes a cross-region call?

The call adds travel, connection, timeout, trust, and partial-failure behaviour. A local exception and an unknown remote outcome are different. Revisit deadlines, operation identity, encoding, credentials, and fallback, measuring the whole path. Preserve the business meaning while changing the communication arrangement.

Remember: Label the responsibility and trust of each intermediary rather than inferring behaviour from a product name.

Try applying it: Which policy belongs at the gateway, and which needs the service's knowledge of resource ownership?

Unit 3 exercises → · Unit 3 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. RFC 9110: HTTP Semantics, June 2022 ↗

   Original standard. HTTP semantics; application commit policy is separate.

   Access reviewed · 21 October 2026

2. RFC 9000: QUIC Transport, 2021 ↗

   Original specification. QUIC streams and transport behavior; application walkthrough is author-created.

   Access reviewed · 21 October 2026

3. RFC 8446: TLS 1.3, 2018 ↗

   Original specification. Secure transport handshake and protocol scope.

   Access reviewed · 21 October 2026

Discover, route & survive the network

Prerequisites: 3.3 · Intermediaries & persistent connections 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

The server commits, but the client receives no reply. Follow routing, discovery, caches, and bounded connection use to understand why a healthy destination does not guarantee a known result.

3.4.1 · NAT, firewalls, ingress, egress

A client opens an outbound connection, but no reply arrives. The problem may lie outside the remote application: a firewall may block traffic, or a network address translation table may have exhausted or expired its mappings.

NAT rewrites network addressing. A firewall applies traffic rules. Ingress is traffic entering a boundary, and egress is traffic leaving it. Show these functions when tracing the path in and out. Choose connection lifetimes and outbound policies together. Segmentation can reduce which services are reachable, but an allowed connection must still authenticate and authorise its requests, particularly in a shared tenant environment.

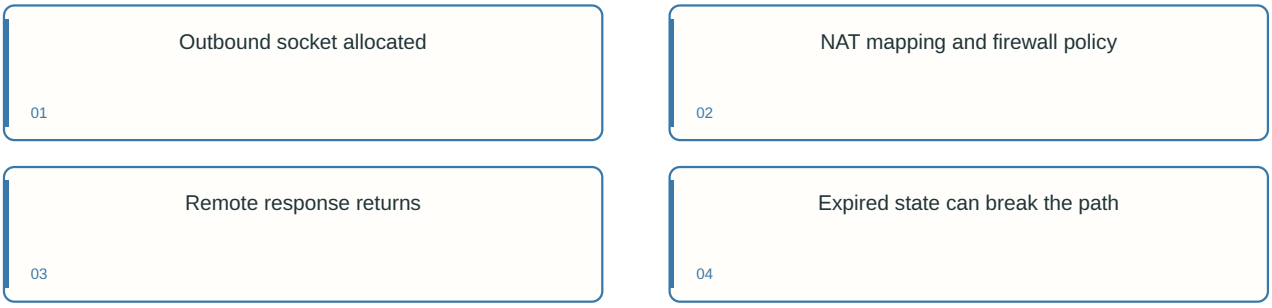


FIG. 3.4.1 **NAT, firewalls, ingress, egress · the example**

NAT changes addressing, firewalls enforce traffic rules, and ingress/egress describe traffic entering or leaving a boundary.

Author-created illustration. Numbers are illustrative unless stated otherwise.

3.4.2 · Service discovery

An application wants to call “the payment service” even as instances are replaced. Service discovery maps that logical name to usable endpoints. It may rely on DNS, a registry, or the platform's control plane.

Discovery information can be stale. An instance might fail after being advertised, or a healthy instance might already lack spare capacity. Cache results to avoid making every call depend on a lookup, but refresh them deliberately. A client must still cope with a bad endpoint, using bounded retries within its deadline. Successful discovery establishes a possible destination; it does not establish that the operation will succeed.

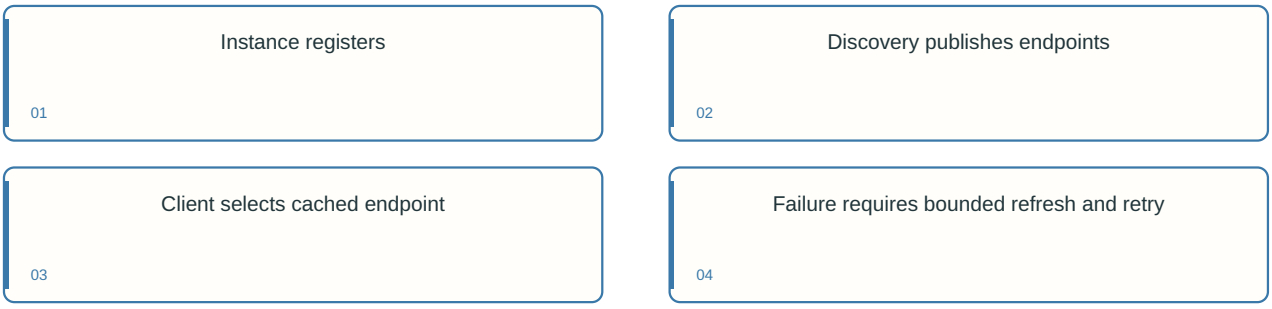


FIG. 3.4.2 **Service discovery · the example**

Service discovery maps logical services to endpoints whose readiness and capacity still require handling.

Author-created illustration. Numbers are illustrative unless stated otherwise.

3.4.3 · CDN and edge networking

A user requests a public image. A nearby CDN cache can return it without contacting the origin server. On a miss, the edge fetches the image and stores a representation according to its caching policy.

The cache key determines which requests share a copy, and freshness rules determine when that copy may be reused. These are correctness decisions. A missing tenant or authorisation dimension can make a cache serve someone else's content. Public images and private account pages need different policies. CDNs reduce some network travel and origin traffic, while adding copies whose expiry, purge, and failure behaviour must be understood.

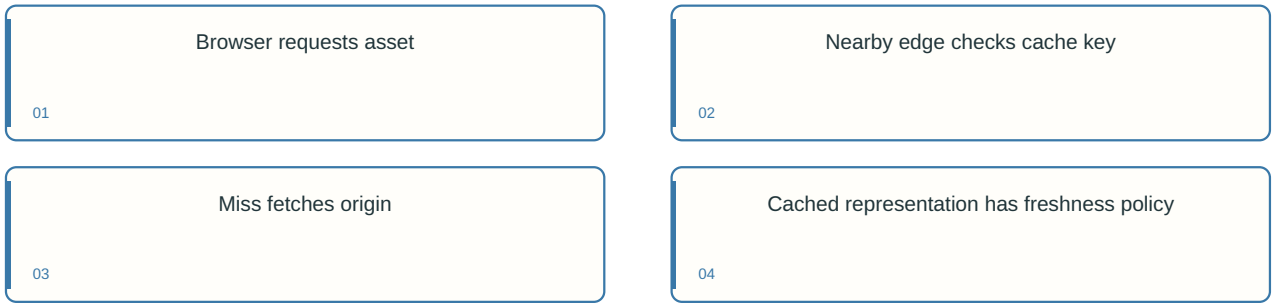


FIG. 3.4.3 **CDN and edge networking · the example**

A CDN caches permitted representations near users, with keys, freshness, and origin recovery requirements.

Author-created illustration. Numbers are illustrative unless stated otherwise.

3.4.4 · Network partitions, timeouts, packet loss

A client sends a request and waits until its deadline. It sees a timeout. Did the server receive the request, perform it, or commit its write? The timeout alone cannot answer these questions. The request or its reply may have been delayed or lost.

A network partition prevents some participants from communicating even while other paths work. Treat such partial failures as normal possibilities. Distinguish connection-setup, idle, and overall request limits. Unbounded waiting can exhaust resources; blind retrying can repeat an effect. Use operation identifiers or status queries where the client needs to recover an uncertain outcome, and keep retries inside an overall deadline.

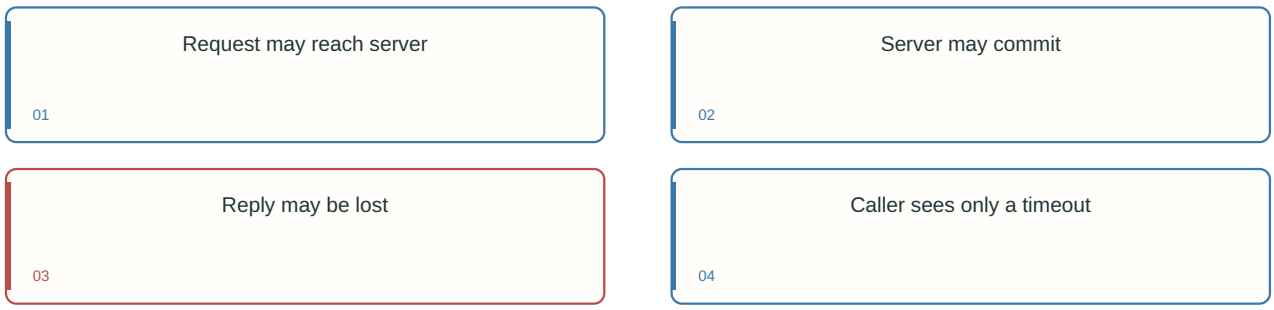


FIG. 3.4.4 **Network partitions, timeouts, packet loss · the example**

A timeout ends the caller's wait without establishing whether the remote operation executed.

Author-created illustration. Numbers are illustrative unless stated otherwise.

3.4.5 · Connection pooling and keepalive

A database connection pool lets requests borrow existing connections. Suppose there are twenty connections and each is held for an assumed average of 50 ms. If every connection remains busy, the simple service-capacity calculation is 20/0.05 = 400 operations per second before overhead and variability.

A request can also wait to borrow a connection, so include that wait in its deadline. Oversizing the pool may merely overload the database; a leaked connection gradually reduces usable capacity. Keepalive can help detect some broken connections, but says little about whether a business operation will succeed. Measure queueing and hold times before changing the pool size.

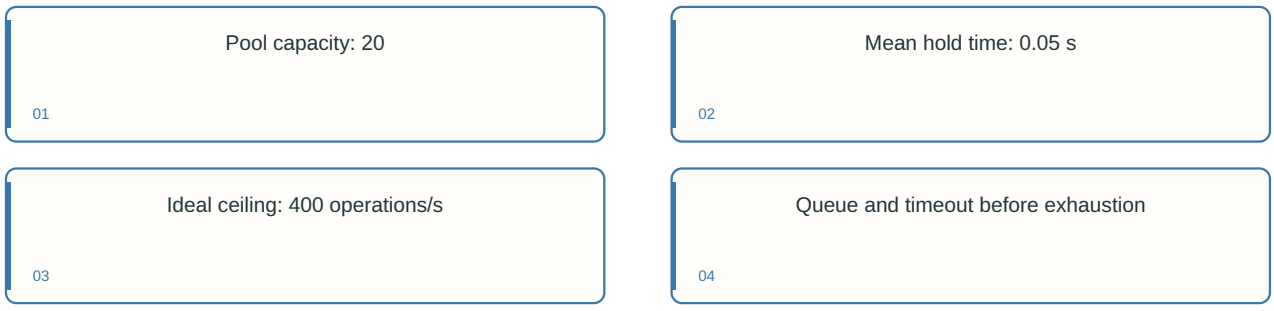


FIG. 3.4.5 **Connection pooling and keepalive · the example**

A pool bounds and reuses connections; acquisition waiting and resource hold time belong in capacity and deadline reasoning.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Try the mechanism in code

PYTHON · ARITHMETIC MODEL

```
cold = dict(dns=20, tcp=40, tls=40, request_and_work=55)
assert sum(cold.values()) == 155
warm = dict(request_and_work=55)
assert sum(warm.values()) == 55
```

The cold path totals 155 ms. The warm model reuses the assumed DNS, TCP, and TLS setup, leaving the same 55 ms request/work stage.

Real reuse depends on cache, protocol, connection state, and overlap. This code performs arithmetic; it does not open a connection or measure a protocol.

Download the executable mechanism collection ↓ · Follow the connected complete design →

▼ Think further · What changes when a local dependency becomes a cross-region call?

The call adds travel, connection, timeout, trust, and partial-failure behaviour. A local exception and an unknown remote outcome are different. Revisit deadlines, operation identity, encoding, credentials, and fallback, measuring the whole path. Preserve the business meaning while changing the communication arrangement.

Remember: A timeout limits waiting; it does not identify whether remote execution occurred.

Try applying it: How would a client recover after losing the reply to a request that may have changed state?

Unit 3 exercises → · Unit 3 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. RFC 9110: HTTP Semantics, June 2022 ↗

   Original standard. HTTP semantics; application commit policy is separate.

   Access reviewed · 21 October 2026

2. RFC 9000: QUIC Transport, 2021 ↗

   Original specification. QUIC streams and transport behavior; application walkthrough is author-created.

   Access reviewed · 21 October 2026

3. RFC 8446: TLS 1.3, 2018 ↗

   Original specification. Secure transport handshake and protocol scope.

   Access reviewed · 21 October 2026

PART 1 / UNIT 04

Runtime, Processes, Threads, and Concurrency

A thousand uploads can wait on storage or compete for CPU. First identify the process, threads, memory, and resource ownership before selecting an execution model.

The running example

Build a service that processes 1,000 file uploads. Compare sequential execution, threads, asynchronous I/O, and worker queues. Identify which architecture helps under which workload.

Unit 4 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

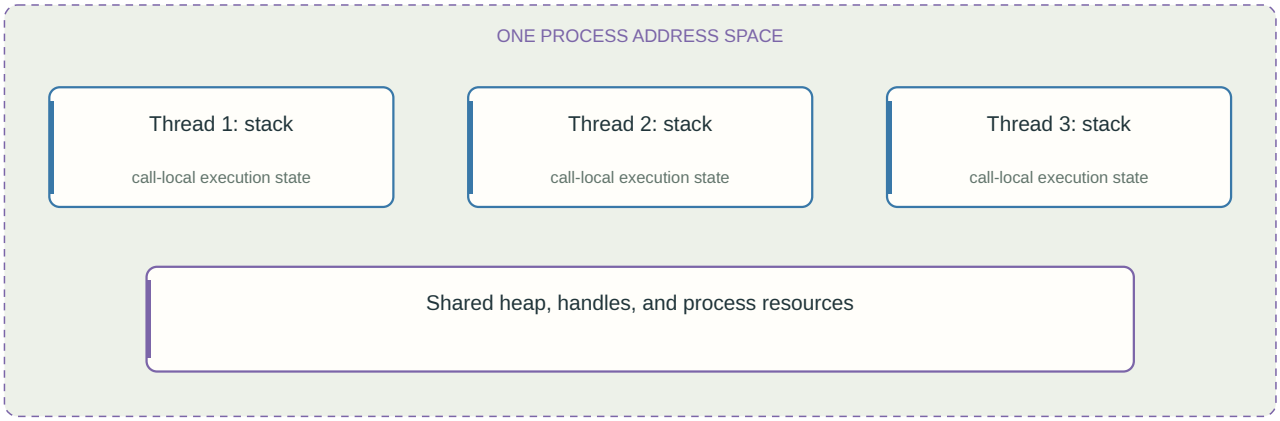


FIG. 4.0.1 **Threads share a heap but have execution-local stacks**

One process contains three thread contexts and a shared heap/resource region. Source-module boundaries do not isolate their failures.

Author-created illustration. Numbers are illustrative unless stated otherwise.

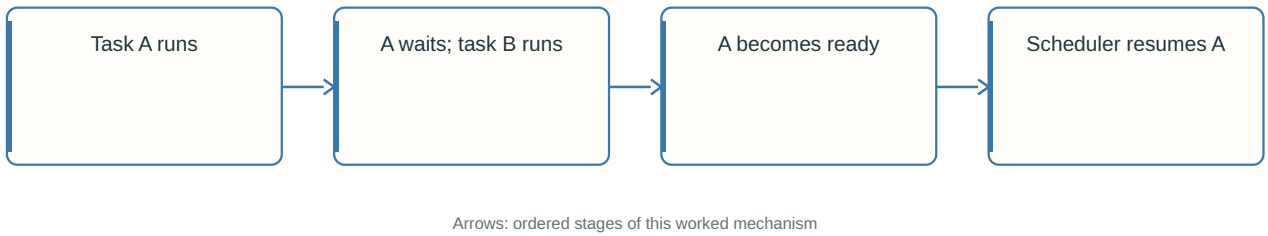
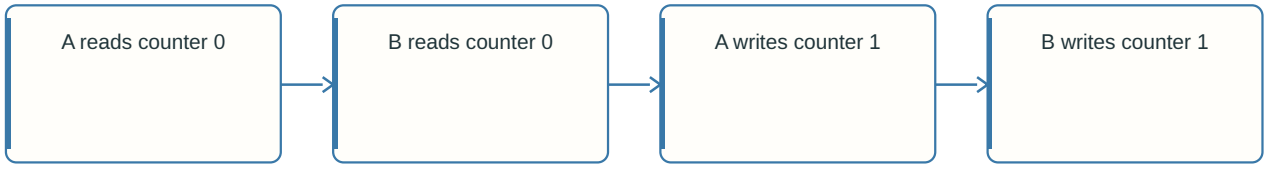


FIG. 4.0.2 **scheduling timeline**

Concurrency interleaves activities; parallel execution requires actual simultaneous resources.

Author-created illustration. Numbers are illustrative unless stated otherwise.

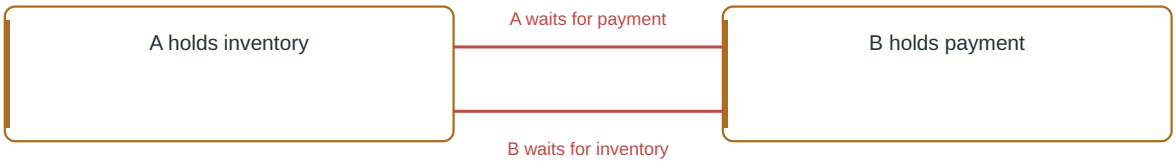


Arrows: ordered stages of this worked mechanism

FIG. 4.0.3 **race-condition interleaving**

Two increments become one without an atomic read-modify-write boundary.

Author-created illustration. Numbers are illustrative unless stated otherwise.

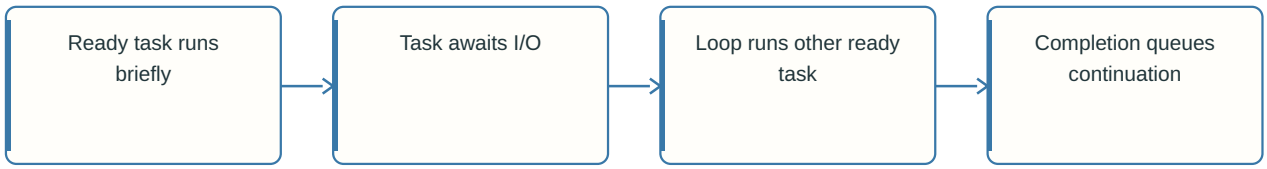


Two opposing wait dependencies form a cycle; neither holder can proceed.

FIG. 4.0.4 **A circular wait prevents both participants progressing**

A holds inventory and waits for payment; B holds payment and waits for inventory. Consistent resource ordering or redesigned ownership can remove this cycle.

Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 4.0.5 **event-loop flow**

The event loop cannot serve other work during a long blocking callback.

Author-created illustration. Numbers are illustrative unless stated otherwise.

The runtime owns your resources

Prerequisites: Introduction to System Design 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A thousand uploads can wait on storage or compete for CPU. First identify the process, threads, memory, and resource ownership before selecting an execution model.

4.1.1 · Program vs process vs thread

The same program can run twice at the same time. Each running instance is a process, with its own address space and resources. Inside a process, threads provide separate paths of execution. They commonly share heap memory but have their own stacks and scheduling state.

These runtime boundaries differ from source files and modules. A crash of one process can stop all its threads, even if they execute different modules. Separate processes can provide useful isolation, while threads make sharing data convenient. Sharing introduces coordination needs: neither choosing threads nor dividing code into classes automatically makes concurrent updates safe.

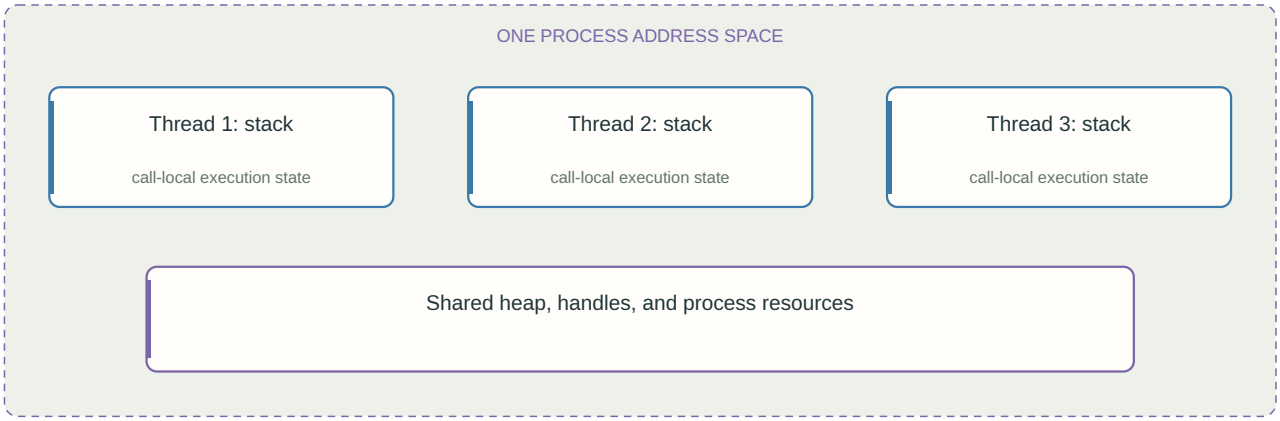


FIG. 4.1.1 **Threads share a heap but have execution-local stacks**

One process contains three thread contexts and a shared heap/resource region. Source-module boundaries do not isolate their failures.

Author-created illustration. Numbers are illustrative unless stated otherwise.

4.1.2 · Memory, stack, heap, and object lifetimes

When an upload request calls functions, the runtime keeps call information on a stack. Dynamically managed objects, such as a large upload buffer, commonly live on the heap. The details depend on the language and runtime.

An object's lifetime need not end when the initiating function returns. If a background task still holds a reference to the buffer, the buffer can remain in memory. Track who keeps each resource and when that ownership ends. Files and database connections need timely cleanup as well. A garbage collector managing memory does not establish when every external resource is closed. Bound the number and size of outstanding uploads.

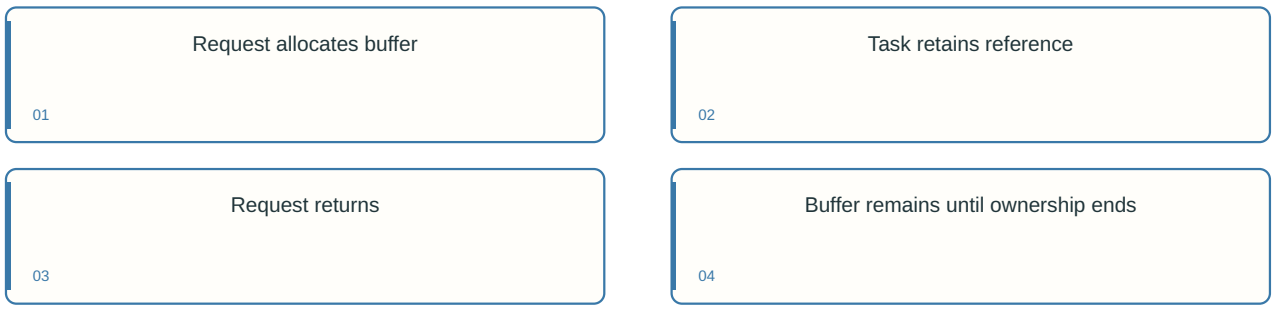


FIG. 4.1.2 **Memory, stack, heap, and object lifetimes · the example**

Stack, heap, and references have runtime-specific lifetimes; external resources also need explicit timely release.

Author-created illustration. Numbers are illustrative unless stated otherwise.

4.1.3 · Scheduling and context switching

A machine may have more runnable threads than available cores. Its scheduler chooses which thread runs next. A context switch saves one execution state and restores another, and may disturb useful CPU-cache contents.

More threads help when some are waiting and others can use an idle CPU. When the CPU is already saturated, more runnable threads can add overhead and waiting. A blocked I/O call can leave the CPU free for another thread; a busy-wait loop consumes it. Inspect CPU utilisation, runnable queues, and lock contention before choosing a thread count. Starting a thread for every incoming request allows resource use to grow without a bound.

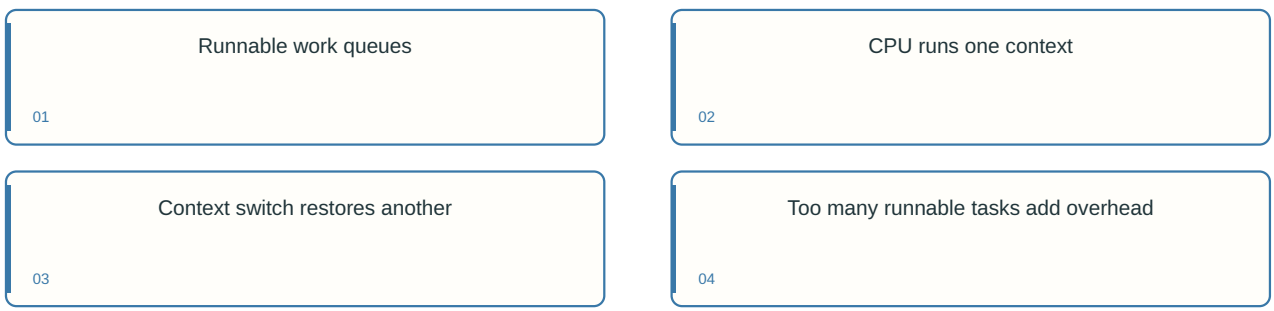


FIG. 4.1.3 **Scheduling and context switching · the example**

Scheduling assigns CPU time; context switches and excessive runnable work can add overhead and waiting.

Author-created illustration. Numbers are illustrative unless stated otherwise.

4.1.4 · CPU-bound vs I/O-bound workloads

Assume 1,000 uploads each spend 100 ms waiting for storage and require negligible CPU work. Handling them sequentially takes about 100 seconds of waiting. Allowing fifty independent waits at once gives an ideal waiting-time lower bound near two seconds, before overhead and bandwidth limits.

This workload is I/O-bound: waiting is the main limit. Compression may instead be CPU-bound because useful time is spent computing. Extra asynchronous tasks do not create extra CPU cores. For computation, investigate algorithms and actual parallel execution. For waiting, concurrency can use idle periods, provided the storage service and connections can support it. Measure which resource limits the real workload.

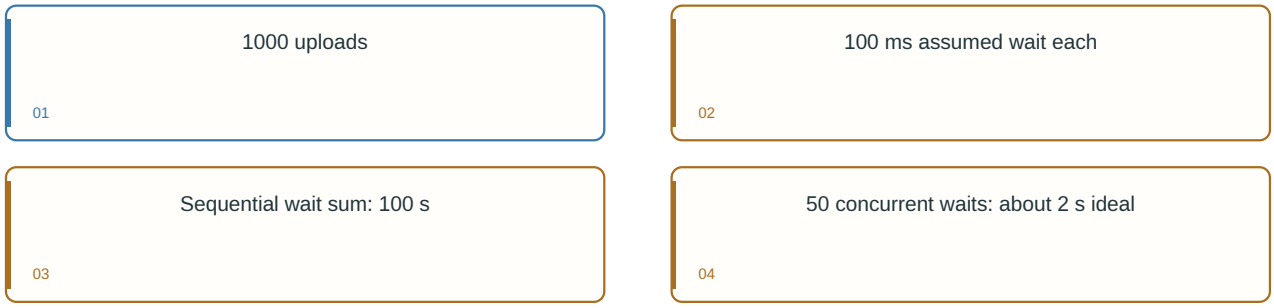


FIG. 4.1.4 **CPU-bound vs I/O-bound workloads · the example**

CPU-bound work is limited by computation, while I/O-bound work spends important time waiting on external operations.

Author-created illustration. Numbers are illustrative unless stated otherwise.

4.1.5 · Synchronous vs asynchronous execution

A caller can start an operation and wait for its result before proceeding; this is synchronous interaction. With an asynchronous interface, initiation and later completion are represented separately, so the caller can arrange other work while waiting.

A different question is whether the thread itself blocks. An async function can still call a blocking library operation and stall the thread running it. Trace the implementation beneath the interface, including drivers. Choose a model that keeps waiting and resource usage manageable for the workload. The word “async” describes an interface or execution arrangement; it does not establish that every operation underneath is nonblocking.

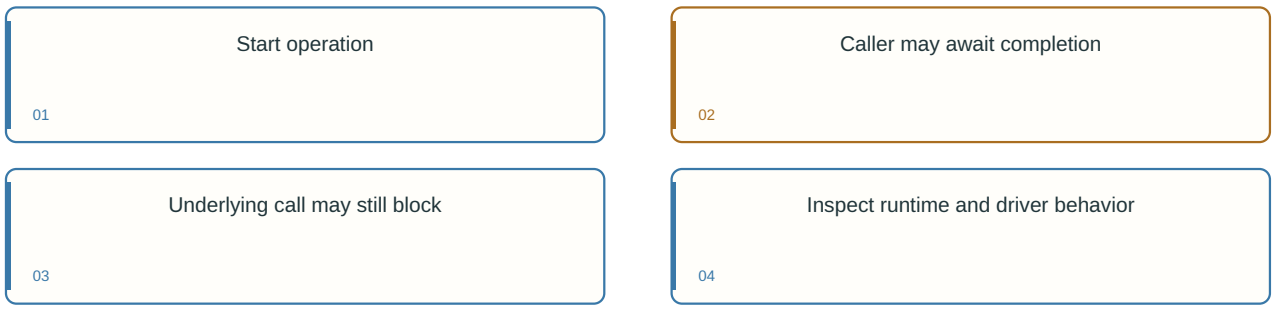


FIG. 4.1.5 **Synchronous vs asynchronous execution · the example**

Asynchronous interfaces separate initiation from completion; their underlying operations can still block.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · Who should own accepted upload work after its initiating request is cancelled?

If cancellation only ends the connection but the business still requires processing, a durable job or workflow must retain identity, input, and progress. Cancel and release request-owned resources without discarding that obligation. If the work itself should stop, persist the chosen cancellation and block inappropriate later completion.

Remember: Match concurrency to the limiting resource and account for what remains alive while work waits.

Try applying it: Which resource changes first if uploads become CPU-heavy compression jobs?

Unit 4 exercises → · Unit 4 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Operating Systems: Three Easy Pieces ↗

   University textbook. Runtime, concurrency, and persistence.

   Access reviewed · 21 October 2026

2. The Go Memory Model ↗

   Language specification · accessed 21 Oct 2026. One concrete memory-model contract; other languages differ.

   Access reviewed · 21 October 2026

Tasks, threads & event loops

Prerequisites: 4.1 · The runtime owns your resources 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

One event-loop callback computes for two seconds while other tasks are ready. Trace readiness and execution to see how nonblocking I/O and bounded pools affect progress.

4.2.1 · Blocking vs nonblocking I/O

An event loop starts a network read and arranges to be notified when progress is possible. While the I/O waits, the loop can run another ready task. Nonblocking I/O avoids holding the thread idle throughout that wait.

The mechanism still needs readiness or completion notifications. Repeatedly checking an unfinished operation without yielding can consume CPU instead of saving it. Use the platform's intended interface. If a required library blocks, it may need a separate, bounded executor. Limit both active blocking jobs and queued jobs, and define cancellation and cleanup. Nonblocking submission does not mean the requested work has finished.

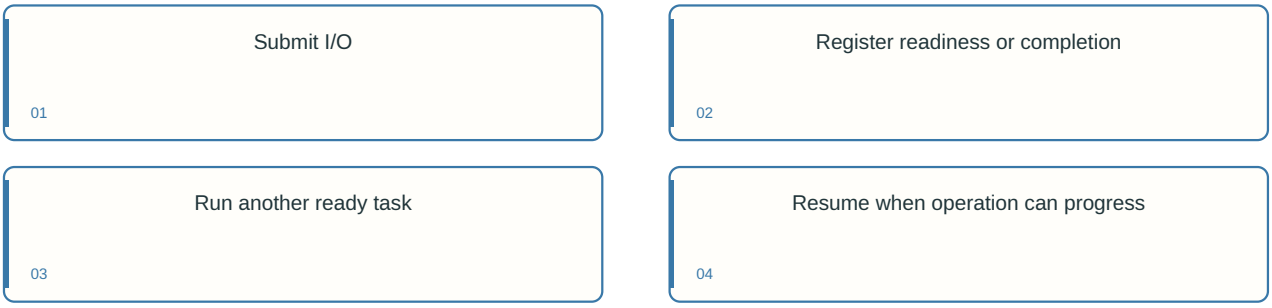


FIG. 4.2.1 **Blocking vs nonblocking I/O · the example**

Nonblocking I/O lets a thread perform other ready work during waits, using supported readiness or completion mechanisms.

Author-created illustration. Numbers are illustrative unless stated otherwise.

4.2.2 · Concurrency vs parallelism

Imagine two uploads on one core. Upload A starts, then waits for storage. The core works on B during that wait. Both activities are in progress over overlapping periods, so they are concurrent.

If two cores execute A and B at the same instant, there is parallelism as well. Concurrency concerns overlapping progress; parallelism concerns simultaneous execution. A one-core program can still have a race if operations are interleaved between a read and its matching write. Decide separately how much work may be in progress and how much can execute at once. Both choices affect resource use, while coordination determines correctness.

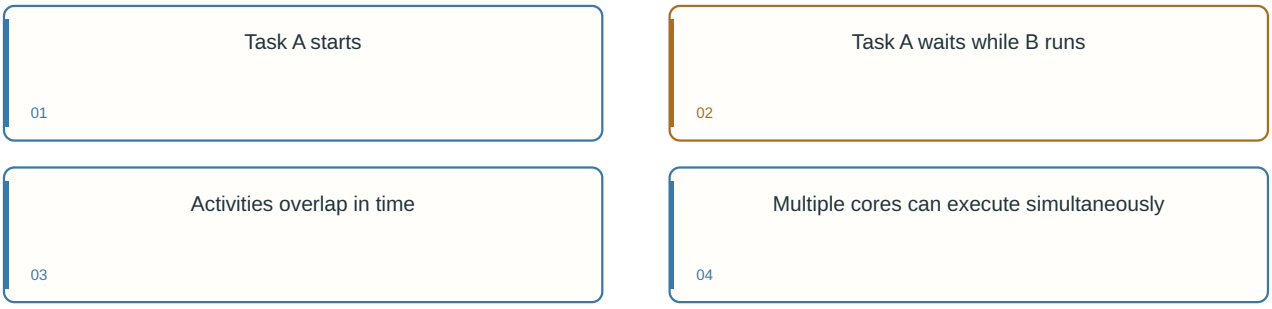


FIG. 4.2.2 **Concurrency vs parallelism · the example**

Concurrency permits overlapping progress; parallelism requires simultaneous execution resources.

Author-created illustration. Numbers are illustrative unless stated otherwise.

4.2.3 · Event loops, tasks, futures, promises

A task awaits a future representing a pending result. The event loop runs other ready work. When the result becomes available, the task's continuation becomes eligible to run. A promise or future is the runtime's representation of that later result; exact behaviour depends on the runtime.

Now suppose one callback computes continuously for two seconds on the only event-loop thread. Other ready callbacks must wait, even if their I/O already finished. Keep long computation away from the loop or split work appropriately. Scheduling a future also does not make the work durable: if its process dies, a memory-only task may disappear.

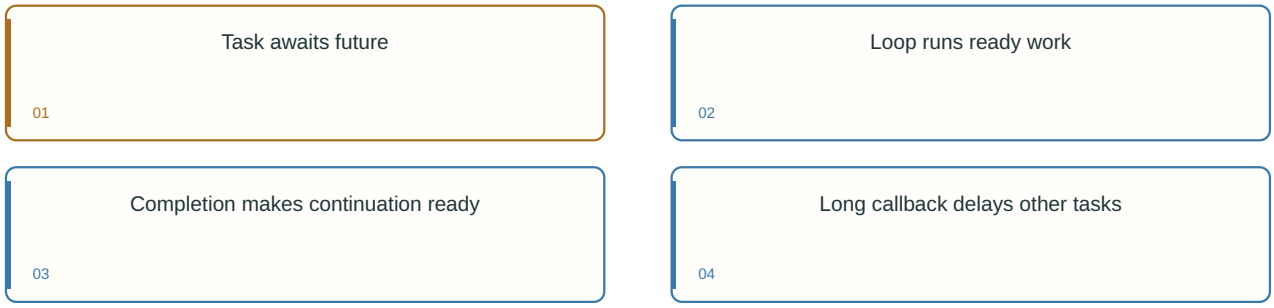


FIG. 4.2.3 **Event loops, tasks, futures, promises · the example**

An event loop runs ready work; futures or promises represent later results under the runtime's rules.

Author-created illustration. Numbers are illustrative unless stated otherwise.

4.2.4 · Thread pools and worker pools

Suppose eight workers process uploads. The pool limits active execution to eight jobs, but accepting a thousand large buffers into a waiting queue can still exhaust memory. Execution limits and admission limits solve different problems.

Use a bounded queue as well as a bounded worker count. Where practical, store large upload bytes separately and queue compact references. Decide what happens when the queue is full: reject, wait until a deadline, or use another supported path. A durable queue is needed if accepting the work promises survival across process loss. Count queued payloads, active buffers, and open resources when choosing the limits.

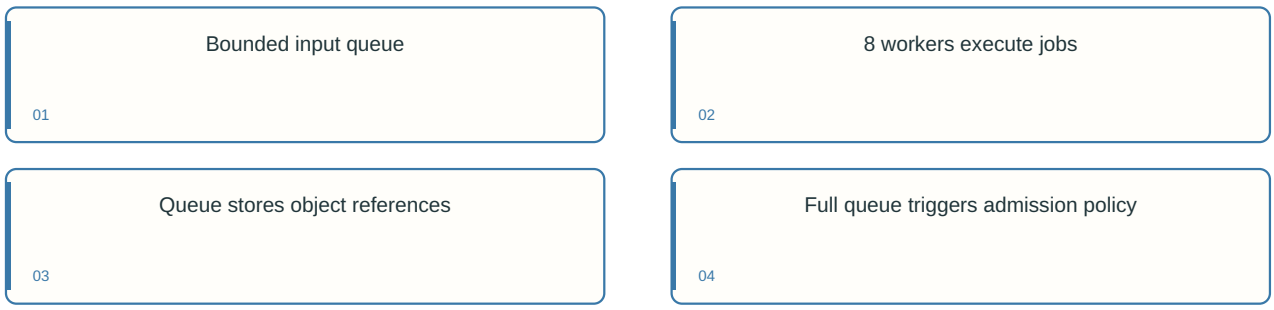


FIG. 4.2.4 **Thread pools and worker pools · the example**

A worker pool bounds active jobs; a bounded queue separately limits waiting jobs and retained payloads.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · Who should own accepted upload work after its initiating request is cancelled?

If cancellation only ends the connection but the business still requires processing, a durable job or workflow must retain identity, input, and progress. Cancel and release request-owned resources without discarding that obligation. If the work itself should stop, persist the chosen cancellation and block inappropriate later completion.

Remember: Overlapping tasks do not necessarily execute in parallel, and an event loop still needs short runnable turns.

Try applying it: Why can adding async tasks leave a CPU-bound loop unresponsive?

Unit 4 exercises → · Unit 4 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Operating Systems: Three Easy Pieces ↗

   University textbook. Runtime, concurrency, and persistence.

   Access reviewed · 21 October 2026

2. The Go Memory Model ↗

   Language specification · accessed 21 Oct 2026. One concrete memory-model contract; other languages differ.

   Access reviewed · 21 October 2026

Shared memory & coordination

Prerequisites: 4.2 · Tasks, threads & event loops 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Two increments leave a counter at one. Trace each read and write, then choose a coordination boundary that covers the whole rule rather than only one assignment.

4.3.1 · Shared memory vs message passing

Two threads can update a shared inventory map directly, provided they coordinate access. Alternatively, one owner can keep the map private and process inventory commands sent by other tasks. These are shared-memory and message-passing approaches.

A single owner can make some local rules easier to preserve, but its messages may repeat and the owner can fail. Define delivery and recovery behaviour. Sending a pointer to an object that both sides can still mutate has not transferred exclusive ownership. Choose the arrangement by where state belongs and how much communication it needs. Both approaches need clear rules for concurrent access and failure.

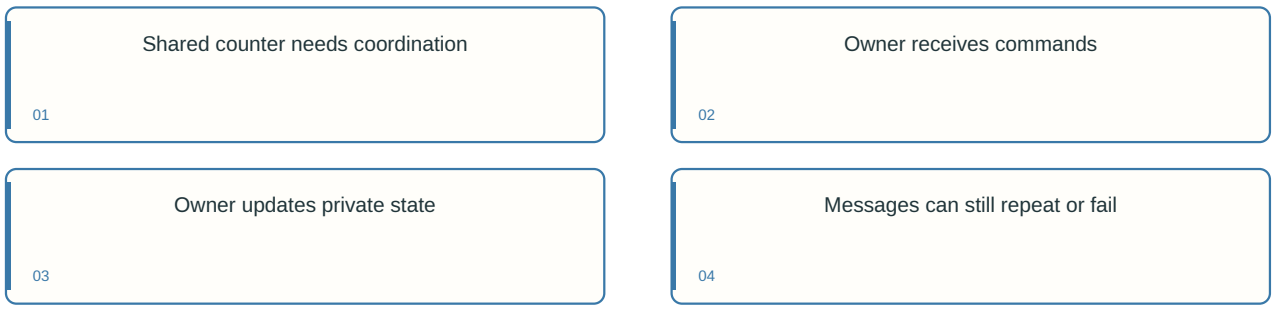


FIG. 4.3.1 **Shared memory vs message passing · the example**

Shared memory needs access coordination; message passing needs ownership, delivery, and recovery rules.

Author-created illustration. Numbers are illustrative unless stated otherwise.

4.3.2 · Race conditions and critical sections

Start a counter at zero. A reads zero. B reads zero before A writes. A adds one and writes one; B also adds one to its earlier value and writes one. Two increments have produced a final value of one.

This is a lost update caused by a race: the outcome depends on how operations interleave. Protect the complete read–modify–write sequence as a critical section, using a suitable lock or atomic operation. Locking only the final assignment leaves the earlier reads unprotected. The protection must be shared by all relevant writers. A mutex inside one process cannot coordinate unrelated processes writing a database.

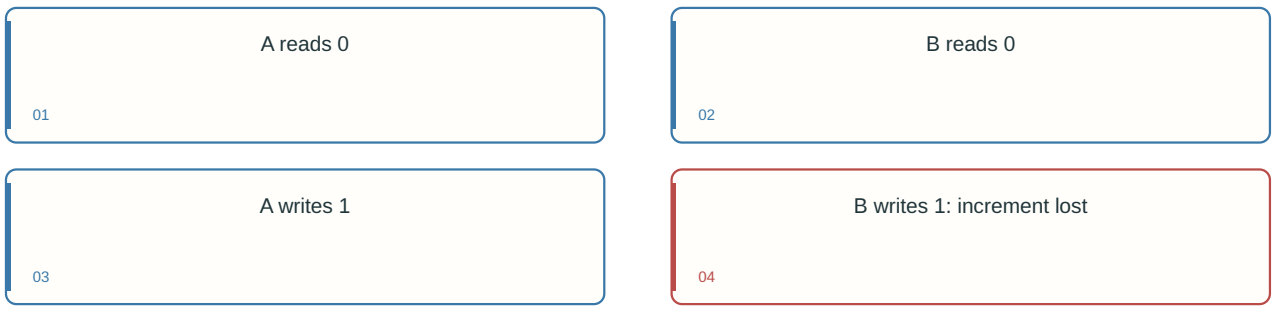


FIG. 4.3.2 **Race conditions and critical sections · the example**

A race can make an outcome depend on interleaving; protect the full critical state transition.

Author-created illustration. Numbers are illustrative unless stated otherwise.

4.3.3 · Mutexes, semaphores, atomics, condition variables

Different coordination tools answer different questions. A mutex allows one holder into a protected section. A semaphore permits a limited number of holders. An atomic operation makes a specified action indivisible under its defined memory-order rules.

A condition variable lets a thread wait for a condition while releasing a lock, then reacquire it before proceeding. Recheck the condition in a loop: waking does not establish that it is still true. Choose the protected state by the invariant. If a transfer changes two balances together, making each balance individually atomic does not automatically make the whole transfer atomic.

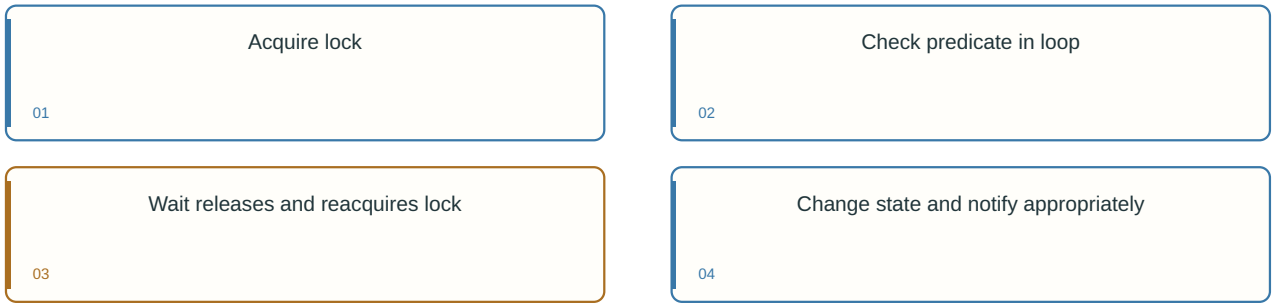


FIG. 4.3.3 **Mutexes, semaphores, atomics, condition variables · the example**

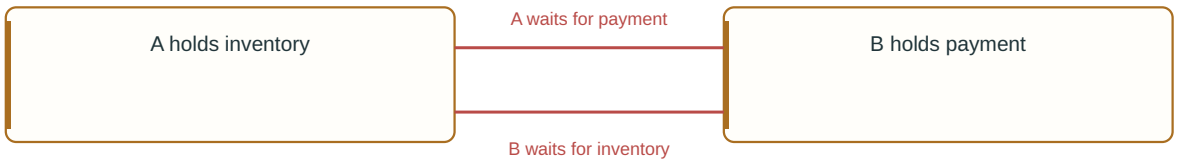
Mutexes, semaphores, atomics, and condition variables coordinate different kinds of access, permits, operations, and waiting.

Author-created illustration. Numbers are illustrative unless stated otherwise.

4.3.4 · Deadlocks, starvation, livelocks

A holds the inventory lock and waits for the payment lock. B holds the payment lock and waits for inventory. Neither can continue. This circular wait is a deadlock.

Starvation occurs when a participant repeatedly fails to obtain the service or resource it needs. Livelock occurs when participants remain active but make no useful progress, for example by repeatedly yielding and retrying in the same pattern. Consistent lock ordering can prevent the illustrated deadlock. Shorter critical sections or different ownership may help too. A timeout needs cleanup and rollback; repeating the same failed coordination indefinitely can merely replace one problem with another.



Two opposing wait dependencies form a cycle; neither holder can proceed.

FIG. 4.3.4 **A circular wait prevents both participants progressing**

A holds inventory and waits for payment; B holds payment and waits for inventory. Consistent resource ordering or redesigned ownership can remove this cycle.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · Who should own accepted upload work after its initiating request is cancelled?

If cancellation only ends the connection but the business still requires processing, a durable job or workflow must retain identity, input, and progress. Cancel and release request-owned resources without discarding that obligation. If the work itself should stop, persist the chosen cancellation and block inappropriate later completion.

Remember: Protect the complete invariant at the actual shared-state boundary.

Try applying it: Which lock must both callers use, and why would separate per-caller locks fail?

Unit 4 exercises → · Unit 4 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Operating Systems: Three Easy Pieces ↗

   University textbook. Runtime, concurrency, and persistence.

   Access reviewed · 21 October 2026

2. The Go Memory Model ↗

   Language specification · accessed 21 Oct 2026. One concrete memory-model contract; other languages differ.

   Access reviewed · 21 October 2026

Visibility, cancellation & overload

Prerequisites: 4.3 · Shared memory & coordination 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A request ends while its child work still holds a large buffer and a connection. Connect publication, ownership, cancellation, and admission to keep shared state and resource use controlled.

4.4.1 · Memory visibility and happens-before relationships

A thread constructs an object and publishes a reference to another thread. For the reader to use it correctly, the language's memory model must guarantee visibility of the initialised fields. An earlier wall-clock time alone does not supply that guarantee.

Happens-before is an ordering relation used by memory models to describe which effects must be visible. Synchronisation mechanisms, such as an appropriate channel or lock, can establish the required relationship. Use the rules of the actual language and runtime rather than assuming all machines behave the same way. The Go memory model is one specified example. Safe publication matters even when the object will not change afterward.

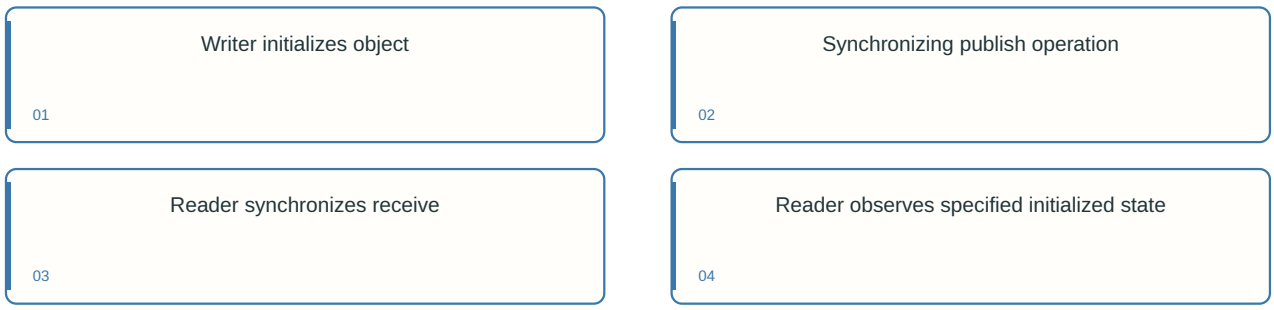


FIG. 4.4.1 **Memory visibility and happens-before relationships · the example**

Happens-before describes memory-model ordering and visibility; earlier wall-clock time alone does not establish it.

Author-created illustration. Numbers are illustrative unless stated otherwise.

4.4.2 · Thread safety and immutability

Suppose task records never change after construction. Readers can share those immutable values without competing to modify their fields. But a mutable dictionary that holds the records can still have concurrent inserts and removals.

Thread safety is a claim about allowed concurrent use of the whole operation or object. Immutability removes some mutation problems; publication and surrounding containers still need coordination. Other approaches include keeping data inside one owner, copying values, or using locks. State which access is allowed and who protects it. Returning a writable internal collection lets callers bypass the rules even if each record looks safe on its own.

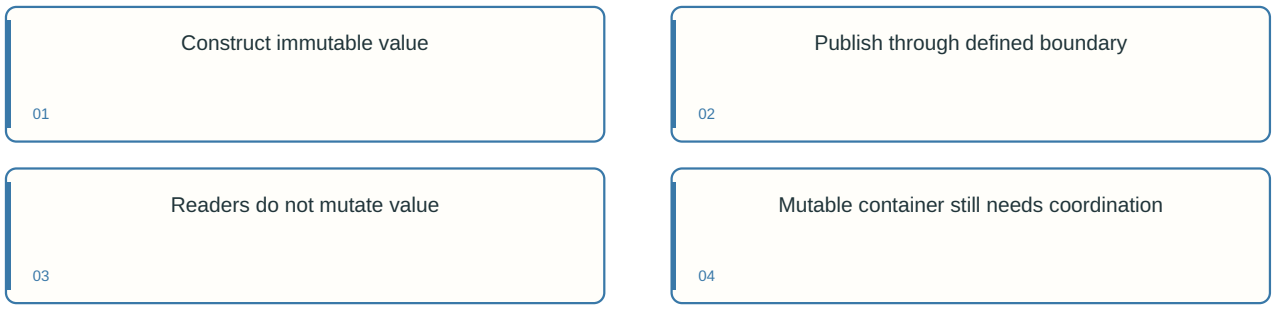


FIG. 4.4.2 **Thread safety and immutability · the example**

Thread safety states allowed concurrent behaviour; immutable values still need valid publication and surrounding ownership.

Author-created illustration. Numbers are illustrative unless stated otherwise.

4.4.3 · Resource exhaustion and backpressure

Assume jobs arrive at 150 per second while workers complete 100 per second. If every job is accepted, the waiting queue grows by fifty each second. A larger queue postpones running out of space but leaves the mismatch unchanged.

Backpressure slows or limits upstream work according to downstream capacity. Use bounded queues, admission deadlines, and clear overload responses. Keep queued payloads compact. Monitor the age of the oldest job as well as the number waiting: a short queue of stuck jobs may still be harmful. When a dependency slows, unlimited concurrency turns its delay into memory, connection, and thread exhaustion elsewhere.

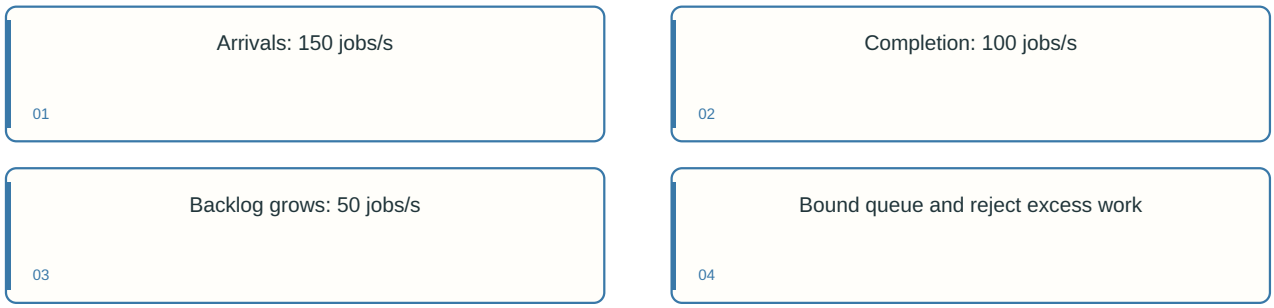


FIG. 4.4.3 **Resource exhaustion and backpressure · the example**

Backpressure limits upstream work according to available capacity; larger queues do not repair sustained overload.

Author-created illustration. Numbers are illustrative unless stated otherwise.

4.4.4 · Cancellation and structured concurrency

An upload request starts several child tasks. If its deadline expires, someone must cancel work that is no longer needed and release local resources. Structured concurrency gives those child tasks a parent scope responsible for their completion and failure.

Cancellation is a request to stop, with behaviour defined by the runtime and operation. It cannot prove that a remote side effect was undone. A payment may already have committed. Use reliable local cleanup for files and connections, and a status or reconciliation process for uncertain remote effects. If accepted work must continue after the request process dies, give it a durable job record and an explicit owner.

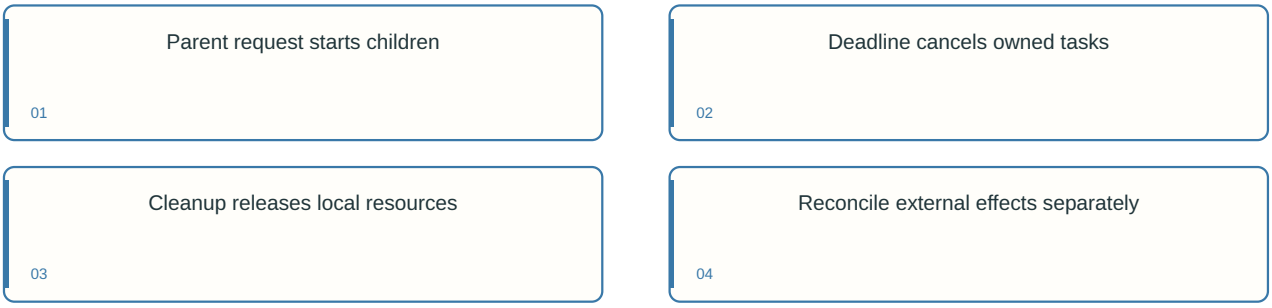


FIG. 4.4.4 **Cancellation and structured concurrency · the example**

Structured concurrency gives child work a scope owner; cancellation requires cleanup and cannot automatically undo remote effects.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Try the mechanism in code

PSEUDOCODE · BOUNDED UPLOAD OWNERSHIP

```
queue = bounded_queue(capacity=100)
start_workers(count=50)
for upload in arrivals:
    reference = persist_upload_or_reject(upload)
    queue.put_with_deadline(reference)
worker():
    while reference = queue.take():
        try: process(reference)
        finally: release_owned_resources()
```

The queue holds at most one hundred waiting references, and fifty workers limit active jobs. The large bytes remain in storage.

A failed handoff still needs an explicit durable-job or rejection policy. A process-local queue alone cannot promise to resume accepted work after its process disappears.

Download the executable mechanism collection ↓ · Follow the connected complete design →

THE WORKBENCH · TRY CHANGING THE INPUTS

What happens when jobs arrive faster than they finish?

Arrived: **9,000** · completed: **6,000** · waiting: **100** · rejected: **2,900**.

Every arrived job is completed, waiting, or rejected. A larger waiting limit lets more jobs wait, but workers still finish at the same rate.

The queue starts empty. Jobs arrive and finish at constant rates, treated as a continuous flow. Increase the waiting limit and compare waiting with rejection. The model omits startup time, scheduling, and differences between individual jobs.

1,000 uploads · compare the resource that dominates

Assume each independent upload needs exactly 100 ms of waiting for storage and no meaningful CPU work. Ignore startup, connection limits, bandwidth, queue overhead, and variance. Sequential execution then needs 100 seconds; allowing 50 independent waits at once has an ideal lower bound of 2 seconds. These are constructed lower bounds, not measurements.

`Same I/O workload, different execution and recovery boundaries`

Execution model	Ideal elapsed time	What must be bounded	Crash recovery
Sequential	100 s	One active upload; backlog elsewhere	Requires a durable record
50 threads with blocking I/O	2 s	Threads, stacks, connections	Memory alone loses accepted work
Async I/O with 50 in flight	2 s	Tasks, buffers, sockets	Memory alone loses accepted work
Durable queue, 50 workers	At least 2 s, plus queue overhead	Admission, queue age, worker count	Durable intent plus retry and idempotent completion

Change the workload: each upload instead requires 20 ms of CPU and negligible I/O. The batch needs 20 CPU-seconds. One executing core therefore needs at least 20 seconds; eight truly parallel cores have an ideal lower bound of 2.5 seconds. An event loop creates no extra cores. Runtime restrictions can also prevent CPU-heavy threads from running in parallel. Measure overhead and contention before choosing

threads or processes. A durable queue changes acknowledgment and recovery behavior; it does not erase service demand.

▼ Think further · Who should own accepted upload work after its initiating request is cancelled?

If cancellation only ends the connection but the business still requires processing, a durable job or workflow must retain identity, input, and progress. Cancel and release request-owned resources without discarding that obligation. If the work itself should stop, persist the chosen cancellation and block inappropriate later completion.

Remember: Safe concurrent use includes visibility, lifetimes, and bounded waiting as well as individual state updates.

Try applying it: What must happen to local resources and required durable work when the parent request is cancelled?

Unit 4 exercises → · Unit 4 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Operating Systems: Three Easy Pieces ↗

   University textbook. Runtime, concurrency, and persistence.

   `Access reviewed · 21 October 2026`

2. The Go Memory Model ↗

   Language specification · accessed 21 Oct 2026. One concrete memory-model contract; other languages differ.

   `Access reviewed · 21 October 2026`

PART 2 / UNIT 05

Object-Oriented and Modular Software Design

One payment class checks orders, creates an SDK, and stores results. Divide the responsibilities, then examine which object relationships and contracts let those decisions change safely.

The running example

Start with a poorly designed payment-processing class. Refactor it into appropriate components, showing which problems each design improvement solves.

Unit 5 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

Responsibility	Before	After
Validation	Payment god class	Task/domain policy
Provider calls	Hardcoded SDK creation	Injected gateway adapter
Persistence	Mixed with rendering	Owned repository contract

FIG. 5.0.1 **Before/after class diagrams**

Refactoring moves knowledge while preserving approved, declined, and uncertain outcomes.

Author-created illustration. Numbers are illustrative unless stated otherwise.

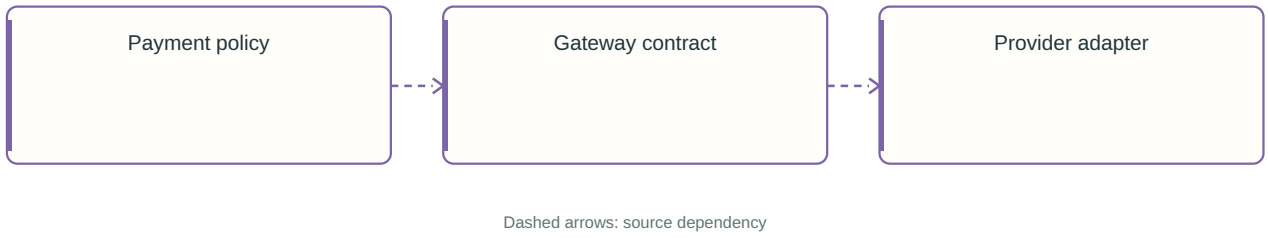


FIG. 5.0.2 **dependency graph**

Dashed arrows denote source dependencies around an application-owned contract.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Variation	Inheritance approach	Composition approach
Channel × content	Subclass combinations	Content receives transport
Behavioral contract	Subclass must substitute	Collaborator contract explicit
Cost	Hierarchy coupling	Wiring and interfaces

FIG. 5.0.3 **inheritance vs composition**

Composition can separate independent variation; inheritance is appropriate only with valid behavioral substitution.

Author-created illustration. Numbers are illustrative unless stated otherwise.

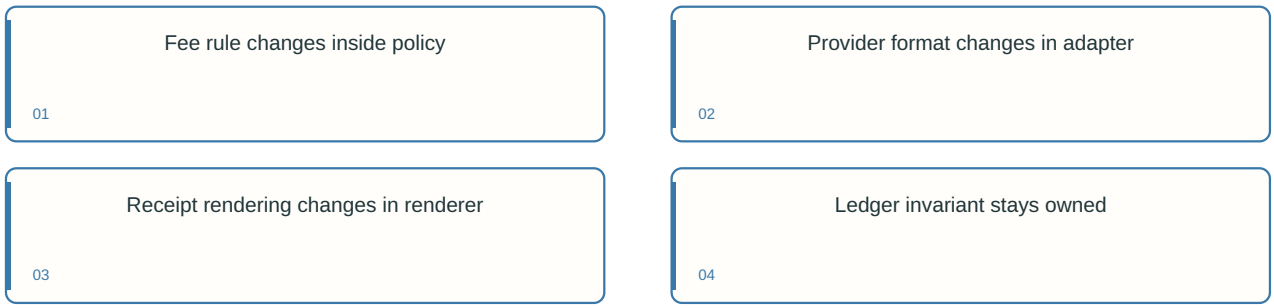


FIG. 5.0.4 **cohesion/coupling illustrations**

Cohesion and coupling are evaluated through concrete change propagation, not class-count metrics.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Model behavior with objects and values

Prerequisites: Introduction to System Design 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

One payment class checks orders, creates an SDK, and stores results. Divide the responsibilities, then examine which object relationships and contracts let those decisions change safely.

5.1.1 · Abstraction, encapsulation, polymorphism, inheritance

Imagine one payment class that checks an order, constructs a provider SDK, calculates fees, and writes audit records. A fee change and a provider change both require editing it. Begin by separating those kinds of work.

Abstraction lets callers request a payment without knowing the provider protocol. Encapsulation controls access to payment state so changes follow its rules. Polymorphism lets different implementations satisfy the same sending contract. Inheritance is one way to specialise behaviour, but the other benefits can also be achieved with composed objects or functions. Keep rules such as valid amounts and allowed payment transitions under application control.

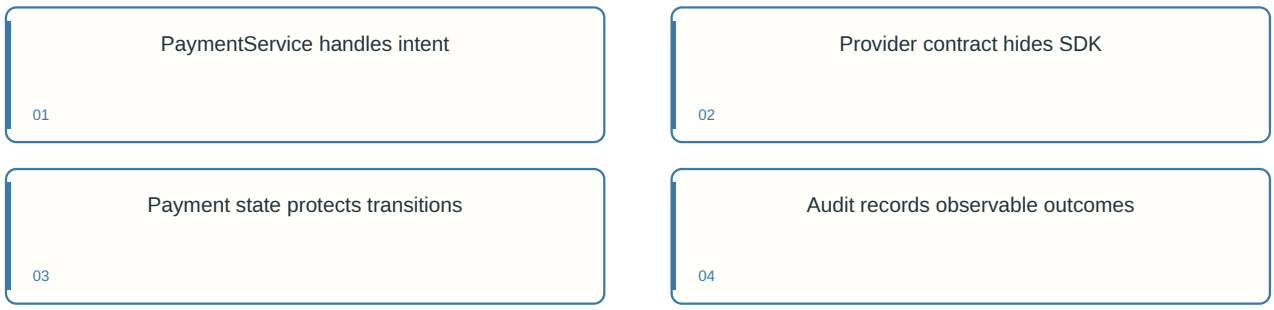


FIG. 5.1.1 **Abstraction, encapsulation, polymorphism, inheritance · the example**

Abstraction presents needed behaviour; encapsulation controls representation; polymorphism permits compatible implementations; inheritance specialises a relationship.

Author-created illustration. Numbers are illustrative unless stated otherwise.

5.1.2 · Composition vs inheritance

Suppose callers use an Account contract that permits withdrawal. A ReadOnlyAccount subclass that always rejects withdrawal changes an assumption those callers rely on. Sharing fields through inheritance has not established a valid behavioural relationship.

Composition assembles collaborators instead. A payment service can receive a fraud policy and a gateway, varying either without creating a subclass for every combination. It adds wiring and contracts, so use it where those variations are real. Inheritance remains useful when the specialised object consistently fulfils the inherited contract. Judge the relationship by what callers may do, rather than by similar fields or convenient code reuse.

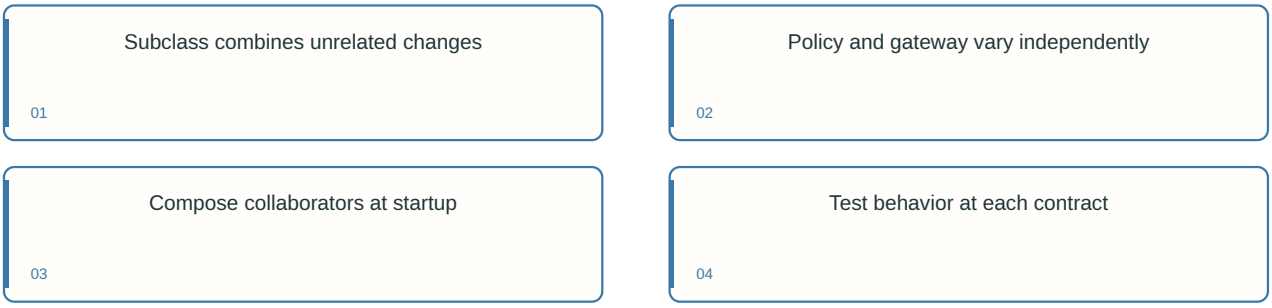


FIG. 5.1.2 **Composition vs inheritance · the example**

Composition combines collaborators, while inheritance requires subclasses to preserve their callers' behavioural expectations.

Author-created illustration. Numbers are illustrative unless stated otherwise.

5.1.3 · Interfaces and contracts

What does charge(100) mean? It might mean one hundred dollars, one dollar expressed in cents, or another currency entirely. A gateway interface needs more than a method signature.

State the currency, amount units, operation identity, and possible outcomes. A payment may be approved, declined, or uncertain, with a provider reference when available. A timeout after submission is uncertain because the provider may already have charged. This behavioural contract lets the application replace adapters without silently changing payment meaning. Test the real adapter as well as a substitute: a fake returning “approved” cannot establish how the provider handles lost replies or repeated operations.

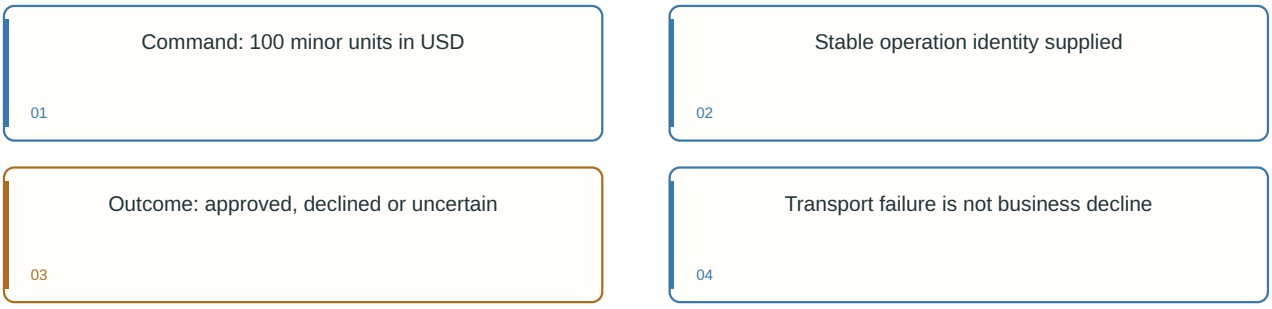


FIG. 5.1.3 **Interfaces and contracts · the example**

A contract specifies valid inputs, outcomes, effects, errors, and relevant timing or repetition rules beyond a signature.

Author-created illustration. Numbers are illustrative unless stated otherwise.

5.1.4 · High cohesion and low coupling

A fee rule belongs with related payment rules. An SMS receipt formatter has a different reason to change. If adding SMS requires editing ledger arithmetic, the responsibilities are entangled.

Cohesion describes how closely the work inside a module belongs together. Coupling describes its dependencies on other modules. A focused payment module can use a narrow gateway contract without knowing provider field names. Test the arrangement with likely changes: where would a new receipt format or provider go? Avoid dividing work so finely that a simple action becomes difficult to follow. Extract a module when it protects a rule, hides changeable details, or gives a real change a clear home.

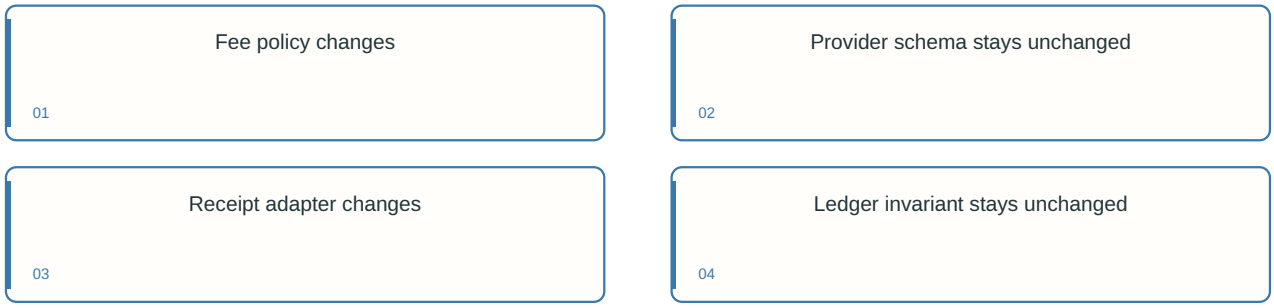


FIG. 5.1.4 **High cohesion and low coupling · the example**

Cohesion concerns related work inside a module; coupling concerns dependencies and knowledge between modules.

Author-created illustration. Numbers are illustrative unless stated otherwise.

5.1.5 · SOLID principles, including their limitations

SOLID collects five design ideas. Single responsibility asks whether a module has a focused reason to change. Open/closed encourages useful extension points. Substitution requires implementations to preserve callers' expected behaviour. Interface segregation favours contracts suited to their callers. Dependency inversion places important policy behind abstractions it controls.

For example, the payment service can use a small gateway interface implemented by a provider adapter. These are guidelines for judging a design, not a demand for an interface around every class. An elaborate extension system can cost more to understand than a direct edit. Apply each idea to an actual change or testing problem.

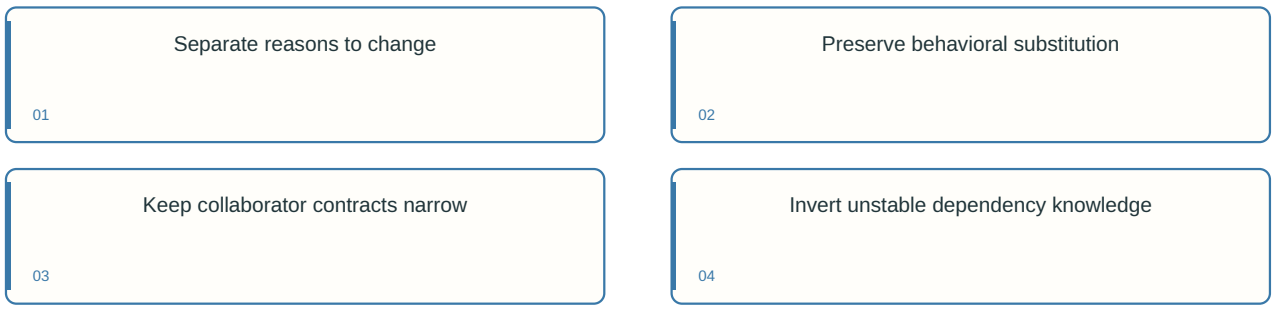


FIG. 5.1.5 **SOLID principles, including their limitations · the example**

SOLID offers guidelines for responsibilities, useful extension, substitution, focused interfaces, and dependency direction.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · Which payment rules should remain the same when objects are replaced by pure functions?

Currency, units, allowed transitions, operation identity, and uncertainty remain business rules regardless of representation. Pure decision functions can make them easier to test. A separate effect interpreter still needs authorisation, transactions, provider identity, and reconciliation. Refactoring should preserve observed outcomes, not merely move methods into functions.

Remember: Organise payment rules and provider details so callers rely on meaningful behaviour.

Try applying it: Which part should change when the provider renames a field but the payment rule stays the same?

Unit 5 exercises → · Unit 5 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. On the criteria to be used in decomposing systems into modules — D. L. Parnas, 1972 ↗

   Original research · ACM DOI. Module decomposition criteria; metadata and abstract verified.

   Access reviewed · 21 October 2026

2. The Catalog of Design Patterns — Refactoring Guru ↗

   Engineering teaching reference. Pattern taxonomy; miniature examples and judgments are original to this atlas.

   Access reviewed · 21 October 2026

Principles under pressure

Prerequisites: 5.1 · Model behavior with objects and values 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Two pieces of code look alike, but do they express the same rule? Apply design heuristics to actual changes instead of treating every repetition or method chain as a defect.

5.2.1 · DRY, KISS, YAGNI

Suppose every gateway contains a copy of the same currency-conversion rule. A policy change can leave the copies disagreeing. This is duplicated knowledge: the concern behind DRY, “Don't Repeat Yourself.” Two similar-looking checks for genuinely different policies need not be merged.

KISS asks us to keep the required design understandable. YAGNI, “You Aren't Gonna Need It,” discourages building speculative features before there is a need. Together, these ideas suggest a practical approach: identify the rule and its owner, share that knowledge where appropriate, and keep the implementation small enough to change. Similar syntax alone is weak evidence that two responsibilities belong together.

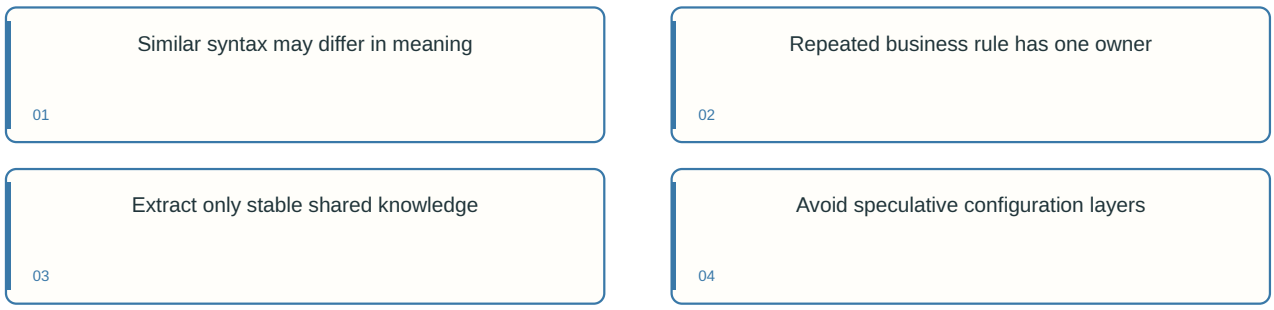


FIG. 5.2.1 **DRY, KISS, YAGNI · the example**

DRY avoids duplicated knowledge; KISS favours understandable required design; YAGNI defers unsupported speculative capability.

Author-created illustration. Numbers are illustrative unless stated otherwise.

5.2.2 · Law of Demeter

Consider order.customer.account.settings.currency. The caller now knows the shape of several objects. Changing the account's settings representation may force that caller to change too.

The Law of Demeter encourages limiting such knowledge of distant collaborators. An operation such as order.paymentCurrency() can express what the caller actually needs. The value is the reduced dependence on internal structure. Counting dots is not the objective: a fluent builder or a deliberate traversal of data can be clear. Avoid adding forwarding methods that hide nothing useful. Ask which representation details the caller would otherwise have to understand and maintain.

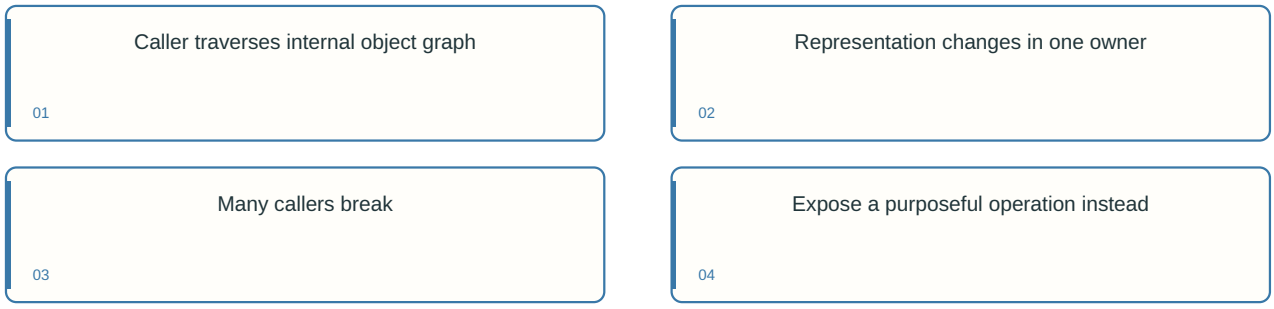


FIG. 5.2.2 **Law of Demeter · the example**

The Law of Demeter discourages unnecessary dependence on distant objects' internal structure.

Author-created illustration. Numbers are illustrative unless stated otherwise.

5.2.3 · Tell, Don't Ask

A caller reads an account balance, subtracts an amount, and writes the result. It has taken responsibility for a rule that belongs with the account. Another caller may implement the rule differently, and simultaneous updates can conflict.

“Tell, Don't Ask” suggests asking the owner to reserve(amount). The owner can check the allowed balance and perform the change under the appropriate coordination. Reading balances for a report remains useful. Keep reads predictable and effects explicit. Moving a method onto an object also does not supply concurrency control: when several processes share the account, its storage boundary must enforce the rule atomically.

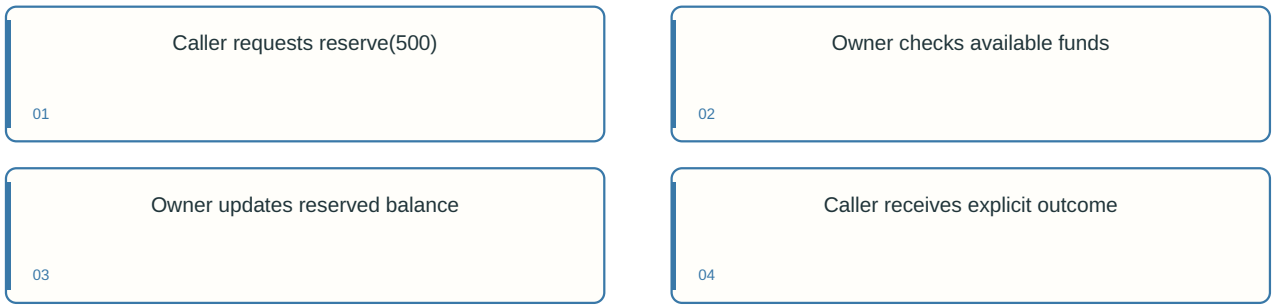


FIG. 5.2.3 **Tell, Don't Ask · the example**

Tell, Don't Ask favours sending intentions to the owner that can protect the relevant state rule.

Author-created illustration. Numbers are illustrative unless stated otherwise.

5.2.4 · Command-query separation

A call named payment.status() should report the payment's state. If reading status unexpectedly charges a card, callers cannot use it safely as a query.

Command-query separation distinguishes operations that change state from those that report information. A create command may still return a useful result, such as the new identifier, provided its effect is clear. Document that convention. CQRS goes further by giving commands and queries different models where their needs justify it. Ordinary command-query separation does not require separate databases, services, or event histories. Begin with clear operation meanings before considering a larger architecture.

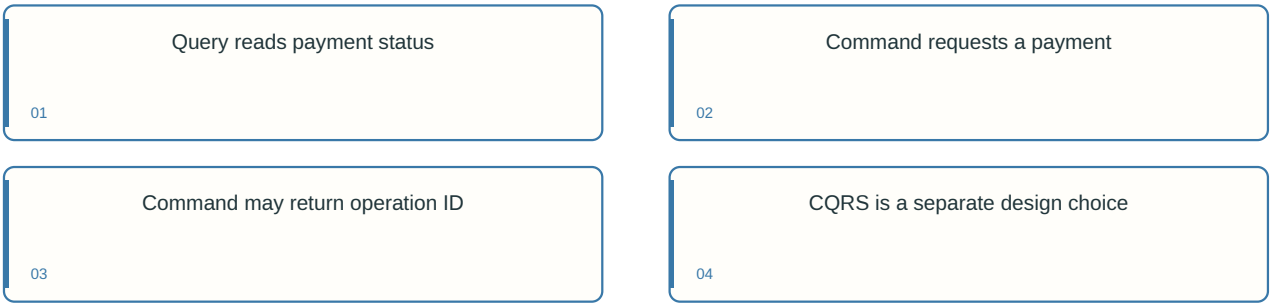


FIG. 5.2.4 **Command-query separation · the example**

Command-query separation makes state-changing operations distinguishable from reads without requiring separate deployments.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · Which payment rules should remain the same when objects are replaced by pure functions?

Currency, units, allowed transitions, operation identity, and uncertainty remain business rules regardless of representation. Pure decision functions can make them easier to test. A separate effect interpreter still needs authorisation, transactions, provider identity, and reconciliation. Refactoring should preserve observed outcomes, not merely move methods into functions.

Remember: Share knowledge that has one meaning and owner; keep effects and intentions explicit.

Try applying it: When would merging similar validation code create a dependency between unrelated policies?

Unit 5 exercises → · Unit 5 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. On the criteria to be used in decomposing systems into modules — D. L. Parnas, 1972 ↗

   Original research · ACM DOI. Module decomposition criteria; metadata and abstract verified.

   Access reviewed · 21 October 2026

2. The Catalog of Design Patterns — Refactoring Guru ↗

   Engineering teaching reference. Pattern taxonomy; miniature examples and judgments are original to this atlas.

   Access reviewed · 21 October 2026

Side effects, domain models & collaboration

Prerequisites: 5.2 · Principles under pressure 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A payment keeps its identity while its status changes; a Money value is defined by its amount and currency. Use these differences to organise collaboration and test observable outcomes.

5.3.1 · Immutability and side-effect management

Suppose several helpers edit the same payment summary. To understand its final value, a reader must track every mutation. An immutable value gives a simpler alternative: calculate a new summary and leave the old one unchanged.

Keep pure calculations, such as fee arithmetic, separate from effects such as provider calls and database writes. Inputs and returned values then make the calculation easier to test. Copying and allocation have costs, which should be measured when significant. Immutability also has a limited scope: an immutable payment command can still be executed twice, and it cannot undo an external charge. Execution still needs authorisation and an operation identity.

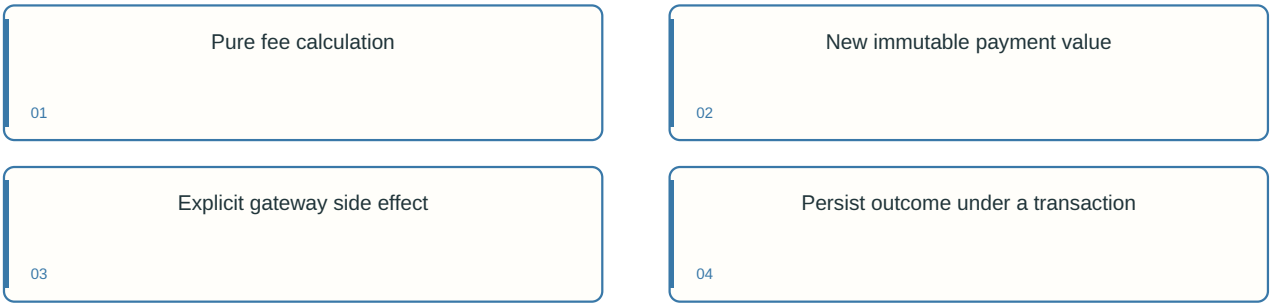


FIG. 5.3.1 **Immutability and side-effect management · the example**

Immutable values and explicit effects make changes easier to follow, with allocation and execution costs still considered.

Author-created illustration. Numbers are illustrative unless stated otherwise.

5.3.2 · Domain models, entities, value objects

Payment 42 is still the same payment after its status changes. Its identity matters, so it is an entity. Two Money values with the same currency and minor-unit amount can represent the same value without separate identities; they are value objects.

A domain model expresses business concepts and their rules. Money arithmetic should reject incompatible currencies unless an explicit conversion rule applies. A domain service can express a rule that does not belong naturally to one entity. Put important rules where callers can consistently use them. Simple transport data need not become an elaborate object model, but scattering payment invariants across controllers makes them difficult to preserve.

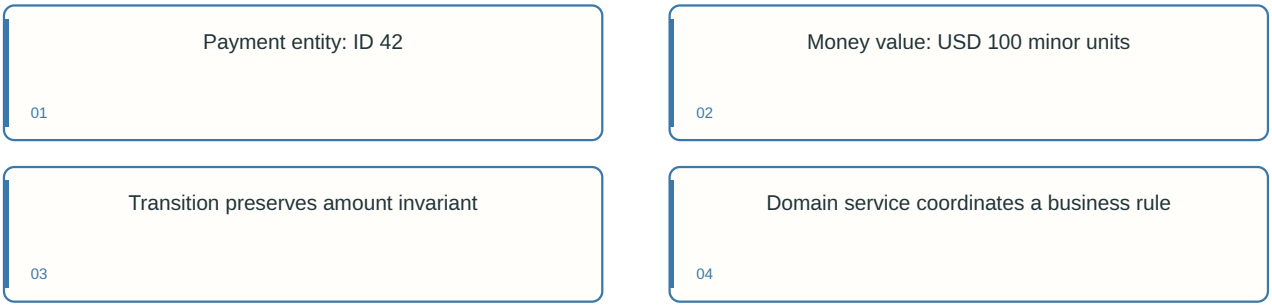


FIG. 5.3.2 **Domain models, entities, value objects · the example**

Entities retain identity; value objects represent equal values; domain models express meaningful concepts and rules.

Author-created illustration. Numbers are illustrative unless stated otherwise.

5.3.3 · Object collaboration

During checkout, an orchestrator asks inventory to reserve items, asks payments to authorise, and records the order's progress. Each collaborator owns its own rules and returns an explicit result.

An object collaboration describes these requests and responsibilities, rather than only the fields each object holds. Inventory should not quietly edit payment records. If one step fails after another succeeds, the orchestrator needs enough information to recover or compensate. Also consider where calls run. Many small in-process calls may be cheap; moving the same conversation across a network adds waiting and partial failures. Design a remote contract for that boundary rather than automatically copying the object interface.

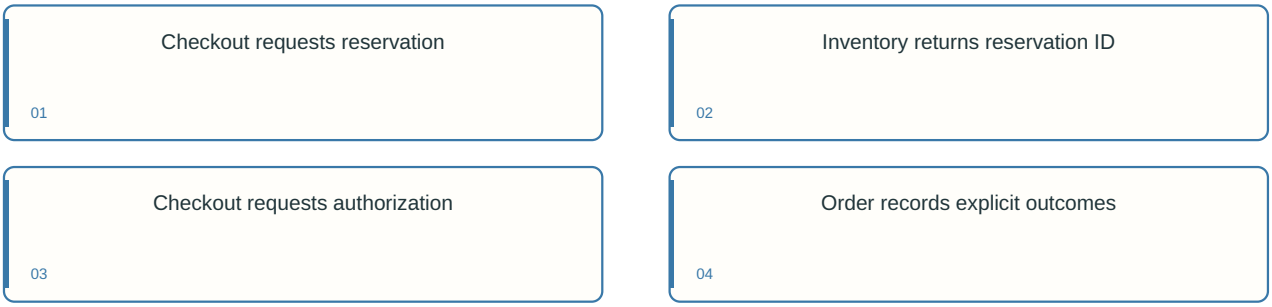


FIG. 5.3.3 **Object collaboration · the example**

Object collaboration describes requests, results, and ownership between components, including partial-failure handling.

Author-created illustration. Numbers are illustrative unless stated otherwise.

5.3.4 · Testability and design for change

To test a declined payment, give the service a test gateway that returns “declined.” The test can then check the order's resulting state without contacting a real provider. Controllable inputs and visible outcomes make such tests useful.

A test that only expects a particular sequence of private method calls may fail after a harmless refactor while missing a business error. Prefer checks of the required result and state. Keep integration checks for the real boundaries: encoding, transactions, timeouts, and cleanup. A substitute gateway helps test orchestration but cannot establish the actual provider's behaviour. Choose evidence appropriate to the claim being checked.

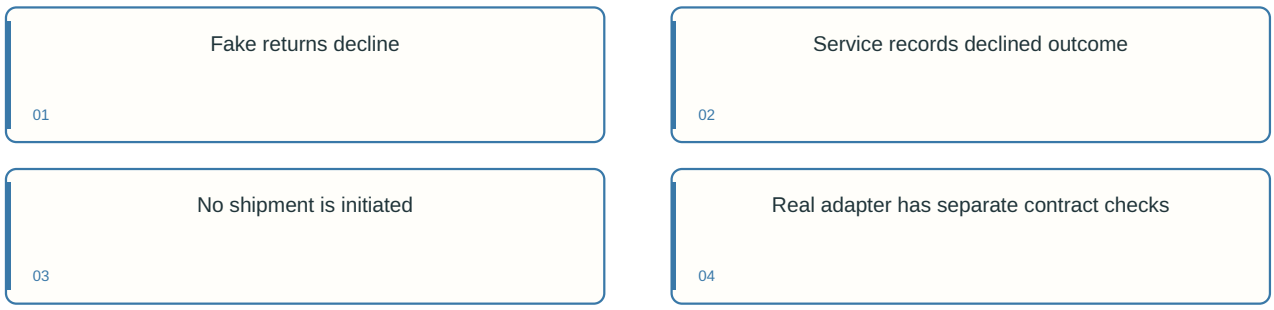


FIG. 5.3.4 **Testability and design for change · the example**

Testable design gives controlled inputs and visible outcomes; substitutes and real integration tests establish different evidence.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · Which payment rules should remain the same when objects are replaced by pure functions?

Currency, units, allowed transitions, operation identity, and uncertainty remain business rules regardless of representation. Pure decision functions can make them easier to test. A separate effect interpreter still needs authorisation, transactions, provider identity, and reconciliation. Refactoring should preserve observed outcomes, not merely move methods into functions.

Remember: Keep identity, values, effects, and coordination distinct enough to reason about their changes.

Try applying it: Which checks can a fake gateway establish, and which require the real storage or provider boundary?

Unit 5 exercises → · Unit 5 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. On the criteria to be used in decomposing systems into modules — D. L. Parnas, 1972 ↗

   Original research · ACM DOI. Module decomposition criteria; metadata and abstract verified.

   Access reviewed · 21 October 2026

2. The Catalog of Design Patterns — Refactoring Guru ↗

   Engineering teaching reference. Pattern taxonomy; miniature examples and judgments are original to this atlas.

   Access reviewed · 21 October 2026

Design for change, testing & functions

Prerequisites: 5.3 · Side effects, domain models & collaboration 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Refactoring a payment class should preserve approval, decline, and uncertainty. Follow those behaviours through structural changes and compare object-based and functional ways to express the decisions.

5.4.1 · Refactoring and recognizing code smells

Before reorganising a large payment class, capture its important behaviours: successful payment, decline, timeout, and storage failure. In particular, keep an uncertain provider outcome distinct from a definite decline.

Refactoring changes structure while preserving the intended behaviour. Extract input parsing, fee calculation, provider communication, and storage in small steps, checking those behaviours after each change. A long method or repeated conditional suggests a place to investigate; it does not automatically call for a design pattern. Tests focused on outcomes allow code to move without tying the design to every old internal call. If the business behaviour changes, describe and test that change explicitly.

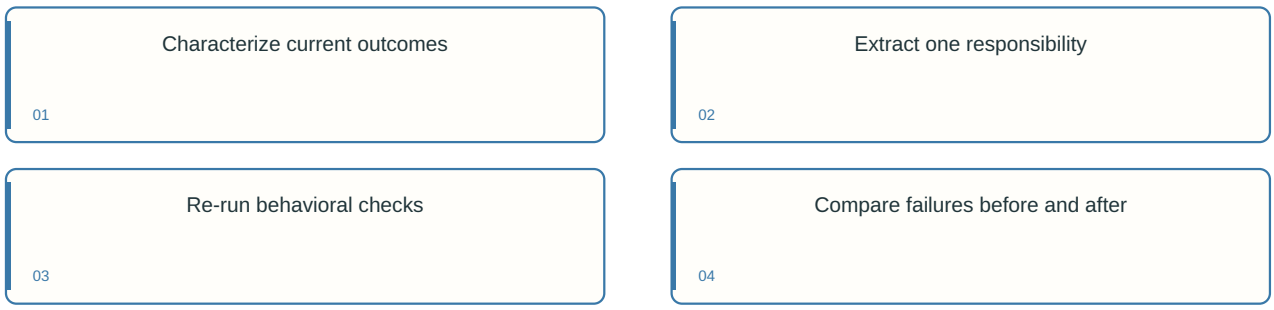


FIG. 5.4.1 **Refactoring and recognizing code smells · the example**

Refactoring improves structure while preserving the intended observable behaviour.

Author-created illustration. Numbers are illustrative unless stated otherwise.

5.4.2 · Functional programming alternatives to object-centric design

We can express payment decisions using functions as well as objects. A pure function decide(state, command) can return a new state, events, or requests for external actions. Another part of the application carries out the database write or provider call.

Because the decision function has explicit inputs and no hidden effects, it is easier to test and replay. The effect-performing part still needs durable progress, ownership, and duplicate handling. Calling a pure decision function twice is different from charging a provider twice. Choose functions and objects according to which makes the rules and collaboration easiest to see; they can be used together in the same design.

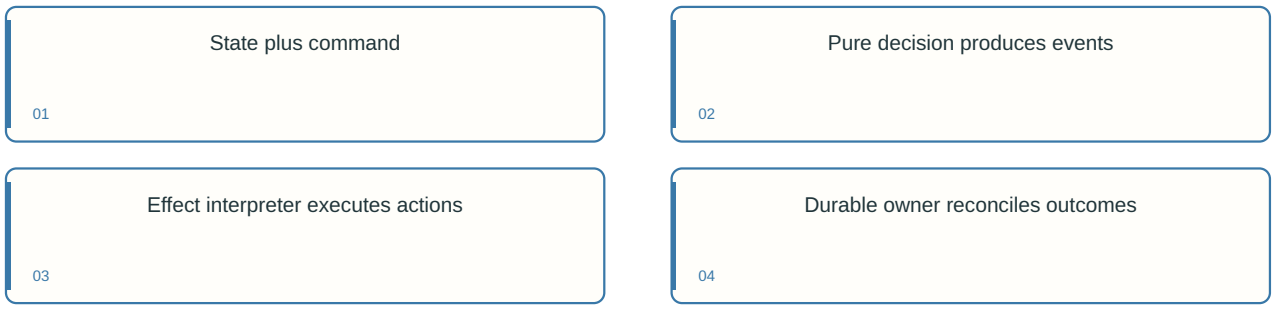


FIG. 5.4.2 **Functional programming alternatives to object-centric design · the example**

Pure functions can express decisions, while an explicit effect layer owns durable execution and external calls.

Author-created illustration. Numbers are illustrative unless stated otherwise.

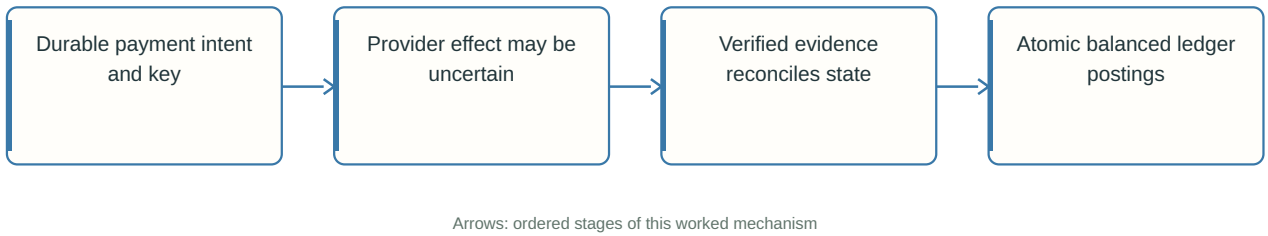


FIG. 5.4.3 **Follow the related complete design**

Persist intent, obtain or reconcile provider evidence, and commit the required local result.

Author-created illustration. Numbers are illustrative unless stated otherwise.

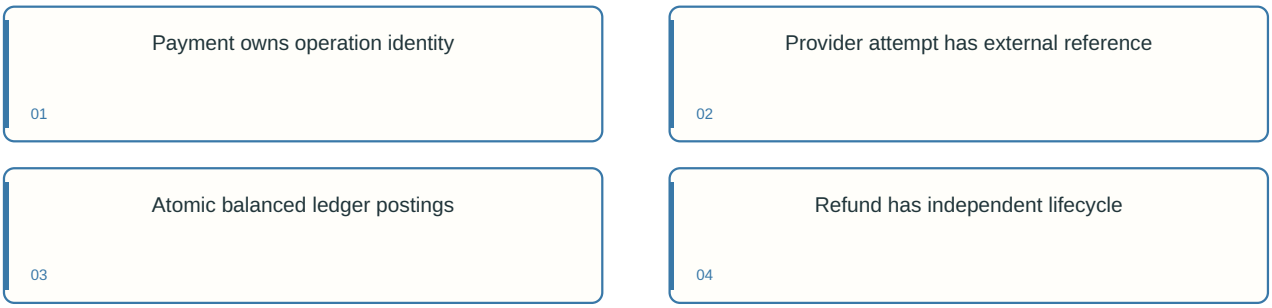


FIG. 5.4.4 **State and ownership in the related design**

Operation identity links provider evidence to payment state and balanced accounting entries.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Try the mechanism in code

PYTHON · INJECTED COLLABORATOR

```
class RecordingGateway:
    def __init__(self): self.calls = []
    def charge(self, command):
        self.calls.append(command)
        return "uncertain"

def pay(command, gateway):
    return gateway.charge(command)

gateway = RecordingGateway()
assert pay({"amount_minor":100,"currency":"USD"}, gateway) == "uncertain"
```

The recording gateway receives the command and returns uncertainty. Passing it as an argument makes this behaviour easy to exercise without a real provider.

A complete payment service also checks money, persists progress, and reconciles outcomes. This small example isolates the dependency; it does not implement those other obligations.

Download the executable mechanism collection ↓ · Follow the connected complete design →

▼ Think further · Which payment rules should remain the same when objects are replaced by pure functions?

Currency, units, allowed transitions, operation identity, and uncertainty remain business rules regardless of representation. Pure decision functions can make them easier to test. A separate effect interpreter still needs authorisation, transactions, provider identity, and reconciliation. Refactoring should preserve observed outcomes, not merely move methods into functions.

Remember: Improve structure while preserving the required outcomes and making external effects visible.

Try applying it: What regression test would catch a timeout accidentally being reclassified as a decline?

Unit 5 exercises → · Unit 5 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. On the criteria to be used in decomposing systems into modules — D. L. Parnas, 1972 ↗

   Original research · ACM DOI. Module decomposition criteria; metadata and abstract verified.

   Access reviewed · 21 October 2026

2. The Catalog of Design Patterns — Refactoring Guru ↗

   Engineering teaching reference. Pattern taxonomy; miniature examples and judgments are original to this atlas.

   Access reviewed · 21 October 2026

PART 2 / UNIT 06

Design Patterns, Dependency Injection, and Object Lifecycles

A notification service constructs its own provider and is difficult to test. Move that choice to startup, then distinguish supplying a dependency from choosing the interface policy relies on.

The running example

Create a notification platform supporting email, SMS, and push notifications. Evolve it from hardcoded classes to DI, Strategy, Factory, and Decorator patterns.

Unit 6 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

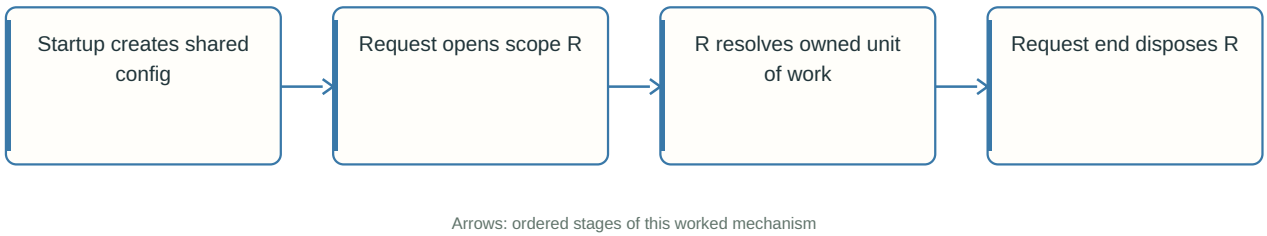


FIG. 6.0.1 **Object creation timelines**

Construction, borrowing, and disposal form a timeline; background jobs need independent scopes.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Resolution	Request scope 1	Request scope 2
Scoped: first / second	A / A	B / B
Transient: first / second	C / D	E / F
Singleton in this process	S / S	S / S

FIG. 6.0.2 **Scope identities make lifetime semantics visible**

These are identity observations in one application. Framework registration and disposal rules must still be checked.

Author-created illustration. Numbers are illustrative unless stated otherwise.

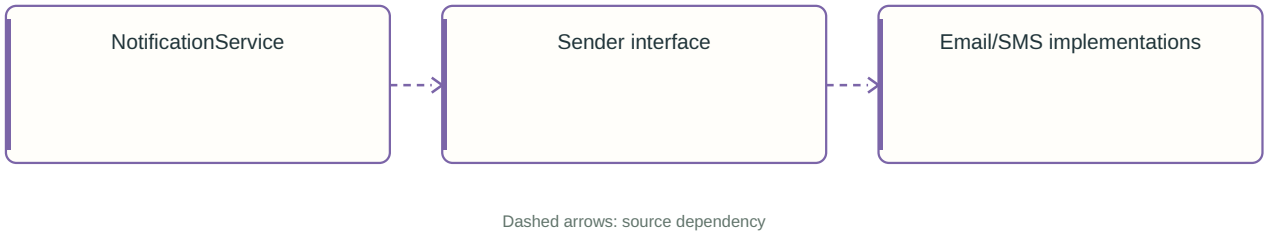


FIG. 6.0.3 **UML class relationships**

Roles are source-level class/interface relationships, not separate deployed services.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Question	Before inversion	After inversion
High-level dependency	Concrete provider SDK	Application-owned contract
Creation	Inside service	Composition root
Testing	Real effect coupled	Controlled collaborator

FIG. 6.0.4 **dependency inversion before/after**

Injection and inversion are different: supplying a concrete SDK alone may retain its semantic coupling.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Compare	Principal intent	Extra responsibility
Adapter / Decorator	Translate / add behavior	Preserve outcome meaning
Factory / DI	Create / supply	Define returned lifetime
Strategy / State	Algorithm / lifecycle	Validate allowed state change

FIG. 6.0.5 **pattern comparisons**

Pattern intent and contract matter more than all three being implemented as wrappers or interfaces.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Dependencies, inversion & composition

Prerequisites: Object-Oriented and Modular Software Design 6 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A notification service constructs its own provider and is difficult to test. Move that choice to startup, then distinguish supplying a dependency from choosing the interface policy relies on.

6.1.1 · Dependency, dependency direction, and inversion

A notification service that imports an email provider's SDK must understand that provider's types and errors. A provider change can then reach the application's notification rules. The source dependency points from the service to the SDK.

Dependency inversion changes which interface the service relies on. Define a Sender contract in the application, then make the provider adapter implement it. Runtime calls still execute that adapter, but the notification policy no longer needs the SDK's details. Check the boundary's data and errors as well as its imports. Moving a constructor into a helper does little if provider-specific meanings still pass through every caller.

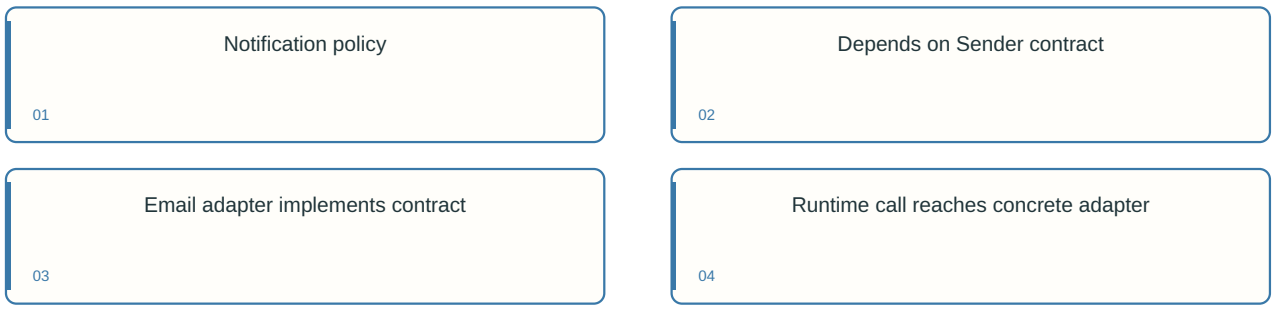


FIG. 6.1.1 **Dependency, dependency direction, and inversion · the example**

Dependency inversion lets important policy rely on suitable abstractions rather than changeable implementation details.

Author-created illustration. Numbers are illustrative unless stated otherwise.

6.1.2 · Inversion of Control (IoC)

In an ordinary function call, our code decides when to invoke another function. With an event loop, we register a callback and the loop decides when to call it. Some control has moved to an external coordinator. This is Inversion of Control, or IoC.

Frameworks also invoke request handlers or arrange their construction. Dependency Injection is one technique that can be used within these arrangements; IoC is the broader idea. Understand the framework's callback lifetime, ordering, and error behaviour. Delegating control changes where these decisions are made, while the application still needs to choose and use them correctly.

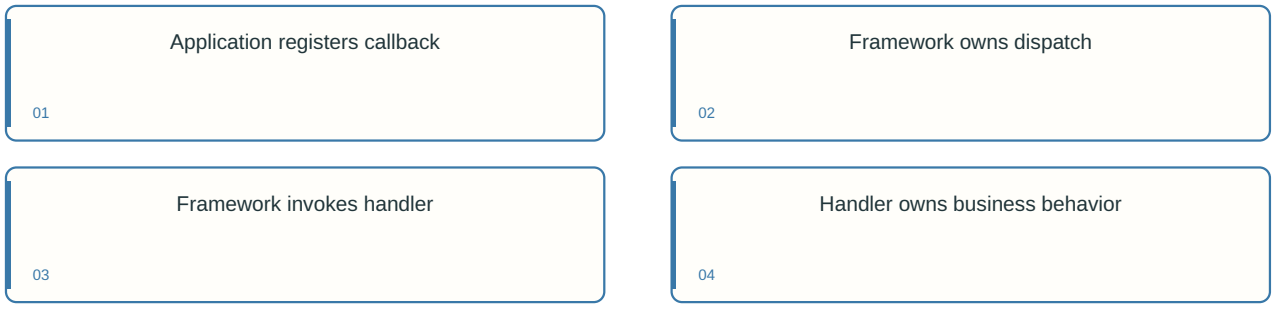


FIG. 6.1.2 **Inversion of Control (IoC) · the example**

Inversion of Control delegates some calling or construction decisions to an external coordinator or framework.

Author-created illustration. Numbers are illustrative unless stated otherwise.

6.1.3 · Dependency Injection (DI)

Compare a notification service that constructs EmailSender internally with one created as NotificationService(sender). In the second version, another part of the program supplies the collaborator. This is Dependency Injection, or DI.

The supplied sender might be an email adapter, an SMS adapter, or a test sender that records messages. Ordinary constructor or function arguments are sufficient; a container is optional. Keep the dependency focused. Passing one enormous infrastructure object can still expose far more knowledge than the service needs. The value of injection comes from visible collaborators and a meaningful contract, rather than the presence of a particular framework.

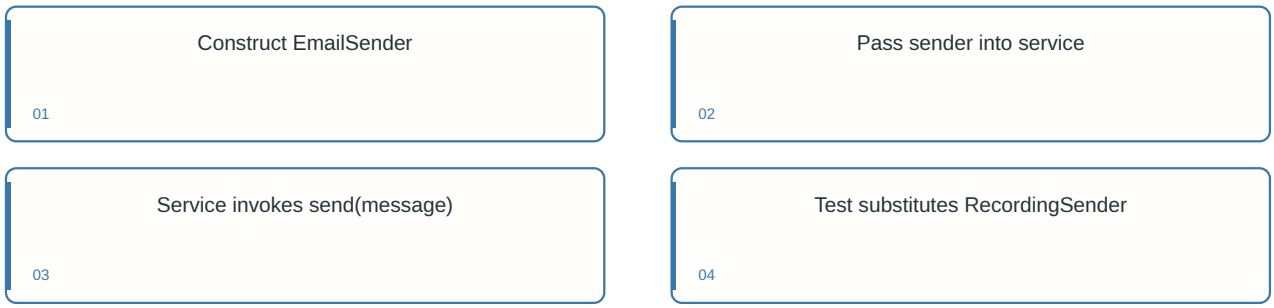


FIG. 6.1.3 **Dependency Injection (DI) · the example**

Dependency Injection supplies collaborators through construction or calls instead of hiding their creation or lookup.

Author-created illustration. Numbers are illustrative unless stated otherwise.

6.1.4 · Constructor, property/setter, and method injection

If an object cannot work without a sender, constructor injection supplies it before the object is used. Setter injection supplies a dependency later, which may suit optional configuration but introduces a period when the object is not fully ready.

Method injection passes a collaborator for one operation. This can express a per-call dependency without storing it for the object's whole lifetime. Choose by validity and ownership. For example, a long-lived service should not retain a request's database session after that request ends. Different frameworks provide different facilities, but the design question remains: when is the dependency required, and how long may the receiver keep it?

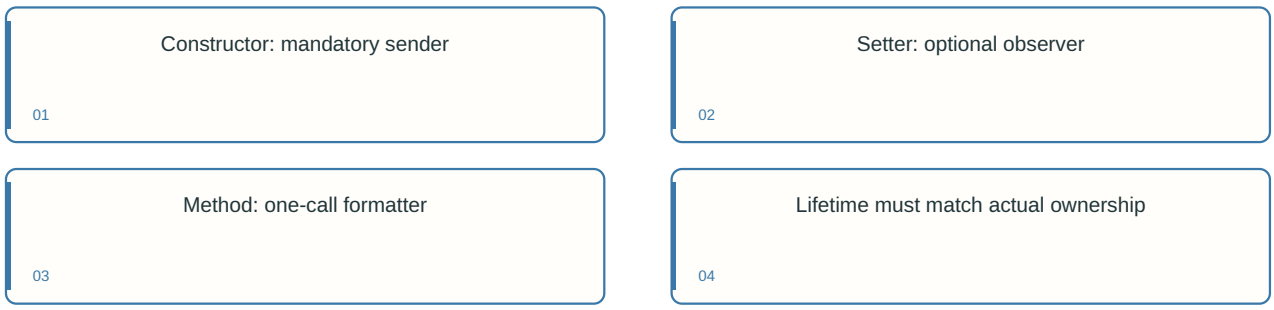


FIG. 6.1.4 **Constructor, property/setter, and method injection · the example**

Constructor, setter, and method injection provide dependencies at different points with different validity and lifetime implications.

Author-created illustration. Numbers are illustrative unless stated otherwise.

6.1.5 · Dependency inversion principle vs DI

Suppose Service(provider_sdk) receives its SDK through a constructor. This uses DI, but the service may still depend on provider-specific request types and exceptions. Injection alone has not isolated its policy from the provider.

The dependency inversion principle concerns the direction of knowledge: important policy relies on suitable abstractions, while implementations conform to them. DI concerns how collaborators are supplied. The two often work together but answer different questions. Inspect the actual interface, outcomes, and resource handles crossing the boundary. A manually assembled service can respect dependency inversion without a container; constructor syntax alone cannot establish that its contract is well chosen.

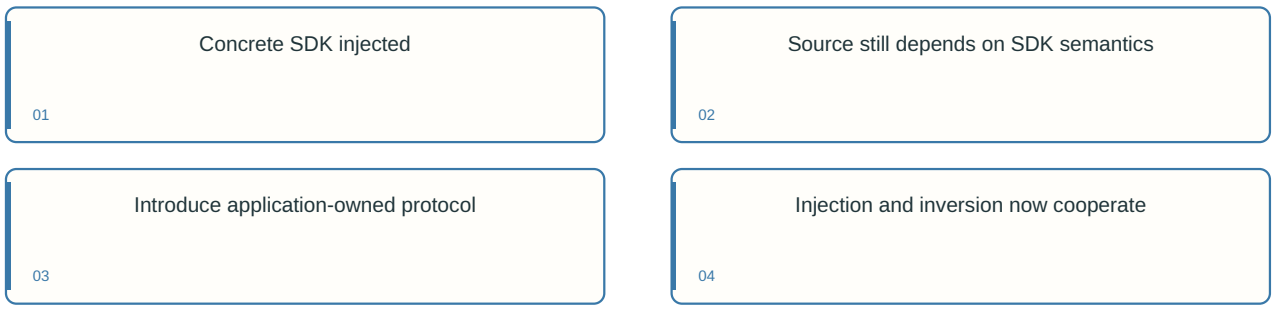


FIG. 6.1.5 **Dependency inversion principle vs DI · the example**

Dependency inversion concerns knowledge direction; DI concerns supplying collaborators. Either term alone does not establish the other.

Author-created illustration. Numbers are illustrative unless stated otherwise.

6.1.6 · Composition roots and DI containers

At startup, we can write repository = SQLiteRepository(path), then service = NotificationService(repository, sender). Keeping these construction choices together makes the application's object relationships visible. This part of the program is the composition root.

A DI container automates construction and manages registered lifetimes. It can simplify a large graph, but also hide which dependency is resolved and postpone some mistakes until runtime. Validate registrations early and make resource ownership clear. For a few collaborators, explicit construction may be easier to read. Use a container where its actual wiring and lifetime support justify the additional machinery.

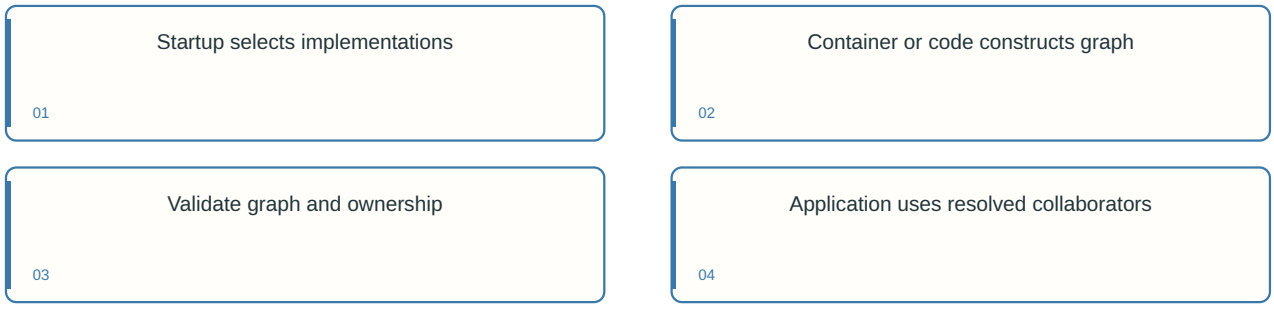


FIG. 6.1.6 **Composition roots and DI containers · the example**

A composition root assembles collaborators; a DI container automates selected construction and lifetime rules.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · How should resources be owned when a job outlives its request and user session?

Give the job an explicit lifetime and scope, with durable identifiers rather than captured request objects. Create or borrow resources under a documented owner and close them at the appropriate boundary. Define whether current membership must be rechecked; a long-lived job must not silently inherit expired user authority.

Remember: Injection supplies collaborators; inversion determines which abstractions and meanings the policy depends on.

Try applying it: Can a constructor receive an SDK while still exposing provider details throughout the service?

Unit 6 exercises → · Unit 6 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Inversion of Control Containers and the Dependency Injection pattern — Martin Fowler, 2004 ↗

   Original engineering article. Dependency injection and wiring.

   Access reviewed · 21 October 2026

2. The Catalog of Design Patterns — Refactoring Guru ↗

   Engineering teaching reference. Pattern taxonomy; miniature examples and judgments are original to this atlas.

   Access reviewed · 21 October 2026

Lifetimes, ownership & cleanup

Prerequisites: 6.1 · Dependencies, inversion & composition 6 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A background job uses a connection after its request scope has closed. Track creation, reuse, ownership, and disposal across the lifetimes involved.

6.2.1 · Transient, scoped, singleton, and framework-specific lifetimes

Resolve a scoped object twice in request one: both references should point to A. Resolve it twice in request two: both should point to B. That is reuse within a scope. A transient resolution creates a new instance, while a singleton is reused within the framework's stated container or application boundary.

Those words describe lifetimes, not universal global identities. Another process can have its own singleton. A shared singleton must also be safe for its allowed concurrent callers. Check the framework's exact naming, resolution, and disposal rules. A lifetime label does not establish thread safety or mean that the object can outlive its dependencies.

Resolution	Request scope 1	Request scope 2
Scoped: first / second	A / A	B / B
Transient: first / second	C / D	E / F
Singleton in this process	S / S	S / S

FIG. 6.2.1 **Scope identities make lifetime semantics visible**

These are identity observations in one application. Framework registration and disposal rules must still be checked.

Author-created illustration. Numbers are illustrative unless stated otherwise.

6.2.2 · Request scopes, session scopes, and custom scopes

An HTTP request finishes and its scoped database session is closed. A background job that kept that session now tries to use it. The job has outlived a resource it borrowed.

A request scope covers one request. A session scope can span several requests, while a job can have its own scope with an explicit end. Create an appropriate scope for background work and pass durable identifiers rather than retaining request-owned objects. Long-lived scopes also retain data and may create privacy or memory costs. A scope is an ownership arrangement; it is not automatically a database transaction or a guarantee about the user's current permissions.

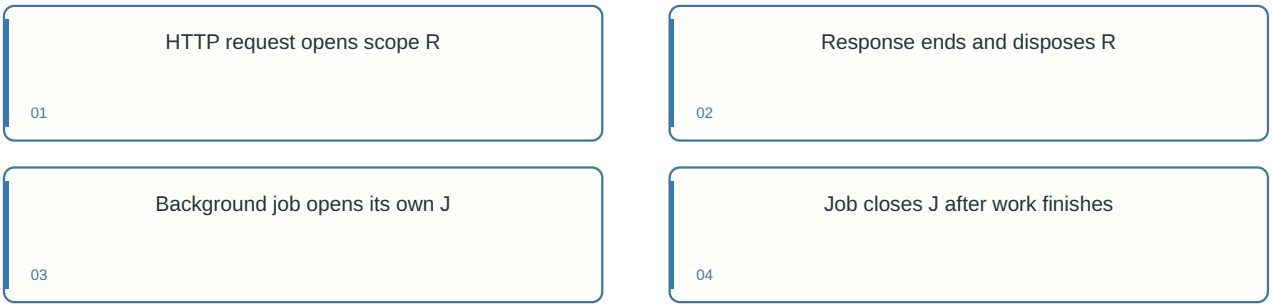


FIG. 6.2.2 **Request scopes, session scopes, and custom scopes · the example**

Scopes define resource lifetimes for requests, sessions, or jobs; their meanings need explicit ownership and cleanup.

Author-created illustration. Numbers are illustrative unless stated otherwise.

6.2.3 · Object creation vs object ownership

A factory opens a connection and returns it. Who must close it? The caller might own it, or the factory might be returning a shared resource managed elsewhere. Holding a reference does not answer the ownership question.

Specify ownership in the contract. Two components must not both assume they may close the same shared connection. Consider a repository that returns a lazy query result: reading it may still need the repository's connection. Closing the repository before consuming the result breaks that lifetime. Return bounded materialised data when appropriate, or return an explicit streaming resource whose owner is responsible for closing it.

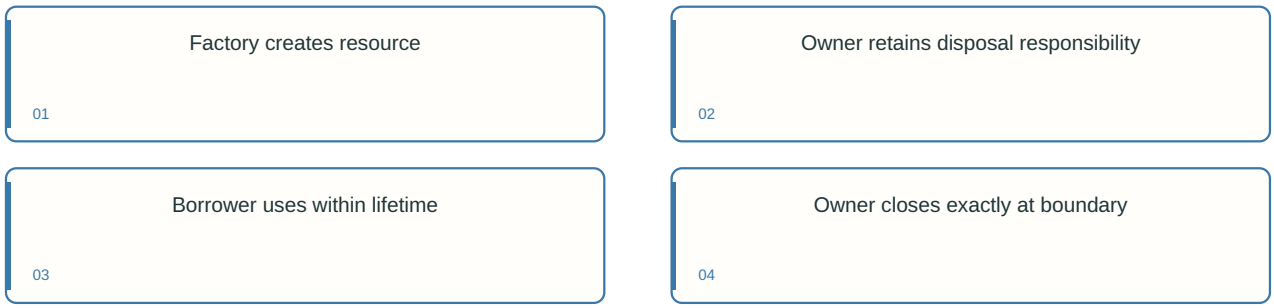


FIG. 6.2.3 **Object creation vs object ownership · the example**

Creating or holding a resource does not determine who owns its disposal.

Author-created illustration. Numbers are illustrative unless stated otherwise.

6.2.4 · Disposal, cleanup, and resource lifecycles

A request borrows a connection, then fails while processing. Unless cleanup returns the connection, later healthy requests have less pool capacity. A small leak repeated over many failures can stop the application.

Arrange cleanup for success, failure, and cancellation using structured scopes or finally-style code. Close streams, release permits, and roll back unfinished transactions. Memory garbage collection does not guarantee timely release of these external resources. If cleanup also fails, preserve the original problem and record the cleanup failure. Resource management is part of system capacity and correctness, rather than work that can be postponed until the process exits.

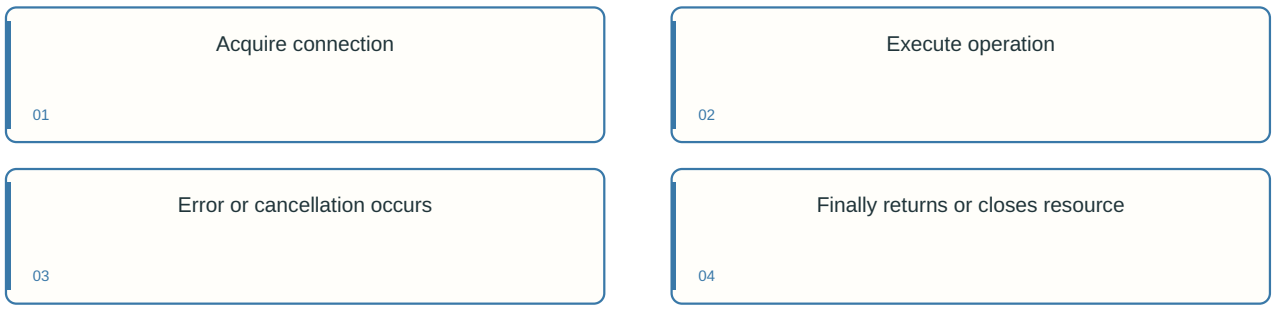


FIG. 6.2.4 **Disposal, cleanup, and resource lifecycles · the example**

Cleanup must release owned resources on success, failure, and cancellation under the relevant contract.

Author-created illustration. Numbers are illustrative unless stated otherwise.

6.2.5 · Lazy initialization

A report generator may never be used during a particular run. Lazy initialisation waits until the first report request before constructing it, saving unused startup work. That first request now pays the construction cost and may encounter its failure.

Two first callers can also arrive together. Coordinate initialisation and safe publication using the runtime's supported mechanism. Decide whether a failed initialisation is retried or remembered. For essential resources, starting them before advertising readiness may provide clearer behaviour than failing the first real user request. Lazy creation changes when work happens; it still needs lifetime, error, and concurrency rules.

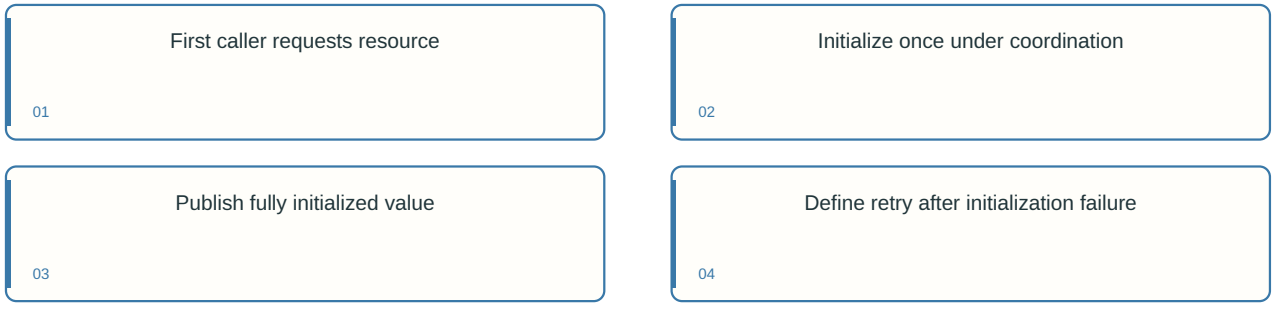


FIG. 6.2.5 **Lazy initialization · the example**

Lazy initialisation defers work to first use, requiring concurrency, publication, failure, and latency decisions.

Author-created illustration. Numbers are illustrative unless stated otherwise.

6.2.6 · Dependency cycles and captive dependencies

Suppose A needs B to be constructed, while B needs A. This dependency cycle may reveal mixed responsibilities or a missing coordinator. Hiding a lookup in a global registry can postpone the problem without clarifying ownership.

Another lifetime error is a captive dependency: a singleton stores a request-scoped unit of work and keeps using it after the request ends. Refactor the operation to receive a currently valid dependency or a narrow factory that creates an appropriate scope. Preserve the correct user and tenant context as well. Passing a container validation check is useful, but the real requirement is that every use occurs within the dependency's valid lifetime.

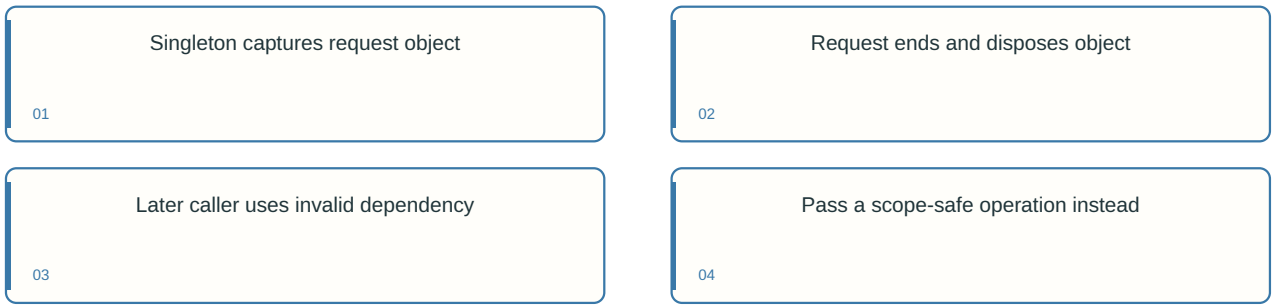


FIG. 6.2.6 **Dependency cycles and captive dependencies · the example**

Cycles can signal unclear responsibilities; captive dependencies retain collaborators beyond their valid shorter lifetime.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · How should resources be owned when a job outlives its request and user session?

Give the job an explicit lifetime and scope, with durable identifiers rather than captured request objects. Create or borrow resources under a documented owner and close them at the appropriate boundary. Define whether current membership must be rechecked; a long-lived job must not silently inherit expired user authority.

Remember: A reference remains valid only under its resource's ownership and lifetime contract.

Try applying it: How should a job obtain resources if it must outlive the request that created it?

Unit 6 exercises → · Unit 6 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Inversion of Control Containers and the Dependency Injection pattern — Martin Fowler, 2004 ↗

   Original engineering article. Dependency injection and wiring.

   Access reviewed · 21 October 2026

2. The Catalog of Design Patterns — Refactoring Guru ↗

   Engineering teaching reference. Pattern taxonomy; miniature examples and judgments are original to this atlas.

   Access reviewed · 21 October 2026

Injection, locators & factories

Prerequisites: 6.2 · Lifetimes, ownership & cleanup 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A service receives one container and resolves everything from it. Compare those hidden lookups with explicit collaborators and a narrow factory for legitimate runtime choices.

6.3.1 · DI vs service locator

A constructor that lists Sender and Repository tells us what the service needs. A method that silently calls global_registry.resolve(...) reveals that need only during execution. The latter is a service locator.

Locators can have a deliberate role at an integration boundary, but using them throughout application code hides dependencies and makes isolated tests harder. Injecting the entire container into a service can preserve the same hidden lookups under a different name. Resolve collaborators near startup or the relevant scope boundary, then pass the specific contracts needed. This makes missing dependencies and ownership easier to review.

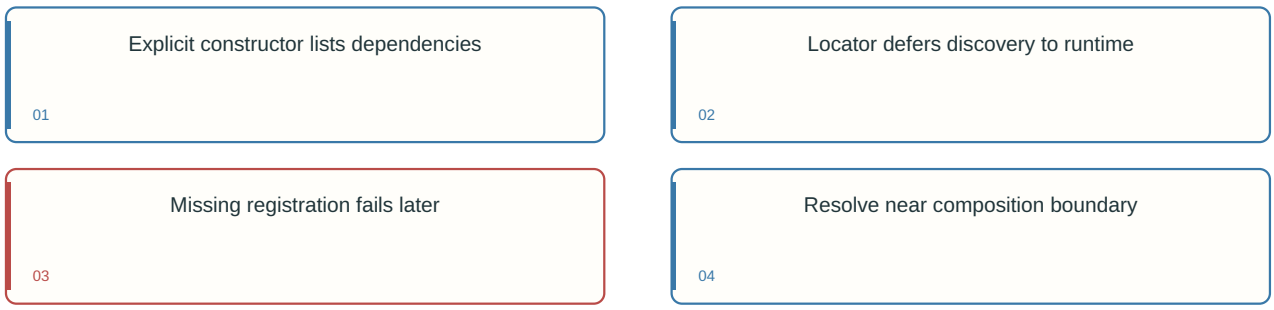


FIG. 6.3.1 **DI vs service locator · the example**

A service locator hides dependencies behind runtime lookup; DI can make them explicit at the appropriate boundary.

Author-created illustration. Numbers are illustrative unless stated otherwise.

6.3.2 · DI vs factory pattern

A service may receive one sender chosen at startup. Alternatively, it may receive a sender factory because each notification chooses its channel at runtime. Injection supplies the collaborator; the factory performs a construction or selection operation when asked.

The two techniques can work together. A factory also needs a lifetime contract: does it return a fresh caller-owned sender, a shared singleton, or a scoped object? A caller must not close a shared sender as if it had created a private one. Keep provider-specific construction inside the factory while making ownership and possible failure clear to its callers.

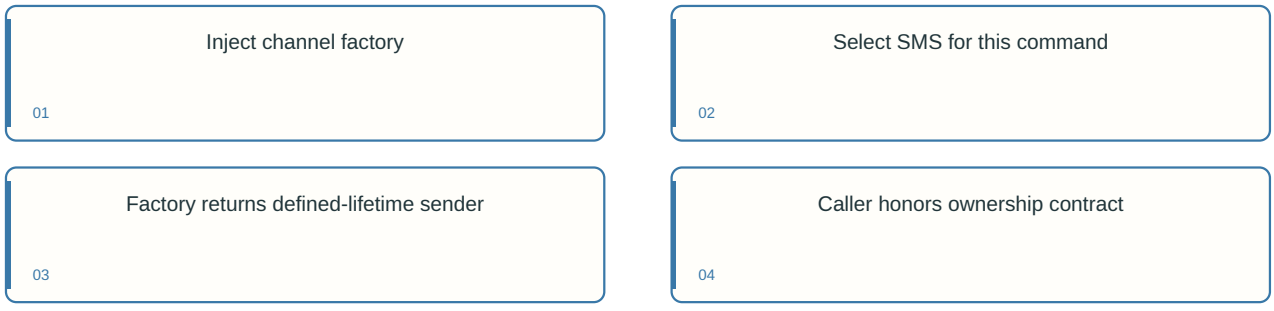


FIG. 6.3.2 **DI vs factory pattern · the example**

A factory chooses or creates a collaborator; injection supplies it or the factory, with ownership still specified.

Author-created illustration. Numbers are illustrative unless stated otherwise.

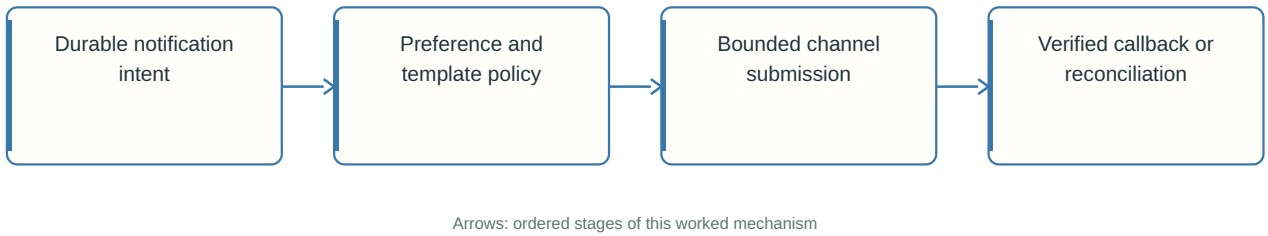


FIG. 6.3.3 **Follow the related complete design**

Durable intent reaches a bounded channel worker and a provider with explicit outcome meanings.

Author-created illustration. Numbers are illustrative unless stated otherwise.

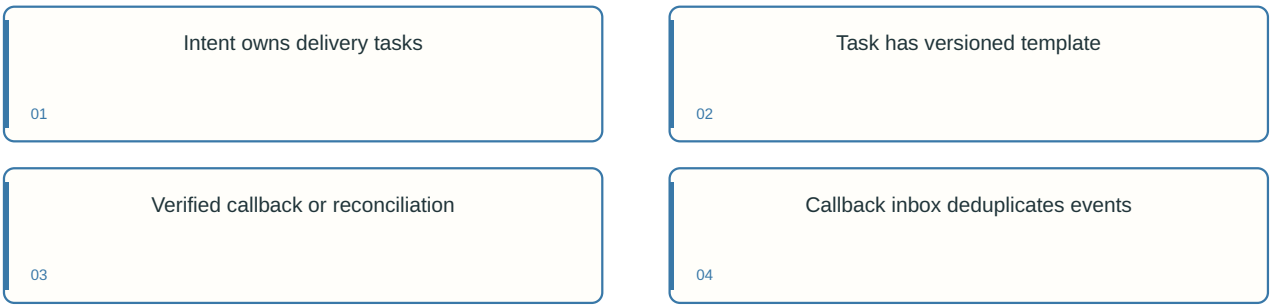


FIG. 6.3.4 **State and ownership in the related design**

Intent, delivery tasks, attempts, and callbacks record different stages of the notification.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · How should resources be owned when a job outlives its request and user session?

Give the job an explicit lifetime and scope, with durable identifiers rather than captured request objects. Create or borrow resources under a documented owner and close them at the appropriate boundary. Define whether current membership must be rechecked; a long-lived job must not silently inherit expired user authority.

Remember: Use factories for creation decisions and explicit dependencies for work the caller already knows it needs.

Try applying it: What must a sender factory say about whether its returned object may be closed by the caller?

Unit 6 exercises → · Unit 6 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Inversion of Control Containers and the Dependency Injection pattern — Martin Fowler, 2004 ↗

   Original engineering article. Dependency injection and wiring.

   `Access reviewed · 21 October 2026`

2. The Catalog of Design Patterns — Refactoring Guru ↗

   Engineering teaching reference. Pattern taxonomy; miniature examples and judgments are original to this atlas.

   `Access reviewed · 21 October 2026`

Creational patterns

Prerequisites: 6.3 · Injection, locators & factories 6 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Creating a notification can require choosing a writer, a compatible family, or a valid combination of fields. These are different construction problems with different useful patterns.

6.4.1 · Factory Method, Abstract Factory, Builder, Prototype, Singleton

Construction becomes a design problem when there are several valid ways to create something. Factory Method varies a creation operation. Abstract Factory produces a compatible family of objects. Builder assembles and validates a complex value in stages. Prototype copies a configured object. Singleton restricts access to an instance within a specified boundary.

The individual examples below show what problem each solves. A direct constructor remains useful when it already expresses all required inputs. Extra construction machinery needs a reason: invalid partially built values, accidental sharing in copies, and global mutable state can introduce problems of their own. Choose the pattern by the variation that the application actually needs.

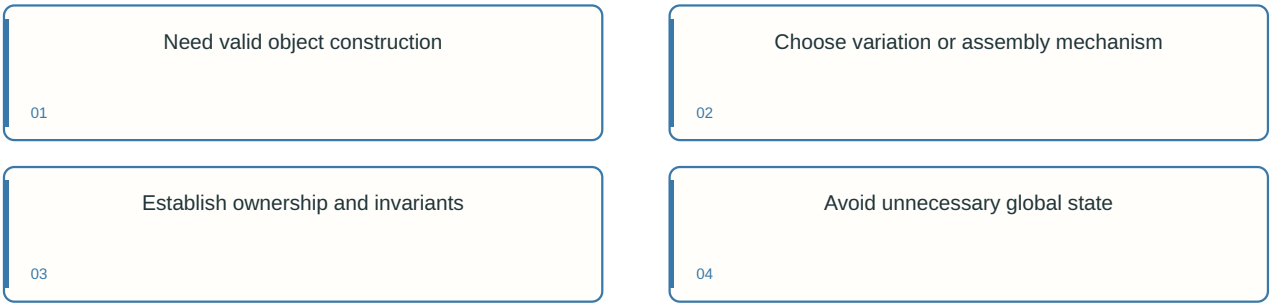


FIG. 6.4.1 **Factory Method, Abstract Factory, Builder, Prototype, Singleton · the example**

Creational patterns organise different construction choices, families, staged assembly, copying, and bounded instance access.

Author-created illustration. Numbers are illustrative unless stated otherwise.

PATTERN / MECHANISM, USE, AND COST

Factory Method

A report exporter needs a writer, but different exporters may choose different formats. Its processing code can call make_writer(), then use the returned writer without choosing the concrete type itself. Factory Method makes that creation step a variation point.

Use it when the creator has a meaningful construction choice to customise. A supplied function may be sufficient; inheritance is not always needed. Abstract Factory deals with a different scale of choice: a compatible family of products rather than one creation operation.

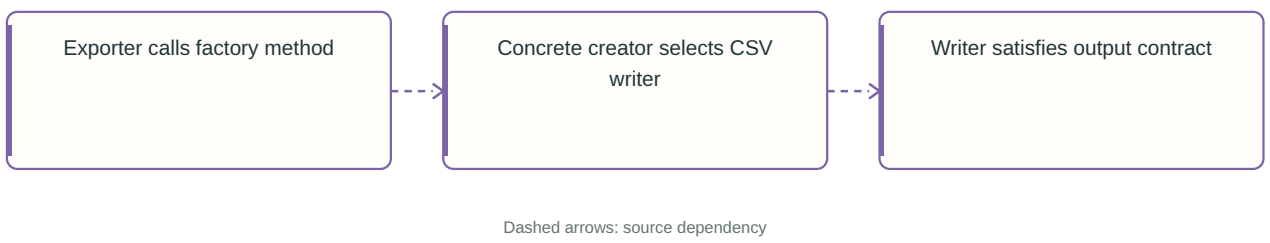


FIG. 6.4.2 **Factory Method · the example**

Varies a creation operation so processing code can use a product without selecting its concrete implementation.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
export(rows):
    writer = make_writer()
    writer.write(rows)
```

PATTERN / MECHANISM, USE, AND COST

Abstract Factory

An HTML notification needs a renderer and a compatible template. Selecting them independently could mix components that expect different content models. Abstract Factory creates the related objects as one family.

The client asks the selected factory for each collaborator, keeping the family choice together. This helps when the application switches complete families. Adding a new kind of product can require changes to every factory, so there is a maintenance cost. If one stable set of objects is all that is needed, direct construction may be clearer.

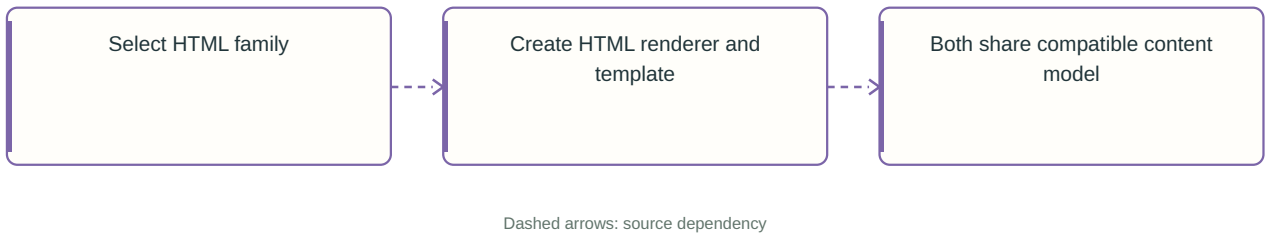


FIG. 6.4.3 **Abstract Factory · the example**

Creates a compatible family of collaborators through one selected construction interface.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
family = HtmlNotificationFactory()
renderer = family.renderer()
template = family.template()
renderer.render(template, data)
```

PATTERN / MECHANISM, USE, AND COST

Builder

A notification requires a recipient and template, with optional locale and expiry. A builder collects these choices and checks them before creating the usable notification.

The final build step should reject missing or incompatible fields. Until then, the partially assembled value should not be treated as ready to send. Builder is useful when staged construction improves clarity. For a small value, a constructor with named parameters may already express everything safely. Its purpose is controlled assembly, rather than creating a family of different products.

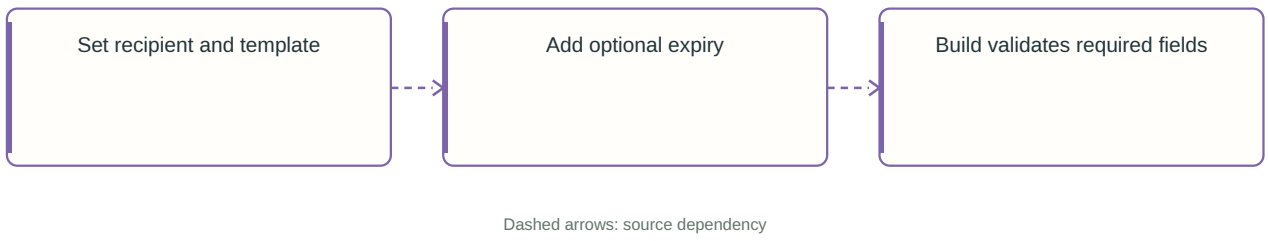


FIG. 6.4.4 **Builder · the example**

Collects construction choices and validates them before producing a usable value.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
builder.to(recipient).template(template_id)
notification = builder.expires_at(deadline).build()
# build rejects missing recipient or incompatible fields
```

PATTERN / MECHANISM, USE, AND COST

Prototype

A report layout contains a carefully configured set of values. Prototype creates another layout by copying that configured instance rather than rebuilding every setting.

Specify what the copy owns. A shallow copy can leave both layouts referring to the same mutable object; a deeper copy may duplicate more state. Copying a connection handle does not create an independently owned connection. Shared immutable font data can remain shared, while editable layout values need suitable ownership. Use copying when those rules are clearer than constructing fresh request state from immutable configuration.

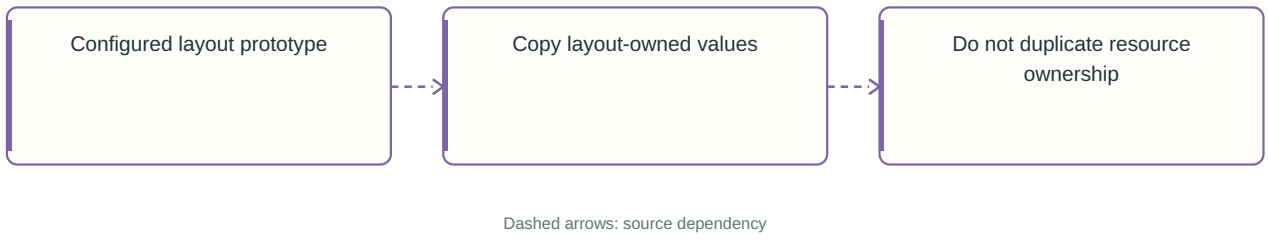


FIG. 6.4.5 **Prototype · the example**

Creates an object by copying configured state under explicit sharing and ownership rules.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
layout = prototype.clone_owned_values()
layout.title = "Monthly report"
# Shared immutable font data remains shared
```

PATTERN / MECHANISM, USE, AND COST

Singleton

An application may load one immutable configuration object and share it with its services. A Singleton arrangement restricts access to an instance within a stated program or container boundary. Another process may still have its own instance.

Keep that boundary and ownership explicit. Shared mutable request state can cause races and test interference. Container-managed singleton lifetime and a class offering hidden global access are related but different arrangements. Passing an application-owned instance to its users often makes dependencies easier to see and tests easier to isolate.

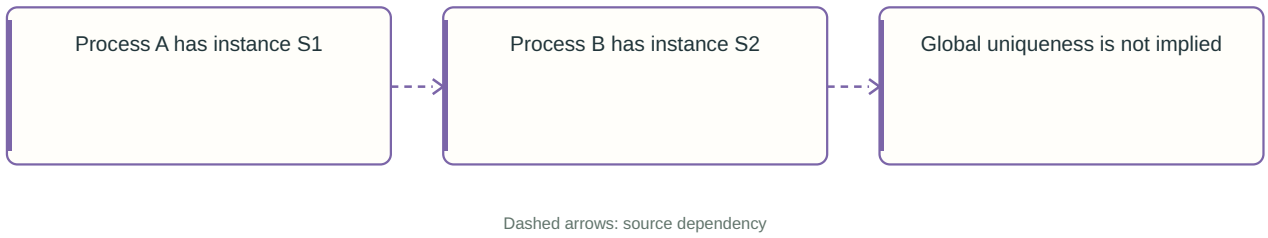


FIG. 6.4.6 **Singleton · the example**

Restricts instance access within a specified program or container boundary, without implying global uniqueness or safe mutation.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
configuration = load_immutable_configuration()
service = Service(configuration)
# One application-owned instance; no hidden global lookup
```

▼ Think further · How should resources be owned when a job outlives its request and user session?

Give the job an explicit lifetime and scope, with durable identifiers rather than captured request objects. Create or borrow resources under a documented owner and close them at the appropriate boundary. Define whether current membership must be rechecked; a long-lived job must not silently inherit expired user authority.

Remember: Choose construction machinery for a real variation, and define validity and ownership of the result.

Try applying it: When would named constructor parameters be clearer than a builder?

Unit 6 exercises → · Unit 6 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Inversion of Control Containers and the Dependency Injection pattern — Martin Fowler, 2004 ↗

   Original engineering article. Dependency injection and wiring.

   Access reviewed · 21 October 2026

2. The Catalog of Design Patterns — Refactoring Guru ↗

   Engineering teaching reference. Pattern taxonomy; miniature examples and judgments are original to this atlas.

   Access reviewed · 21 October 2026

Structural patterns

Prerequisites: 6.4 · Creational patterns 8 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A wrapper can translate an interface, add metrics, control access, or simplify a subsystem. Follow the intent and contract of each structural pattern to tell them apart.

6.5.1 · Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy

Several patterns place objects between collaborators, but their purposes differ. Adapter translates an interface. Bridge separates two dimensions that vary independently. Composite gives leaves and groups a common way to be used. Decorator adds behaviour around an existing interface.

Facade presents a focused entry to a subsystem. Flyweight shares suitable immutable data. Proxy controls access to a target. The examples below make these differences concrete. Preserve the intended contract when composing them: adding metrics can be straightforward, while adding retries to a payment sender changes the risk of repeated effects. A common diagram shape does not make these patterns interchangeable.

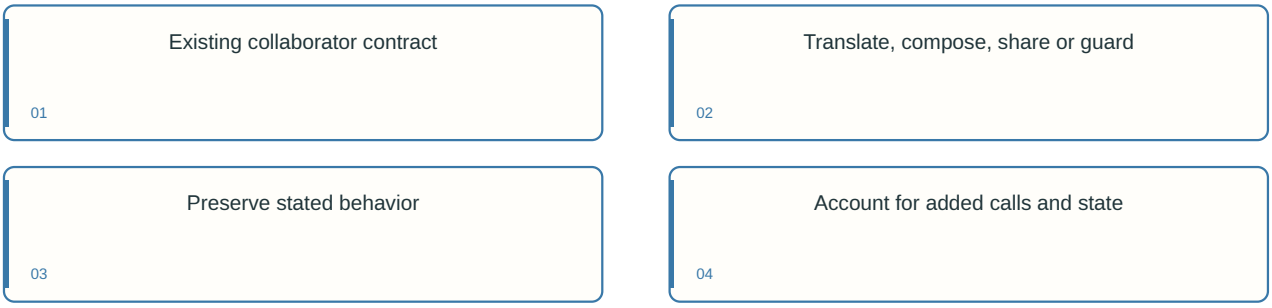


FIG. 6.5.1 **Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy · the example**

Structural patterns organise interface translation, composition, grouping, added behaviour, simplified access, sharing, and mediation to a target.

Author-created illustration. Numbers are illustrative unless stated otherwise.

PATTERN / MECHANISM, USE, AND COST

Adapter

An SMS SDK returns provider-specific status codes. The application's Sender contract expects accepted, declined, or uncertain with a reference. An Adapter translates between these interfaces.

The translation must preserve meaning. A provider timeout should not become a definite decline just because that is easier to encode. Keep SDK-specific types inside the adapter where practical, and test its mapping against the provider contract. Decorator usually adds behaviour around the same interface; Adapter's main job is to reconcile an interface or representation mismatch.

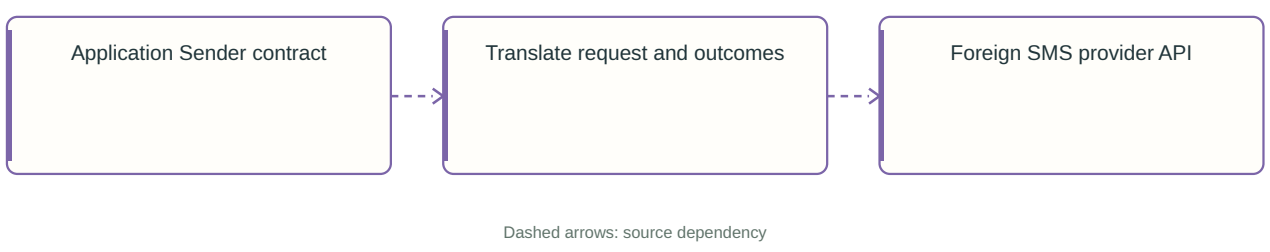


FIG. 6.5.2 **Adapter · the example**

Translates an existing interface or representation into a caller’s required behavioural contract.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
send(message):
    raw = sdk.submit(encode(message))
    return map_provider_outcome(raw)
```

PATTERN / MECHANISM, USE, AND COST

Bridge

Suppose notifications vary by content—alerts or receipts—and by transport—email or SMS. Subclassing every combination produces many similar classes. A Bridge lets each content object use a transport contract through composition.

The two dimensions can then vary independently. This needs a well-chosen shared contract and some wiring. Adapter usually connects an existing mismatch; Bridge deliberately separates variations in the design. It is useful when those variations are real, rather than when one fixed sender would serve the application more simply.

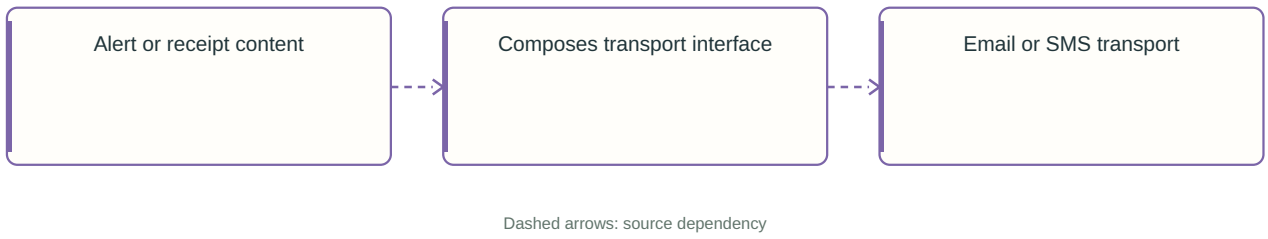


FIG. 6.5.3 **Bridge · the example**

Uses composition to let two dimensions of variation evolve independently.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
class Receipt:
    constructor(transport): self.transport = transport
    deliver(data): return self.transport.send(format_receipt(data))
```

PATTERN / MECHANISM, USE, AND COST

Composite

A notification audience can be one user or a group containing other audiences. Composite gives leaves and groups a compatible interface so a caller can traverse the audience structure.

Define duplicates, cycles, depth or size limits, and how failures are reported. A send to a group can partly succeed even if its method looks like a send to one user. The shared interface does not make the whole group atomic. Use a simple collection loop when the domain has no useful tree-shaped structure.

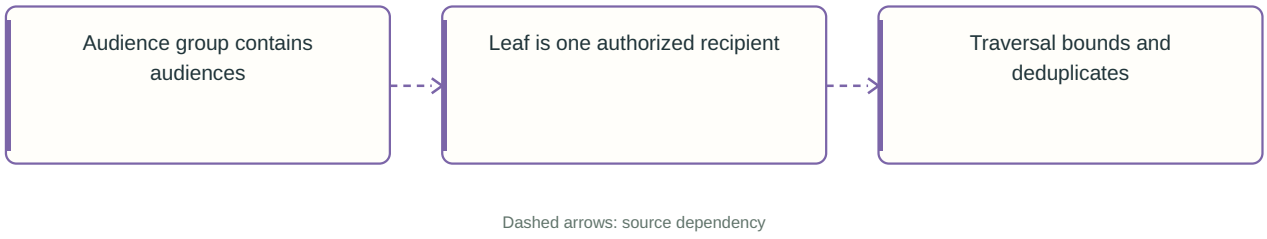


FIG. 6.5.4 **Composite · the example**

Lets callers use leaves and groups through a compatible interface, with explicit traversal and partial-failure rules.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
recipients(node, seen):
    if node.id in seen: return []
    seen.add(node.id)
    return node.users_or_recursive_children(seen)
```

PATTERN / MECHANISM, USE, AND COST

Decorator

A metrics wrapper records how long Sender.send takes, then returns its result. Decorator adds behaviour around a collaborator while retaining its interface.

Order can change meaning: placing metrics inside a retry wrapper counts attempts, while placing it outside can measure one logical call. Retrying also changes the risk of duplicate effects, so it needs a safe repetition contract. Use decorators where composition makes such behaviour clear. Avoid burying essential business decisions in a long wrapper stack that callers cannot easily inspect.

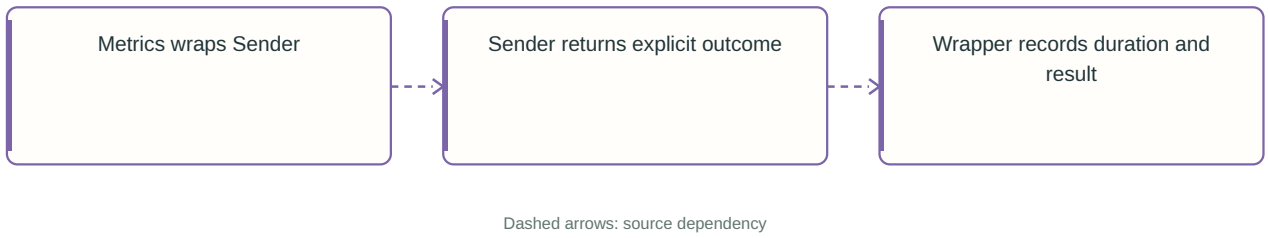


FIG. 6.5.5 **Decorator · the example**

Adds behaviour around a collaborator while retaining its interface, with composition order affecting meaning.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
send(message):
    start = clock()
    try: return inner.send(message)
    finally: metrics.observe(clock() - start)
```

PATTERN / MECHANISM, USE, AND COST

Facade

Submitting a notification may involve validation, template selection, and saving a delivery plan. A Facade offers one focused operation that coordinates this subsystem for its clients.

The caller no longer needs to know every internal step. The response must still distinguish accepted work from completed delivery. Keep the facade focused; accumulating every unrelated operation turns it into a large coordinator that is difficult to change. A Proxy commonly stands in for the same target interface, while a Facade primarily offers a simpler entry to several cooperating parts.

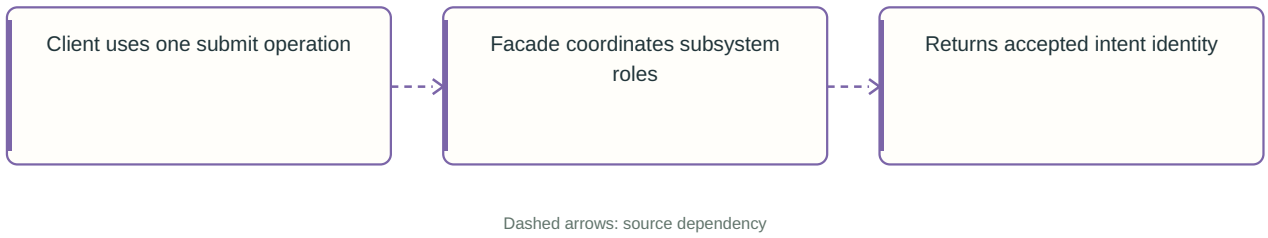


FIG. 6.5.6 **Facade · the example**

Offers a focused entry to cooperating subsystem operations without hiding consequential outcomes.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
submit(command):
    validated = validate(command)
    return intents.persist(render_plan(validated))
```

PATTERN / MECHANISM, USE, AND COST

Flyweight

Thousands of notifications may use one parsed template. Flyweight shares that reusable, immutable part while each rendering supplies its own recipient and variables.

Separating shared content from per-use context reduces repeated storage. Mixing recipient state into the shared template could leak data between users. Measure whether memory pressure justifies extra pooling or caching; ordinary immutable reuse may be enough. Sharing also leaves version selection, cache expiry, and resource lifetime to be designed. The pattern identifies what can be shared, rather than solving every cache-management problem.

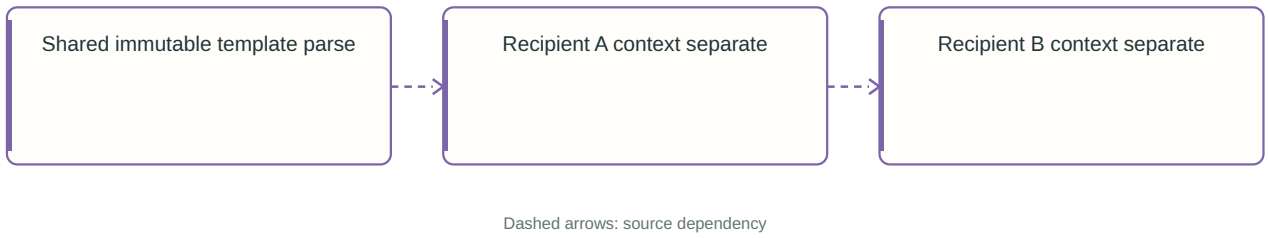


FIG. 6.5.7 **Flyweight · the example**

Shares suitable immutable intrinsic state while keeping per-use context separate.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
template = parsed_templates.get(version)
output_a = render(template, context_a)
output_b = render(template, context_b)
```

PATTERN / MECHANISM, USE, AND COST

Proxy

Before reading a document, a Proxy can check whether the actor currently has access, then forward the call to the target. It presents the target's interface while controlling access or another aspect of its use.

Other proxies defer creation or represent a remote target. A remote call can fail or take substantial time, so its contract must expose the relevant uncertainty. Avoid interfaces that look cheap while hiding unbounded network retries or loads. A Facade simplifies subsystem operations; a Proxy usually represents the same conceptual target with controlled access.

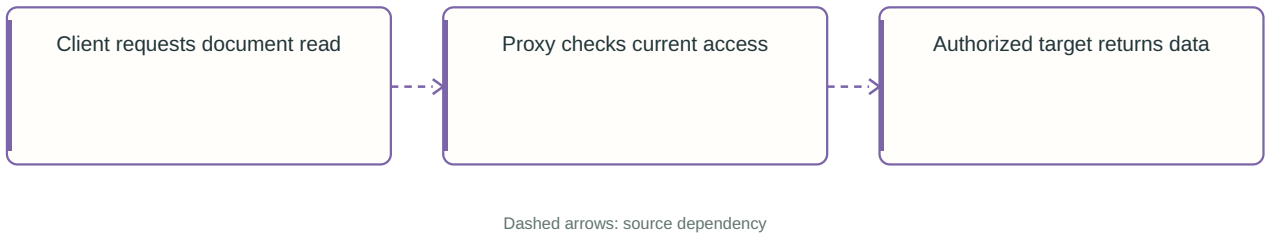


FIG. 6.5.8 **Proxy · the example**

Represents a target interface while controlling access, creation, or location with explicit cost and failure behaviour.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
read(document_id, actor):
    authorize(actor, document_id)
    return target.read(document_id)
```

▼ Think further · How should resources be owned when a job outlives its request and user session?

Give the job an explicit lifetime and scope, with durable identifiers rather than captured request objects. Create or borrow resources under a documented owner and close them at the appropriate boundary. Define whether current membership must be rechecked; a long-lived job must not silently inherit expired user authority.

Remember: Similar object diagrams can hide different purposes and different changes to behaviour.

Try applying it: Why does adding retries around a payment sender require more reasoning than adding timing metrics?

Unit 6 exercises → · Unit 6 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Inversion of Control Containers and the Dependency Injection pattern — Martin Fowler, 2004 ↗

   Original engineering article. Dependency injection and wiring.

   Access reviewed · 21 October 2026

2. The Catalog of Design Patterns — Refactoring Guru ↗

   Engineering teaching reference. Pattern taxonomy; miniature examples and judgments are original to this atlas.

   Access reviewed · 21 October 2026

Behavioral patterns

Prerequisites: 6.5 · Structural patterns 11 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A changing algorithm, a lifecycle transition, and a request pipeline each vary for different reasons. Use behavioural patterns to place the right decision where it can evolve clearly.

6.6.1 · Strategy, Observer, Command, State, Chain of Responsibility, Template Method, Iterator, Mediator, Memento, Visitor

Behavioural patterns make different kinds of decisions and coordination explicit. Strategy supplies an algorithm. Observer distributes notifications. Command represents a requested action. State organises behaviour by lifecycle. Chain of Responsibility chooses handlers, while Template Method fixes a process with selected extension points.

Iterator controls traversal; Mediator coordinates collaborators; Memento captures a recoverable local state; Visitor adds operations over a known object structure. Read their individual examples as choices about where change should be easy. A pattern is useful when its extension point matches the application. A small conditional can remain clearer when no such variation is needed.

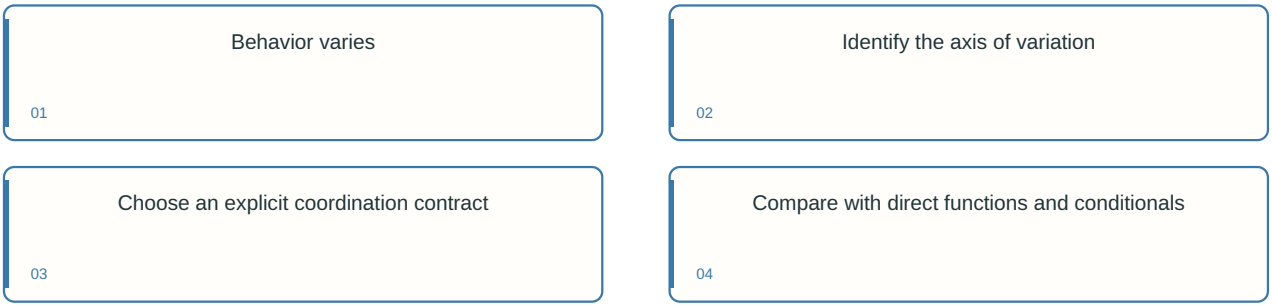


FIG. 6.6.1 **Strategy, Observer, Command, State, Chain of Responsibility, Template Method, Iterator, Mediator, Memento, Visitor · the example**

Behavioural patterns organise algorithms, notifications, intentions, state transitions, traversal, and coordination at chosen extension points.

Author-created illustration. Numbers are illustrative unless stated otherwise.

PATTERN / MECHANISM, USE, AND COST

Strategy

An order's price can be calculated under different pricing policies. Strategy supplies one calculation behind a contract, letting the caller use the selected policy without embedding every algorithm.

Strategies must agree on amounts, units, errors, and effects. If there is only one stable formula, a function may be sufficient. Strategy varies a selected policy; State varies behaviour according to an entity's lifecycle and history. Choose the former when a policy is meant to be interchangeable, rather than letting a caller arbitrarily replace a payment's valid lifecycle state.

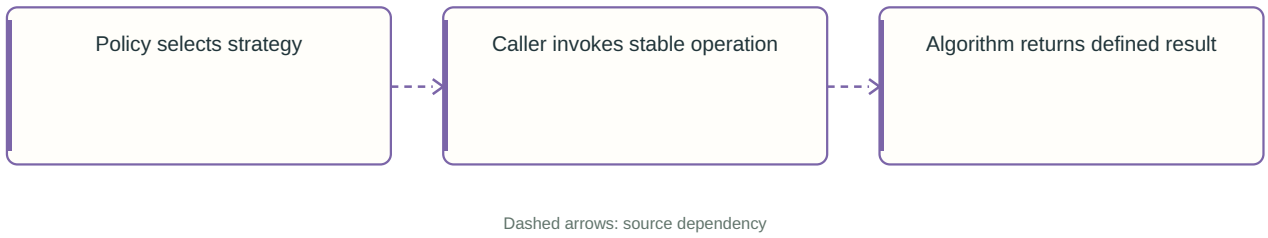


FIG. 6.6.2 **Strategy · the example**

Supplies an interchangeable algorithm or policy behind a stable behavioural contract.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
price = pricing_strategy.calculate(order)
require(price.currency == order.currency)
```

PATTERN / MECHANISM, USE, AND COST

Observer

A subject can notify several subscribed observers when something changes. Observer is useful for one-to-many reactions without making the subject call each concrete implementation directly.

Decide whether callbacks run inline, in what order, and what happens if one throws or runs slowly. Remove subscriptions when their owners end so they do not retain unwanted objects. A local callback list exists only in process memory. When a required reaction must survive process loss, use an appropriate durable publication mechanism rather than treating local observation as a message broker.

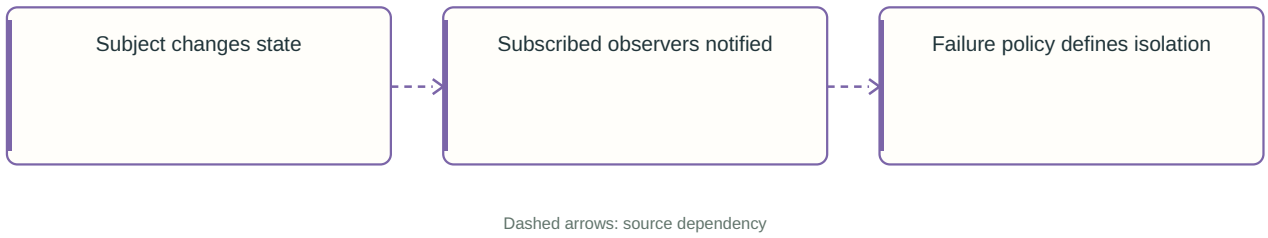


FIG. 6.6.3 **Observer · the example**

Notifies subscribed listeners under defined timing, lifetime, ordering, and failure rules.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
for observer in snapshot(subscribers):
    notify_under_defined_failure_policy(observer, event)
```

PATTERN / MECHANISM, USE, AND COST

Command

SendReceipt(order_id, operation_id) records an intention that a handler can execute later. Command gives a requested action explicit data and identity. An event instead records a fact that has already been accepted.

For durable work, prefer serialisable domain data over a closure holding a request's live connections. At execution time, the handler must check current validity and the chosen authorisation policy. Retries need duplicate handling. Representing an intention makes scheduling and audit easier, but storing a command alone does not establish safe repeated external execution.

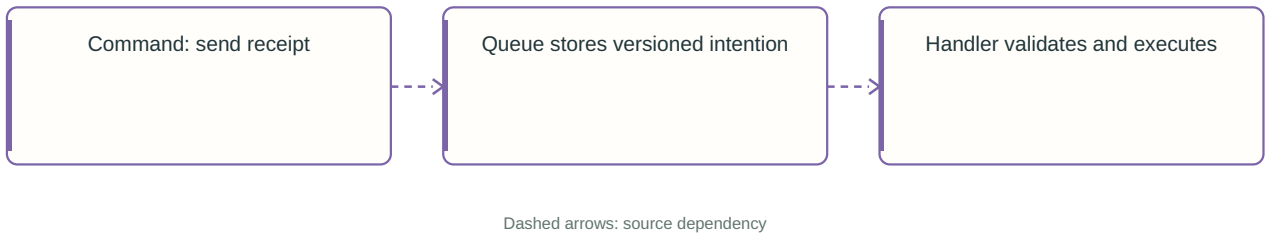


FIG. 6.6.4 **Command · the example**

Represents a requested action as explicit data and identity for an appropriate handler.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
command = SendReceipt(order_id, operation_id)
handler.handle(command)
# Handler checks current authorization and state
```

PATTERN / MECHANISM, USE, AND COST

State

A pending payment and a captured payment accept different actions. State organises behaviour around that lifecycle. A guarded transition changes the state when the required evidence arrives.

A small transition table can be clearer than a class for every state. Whichever representation is used, persist related state changes atomically and keep invalid transitions explicit. A caller cannot simply select “captured” as if choosing an interchangeable algorithm: the payment's history and evidence constrain it. This distinguishes lifecycle-dependent State from a selected Strategy.



FIG. 6.6.5 **State · the example**

Organises behaviour around a lifecycle whose guarded transitions follow entity history.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
require((payment.state, event.kind) in allowed_transitions)
payment.state = next_state(payment.state, event)
```

PATTERN / MECHANISM, USE, AND COST

Chain of Responsibility

A request passes through authentication, rate limiting, and parsing before reaching the application. A Chain of Responsibility gives each handler defined choices: reject, handle, transform, or continue.

Specify the order and continuation rules. Forgetting to invoke the next handler can silently swallow work; placing expensive parsing before admission can change cost and exposure. A chain is a coordination structure, not a durable workflow protocol. If several stages must complete across crashes, they also need persistent progress and recovery rules.

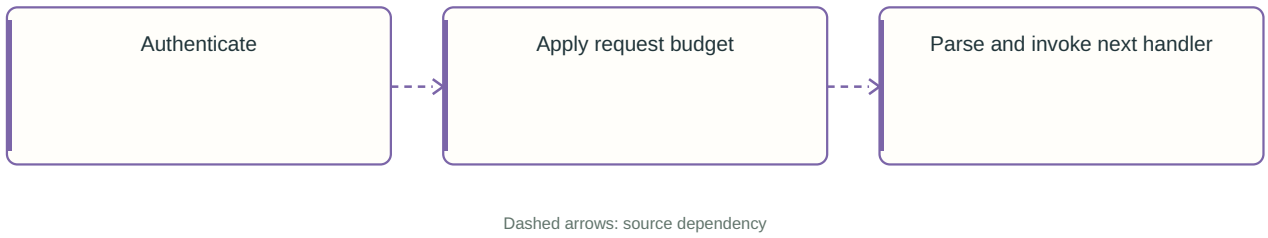


FIG. 6.6.6 **Chain of Responsibility · the example**

Passes a request through ordered handlers with explicit continuation, rejection, and handling rules.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
auth(rate_limit(parse(application)))(request)
# Each handler has explicit rejection and continuation rules
```

PATTERN / MECHANISM, USE, AND COST

Template Method

Every import job must validate input, transform it, and commit the result in that order. Template Method fixes the overall sequence while allowing selected steps, such as format conversion, to vary.

The base process must preserve its required checks when subclasses customise a step. Inheritance is a common implementation, but independent variations may be easier to express through composed strategies. Use the pattern when a stable process really has controlled extension points. Letting subclasses bypass validation or commit ordering undermines the very sequence the template was meant to protect.

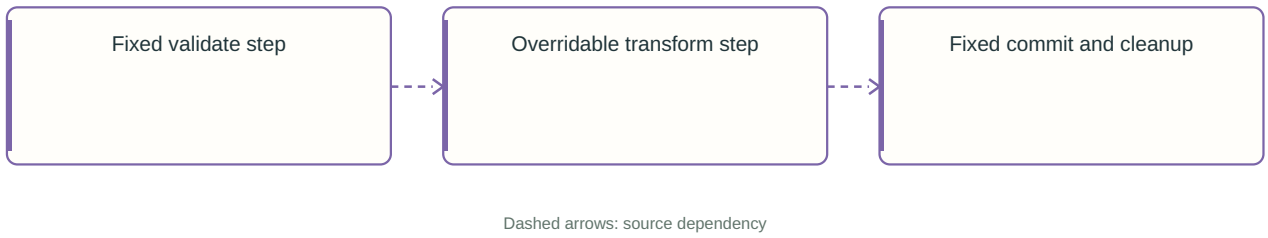


FIG. 6.6.7 **Template Method · the example**

Preserves a process skeleton while allowing selected implementation steps to vary.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
run(input):
    validate(input)
    result = transform(input)
    return commit(result)
```

PATTERN / MECHANISM, USE, AND COST

Iterator

A caller asks an Iterator for the next item without needing to know whether it comes from an array, tree, or database cursor. The iterator keeps the traversal position.

A lazy database iterator can also hold a connection. A slow reader may keep that resource open, while closing its owner too early can make the next read fail. Define closing, end-of-stream, ordering, and behaviour under mutation. For a small bounded result, materialising a list and releasing the connection before returning may provide a simpler contract.

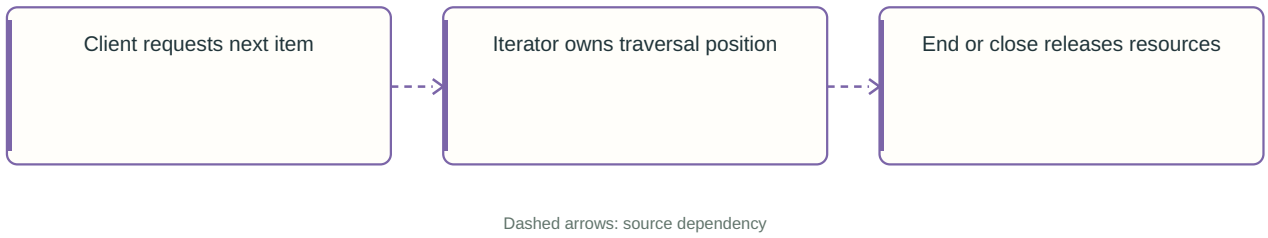


FIG. 6.6.8 **Iterator · the example**

Exposes traversal without revealing collection representation, while retaining ordering and resource-lifetime obligations.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
with open_bounded_cursor(query) as rows:
    for row in rows:
        consume(row)
```

PATTERN / MECHANISM, USE, AND COST

Mediator

Inventory and payments need coordinated steps during checkout. A Mediator can receive their results and decide the next action, reducing direct knowledge between the collaborators.

Give the coordinator a focused responsibility and explicit failure handling. Otherwise, it can become a large object that knows every detail of the application. In a distributed setting, a durable workflow orchestrator can carry this coordination across restarts, but the object pattern alone supplies no persistence. Separate the coordination idea from the mechanism needed to recover its progress.

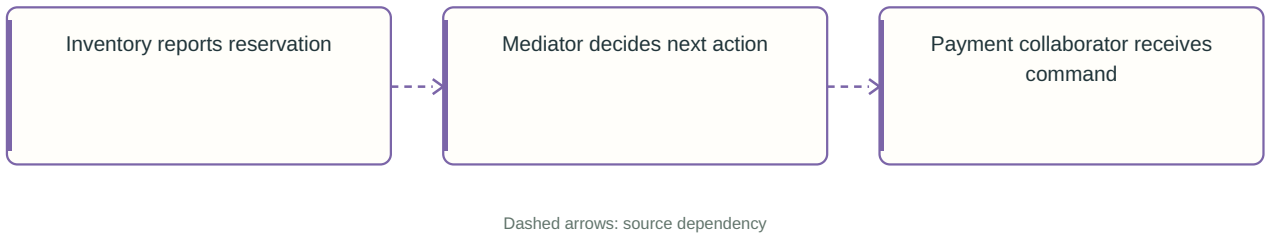


FIG. 6.6.9 **Mediator · the example**

Coordinates collaborators through a focused owner instead of many direct relationships.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
reservation = inventory.reserve(command)
outcome = payments.authorize(command.operation_id)
coordinate_outcomes(reservation, outcome)
```

PATTERN / MECHANISM, USE, AND COST

Memento

Before an edit, a draft captures the values needed to restore its earlier local state. Memento provides such a supported snapshot without giving every caller access to arbitrary internals.

Define which values are copied and what restoration means. References to shared mutable objects can make a supposed snapshot change later. Restoring the draft also cannot retract an email or reverse a payment already sent elsewhere. Use the pattern where local capture and restoration are well defined; external compensation, backups, and event histories address different recovery needs.

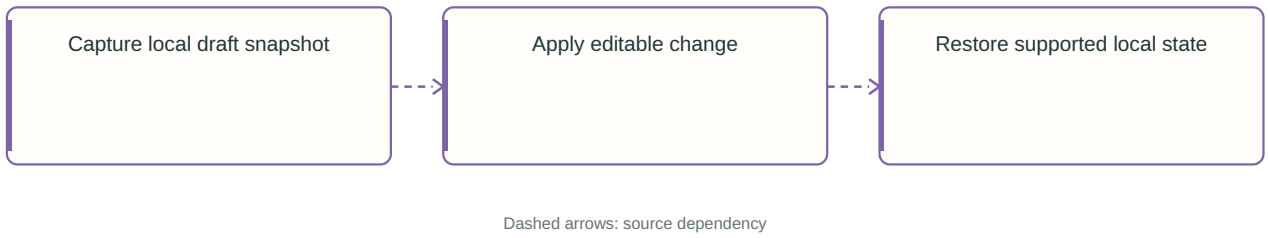


FIG. 6.6.10 **Memento · the example**

Captures and restores supported local state without claiming to reverse independent external effects.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
before = draft.snapshot_owned_values()
draft.apply(edit)
draft.restore(before)
# No claim about undoing external effects
```

PATTERN / MECHANISM, USE, AND COST

Visitor

A document tree contains known node types such as text and image. A Visitor supplies an operation, such as export or validation, that handles each supported node type.

This makes adding operations easier without placing every operation on every node. Adding a new node kind can require updating all visitors, so the trade-off reverses for structural change. Pattern matching may be a simpler expression in some languages. Make unknown-node handling explicit so a new document version does not silently lose content during export.



FIG. 6.6.11 **Visitor · the example**

Adds operations across a known structure, trading easier new operations for more work when element types change.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
visit(node):
    match node.kind:
        case "text": return export_text(node)
        case "image": return export_image(node)
        case _: raise UnsupportedNode
```

▼ Think further · How should resources be owned when a job outlives its request and user session?

Give the job an explicit lifetime and scope, with durable identifiers rather than captured request objects. Create or borrow resources under a documented owner and close them at the appropriate boundary. Define whether current membership must be rechecked; a long-lived job must not silently inherit expired user authority.

Remember: Choose the extension point that matches the application's actual source of change.

Try applying it: How does choosing a pricing strategy differ from moving a payment into its next valid state?

Unit 6 exercises → · Unit 6 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Inversion of Control Containers and the Dependency Injection pattern — Martin Fowler, 2004 ↗

   Original engineering article. Dependency injection and wiring.

   Access reviewed · 21 October 2026

2. The Catalog of Design Patterns — Refactoring Guru ↗

   Engineering teaching reference. Pattern taxonomy; miniature examples and judgments are original to this atlas.

   Access reviewed · 21 October 2026

Enterprise patterns & boundaries

Prerequisites: 6.6 · Behavioral patterns 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A reservation update needs data access, eligibility checks, and a commit decision. Compare the patterns that make these responsibilities explicit without inventing a transaction across unrelated systems.

6.7.1 · Repository, Unit of Work, Specification, CQRS, and dependency boundaries

A repository offers application-oriented operations for stored data. A Unit of Work groups changes under one commit decision. A Specification represents a reusable rule for selecting or accepting values. CQRS uses different models for changing data and reading it.

For example, an order update can use a Unit of Work to commit its related local changes together. The owner must also handle rollback and cleanup. These boundaries should expose important failure differences: a local database commit and a remote provider call are not one atomic operation. CQRS may use one process and database; event sourcing and separate deployments are additional decisions, not requirements of the pattern.

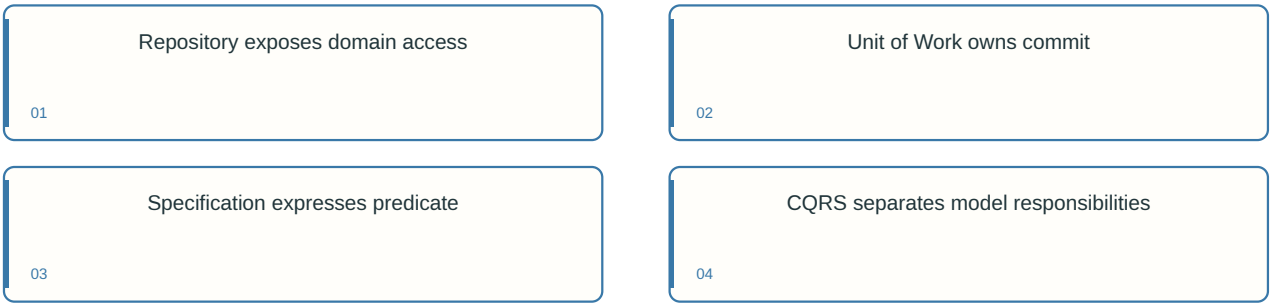


FIG. 6.7.1 **Repository, Unit of Work, Specification, CQRS, and dependency boundaries · the example**

Repository, Unit of Work, Specification, and CQRS clarify data access, commit ownership, predicates, and distinct read/write models.

Author-created illustration. Numbers are illustrative unless stated otherwise.

PATTERN / MECHANISM, USE, AND COST

Repository

A reservation service needs to load and save reservations while preserving their rules. Repository provides application-oriented data operations and keeps SQL or other storage details in an adapter.

State transaction ownership, error outcomes, query bounds, and returned-value lifetimes. A repository per table may not match an aggregate's required atomic boundary. Avoid concealing expensive remote calls or live cursors behind a misleadingly simple name. Use the boundary where it hides useful knowledge, and keep real storage integration checks: a test substitute cannot demonstrate actual commit and recovery behaviour.

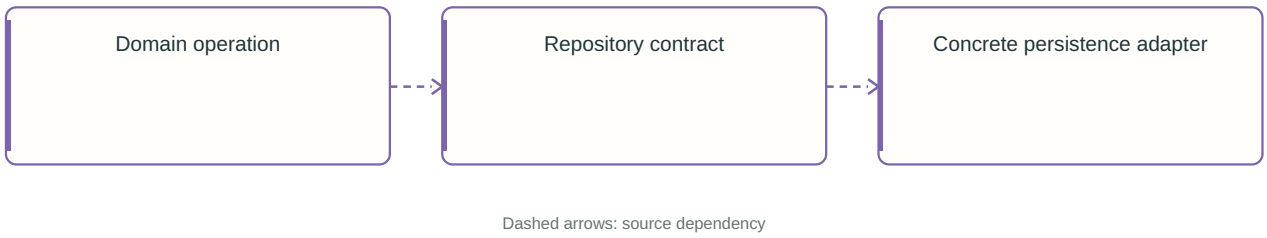


FIG. 6.7.2 **Repository · the example**

Offers application-oriented persistence operations through a contract defining transactions, bounds, errors, and lifetimes.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
reservation = repository.get_for_update(reservation_id)
reservation.confirm(payment_evidence)
repository.save(reservation)
```

PATTERN / MECHANISM, USE, AND COST

Unit of Work

Reserving inventory and inserting the local reservation should either both commit or both fail. Unit of Work groups these changes under one explicit commit decision and owns their transaction resources.

All participating local repositories must respect that owner instead of committing independently. Keep the unit short and arrange rollback and cleanup. Calling a remote payment provider inside the same language scope does not make its charge part of the database transaction. External effects need their own protocol and recovery. The useful boundary is the actual transaction, rather than the indentation of the code.

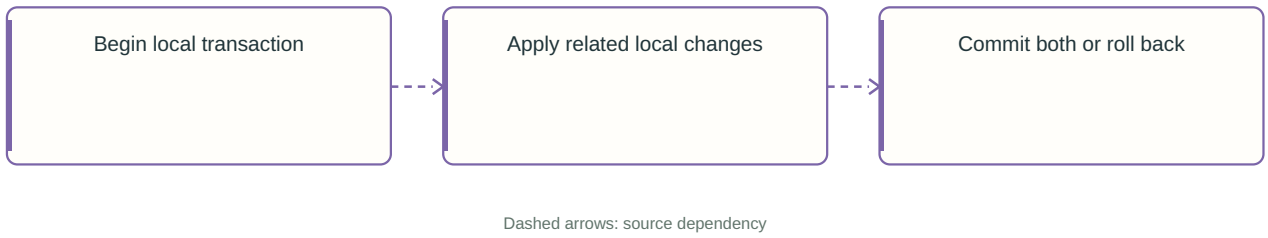


FIG. 6.7.3 **Unit of Work · the example**

Owns related local changes under one commit and cleanup boundary.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
with unit_of_work() as uow:
    uow.inventory.reserve(seat)
    uow.reservations.add(record)
    uow.commit()
```

PATTERN / MECHANISM, USE, AND COST

Specification

A refund policy can express “this payment is eligible at this time” as a named predicate. Specification makes such a selection or eligibility rule reusable and, where useful, composable.

The predicate describes a decision based on particular state. That state can change before a refund is applied, so the atomic transition must recheck the relevant version or conditions. When translating a specification into SQL, preserve time, null, and access semantics. A plain function is often enough; add an expression framework only when genuine composition needs justify it.

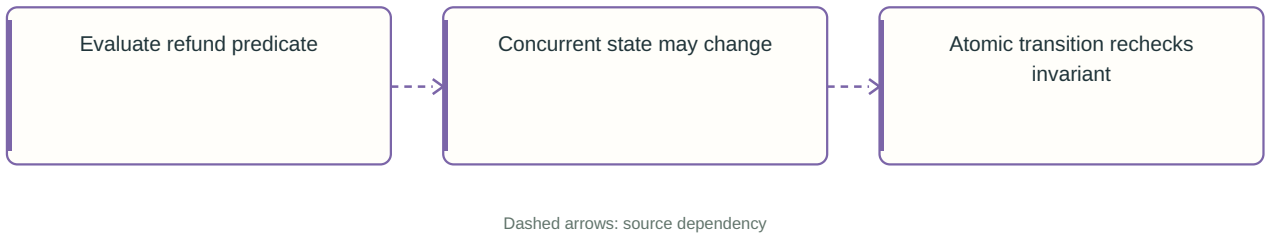


FIG. 6.7.4 **Specification · the example**

Names a reusable eligibility or selection predicate whose later state-changing use may need atomic revalidation.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
eligible = RefundPolicy.satisfied_by(payment, now)
if eligible:
    repository.refund_if_still_eligible(payment.version)
```

PATTERN / MECHANISM, USE, AND COST

CQRS

Capturing a payment needs a model that protects allowed transitions. Displaying an account statement may need a different, read-friendly shape. CQRS separates these command and query models when their needs justify it.

A projection can build the read model from accepted changes, but then it may lag and must be rebuildable. State that freshness contract. Both models can exist in one process and one database; event sourcing and separate services are optional choices. When one simple model already serves both uses, the additional synchronisation and maintenance may not be worthwhile.

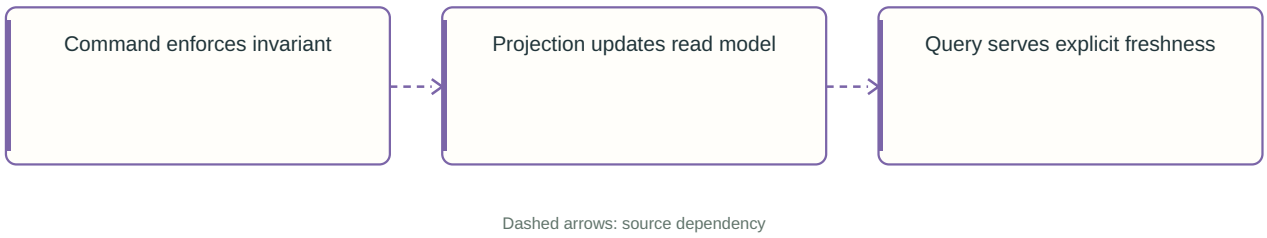


FIG. 6.7.5 **CQRS · the example**

Uses different command and query models where their needs justify extra projection and freshness work.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
commands.capture(payment_id, operation_id)
statement = queries.account_statement(account_id)
# Projection lag is part of the query contract
```

Try the mechanism in code

PYTHON · LIFETIME IDENTITIES

```
singleton = object()
def new_scope(): return {"scoped": object()}
a, b = new_scope(), new_scope()
assert a["scoped"] is a["scoped"]
assert a["scoped"] is not b["scoped"]
assert object() is not object()
assert singleton is singleton
```

Within a scope, the same scoped identity is reused. Different scopes have different objects, while the process-owned singleton stays the same.

This models identity only. An actual framework also defines registration and disposal. A request-owned resource cannot safely become a retained singleton dependency just because a reference remains available.

Download the executable mechanism collection ↓ · Follow the connected complete design →

▼ Think further · How should resources be owned when a job outlives its request and user session?

Give the job an explicit lifetime and scope, with durable identifiers rather than captured request objects. Create or borrow resources under a documented owner and close them at the appropriate boundary. Define whether current membership must be rechecked; a long-lived job must not silently inherit expired user authority.

Remember: State which rule, model, and transaction each persistence pattern owns.

Try applying it: Which parts of a reservation and external payment can a local Unit of Work actually commit together?

Unit 6 exercises → · Unit 6 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Inversion of Control Containers and the Dependency Injection pattern — Martin Fowler, 2004 ↗

   Original engineering article. Dependency injection and wiring.

   Access reviewed · 21 October 2026

2. The Catalog of Design Patterns — Refactoring Guru ↗

   Engineering teaching reference. Pattern taxonomy; miniature examples and judgments are original to this atlas.

   Access reviewed · 21 October 2026

PART 2 / UNIT 07

Application Architecture and Domain-Driven Design

A shopping application can separate input, policy, and storage without putting them on different machines. Compare architecture styles by their dependency rules and the changes they help contain.

The running example

Design an online shopping application first as a modular monolith, then examine the conditions under which selected domains should become independent services.

Unit 7 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

View	Boundary rule	Runtime implication
Layered	Separate technical roles	Can remain one process
Hexagonal	Ports express app needs	Adapters call in or out
Clean/Onion	Policy dependencies inward	Runtime effects still reach adapters

FIG. 7.0.1 **Layered vs Hexagonal**

The styles organize source responsibilities, not mandatory physical deployment layers.

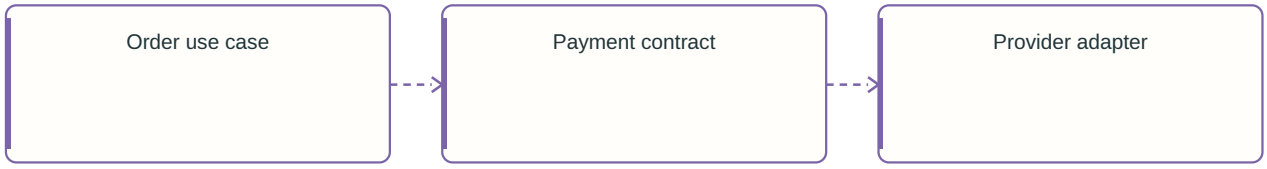
Author-created illustration. Numbers are illustrative unless stated otherwise.

Design	Deployment	Main cost
Modular monolith	One release unit	Must enforce internal ownership
Microservices	Independent processes	Network, versions, operations
Distributed monolith	Many coupled processes	Costs without independence

FIG. 7.0.2 **modular monolith vs microservices**

Choose according to actual independent operation and change needs.

Author-created illustration. Numbers are illustrative unless stated otherwise.



Dashed arrows: source dependency

FIG. 7.0.3 **dependency arrows**

Dependency arrows express knowledge; runtime execution reaches the selected adapter.

Author-created illustration. Numbers are illustrative unless stated otherwise.



Dashed arrows: source dependency

FIG. 7.0.4 **bounded context map**

Bounded contexts retain different valid meanings rather than sharing every field globally.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Layers, monoliths & ports

Prerequisites: Object-Oriented and Modular Software Design, Design Patterns, Dependency Injection, and Object Lifecycles

6 original figures

Blue: normal flow · Green: healthy · Amber: caution · Red: failure · Purple: control / shared infrastructure · Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A shopping application can separate input, policy, and storage without putting them on different machines. Compare architecture styles by their dependency rules and the changes they help contain.

7.1.1 · Layered/N-tier architecture

A shopping request arrives through a controller, invokes an application operation, applies pricing rules, and stores the order. Layered architecture gives these different kinds of work clear places: presentation, application coordination, domain policy, and infrastructure.

The layers can all run in one process. Deploying them as separate physical tiers is another choice. Use the division to protect useful rules: a controller should not bypass order validation simply because it can reach an SQL helper. Follow a change, such as a new pricing rule, to see whether the boundary helps. Classes that only forward every call add work without necessarily hiding any detail.

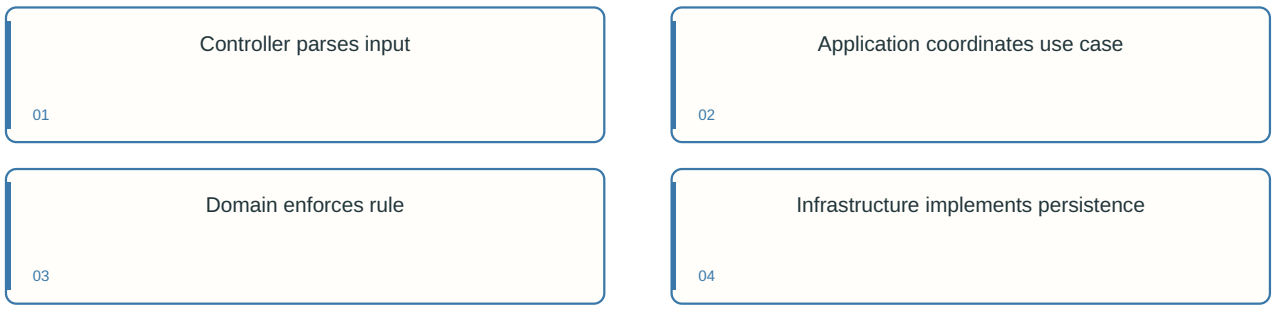


FIG. 7.1.1 **Layered/N-tier architecture · the example**

Layered architecture separates kinds of responsibility; physical tiers are a separate deployment choice.

Author-created illustration. Numbers are illustrative unless stated otherwise.

7.1.2 · Modular monoliths

Catalog, orders, and payments can be modules of one deployable application. Each exposes operations and owns its data, while calls between them remain local. This is a modular monolith.

The word “modular” matters. If every module writes every table, changing a payment record can still break the whole program. Use agreed interfaces and enforce dependency rules through review or useful automated checks. This arrangement keeps deployment and local transactions relatively simple while giving business responsibilities separate homes. Later extraction remains possible, but it is an option supported by the boundaries, rather than a benefit obtained automatically by naming folders.

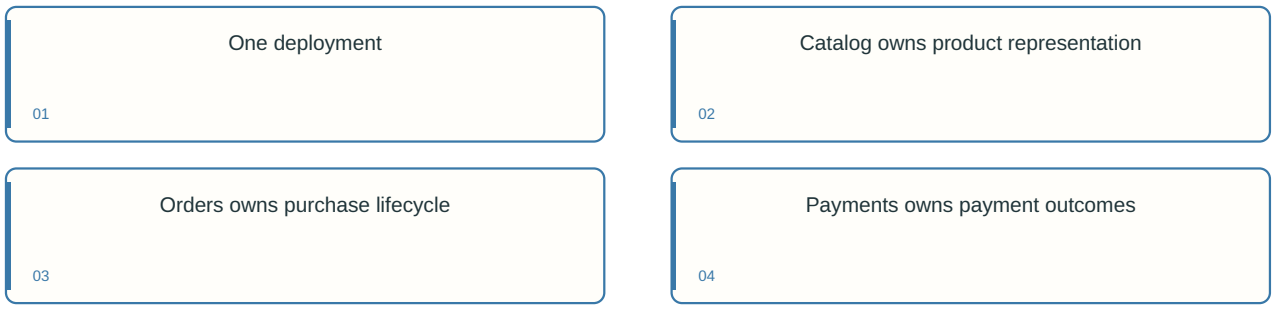


FIG. 7.1.2 **Modular monoliths · the example**

A modular monolith has one release unit with enforced internal contracts and data ownership.

Author-created illustration. Numbers are illustrative unless stated otherwise.

7.1.3 · Monolith vs microservices

Suppose inventory needs to scale independently and has a team ready to operate it. Making it a separate service may provide a useful deployment and ownership boundary. It also replaces some local calls with network calls that can time out or partially fail.

Microservices aim for useful independent deployment and operation. A monolith can be well organised, while several services can remain tightly coupled. If five services must release together for each change, ask how much independence was gained. Compare the motivation for extraction with the added work in data coordination, compatibility, monitoring, and recovery. A process boundary should solve a demonstrated problem.

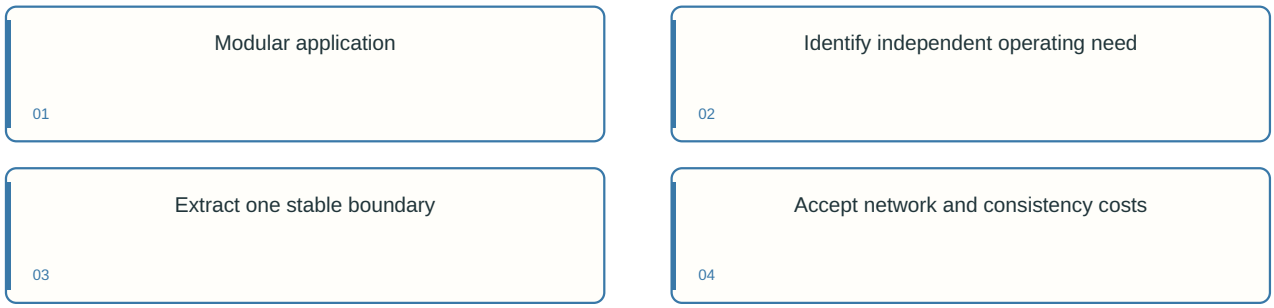


FIG. 7.1.3 **Monolith vs microservices · the example**

Microservices seek useful independent deployment and operation while adding network, version, and coordination work.

Author-created illustration. Numbers are illustrative unless stated otherwise.

7.1.4 · Clean Architecture

An order rule should be testable without constructing an HTTP response or starting a web framework. Clean Architecture supports this by making framework and storage details depend on interfaces used by the application's policy.

The inward arrows in its usual diagrams describe source dependencies. During execution, a use case can still call an outer storage adapter through an injected interface. Avoid making core rules depend on ORM-specific records or transport response objects. The separation costs mappings and contracts, so choose a proportionate version for a small application. The important question is whether policy can be understood and exercised without unrelated infrastructure details.

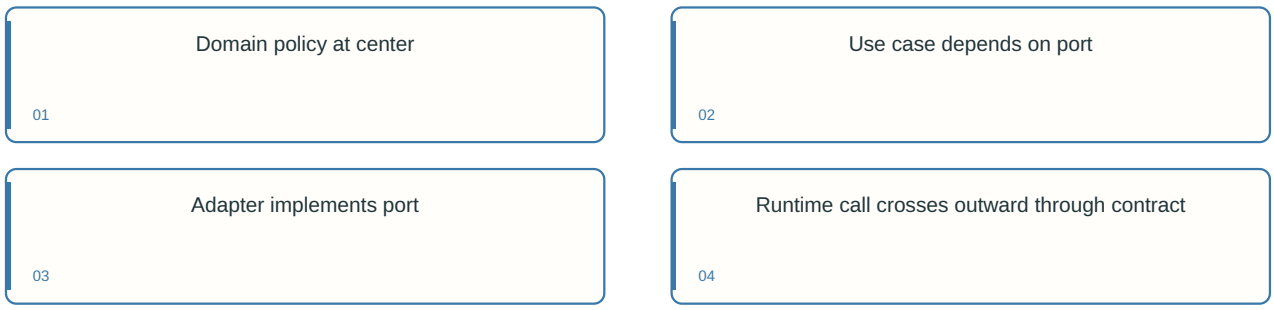


FIG. 7.1.4 **Clean Architecture · the example**

Clean Architecture directs source dependencies toward policy through application-owned abstractions.

Author-created illustration. Numbers are illustrative unless stated otherwise.

7.1.5 · Hexagonal/Ports and Adapters

An order can be created from a web request or a command-line tool. Both adapters can invoke the same application operation. Likewise, an SQL adapter and a test repository can satisfy the application's storage needs.

Hexagonal architecture calls these application-facing contracts ports and their implementations adapters. An inbound port expresses an operation the application offers; an outbound port expresses work it needs from elsewhere. Design ports around those needs rather than copying an entire vendor API. The drawing does not imply separate machines. Test application decisions with controlled adapters, then separately check real encoding, transactions, and failure behaviour at the external boundaries.

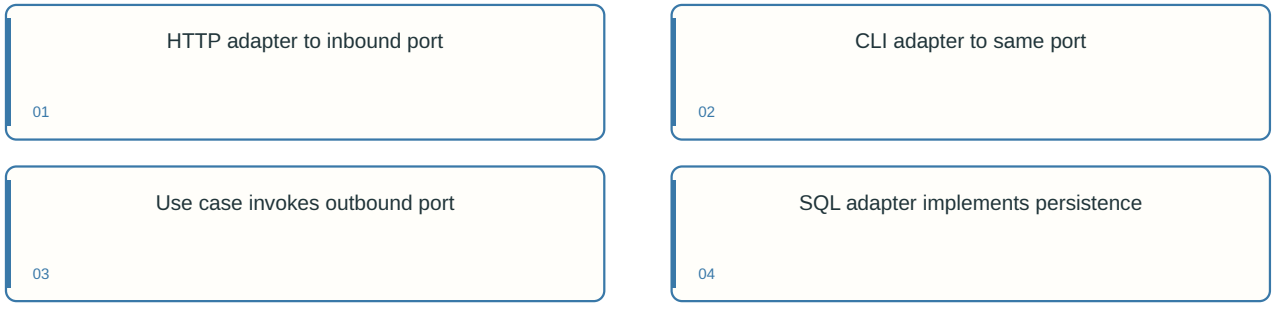


FIG. 7.1.5 **Hexagonal/Ports and Adapters · the example**

Hexagonal architecture uses application ports and adapters to separate core operations from external mechanisms.

Author-created illustration. Numbers are illustrative unless stated otherwise.

7.1.6 · Onion Architecture

Onion Architecture arranges domain policy, application services, and infrastructure so that dependencies point toward the core rules. An order rule can then use an application-owned repository contract rather than import database-specific code.

Its vocabulary overlaps with Clean and Hexagonal Architecture, and teams draw the layers differently. Compare the actual dependency and ownership rules instead of treating the shape as a guarantee. Can pricing run without a web server? Does a failed write remain visible to the caller? These questions reveal whether the separation helps. Extra layers are worthwhile when they protect such properties, rather than simply matching a diagram.

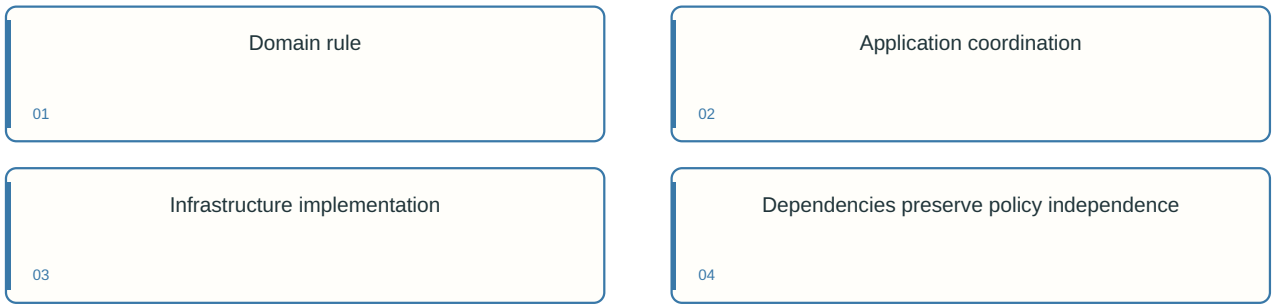


FIG. 7.1.6 **Onion Architecture · the example**

Onion Architecture organises inward policy dependencies; concrete contracts matter more than diagram shape.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · What evidence would justify extracting payments into a separate service?

Look for stable capability ownership, different scaling or isolation needs, and an operating team prepared for network and version failures. Compare those benefits with added data coordination and recovery. Preserve existing payment invariants and histories through migration. A separate deployment is useful only when the required independence can actually be exercised.

Remember: Code boundaries and deployment boundaries solve related but different problems.

Try applying it: What benefit would justify turning a well-separated module into a remote service?

Unit 7 exercises → · Unit 7 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Architecture styles — Microsoft Architecture Center ↗

   Official guidance · accessed 21 Oct 2026. Style comparisons; provider-specific recommendations are not mandatory in the atlas.

   Access reviewed · 21 October 2026

2. Bounded Context — Martin Fowler, 2014 ↗

   Original engineering explanation. Model boundaries and language.

   Access reviewed · 21 October 2026

3. Microservices — James Lewis and Martin Fowler, 2014 ↗

   Original engineering article. Architecture characteristics and costs; no exact company architecture is asserted here.

   Access reviewed · 21 October 2026

Model a bounded context

Prerequisites: 7.1 · Layers, monoliths & ports 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Catalog and fulfilment use different meanings for a product. Model those meanings with domain experts, then choose where rules need an atomic aggregate and which facts should become events.

7.2.1 · Domain-Driven Design (DDD)

Ask a shop's merchandising and warehouse staff what “available stock” means. Merchandising may mean stock that can be advertised. A reservation worker may mean units that can still be promised to a buyer. Treating both as one unexamined field can produce wrong decisions.

Domain-Driven Design develops a model with the people who understand the business. The model names concepts and makes their rules explicit. Use this effort where business meaning is complex or disputed. A simple lookup table may need only straightforward data handling. DDD's value comes from clearer decisions and shared understanding, rather than assigning a class to every database row.

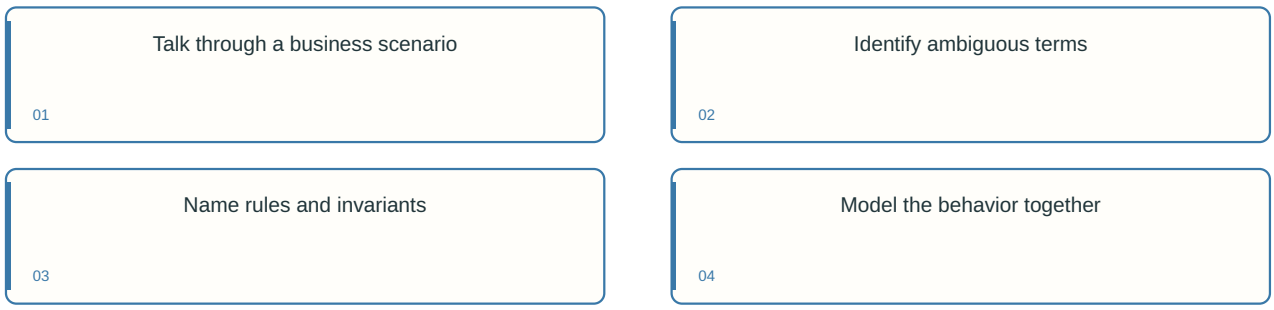


FIG. 7.2.1 **Domain-Driven Design (DDD) · the example**

Domain-Driven Design develops a shared business model with domain experts where rules and meanings need clarification.

Author-created illustration. Numbers are illustrative unless stated otherwise.

7.2.2 · Bounded contexts and ubiquitous language

Catalog's Product may describe a title, images, and marketing categories. Fulfilment's Item may describe a physical unit and its shelf location. Both models can be correct for their own work.

A bounded context states where a model and its vocabulary apply. Within it, a ubiquitous language gives team members and code consistent names for the business concepts. Between contexts, translate deliberately and identify who owns each fact. One universal Product schema may obscure real differences. A context is a modelling boundary; deciding whether it becomes a separate service, repository, or team requires further reasons.

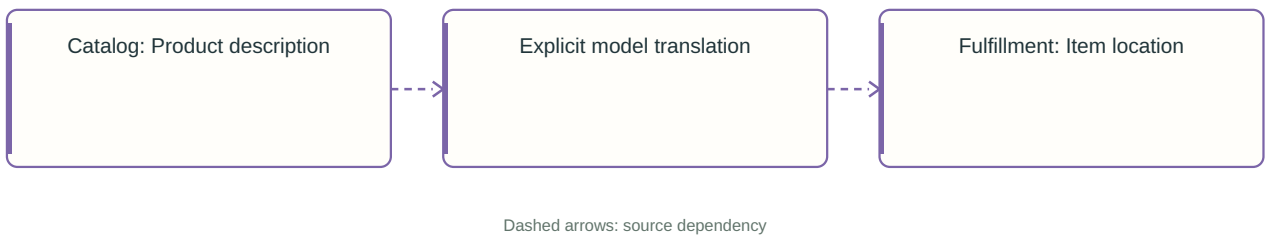


FIG. 7.2.2 **Context boundaries protect different business meanings**

The source-dependency boundary is not automatically a network deployment. Translation prevents one universal schema from erasing meaningful domain differences.

Author-created illustration. Numbers are illustrative unless stated otherwise.

7.2.3 · Aggregates, entities, value objects, domain services

An order's line items and total must agree when it is committed. We can give an Order root responsibility for changing those related values together. An aggregate is this kind of consistency boundary, with a root controlling changes to its members.

Entities have identity through change; value objects are distinguished by their values. A domain service can express a rule that does not belong to one entity. Keep an aggregate focused on invariants that need a common atomic change. Putting the entire shop into one aggregate creates contention and loading costs. Rules spanning aggregates need an explicit coordination design; an object reference alone does not create a distributed transaction.

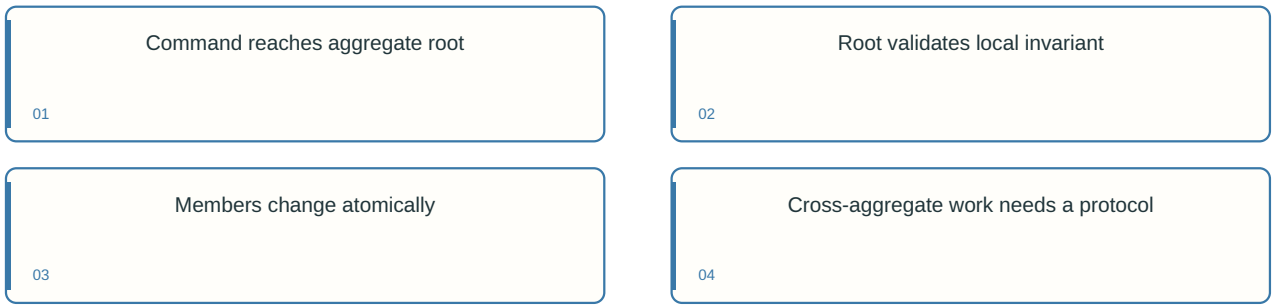


FIG. 7.2.3 **Aggregates, entities, value objects, domain services · the example**

An aggregate groups changes requiring one consistency boundary, with a root controlling its members' transitions.

Author-created illustration. Numbers are illustrative unless stated otherwise.

7.2.4 · Domain events

After an order is accepted, other parts of the system may need to know. OrderPlaced records a business fact. Give it a stable identity, clear time meaning, and information that its intended consumers can interpret across versions.

Creating an event object in memory is not the same as publishing it reliably. The process may commit the order and die before sending the event. An outbox records order state and publication intent in one local transaction, then relays the event. Consumers still need to handle repeated and delayed delivery. Define a fact's meaning before deciding how the broker will carry it.

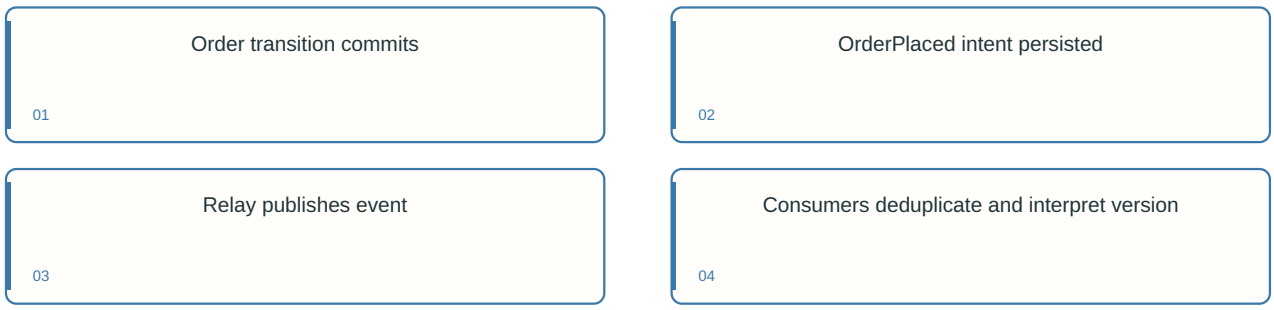


FIG. 7.2.4 **Domain events · the example**

A domain event records a meaningful accepted fact; durable publication needs a separate explicit mechanism.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · What evidence would justify extracting payments into a separate service?

Look for stable capability ownership, different scaling or isolation needs, and an operating team prepared for network and version failures. Compare those benefits with added data coordination and recovery. Preserve existing payment invariants and histories through migration. A separate deployment is useful only when the required independence can actually be exercised.

Remember: A model is useful within a context whose vocabulary, ownership, and invariants are understood.

Try applying it: Which purchase-time facts should an order retain when the current catalog changes?

Unit 7 exercises → · Unit 7 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Architecture styles — Microsoft Architecture Center ↗

   Official guidance · accessed 21 Oct 2026. Style comparisons; provider-specific recommendations are not mandatory in the atlas.

   Access reviewed · 21 October 2026

2. Bounded Context — Martin Fowler, 2014 ↗

   Original engineering explanation. Model boundaries and language.

   Access reviewed · 21 October 2026

3. Microservices — James Lewis and Martin Fowler, 2014 ↗

   Original engineering article. Architecture characteristics and costs; no exact company architecture is asserted here.

   Access reviewed · 21 October 2026

Keep business policy inside

Prerequisites: 7.2 · Model a bounded context 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

An order rule should not be rewritten in controllers, broker configuration, and provider adapters. Assign the business decision, its coordination, and its technical effects to clear owners.

7.3.1 · Dependency rules and module boundaries

Suppose the orders module updates a payment table directly. It now knows the payment representation and can bypass payment rules. A payment interface gives that interaction a controlled place.

Dependency rules specify which modules may know about or call others. Enforce important boundaries with appropriate import checks, database permissions, or architecture tests. A cycle can reveal that coordination has no clear owner. Shared helpers also need scrutiny: one “common model” can accidentally force unrelated contexts to change together. Where meanings genuinely differ, use an explicit translation rather than sharing a type merely because its fields currently look similar.

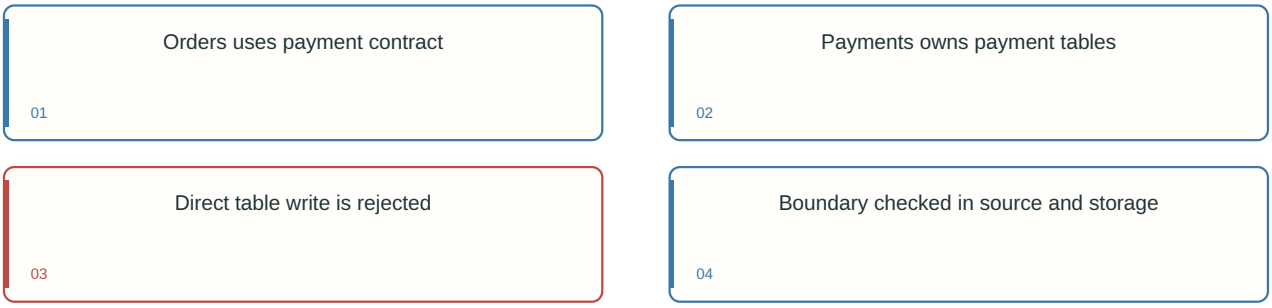


FIG. 7.3.1 **Dependency rules and module boundaries · the example**

Module dependency rules limit knowledge and bypass paths, using review and useful enforcement where appropriate.

Author-created illustration. Numbers are illustrative unless stated otherwise.

7.3.2 · Business logic vs orchestration vs infrastructure

“A refunded order must not ship” is a business rule. Asking the payment service for a refund and then updating progress is orchestration. Encoding and sending a provider request is infrastructure work.

These responsibilities may begin in one function, but distinguishing them helps assign tests and handle failures. Test the shipping rule as a decision. Test orchestration across success and partial failure. Test the provider adapter against its actual protocol. Moving a sequence onto a queue does not make its business rules belong to the broker. Keep the owner of each decision visible even when the execution arrangement changes.

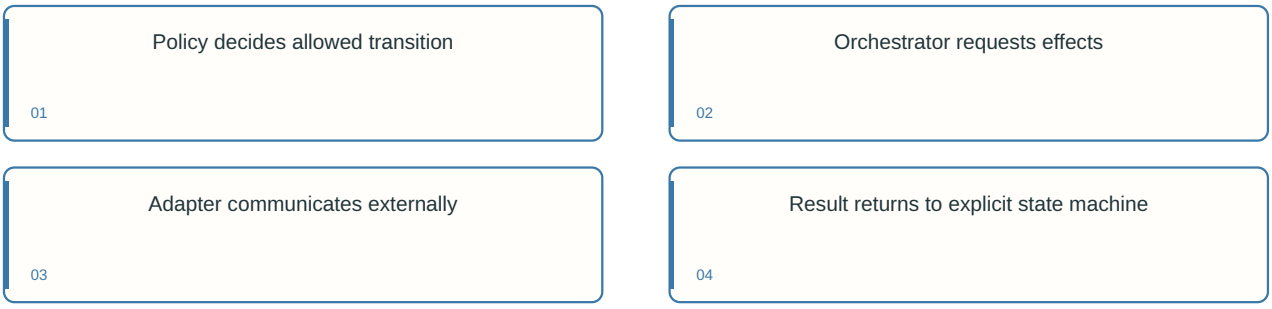


FIG. 7.3.2 **Business logic vs orchestration vs infrastructure · the example**

Business logic decides outcomes, orchestration sequences work, and infrastructure performs technical effects.

Author-created illustration. Numbers are illustrative unless stated otherwise.

7.3.3 · Controllers, services, repositories, adapters

A controller reads a request and builds a response. An application service coordinates an operation. A repository offers data operations, and an adapter connects a contract to a particular protocol or technology.

These are roles, not required class-name endings. A service can be a function; a domain service is not necessarily a network service. Examine the work the code performs. If a controller calculates prices and retries transactions, creating an empty “services” directory has not moved those responsibilities. Choose names and boundaries that help readers follow one real operation from its input to its result.

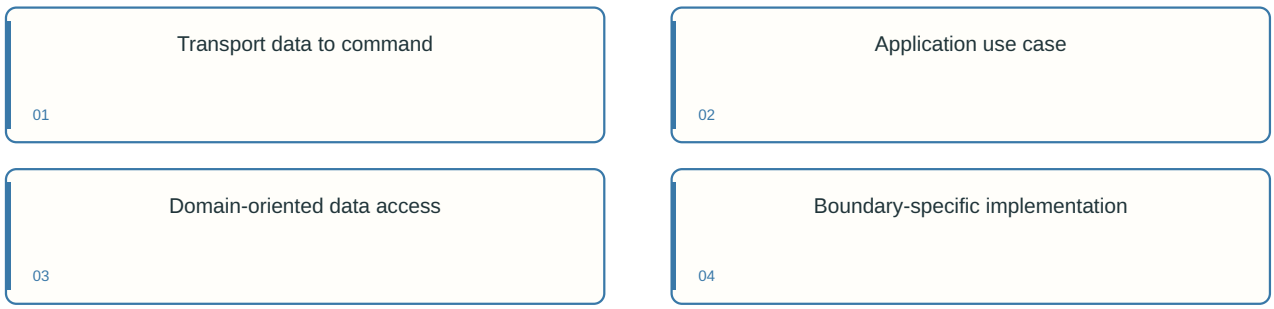


FIG. 7.3.3 **Controllers, services, repositories, adapters · the example**

Controllers, services, repositories, and adapters are responsibilities, not mandatory class-name conventions.

Author-created illustration. Numbers are illustrative unless stated otherwise.

7.3.4 · Anti-corruption layers

A legacy payment provider may use “settled” for an authorisation that our application still considers pending. Copying that label directly would import the provider's meaning into our own model.

An anti-corruption layer translates the external model into the local one. It maps known states, retains useful provider references, and handles unknown meanings explicitly. This adds mapping and maintenance work, but keeps external quirks from spreading through business code. Never turn an unmapped result silently into success. Where the provider's state is uncertain or different from ours, preserve enough information to investigate and reconcile it.

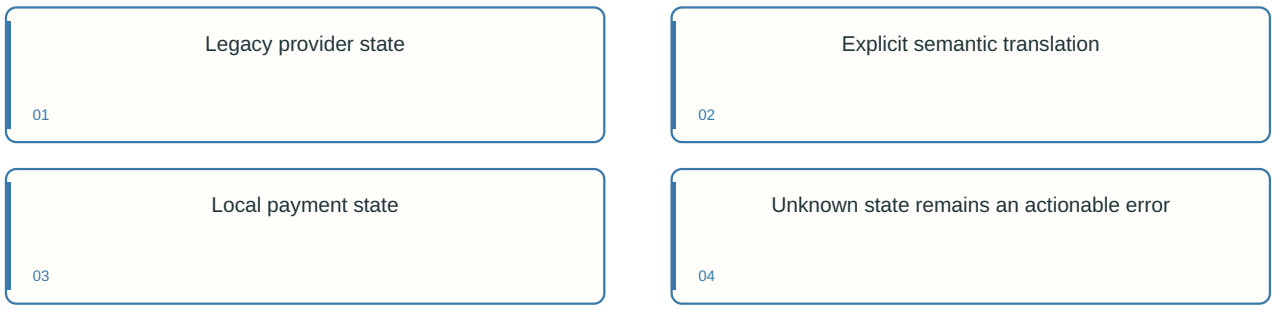


FIG. 7.3.4 **Anti-corruption layers · the example**

An anti-corruption layer translates external meanings so they do not silently redefine the local model.

Author-created illustration. Numbers are illustrative unless stated otherwise.

7.3.5 · Event-driven architecture

An accepted order can trigger a receipt without making checkout wait for email delivery. Event-driven architecture allows components to react independently to recorded events. This can remove a synchronous dependency from the user's path.

Decide which steps may actually be delayed. Inventory and payment requirements may require stronger coordination than receipts. Event processing also adds delivery, ordering, replay, and monitoring work. Name the owner of each event and state how it is persisted, recognised on redelivery, and recovered after a failure. A broker transports messages; the application must still arrange for the required business outcome.

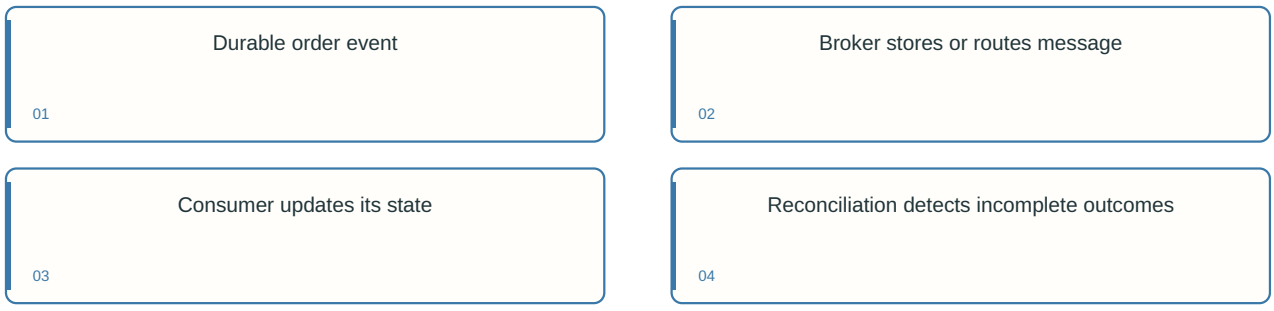


FIG. 7.3.5 **Event-driven architecture · the example**

Event-driven components react independently, with delivery, ordering, replay, and progress obligations.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · What evidence would justify extracting payments into a separate service?

Look for stable capability ownership, different scaling or isolation needs, and an operating team prepared for network and version failures. Compare those benefits with added data coordination and recovery. Preserve existing payment invariants and histories through migration. A separate deployment is useful only when the required independence can actually be exercised.

Remember: Preserve domain meaning while translating transport, storage, and external models at explicit boundaries.

Try applying it: How would an anti-corruption layer handle a provider status whose name differs from your own model?

Unit 7 exercises → · Unit 7 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Architecture styles — Microsoft Architecture Center ↗

   Official guidance · accessed 21 Oct 2026. Style comparisons; provider-specific recommendations are not mandatory in the atlas.

   `Access reviewed · 21 October 2026`

2. Bounded Context — Martin Fowler, 2014 ↗

   Original engineering explanation. Model boundaries and language.

   `Access reviewed · 21 October 2026`

3. Microservices — James Lewis and Martin Fowler, 2014 ↗

   Original engineering article. Architecture characteristics and costs; no exact company architecture is asserted here.

   `Access reviewed · 21 October 2026`

Evolution without a distributed monolith

Prerequisites: 7.3 · Keep business policy inside 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Five services always deploy together and share writable tables. Review whether the process split delivered independence, and use evidence to evolve toward a more manageable design.

7.4.1 · Serverless and function-oriented architectures

A cloud function can process an uploaded file when an event arrives. The platform manages invocation and some execution capacity, but the function still has deadlines, concurrency limits, credentials, and possible retries.

Treat function-local memory as temporary unless the platform guarantees otherwise. Store required progress durably, and recognise repeated invocations before applying repeated effects. Small independently triggered tasks can fit this arrangement well. A long sequence with human review or remote uncertainty needs a durable owner for its state, often a workflow engine. “Serverless” changes operating responsibilities; it leaves application correctness and recovery decisions to be designed.

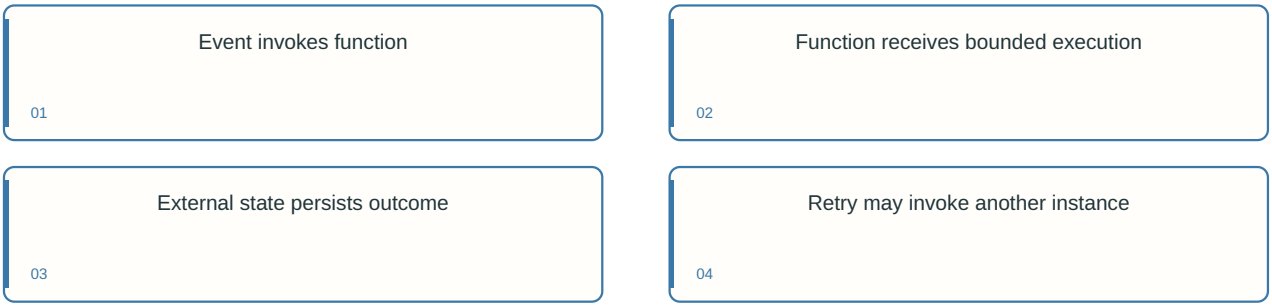


FIG. 7.4.1 **Serverless and function-oriented architectures · the example**

Serverless execution delegates platform work while retaining limits, state, identity, and retry requirements.

Author-created illustration. Numbers are illustrative unless stated otherwise.

7.4.2 · Evolutionary architecture

The application must keep tenant data isolated while releases change its internals. A check that attempts a cross-tenant read can protect that property during development. Similar checks can protect module boundaries, migration compatibility, or a latency budget.

Evolutionary architecture uses such feedback to change a system while preserving important qualities. A fitness check should express a useful property, rather than freeze an incidental class layout. Record why a choice exists and when evidence should prompt reconsideration. Evolution may involve removing an unnecessary service or simplifying a contract. It is a process of guided change, not an obligation to replace the architecture repeatedly.

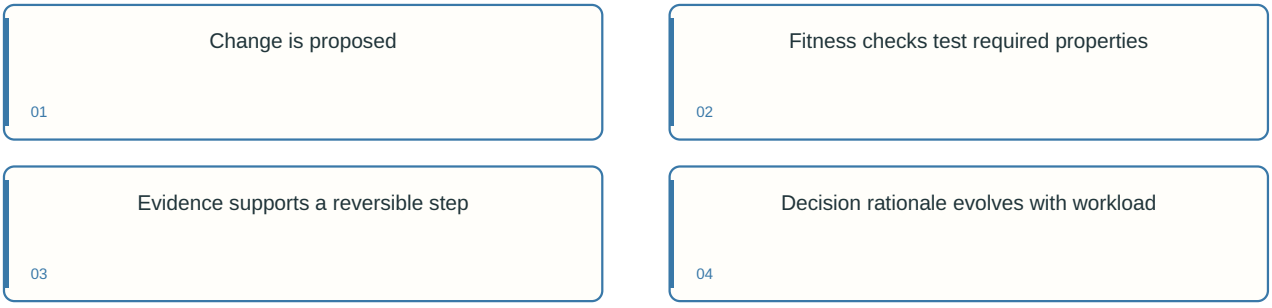


FIG. 7.4.2 **Evolutionary architecture · the example**

Evolutionary architecture changes through evidence while checks protect important required properties.

Author-created illustration. Numbers are illustrative unless stated otherwise.

7.4.3 · Technical vs business boundaries

Controllers and database adapters are technical groupings. Orders and fulfilment are business capabilities. The two kinds of division can coexist: each business module can have its own internal input, policy, and storage code.

Making separate network services for “validation” and “SQL” may force every business operation to call both. Begin with ownership of decisions and facts, then choose the technical arrangement inside those boundaries. A bounded context describes a model's scope. It does not automatically dictate a deployment or team structure. Follow actual business changes to see which parts need to evolve together.

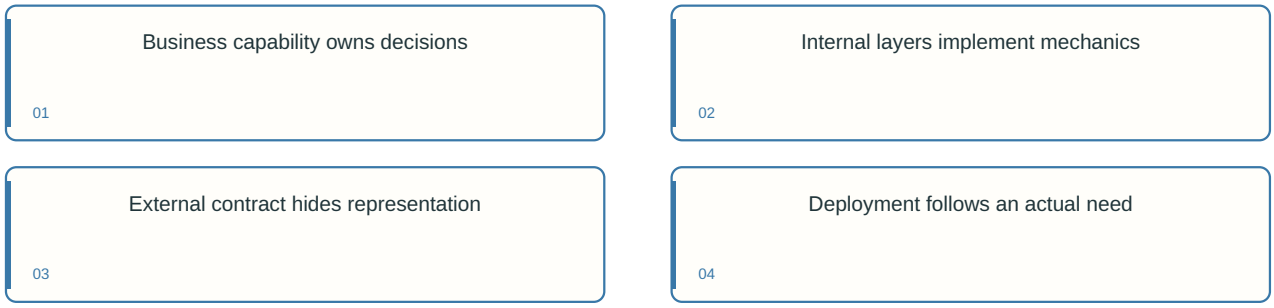


FIG. 7.4.3 **Technical vs business boundaries · the example**

Business boundaries group capabilities; technical layers organise mechanisms within or across them.

Author-created illustration. Numbers are illustrative unless stated otherwise.

7.4.4 · Distributed monolith antipattern

Five services share writable tables and must be released together. Most requests synchronously pass through all five. Although deployed separately, they cannot change or fail independently. This is commonly called a distributed monolith.

Network boundaries now add timeouts and debugging work without delivering the expected independence. Clarify data ownership and contracts before promising independent services. Sometimes consolidating unstable boundaries into a modular application is the easier improvement. In other cases, redesigning the interactions can justify the separation. Judge progress by independent behaviour and manageable failures, rather than the number of deployed processes.

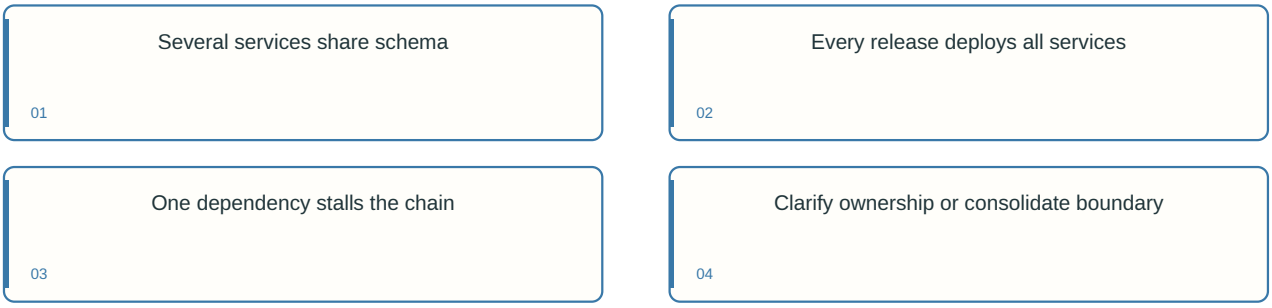


FIG. 7.4.4 **Distributed monolith antipattern · the example**

A distributed monolith has multiple processes without useful independent change or operation.

Author-created illustration. Numbers are illustrative unless stated otherwise.

7.4.5 · When not to use microservices

A small team is still discovering how orders and fulfilment should work. Splitting them into many services immediately would add contract and operating work while the boundaries are changing. A modular monolith can provide a simpler starting point.

Consider extraction when a stable capability has distinct scaling, isolation, or ownership needs and the team can operate it. Include distributed transactions, version compatibility, telemetry, security, and incident response in the cost. Neither low traffic nor a service count settles the choice. Prefer the design that meets the actual requirements and can be changed and recovered by the people responsible for it.

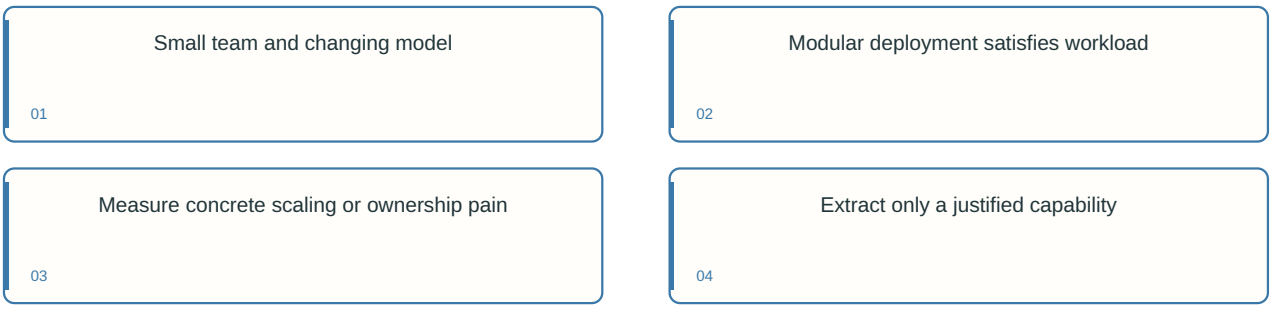


FIG. 7.4.5 **When not to use microservices · the example**

Microservice extraction needs concrete benefits and operating capability sufficient to justify its added coordination cost.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Try the mechanism in code

PSEUDOCODE · MODULE CONTRACT

```
place_order(command):
    priced = catalog.quote(command.items)
    reservation = inventory.reserve(command.items)
    payment = payments.authorize(command.operation_id, priced.total)
    return orders.record_outcomes(priced, reservation, payment)
# Modules expose contracts; they do not update each other's private tables.
```

The modules communicate through their operations rather than writing each other's private tables. Their results let the caller record what actually happened.

Calling three remote services in this order does not create one atomic transaction. Some local changes can share a deliberate transaction, while external effects need a workflow and recovery rules.

Download the executable mechanism collection ↓ · Follow the connected complete design →

▼ Think further · What evidence would justify extracting payments into a separate service?

Look for stable capability ownership, different scaling or isolation needs, and an operating team prepared for network and version failures. Compare those benefits with added data coordination and recovery. Preserve existing payment invariants and histories through migration. A separate deployment is useful only when the required independence can actually be exercised.

Remember: Change architecture for demonstrated ownership, load, or failure needs while protecting existing properties.

Try applying it: When might consolidating services make the system easier to change and recover?

Unit 7 exercises → · Unit 7 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Architecture styles — Microsoft Architecture Center ↗

   Official guidance · accessed 21 Oct 2026. Style comparisons; provider-specific recommendations are not mandatory in the atlas.

   Access reviewed · 21 October 2026

2. Bounded Context — Martin Fowler, 2014 ↗

   Original engineering explanation. Model boundaries and language.

   Access reviewed · 21 October 2026

3. Microservices — James Lewis and Martin Fowler, 2014 ↗

   Original engineering article. Architecture characteristics and costs; no exact company architecture is asserted here.

   Access reviewed · 21 October 2026

PART 2 / UNIT 08

API Design and Communication Contracts

A client asks to create a payment. It needs a clear resource identity, valid amount representation, possible states, and bounded ways to read results. Build the contract before selecting request syntax.

The running example

Design a payment API with create-payment, payment-status, refunds, idempotency protection, duplicate requests, and failed network responses.

Unit 8 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

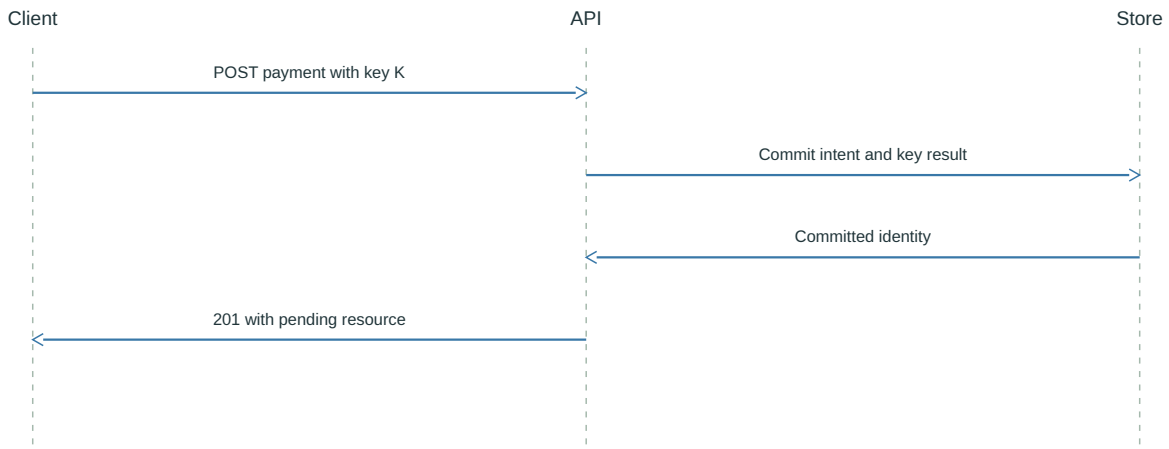
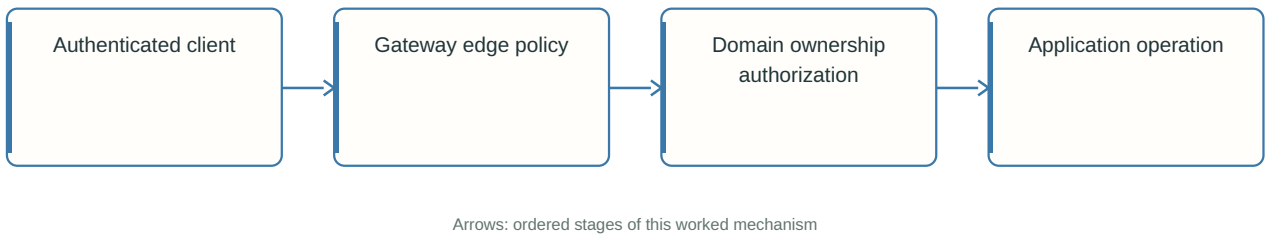


FIG. 8.0.1 **REST request flow**

HTTP creation and financial completion are different outcomes.

Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 8.0.2 **API gateway topology**

An edge gateway cannot decide every resource-specific permission without the appropriate domain context.

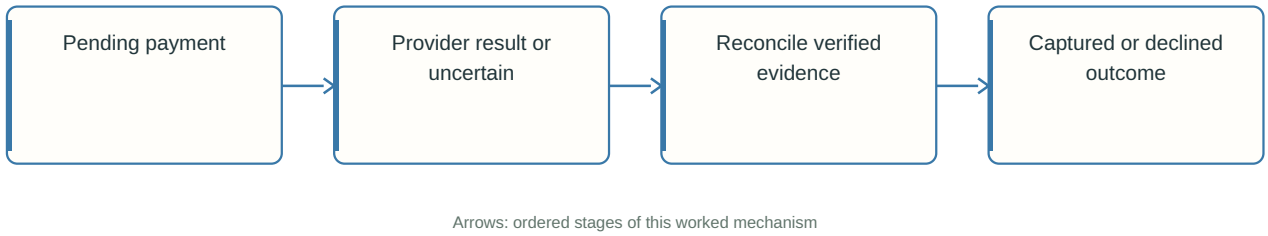
Author-created illustration. Numbers are illustrative unless stated otherwise.

Contract	Caller observes	State owner
Synchronous completion	Waits for final result	Operation still may be uncertain
Asynchronous acceptance	Receives operation ID	Durable workflow
Status query	Reads current progress	Authoritative or stated-freshness view

FIG. 8.0.3 **synchronous vs asynchronous API**

An accepted operation needs durable ownership beyond the request lifetime.

Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 8.0.4 **payment state-machine diagram**

Uncertain is not synonymous with declined; refunds form a separately tracked lifecycle.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Model resources and messages

Prerequisites: Introduction to System Design, Networking Foundations for System Design 6 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A client asks to create a payment. It needs a clear resource identity, valid amount representation, possible states, and bounded ways to read results. Build the contract before selecting request syntax.

8.1.1 · API fundamentals and interface contracts

A client calls create-payment. It needs to know the amount units, currency, permitted caller, returned states, and what to do if no reply arrives. A field schema alone cannot tell it whether retrying might charge twice.

An API is an interface together with a communication contract. Describe inputs, results, effects, errors, timing, operation identity, and supported evolution. Begin with concrete histories: successful creation, invalid input, a repeated request, and a lost reply after submission. These expose decisions a transport framework cannot make for you. Keep a definite provider decline distinct from an operation whose outcome is still uncertain.

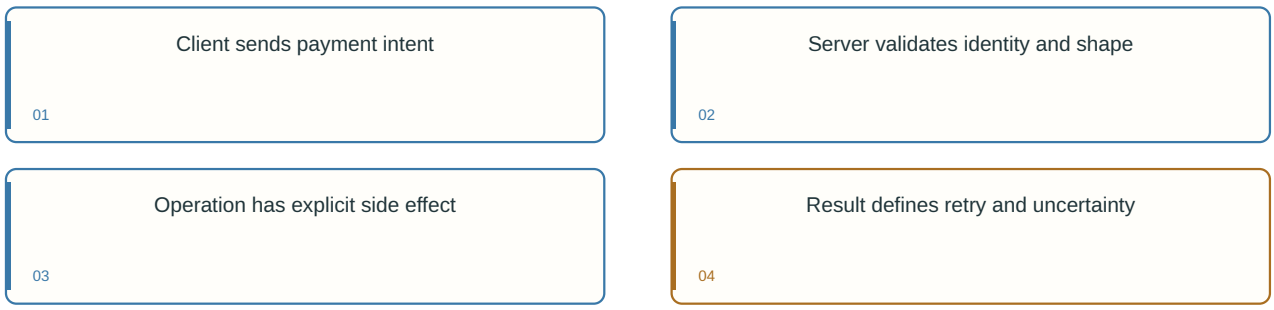


FIG. 8.1.1 **API fundamentals and interface contracts · the example**

An API is a behavioural communication contract, including identity, effects, failures, timing, and supported change.

Author-created illustration. Numbers are illustrative unless stated otherwise.

8.1.2 · REST, GraphQL, gRPC, RPC

The same payment capability can be exposed in several ways. REST emphasises resources and HTTP semantics. GraphQL lets clients query a typed graph. gRPC commonly supplies schema-defined remote calls and streaming. RPC more generally models operations called across a boundary.

Choose by what clients need, how the API will evolve, and the cost of serving it. GraphQL can reduce unwanted data transfer but needs query-cost and field-access controls. An RPC name can make an action clear while leaving retries and compatibility to be specified. Compare complete operation histories and tooling, rather than assuming a request syntax establishes reliability or performance.

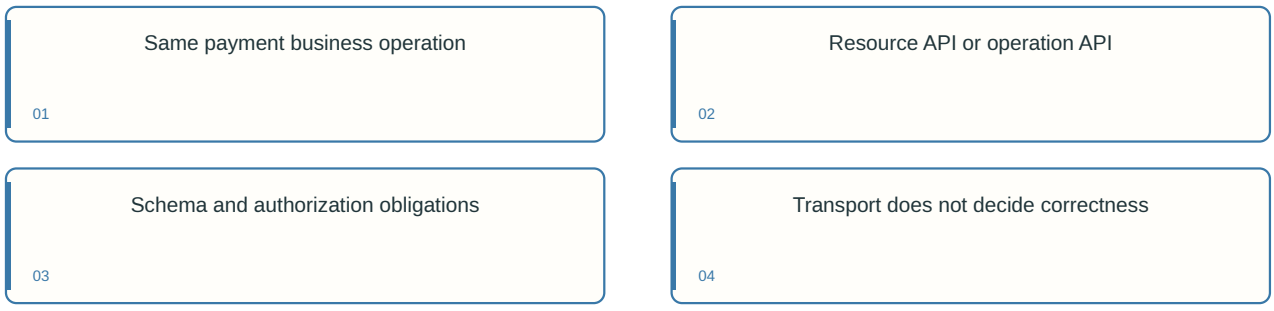


FIG. 8.1.2 **REST, GraphQL, gRPC, RPC · the example**

REST, GraphQL, and RPC styles organise interaction differently; none alone supplies safe retries or compatibility.

Author-created illustration. Numbers are illustrative unless stated otherwise.

8.1.3 · Resource modeling

A payment and its refund are related, but they have different identities and lifecycles. A refund may be requested later, authorised separately, and succeed or fail independently.

One resource model uses POST /payments, GET /payments/{id}, and POST /payments/{id}/refunds. The refund then has its own amount, status, and operation identifier. Resource modelling gives these continuing business objects clear identities. Avoid a generic /execute endpoint that hides every meaning, while also avoiding a forced CRUD model that makes domain actions unclear. Let ownership and allowed state transitions guide the interface.

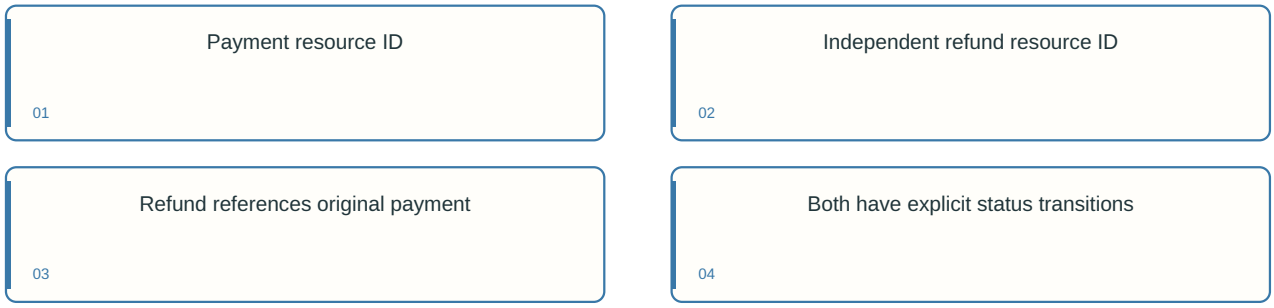


FIG. 8.1.3 **Resource modeling · the example**

Resource modelling gives business objects and operations stable identities and meaningful lifecycles.

Author-created illustration. Numbers are illustrative unless stated otherwise.

8.1.4 · Request/response schemas

A request can contain valid JSON yet ask to charge a negative amount. Parsing establishes structural validity; application validation checks business meaning. Check both.

Bound body size, collection length, and supported encodings before expensive work. Decide how unknown fields are handled so clients can evolve predictably. Return a stable machine-readable error code with a useful, safe message and a request identifier. Avoid exposing secrets or raw provider internals. Consider strict clients as well as tolerant ones: a new field can affect a client that rejects unknown input. The schema and its compatibility policy belong together.

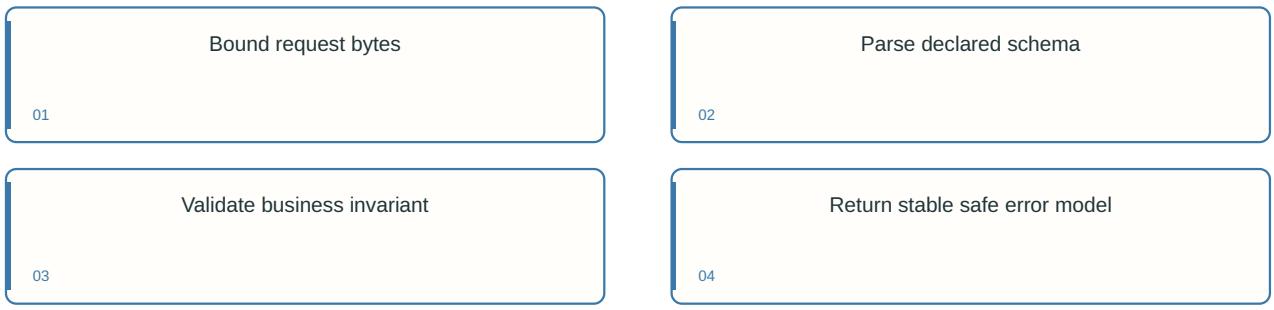


FIG. 8.1.4 **Request/response schemas · the example**

Schemas describe structure; validation also checks business meaning, size limits, and safe error behaviour.

Author-created illustration. Numbers are illustrative unless stated otherwise.

8.1.5 · Status codes and error models

The server returns 201 after creating a payment resource whose status is pending. The resource exists; money has not necessarily been captured. Similarly, 202 indicates acceptance for processing rather than final approval.

HTTP status describes the protocol response, while the body can describe the business operation's state. A timeout supplies neither a definitive approval nor a decline. Use application error codes to distinguish invalid input, conflicts, denied access, and uncertain submitted work. This lets clients choose an appropriate next action. Turning every provider problem into the same generic error can hide whether the caller should correct input, wait, or query an existing operation.

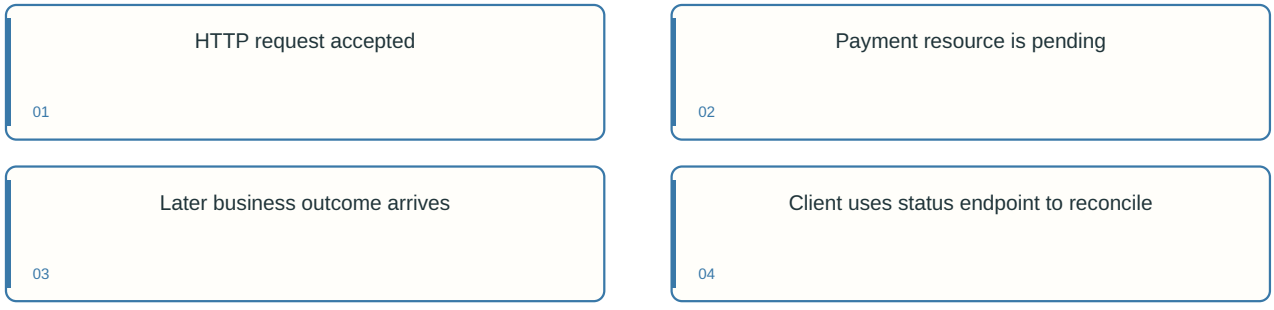


FIG. 8.1.5 **Status codes and error models · the example**

Protocol status and business state can differ: a created resource may still represent pending work.

Author-created illustration. Numbers are illustrative unless stated otherwise.

8.1.6 · Pagination, filtering, sorting

A request for every payment can return too much data and consume too much memory. Pagination divides the collection into bounded responses. Define maximum page size, permitted filters, ordering, and access checks.

Sorting only by creation time can leave rows with equal timestamps in an unstable order. Add a unique tie-breaker such as the payment identifier. Decide what happens while the collection changes between page requests. Neither offset nor cursor pagination automatically gives the client one frozen snapshot. Enforce tenant access in the query itself, rather than retrieving a large shared collection and hoping the client filters it correctly.

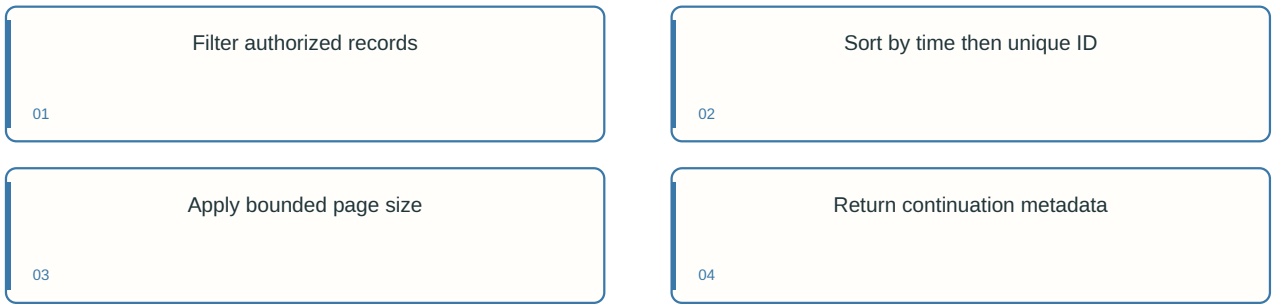


FIG. 8.1.6 **Pagination, filtering, sorting · the example**

Pagination bounds query and response work using defined order, filtering, size, and access rules.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · Can finite storage support a permanent promise to recognise every old idempotency key?

Not for an unbounded set of arbitrary distinct operations unless another constraint supplies a bound or persistent evidence elsewhere. Define key retention, expiry, reuse, and client expectations. A permanent business identity may require a durable record rather than an expiring cache. Make the promise fit the information the system can retain.

Remember: An API describes behaviour and meaning as well as message shape.

Try applying it: Why can creating a payment resource succeed before the provider captures money?

Unit 8 exercises → · Unit 8 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. RFC 9110: HTTP Semantics, June 2022 ↗

   Original standard. HTTP semantics; application commit policy is separate.

   Access reviewed · 21 October 2026

2. OWASP API Security Top 10, 2023 edition ↗

   Project documentation. Threat taxonomy for Unit 18.

   Access reviewed · 21 October 2026

3. RFC 9700: OAuth 2.0 Security Best Current Practice, 2025 ↗

   Original protocol security guidance. OAuth threat and flow guidance; implementation should use maintained protocol libraries.

   Access reviewed · 21 October 2026

Compatibility & idempotency

Prerequisites: 8.1 · Model resources and messages 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A collection changes between pages, and a payment reply disappears before the client receives it. Use stable continuation and operation identities to give repeated calls a defined meaning.

8.2.1 · Cursor vs offset pagination

Suppose a client reads the first ten rows, then asks to skip ten. A new row inserted near the beginning shifts positions, so the second page may repeat or omit an item. This is a difficulty with offset pagination on changing data.

A keyset cursor instead records the last ordered key, such as (created_at,id), and continues after that key. It can reduce skipping cost and these shifting-position problems. Updates to existing rows still need a stated consistency policy. Validate opaque cursors or protect their integrity, and bind them to the applicable filters and tenant. A cursor must not become a way to escape the original query's access rules.

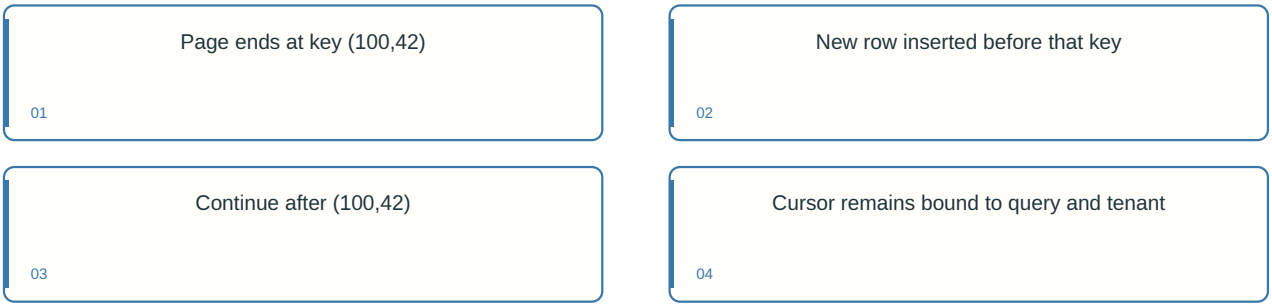


FIG. 8.2.1 **Cursor vs offset pagination · the example**

Keyset cursors continue after ordered keys; offsets skip positions that can shift as data changes.

Author-created illustration. Numbers are illustrative unless stated otherwise.

8.2.2 · API versioning and compatibility

A payment amount changes from cents to dollars while its JSON type remains numeric. Old clients can now submit a hundred times the intended amount. Compatibility concerns meaning as well as syntax.

Adding an optional field is safer for tolerant clients than changing an existing field's meaning, but strict validators and enum handling may still break. Use an explicit version policy, an overlap period, and evidence about old consumers. Test representative clients. A versioned URL is one tool; it does not migrate callers or update stored events. Plan the transition for every reader that still relies on the earlier contract.

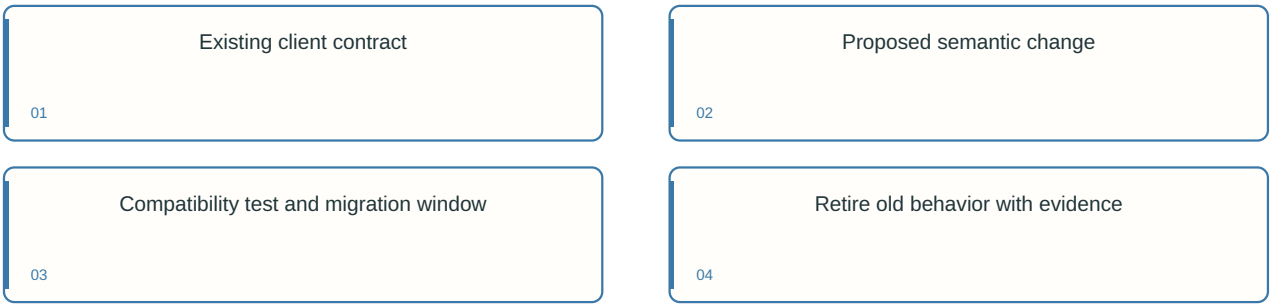


FIG. 8.2.2 **API versioning and compatibility · the example**

Compatibility preserves supported meanings and client behaviour across a planned transition.

Author-created illustration. Numbers are illustrative unless stated otherwise.

8.2.3 · Idempotency and idempotency keys

A client retries a payment after losing its reply. It sends the same idempotency key so the server can recognise the original logical operation. The server must bind that key to the authenticated owner, request content, progress, and result.

Enforce uniqueness and the required atomic updates in storage. If the same key arrives with a different amount, return a conflict instead of replaying an unrelated result. Define how long the key is remembered. A provider timeout may require querying or reconciling that provider before another submission. Local key storage alone cannot promise one external charge if the provider does not support the necessary operation identity and recovery behaviour.

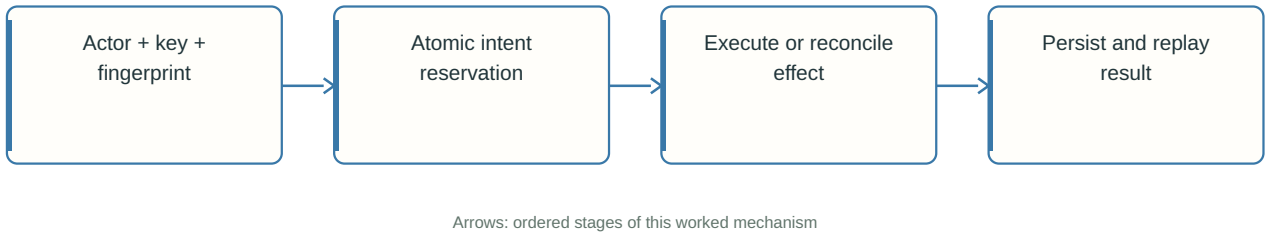


FIG. 8.2.3 **A payment retry follows durable operation identity**

The same key and payload return the established operation; a different payload conflicts. Local identity does not encompass arbitrary provider effects.

Author-created illustration. Numbers are illustrative unless stated otherwise.

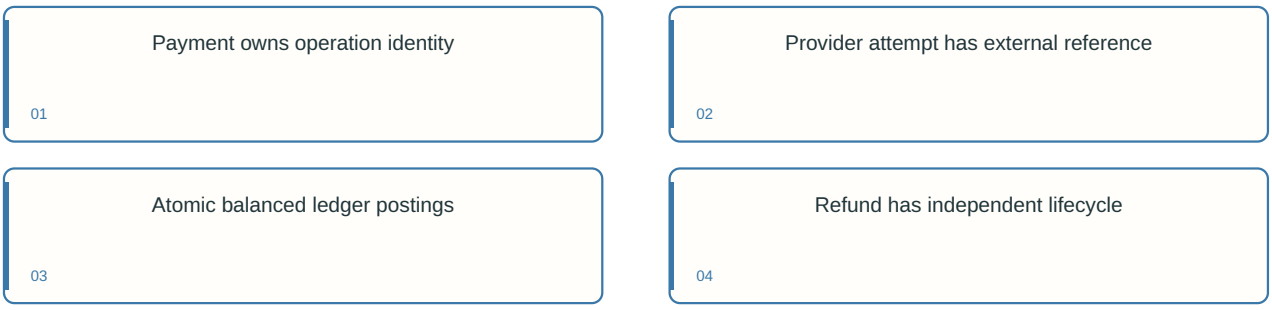


FIG. 8.2.4 **State and ownership in the related design**

Operation identity links provider evidence to payment state and balanced accounting entries.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · Can finite storage support a permanent promise to recognise every old idempotency key?

Not for an unbounded set of arbitrary distinct operations unless another constraint supplies a bound or persistent evidence elsewhere. Define key retention, expiry, reuse, and client expectations. A permanent business identity may require a durable record rather than an expiring cache. Make the promise fit the information the system can retain.

Remember: Compatibility preserves meanings; cursors and idempotency keys need scope, validation, and recovery rules.

Try applying it: What should happen when an old idempotency key is reused with a different amount?

Unit 8 exercises → · Unit 8 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. RFC 9110: HTTP Semantics, June 2022 ↗

   Original standard. HTTP semantics; application commit policy is separate.

   Access reviewed · 21 October 2026

2. OWASP API Security Top 10, 2023 edition ↗

   Project documentation. Threat taxonomy for Unit 18.

   Access reviewed · 21 October 2026

3. RFC 9700: OAuth 2.0 Security Best Current Practice, 2025 ↗

   Original protocol security guidance. OAuth threat and flow guidance; implementation should use maintained protocol libraries.

   Access reviewed · 21 October 2026

Control access, load & long operations

Prerequisites: 8.2 · Compatibility & idempotency 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A signed-in client can still ask for another tenant's payment or overload an export. Combine access, resource limits, deadlines, and durable progress with clear acknowledgment boundaries.

8.3.1 · Authentication vs authorization at API boundaries

A user signs in successfully, then requests payment 42. The server still needs to check whether that user owns or may view payment 42. Knowing an identity and granting an action are separate decisions.

Authentication establishes identity under the chosen trust mechanism. Authorisation checks permission for this action on this resource. Derive tenant context from validated membership rather than trusting an arbitrary tenant_id field. A gateway can perform coarse checks, but the service with ownership information must enforce the resource rule. Apply appropriate identity checks to internal service callers too. An obscure identifier reduces guessing; it is not an access-control mechanism.

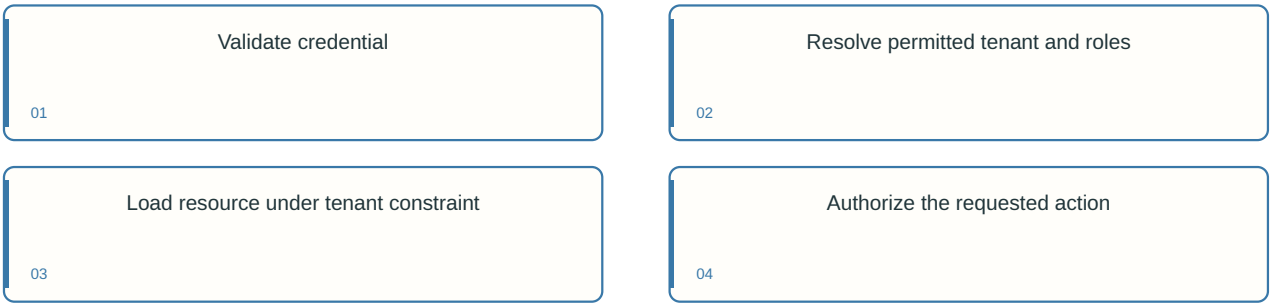


FIG. 8.3.1 **Authentication vs authorization at API boundaries · the example**

Authentication identifies a caller; authorisation checks the permitted action on the particular resource.

Author-created illustration. Numbers are illustrative unless stated otherwise.

8.3.2 · Rate limits and quotas

A client makes too many expensive validation requests at once. A rate limit controls that pace, often with a specified burst allowance. A daily export quota instead caps accumulated use over a longer period or resource dimension.

State the identity being limited, window, cost unit, retry guidance, and behaviour if the limiter fails. Per-tenant controls can protect neighbours from one noisy client. Per-user limits alone may miss coordinated abuse. Across regions, a strict global limit needs shared coordination or safely allocated local budgets. If enforcement is approximate, specify the approximation rather than silently giving every region the full global allowance.

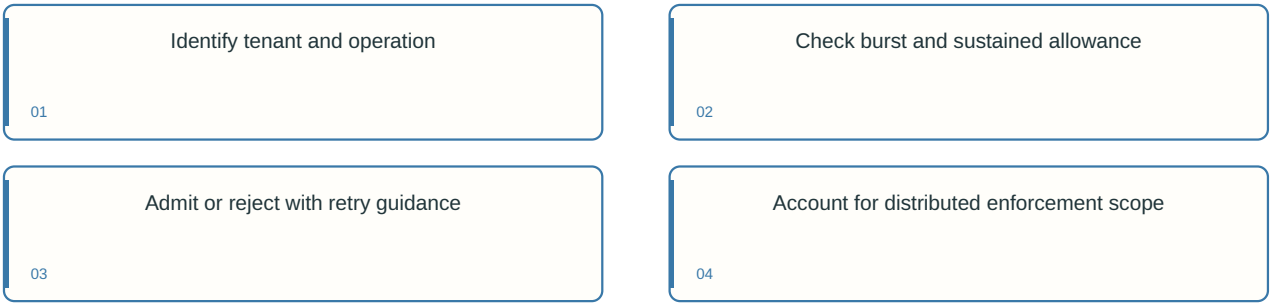


FIG. 8.3.2 **Rate limits and quotas · the example**

Rate limits control pace and bursts; quotas cap accumulated use under defined scope and failure rules.

Author-created illustration. Numbers are illustrative unless stated otherwise.

8.3.3 · Retries and timeouts

A payment request has an overall deadline. Connection acquisition, each attempt, and any backoff must fit inside it. Passing only a fresh full timeout to every downstream call can make the total wait exceed the client's intended limit.

Retry transient work only where repetition is safe. Exponential backoff with jitter spreads attempts but does not add capacity. If four layers each allow three attempts, one intent can cause up to $3^4 = 81$ downstream attempts. Choose a shared retry budget and propagate remaining time. Side effects additionally need a stable operation identity and a way to resolve outcomes that may already have committed.

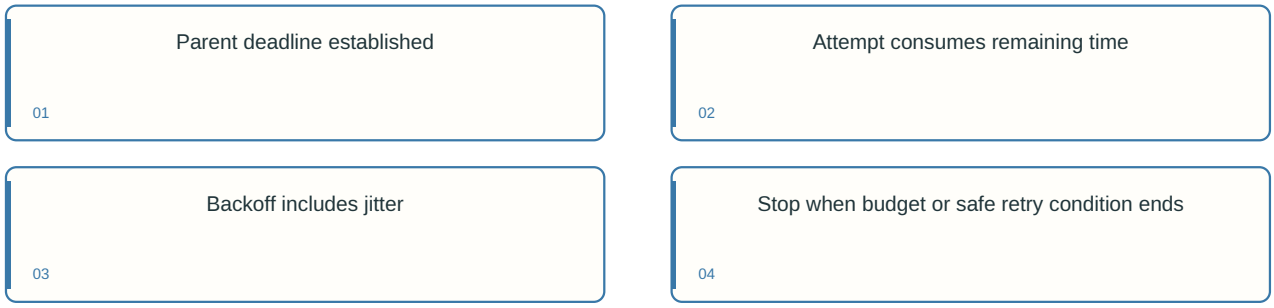


FIG. 8.3.3 **Retries and timeouts · the example**

Retries spend bounded time and capacity and need safe repetition, particularly for uncertain effects.

Author-created illustration. Numbers are illustrative unless stated otherwise.

8.3.4 · Streaming APIs and long-running operations

A report takes minutes to generate. Instead of holding the initial request open, the server can return an operation resource that the client polls or subscribes to. Required progress must be stored beyond the request's memory.

A streaming API returns a sequence of messages and needs framing, buffer limits, cancellation, and possibly a resume policy. Decide whether disconnecting cancels the business job or only the connection. A slow reader should not accidentally hold an unbounded database transaction. Use bounded pages or an explicitly owned streaming resource. The interface must explain how work is tracked after the initiating client has gone away.

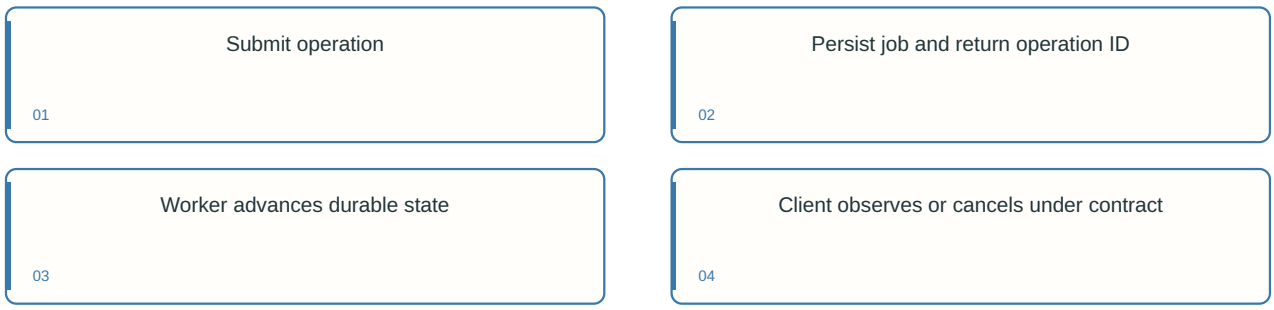


FIG. 8.3.4 **Streaming APIs and long-running operations · the example**

Long-running and streaming APIs need explicit ownership, progress, cancellation, buffering, and recovery contracts.

Author-created illustration. Numbers are illustrative unless stated otherwise.

8.3.5 · Webhooks and delivery acknowledgments

A payment service sends a webhook announcing a completed operation. The receiver commits its local effect, but the success reply is lost. The sender retries, so the same event arrives again.

Give webhooks stable event identifiers and a documented signature and timestamp-verification policy. The receiver should persist the required inbox or effect before acknowledging, then recognise redelivery. The sender needs bounded retries and a recovery policy. A 2xx reply establishes only what the receiver's contract promises; it does not mean every later downstream action is complete. Treat the acknowledgment as a named step in the delivery protocol.

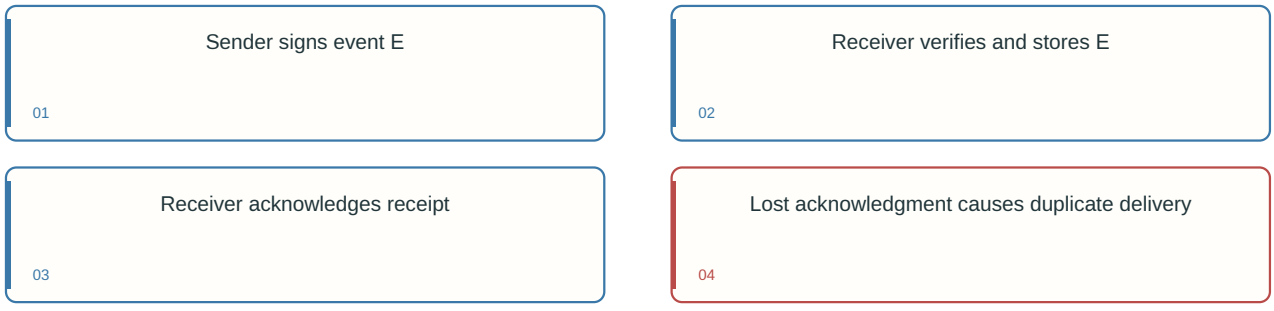


FIG. 8.3.5 **Webhooks and delivery acknowledgments · the example**

Webhooks are asynchronous deliveries requiring authenticity, stable event identity, and defined acknowledgment behaviour.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · Can finite storage support a permanent promise to recognise every old idempotency key?

Not for an unbounded set of arbitrary distinct operations unless another constraint supplies a bound or persistent evidence elsewhere. Define key retention, expiry, reuse, and client expectations. A permanent business identity may require a durable record rather than an expiring cache. Make the promise fit the information the system can retain.

Remember: Admission, permission, and completion answer different questions and need separate enforcement.

Try applying it: What can a webhook receiver acknowledge safely if its effect must survive a crash?

Unit 8 exercises → · Unit 8 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. RFC 9110: HTTP Semantics, June 2022 ↗

   Original standard. HTTP semantics; application commit policy is separate.

   Access reviewed · 21 October 2026

2. OWASP API Security Top 10, 2023 edition ↗

   Project documentation. Threat taxonomy for Unit 18.

   Access reviewed · 21 October 2026

3. RFC 9700: OAuth 2.0 Security Best Current Practice, 2025 ↗

   Original protocol security guidance. OAuth threat and flow guidance; implementation should use maintained protocol libraries.

   Access reviewed · 21 October 2026

Evolve a public contract

Prerequisites: 8.3 · Control access, load & long operations 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

An event saved today may be replayed a year later. Keep schemas, consumer expectations, gateway roles, and documentation usable across that lifetime.

8.4.1 · Schema evolution

A restored consumer replays last year's events. It encounters fields and status values no current request uses. Deleting the old decoder can therefore break recovery even if today's API traffic works.

Schema evolution applies to events and stored records as well as live requests. Preserve field meanings, defaults, version identity, and handling of unknown values. Test historical samples and any transformations used to read them. An extra field can also affect signatures or uniqueness rules, so “additive” does not automatically mean harmless. Plan compatibility for the full life of the data, including replay, backup restoration, and delayed clients.

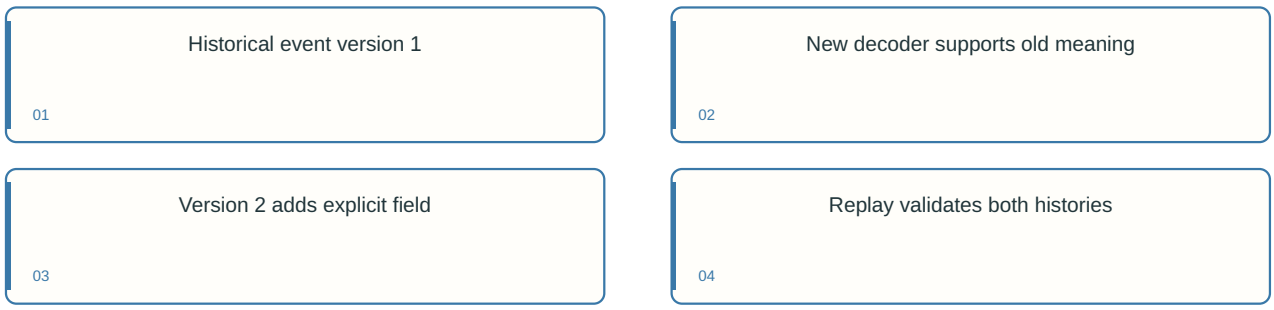


FIG. 8.4.1 **Schema evolution · the example**

Schema evolution must preserve supported interpretation of stored events, historical data, and delayed readers.

Author-created illustration. Numbers are illustrative unless stated otherwise.

8.4.2 · Consumer-driven contracts

A consumer expects a payment status to be either pending or final. A provider test that checks only “status is a string” misses that assumption. Consumer-driven contracts record what actual clients need from their provider.

Run relevant provider checks before releasing a change, including representative errors and state behaviour. Keep the contracts focused on meaningful obligations rather than incidental JSON formatting. Schema tests alone cannot establish retry safety, authorisation, or pagination correctness. Use concrete histories for these behaviours. The contract is useful when it protects a real dependency while still allowing the provider to change internal implementation details.

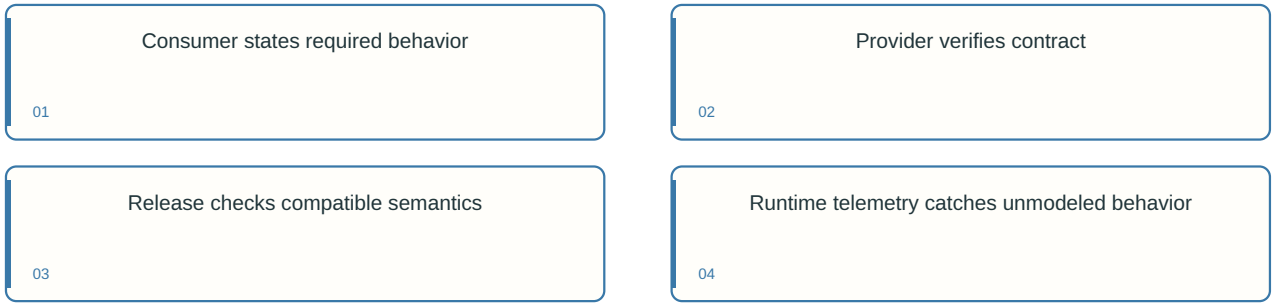


FIG. 8.4.2 **Consumer-driven contracts · the example**

Consumer-driven contracts protect meaningful assumptions of actual clients without freezing incidental formatting.

Author-created illustration. Numbers are illustrative unless stated otherwise.

8.4.3 · API gateways and BFF

A mobile screen needs three small pieces of data from several services. A Backend for Frontend, or BFF, can assemble the response around that client's needs. A gateway usually handles shared edge concerns, such as routing and coarse admission.

Both add operating work and can become bottlenecks. Keep ownership checks in services that know the resource rules, and give the aggregation layer a clear owner. A universal BFF that every product must edit can reduce independence. If results are cached, define the key, tenant scope, and freshness. Aggregating private results does not make them safe for a shared public cache.

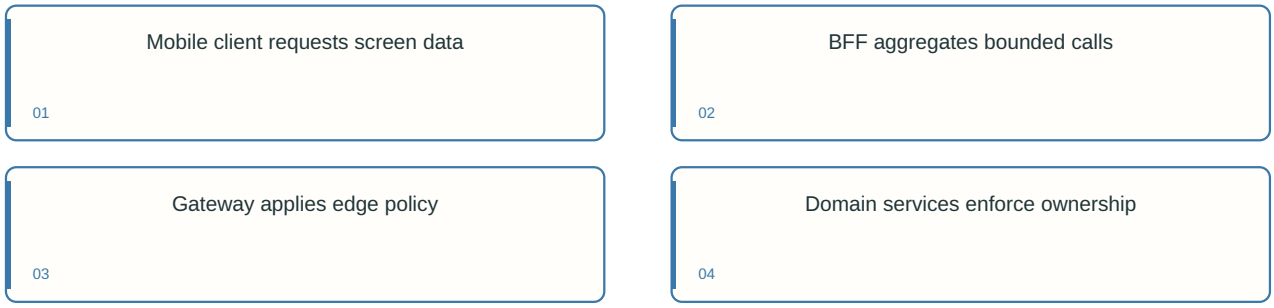


FIG. 8.4.3 **API gateways and BFF · the example**

Gateways provide shared boundary policy; BFFs adapt interactions to particular client needs.

Author-created illustration. Numbers are illustrative unless stated otherwise.

8.4.4 · Public vs internal APIs

Two internal services may release at different times and communicate over a network. Their API therefore still has compatibility, trust, load, and failure concerns. “Internal” does not mean that every request is valid or authorised.

A public API usually needs longer compatibility commitments, discoverability, abuse protection, and support processes. State the intended audience and lifetime of both kinds. Internal interfaces can become important dependencies whose accidental behaviour is difficult to change. Document their meaningful guarantees early, and apply proportionate authentication, validation, and resource limits instead of relying solely on location inside a network.

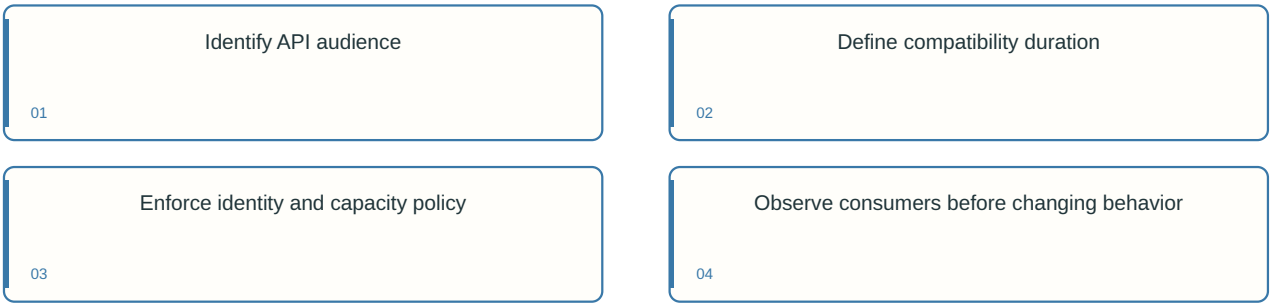


FIG. 8.4.4 **Public vs internal APIs · the example**

Internal APIs still cross trust, version, and failure boundaries; public APIs add explicit audience and support commitments.

Author-created illustration. Numbers are illustrative unless stated otherwise.

8.4.5 · API documentation and discoverability

A developer needs to create a payment and recover if the reply is lost. Documentation that shows only a successful request leaves the most consequential decision unexplained.

Include complete examples for validation errors, access denial, pagination, operation keys, timeouts, and pending states. A machine-readable schema can generate client types, while prose explains the behaviour those types cannot express. Keep examples aligned with the implementation and test the important ones. A sandbox must clearly distinguish simulated effects from real charges. Good discoverability means a reader can find the operation, understand its scope, and choose a supported recovery action when the usual sequence is interrupted.

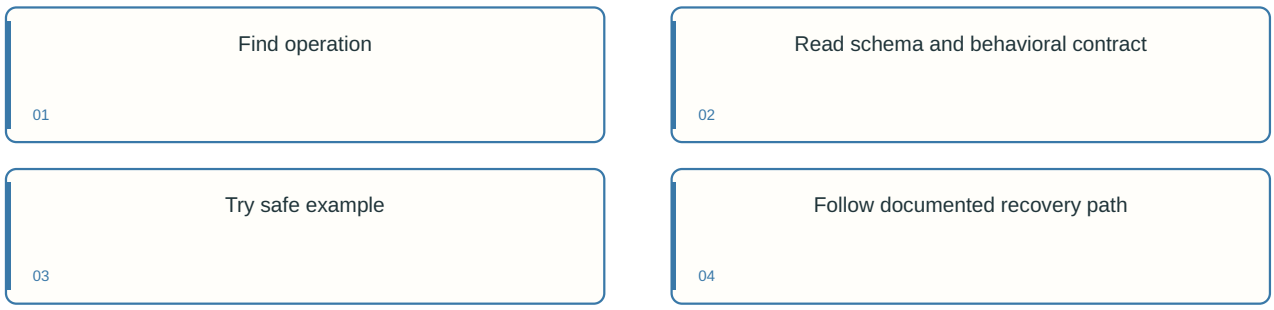


FIG. 8.4.5 **API documentation and discoverability · the example**

Useful API documentation explains complete success and failure histories beyond machine-readable message shape.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Try the mechanism in code

PYTHON · KEYSET CONTINUATION

```
from mechanisms import keyset_page
rows = [(1,1,"A"), (1,2,"B"), (2,3,"C")]
assert keyset_page(rows, after=(1,1)) == rows[1:]
```

The ordered time/ID pair gives equal timestamps a stable tie-breaker. Continuing after (1,1) returns the later rows in this fixed fixture.

A real cursor also needs validation, query and tenant scope, and a rule for changes between pages. Keyset continuation does not itself freeze a changing dataset.

Download the executable mechanism collection ↓ · Follow the connected complete design →

▼ Think further · Can finite storage support a permanent promise to recognise every old idempotency key?

Not for an unbounded set of arbitrary distinct operations unless another constraint supplies a bound or persistent evidence elsewhere. Define key retention, expiry, reuse, and client expectations. A permanent business identity may require a durable record rather than an expiring cache. Make the promise fit the information the system can retain.

Remember: Supported consumers include delayed and recovering readers, not only today's newest client.

Try applying it: Which historical samples would you retain to check that a new decoder still supports recovery?

Unit 8 exercises → · Unit 8 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. RFC 9110: HTTP Semantics, June 2022 ↗

   Original standard. HTTP semantics; application commit policy is separate.

   Access reviewed · 21 October 2026

2. OWASP API Security Top 10, 2023 edition ↗

   Project documentation. Threat taxonomy for Unit 18.

   Access reviewed · 21 October 2026

3. RFC 9700: OAuth 2.0 Security Best Current Practice, 2025 ↗

   Original protocol security guidance. OAuth threat and flow guidance; implementation should use maintained protocol libraries.

   Access reviewed · 21 October 2026

PART 3 / UNIT 09

Data Models and Storage Technologies

A ledger, profile, chat history, and search index do not make the same promises. Compare their reads, changes, and recovery needs before assigning a storage family.

The running example

Choose storage technologies for user profiles, financial transactions, chat messages, event logs, and uploaded PDFs. Justify each decision.

Unit 9 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

Access pattern	Candidate family	Important cost
Relationships and invariants	Relational	Index/write and transaction cost
Nested aggregate	Document	Cross-document rules
Direct key access	Key-value	Secondary query maintenance
Relationship traversal	Graph	Degree and traversal bounds

FIG. 9.0.1 **Database taxonomy**

Families describe useful access patterns; no category label establishes all guarantees.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Layout	Physical grouping	Suitable operation
Row	Fields of one record	Bounded point read
Column	One field across records	Analytical aggregation
Object	Named immutable bytes	Large file transfer

FIG. 9.0.2 **physical storage representations**

Logical data modeling and physical layout are separate decisions.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Fact	Normalized source	Legitimate copy
Current profile address	Customer profile	Cache with freshness policy
Purchase-time address	Order history	Historical fact, not stale profile
Display name	User record	Projection with update policy

FIG. 9.0.3 **normalized vs denormalized schema**

Not all duplication is an anomaly: distinguish historical snapshots from copies meant to track current state.

Author-created illustration. Numbers are illustrative unless stated otherwise.

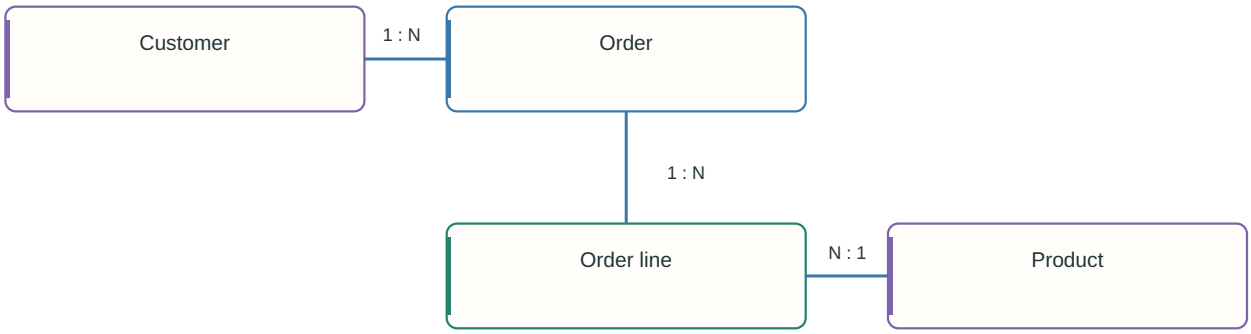


FIG. 9.0.4 **An order schema preserves relationships and history**

Customer has many Orders, Order has many Lines, and each Line references a Product identity while retaining purchase-time values.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Storage promises & database families

Prerequisites: Introduction to System Design 9 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A ledger, profile, chat history, and search index do not make the same promises. Compare their reads, changes, and recovery needs before assigning a storage family.

9.1.1 · What data storage must guarantee

A chat history, a payment ledger, and a thumbnail cache all store data, but losing a copy has different consequences. Before choosing technology, state what is accepted, when it is committed, how callers read it, and which failures it must survive.

Identify the authoritative record and the recovery procedure. A database category does not establish its configured isolation, durability, or backup behaviour. Begin with access patterns and invariants, then compare implementations against them. This prevents a familiar product name from standing in for decisions about the actual service being built.

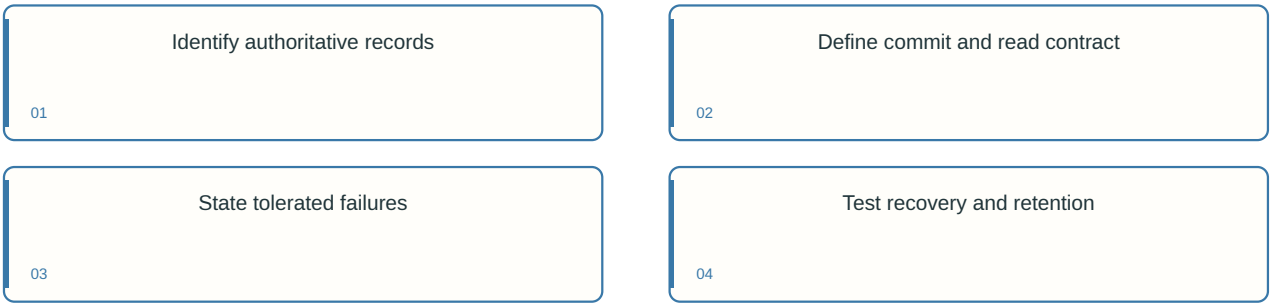


FIG. 9.1.1 **What data storage must guarantee · the example**

Storage guarantees state accepted outcomes, observation rules, ownership, retention, and recovery under specified failures.

Author-created illustration. Numbers are illustrative unless stated otherwise.

9.1.2 · Relational databases and SQL

A ledger can represent accounts and entries as related tables. Keys identify rows, foreign keys protect appropriate relationships, and transactions can group changes. SQL expresses operations on this relational model.

Joins let queries combine related facts. Their cost depends on indexes, row counts, data distribution, and the chosen execution plan; a join is not inherently a scaling mistake. Normalised facts often provide a clear starting point for ownership and consistency. If a measured read path needs a repeated or precomputed representation, add it deliberately and state how it stays correct. The relational model and the physical query cost are related but different questions.

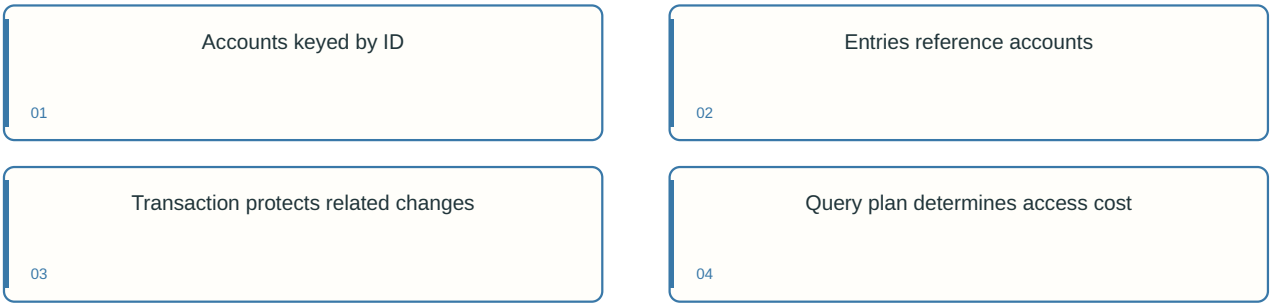


FIG. 9.1.2 **Relational databases and SQL · the example**

Relational storage models facts with keys and constraints; SQL expresses operations whose physical cost needs investigation.

Author-created illustration. Numbers are illustrative unless stated otherwise.

9.1.3 · Document databases

A user profile and its preferences can be stored as one nested document. A document database gives that unit an identity and often allows flexible structures. This can suit data usually read or changed together.

Decide which structure is valid and how it evolves. Flexibility moves some checking into application conventions; it does not remove the schema. Also ask how large the document can grow and whether changes across documents need a shared transaction. Placing every chat message in one ever-growing user document mixes different growth and ownership needs. Design the document boundary around actual operations rather than convenient JSON syntax.

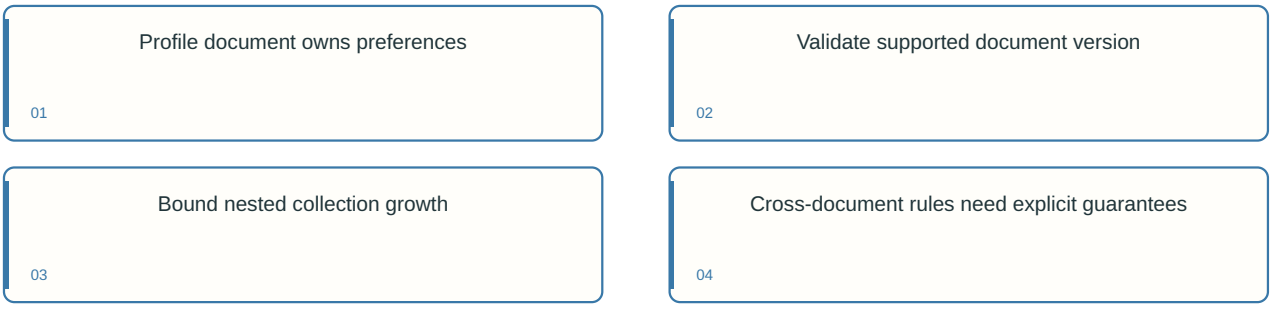


FIG. 9.1.3 **Document databases · the example**

Document storage groups nested data by identity, with explicit size, schema, index, and cross-document rules.

Author-created illustration. Numbers are illustrative unless stated otherwise.

9.1.4 · Key-value databases

A session lookup already knows its identifier. A key-value store can retrieve the corresponding value directly, making this a natural access pattern. A key such as tenant:42:session:7 also makes the intended identity scope visible.

Other queries, such as finding values by an arbitrary field, may need separately maintained structures. Check the selected store's read, write, expiry, and durability contract; key-value products do not all behave alike. A session's expiry and revocation policy must also be explicit. Retaining an old value after permissions change can be a security problem even when the key lookup itself is correct.

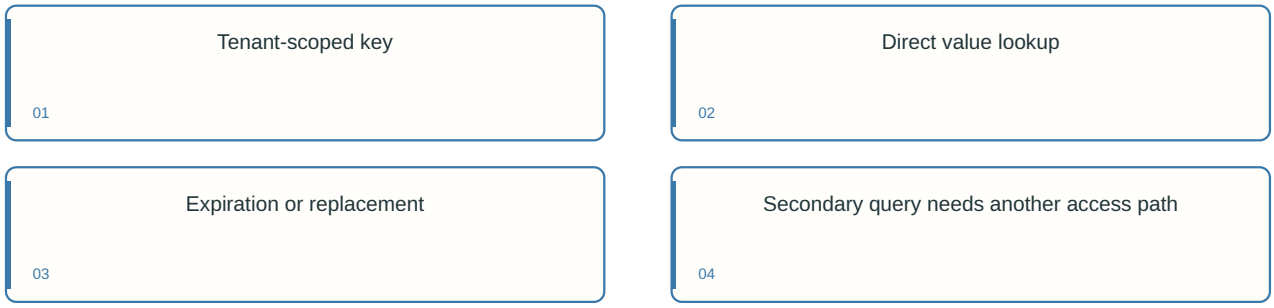


FIG. 9.1.4 **Key-value databases · the example**

Key-value storage serves keyed operations; other queries and expiry need their own supported structures and policy.

Author-created illustration. Numbers are illustrative unless stated otherwise.

9.1.5 · Wide-column stores

A conversation history is often read by conversation identifier and message order. A wide-column design can use the conversation as a partition key and a sequence as an ordering dimension, supporting bounded history reads.

Work through the largest room as well as the typical one. One huge or popular partition can dominate traffic despite many storage nodes. Model the queries first, and duplicate representations for different queries only with a maintenance plan. State which joins or integrity rules the application must enforce. The common storage pattern is useful guidance; exact guarantees still depend on the implementation.

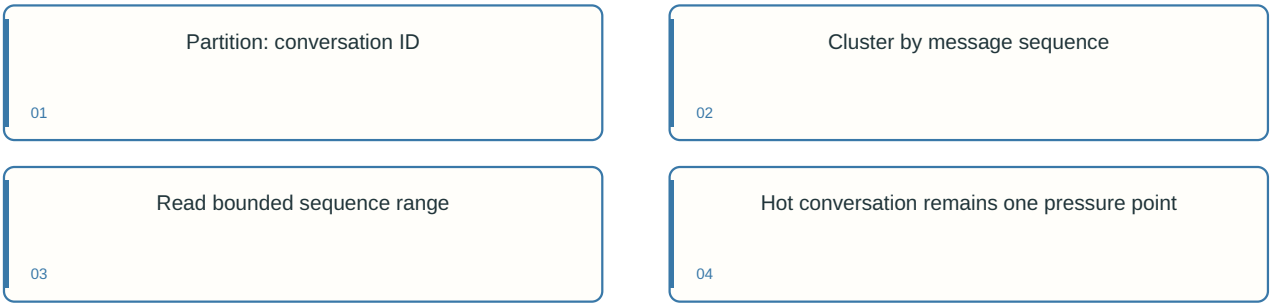


FIG. 9.1.5 **Wide-column stores · the example**

Wide-column designs commonly combine partition keys with ordered dimensions, requiring bounded partitions and query-driven modelling.

Author-created illustration. Numbers are illustrative unless stated otherwise.

9.1.6 · Graph databases

A fraud investigator may follow transfers from an account to devices, other accounts, and further transfers. When variable-depth relationships are central to the query, graph storage can make traversals convenient.

A relational model can also represent edges, so compare actual operations rather than assuming relationships require a graph database. Bound traversal depth and fan-out: one high-degree node can create enormous work. Apply access checks during expansion, not only after collecting results. A traversal that crosses tenant boundaries can expose data even if its starting node was authorised. Query usefulness, cost, and permission rules must be designed together.

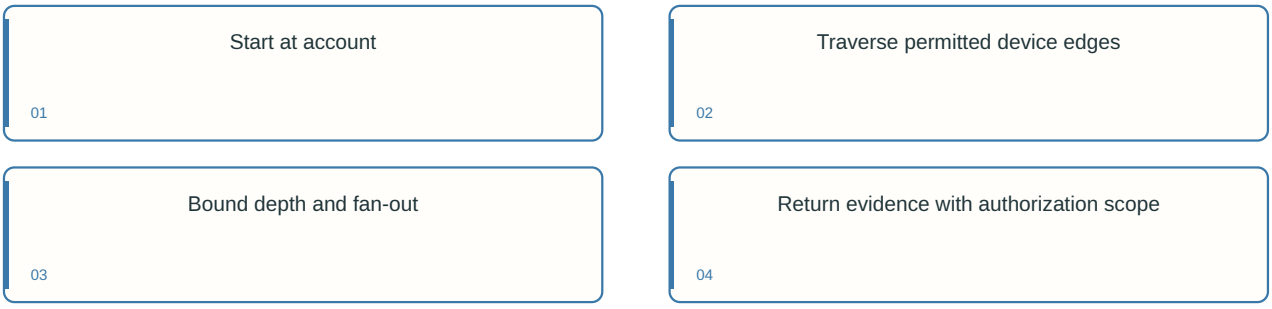


FIG. 9.1.6 **Graph databases · the example**

Graph storage supports relationship traversal whose depth, degree, cost, and access scope need limits.

Author-created illustration. Numbers are illustrative unless stated otherwise.

9.1.7 · Time-series databases

A temperature measurement has a time, value, and dimensions such as device identity. Time-series storage can exploit nearby timestamps, compression, and time-based retention for these measurements.

Choose dimensions carefully. A unique request identifier on every metric can create too many distinct series. Also decide how to handle late readings, clock errors, corrections, and downsampling. A device timestamp records what its clock reported; it does not automatically establish causal order. Preserve ingestion time or a sequence where investigation needs that distinction. Time-based organisation helps when it matches the measurements and queries, rather than simply because a row contains a date.

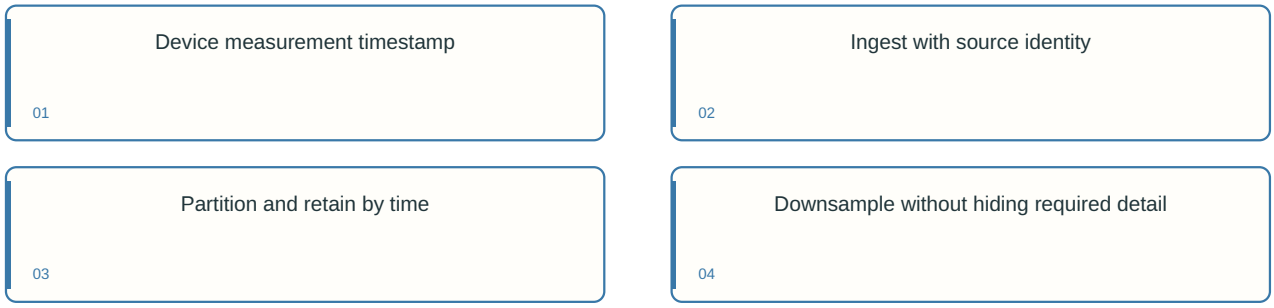


FIG. 9.1.7 **Time-series databases · the example**

Time-series data combines measurements, times, and dimensions with explicit cardinality, lateness, and retention handling.

Author-created illustration. Numbers are illustrative unless stated otherwise.

9.1.8 · Search indexes

A search query needs matching and ranking rather than a transaction over the original record. An inverted index maps terms to document references and supports this retrieval path. It is commonly built from another authoritative store.

If indexing is asynchronous, a new document may appear late and deleted content may remain in the index temporarily. Permission enforcement must remain correct during that lag. Keep source identity and version information so updates can be ordered and the index rebuilt. A search result is a derived serving representation; treating it as the only recoverable copy creates different and usually much harder ownership obligations.

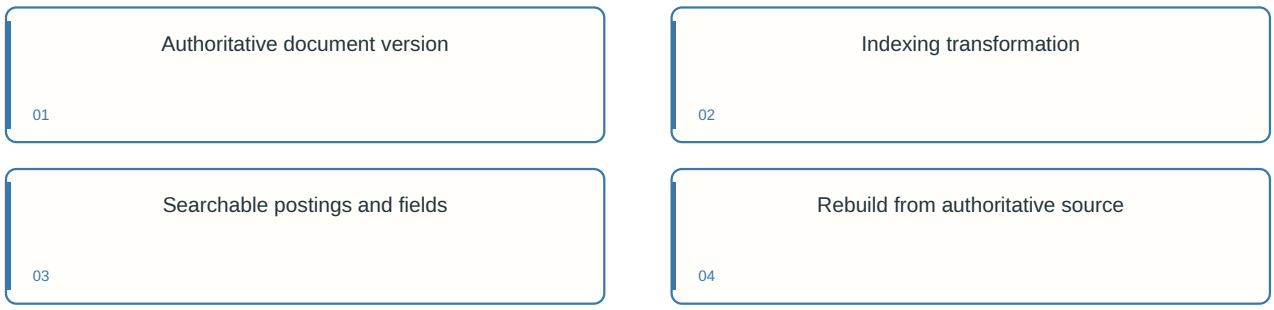


FIG. 9.1.8 **Search indexes · the example**

Search indexes are retrieval representations with source-version, freshness, access, and rebuild obligations.

Author-created illustration. Numbers are illustrative unless stated otherwise.

9.1.9 · Vector indexes: introductory architecture only

An embedding converts content into a numerical vector. A vector index finds nearby vectors, often approximately, to retrieve candidate documents with similar representations. Its architecture must balance recall, memory, updates, distance calculations, and filtering.

Preserve the source document and version that produced each embedding. The index is derived and needs a rebuild path. Approximate search can miss relevant items, and a similarity score establishes neither factual truth nor access permission. Apply the required tenant and resource checks to candidates. Choose this structure for a demonstrated similarity-search need, rather than as a replacement for authoritative records or exact constraints.

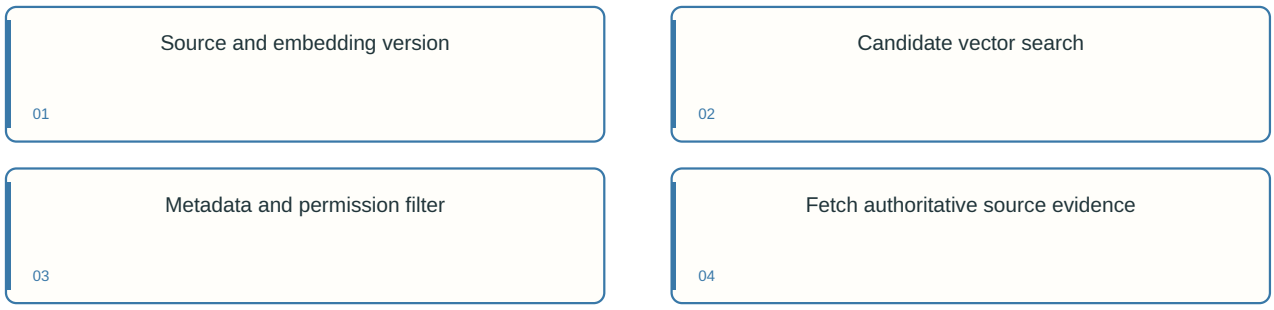


FIG. 9.1.9 **Vector indexes: introductory architecture only · the example**

Vector indexes retrieve similarity candidates from embeddings; scores establish neither truth nor access permission.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · How should restoration from a historical backup respect later deletion decisions?

Restore into a controlled path, apply relevant deletion decisions, and reconcile raw and derived copies before serving. Rebuild indexes from the permitted current state rather than blindly expose an older snapshot. Keep deletion evidence narrow and purpose-limited. The backup and live-data lifecycle need a combined recovery procedure.

Remember: Choose storage from operations and invariants, then verify the selected implementation's guarantees.

Try applying it: What would make a document model suitable for a profile but unsuitable for an unbounded chat history?

Unit 9 exercises → · Unit 9 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Carnegie Mellon Database Group: courses ↗

   University courses. Database internals reading.

   Access reviewed · 21 October 2026

2. PostgreSQL 18: Index Types ↗

   Official versioned documentation. Supported access structures and their purposes.

   Access reviewed · 21 October 2026

Files, objects & analytical layouts

Prerequisites: 9.1 · Storage promises & database families 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A PDF, a checkout row, and an analytical revenue scan read different shapes of data. Separate storage interfaces and physical layouts from the logical facts they represent.

9.2.1 · Object storage vs block vs file storage

Object storage offers named blobs and metadata. Block storage offers addressable blocks. File storage offers a filesystem namespace and operations on files. These interfaces suit different callers.

A PDF upload often fits object storage with separate application metadata. A database engine may require a filesystem backed by suitable block storage. Check the actual protocol and implementation rather than assuming category-wide guarantees. An uploaded object and a database reference are also separate changes: either can exist without the other after a failure. Track incomplete uploads, orphaned objects, and deletion so the application knows when a file is actually usable.

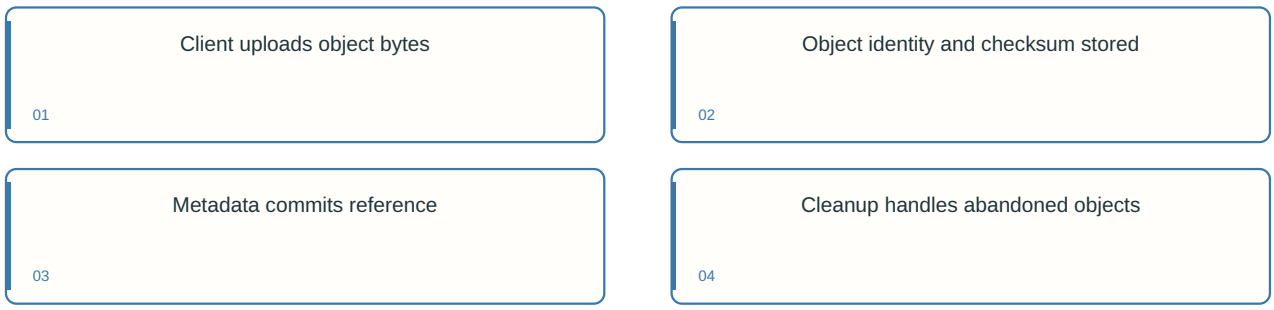


FIG. 9.2.1 **Object storage vs block vs file storage · the example**

Object, block, and file interfaces expose different byte-access and lifecycle contracts.

Author-created illustration. Numbers are illustrative unless stated otherwise.

9.2.2 · OLTP vs OLAP

Checkout performs short transactions over current orders. A revenue report may scan years of payments. OLTP describes the first kind of transactional workload; OLAP describes the second kind of analytical work.

A large report on the checkout database can compete for memory and I/O. Copying or transforming data into an analytical store can reduce that interference, but adds lag, lineage, and reconciliation requirements. One engine may serve both uses adequately at a smaller scale. Measure their interaction before introducing another system, and decide how current analytical results need to be. The labels explain workload differences, not mandatory product assignments.

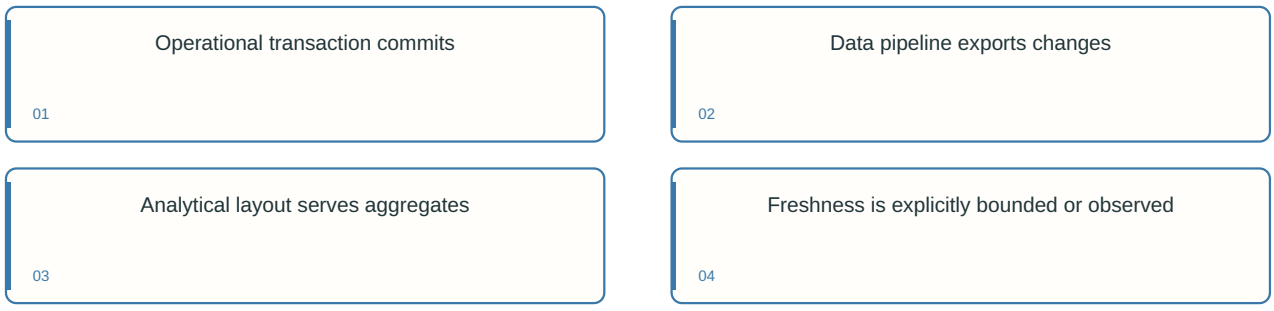


FIG. 9.2.2 **OLTP vs OLAP · the example**

OLTP serves short operational transactions; OLAP serves analytical scans and aggregation, often with different interference costs.

Author-created illustration. Numbers are illustrative unless stated otherwise.

9.2.3 · Row-oriented vs column-oriented storage

To display one customer's full record, it is convenient to read fields stored together in a row. To sum revenue over millions of records, reading only an amount column can avoid unrelated fields and support compression.

Row-oriented storage groups fields by record; column-oriented storage groups values by field. Each changes locality and the cost of reads and updates. Modern engines can combine techniques, so inspect the actual access path. Storage layout does not replace a sound logical model or useful indexes. Choose it according to which bytes the important queries need to read and how frequently those values change.

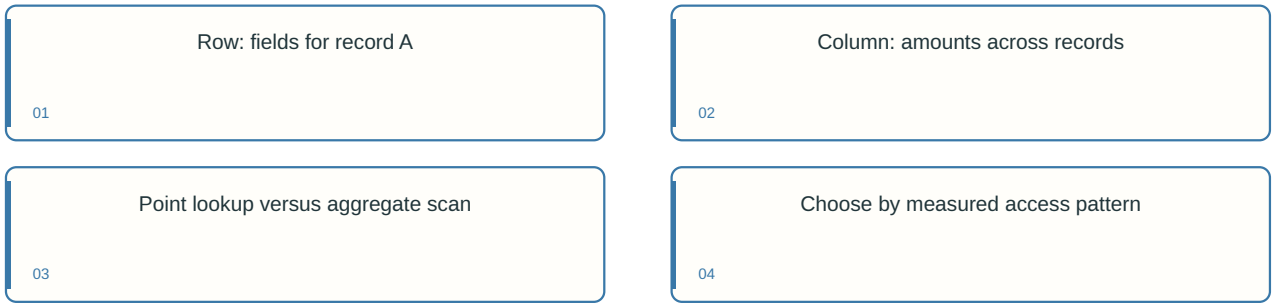


FIG. 9.2.3 **Row-oriented vs column-oriented storage · the example**

Row and column layouts group physical values differently to support different read and update patterns.

Author-created illustration. Numbers are illustrative unless stated otherwise.

9.2.4 · Normalization vs denormalization

A customer's current address belongs in their profile. A completed order may also keep the address used for that shipment. This repetition preserves a historical fact; changing the profile should not rewrite the old order.

Normalisation reduces duplicated facts and update anomalies. Denormalisation deliberately repeats or derives data for a purpose, often a faster read. State which meaning a copy has. A cached author name might be a historical snapshot or a value that should follow future edits. Without that distinction, developers cannot know whether a difference is correct history or stale data. Every maintained copy needs an update and recovery rule.

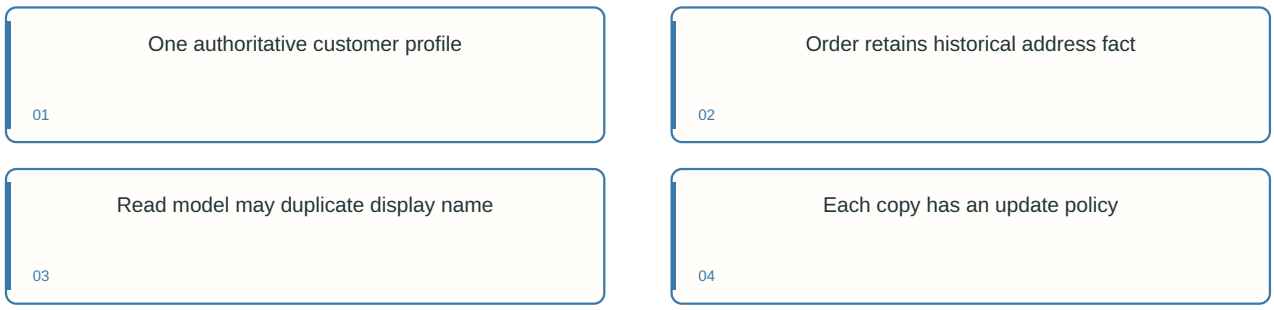


FIG. 9.2.4 **Normalization vs denormalization · the example**

Normalisation reduces repeated facts; deliberate copies require a clear historical or maintained-current meaning.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · How should restoration from a historical backup respect later deletion decisions?

Restore into a controlled path, apply relevant deletion decisions, and reconcile raw and derived copies before serving. Rebuild indexes from the permitted current state rather than blindly expose an older snapshot. Keep deletion evidence narrow and purpose-limited. The backup and live-data lifecycle need a combined recovery procedure.

Remember: Group bytes for the access pattern while preserving the meaning of current and historical facts.

Try applying it: Why should changing a customer's address leave the shipping address of an old order unchanged?

Unit 9 exercises → · Unit 9 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Carnegie Mellon Database Group: courses ↗

   University courses. Database internals reading.

   Access reviewed · 21 October 2026

2. PostgreSQL 18: Index Types ↗

   Official versioned documentation. Supported access structures and their purposes.

   Access reviewed · 21 October 2026

Model relationships and ownership

Prerequisites: 9.2 · Files, objects & analytical layouts 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

An order line refers to an order and a product, but those records have different lifecycles. Model keys, cardinalities, and ownership so data stays meaningful as it changes.

9.3.1 · ER diagrams and relational modeling

An order has several lines. Each line belongs to one order, refers to a product identity, and retains the purchase-time values needed for its history. An entity-relationship model makes those identities and cardinalities explicit.

Show keys, optional and required links, and the constraints that protect them. Also consider lifecycle: deleting a product must not remove the financial meaning of an old order. A diagram is more useful when it states these rules rather than merely joining table names. Translate the relevant constraints into the database or application so they are actually enforced.

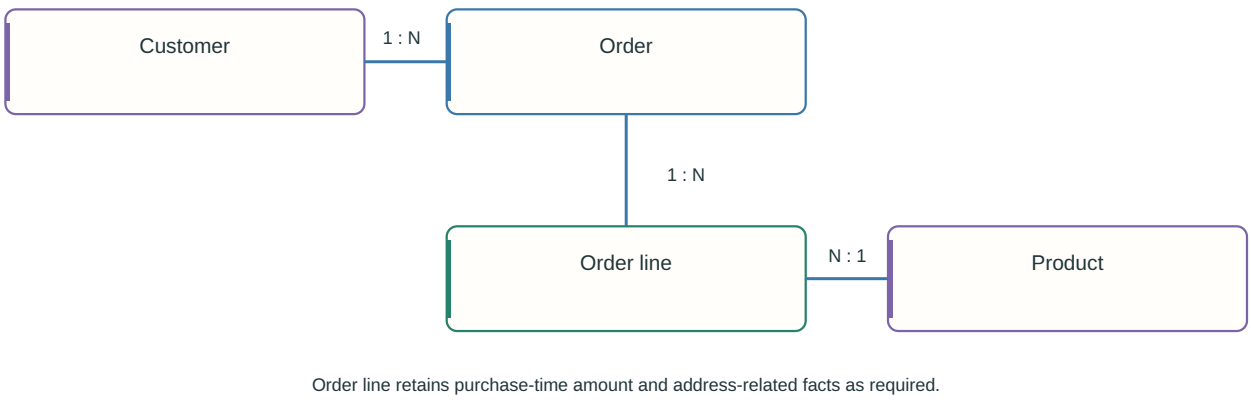


FIG. 9.3.1 **An order schema preserves relationships and history**

Customer has many Orders, Order has many Lines, and each Line references a Product identity while retaining purchase-time values.

Author-created illustration. Numbers are illustrative unless stated otherwise.

9.3.2 · Referential integrity

An order line should not point to an order that does not exist. A suitable foreign key can enforce that relationship in a relational database. This is referential integrity.

Decide deletion behaviour deliberately: automatically deleting dependent records may be wrong for historical payments. An application check before insertion can also race with deletion unless an atomic mechanism protects the relationship. Across separate services, one local foreign key cannot enforce a remote record's lifetime. Use stable identifiers, explicit ownership contracts, and reconciliation appropriate to the business rule. The integrity requirement stays important even when its enforcement crosses different storage boundaries.

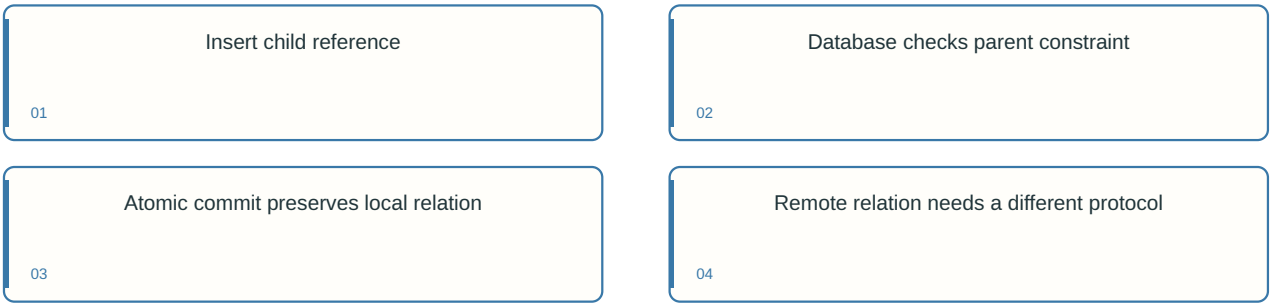


FIG. 9.3.2 **Referential integrity · the example**

Referential integrity protects valid relationships using mechanisms appropriate to the ownership and transaction boundary.

Author-created illustration. Numbers are illustrative unless stated otherwise.

9.3.3 · Data ownership

Payments records whether a charge was accepted. Analytics may keep a copy for reports, but should not rewrite that financial outcome. Data ownership identifies who may define and change each fact.

If several services directly edit one table, they share transaction and release assumptions even if their source repositories are separate. Provide controlled read and change contracts, with only the permissions each consumer needs. Ownership also includes schema changes, incident response, retention, and recovery. Naming a table owner is a start; the design must give that owner a way to preserve its rules while others use the data.

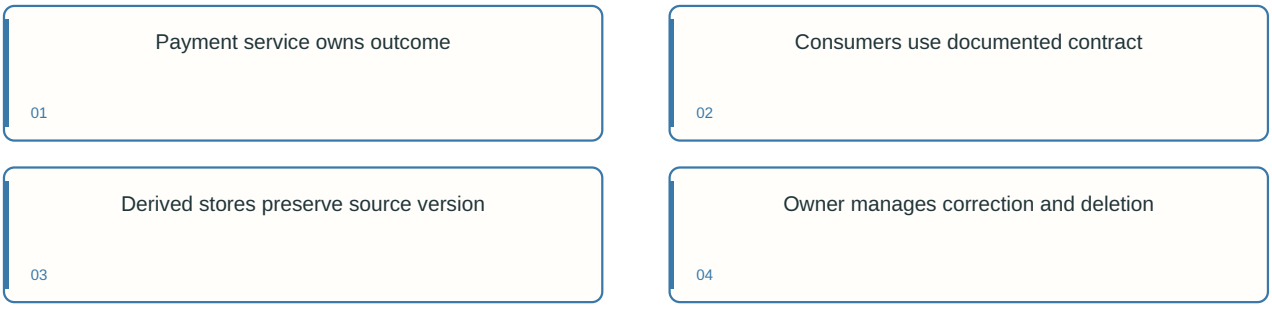


FIG. 9.3.3 **Data ownership · the example**

Data ownership includes who changes a fact and who maintains its access, schema, retention, and recovery.

Author-created illustration. Numbers are illustrative unless stated otherwise.

9.3.4 · Choosing storage based on access patterns

List the operations before assigning an engine. Profiles need particular lookups and edits. Financial entries need transaction and audit rules. Chat history needs bounded conversation/time reads. PDFs need large-byte storage and lifecycle management.

Several of these needs may be met by one system. A relational or document model could both serve a profile, depending on its actual constraints. Add another engine when its capability solves a demonstrated problem. Each additional store introduces backup, permissions, migration, monitoring, and expertise costs. Prefer a storage choice justified by the operations it supports over one chosen merely to include every database category in the diagram.

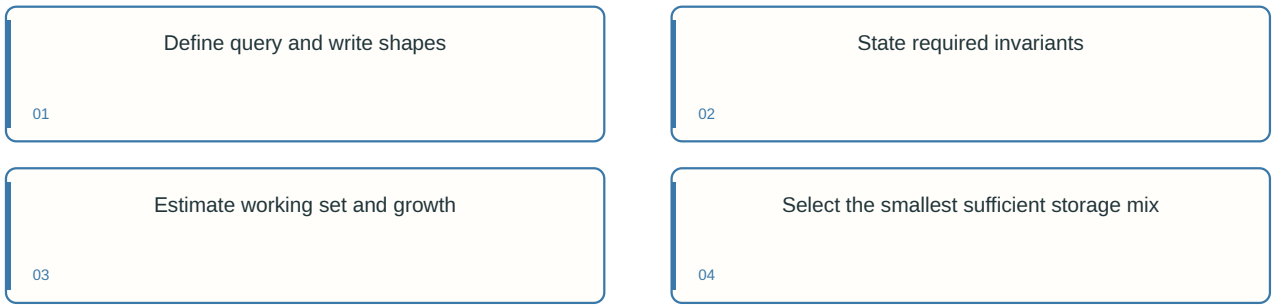


FIG. 9.3.4 **Choosing storage based on access patterns · the example**

Storage selection follows actual queries and invariants, including the complete cost of another engine.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · How should restoration from a historical backup respect later deletion decisions?

Restore into a controlled path, apply relevant deletion decisions, and reconcile raw and derived copies before serving. Rebuild indexes from the permitted current state rather than blindly expose an older snapshot. Keep deletion evidence narrow and purpose-limited. The backup and live-data lifecycle need a combined recovery procedure.

Remember: Relationships need both an expressed rule and a mechanism that preserves it.

Try applying it: What must happen to completed order history when a product is removed from sale?

Unit 9 exercises → · Unit 9 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Carnegie Mellon Database Group: courses ↗

   University courses. Database internals reading.

   Access reviewed · 21 October 2026

2. PostgreSQL 18: Index Types ↗

   Official versioned documentation. Supported access structures and their purposes.

   Access reviewed · 21 October 2026

Access patterns, retention & operational cost

Prerequisites: 9.3 · Model relationships and ownership 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Adding a search engine also adds another recovery and deletion path. Follow raw and derived copies through retention and operation before deciding that another store is worth maintaining.

9.4.1 · Data retention and lifecycle

A document's lifecycle includes its original bytes, extracted fields, indexes, archives, and recoverable copies. Deleting only the live row may leave other copies usable.

Define purpose, retention, expiration, archival, and deletion behaviour from the start. Dropping time partitions can efficiently expire old data; deleting one user's records needs identity-aware propagation. A minimal deletion marker may be needed to prevent old events from recreating removed data, but that marker can itself retain personal information. Keep only what its purpose needs. Test restoration from an old backup so required deletion decisions are reapplied instead of silently reversed.

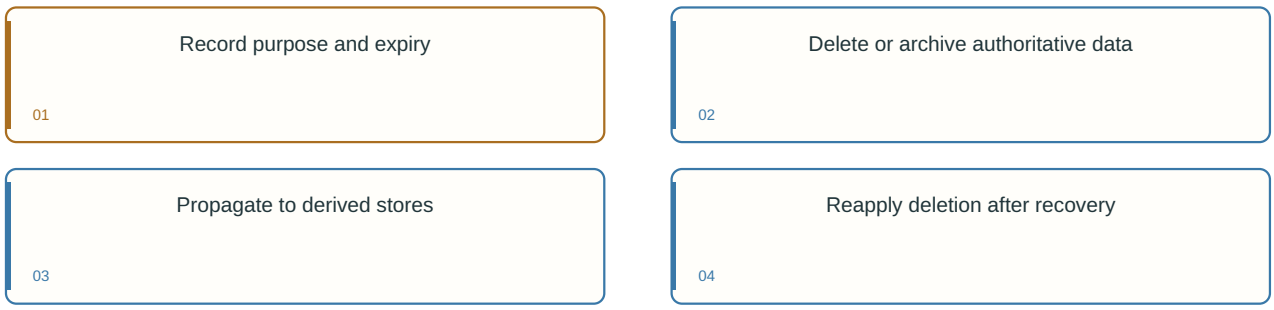


FIG. 9.4.1 **Data retention and lifecycle · the example**

Retention follows raw and derived copies through expiry, archival, deletion, replay, and recovery.

Author-created illustration. Numbers are illustrative unless stated otherwise.

9.4.2 · Polyglot persistence and its operational costs

A shop adds a search engine because its current queries cannot meet a demonstrated relevance need. It now uses different stores for different workloads: polyglot persistence.

The extra capability comes with indexing lag, rebuilds, permissions, schema changes, and another recovery procedure. A change across two stores is not automatically one transaction. Identify which record is authoritative and which representation can be rebuilt, then monitor progress between them. Compare the complete operating cost with the benefit of the new query path. Keeping fewer engines can be an advantage when one already meets the requirements.

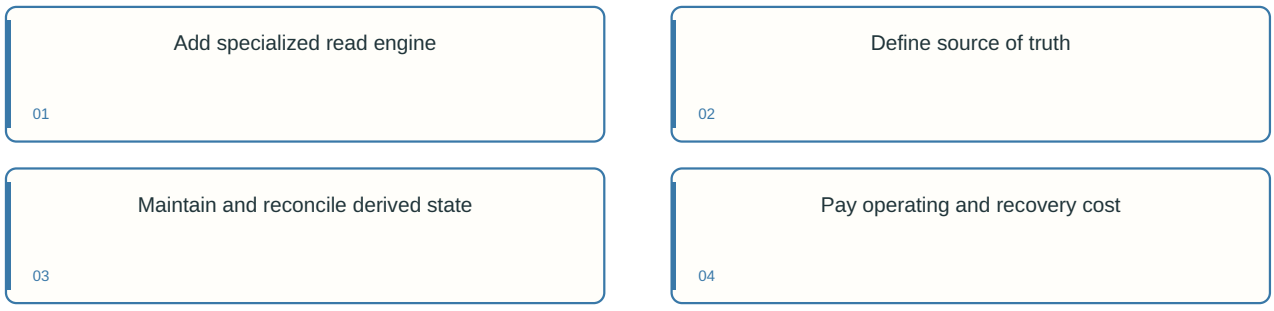


FIG. 9.4.2 **Polyglot persistence and its operational costs · the example**

Polyglot persistence combines engines for useful capabilities while adding operating and cross-store coordination work.

Author-created illustration. Numbers are illustrative unless stated otherwise.

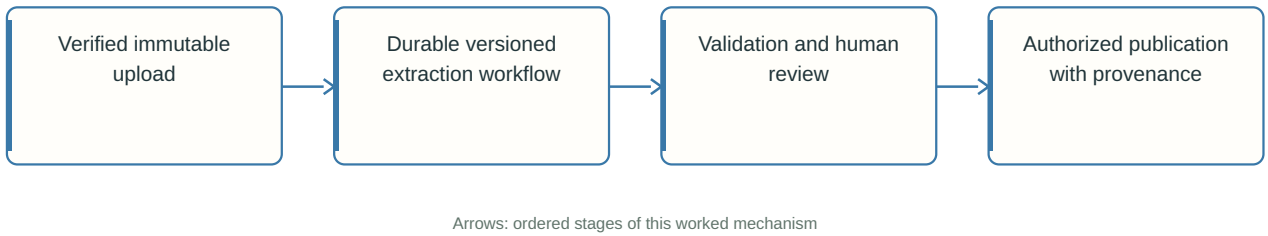


FIG. 9.4.3 **Follow the related complete design**

Ingestion, extraction, review, and publication have distinct durable progress.

Author-created illustration. Numbers are illustrative unless stated otherwise.

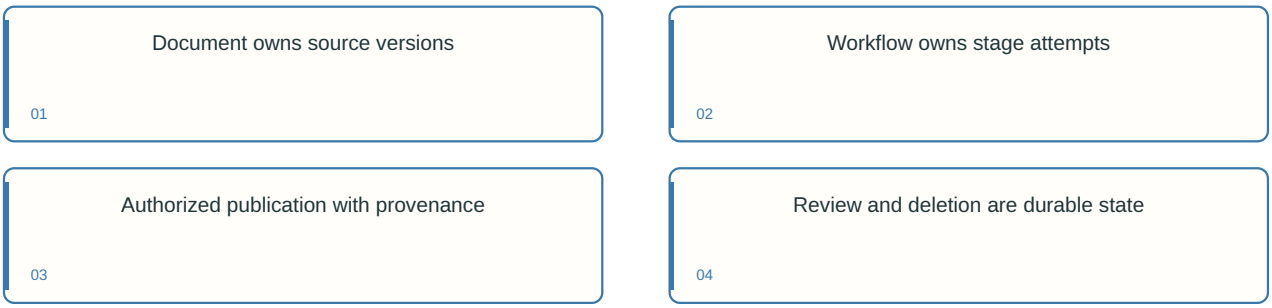


FIG. 9.4.4 **State and ownership in the related design**

Fields and decisions retain tenant, input, transform, and evidence relationships.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Try the mechanism in code

PSEUDOCODE · BYTES AND METADATA

```
upload = create_authorized_upload(tenant, expected_size)
object = store_and_verify_bytes(upload)
with transaction():
    insert_document(tenant, object.version, object.digest)
    insert_processing_intent(object.version)
# Orphan-byte cleanup handles incomplete metadata handoff.
```

The metadata points to a verified immutable object version and records the processing obligation. If metadata commit fails, the object can remain orphaned and needs reconciliation or cleanup.

The local transaction does not include the earlier separate object upload. Track that handoff explicitly instead of describing it as one cross-service commit.

Download the executable mechanism collection ↓ · Follow the connected complete design →

▼ Think further · How should restoration from a historical backup respect later deletion decisions?

Restore into a controlled path, apply relevant deletion decisions, and reconcile raw and derived copies before serving. Rebuild indexes from the permitted current state rather than blindly expose an older snapshot. Keep deletion evidence narrow and purpose-limited. The backup and live-data lifecycle need a combined recovery procedure.

Remember: Additional engines earn their cost through a demonstrated capability and a complete lifecycle.

Try applying it: How could restoring an old backup bring deleted content back into a search index?

Unit 9 exercises → · Unit 9 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Carnegie Mellon Database Group: courses ↗

   University courses. Database internals reading.

   Access reviewed · 21 October 2026

2. PostgreSQL 18: Index Types ↗

   Official versioned documentation. Supported access structures and their purposes.

   Access reviewed · 21 October 2026

PART 3 / UNIT 10

Database Internals, Transactions, and Concurrency Control

A query finds a row through pages, indexes, memory, and storage. Follow that path to understand which physical work an index avoids and which costs remain.

The running example

Two users attempt to purchase the last available ticket. Demonstrate the lost-update race, show why naive reads fail, and implement valid alternatives using transactions and concurrency control.

Unit 10 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

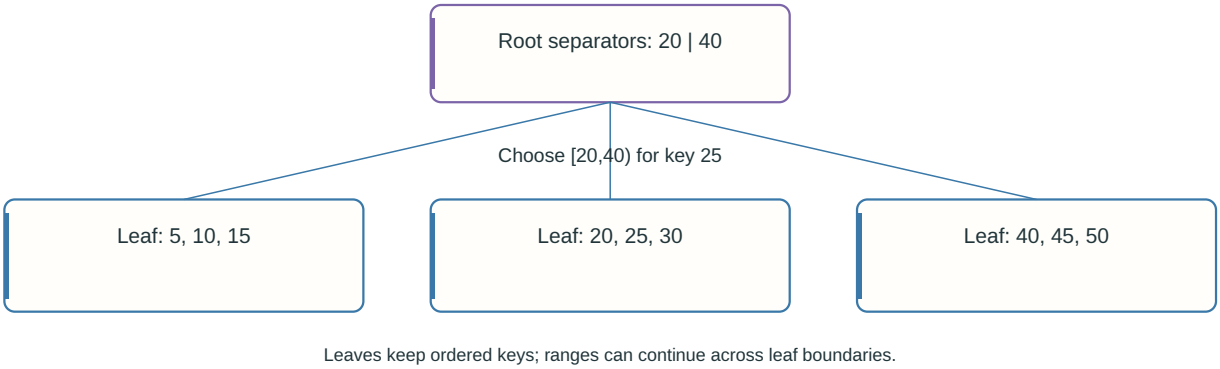


FIG. 10.0.1 **B+tree lookup narrows an ordered key range**

Illustrative keys and leaves demonstrate lookup and range locality. Exact database page layout is implementation-specific.

Author-created illustration. Numbers are illustrative unless stated otherwise.

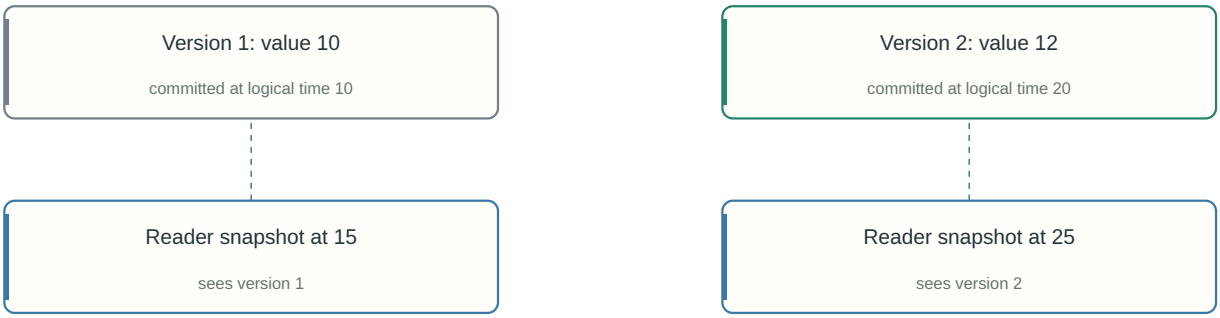
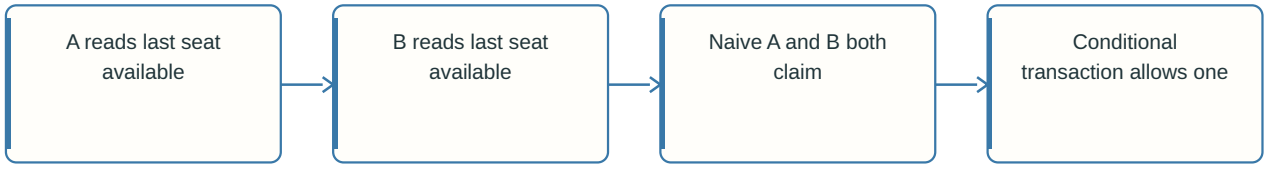


FIG. 10.0.2 **Two snapshots can legitimately see different versions**

A reader whose snapshot precedes version two can keep seeing version one while a later reader sees version two. MVCC alone does not imply serializability.

Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 10.0.3 **concurrent transaction interleavings**

The observable acknowledgment history, not only the final row, reveals overselling.

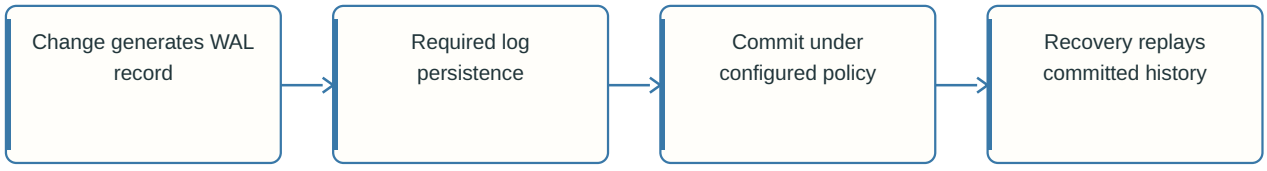
Author-created illustration. Numbers are illustrative unless stated otherwise.

Holder	Owns	Waits for
Transaction A	Row X	Row Y
Transaction B	Row Y	Row X
Result	Circular wait	Abort/retry or consistent ordering

FIG. 10.0.4 **lock wait graph**

A lock-wait graph contains a cycle when each transaction waits on a resource held by the other.

Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 10.0.5 **WAL and recovery**

Data pages can lag the log under a correct recovery protocol; a checkpoint is not a backup.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Pages, indexes & query plans

Prerequisites: Runtime, Processes, Threads, and Concurrency, Data Models and Storage Technologies 6 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A query finds a row through pages, indexes, memory, and storage. Follow that path to understand which physical work an index avoids and which costs remain.

10.1.1 · How databases store and retrieve records

A lookup by task identifier looks simple to its caller. Inside the database, it may traverse an index, fetch pages into memory, check which row version is visible, and return a result. Each step consumes a different resource.

Following this path helps explain why one query waits for disk while another uses CPU or waits for a lock. The operating system and storage device also affect the result. Inspect the query plan and resource behaviour before replacing the database or adding servers. An improvement is easier to justify when it addresses the expensive step rather than an assumed weakness of the database category.

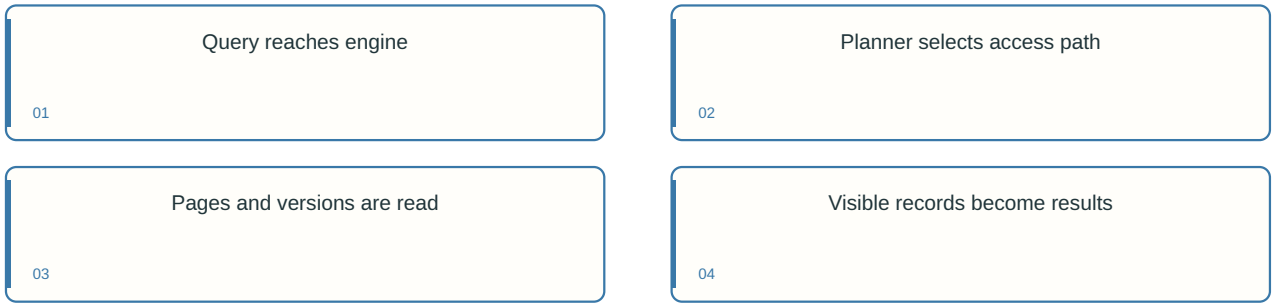


FIG. 10.1.1 **How databases store and retrieve records · the example**

A record lookup performs physical and visibility work through indexes, pages, memory, and storage.

Author-created illustration. Numbers are illustrative unless stated otherwise.

10.1.2 · Pages, records, indexes, and buffer pools

Many database engines organise storage into pages containing records or index entries. A buffer pool keeps useful pages in memory. Finding a page there avoids some device I/O, although execution and synchronisation still take work.

Suppose a query needs a million scattered pages. Extra web servers may simply send more random reads to the same storage. The actively used pages and their locality matter more than row count alone. Inspect what the query touches and how much fits in memory. Page size, layout, and replacement policy differ by engine, so use its actual implementation when explaining detailed behaviour.

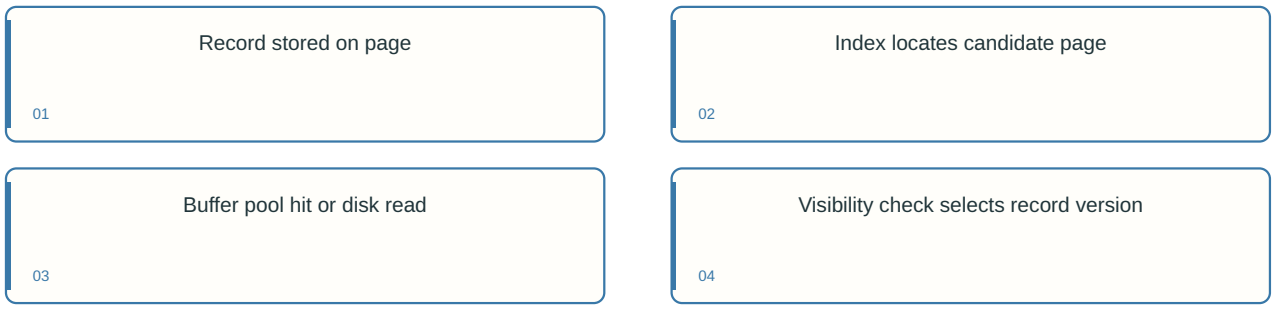


FIG. 10.1.2 **Pages, records, indexes, and buffer pools · the example**

Pages organise records; buffer pools cache pages, while query locality determines which work can be avoided.

Author-created illustration. Numbers are illustrative unless stated otherwise.

10.1.3 · B-tree/B+tree and hash indexing

An ordered index helps find keys 20 through 35 without checking every record. B-tree-family structures keep ordered separators along a bounded-height path. B+tree designs commonly keep records or references at leaves and support continuing a range through neighbouring leaves.

A hash index instead targets equality lookup under its implementation; it does not supply ordered range traversal. Choose by predicates, ordering, engine support, and update cost. Even a short tree path can require page reads, and returning many matching rows still takes time. Algorithmic complexity is useful, but query cost also depends on storage locality and the amount of output.

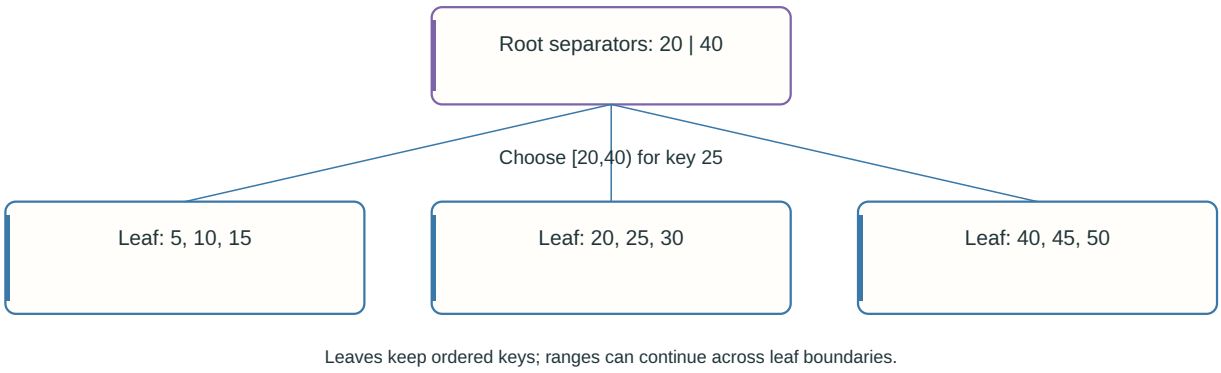


FIG. 10.1.3 **B+tree lookup narrows an ordered key range**

Illustrative keys and leaves demonstrate lookup and range locality. Exact database page layout is implementation-specific.

Author-created illustration. Numbers are illustrative unless stated otherwise.

10.1.4 · LSM trees and write amplification

An LSM-based engine collects writes in memory, flushes sorted runs, and later merges them through compaction. This can turn many small changes into more sequential storage work.

Reads may consult several structures, and compaction writes some data again. Write amplification compares physical bytes written with the logical change size. Its value depends on the layout and workload. Compaction also competes for bandwidth and can cause stalls when it falls behind. Observe sustained ingestion and tail latency, rather than only a short burst that fits in memory. Write-oriented organisation changes costs; it does not remove limits or make reads free.

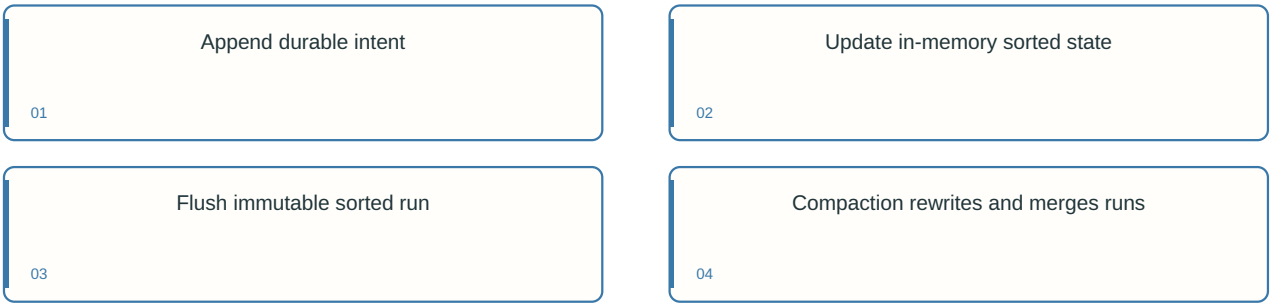


FIG. 10.1.4 **LSM trees and write amplification · the example**

LSM organisation batches sorted writes and compaction, trading ingestion behaviour against read work and amplification.

Author-created illustration. Numbers are illustrative unless stated otherwise.

10.1.5 · Query planning and EXPLAIN

A SQL query describes the desired result. The planner chooses how to obtain it, using statistics and estimated costs. EXPLAIN shows the selected plan; an execution-analysis form can show actual work, but may execute the statement and its side effects.

Suppose the planner expects ten rows and finds a million. A join strategy that looked cheap can become expensive. Compare estimates with actual row counts, filters, sorts, scans, and buffer activity. Statistics or a query rewrite may be the right first correction. Check the selected engine's command behaviour before analysing a statement that changes data.

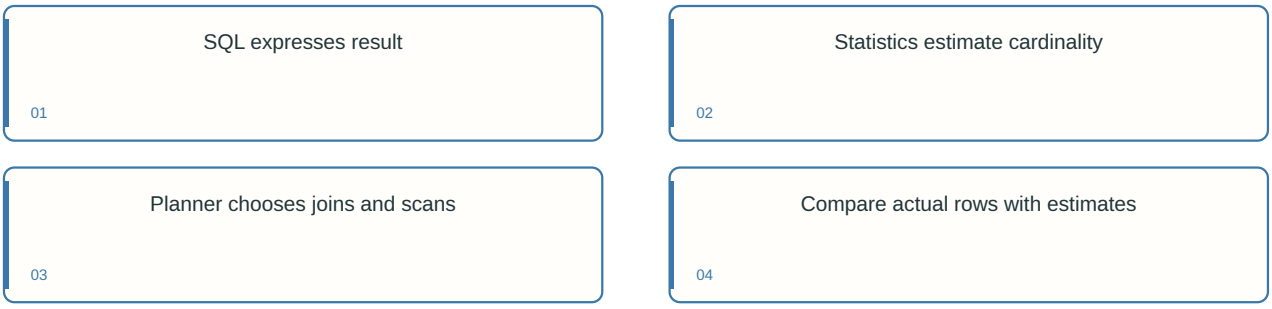


FIG. 10.1.5 **Query planning and EXPLAIN · the example**

A query planner chooses physical execution using estimates; compare plans with actual work where safely supported.

Author-created illustration. Numbers are illustrative unless stated otherwise.

10.1.6 · Index selection and query optimization

A tenant-scoped list query filters by tenant and sorts by time and identifier. An index on (tenant_id,created_at,id) may support that access path, depending on the engine and query.

An index also consumes space and must be updated on writes. Redundant or unused indexes impose that cost without helping the important queries. A larger covering index can avoid some record fetches but reduce memory locality. Test representative values and distributions: a small uniform dataset may conceal a tenant with most of the rows. Select indexes to serve demonstrated operations, then inspect the resulting plan and write cost.

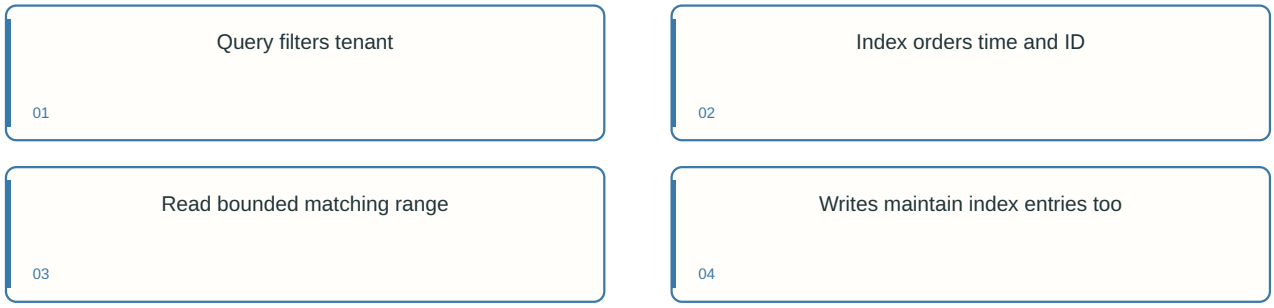


FIG. 10.1.6 **Index selection and query optimization · the example**

Indexes improve selected reads while adding space, maintenance, and write costs.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · What changes when one local transaction is divided across independent databases?

A former all-or-nothing invariant may no longer share one commit boundary. Identify which related changes require agreement, whether a distributed transaction is justified, or whether a workflow with explicit intermediate states and compensation fits. Local success at each store does not guarantee the combined business outcome.

Remember: Inspect the real plan and data distribution before changing indexes or hardware.

Try applying it: Why could an index that helps reads still reduce write throughput?

Unit 10 exercises → · Unit 10 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. PostgreSQL 18: Transaction Isolation ↗

   Official versioned documentation. Specific PostgreSQL isolation behavior; no all-database equivalence claim.

   Access reviewed · 21 October 2026

2. PostgreSQL 18: Index Types ↗

   Official versioned documentation. Supported access structures and their purposes.

   Access reviewed · 21 October 2026

3. PostgreSQL 18: Write-Ahead Logging ↗

   Official versioned documentation. Logging and recovery ordering; durability remains configuration-dependent.

   Access reviewed · 21 October 2026

4. RocksDB Overview ↗

   Official implementation documentation · accessed 21 Oct 2026. An LSM-based engine; exact tuning is implementation-specific.

   Access reviewed · 21 October 2026

Commit, logging & versions

Prerequisites: 10.1 · Pages, indexes & query plans 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A reader sees an earlier row while a writer commits a new version. Connect transaction boundaries, logs, and visibility to explain both recovery and the observations readers may make.

10.2.1 · ACID properties

A transaction transfers an amount between two accounts. Atomicity means its related database changes are accepted together or abandoned together. Isolation controls what concurrent transactions may observe. Durability concerns keeping committed data under the stated configuration and failure assumptions.

Consistency, the C in ACID, concerns preserving the required invariants when application logic and constraints correctly express them. The engine cannot infer every business rule. Atomically transferring the wrong amount still produces the wrong business outcome. ACID separates useful properties so we can ask the right question of each. A transaction's existence alone does not specify its isolation level or validate its calculation.

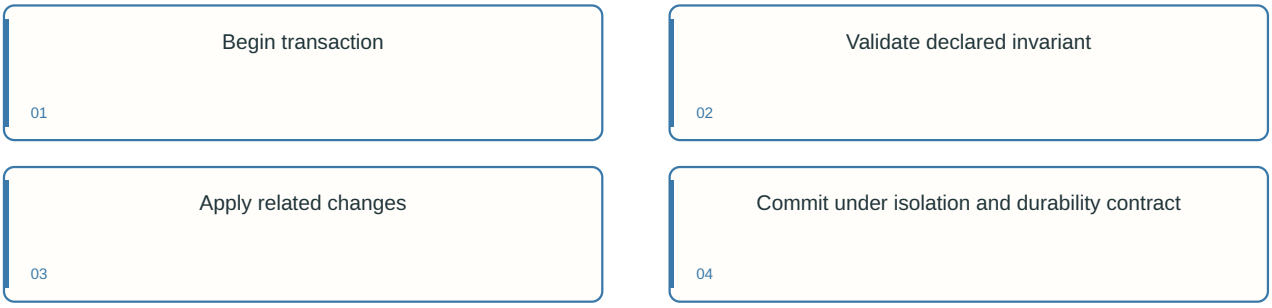


FIG. 10.2.1 **ACID properties · the example**

ACID separates atomicity, invariant preservation, isolation, and conditional durability; application rules still need correct expression.

Author-created illustration. Numbers are illustrative unless stated otherwise.

10.2.2 · Transaction commit and rollback

The application changes rows and asks the database to commit. Only after a successful commit should it report success if that report promises a saved result. A rollback abandons uncommitted transaction work.

Now lose the connection while commit is being decided. The caller may be unsure whether the database accepted the transaction. Recovery needs an operation identity or another way to find the outcome. Also separate local rollback from external actions: rolling back SQL does not reverse a provider charge already made over HTTP. Keep such uncertainty explicit rather than blindly replaying an entire operation containing unprotected remote effects.

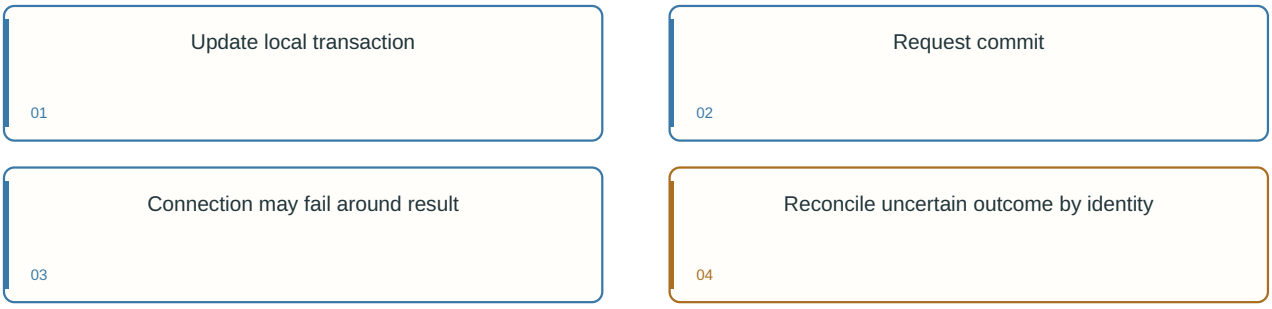


FIG. 10.2.2 **Transaction commit and rollback · the example**

Commit accepts a transaction under the engine protocol; rollback abandons uncommitted work and cannot reverse unrelated remote effects.

Author-created illustration. Numbers are illustrative unless stated otherwise.

10.2.3 · Write-ahead logging

A database may commit while some modified data pages are still awaiting their final storage writes. With write-ahead logging, it first persists the recovery information needed to reconstruct the accepted work under its protocol.

On recovery, the engine uses the log to recover the appropriate state. Checkpoints reduce recovery work by advancing what has been written; they are not backups. Acknowledgment durability depends on sync and replication settings. Losing the only device that holds both log and data can still lose everything. Logging protects particular recovery steps, while independent copies and correct application changes address different failure classes.

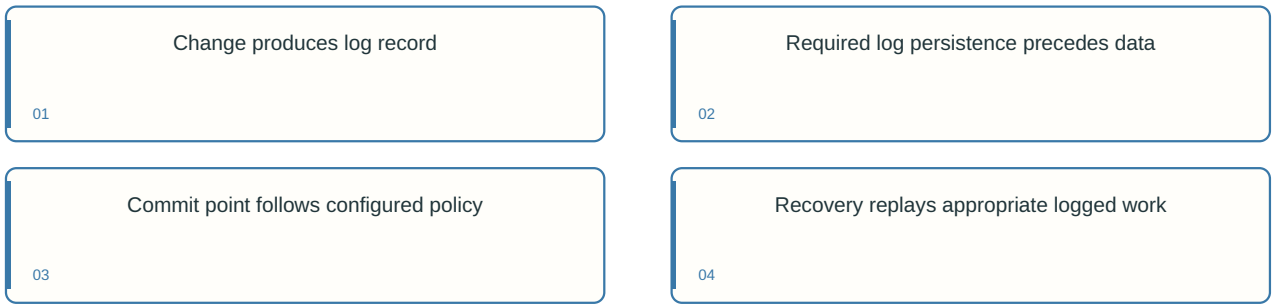


FIG. 10.2.3 **Write-ahead logging · the example**

Write-ahead logging preserves recovery information before corresponding data pages, under configured persistence rules.

Author-created illustration. Numbers are illustrative unless stated otherwise.

10.2.4 · MVCC

A reader starts while another transaction updates a row. With multiversion concurrency control, or MVCC, the engine can keep multiple versions and choose the one visible under the reader's snapshot rules.

This reduces some direct reader/writer interference, but the snapshot contract still matters. A long reader can retain old versions and delay cleanup. Two transactions can read the same multi-row rule, change different rows, and together violate it. That is possible write skew under suitable snapshot semantics. Learn the selected isolation and conflict rules; the presence of versions alone does not establish serializable execution.

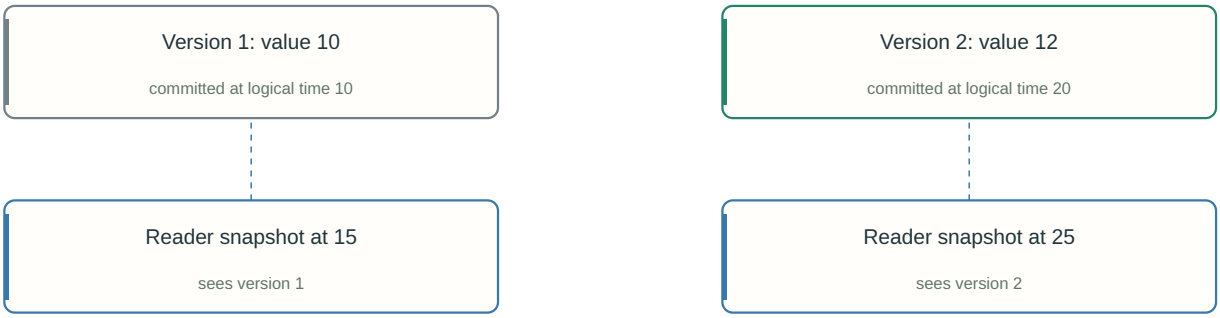


FIG. 10.2.4 **Two snapshots can legitimately see different versions**

A reader whose snapshot precedes version two can keep seeing version one while a later reader sees version two. MVCC alone does not imply serializability.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · What changes when one local transaction is divided across independent databases?

A former all-or-nothing invariant may no longer share one commit boundary. Identify which related changes require agreement, whether a distributed transaction is justified, or whether a workflow with explicit intermediate states and compensation fits. Local success at each store does not guarantee the combined business outcome.

Remember: Commitment, durable recovery, and snapshot visibility are distinct parts of the database contract.

Try applying it: Why does an engine using MVCC still need an explicit isolation policy?

Unit 10 exercises → · Unit 10 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. PostgreSQL 18: Transaction Isolation ↗

   Official versioned documentation. Specific PostgreSQL isolation behavior; no all-database equivalence claim.

   Access reviewed · 21 October 2026

2. PostgreSQL 18: Index Types ↗

   Official versioned documentation. Supported access structures and their purposes.

   Access reviewed · 21 October 2026

3. PostgreSQL 18: Write-Ahead Logging ↗

   Official versioned documentation. Logging and recovery ordering; durability remains configuration-dependent.

   Access reviewed · 21 October 2026

4. RocksDB Overview ↗

   Official implementation documentation · accessed 21 Oct 2026. An LSM-based engine; exact tuning is implementation-specific.

   Access reviewed · 21 October 2026

Races, anomalies & isolation

Prerequisites: 10.2 · Commit, logging & versions 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Two buyers want one seat, while two doctors can jointly leave nobody on call. Trace the different conflicts and choose protection for the whole invariant.

10.3.1 · Locking and optimistic concurrency control

Two buyers see seat version seven. Each tries UPDATE ... WHERE version=7, changing the owner and advancing the version. After one succeeds, the other's condition no longer matches, so it changes zero rows.

This is optimistic concurrency control: validate the state assumption when applying the change. Zero rows means conflict, requiring a fresh read and business decision. Pessimistic locking coordinates before conflicting work instead. Locks can wait or deadlock; optimistic retries can waste effort under heavy contention. Choose according to conflict frequency and operation cost, and protect the whole invariant rather than treating every affected-row result as success.

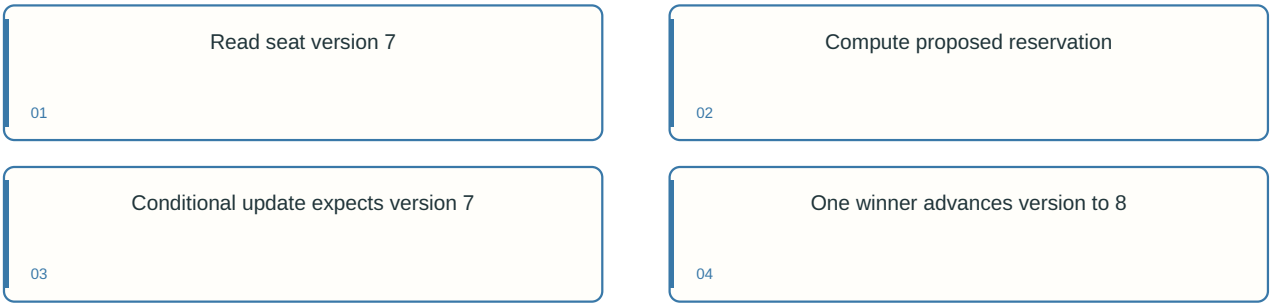


FIG. 10.3.1 **Locking and optimistic concurrency control · the example**

Locks coordinate conflicting access; optimistic checks validate expected state before applying changes.

Author-created illustration. Numbers are illustrative unless stated otherwise.

10.3.2 · Dirty reads, nonrepeatable reads, phantom reads

A dirty read observes another transaction's uncommitted change. A nonrepeatable read sees a row change between two reads. A phantom changes the set of rows matching a repeated query—for example, a fourth reservation appears after a query first found three.

These names describe histories worth distinguishing, but whether they occur depends on the chosen engine and isolation level. Write the problematic sequence in order and check the documented behaviour. Do not assume equal isolation names mean identical implementations. A test of one row read also cannot establish every predicate or write-conflict guarantee needed by the application.

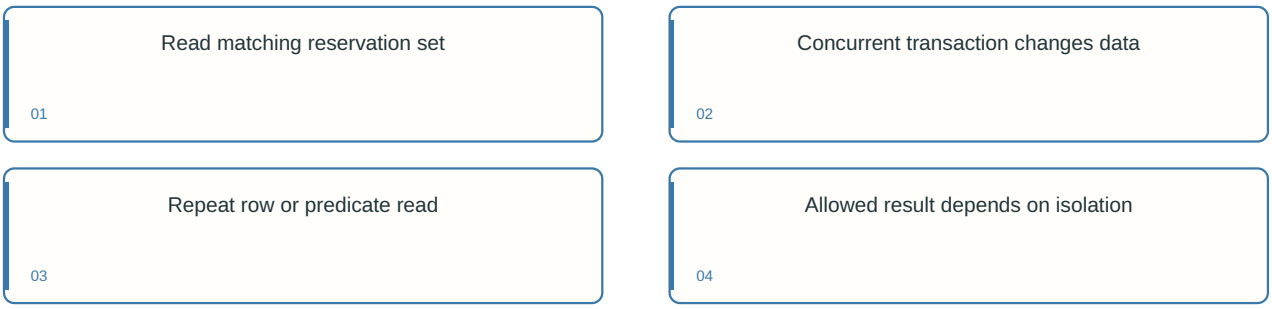


FIG. 10.3.2 **Dirty reads, nonrepeatable reads, phantom reads · the example**

Dirty, nonrepeatable, and phantom reads describe different concurrent histories whose occurrence depends on isolation implementation.

Author-created illustration. Numbers are illustrative unless stated otherwise.

10.3.3 · Lost updates, write skew, serialization anomalies

Two doctors are on call. Each transaction checks that the other doctor remains on call, then marks its own doctor off. If both see the original snapshot, they update different rows but leave nobody on call.

This is write skew: individually plausible updates violate a shared rule. A lost update instead overwrites another writer's result. More generally, a serialization anomaly has no equivalent valid serial ordering. Protect the actual invariant with suitable serialization, coordinated locking, or a shared conflict point. Row-level protection alone does not necessarily cover a condition spanning several rows.

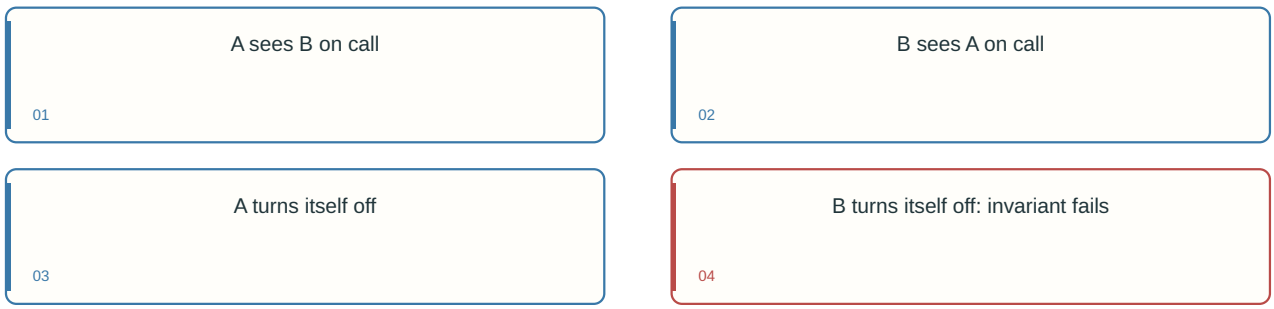


FIG. 10.3.3 **Lost updates, write skew, serialization anomalies · the example**

Lost updates overwrite work; write skew can violate a multi-row rule even when separate rows are protected.

Author-created illustration. Numbers are illustrative unless stated otherwise.

10.3.4 · Isolation levels and implementation differences

The selected engine's isolation implementation is part of the application contract. In PostgreSQL 18, ordinary Read Committed queries use statement snapshots. Repeatable Read gives a transaction snapshot but can still allow serialization anomalies. Serializable adds conflict handling that can reject a transaction.

These rules are version- and statement-sensitive; other engines can differ. When a transaction must retry, restart the whole affected transaction using fresh reads, with bounded attempts. Repeating only its last statement can retain assumptions from a snapshot that is no longer valid. Keep externally visible effects safe under replay as well. An isolation name is a starting point for reading the actual guarantees.

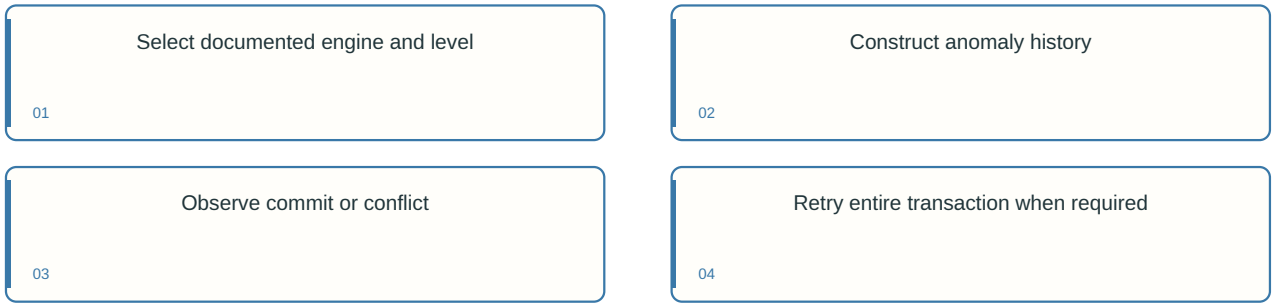


FIG. 10.3.4 **Isolation levels and implementation differences · the example**

Isolation behaviour depends on engine, version, and statement form; retries require fresh whole-transaction reasoning.

Author-created illustration. Numbers are illustrative unless stated otherwise.

10.3.5 · Serializable execution and serializability

Serializable transactions produce a committed result equivalent to some serial transaction order. That order need not be the same as wall-clock order between separate callers.

Linearizability concerns operations respecting real-time precedence, as if each took effect at one point between its invocation and reply. Strict serializability combines real-time constraints with transaction serialization. Keep these properties separate when stating a requirement. A serializable transaction may abort and need retry, so protect external effects in any replayed code. Stronger concurrency guarantees help preserve a correctly expressed rule; they cannot correct a wrong fee calculation or supply every desired availability property.

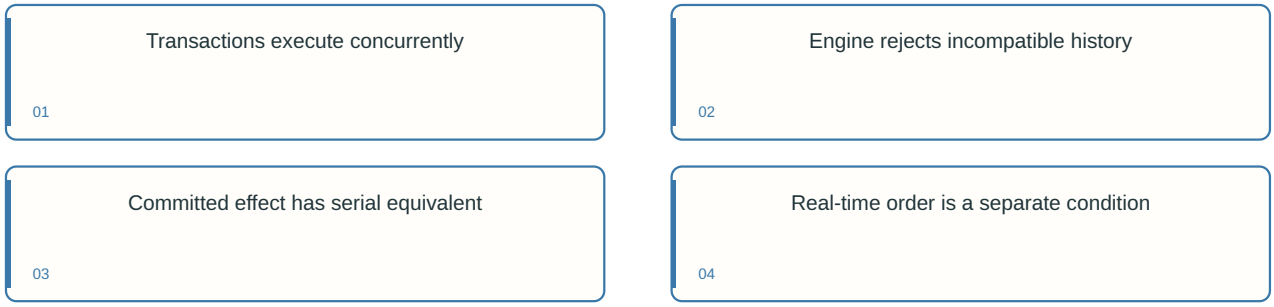


FIG. 10.3.5 **Serializable execution and serializability · the example**

Serializability concerns serial transaction equivalence; linearizability also concerns operation real-time precedence.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · What changes when one local transaction is divided across independent databases?

A former all-or-nothing invariant may no longer share one commit boundary. Identify which related changes require agreement, whether a distributed transaction is justified, or whether a workflow with explicit intermediate states and compensation fits. Local success at each store does not guarantee the combined business outcome.

Remember: Concurrency control must match the rule being preserved, not just the individual row being updated.

Try applying it: What does zero affected rows mean after an optimistic seat claim?

Unit 10 exercises → · Unit 10 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. PostgreSQL 18: Transaction Isolation ↗

   Official versioned documentation. Specific PostgreSQL isolation behavior; no all-database equivalence claim.

   `Access reviewed · 21 October 2026`

2. PostgreSQL 18: Index Types ↗

   Official versioned documentation. Supported access structures and their purposes.

   `Access reviewed · 21 October 2026`

3. PostgreSQL 18: Write-Ahead Logging ↗

   Official versioned documentation. Logging and recovery ordering; durability remains configuration-dependent.

   `Access reviewed · 21 October 2026`

4. RocksDB Overview ↗

   Official implementation documentation · accessed 21 Oct 2026. An LSM-based engine; exact tuning is implementation-specific.

   `Access reviewed · 21 October 2026`

Pools, recovery & ticket correctness

Prerequisites: 10.3 · Races, anomalies & isolation 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A pool is full and the backup job says it succeeded. Examine connection ownership, multi-statement transactions, and actual restore evidence before making a reliability claim.

10.4.1 · Database connection pooling

Assume ten database connections, each held for an average of 100 ms. The simple fully busy service bound is 10/0.1 = 100 operations per second before other constraints and variability.

A pool limits connections but also creates acquisition waits. Increasing its size can add contention instead of throughput. Measure hold time, lock waits, idle transactions, and queue age. If callers retain a connection while waiting for an unrelated HTTP service, shorten that ownership interval where the transaction allows it. Include acquisition in the request deadline. Pool capacity is useful only when the database can perform the work it admits.

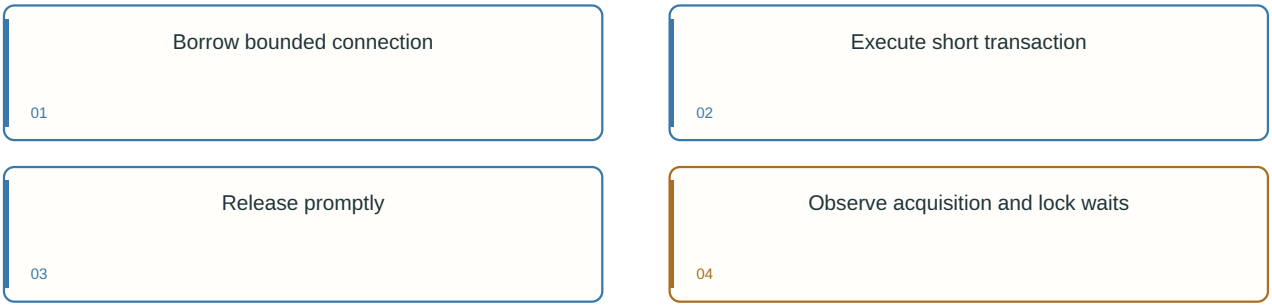


FIG. 10.4.1 **Database connection pooling · the example**

Database pools reuse bounded connections, with acquisition, hold time, and downstream contention included in deadlines.

Author-created illustration. Numbers are illustrative unless stated otherwise.

10.4.2 · Transactions across multiple operations

Reserving a seat and inserting the corresponding local reservation must agree. If one commits without the other, the records no longer describe one valid outcome. Put both statements inside a deliberate transaction boundary.

Check what the libraries actually do. An ORM save call does not establish that every related action shares the same transaction. Across stores or remote services, use an explicit coordination protocol and reconciliation where needed. Keep a local transaction short while covering the required invariant. Do not let a returned lazy result depend on resources already closed when the operation's scope ended.

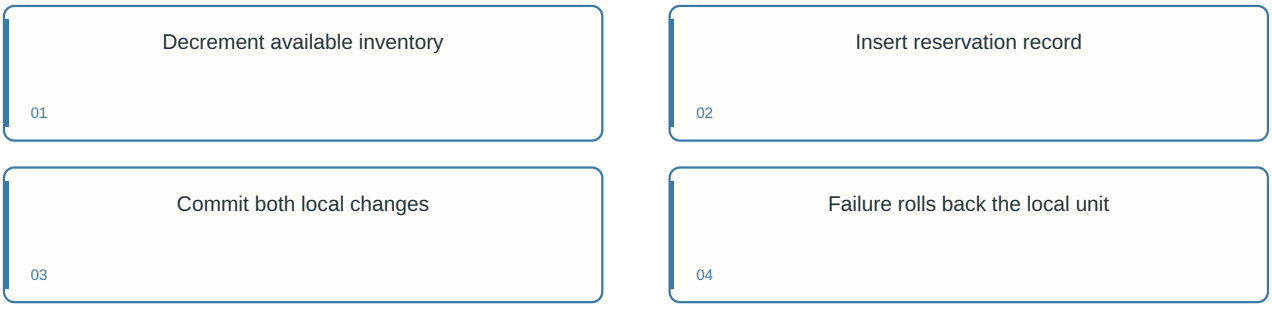


FIG. 10.4.2 **Transactions across multiple operations · the example**

Related statements need an explicit common transaction when their invariant requires one all-or-nothing change.

Author-created illustration. Numbers are illustrative unless stated otherwise.

10.4.3 · Backup, recovery, and durability

A backup job reports success. We still need to know whether its data can be decrypted, restored, and used by the application. Test restoration with the required schema, credentials, and business checks, and measure how long it takes.

Replication maintains live copies and may repeat an accidental deletion. Backups preserve recoverable history under their retention policy. Point-in-time recovery also needs the appropriate log history. State which acknowledged writes can be recovered after each failure. Owning a backup file is useful evidence of a file, but the recovery promise requires a tested procedure and suitable independent storage.

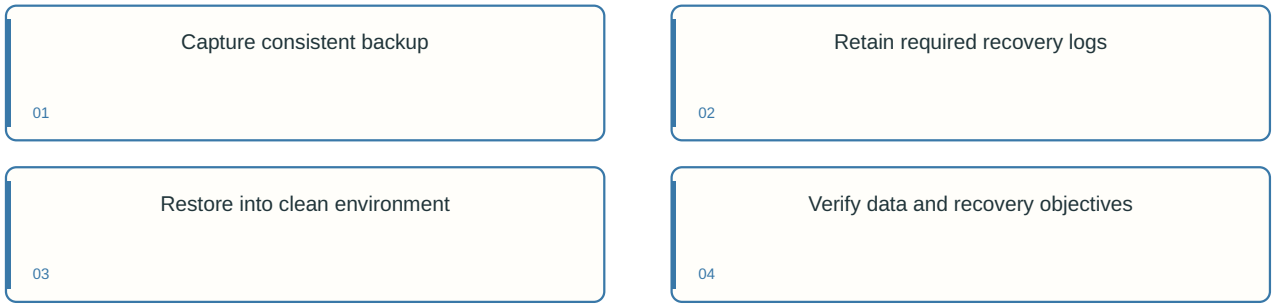


FIG. 10.4.3 **Backup, recovery, and durability · the example**

Recovery needs tested backups, logs, keys, compatible applications, and an explicit acknowledged-data policy.

Author-created illustration. Numbers are illustrative unless stated otherwise.

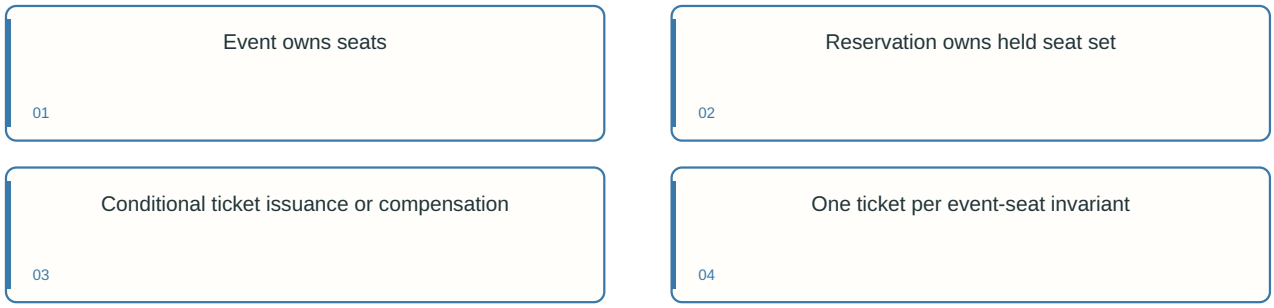


FIG. 10.4.4 **State and ownership in the related design**

Seat, hold, payment, and ticket records must preserve their defined ownership relationships.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Try the mechanism in code

PYTHON · CONDITIONAL SEAT CLAIM

```
from mechanisms import open_demo_store, claim_seat
db = open_demo_store()
db.execute("INSERT INTO seats VALUES (1,NULL)")
db.commit()
assert claim_seat(db, 1, "A")
assert not claim_seat(db, 1, "B")
db.close()
```

A claims the available seat; B's later conditional claim fails. The affected-row result determines whether the change occurred. The executable checks also use two connections to the same store.

This protects a local ownership rule. Payment, hold expiry, and stale callbacks need connected but separate workflow logic.

Download the executable mechanism collection ↓ · Follow the connected complete design →

THE WORKBENCH · TRY CHANGING THE INPUTS

Let two callers claim the last seat

1. A reads available.
2. B reads available.
3. A writes owner A and reports success.
4. B overwrites owner B and reports success.

Two callers believe they won. Reading the final row would show B, but would miss the earlier success reported to A.

Compare the two policies using the same order of steps. Does either caller learn that the seat has already been taken? This display replays a fixed example. The downloadable SQLite example separately tests the atomic conditional update.

▼ Think further · What changes when one local transaction is divided across independent

databases?

A former all-or-nothing invariant may no longer share one commit boundary. Identify which related changes require agreement, whether a distributed transaction is justified, or whether a workflow with explicit intermediate states and compensation fits. Local success at each store does not guarantee the combined business outcome.

Remember: Transaction scope and tested recovery must agree with the operation's promises.

Try applying it: Which check establishes that a restored booking database and its application are usable together?

Unit 10 exercises → · Unit 10 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. PostgreSQL 18: Transaction Isolation ↗

   Official versioned documentation. Specific PostgreSQL isolation behavior; no all-database equivalence claim.

   `Access reviewed · 21 October 2026`

2. PostgreSQL 18: Index Types ↗

   Official versioned documentation. Supported access structures and their purposes.

   `Access reviewed · 21 October 2026`

3. PostgreSQL 18: Write-Ahead Logging ↗

   Official versioned documentation. Logging and recovery ordering; durability remains configuration-dependent.

   `Access reviewed · 21 October 2026`

4. RocksDB Overview ↗

   Official implementation documentation · accessed 21 Oct 2026. An LSM-based engine; exact tuning is implementation-specific.

   `Access reviewed · 21 October 2026`

PART 3 / UNIT 11

Caching and Content Delivery

A product is read repeatedly between edits. Work out which cost a cache can avoid, how much active data fits, and which layer should hold the reusable representation.

The running example

A popular product page receives enormous traffic. Introduce caching gradually and examine stale prices, thundering herds, inventory correctness, and failure behavior.

Unit 11 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

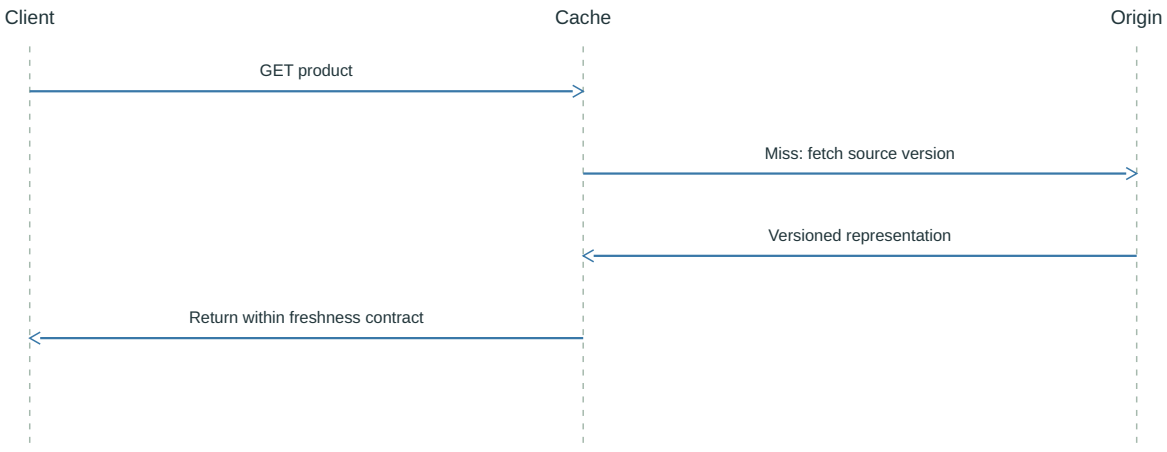
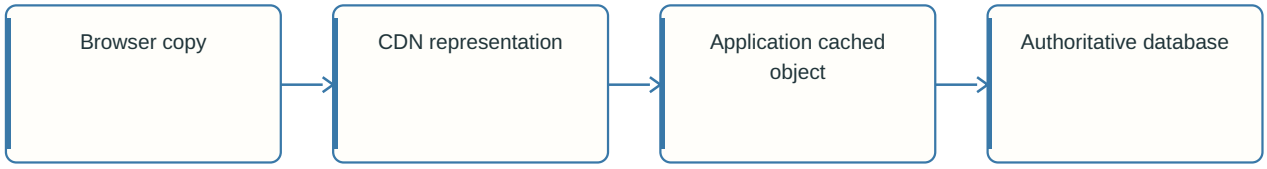


FIG. 11.0.1 **Cache hit/miss sequence**

On a hit, the origin exchange is omitted. Cache-aside population still needs race reasoning.

Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 11.0.2 **cache hierarchy**

These are distinct copies with separate freshness, identity, and invalidation policies.

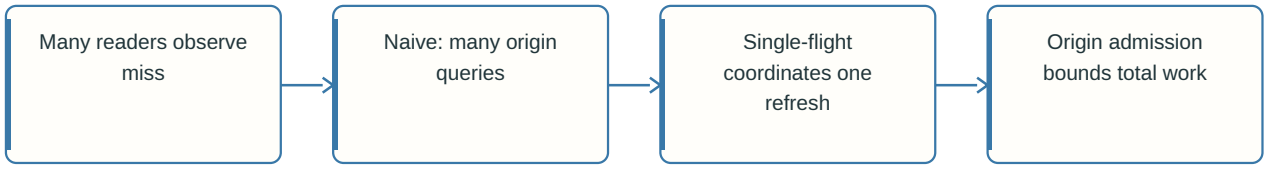
Author-created illustration. Numbers are illustrative unless stated otherwise.

Policy	After A,B,C,A	Insert D: evicted
Ideal LRU	Order B,C,A	B
FIFO	Insertion A,B,C	A

FIG. 11.0.3 **LRU and FIFO disagree after the same accesses**

Capacity is three. Both policies saw the same requests; policy metadata changes the decision.

Author-created illustration. Numbers are illustrative unless stated otherwise.

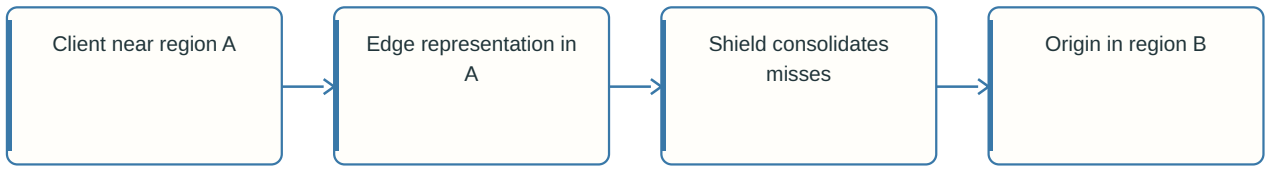


Arrows: ordered stages of this worked mechanism

FIG. 11.0.4 **stampede timeline**

A cache outage or synchronized expiry must not create an unbounded origin reconstruction storm.

Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 11.0.5 **geographic CDN delivery**

Geographic delivery reduces some distance and origin work while adding copies and freshness obligations.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Locality and cache placement

Prerequisites: Networking Foundations for System Design, Data Models and Storage Technologies 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A product is read repeatedly between edits. Work out which cost a cache can avoid, how much active data fits, and which layer should hold the reusable representation.

11.1.1 · Why caching works

A product description is read thousands of times between edits. Reusing a stored response can avoid repeating the database lookup or calculation. This reuse is why caching can help.

Assume a cache hit takes 1 ms, a miss takes 50 ms, and 90% of requests hit. The simple average service time is 0.9×1 + 0.1×50 = 5.9 ms, excluding queueing and other work. Compare this saving with memory, lookup, and maintenance costs. A result used only once may be a poor cache candidate. Hit ratio also does not establish tail latency: slow misses and queues can still dominate some users' waits.

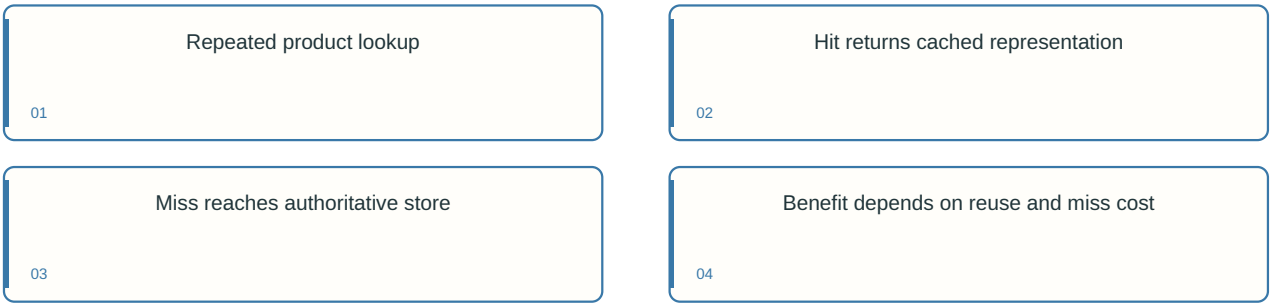


FIG. 11.1.1 **Why caching works · the example**

Caching saves repeated work when reuse outweighs access, storage, and maintenance costs under the required semantics.

Author-created illustration. Numbers are illustrative unless stated otherwise.

11.1.2 · Cache locality and the working set

A user repeatedly opens the same product: that is temporal locality. Reading neighbouring records together illustrates spatial locality. A working set is the data actively needed during a relevant interval.

A cache much smaller than that set may continually evict entries just before they are reused. A large scan can also displace useful entries under some policies. Measure bytes and object sizes as well as key counts, and distinguish frequently used data from the full historical collection. Preloading every object can fill memory with cold content. Admission rules decide what deserves a place before eviction decides what must leave.

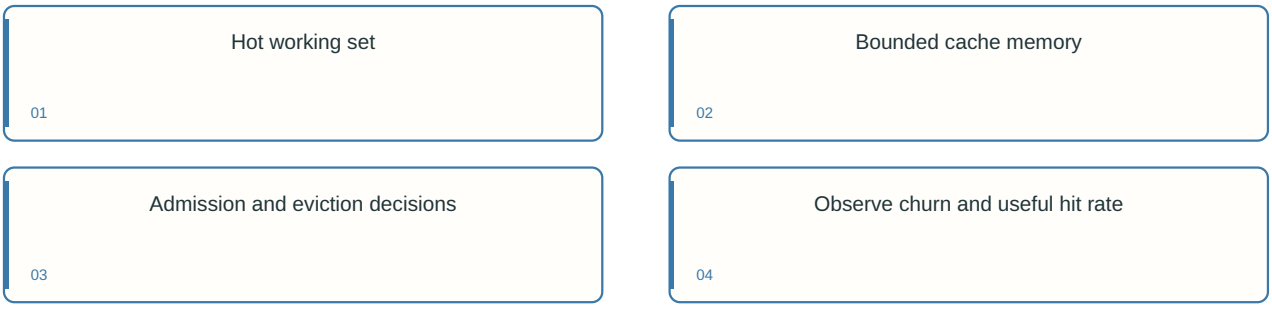


FIG. 11.1.2 **Cache locality and the working set · the example**

Locality and the active working set determine which data can usefully fit and remain in a cache.

Author-created illustration. Numbers are illustrative unless stated otherwise.

11.1.3 · In-process vs distributed caches

Two application instances each keep a price in memory. One has price 10 and another price 12. Their in-process caches provide fast access but create separate copies whose freshness needs coordination.

A distributed cache provides a shared network-accessible layer, adding encoding, connection, and failure costs. Choose by reuse patterns, duplicated memory, and required behaviour when a copy is stale or unavailable. Ordinary cached stock counts should not become the sole authority for exclusive allocation. Placing the cache elsewhere changes its access and failure boundaries; it does not by itself resolve the application's consistency rule.

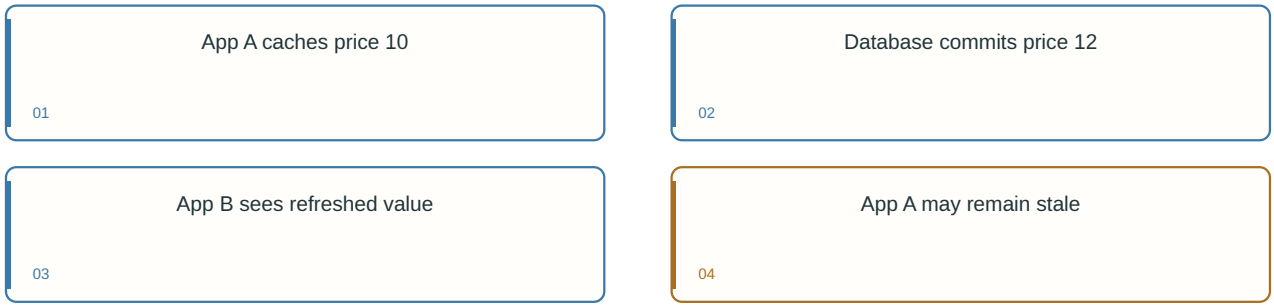


FIG. 11.1.3 **In-process vs distributed caches · the example**

In-process caches duplicate local copies; distributed caches add a shared network boundary and its failure behaviour.

Author-created illustration. Numbers are illustrative unless stated otherwise.

11.1.4 · Browser, CDN, application, and database caches

A page can be answered by a browser cache, a CDN, an application cache, or a database buffer pool. Each stores a different kind of copy. Database pages and a rendered account response also have different keys and update rules.

Draw the layers actually used and specify freshness at each. Invalidating an application object does not clear a browser's response. A public edge cache with a poorly scoped key can expose personal data. When diagnosing a stale result, identify which layer answered and which version it held. “The cache was cleared” is incomplete unless it names the particular copy involved.

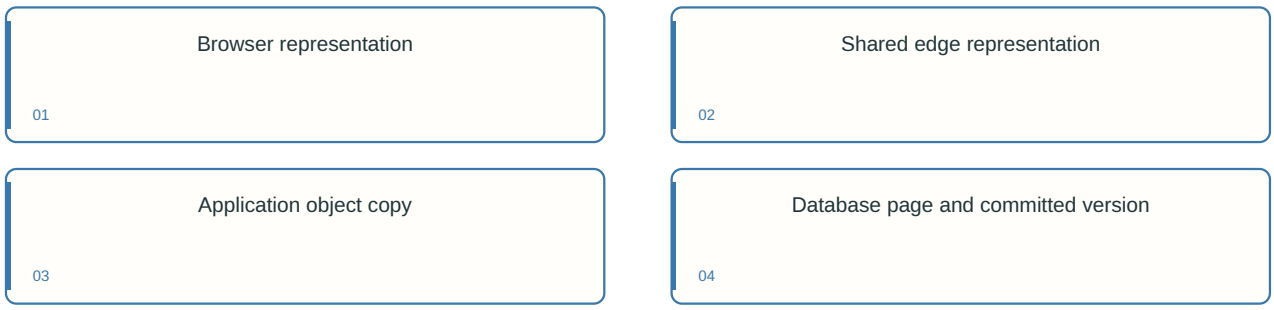


FIG. 11.1.4 **Browser, CDN, application, and database caches · the example**

Browser, edge, application, and database caches hold different copies with independent freshness and trust requirements.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · When should a product stop serving rather than return a stale cached result?

Stop when the operation requires current authority or the permitted stale interval has ended. An old description may be allowed, while allocating scarce stock or deciding current access may not be. State which actions degrade and which wait or reject, with visible status and a recovery path.

Remember: Cache value comes from useful reuse under a valid freshness and access policy.

Try applying it: Why can a high hit ratio coexist with slow or incorrect user responses?

Unit 11 exercises → · Unit 11 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. RFC 9111: HTTP Caching, 2022 ↗

   Original standard. HTTP cache semantics; application cache races are separate examples.

   Access reviewed · 21 October 2026

2. Redis: Key eviction ↗

   Official documentation · accessed 21 Oct 2026. Implementation policies can approximate ideal LRU/LFU.

   Access reviewed · 21 October 2026

3. Content delivery networks — Cloudflare Learning Center ↗

   Official engineering explanation. CDN role; atlas delivery and traffic numbers are illustrative.

   Access reviewed · 21 October 2026

Copies, freshness & eviction

Prerequisites: 11.1 · Locality and cache placement 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A delayed reader inserts yesterday's price after a writer invalidates it. Follow cache population, expiry, versioning, and eviction as separate decisions.

11.2.1 · Cache-aside, read-through, write-through, write-behind

With cache-aside, application code loads from the source after a miss and populates the cache. Read-through places that loading behind the cache interface. Write-through includes backing-store work in the write path; write-behind delays it. Exact ordering varies by implementation.

Trace failures explicitly. A volatile write-behind cache can lose acknowledged changes before backing persistence. Cache-aside also has a race: a reader fetches version one, a writer commits version two and invalidates, then the delayed reader inserts version one. Where freshness requires stronger behaviour, use an appropriate version or coordination protocol. A pattern name alone does not resolve these event sequences.

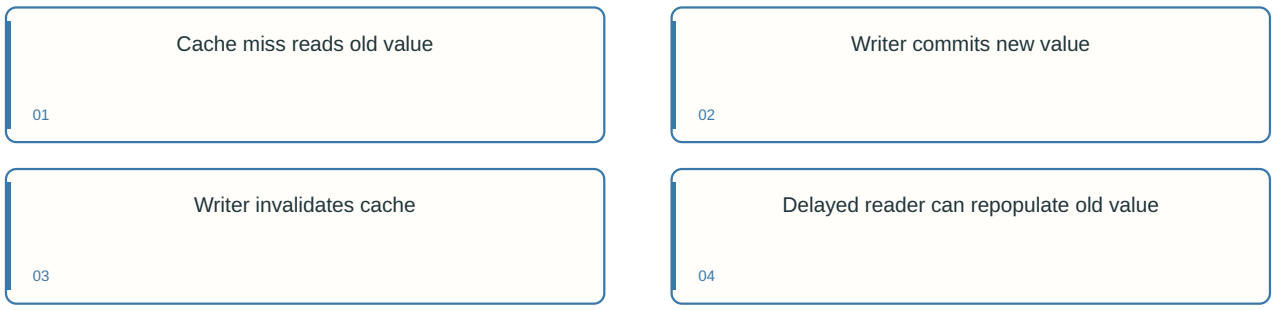


FIG. 11.2.1 **Cache-aside, read-through, write-through, write-behind · the example**

Cache population and write patterns specify ordering and failure gaps; their names alone do not remove races.

Author-created illustration. Numbers are illustrative unless stated otherwise.

11.2.2 · TTL and cache expiration

A cache receives a value that was already five minutes old and gives it a one-minute TTL. The copy may then remain six minutes behind its source. TTL describes residence after insertion, rather than necessarily the age of the fact.

Expiration also differs from eviction due to memory pressure. Choose lifetimes using tolerated staleness and refresh behaviour, and consider client-held copies. Adding jitter can spread expirations that would otherwise happen together. For a correctness-critical action, revalidate against the authority when required. A short display-cache TTL is useful only if it matches the promise made about that display.

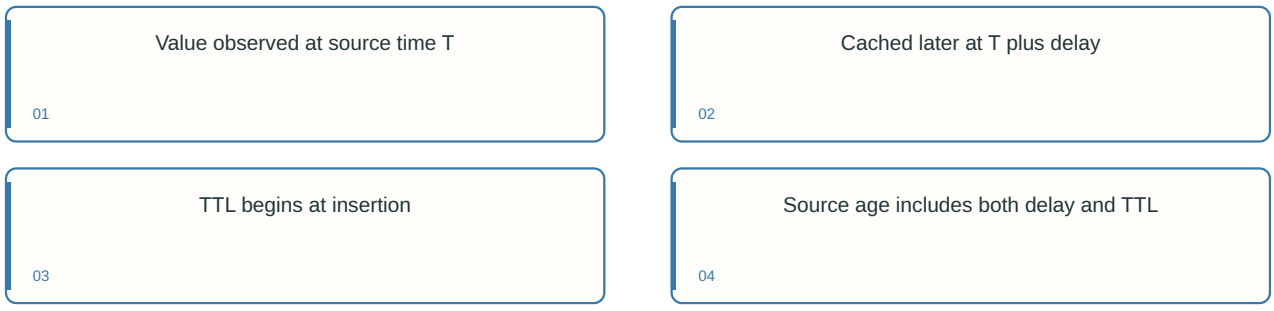


FIG. 11.2.2 **TTL and cache expiration · the example**

TTL limits intended residence after insertion, not necessarily the source age of the inserted fact.

Author-created illustration. Numbers are illustrative unless stated otherwise.

11.2.3 · Cache consistency and invalidation

A writer updates the source and deletes a cache entry. A delayed reader can still repopulate it from an earlier read, leaving an old value after the invalidation. Simply deleting after writing does not prevent every race.

Versioned entries, conditional population, or source-version checks can support a stronger protocol. Observe delayed or repeated invalidation events and their progress. Choose freshness per operation. Showing an older price may be acceptable if checkout verifies the actual price; allocating the last item from an old count can oversell it. Invalidation must serve the business rule, rather than merely keep copies approximately similar.

Location	Version and value	Authority
Database	v2: price 12	current authoritative value
App cache A	v1: price 10	stale representation
Checkout	revalidates database	does not trust display cache

FIG. 11.2.3 **A stale cache is a separate state copy**

Invalidation and propagation policy decide when copies agree. The critical checkout operation uses the required authoritative contract.

Author-created illustration. Numbers are illustrative unless stated otherwise.

11.2.4 · Eviction: LRU, LFU, FIFO, and alternatives

At capacity three, access A,B,C,A,D. Ideal LRU treats the second access to A as recent and evicts B. FIFO preserves insertion order and evicts A. LFU favours frequently used entries but also needs a tie-breaking rule.

Real cache policies may approximate these ideals. Test the actual policy with representative object sizes and request patterns. Eviction decides what leaves; admission decides what enters. Neither can make an arbitrarily large working set fit into memory. A policy that performs well on one tiny fixture may behave poorly during scans or changing popularity, so measure the workload it is meant to serve.

Policy	After A,B,C,A	Insert D: evicted
Ideal LRU	Order B,C,A	B
FIFO	Insertion A,B,C	A

FIG. 11.2.4 **LRU and FIFO disagree after the same accesses**

Capacity is three. Both policies saw the same requests; policy metadata changes the decision.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · When should a product stop serving rather than return a stale cached result?

Stop when the operation requires current authority or the permitted stale interval has ended. An old description may be allowed, while allocating scarce stock or deciding current access may not be. State which actions degrade and which wait or reject, with visible status and a recovery path.

Remember: Residence time and eviction policy do not by themselves establish current source data.

Try applying it: How can a one-minute TTL serve a value that is more than one minute behind its source?

Unit 11 exercises → · Unit 11 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. RFC 9111: HTTP Caching, 2022 ↗

   Original standard. HTTP cache semantics; application cache races are separate examples.

   Access reviewed · 21 October 2026

2. Redis: Key eviction ↗

   Official documentation · accessed 21 Oct 2026. Implementation policies can approximate ideal LRU/LFU.

   Access reviewed · 21 October 2026

3. Content delivery networks — Cloudflare Learning Center ↗

   Official engineering explanation. CDN role; atlas delivery and traffic numbers are illustrative.

   Access reviewed · 21 October 2026

Hot keys, stampedes & negative results

Prerequisites: 11.2 · Copies, freshness & eviction 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A cache restarts just as a product becomes popular. Control duplicate loading, absent lookups, warming, and permitted stale results before the origin becomes overwhelmed.

11.3.1 · Hot keys

A single product becomes extremely popular. Even a large cache cluster can overload the one owner serving that key. Adding nodes mainly gives other keys more capacity unless the design also distributes this key's traffic.

For reads, local copies, replicas, or coalescing repeated work may help, with an explicit freshness policy. For writes, determine whether the value can be split or combined safely. An aggregate counter may allow distributed increments; exclusive stock allocation needs a stronger rule. Identify whether the pressure is transfer volume, recomputation, object size, or serialised updates before selecting a remedy.

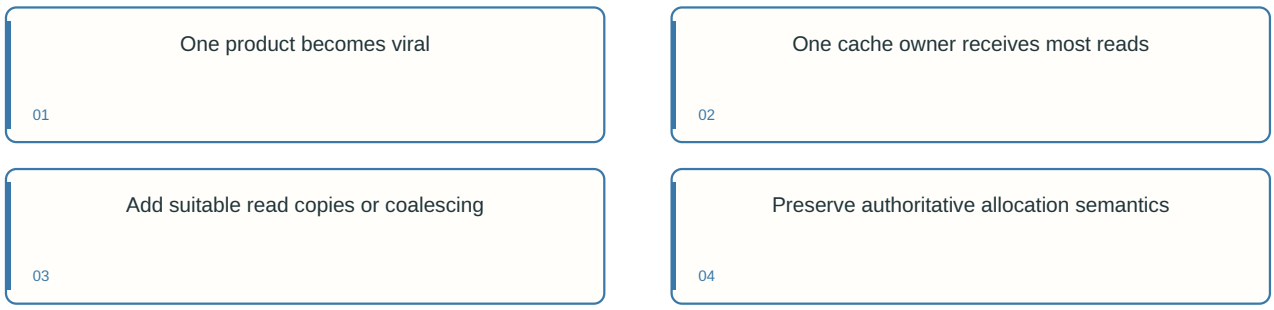


FIG. 11.3.1 **Hot keys · the example**

A hot key concentrates work on an owner and needs a remedy suited to its actual read or write pressure.

Author-created illustration. Numbers are illustrative unless stated otherwise.

11.3.2 · Cache stampedes, penetration, and avalanche

Many requests miss the same key and all recompute it: a cache stampede. Repeated requests for nonexistent keys are often called penetration. Broad expiry or cache loss can cause an avalanche of origin traffic.

Use controls suited to the cause. Single-flight loading combines concurrent work for one key. Negative caching can remember a genuine absence. TTL jitter spreads expiry, while bounded refresh and origin admission protect the source. A refresh lock needs its own failure and expiry behaviour. Test cache loss: allowing every application instance to rebuild everything at once can overload the database that remains responsible for the real data.

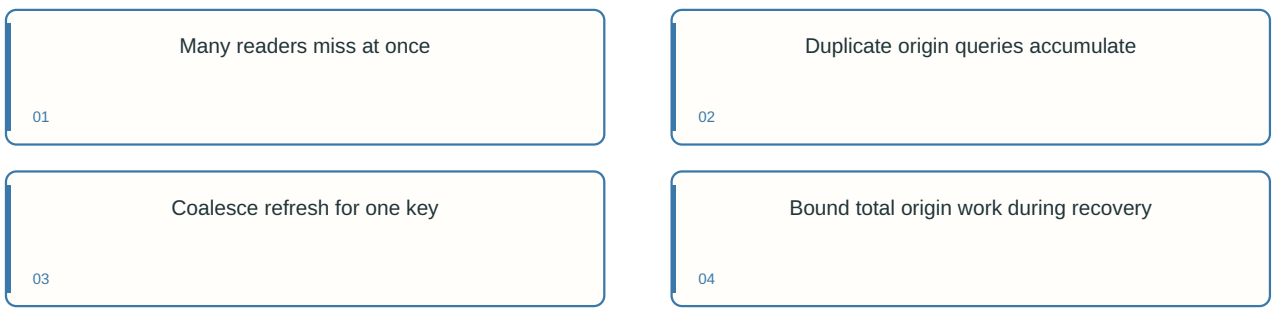


FIG. 11.3.2 **Cache stampedes, penetration, and avalanche · the example**

Stampedes, absent-key traffic, and broad expiry need controlled loading and origin protection.

Author-created illustration. Numbers are illustrative unless stated otherwise.

11.3.3 · Negative caching

A client repeatedly asks for a product that does not exist. A short-lived “not found” entry can prevent repeated source lookups. This is negative caching.

Distinguish absence from failure. A temporarily unreachable database has not established that the product is missing. Likewise, one user's denied access must not poison another user's cache entry. Scope keys correctly and use deliberate lifetimes. If the product is created just after its absence is cached, it may stay invisible until expiry or invalidation. The policy should state whether that delay is acceptable and how newly created records become visible.

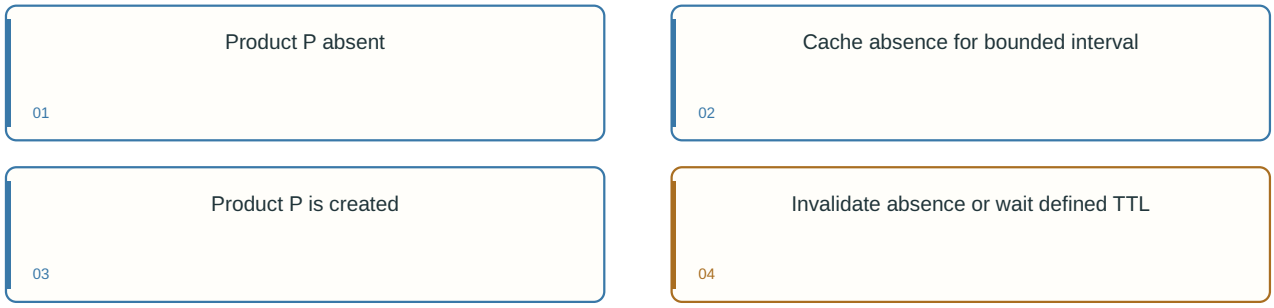


FIG. 11.3.3 **Negative caching · the example**

Negative caching stores a scoped absence or failure meaning for a deliberate lifetime.

Author-created illustration. Numbers are illustrative unless stated otherwise.

11.3.4 · Stale-while-revalidate

A cached product description expires while its source is slow. Stale-while-revalidate can return an allowed older copy and refresh it in the background. The user avoids the immediate wait, and combined refresh work can protect the origin.

Define separate ages for fresh data, permitted stale data, and unusable data. Repeated refresh failures should not silently extend stale serving forever. A description may tolerate this arrangement; an old payment reply cannot authorise a new charge. Make degradation visible where appropriate and preserve authoritative checks for operations whose correctness depends on current state. Faster responses have value only within the agreed freshness rule.

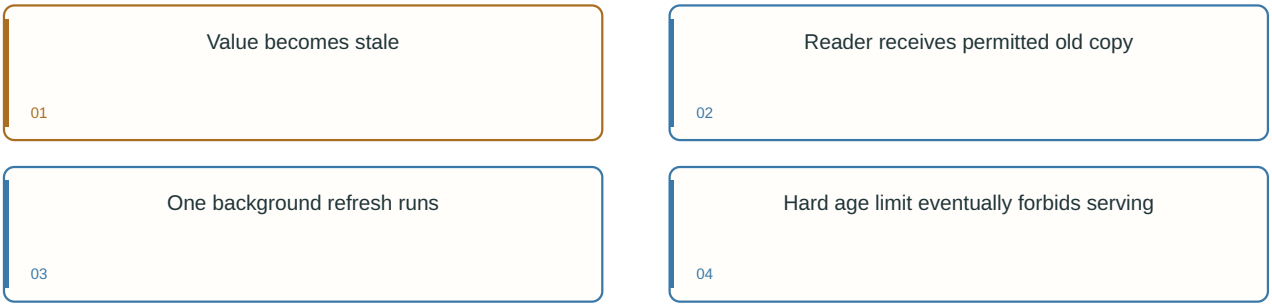


FIG. 11.3.4 **Stale-while-revalidate · the example**

Stale-while-revalidate serves an allowed older value while refresh proceeds, with a limit on unusable age.

Author-created illustration. Numbers are illustrative unless stated otherwise.

11.3.5 · Cache warming and prefetching

New application instances start with empty caches. Warming loads likely useful entries before normal demand; prefetching predicts what will be needed soon. Both spend bandwidth and memory before a user actually asks.

Load a measured working set gradually rather than scanning the entire database from every new instance. Check whether prefetched values are used and whether they evict more valuable entries. Staggering a rollout, sharing a cache, or limiting admission may solve a cold-start problem more simply. Stop obsolete speculative work. The useful question is how much future work was avoided, rather than how many entries were loaded.

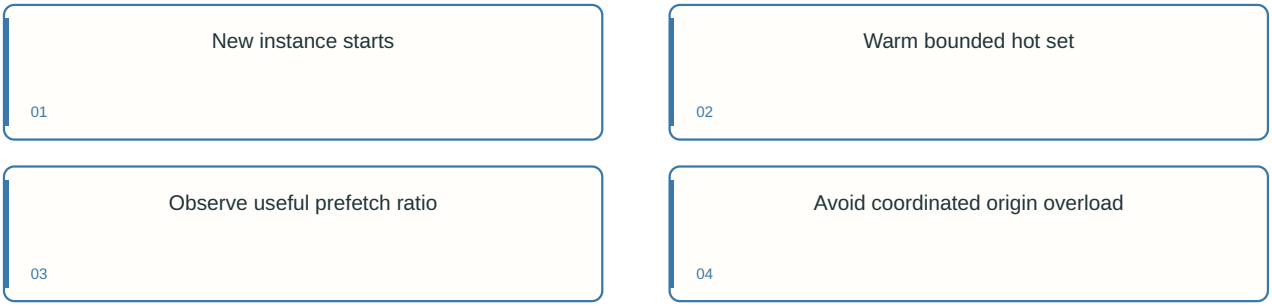


FIG. 11.3.5 **Cache warming and prefetching · the example**

Warming and prefetching spend speculative work that should be justified by actual future reuse.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · When should a product stop serving rather than return a stale cached result?

Stop when the operation requires current authority or the permitted stale interval has ended. An old description may be allowed, while allocating scarce stock or deciding current access may not be. State which actions degrade and which wait or reject, with visible status and a recovery path.

Remember: Bound refresh and fallback work while keeping the accepted stale and absent meanings explicit.

Try applying it: What should happen if background refresh keeps failing past the allowed stale age?

Unit 11 exercises → · Unit 11 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. RFC 9111: HTTP Caching, 2022 ↗

   Original standard. HTTP cache semantics; application cache races are separate examples.

   `Access reviewed · 21 October 2026`

2. Redis: Key eviction ↗

   Official documentation · accessed 21 Oct 2026. Implementation policies can approximate ideal LRU/LFU.

   `Access reviewed · 21 October 2026`

3. Content delivery networks — Cloudflare Learning Center ↗

   Official engineering explanation. CDN role; atlas delivery and traffic numbers are illustrative.

   `Access reviewed · 21 October 2026`

Edges, partitions & the price of a cache

Prerequisites: 11.3 · Hot keys, stampedes & negative results 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A cache cluster and CDN add more copies and another failure path. Compare their operations with source correctness and recovery capacity, including whether the cache should exist at all.

11.4.1 · Redis-like data structures

A set can track identifiers, a sorted set can organise scores or timestamps, and a hash can group fields. Redis-like systems expose such structures with operations suited to particular tasks.

Choose by the operations needed, then examine their atomic scope. One atomic command does not make an arbitrary sequence of commands atomic. A script or transaction may protect the local sequence, subject to the selected system's rules and key placement. Expiry, persistence, and replica behaviour remain separate decisions. A convenient list or stream is not automatically a durable queue with the required acknowledgment and recovery semantics.

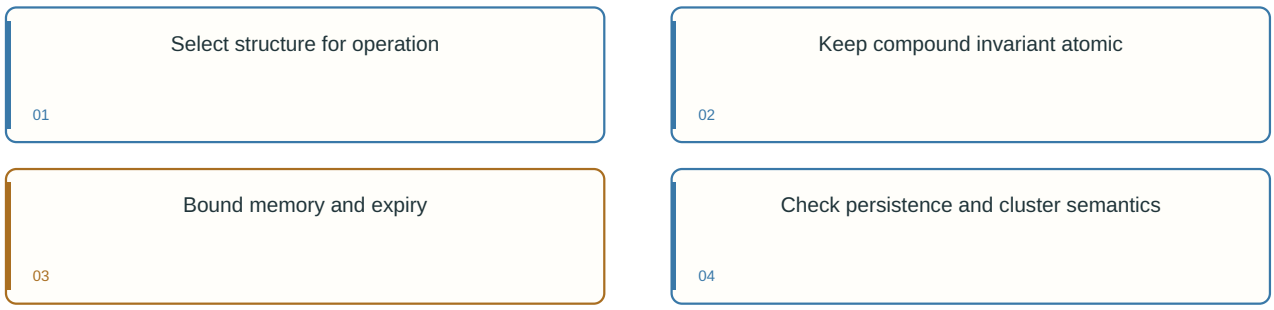


FIG. 11.4.1 **Redis-like data structures · the example**

Redis-like data structures offer specific operations; local command atomicity does not automatically cover a larger workflow.

Author-created illustration. Numbers are illustrative unless stated otherwise.

11.4.2 · CDN architecture and origin shielding

An origin shield collects edge misses before they reach the source, reducing duplicate origin work. A CDN places permitted representations near users. Both depend on correct keys and sharing rules.

Decide how query parameters, headers, identities, and content versions affect the representation. A signed URL is a capability with a scope and lifetime, not permission to ignore private-data policy. Purge propagation follows the provider contract and may not remove copies already kept by clients. Immutable versioned URLs can simplify public asset updates. Trace every remaining copy when immediate revocation matters; proximity and bandwidth savings do not establish access correctness.

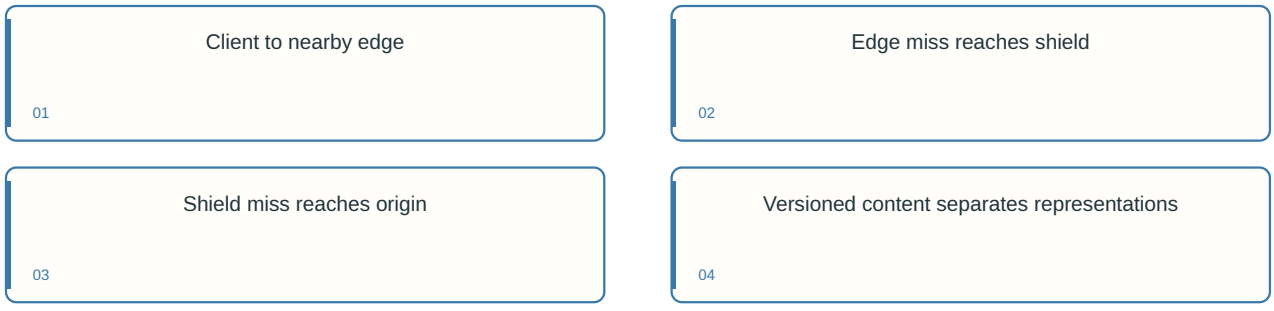


FIG. 11.4.2 **CDN architecture and origin shielding · the example**

CDN edges and shields distribute suitable representations under explicit key, capability, freshness, and purge rules.

Author-created illustration. Numbers are illustrative unless stated otherwise.

11.4.3 · Cache partitioning and replication

Partitioning assigns different keys to different owners. Replication makes additional copies under a protocol. A distributed cache may use both, but failover can still lose, evict, or expose delayed entries.

Specify whether the application falls back, serves permitted stale data, or rejects when the cache fails. Stable hashing can reduce key remapping during membership changes, while leaving hot-key traffic uneven. Keep source and cache recovery capacity explicit. The real peak may arrive when a cache cluster restarts and every missing value needs reconstruction. Test that scenario rather than sizing the origin only for a warm-cache hit ratio.

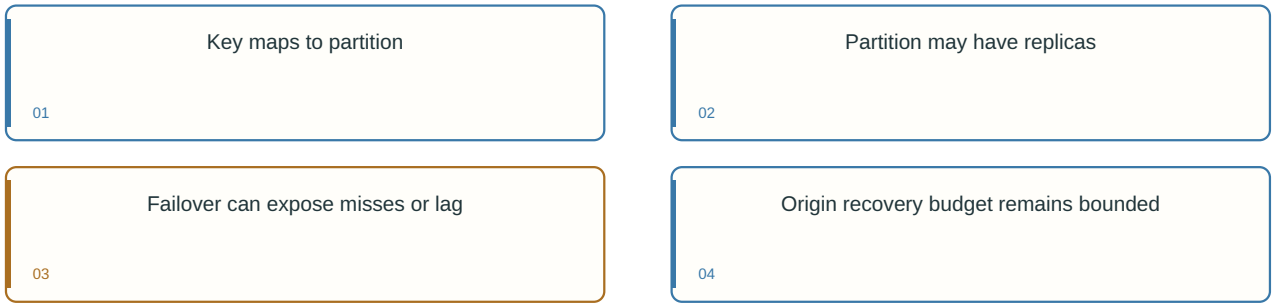


FIG. 11.4.3 **Cache partitioning and replication · the example**

Cache partitions and replicas need failure, remapping, fallback, and rebuild-capacity policies.

Author-created illustration. Numbers are illustrative unless stated otherwise.

11.4.4 · When caching is harmful

A cache adds a lookup, encoding, memory use, and copy-maintenance work. If results are rarely reused, those costs can exceed the work saved. If its key omits authorisation scope, the cache can also serve the wrong user's result.

Test with the cache empty and unavailable. A warm cache can hide insufficient source capacity until the day it disappears. Sometimes a better query or index removes the bottleneck with fewer freshness and recovery obligations. Keep caching where measured reuse and the agreed correctness policy justify it. Removing an unhelpful cache can simplify both performance and reasoning.

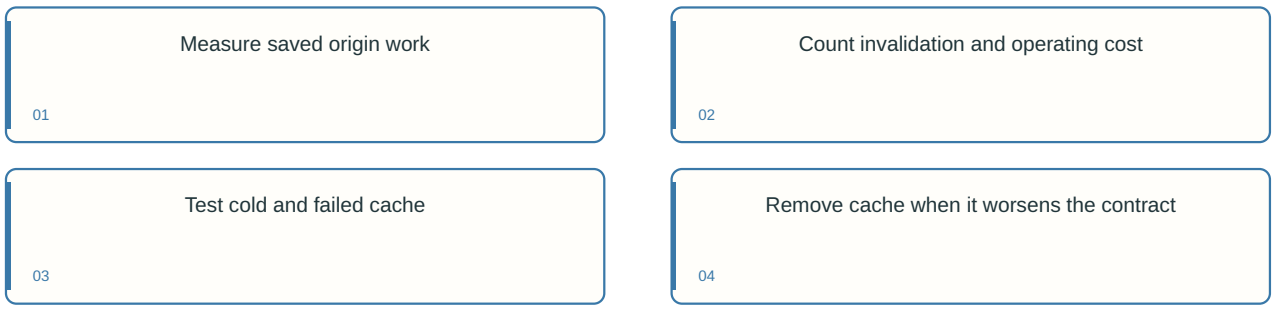


FIG. 11.4.4 **When caching is harmful · the example**

Caching can be harmful when it saves little, weakens scope, hides origin limits, or adds excessive lifecycle work.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Try the mechanism in code

PYTHON · IDEAL LRU

```
from mechanisms import LRU
cache = LRU(3)
for key in "ABC": cache.put(key, key)
cache.get("A")
assert cache.put("D", "D") == ("B", "B")
```

After A,B,C,A, ideal LRU considers B the least recently used. Adding D evicts B, as the assertion checks.

The fixture models entry count and exact recency. Actual caches can approximate the policy and use byte or expiry limits. Eviction alone says nothing about whether a loaded value is current.

Download the executable mechanism collection ↓ · Follow the connected complete design →

THE WORKBENCH · TRY CHANGING THE INPUTS

Follow which cache entry is used least recently

1. A: miss · oldest → newest: A
2. B: miss · oldest → newest: A, B
3. C: miss · oldest → newest: A, B, C
4. A: hit · oldest → newest: B, C, A
5. D: miss, evict B · oldest → newest: C, A, D

The cache holds a fixed number of entries. Each access loads a missing key or moves an existing key to the most recently used end. Real caches may use approximate eviction, byte limits, and expiry. Choosing an entry to evict does not tell us whether its value is fresh.

▼ Think further · When should a product stop serving rather than return a stale cached result?

Stop when the operation requires current authority or the permitted stale interval has ended. An old description may be allowed, while allocating scarce stock or deciding current access may not be. State which actions degrade and which wait or reject, with visible status and a recovery path.

Remember: Evaluate each cache by saved work, maintained semantics, and the cost of losing it.

Try applying it: Which workload would show that removing a cache improves the whole system?

Unit 11 exercises → · Unit 11 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. RFC 9111: HTTP Caching, 2022 ↗
   Original standard. HTTP cache semantics; application cache races are separate examples.
   Access reviewed · 21 October 2026

2. Redis: Key eviction ↗

   Official documentation · accessed 21 Oct 2026. Implementation policies can approximate ideal LRU/LFU.

   Access reviewed · 21 October 2026

3. Content delivery networks — Cloudflare Learning Center ↗

   Official engineering explanation. CDN role; atlas delivery and traffic numbers are illustrative.

   Access reviewed · 21 October 2026

PART 3 / UNIT 12

Scalability, Load Balancing, and Data Distribution

Adding application instances changes traffic distribution and often connection demand. Find which work is independent, where required state lives, and which signal should drive growth.

The running example

Scale a URL shortener from 100 requests per second to 100,000. Introduce new components only when justified by a demonstrated bottleneck.

Unit 12 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

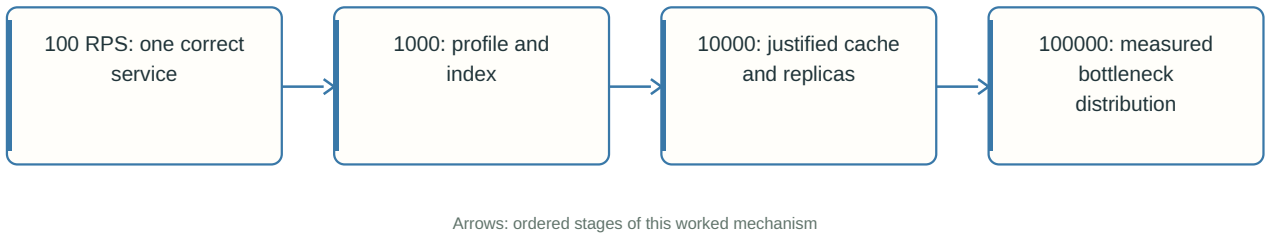


FIG. 12.0.1 **Architecture evolution across four load levels**

These are conceptual teaching checkpoints, not hardware benchmark thresholds.

Author-created illustration. Numbers are illustrative unless stated otherwise.

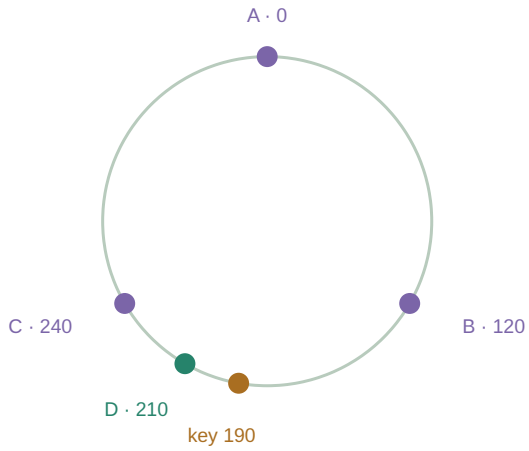


FIG. 12.0.2 **A new owner takes only part of the hash space**

With clockwise-successor ownership, key 190 belongs to C at 240. Adding D at 210 moves it to D; the affected interval is (120,210]. Hash coordinates are illustrative.

Author-created illustration. Numbers are illustrative unless stated otherwise.

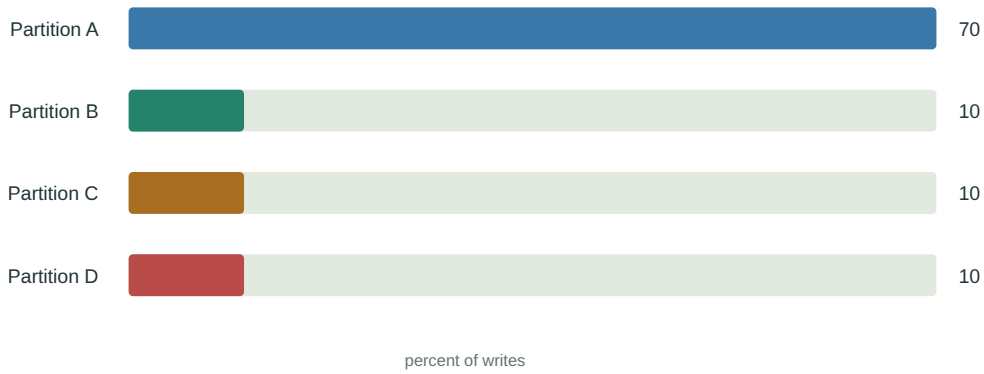
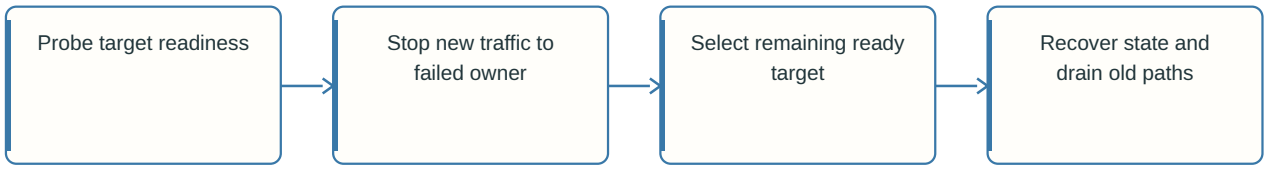


FIG. 12.0.3 **Averages hide a hot partition**

Seventy percent of writes go to A. Adding cold capacity does not split one hot ownership key.

Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 12.0.4 **load-balancer failover**

Routing failover does not by itself establish fresh state or preserve uncommitted work.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Scale compute & route requests

Prerequisites: Requirements, Estimation, and Engineering Trade-offs, Networking Foundations for System Design, Data Models and Storage Technologies, Database Internals, Transactions, and Concurrency Control, Caching and Content Delivery

6 original figures

Blue: normal flow · Green: healthy · Amber: caution · Red: failure · Purple: control / shared infrastructure · Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Adding application instances changes traffic distribution and often connection demand. Find which work is independent, where required state lives, and which signal should drive growth.

12.1.1 · Vertical vs horizontal scaling

A busy application can run on a larger machine or on more machines. Vertical scaling increases one node's resources. Horizontal scaling adds nodes and distributes suitable work.

Neither guarantees proportional throughput. A larger database machine can postpone distribution work, while extra application instances can share independent computation. A common lock or downstream database may remain the limit. Compare startup delay, failure impact, cost, and the work needed to operate the result. Profile the bottleneck first, make the smallest justified change, then measure again. The limiting resource can move after a successful improvement.

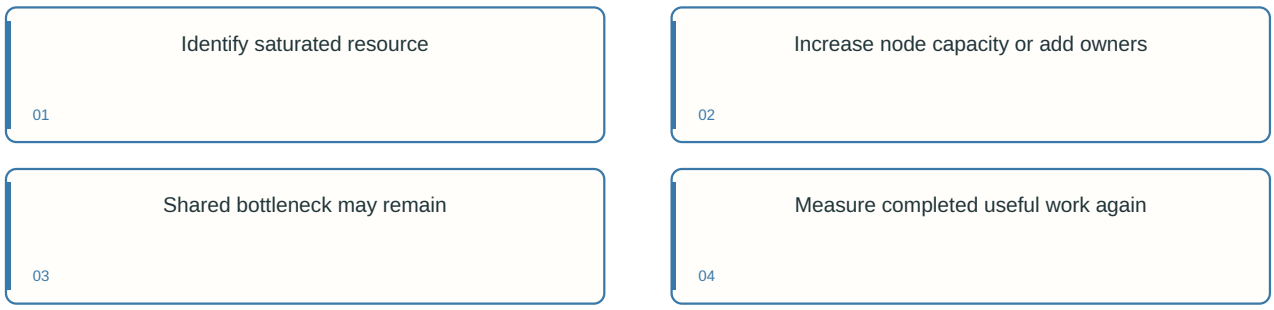


FIG. 12.1.1 **Vertical vs horizontal scaling · the example**

Vertical scaling enlarges a node; horizontal scaling adds nodes, with gains limited by the work that can divide.

Author-created illustration. Numbers are illustrative unless stated otherwise.

12.1.2 · Stateless vs stateful services

A request reaches a replacement web instance. Can that instance handle it using the same authenticated identity and stored state? A stateless application avoids keeping required per-client conversation state only in a particular instance between requests.

It still uses state elsewhere. Local sessions or accepted memory-only jobs can prevent transparent replacement. Move required state to a suitable owner or specify affinity and recovery. Check compatible configuration, state access, and duplicate effects across instances. Distributing requests freely works only when these conditions hold; labelling a service “stateless” does not establish them by itself.

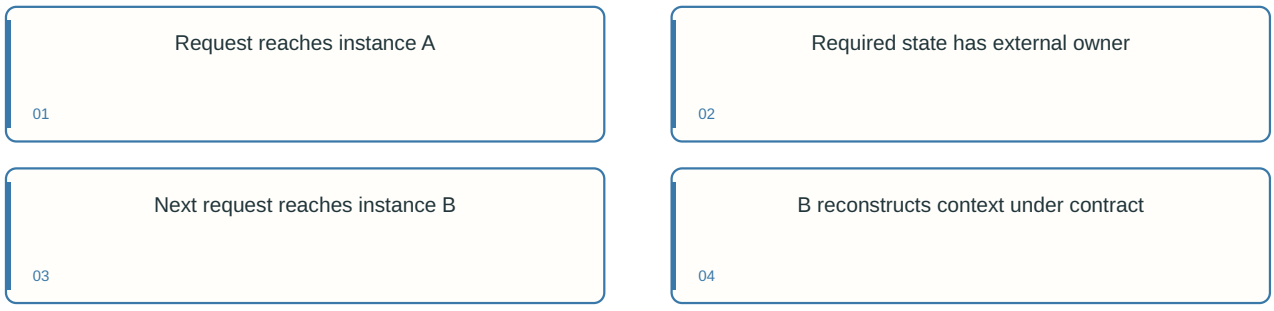


FIG. 12.1.2 **Stateless vs stateful services · the example**

Stateless instances keep no required per-client conversation only locally, while using appropriate external state owners.

Author-created illustration. Numbers are illustrative unless stated otherwise.

12.1.3 · Load distribution algorithms

Round-robin distributes backend selections in turn. Least-connections uses active connections as a load clue. Weights can represent different capacities, and hashing can keep related traffic together.

The best clue depends on the workload. One expensive request can occupy an instance longer than many cheap requests, while a long-lived connection can carry changing traffic. Inspect per-instance saturation and waiting instead of only counting requests. Client-side and server-side balancing also place discovery and retry decisions in different components. Choose a policy that reflects useful load, and remember that routing cannot add capacity to a shared downstream bottleneck.

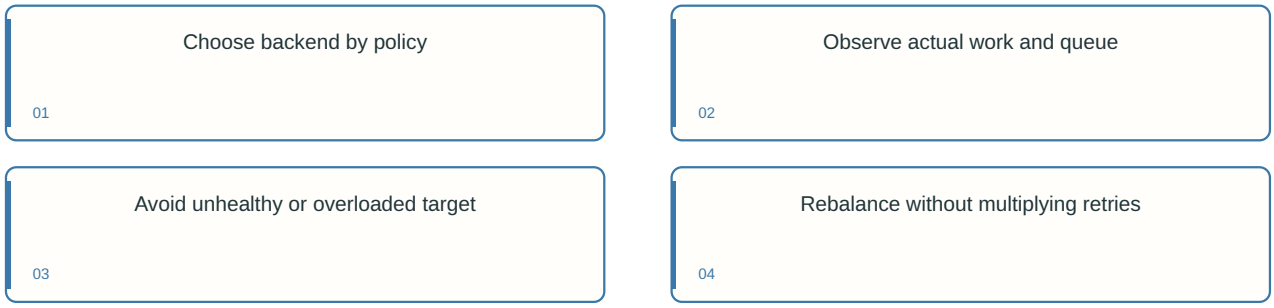


FIG. 12.1.3 **Load distribution algorithms · the example**

Balancing algorithms distribute selections using different load clues whose usefulness depends on request and connection behaviour.

Author-created illustration. Numbers are illustrative unless stated otherwise.

12.1.4 · Health checks and failover

A process responds to a simple ping, but cannot serve real traffic. Another process can serve a reduced response even though a shared optional dependency is down. Health checks need to reflect the routing decision they control.

Liveness concerns whether a process should be restarted. Readiness concerns whether it should receive traffic under the chosen policy. Requiring every dependency in every readiness check can remove all instances during one shared outage. Use bounded, meaningful checks and deliberate degradation. A traffic switch also needs valid state and connection behaviour; routing to a new instance does not recover work that existed only in the old one's memory.

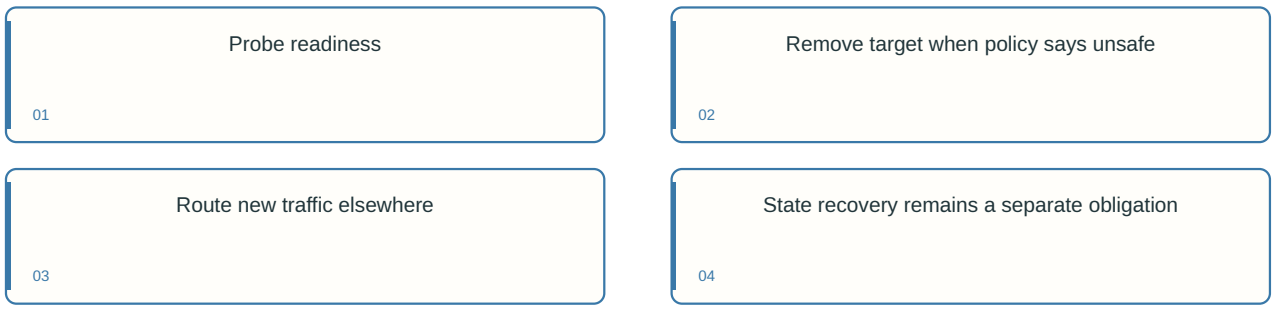


FIG. 12.1.4 **Health checks and failover · the example**

Readiness, liveness, and failover make different decisions and require state and capacity beyond a probe response.

Author-created illustration. Numbers are illustrative unless stated otherwise.

12.1.5 · Sticky sessions

Sticky sessions send a client's requests to the same instance. This can support local session state or improve cache reuse, but also creates an affinity that affects balancing and recovery.

If that instance fails, decide how the client reconnects and how required state is recovered. Heavy users may create uneven load. For replaceable web instances, externally owned sessions can remove some affinity needs. Where sticking is deliberate, document the consequences rather than relying on it invisibly. Affinity can be an optimisation or a constraint; the authoritative location and lifetime of session state still need a clear design.

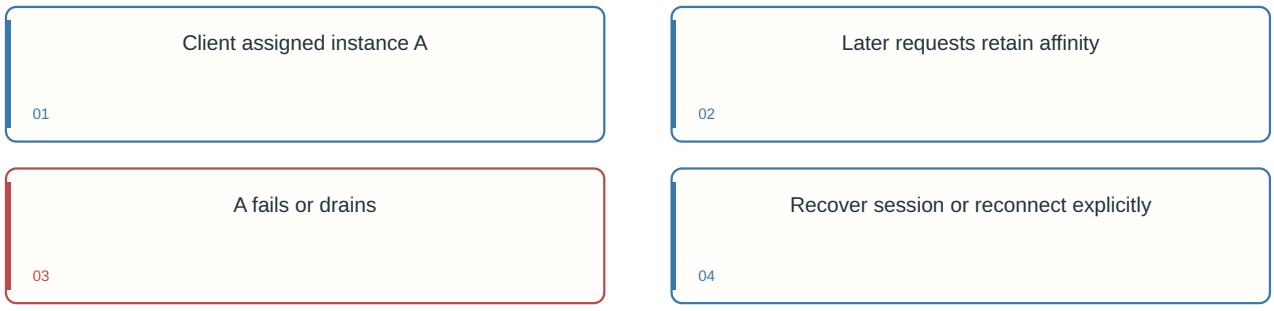


FIG. 12.1.5 **Sticky sessions · the example**

Sticky sessions preserve affinity while adding load-skew and recovery consequences for locally held state.

Author-created illustration. Numbers are illustrative unless stated otherwise.

12.1.6 · Autoscaling metrics

A worker queue's oldest job is getting older even while CPU looks moderate. Scaling solely on CPU might miss the delay users care about. The signal should reflect the constrained work: CPU for computation, queue age for job delay, or connections for some waiting workloads.

Autoscaling also has startup time, minimums, and maximums. Use stabilisation to avoid repeatedly adding and removing instances, and cap spending and downstream pressure. Request-count scaling assumes comparable request costs. Keep existing headroom and admission controls for bursts that arrive before new capacity is ready. Automation reacts to a limit; it does not remove the reaction interval.

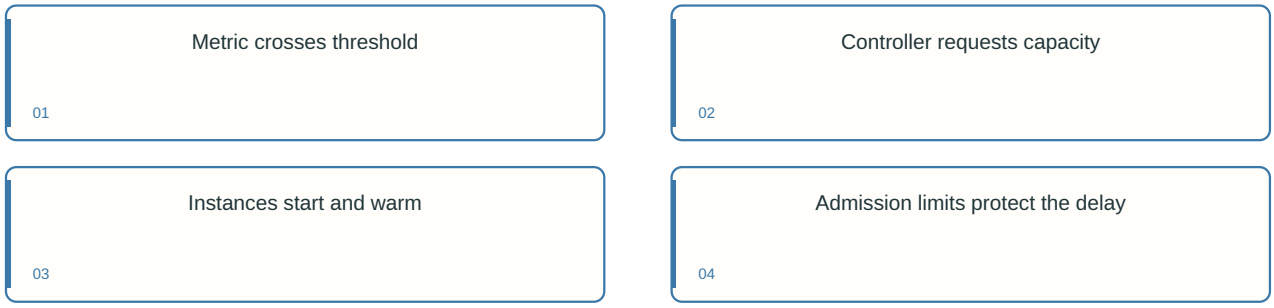


FIG. 12.1.6 **Autoscaling metrics · the example**

Autoscaling reacts to workload signals with delay, bounds, warmup, and downstream-capacity constraints.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · Can more partitions scale one globally hot exclusive ownership decision?

Cold partitions do not divide the authority required for one exclusive key. Read copies or batching can help suitable work, while changing the invariant or allocation model needs a valid protocol. Measure the hot owner and explain how any proposed partition changes its actual work without permitting conflicting winners.

Remember: Scale the constrained work while bounding shared downstream demand.

Try applying it: What must remain true before any healthy instance can serve the next request?

Unit 12 exercises → · Unit 12 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Application best practices — Microsoft Architecture Center ↗

   Official guidance · accessed 21 Oct 2026. Scaling and application boundary review.

   Access reviewed · 21 October 2026

2. Dynamo: Amazon’s highly available key-value store, 2007 ↗

   Original research · author organization. Replication, partitioning, and reconciliation ideas; capstones are original designs, not Dynamo deployments.

   Access reviewed · 21 October 2026

Replicate and partition data

Prerequisites: 12.1 · Scale compute & route requests 6 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A dataset no longer fits its original owner. Compare copies, partitions, stable key placement, and ownership movement without assuming that evenly stored bytes imply evenly distributed traffic.

12.2.1 · Replication

A primary can copy writes to a replica before or after replying. In asynchronous replication, a reply may precede receipt on the replica. If that replica is promoted after losing the primary, recent acknowledged writes may be absent.

Synchronous arrangements wait for a defined replica milestone and can increase delay or block during communication failure. State exactly which copies and milestones acknowledgment requires. Replication can improve reading or continuity while adding lag and recovery rules. Independent failure placement also matters: two copies on one lost disk provide little protection. A copy count is useful only alongside its protocol and failure assumptions.

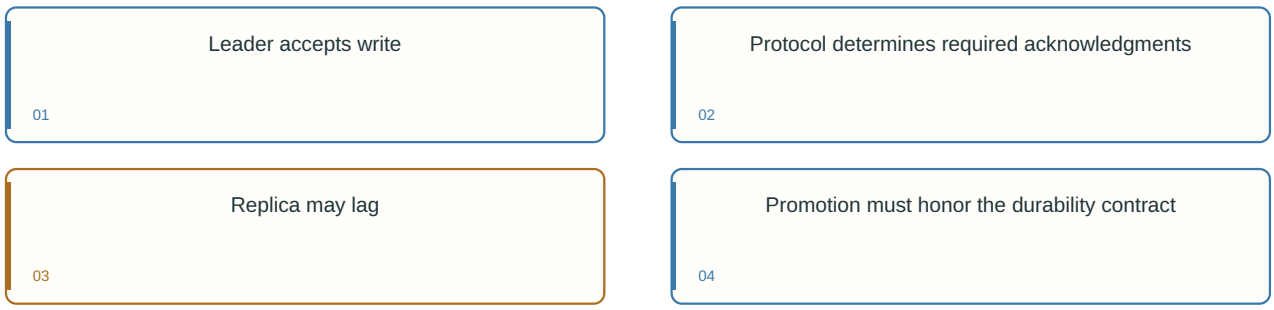


FIG. 12.2.1 **Replication · the example**

Replication maintains copies under explicit acknowledgment, authority, lag, and recovery rules.

Author-created illustration. Numbers are illustrative unless stated otherwise.

12.2.2 · Partitioning and sharding

A dataset can be divided into partitions. When separate storage owners hold those portions, this is commonly called sharding. The key determines where data lives and which queries need several owners.

Tenant-based placement may keep each tenant's data together while leaving one large tenant hot. Cross-shard uniqueness, joins, transactions, and movement need additional design. Keep a routing and relocation procedure, and name where invariants are enforced. Add this complexity when a measured storage or write problem justifies it. A large user count alone does not identify a partition key or establish that one database has reached its limit.

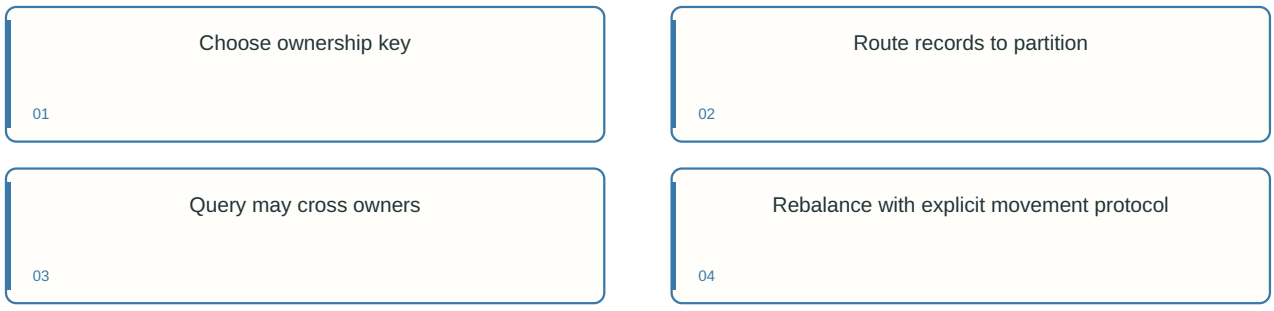


FIG. 12.2.2 **Partitioning and sharding · the example**

Sharding distributes partitions among owners, adding routing, query, transaction, and movement requirements.

Author-created illustration. Numbers are illustrative unless stated otherwise.

12.2.3 · Range partitioning vs hash partitioning

Range partitioning keeps ordered keys together, which can help scans and expiration by time. But if every new event goes into today's range, that owner receives most writes. Hash partitioning spreads keys while making ordered range reads less local.

A time bucket plus a spread dimension can distribute ingestion, at the cost of reading several partitions for one time range. State that fan-out explicitly. Neither technique automatically splits one extremely popular key. Choose by the writes, reads, retention, and skew the application actually has, rather than using “balanced storage” as a substitute for measuring traffic.

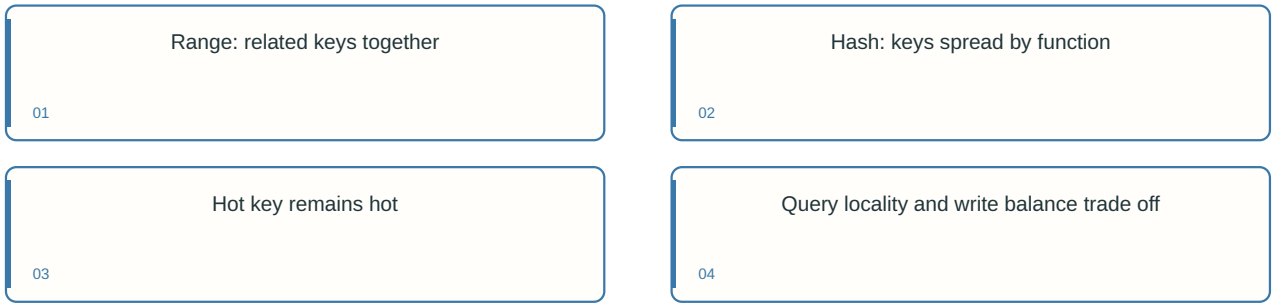


FIG. 12.2.3 **Range partitioning vs hash partitioning · the example**

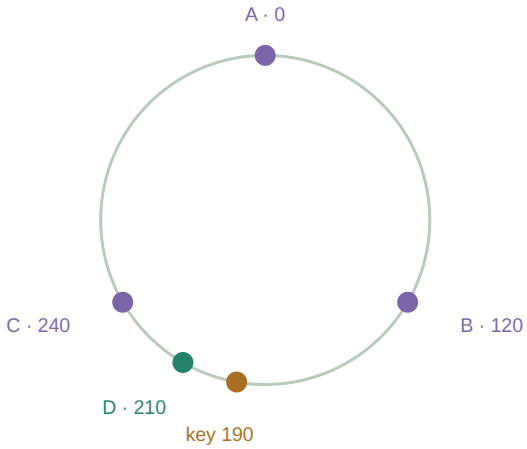
Range placement supports ordered locality; hashing spreads keys, with different scan and hot-key trade-offs.

Author-created illustration. Numbers are illustrative unless stated otherwise.

12.2.4 · Consistent hashing and rendezvous hashing

Modulo routing can move many keys when the owner count changes. Consistent hashing uses a stable placement, often a ring with virtual positions, to limit reassignment. Rendezvous hashing instead scores each key-owner pair and selects the highest eligible score.

Both aim to keep much of the old assignment while membership changes. Neither guarantees equal request traffic: one key can be far more popular than others. Clients must agree about membership and hash behaviour, and data still needs safe movement and replication. Changing the hash function also changes placement, so treat it as a migration rather than a harmless implementation detail.



Clockwise successor: key 190 moves from C to new owner D.

FIG. 12.2.4 **A new owner takes only part of the hash space**

With clockwise-successor ownership, key 190 belongs to C at 240. Adding D at 210 moves it to D; the affected interval is (120,210]. Hash coordinates are illustrative.

Author-created illustration. Numbers are illustrative unless stated otherwise.

12.2.5 · Rebalancing

Moving a partition while requests continue is a coordination operation. Copy its data from a defined starting point, capture changes made during the copy, validate the destination, and establish when it becomes the owner.

Readers and writers need a consistent way to discover that transition, such as a routing epoch. Throttle copying so it leaves capacity for users. Keep the old copy until the new ownership and recovery conditions permit removal. Plan rollback and reconciliation too. A move that deletes the source before proving the destination current can turn a capacity improvement into data loss.

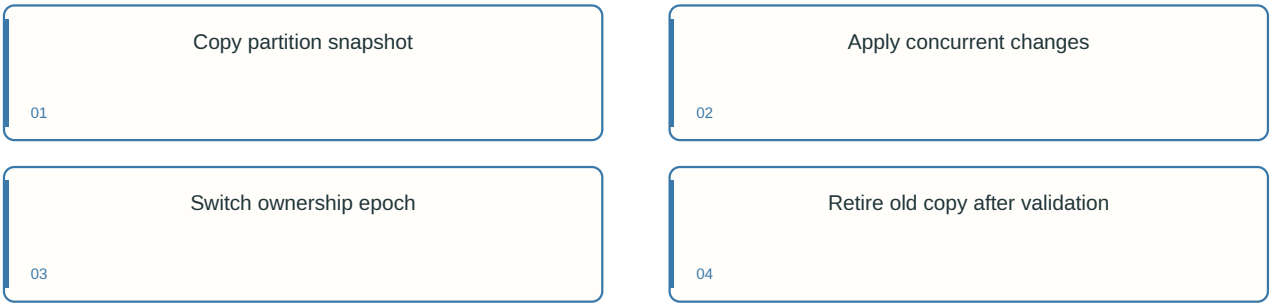


FIG. 12.2.5 **Rebalancing · the example**

Rebalancing transfers current data and authority while preserving concurrent writes, routing, validation, and recovery.

Author-created illustration. Numbers are illustrative unless stated otherwise.

12.2.6 · Hot partitions and skew

Fleet averages look healthy, but one partition handles 70% of writes. More cold partitions do little for that owner. Inspect requests, bytes, and delays by partition to find the actual skew.

A range split helps only if the traffic can divide. One hot entity may require read copies, batching, or a redesigned ownership rule. Randomising writes can spread append pressure but force reads to combine more partitions. Determine whether the issue is volume, size, or a serialised invariant. Choose a remedy that changes that pressure, rather than repeatedly buying capacity elsewhere in the cluster.

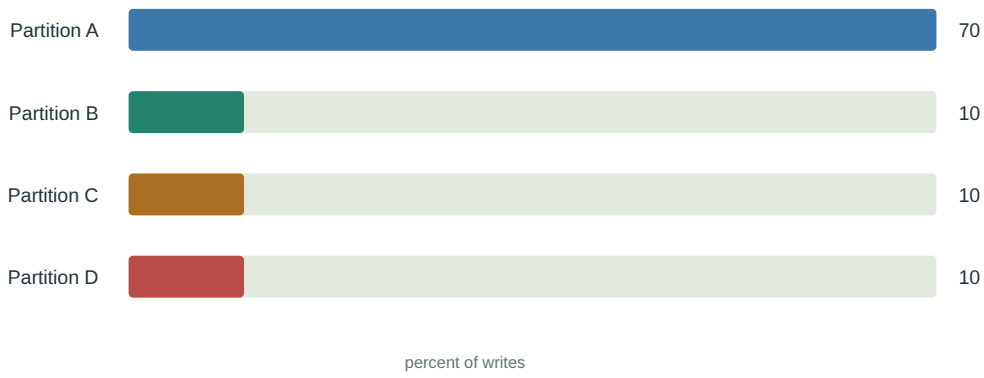


FIG. 12.2.6 **Averages hide a hot partition**

Seventy percent of writes go to A. Adding cold capacity does not split one hot ownership key.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · Can more partitions scale one globally hot exclusive ownership decision?

Cold partitions do not divide the authority required for one exclusive key. Read copies or batching can help suitable work, while changing the invariant or allocation model needs a valid protocol. Measure the hot owner and explain how any proposed partition changes its actual work without permitting conflicting winners.

Remember: Data distribution needs query, invariant, routing, and migration rules as well as a key function.

Try applying it: Why does a new hash owner not automatically solve one extremely popular key?

Unit 12 exercises → · Unit 12 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Application best practices — Microsoft Architecture Center ↗

   Official guidance · accessed 21 Oct 2026. Scaling and application boundary review.

   `Access reviewed · 21 October 2026`

2. Dynamo: Amazon’s highly available key-value store, 2007 ↗

   Original research · author organization. Replication, partitioning, and reconciliation ideas; capstones are original designs, not Dynamo deployments.

   `Access reviewed · 21 October 2026`

Lag, proxies & saturated pools

Prerequisites: 12.2 · Replicate and partition data 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A replica receives a write but has not applied it, while applications wait for connections. Follow actual progress and ownership before changing routing or increasing pool sizes.

12.3.1 · Read replicas and read/write splitting

A user updates a profile on the primary, then reads from a replica that has not applied the change. The next response shows the old value. Read/write splitting needs a freshness policy.

Route critical reads to a sufficiently current owner, track session progress where supported, or explicitly allow delayed results. Classifying by SQL verb alone is unsafe: a SELECT may acquire a lock or participate in a transaction requiring current state. Read replicas can distribute appropriate reads while writes remain elsewhere. Make the operation's required observations part of the routing decision, rather than assuming every read can go to any replica.

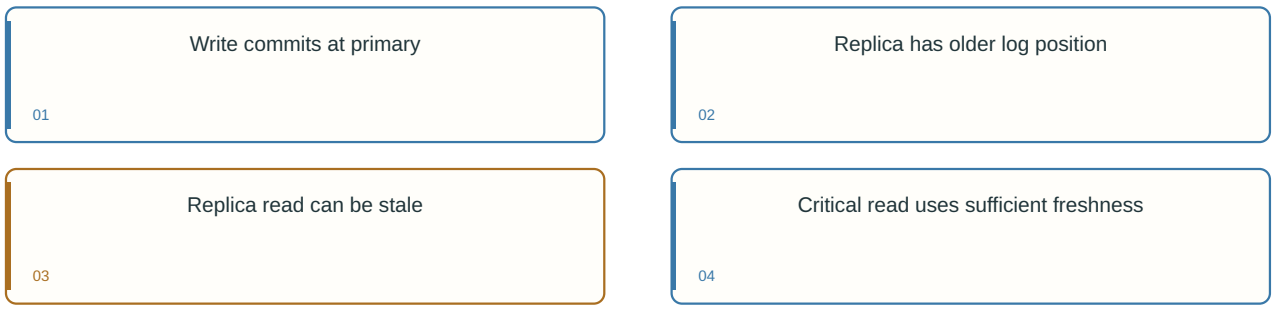


FIG. 12.3.1 **Read replicas and read/write splitting · the example**

Read/write splitting must respect observation and transaction needs, not merely classify by SQL verb.

Author-created illustration. Numbers are illustrative unless stated otherwise.

12.3.2 · Replication lag

A source has accepted changes through position 100, but a replica has applied only through 90. This position difference describes part of its replication lag. The transport may have received newer records that still wait to be applied.

Measure source, receipt, and apply progress where the system supports them. A small average can hide one stalled replica. Time-based lag estimates also need interpretation when traffic varies. Promotion must respect the acknowledgment and possible-loss contract, not just happen quickly. For a user needing read-your-writes, route according to the required version or another supported mechanism rather than relying on the fleet's average freshness.

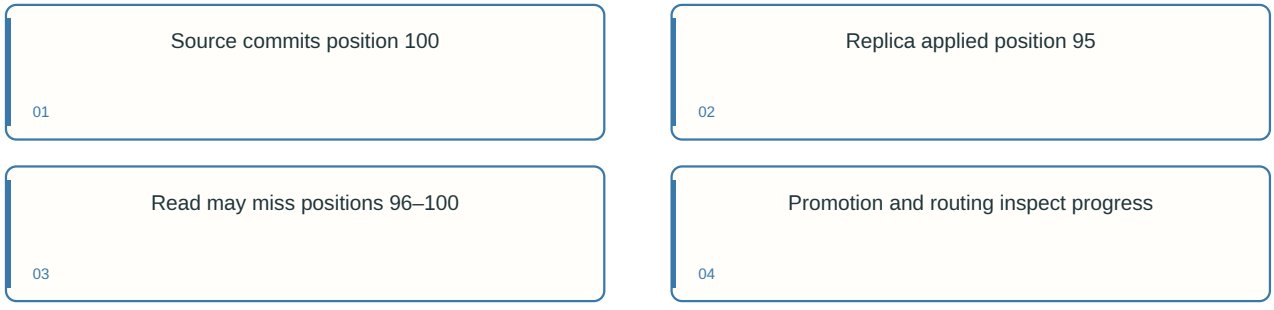


FIG. 12.3.2 **Replication lag · the example**

Replica receipt, persistence, and apply progress can differ and support different freshness or recovery claims.

Author-created illustration. Numbers are illustrative unless stated otherwise.

12.3.3 · Database proxying

A database proxy can route clients or reuse backend connections. Some implementations multiplex several client sessions onto fewer database connections. This can change assumptions about session variables, temporary tables, or long transactions.

Check which state must stay attached to a particular backend. Document where identity, failover, and routing decisions occur, and size the proxy itself. It can become another constrained or failed component. Pooling does not remove transaction limits, and a failed call cannot always be transparently repeated after a connection switch. Choose proxy features against the application's actual session and operation contract.

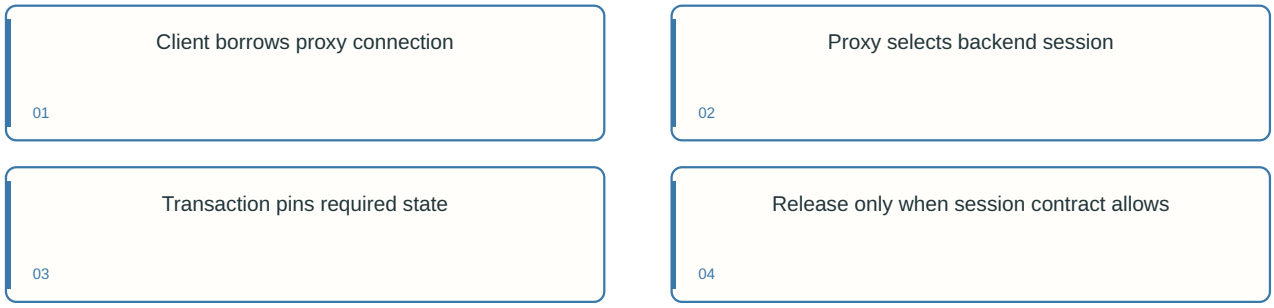


FIG. 12.3.3 **Database proxying · the example**

Database proxies add routing or pooling facilities whose session-state and retry contracts need verification.

Author-created illustration. Numbers are illustrative unless stated otherwise.

12.3.4 · Connection pool saturation

Ten application instances each open a pool of twenty database connections. Adding ten more instances can double connection demand even if the database has no more capacity. Waiting callers may then accumulate both in application pools and inside the database.

Bound the aggregate, reserve recovery capacity, and include acquisition waits in deadlines. A request waiting on a database lock may hold a connection while using little CPU. Measure hold time and contention before increasing pool sizes. Scaling the admission points can worsen an outage when it increases pressure on the shared resource already causing the wait.

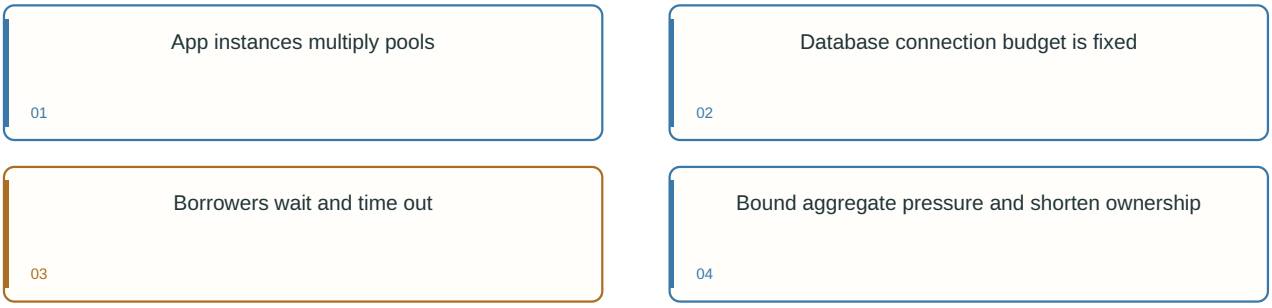


FIG. 12.3.4 **Connection pool saturation · the example**

Pool saturation includes waiting and held connections; aggregate demand across application replicas must remain bounded.

Author-created illustration. Numbers are illustrative unless stated otherwise.

12.3.5 · Bottleneck identification

A request spends most of its time waiting for one database query. Adding web instances is unlikely to fix that path; a query or index change may help more. Another service may instead be limited by independent CPU work and benefit from workers.

Trace useful completions and waiting through the system. Compare CPU, memory, disk, network, connections, and lock contention with offered load and queues. High utilisation is a clue, not proof of the business bottleneck. Change one material factor and measure again. This links an intervention to evidence and shows whether the limit moved elsewhere.

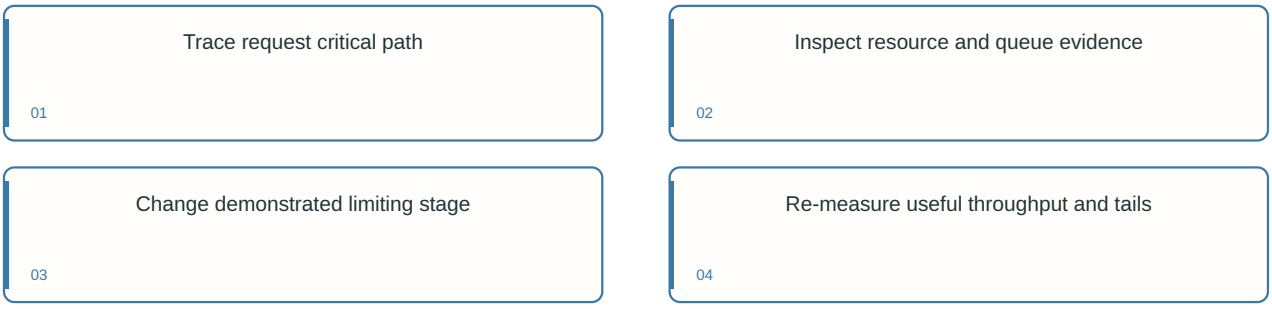


FIG. 12.3.5 **Bottleneck identification · the example**

Bottleneck diagnosis connects useful outcomes and critical-path waits to constrained resources.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · Can more partitions scale one globally hot exclusive ownership decision?

Cold partitions do not divide the authority required for one exclusive key. Read copies or batching can help suitable work, while changing the invariant or allocation model needs a valid protocol. Measure the hot owner and explain how any proposed partition changes its actual work without permitting conflicting winners.

Remember: Read freshness and capacity diagnosis depend on the real progress and wait boundaries.

Try applying it: How can doubling application replicas overload the same database?

Unit 12 exercises → · Unit 12 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Application best practices — Microsoft Architecture Center ↗

   Official guidance · accessed 21 Oct 2026. Scaling and application boundary review.

   `Access reviewed · 21 October 2026`

2. Dynamo: Amazon’s highly available key-value store, 2007 ↗

   Original research · author organization. Replication, partitioning, and reconciliation ideas; capstones are original designs, not Dynamo deployments.

   `Access reviewed · 21 October 2026`

Regions, write paths & scale economics

Prerequisites: 12.3 · Lag, proxies & saturated pools 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

The service needs more capacity and perhaps another region. Compare queues, failure placement, read and write rules, and total operating cost under equal requirements.

12.4.1 · Queueing and overload

A queue can absorb a brief burst when workers later catch up. If arrival stays above completion, the backlog keeps growing. Longer waiting can make a request useless even before memory runs out.

Bound both queue length and age, prioritise required work, and reject or shed work that cannot meet its deadline. A request with a two-second deadline should not be accepted silently behind ten seconds of work. Explain the overload result so callers do not immediately create a retry storm. A queue is a place for controlled waiting, not a way to turn insufficient sustained capacity into sufficient capacity.

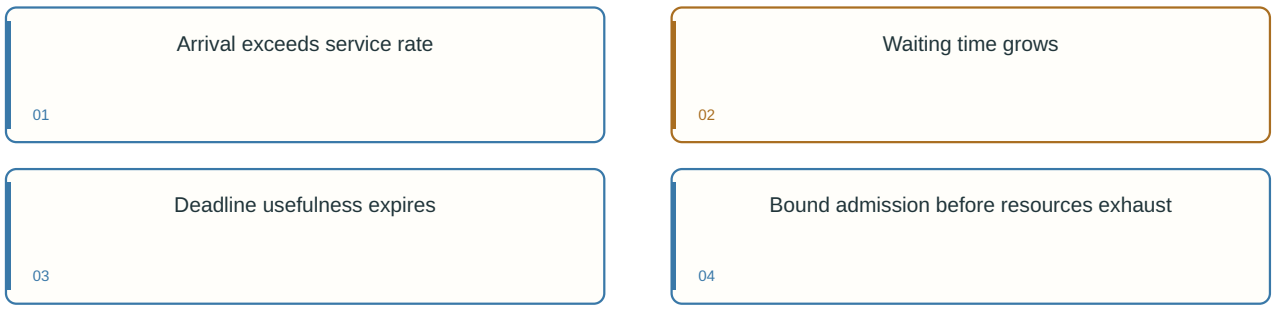


FIG. 12.4.1 **Queueing and overload · the example**

Queues absorb temporary mismatch but sustained overload requires admission, capacity, or workload changes.

Author-created illustration. Numbers are illustrative unless stated otherwise.

12.4.2 · Single points of failure

Two application containers run on one host. They survive a single process failure but not the host's failure. A single point of failure is a dependency whose loss defeats the required service, which may be a machine, router, credential source, or operator-controlled procedure.

Inspect independent failure domains rather than counting process boxes. Redundancy also needs detection, valid state, surviving capacity, and failover. Check recovery dependencies: a data path may be replicated while every replacement instance needs one unavailable configuration service to start. Trace what must work during a failure, not only what runs during normal operation.

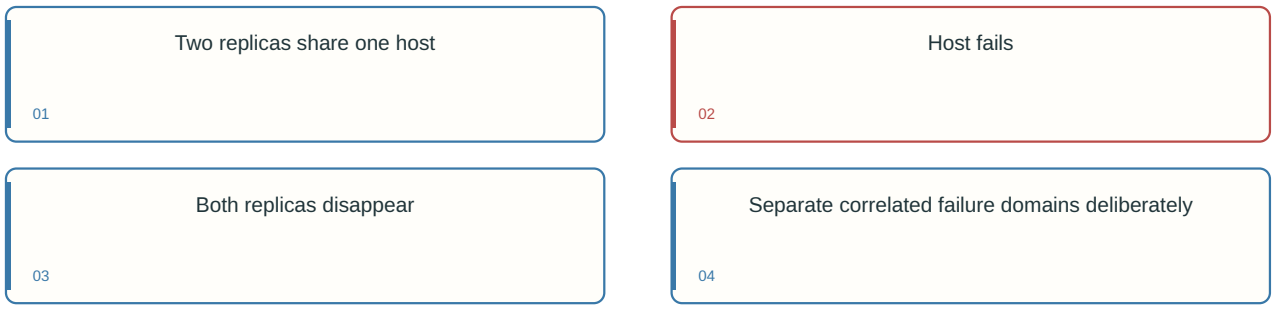


FIG. 12.4.2 **Single points of failure · the example**

A single point of failure can be any shared dependency defeating the required service or recovery.

Author-created illustration. Numbers are illustrative unless stated otherwise.

12.4.3 · Multi-region strategies

A second region can help locality or continuity, but it changes where writes are accepted and how copies agree. Active-passive keeps one current authority and needs promotion and recovery rules. Active-active needs a precise coordination or conflict policy for concurrent activity.

Waiting across regions adds propagation time. Copying asynchronously reduces that wait but may lose recent acknowledged changes during failover. Begin with tolerated interruption, data loss, and locality requirements. Choose write authority and read freshness per operation. A single “multi-region” label cannot establish which work continues during a partition or which state survives it.

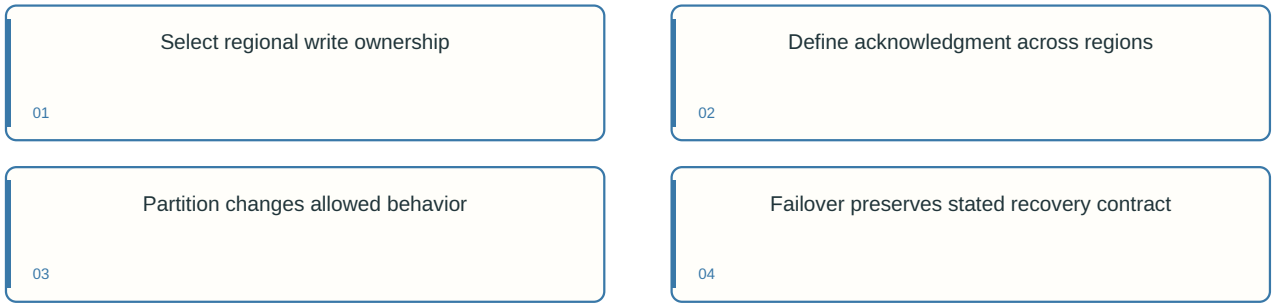


FIG. 12.4.3 **Multi-region strategies · the example**

Multi-region operation needs write authority, read freshness, loss, latency, and rejoin policies.

Author-created illustration. Numbers are illustrative unless stated otherwise.

12.4.4 · Scaling writes vs scaling reads

A product description can be copied to many readers with an allowed freshness delay. Allocating the last seat needs exclusive ownership of that seat. These operations have different scaling options.

Read scaling often uses suitable replicas or caches. Write scaling must divide, reduce, or batch ownership work while preserving its invariants. One hot account or global sequential allocator can remain serialised despite many shards. First remove unnecessary coordination or indexes where justified. Commutative counters may support independent updates; exclusive reservations need a protocol that prevents incompatible winners. Scale the actual rule, rather than assuming reads and writes benefit from the same arrangement.

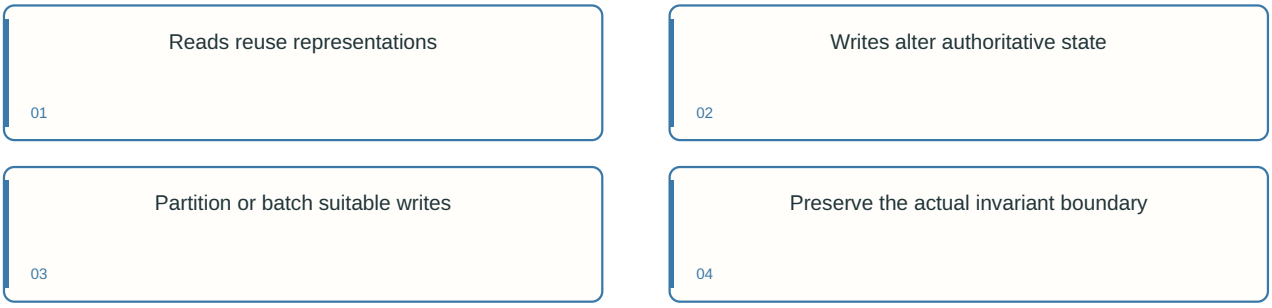


FIG. 12.4.4 **Scaling writes vs scaling reads · the example**

Read scaling can reuse copies; write scaling must divide or reduce ownership work while preserving invariants.

Author-created illustration. Numbers are illustrative unless stated otherwise.

12.4.5 · Scale-out economics

Ten small nodes introduce repeated baseline memory, extra connections, transfer, monitoring, and operating work. They may or may not cost less than one larger node for the required workload.

Compare candidates with equal service targets and spare capacity for the same failures. Include startup, cache warming, migration, and recovery. Managed services can exchange provider charges for some staffing work, but the application still owns its correctness. Cost per successful business outcome is more useful than cost per attempted request when retries differ. Identify the bottleneck before purchasing capacity that cannot increase useful completions.

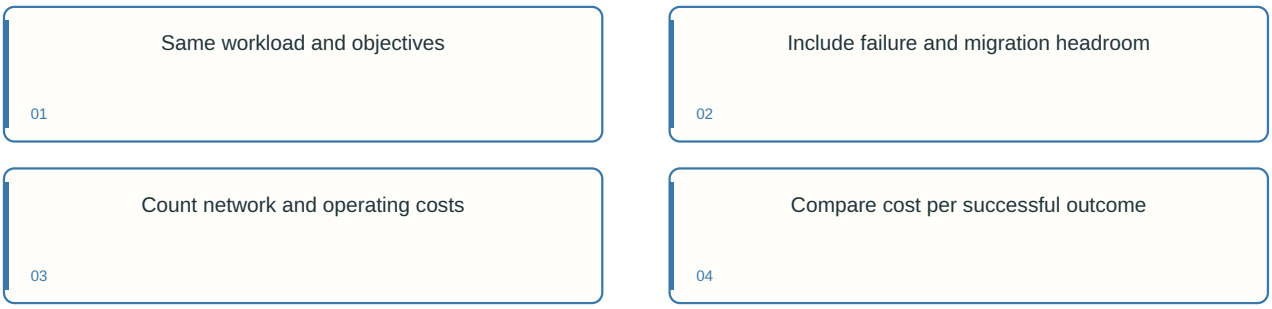


FIG. 12.4.5 **Scale-out economics · the example**

Scale economics includes surviving capacity, coordination, transfer, baseline resources, and operating effort per useful outcome.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Try the mechanism in code

PYTHON · STABLE RENDEZVOUS MAPPING

```
from mechanisms import rendezvous
for key in range(1000):
    old = rendezvous(key, ["A","B","C"])
    new = rendezvous(key, ["A","B","C","D"])
    assert old == new or new == "D"
```

For each key, adding D either preserves the old owner or assigns the key to D. The assertion checks that stable-mapping property across the fixture.

It does not move stored data or synchronise client membership. A popular key still has one selected owner in this model, regardless of how balanced the key counts look.

Download the executable mechanism collection ↓ · Follow the connected complete design →

THE WORKBENCH · TRY CHANGING THE INPUTS

Compare owners before and after adding D

Key 190: old owner **B**; new owner **B**.

Across keys 0–999, 246 change owners. Counts after adding D: A=228, B=252, C=274, D=246.

Every changed key moves to D; keys never move between A, B, and C here. This assignment says nothing about traffic per key or how data is copied.

Change the key and compare its owner before and after D joins. The same key always receives the same scores in this example. Choosing a new owner does not copy data, agree on membership, or spread the traffic of a single popular key.

▼ Think further · Can more partitions scale one globally hot exclusive ownership decision?

Cold partitions do not divide the authority required for one exclusive key. Read copies or batching can help suitable work, while changing the invariant or allocation model needs a valid protocol.

Measure the hot owner and explain how any proposed partition changes its actual work without permitting conflicting winners.

Remember: A bigger topology helps only when its capacity, authority, and failure behaviour match the need.

Try applying it: Which write invariant would remain concentrated despite adding many cold shards?

Unit 12 exercises → · Unit 12 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Application best practices — Microsoft Architecture Center ↗

   Official guidance · accessed 21 Oct 2026. Scaling and application boundary review.

   `Access reviewed · 21 October 2026`

2. Dynamo: Amazon’s highly available key-value store, 2007 ↗

   Original research · author organization. Replication, partitioning, and reconciliation ideas; capstones are original designs, not Dynamo deployments.

   `Access reviewed · 21 October 2026`

PART 3 / UNIT 13

File Systems, Search Infrastructure, and Data Pipelines

A client uploads bytes while the application records metadata. Make both identities and their handoff explicit so an interrupted transfer cannot become verified processing input.

The running example

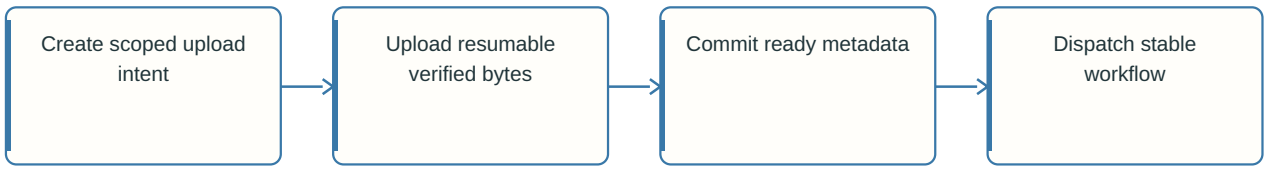
Design a document-processing system accepting millions of files, tracking processing state, extracting metadata, maintaining provenance, and exposing searchable results.

Unit 13 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.



Arrows: ordered stages of this worked mechanism

FIG. 13.0.1 **Upload pipeline**

Input bytes and control metadata have different owners and failure windows.

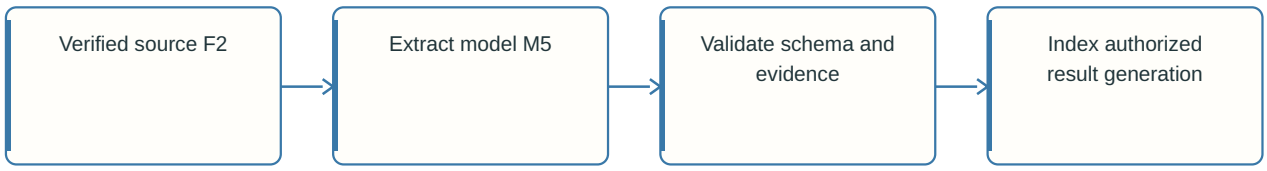
Author-created illustration. Numbers are illustrative unless stated otherwise.

Source / term	Representation	Meaning
D1	blue car	document text
D2	blue bike	document text
blue	D1, D2	posting list
car	D1	posting list

FIG. 13.0.2 **An inverted index is a term-to-document map**

Token analysis creates the lookup representation. Source versions and permissions remain separately owned.

Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 13.0.3 **ingestion DAG**

A stage’s versioned artifact and durable transition make replay inspectable.

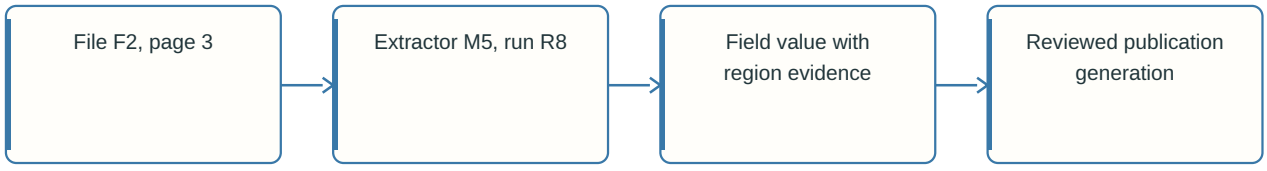
Author-created illustration. Numbers are illustrative unless stated otherwise.

Property	Batch	Streaming
Input	Bounded set	Continuing events
Freshness	Scheduled recomputation	Progress and lateness policy
Recovery	Replay bounded batch	Restore state and offsets

FIG. 13.0.4 **batch vs stream timeline**

Neither mode is automatically correct; output semantics and replay identity must be explicit.

Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 13.0.5 **lineage graph**

Lineage connects exact source and transform identities through derived artifacts.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Upload bytes without losing identity

Prerequisites: API Design and Communication Contracts, Data Models and Storage Technologies, Database Internals, Transactions, and Concurrency Control

5 original figures

Blue: normal flow · Green: healthy · Amber: caution · Red: failure · Purple: control / shared infrastructure · Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A client uploads bytes while the application records metadata. Make both identities and their handoff explicit so an interrupted transfer cannot become verified processing input.

13.1.1 · Binary/object storage

A processing job needs both a PDF's bytes and information about its owner and progress. Store immutable binary content separately when its access and lifecycle differ from the mutable job record.

The record should identify a particular object version, its size or checksum, and ownership. A row pointing to a missing or incomplete object is a broken workflow even if both storage services individually operated correctly. Track upload state and clean up abandoned bytes. Verify the chosen object store's guarantees; “object storage” names an interface category rather than a universal consistency or durability contract.

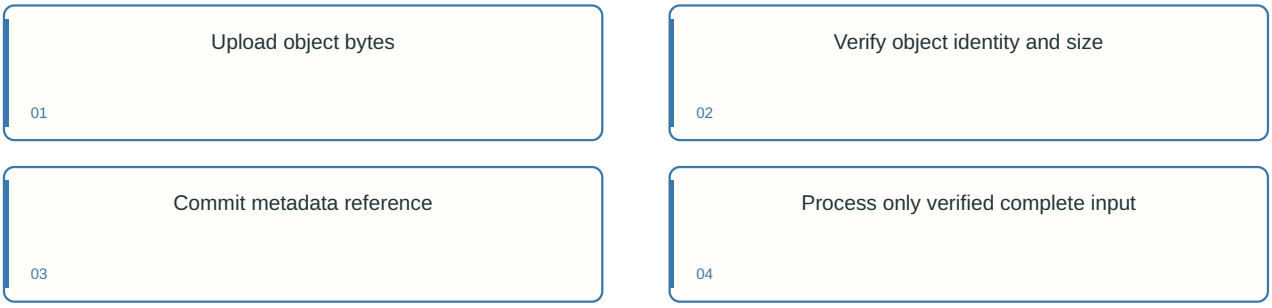


FIG. 13.1.1 **Binary/object storage · the example**

Large binary objects and mutable workflow metadata need stable identities and an explicit lifecycle handoff.

Author-created illustration. Numbers are illustrative unless stated otherwise.

13.1.2 · Large-file upload architecture

Instead of sending a large upload through application workers, the service can issue a narrowly scoped authorisation for the client to stream directly to storage. The application still owns permission, quotas, metadata, and the decision that a file is ready.

A client may abandon the upload or send different bytes than expected. Validate completion, size, and ownership before starting processing. Expire unused upload intents and reconcile unreferenced objects. A signed upload URL is a capability: its scope and lifetime determine what its holder may do. Grant only the intended object operation, rather than broad access to the storage collection.

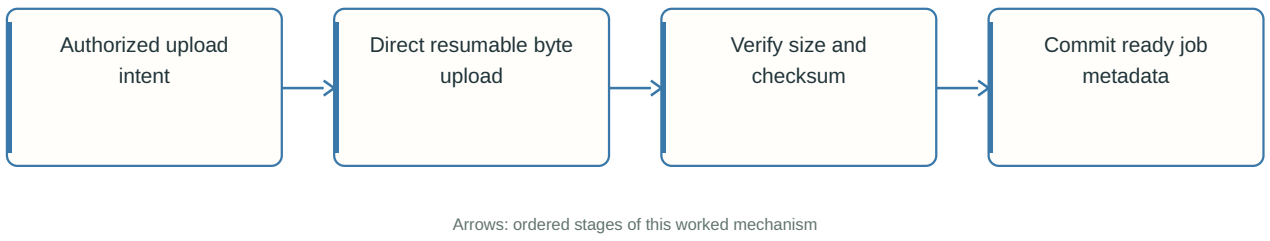


FIG. 13.1.2 **The upload pipeline separates bytes from control**

Incomplete uploads never enter processing as verified input. Metadata failure leaves cleanup/reconciliation work rather than a fictitious cross-service transaction.

Author-created illustration. Numbers are illustrative unless stated otherwise.

13.1.3 · Multipart and resumable uploads

An interrupted large upload should not always restart from byte zero. Multipart upload divides it into named parts and has a separate completion step. Resumable upload remembers confirmed offsets or parts so the client can continue.

Specify how retries identify a part and what happens if the client's source file changes. Acknowledging every part does not by itself establish that the assembled object is the right complete file. Verify the final content under the chosen protocol. Provider entity tags are not universally content hashes. Remove expired incomplete sessions, or abandoned parts can create storage growth that no visible completed file explains.

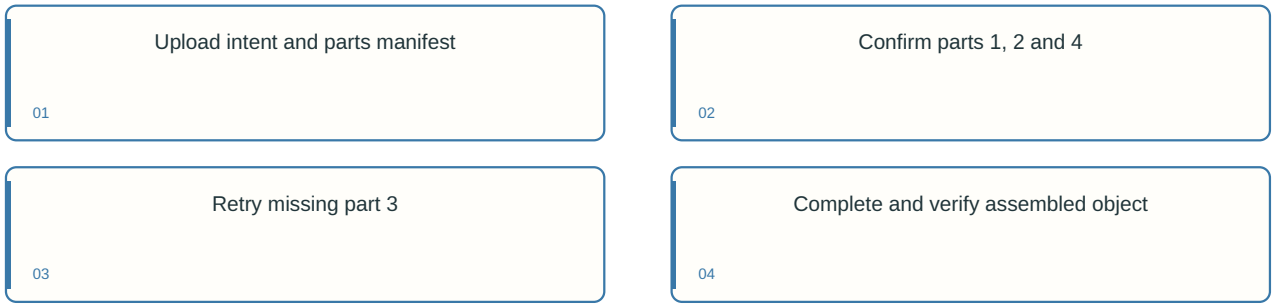


FIG. 13.1.3 **Multipart and resumable uploads · the example**

Resumable or multipart upload tracks confirmed pieces and separately verifies the completed object's identity and integrity.

Author-created illustration. Numbers are illustrative unless stated otherwise.

13.1.4 · Checksums and content addressing

A downloaded file differs from the uploaded bytes. A checksum can help detect corruption according to its algorithm's properties. A suitable cryptographic digest can also identify content under stated collision assumptions.

Content addressing derives an identity from that digest, supporting immutable references or deduplication. Verify the actual stored or received bytes rather than hashing only a client's claim. Matching content does not establish permission to access it or that the content is safe. Cross-tenant deduplication can reveal information through existence checks, so design its scope carefully. Integrity, identity, authorisation, and content safety are related but separate decisions.

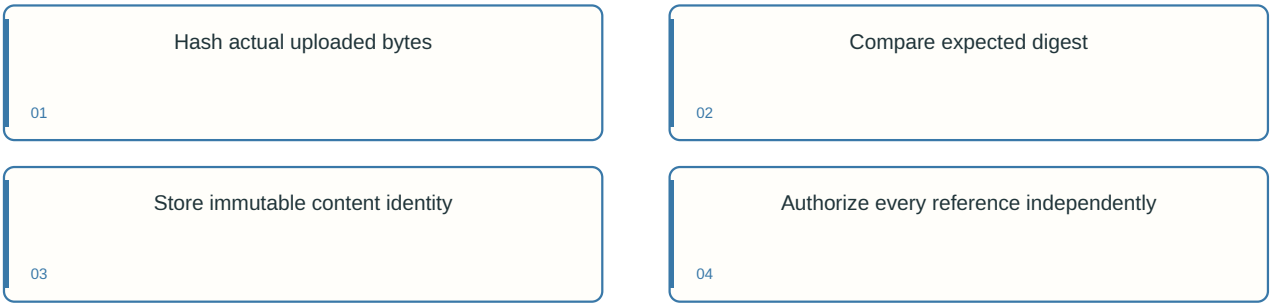


FIG. 13.1.4 **Checksums and content addressing · the example**

Checksums detect corruption under algorithm assumptions; content addressing identifies bytes without establishing permission or safety.

Author-created illustration. Numbers are illustrative unless stated otherwise.

13.1.5 · Metadata vs binary data

A user replaces a document after an extraction completed. If the same input identifier silently refers to new bytes, we can no longer explain which file produced the earlier result.

Give binary versions stable identities. Keep annotations, classifications, attempts, and review status in metadata that names those versions. When the input changes, create a new version and decide which derived results must be recomputed. Retain or remove historical artifacts according to policy. This separation lets the application explain an output without treating mutable processing state as part of the immutable original file.

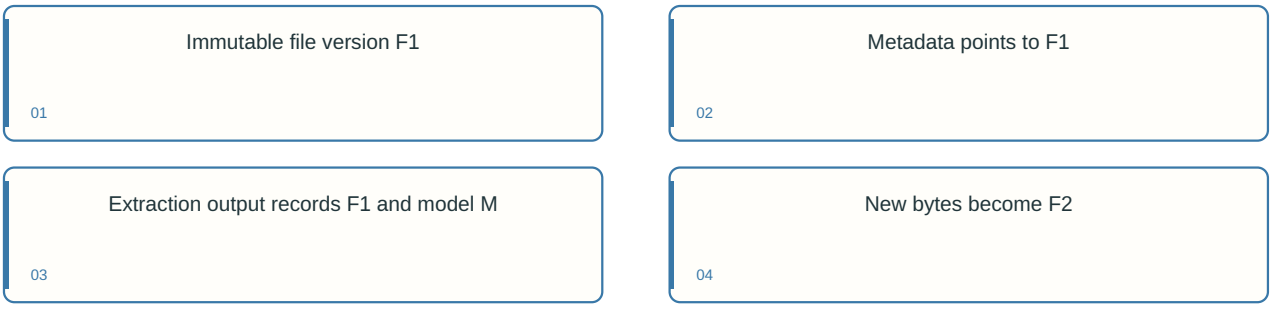


FIG. 13.1.5 **Metadata vs binary data · the example**

Immutable source versions keep changing annotations and processing attempts traceable to the bytes they used.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · How would you investigate an extracted field after both source and model have changed?

Use stable references to the exact source version, region, transform/model version, parameters, and review evidence. Keep the disputed output version identifiable. Re-running a changed or nondeterministic model is not proof of the old result. Retain appropriate evidence under the lifecycle policy and distinguish reproducibility limits from missing provenance.

Remember: Immutable content identity and a tracked lifecycle connect stored bytes to application progress.

Try applying it: What cleanup is needed if the object exists but its metadata transaction fails?

Unit 13 exercises → · Unit 13 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. OpenLineage: core model and documentation ↗

   Official specification documentation · 1.54.0 at access. Dataset, job, and run lineage concepts.

   `Access reviewed · 21 October 2026`

2. Elasticsearch: Reindex documents ↗

   Official documentation · accessed 21 Oct 2026. Implementation reindex controls; full generation-switch protocol in the atlas is an original design.

   `Access reviewed · 21 October 2026`

3. Apache Kafka 4.1: Design ↗

   Official versioned documentation. Partitioning and scoped transactional/processing behavior; external effects are separate.

   `Access reviewed · 21 October 2026`

Build and rebuild a search index

Prerequisites: 13.1 · Upload bytes without losing identity 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Search needs its own representation of source content. Follow term indexing, stage progress, and a generation switch while updates and deletions continue.

13.2.1 · Search engines and inverted indexes

Consider D1 = “blue car” and D2 = “blue bike.” An inverted index records that blue occurs in D1 and D2, while car occurs only in D1. A query can use these postings to find candidates without scanning every document's text.

Tokenisation and normalisation affect which terms are recorded. Updates also need document identity and version: an older extraction must not overwrite a newer representation. Keep recoverable sources and transformation information so the index can be rebuilt. Its retrieval structure has its own freshness and access obligations, even when the original document is stored correctly elsewhere.

Source / term	Representation	Meaning
D1	blue car	document text
D2	blue bike	document text
blue	D1, D2	posting list
car	D1	posting list

FIG. 13.2.1 **An inverted index is a term-to-document map**

Token analysis creates the lookup representation. Source versions and permissions remain separately owned.

Author-created illustration. Numbers are illustrative unless stated otherwise.

13.2.2 · Indexing pipelines

A document can be uploaded successfully while indexing fails. Treat extraction, validation, enrichment, indexing, and publication as distinguishable stages so the application can report where progress stopped.

Each retry should name its input version and intended output, rather than create another searchable document blindly. Give malformed or repeatedly failing files a bounded retry and review path. Publication must also respect current access policy; an old “public” flag copied into an artifact may no longer be valid. A useful completed state names the source and index version whose required work actually finished.

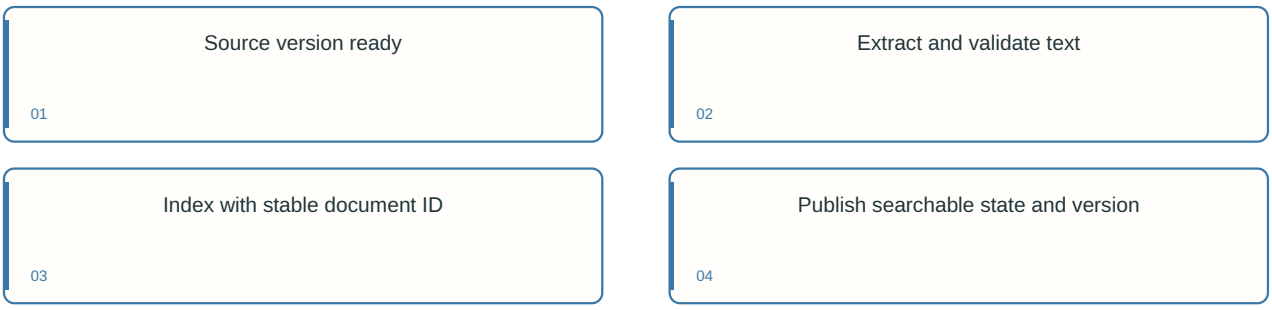


FIG. 13.2.2 **Indexing pipelines · the example**

Indexing stages need identified inputs, repeat-safe output, validated completion, and an owner for failed progress.

Author-created illustration. Numbers are illustrative unless stated otherwise.

13.2.3 · Search ingestion and reindexing

A new analyser needs a fresh search index while users continue searching. Build a new generation from a defined snapshot or source position, then apply changes that occurred during the build.

Include deletions as well as inserts and edits. Validate results before switching a serving alias or routing version. Keep the old usable generation until rollback and compatibility conditions permit removal. Equal document counts do not establish equal versions, permissions, or contents. The transition is safe only when readers know which representation is authoritative and new writes are not lost between the initial copy and the switch.

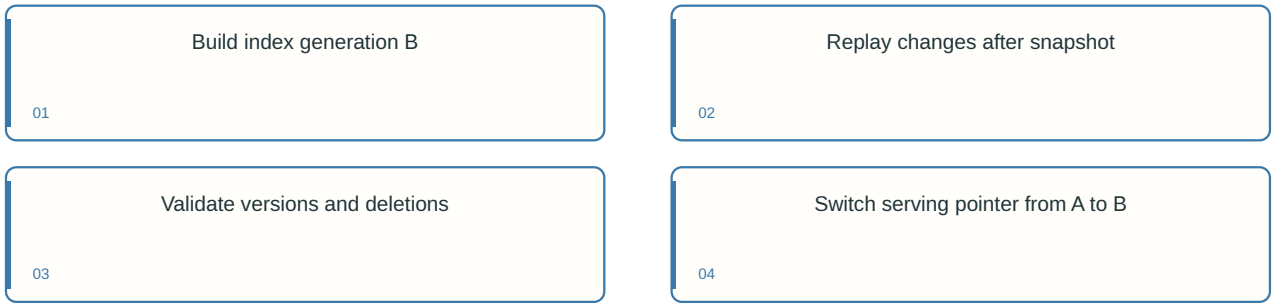


FIG. 13.2.3 **Search ingestion and reindexing · the example**

Reindexing builds and validates a new generation while catching up edits and deletes before serving switches.

Author-created illustration. Numbers are illustrative unless stated otherwise.

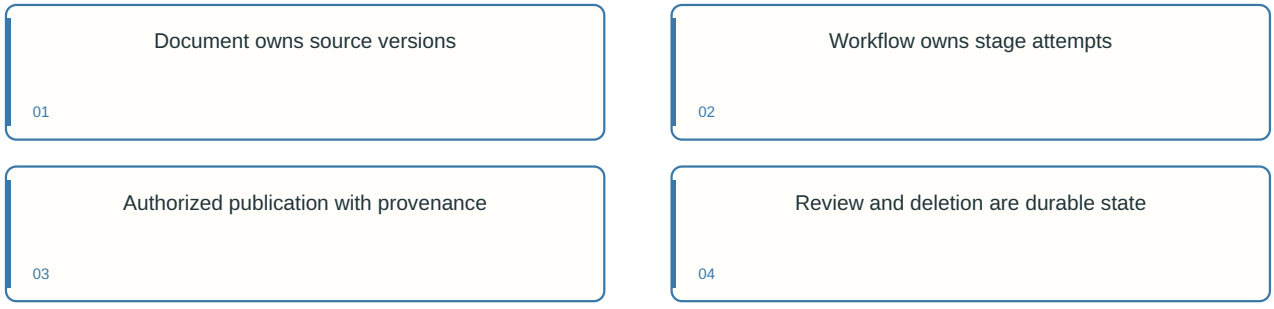


FIG. 13.2.4 **State and ownership in the related design**

Fields and decisions retain tenant, input, transform, and evidence relationships.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · How would you investigate an extracted field after both source and model have changed?

Use stable references to the exact source version, region, transform/model version, parameters, and review evidence. Keep the disputed output version identifiable. Re-running a changed or nondeterministic model is not proof of the old result. Retain appropriate evidence under the lifecycle policy and distinguish reproducibility limits from missing provenance.

Remember: A derived index needs versions, access rules, and a rebuild protocol.

Try applying it: Why do matching document counts fail to prove that a reindex is ready to serve?

Unit 13 exercises → · Unit 13 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. OpenLineage: core model and documentation ↗

   Official specification documentation · 1.54.0 at access. Dataset, job, and run lineage concepts.

   `Access reviewed · 21 October 2026`

2. Elasticsearch: Reindex documents ↗

   Official documentation · accessed 21 Oct 2026. Implementation reindex controls; full generation-switch protocol in the atlas is an original design.

   `Access reviewed · 21 October 2026`

3. Apache Kafka 4.1: Design ↗

   Official versioned documentation. Partitioning and scoped transactional/processing behavior; external effects are separate.

   `Access reviewed · 21 October 2026`

Move data through batch and streams

Prerequisites: 13.2 · Build and rebuild a search index 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A nightly report and an immediate event alert need different freshness. Compare batch, continuous processing, transformation placement, and change capture using the progress each can establish.

13.3.1 · Batch vs streaming pipelines

A nightly batch can recompute yesterday's report from a bounded input. A streaming processor handles continuing input and records progress as it arrives. Choose by the required freshness and recomputation behaviour.

Streaming can reduce delay, but adds late arrivals, partitioned state, and checkpoint recovery. Batch processing has failure and resource costs too. State whether a result is provisional: later input may change a total. A simple scheduled batch is appropriate when hours of delay are acceptable. A low-latency requirement may justify streaming, provided its recovery and state rules are designed as carefully as the normal path.

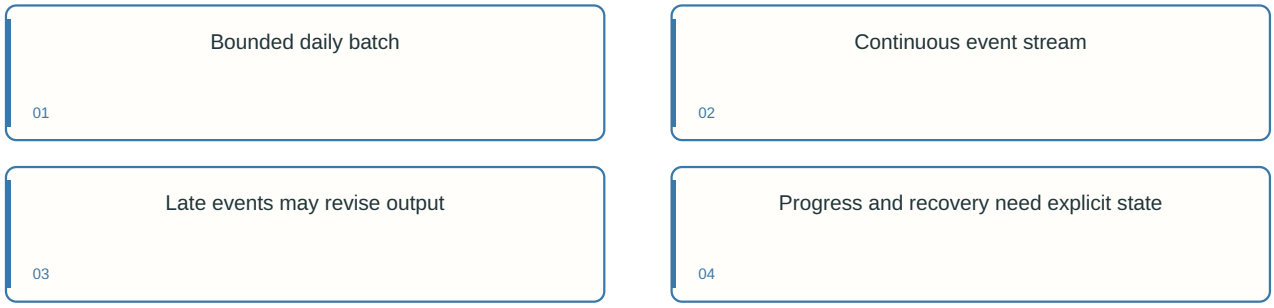


FIG. 13.3.1 **Batch vs streaming pipelines · the example**

Batch works on bounded input; streaming maintains continuing progress, with different freshness and recovery costs.

Author-created illustration. Numbers are illustrative unless stated otherwise.

13.3.2 · ETL vs ELT

An ETL pipeline extracts data, transforms it, then loads the target. ELT loads first and transforms in the target environment. The difference changes where raw data exists and where computation occurs.

Transforming early can minimise sensitive fields before they reach analytics. Loading raw data can support later transformations but adds storage and governance work. In either arrangement, version transformations, validate input and output, and keep ownership and lineage. Neither acronym establishes data quality or repeatability. Choose where each step belongs by its privacy, compute, and query needs rather than treating either sequence as inherently superior.

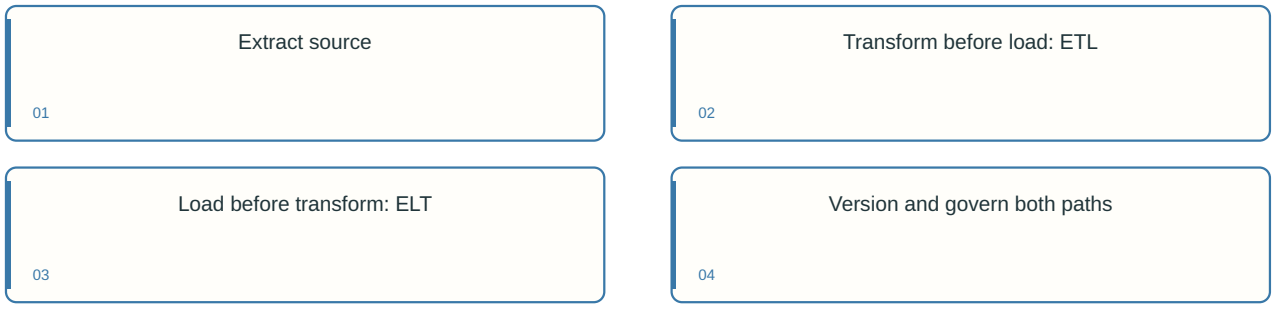


FIG. 13.3.2 **ETL vs ELT · the example**

ETL transforms before target loading; ELT transforms after loading, changing raw-data placement and governance work.

Author-created illustration. Numbers are illustrative unless stated otherwise.

13.3.3 · Change Data Capture (CDC)

A source row changes, and a supported Change Data Capture mechanism records that committed change for downstream consumers. CDC can keep search or analytics current without repeatedly scanning the whole source.

An initial snapshot and later change stream need a consistent handoff position. Otherwise, changes made during copying can be missed or applied wrongly. Preserve source identifiers, positions, and schema information, handle repetition, and monitor lag. A destination may expose a different or delayed view of the source's transactions. CDC transports changes; the destination still needs a clear rule for applying them and serving consistent results.

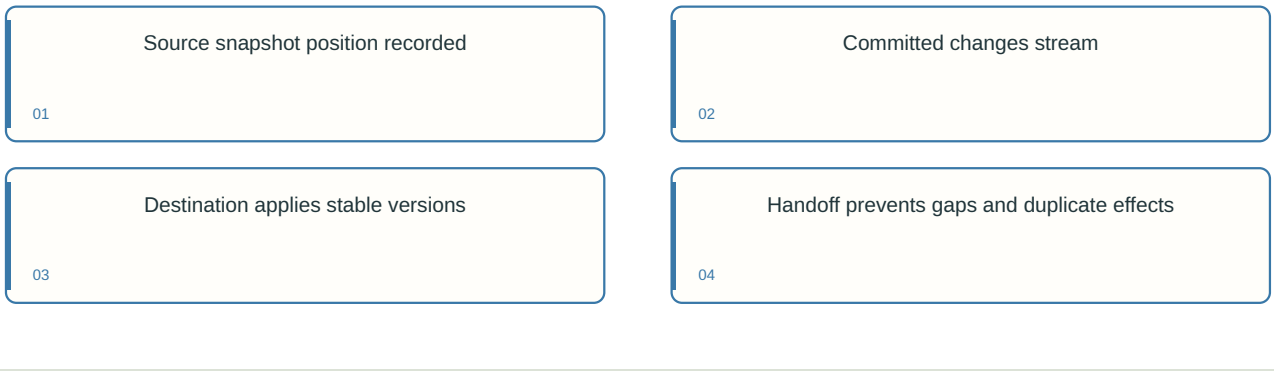


FIG. 13.3.3 **Change Data Capture (CDC) · the example**

CDC transports committed source changes through a supported handoff and ordering mechanism.

Author-created illustration. Numbers are illustrative unless stated otherwise.

13.3.4 · Data lakes and warehouses

A data lake can keep flexible datasets, often in object storage, while a warehouse provides curated analytical structures and serving facilities. Actual product boundaries overlap, so compare their supported operations.

Raw inputs can make replay possible but also accumulate poorly understood and sensitive data. Identify immutable source versions, curated outputs, catalog information, owners, and access policies. Choose file formats and partitions for the scans and recovery work that need to run. Without schema, lineage, and lifecycle management, storing more bytes can make the data harder to trust. Retaining raw data needs a purpose, not just available space.

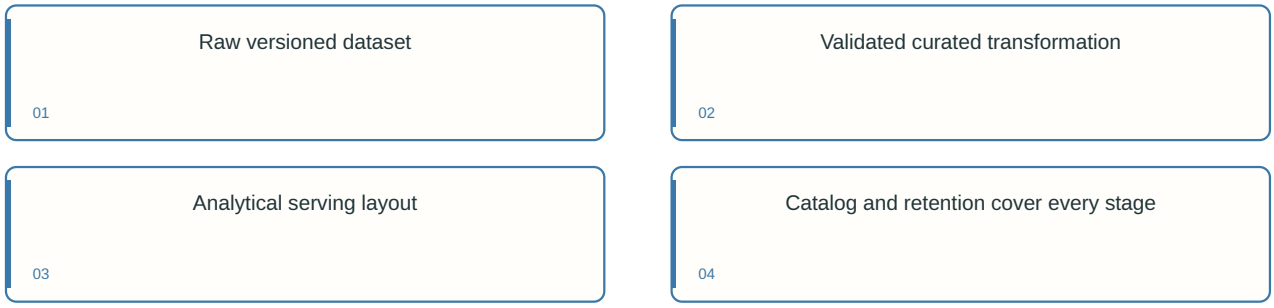


FIG. 13.3.4 **Data lakes and warehouses · the example**

Lakes and warehouses organise raw and curated analytics, requiring ownership, catalog, quality, and lifecycle rules.

Author-created illustration. Numbers are illustrative unless stated otherwise.

13.3.5 · Stream processing fundamentals

An event occurred at noon but reaches a processor three minutes later. Its event time describes the source occurrence; processing time describes when the system handles it. A time-windowed result needs a rule for this late event.

Stream processors maintain keyed state and progress. A watermark estimates how far event-time processing has advanced under lateness assumptions; it is not proof that no earlier event can arrive. Define accepted lateness, corrections, and when a result is final. Checkpoint and external-effect boundaries also matter. Consumers need to know whether a count can change after its first publication.

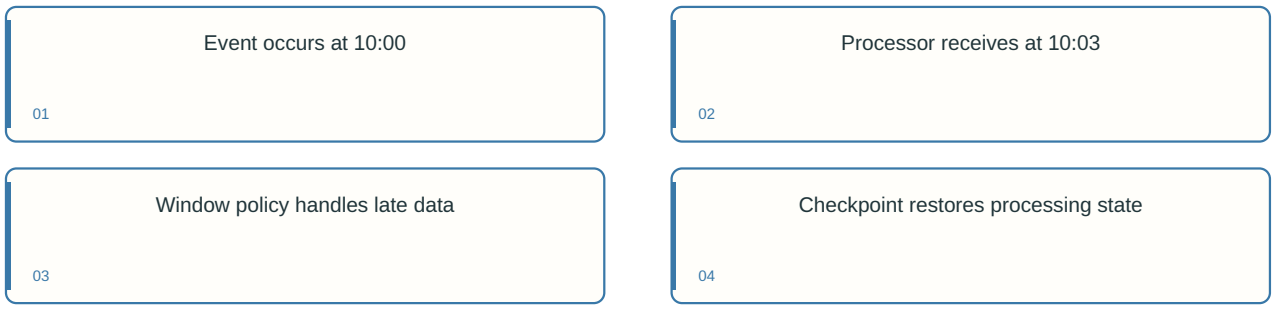


FIG. 13.3.5 **Stream processing fundamentals · the example**

Stream processing needs keyed state, event/processing time, lateness, corrections, and recoverable progress.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · How would you investigate an extracted field after both source and model have changed?

Use stable references to the exact source version, region, transform/model version, parameters, and review evidence. Keep the disputed output version identifiable. Re-running a changed or nondeterministic model is not proof of the old result. Retain appropriate evidence under the lifecycle policy and distinguish reproducibility limits from missing provenance.

Remember: Data movement needs a defined handoff, time meaning, and recovery position.

Try applying it: How should a late event change a result already published for a closed time window?

Unit 13 exercises → · Unit 13 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. OpenLineage: core model and documentation ↗

   Official specification documentation · 1.54.0 at access. Dataset, job, and run lineage concepts.

   `Access reviewed · 21 October 2026`

2. Elasticsearch: Reindex documents ↗

   Official documentation · accessed 21 Oct 2026. Implementation reindex controls; full generation-switch protocol in the atlas is an original design.

   `Access reviewed · 21 October 2026`

3. Apache Kafka 4.1: Design ↗

   Official versioned documentation. Partitioning and scoped transactional/processing behavior; external effects are separate.

   `Access reviewed · 21 October 2026`

Quality, provenance & replay

Prerequisites: 13.3 · Move data through batch and streams 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

An extracted value is disputed after a pipeline update. Trace exact inputs, transformations, checks, and retained copies so the result can be investigated and corrected.

13.4.1 · Data provenance and lineage

An extracted amount is disputed. To investigate it, we need the exact file version, page or region, extractor version, parameters, and any review decision that produced it. Provenance records origins; lineage connects artifacts through transformations.

Store stable references that can actually be checked. An explanatory label alone is weak if the underlying evidence cannot be located. These records support correction and reproducibility without proving every extraction right. They also have their own retention needs: copying raw personal data into a permanent audit record can defeat deletion of the original. Keep useful evidence with an explicit lifecycle.

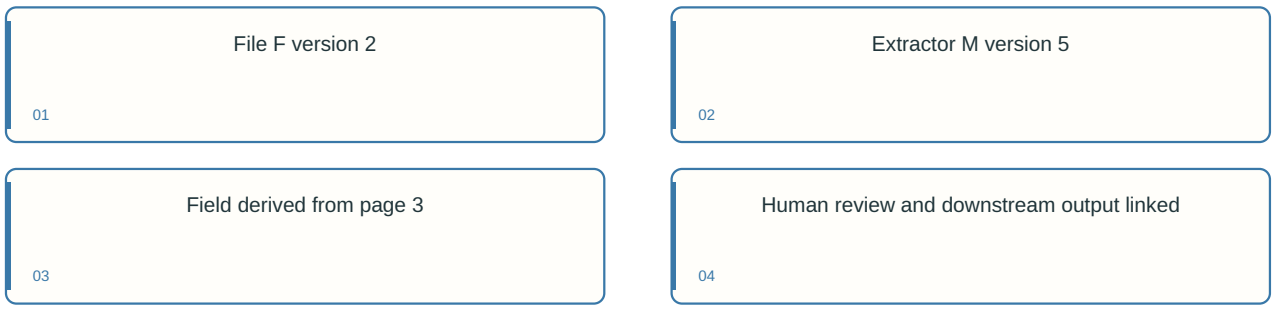


FIG. 13.4.1 **Data provenance and lineage · the example**

Provenance identifies origins; lineage links transformations so outputs can be investigated and corrected.

Author-created illustration. Numbers are illustrative unless stated otherwise.

13.4.2 · Data quality, schema validation, and replay

A record contains both amount and currency, so it passes a structural schema check. If the amount is interpreted in the wrong units, the record is still wrong. Quality checks need semantic rules as well as field presence.

Check identity, ranges, relationships, completeness, and source versions. Quarantine invalid inputs with actionable reasons and enough evidence for repair. Replaying corrected transformations must not repeat external side effects or create duplicate records. Compare source and destination business invariants alongside counts and versions. An empty error queue establishes little if a processor consistently accepts and publishes the wrong meaning.

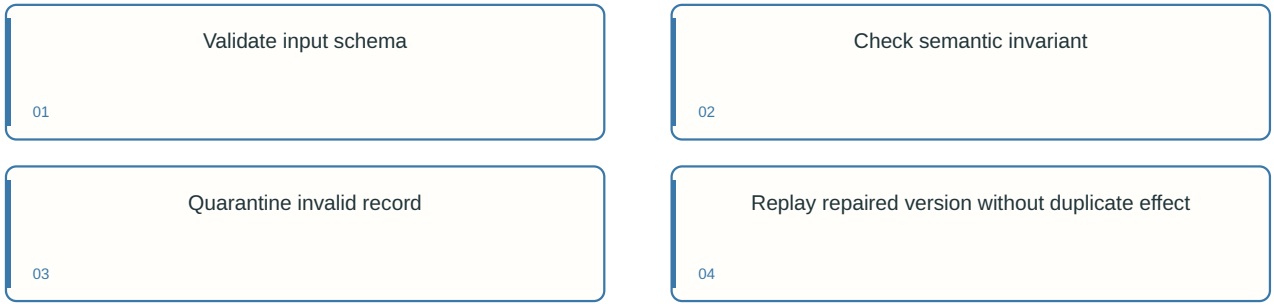


FIG. 13.4.2 **Data quality, schema validation, and replay · the example**

Quality checks validate meaning as well as shape, while replay needs stable versions and safe effect handling.

Author-created illustration. Numbers are illustrative unless stated otherwise.

13.4.3 · Lifecycle policies and retention

Deleting a PDF leaves its extracted text, thumbnail, and search entry behind. A retention policy on original bytes alone has not completed the document's lifecycle.

Track every relevant copy, including logs and recoverable snapshots. A durable deletion workflow can record progress and retry unfinished steps. Minimal deletion markers may be needed to stop old workers or replay from recreating removed results; minimise the personal information retained in them. Maintain indexes that can find a user's objects efficiently. Test restoring older state against the deletion history, so recovery does not silently republish content that should remain removed.

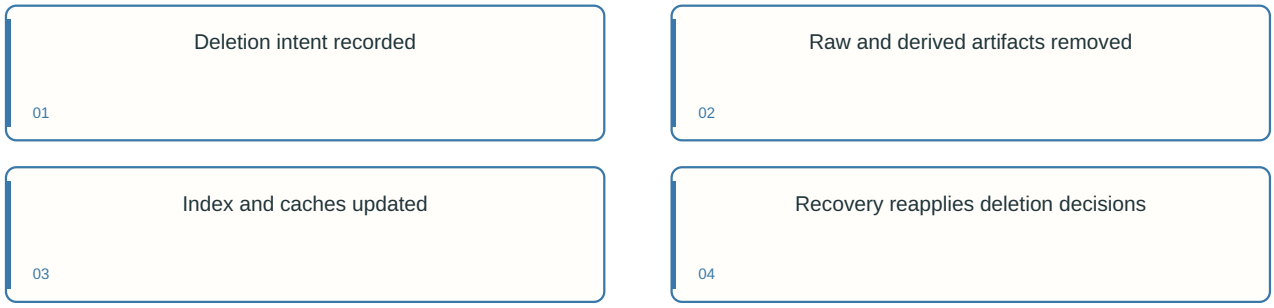


FIG. 13.4.3 **Lifecycle policies and retention · the example**

Lifecycle rules follow originals, intermediates, indexes, logs, and recovery copies through required deletion.

Author-created illustration. Numbers are illustrative unless stated otherwise.

13.4.4 · Designing data-intensive applications

Follow one document from upload to extraction, review, and search. Every step takes an identified input, applies a transformation, and produces state used by the next step. The complete design is this connected set of operations.

For each connection, define the source owner, version, order, retry identity, validation, and recovery. Keep inputs recoverable where rebuilding is required, within the retention policy. Observe progress across stages rather than only queue throughput. A fast broker cannot repair missing provenance or an index serving the wrong tenant's documents. End-to-end correctness depends on how these contracts work together.

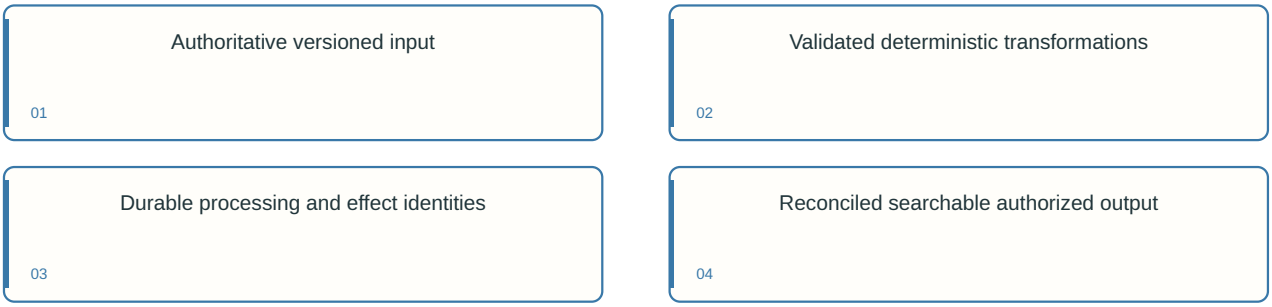


FIG. 13.4.4 **Designing data-intensive applications · the example**

Data-intensive design connects versioned transformations with explicit authority, validation, progress, and recovery.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Try the mechanism in code

PYTHON · SOURCE AND ATTEMPT FENCING

```
from mechanisms import Workflow
run = Workflow("file-version-2", epoch=8)
assert not run.complete(7, "file-version-2", "old")
assert not run.complete(8, "file-version-1", "wrong-input")
assert run.complete(8, "file-version-2", "current")
```

The workflow rejects an old epoch and the wrong input version, then accepts the current combination. These local conditions make the attempt's authority explicit.

A real store must check and apply them atomically and validate the output. Provenance also needs the transform version and relevant source evidence, not just a result filename.

Download the executable mechanism collection ↓ · Follow the connected complete design →

THE WORKBENCH · TRY CHANGING THE INPUTS

Should this worker be allowed to finish?

Reject completion

The run now belongs to epoch 8. A worker from epoch 7 can no longer save its completion.

Choose an attempt and check its ownership epoch, input version, and document state. A real database must check these conditions and save the result atomically. This step does not include an external model call or an object-store write.

▼ Think further · How would you investigate an extracted field after both source and model have changed?

Use stable references to the exact source version, region, transform/model version, parameters, and review evidence. Keep the disputed output version identifiable. Re-running a changed or nondeterministic model is not proof of the old result. Retain appropriate evidence under the lifecycle policy and distinguish reproducibility limits from missing provenance.

Remember: Quality and provenance belong to the output contract and every copy's lifecycle.

Try applying it: What would let an old worker recreate personal data after the source was deleted?

Unit 13 exercises → · Unit 13 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. OpenLineage: core model and documentation ↗
   Official specification documentation · 1.54.0 at access. Dataset, job, and run lineage concepts.
   Access reviewed · 21 October 2026

2. Elasticsearch: Reindex documents ↗

   Official documentation · accessed 21 Oct 2026. Implementation reindex controls; full generation-switch protocol in the atlas is an original design.

   Access reviewed · 21 October 2026

3. Apache Kafka 4.1: Design ↗

   Official versioned documentation. Partitioning and scoped transactional/processing behavior; external effects are separate.

   Access reviewed · 21 October 2026

PART 4 / UNIT 14

Distributed Systems Fundamentals

One service commits while its caller loses the reply. Begin with partial failures and the limits of time observations before trying to order events across machines.

The running example

A social application writes a new post in one region while a friend reads it from another. Examine different consistency contracts and their user-visible effects.

Unit 14 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

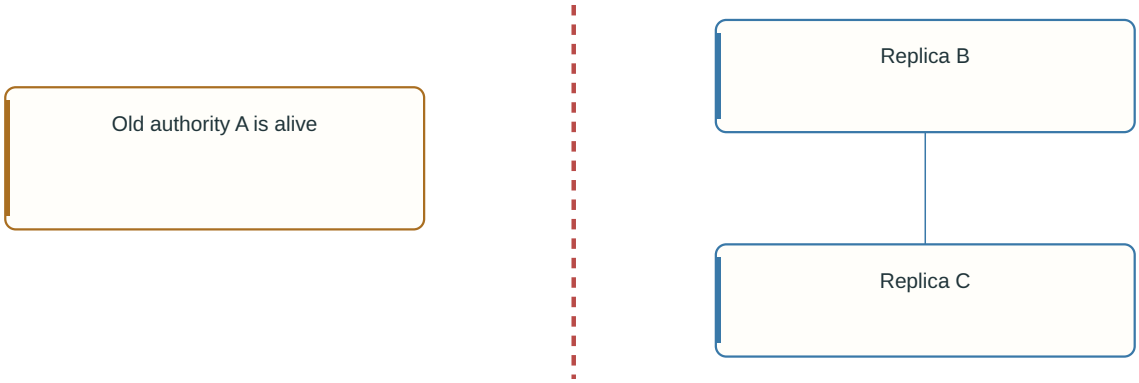


FIG. 14.0.1 **A partition can leave two live but disconnected sides**

Region A remains running while B and C form another reachable side. Correct authority and fencing determine who may make which writes; a missing heartbeat cannot stop A.

Author-created illustration. Numbers are illustrative unless stated otherwise.

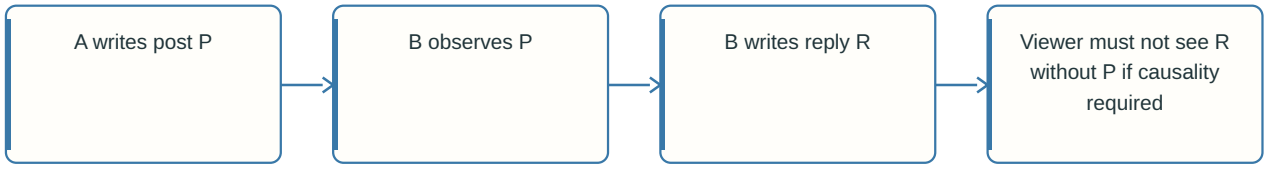


FIG. 14.0.2 **causality timeline**

Causality follows program and message dependencies, not arbitrary wall-clock ordering.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Model	Required constraint	What is not implied
Linearizable	Real-time atomic operations	External-effect transaction
Serializable	Equivalent serial transactions	Real-time order
Causal	Preserve dependencies	One total global order
Eventual	Converge under assumptions	Bounded freshness by itself

FIG. 14.0.3 **consistency model comparisons**

Name the model rather than treating strong and eventual as complete application descriptions.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Set	Members	Intersection
Write W=2	A, B	B
Read R=2	B, C	B
Fixed N=3	R + W = 4 > 3	one shared member

FIG. 14.0.4 **Intersecting quorums still need a protocol**

Intersection preserves a possible witness. Version, membership, persistence, and concurrency rules determine how that witness is used.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Partial failure & the meaning of time

Prerequisites: Networking Foundations for System Design, Runtime, Processes, Threads, and Concurrency, Database Internals, Transactions, and Concurrency Control

7 original figures

Blue: normal flow · Green: healthy · Amber: caution · Red: failure · Purple: control / shared infrastructure · Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

One service commits while its caller loses the reply. Begin with partial failures and the limits of time observations before trying to order events across machines.

14.1.1 · What makes a distributed system difficult?

An order service calls a payment provider. The provider may succeed while the order service cannot receive its reply. Different machines can keep working, stop, or lose communication independently, so there is no single local observation that explains everything.

Distributed design specifies what outcomes are allowed despite this uncertainty. Begin with clear state owners and operations, then introduce additional copies or coordination for a real requirement. More nodes add possible failure combinations alongside capacity. The design therefore needs a protocol: rules for messages, state changes, acknowledgments, and recovery. A diagram showing connected machines does not supply those rules on its own.

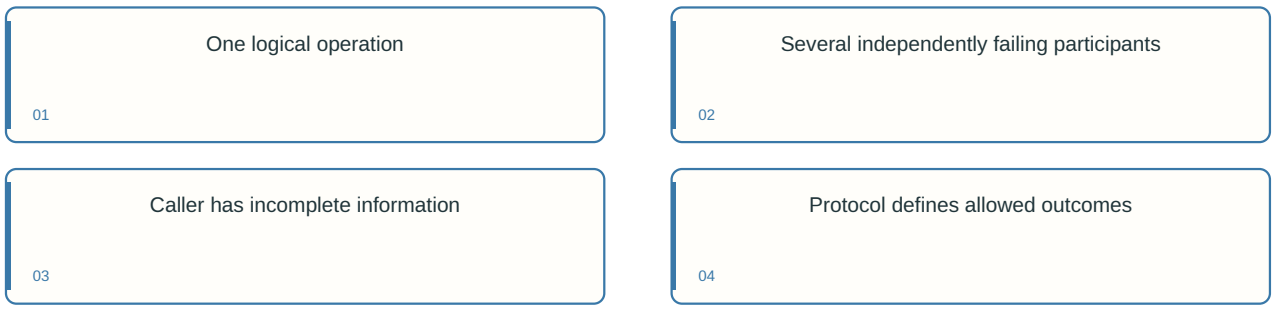


FIG. 14.1.1 **What makes a distributed system difficult · the example**

Distributed participants observe only parts of an operation, so legal outcomes need explicit message and recovery protocols.

Author-created illustration. Numbers are illustrative unless stated otherwise.

14.1.2 · Partial failures

The provider charges a card, but the order service loses the reply. The local order may still be pending while the remote effect already exists. This is a partial failure: some of the overall operation happened and another part did not complete as observed.

Keep “not attempted” separate from “attempted, outcome unknown.” Record an operation or provider reference and reconcile the status. A local rollback cannot erase the remote charge. Compensation is possible only where the business and provider support a suitable further action. Treating every exception as a definite clean failure can lead to resubmitting an effect that already succeeded.

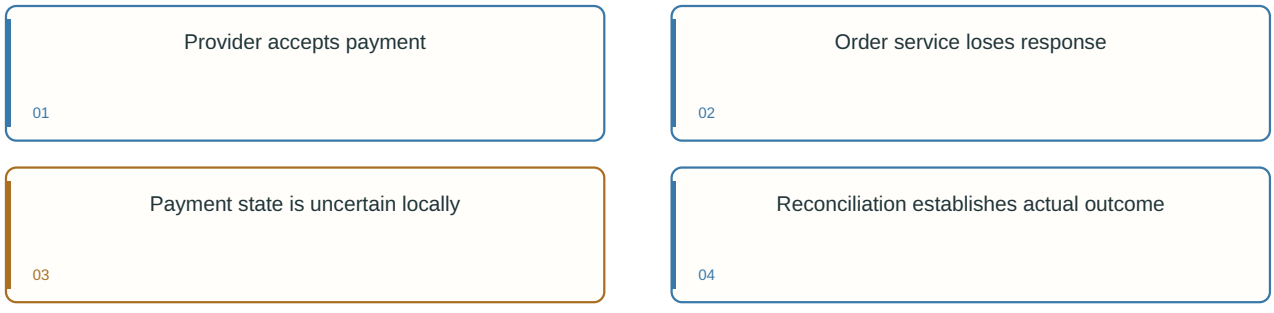


FIG. 14.1.2 **Partial failures · the example**

Partial failure permits one participant's effect to succeed while another fails to observe or record it.

Author-created illustration. Numbers are illustrative unless stated otherwise.

14.1.3 · Network uncertainty

A heartbeat does not arrive. The other machine might have failed, or its message might simply be delayed. For a time, these cases can look identical to the observer.

A timeout is a practical suspicion and a limit on waiting, not proof that the old process stopped. A retry can overlap the delayed original request. Bound waiting to protect resources, while using a protocol to prevent incompatible owners or duplicate effects. State which operations must wait or reject when authority is uncertain. A short timer can improve responsiveness, but correctness must also hold when its suspicion is wrong.

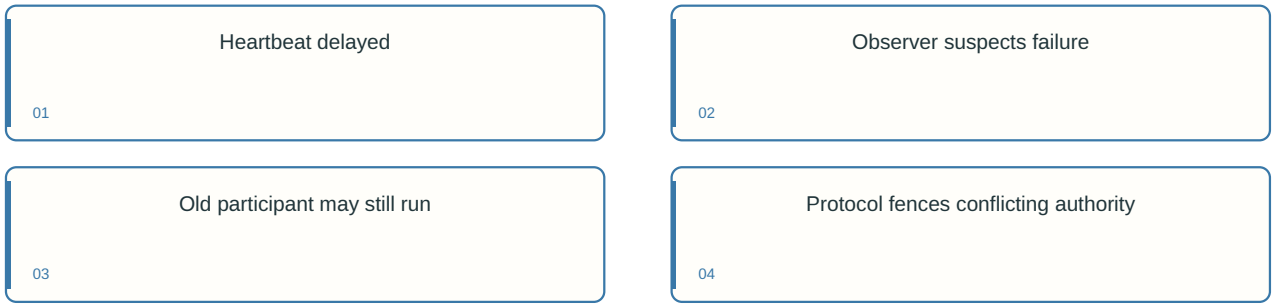


FIG. 14.1.3 **Network uncertainty · the example**

Network delay and failure can appear alike; timeouts bound waiting but cannot prove a remote process stopped.

Author-created illustration. Numbers are illustrative unless stated otherwise.

14.1.4 · Fault models

Suppose a protocol assumes machines either follow its rules or crash. A participant that lies about its stored history is outside that assumption. A fault model states the failures the design claims to tolerate.

Common models include crash-stop, crash-recovery, message loss, partitions, and arbitrary Byzantine behaviour. Recovery also depends on whether required persistent state survives a restart. State these conditions before calculating a replica count. Three copies on one failed device have a different failure exposure from independently placed durable copies. Correctness reasoning is meaningful only for the failure cases and resource assumptions it actually covers.

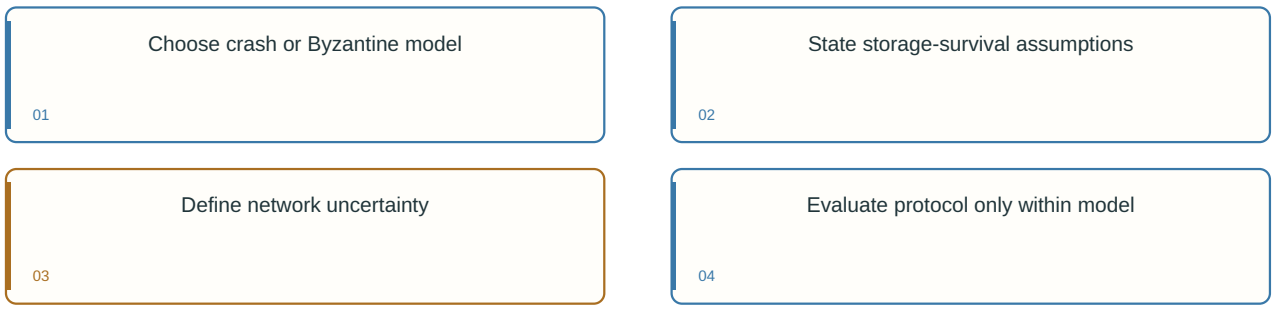


FIG. 14.1.4 **Fault models · the example**

A fault model specifies failures and persistence assumptions a protocol claims to tolerate.

Author-created illustration. Numbers are illustrative unless stated otherwise.

14.1.5 · Time, clocks, and event ordering

Two machines stamp events using their own clocks. Clock drift or adjustment can make the later causal event appear earlier by timestamp. A wall-clock comparison therefore does not necessarily establish what caused what.

Use an appropriate monotonic clock for elapsed durations where available. For expiry, identify the authority for time and the uncertainty the rule permits. An ordering requirement may need a sequence or protocol log rather than client timestamps. Recording source time and observed time separately helps explain delayed events. The design must distinguish user-facing dates, measured durations, and causal relationships instead of asking one timestamp to establish all three.

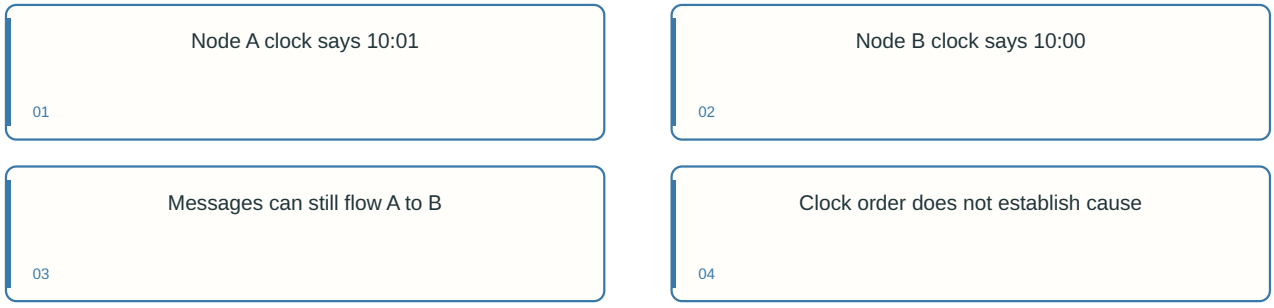


FIG. 14.1.5 **Time, clocks, and event ordering · the example**

Physical timestamps, elapsed durations, and causal order answer different questions and need suitable clocks or protocols.

Author-created illustration. Numbers are illustrative unless stated otherwise.

14.1.6 · Physical vs logical clocks

A physical clock estimates real time. A logical clock records ordering information and need not count seconds. A logical counter can tell us something about event relationships without saying what time of day they occurred.

Use the kind of time appropriate to the question. Displaying a date and ordering protocol actions are different tasks. Even subtracting timestamps from two machines can be misleading if their clock errors are unknown. A combined physical/logical scheme needs its own specified rules. Name which meaning each stored field has so later code does not interpret an ordering counter as a duration or a timestamp as proof of causality.

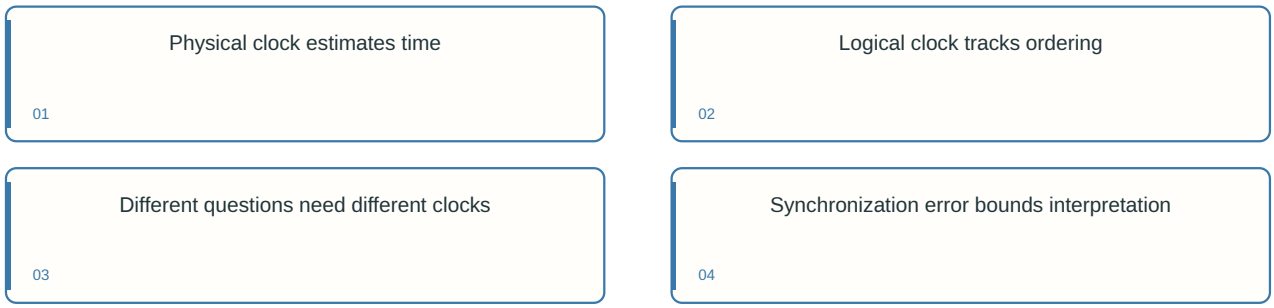


FIG. 14.1.6 **Physical vs logical clocks · the example**

Physical clocks estimate time; logical clocks represent ordering relationships without necessarily measuring seconds.

Author-created illustration. Numbers are illustrative unless stated otherwise.

14.1.7 · Lamport clocks and vector clocks

A process with Lamport counter 5 receives a message carrying 9. It advances to max(5,9)+1 = 10. Local increments and these receive rules ensure that a causal predecessor has a smaller counter. A smaller counter alone does not prove causation.

Vector clocks keep progress for several participants. Vectors [1,0] and [0,1] each contain progress absent from the other, so they represent concurrent versions under the model. This can reveal conflicts but costs metadata. Neither clock decides how to merge two account changes or independently provides a global real-time commit order. The clock describes a relation; the application still needs a valid resolution rule.

Event	Vector	Relation
A only	[1,0]	not before B
B only	[0,1]	not before A
After observing both	[1,1]	dominates both

FIG. 14.1.7 **Vector comparison exposes concurrent events**

Componentwise comparison identifies causality under the chosen participant model; it does not choose a business merge.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · Which operations may continue during a partition while exclusive allocation must wait?

Operations with explicitly permitted stale or independent results may continue. Exclusive allocation needs valid authority and may have to wait or reject when agreement is unavailable. State per-operation observations rather than weaken the entire system silently. Recovery must reconcile progress without introducing another owner.

Remember: A local observation may leave remote outcomes unknown; clocks describe different kinds of order.

Try applying it: Why does a missing heartbeat fail to prove that the old owner has stopped?

Unit 14 exercises → · Unit 14 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Jepsen: Consistency models ↗

   Primary practitioner research reference. Formal-history distinctions and references to underlying literature.

   Access reviewed · 21 October 2026

2. Time, Clocks, and the Ordering of Events in a Distributed System — Lamport, 1978 ↗

   Original research. Ordering reference for Unit 14.

   Access reviewed · 21 October 2026

3. Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services — Gilbert and Lynch, 2002 ↗

   Original research · university-hosted paper. CAP assumptions and impossibility; PDF text extraction is imperfect, original page inspected.

   Access reviewed · 21 October 2026

Consistency as a reader-visible contract

Prerequisites: 14.1 · Partial failure & the meaning of time 6 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A user reads a value, writes another, and switches replicas. Describe the allowed observations before choosing a consistency model or relying on a product's “strong” label.

14.2.1 · Consistency models

A user writes a post, reads it, then switches replicas. What may the next read show? A consistency model specifies which observation histories the system permits.

Write examples before choosing a setting: can a read return an older value, can two readers see different orders, and must a reply appear after its parent post? Different objects can need different rules. A feed count may tolerate delay while an allocation requires one winner. Stronger observations can require more coordination or fewer available serving paths during a partition. Name the model and its operation scope; a bare word such as “strong” leaves important assumptions unstated.

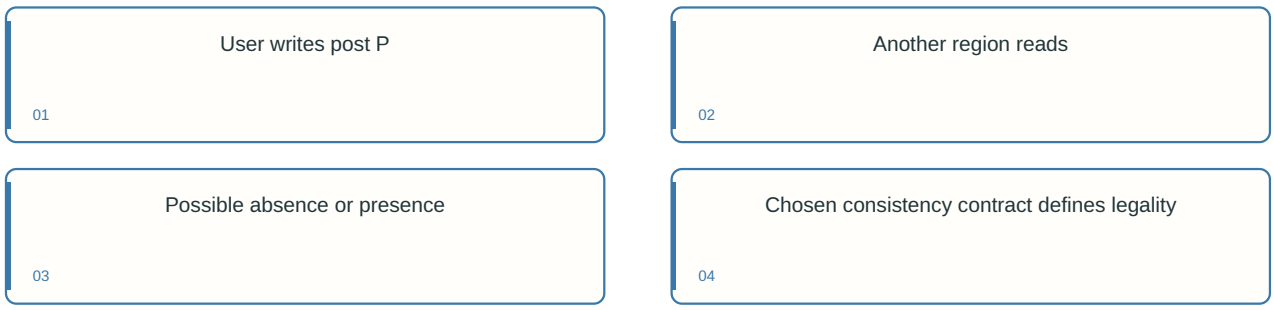


FIG. 14.2.1 **Consistency models · the example**

A consistency model defines permitted observations across operations, sessions, and state copies.

Author-created illustration. Numbers are illustrative unless stated otherwise.

14.2.2 · Strong consistency and eventual consistency

Replicas may briefly disagree after an update, then converge once updates stop and communication and repair succeed. Eventual consistency expresses convergence under such assumptions. It does not by itself specify a maximum delay or decide how conflicts are merged.

“Strong consistency” is often used for linearizable behaviour, but check what the speaker means. A social feed may allow propagation delay while the author still needs to read their own post immediately. State these per-operation contracts. Delayed convergence does not excuse losing an acknowledged update outside the agreed failure policy. Freshness, conflict handling, and durability need their own precise rules.

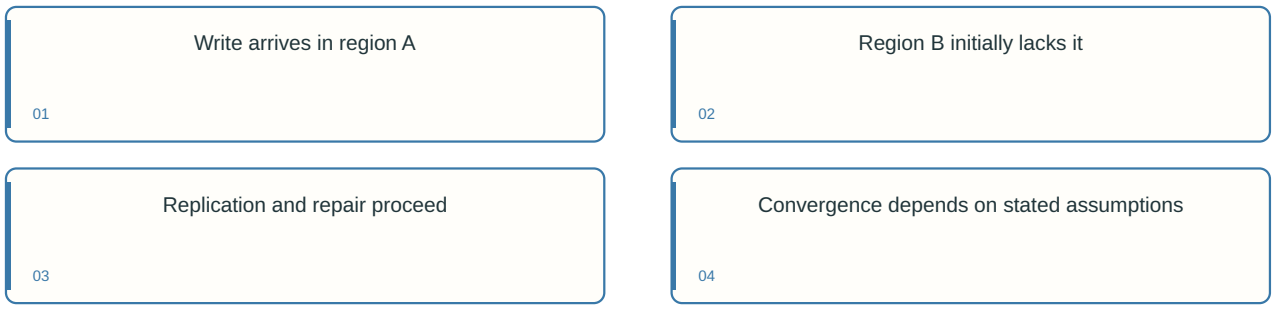


FIG. 14.2.2 **Strong consistency and eventual consistency · the example**

Eventual convergence needs stated repair assumptions and conflict semantics; “strong” needs a named precise model.

Author-created illustration. Numbers are illustrative unless stated otherwise.

14.2.3 · Linearizability

A write of 2 finishes before another client begins a read. For a linearizable register, that read cannot return the superseded value 1 unless an intervening write permits it. Each operation behaves as if it took effect at one point between invocation and response, respecting real-time precedence.

Overlapping operations can have more than one valid ordering. A timed-out operation can still take effect because the client never obtained a definite completed result. Achieving linearizability requires a suitable protocol for writes and reads. Majority arithmetic and timestamps alone leave version choice, concurrency, and recovery rules unresolved.

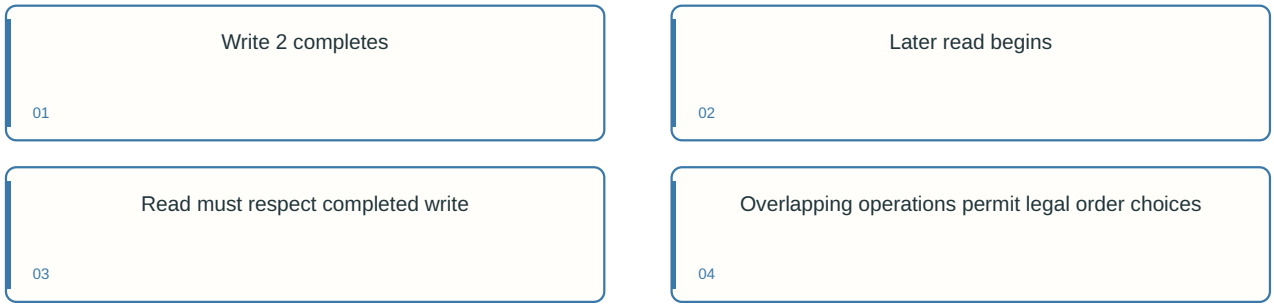


FIG. 14.2.3 **Linearizability · the example**

Linearizability gives operations atomic points between invocation and response while respecting real-time precedence.

Author-created illustration. Numbers are illustrative unless stated otherwise.

14.2.4 · Serializability vs linearizability

Two database transactions can be equivalent to a serial order without that order necessarily following the real-time order observed by separate clients. Serializability concerns whole transactions and their equivalent serial execution.

Linearizability concerns real-time ordering of operations. Strict serializability combines transaction serialization with real-time precedence. Distinguish these when combining transactions and replica reads. Also identify the actual atomic boundary: a serializable local transaction does not bring an external payment provider into the same transaction. The property helps reason about work within its scope; messages and remote effects still need their own ordering and recovery design.

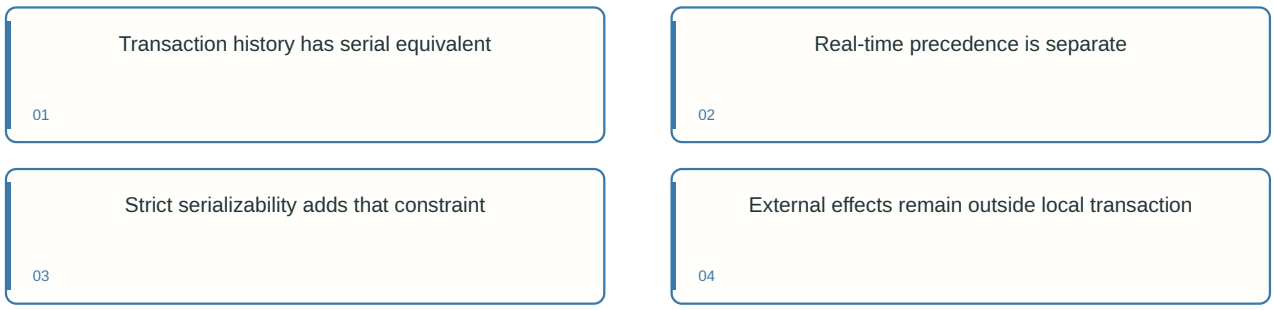


FIG. 14.2.4 **Serializability vs linearizability · the example**

Serializability concerns transaction equivalence; strict serializability additionally respects real-time constraints.

Author-created illustration. Numbers are illustrative unless stated otherwise.

14.2.5 · Sequential and causal consistency

Sequential consistency allows one common ordering that preserves each participant's program order, without necessarily preserving real-time order across participants. Causal consistency preserves dependencies while permitting unrelated concurrent events to appear in different orders.

Consider a reply written only after its author read a post. A causally consistent view should not expose that dependent reply without the parent information it depends on. The system needs dependency tracking or a supported protocol to provide this observation. Ordinary delayed replication alone does not establish it. Use a concrete event history to decide which order matters, rather than requiring a total order for every unrelated action.

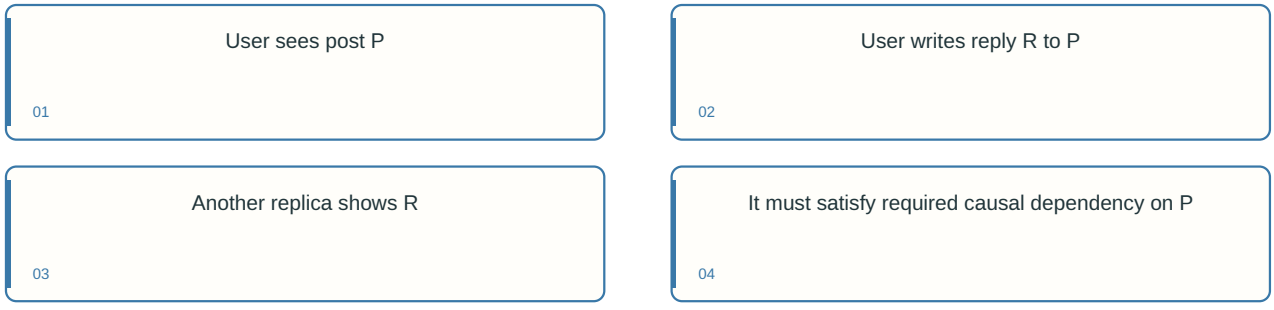


FIG. 14.2.5 **Sequential and causal consistency · the example**

Sequential consistency preserves participant program order; causal consistency preserves dependencies while allowing unrelated orders to differ.

Author-created illustration. Numbers are illustrative unless stated otherwise.

14.2.6 · Read-your-writes and monotonic reads

After editing a profile, a user expects the next read to include that edit. Read-your-writes provides this session guarantee. After seeing version ten, the user also may expect not to see version nine later; that is monotonic reads.

Routing, replication-position tracking, or another suitable protocol can support these rules. They do not necessarily provide global linearizability. Decide how progress follows a session when the user changes devices or an instance fails. Sticky routing alone may lose the guarantee at failover. A read might need to wait, use a current owner, or return an explicit unavailability result rather than silently go backward.

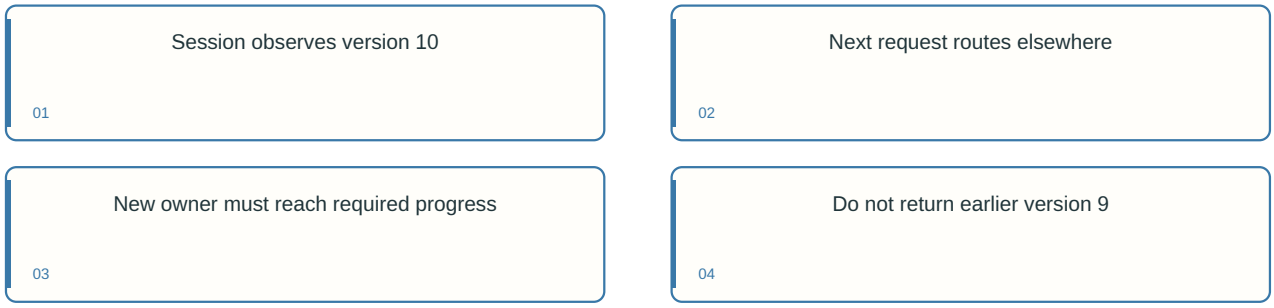


FIG. 14.2.6 **Read-your-writes and monotonic reads · the example**

Read-your-writes and monotonic reads preserve particular session observations across supported routing and recovery paths.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · Which operations may continue during a partition while exclusive allocation must wait?

Operations with explicitly permitted stale or independent results may continue. Exclusive allocation needs valid authority and may have to wait or reject when agreement is unavailable. State per-operation observations rather than weaken the entire system silently. Recovery must reconcile progress without introducing another owner.

Remember: Choose consistency by legal histories and their scope.

Try applying it: Which history distinguishes a session moving backward from a permitted delayed read by another user?

Unit 14 exercises → · Unit 14 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Jepsen: Consistency models ↗

   Primary practitioner research reference. Formal-history distinctions and references to underlying literature.

   Access reviewed · 21 October 2026

2. Time, Clocks, and the Ordering of Events in a Distributed System — Lamport, 1978 ↗

   Original research. Ordering reference for Unit 14.

   Access reviewed · 21 October 2026

3. Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services — Gilbert and Lynch, 2002 ↗

   Original research · university-hosted paper. CAP assumptions and impossibility; PDF text extraction is imperfect, original page inspected.

   Access reviewed · 21 October 2026

Partitions, CAP, PACELC & quorums

Prerequisites: 14.2 · Consistency as a reader-visible contract 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A partition separates machines that all remain alive. Use CAP, ordinary coordination costs, and quorum intersection to identify what each side may safely do.

14.3.1 · CAP theorem, with its actual assumptions

Two groups of replicas cannot communicate during a partition. If both must answer every request, they cannot always preserve CAP's formal atomic-consistency requirement for the shared read/write object. The impossibility concerns those defined properties under the specified partition model.

CAP is therefore not a permanent menu of any two desirable features. Decide what each operation does when coordination is unavailable. Exclusive allocation may reject or wait; a different operation may serve an explicitly weaker view. State the exact property rather than equating formal availability with an ordinary uptime percentage. When the network is functioning, latency and coordination still require separate analysis.

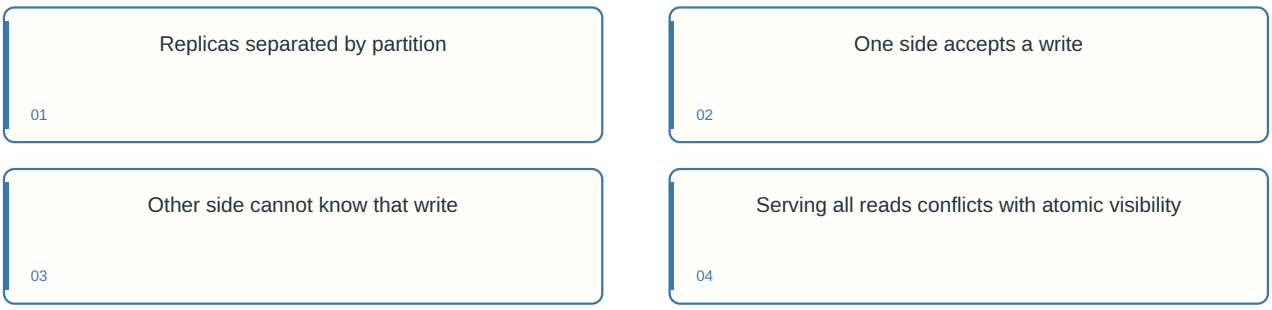


FIG. 14.3.1 **CAP theorem, with its actual assumptions · the example**

CAP concerns the incompatibility of defined atomic consistency and availability guarantees under its partition assumptions.

Author-created illustration. Numbers are illustrative unless stated otherwise.

14.3.2 · PACELC as a useful extension

A write can wait for a distant replica even when the network is healthy. That improves some coordination possibilities but adds travel time. PACELC draws attention to this ordinary latency/consistency trade-off as well as CAP's partition case.

The mnemonic asks: during a partition, what availability/consistency choice is made; else, what latency/consistency trade-off is made? Use it as a review lens rather than a substitute for the protocol. Different operations and configurations can take different paths. Quantify the distances and waits with stated assumptions instead of assigning one permanent letter label to a product.

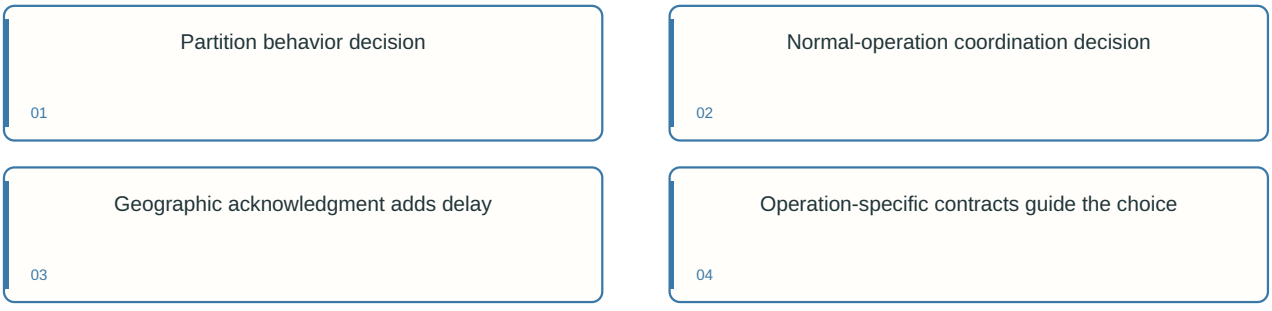


FIG. 14.3.2 **PACELC as a useful extension · the example**

PACELC highlights partition choices and ordinary latency/consistency costs as a design lens, not a complete protocol.

Author-created illustration. Numbers are illustrative unless stated otherwise.

14.3.3 · Quorums and read/write trade-offs

With three fixed replicas, a two-member read set and a two-member write set must overlap. More generally, R+W>N gives read/write intersection, and W>N/2 gives intersection between write quorums.

That is a statement about sets. A reader still needs a correct way to interpret the versions it receives, and writers need ordering, persistence, and concurrency rules. Membership changes or substitute nodes can invalidate a naive fixed-set argument. In N=3,R=2,W=2, at least one queried participant belongs to the completed write set, but that observation alone is not a linearizability proof. Keep the arithmetic and protocol claims separate.

Set	Members	Intersection
Write W=2	A, B	B
Read R=2	B, C	B
Fixed N=3	R + W = 4 > 3	one shared member

FIG. 14.3.3 **Intersecting quorums still need a protocol**

Intersection preserves a possible witness. Version, membership, persistence, and concurrency rules determine how that witness is used.

Author-created illustration. Numbers are illustrative unless stated otherwise.

14.3.4 · Split-brain situations

An old leader pauses. Another leader is elected, but the old process later resumes and tries to write. If both are treated as authorities, the system has split brain.

A missing heartbeat cannot physically stop the earlier owner. An ownership protocol must be paired with enforcement where effects are accepted. For example, a storage receiver can reject requests carrying an older fencing epoch after it has accepted a newer one. Expiring only a local lock variable leaves the paused process able to act elsewhere. Test delayed messages and resumed workers as well as clean crashes, because these histories expose the stale-owner problem.

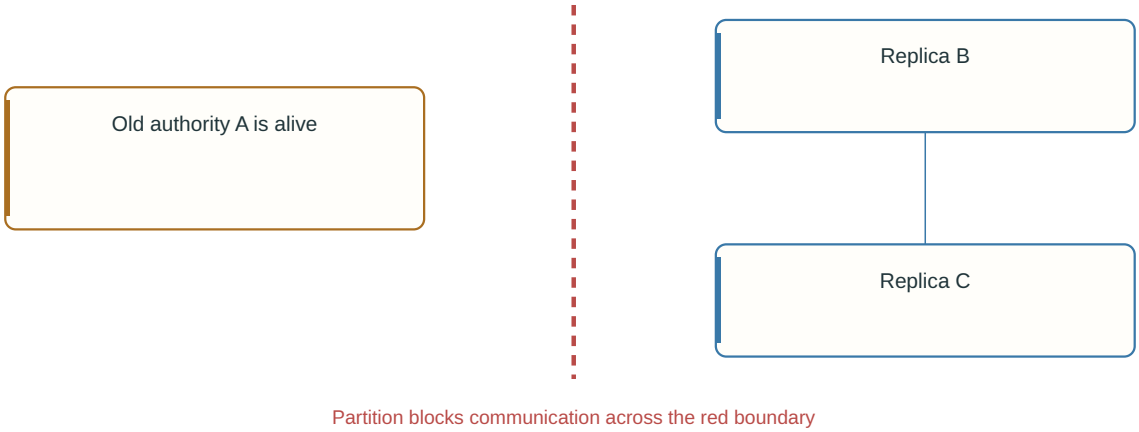


FIG. 14.3.4 **A partition can leave two live but disconnected sides**

Region A remains running while B and C form another reachable side. Correct authority and fencing determine who may make which writes; a missing heartbeat cannot stop A.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · Which operations may continue during a partition while exclusive allocation must wait?

Operations with explicitly permitted stale or independent results may continue. Exclusive allocation needs valid authority and may have to wait or reject when agreement is unavailable. State per-operation observations rather than weaken the entire system silently. Recovery must reconcile progress without introducing another owner.

Remember: Set intersection supplies a protocol ingredient; authority and version rules supply the rest.

Try applying it: Which additional rules would a two-of-three read/write design need before claiming linearizability?

Unit 14 exercises → · Unit 14 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Jepsen: Consistency models ↗

   Primary practitioner research reference. Formal-history distinctions and references to underlying literature.

   Access reviewed · 21 October 2026

2. Time, Clocks, and the Ordering of Events in a Distributed System — Lamport, 1978 ↗

   Original research. Ordering reference for Unit 14.

   Access reviewed · 21 October 2026

3. Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services — Gilbert and Lynch, 2002 ↗

   Original research · university-hosted paper. CAP assumptions and impossibility; PDF text extraction is imperfect, original page inspected.

   Access reviewed · 21 October 2026

Coordination, safety & liveness

Prerequisites: 14.3 · Partitions, CAP, PACELC & quorums 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Some shared decisions need agreement; other work can proceed independently. State both forbidden outcomes and required progress, including the conditions under which progress can pause.

14.4.1 · Distributed coordination

Some decisions need agreement: who owns a partition, which configuration is current, or what order metadata changes follow. Distributed coordination establishes such shared decisions under a failure protocol.

Use it where the invariant needs it. Independent updates that safely commute may not require a shared round on every operation. The coordinator also needs recovery and failure rules; a service name does not make it reliable. Decide how clients behave when it is unavailable. Reusing safe cached configuration can differ from issuing new exclusive ownership. Separating these activities helps preserve useful service without granting incompatible authority.

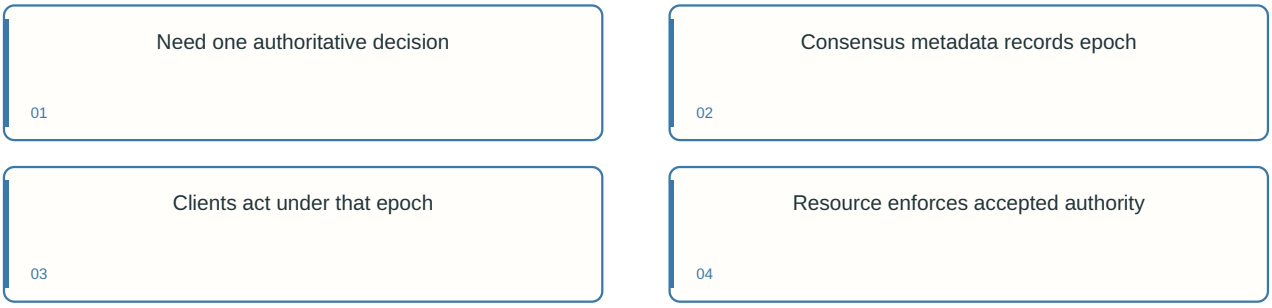


FIG. 14.4.1 **Distributed coordination · the example**

Coordination establishes shared decisions under an explicit failure and progress protocol.

Author-created illustration. Numbers are illustrative unless stated otherwise.

14.4.2 · The fallacies of distributed computing

A design may quietly assume every network call succeeds, arrives instantly, costs nothing, and reaches an unchanged destination. The fallacies of distributed computing collect warnings about such assumptions, including bandwidth, security, infrastructure diversity, and multiple administrators.

Turn each warning into a concrete review question. What is the deadline? Who controls the other endpoint? How is the message versioned and authorised? How much data does fifty-way fan-out transfer? These questions uncover hidden requirements. The list is an engineering aid rather than a protocol guarantee. Its value comes from applying the concerns to real paths and failure histories.

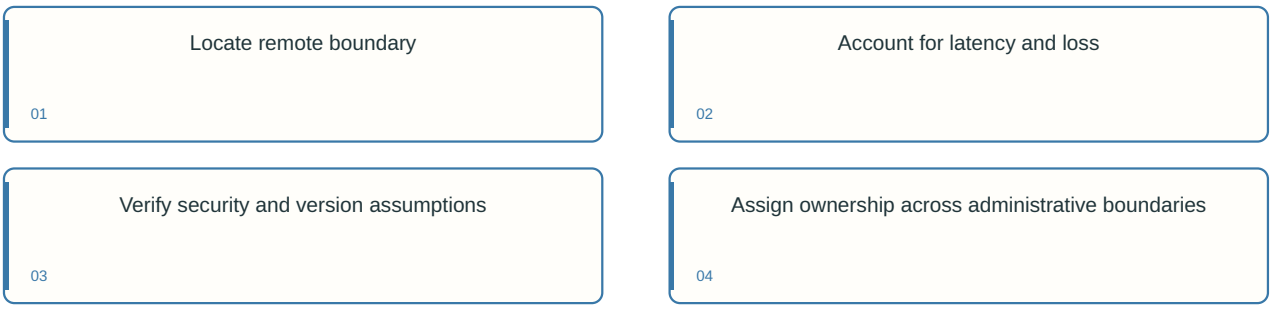


FIG. 14.4.2 **The fallacies of distributed computing · the example**

Distributed-computing fallacies expose hidden assumptions about networks, cost, topology, trust, and administration.

Author-created illustration. Numbers are illustrative unless stated otherwise.

14.4.3 · Safety vs liveness

“No seat is sold to two buyers” excludes a bad outcome: it is a safety property. “Every accepted booking eventually resolves” asks for progress, a liveness obligation that needs stated environmental assumptions.

Rejecting every request can preserve the first property while providing no useful progress. Conversely, replying immediately can appear responsive while breaking exclusive ownership. Write both requirements and identify which failures may delay completion. Some network and failure models prevent unconditional progress guarantees. Expose stuck work so operators can restore progress without inventing a second winner or losing an already accepted operation.

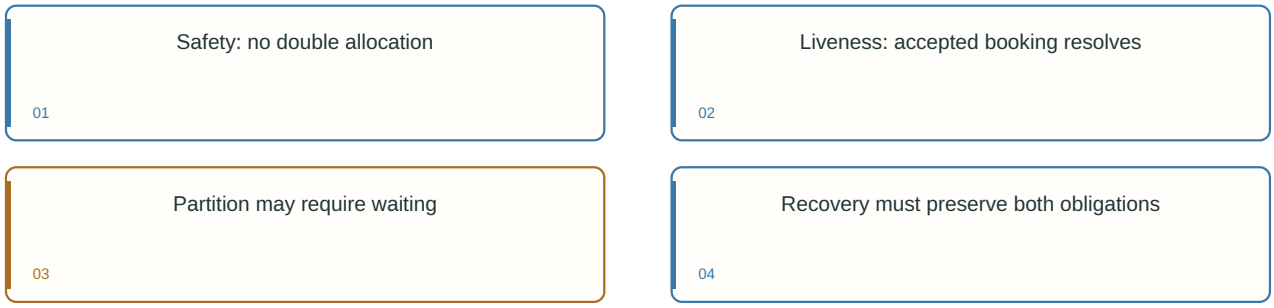


FIG. 14.4.3 **Safety vs liveness · the example**

Safety excludes bad outcomes; liveness requires progress under specified environmental conditions.

Author-created illustration. Numbers are illustrative unless stated otherwise.

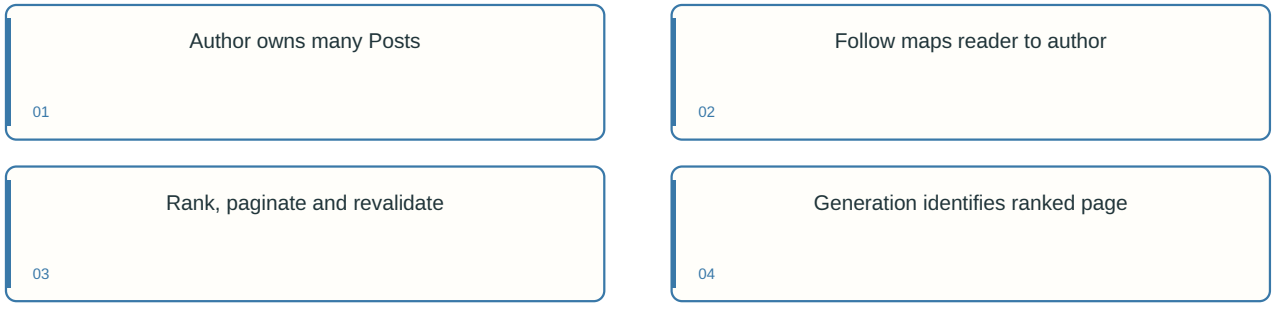


FIG. 14.4.4 **State and ownership in the related design**

The post and access relationships are authoritative; feed candidates are derived references.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Try the mechanism in code

PYTHON · ORDERING MODELS

```
from mechanisms import vector_relation, lamport_receive
assert vector_relation([1,0], [0,1]) == "concurrent"
assert vector_relation([1,0], [1,2]) == "before"
assert lamport_receive(5, 9) == 10
```

Neither [1,0] nor [0,1] dominates the other, so the model reports concurrency. [1,0] precedes [1,2], and receiving 9 at Lamport value 5 advances it to 10.

These are ordering calculations. A smaller Lamport number alone does not prove causation, and neither clock decides a business merge or consensus outcome.

Download the executable mechanism collection ↓ · Follow the connected complete design →

THE WORKBENCH · TRY CHANGING THE INPUTS

When must a read set meet a write set?

Write set example: A, B. Read set example: B, C. Shared members: B.

Every such fixed-set read/write pair must intersect. R + W = 4; N = 3.

A shared replica gives the read a place to encounter a written version. Version selection and membership rules still determine whether the whole protocol is linearizable.

Keep the same N replicas, then vary the number read and written. The calculation shows whether the sets must share a member. To decide which value a read can return, we also need rules for versions, concurrent writes, persistence, repair, and membership changes. Intersection alone does not prove linearizability.

▼ Think further · Which operations may continue during a partition while exclusive allocation must wait?

Operations with explicitly permitted stale or independent results may continue. Exclusive allocation needs valid authority and may have to wait or reject when agreement is unavailable. State per-

operation observations rather than weaken the entire system silently. Recovery must reconcile progress without introducing another owner.

Remember: Safety and liveness are separate obligations under an explicit failure model.

Try applying it: How could a system preserve exclusive ownership while still failing every user's useful request?

Unit 14 exercises → · Unit 14 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Jepsen: Consistency models ↗

   Primary practitioner research reference. Formal-history distinctions and references to underlying literature.

   `Access reviewed · 21 October 2026`

2. Time, Clocks, and the Ordering of Events in a Distributed System — Lamport, 1978 ↗

   Original research. Ordering reference for Unit 14.

   `Access reviewed · 21 October 2026`

3. Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services — Gilbert and Lynch, 2002 ↗

   Original research · university-hosted paper. CAP assumptions and impossibility; PDF text extraction is imperfect, original page inspected.

   `Access reviewed · 21 October 2026`

PART 4 / UNIT 15

Replication, Leader Election, and Consensus

A primary replies before a replica catches up. Compare replication models by write authority, acknowledgment milestones, conflicts, and the history available after failure.

The running example

Three database replicas, one leader, and a network partition. Walk through elections, writes, acknowledgement rules, recovery, and potential data loss.

Unit 15 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

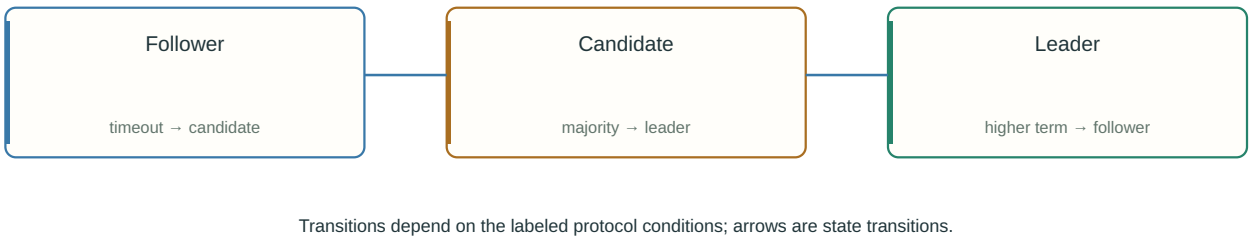
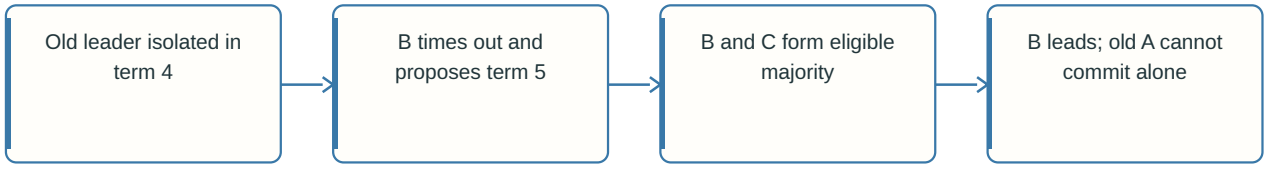


FIG. 15.0.1 **Raft election changes authority under protocol rules**

Election timeout can make a follower a candidate; a valid majority can elect an eligible leader. Observing a higher term returns a participant to follower. This is the role state machine, not the complete protocol.

Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 15.0.2 **election timeline**

The example assumes correct persistent voting/log rules and fixed non-Byzantine membership.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Participant	Durable log suffix	Meaning
Leader term 5	8:t4, 9:t5	current-term entry 9
Follower A	8:t4, 9:t5	required durable replica
Follower B	8:t4	lagging replica

FIG. 15.0.3 **Presence in a log is not the same as commitment**

Within the stated fixed-membership Raft protocol, majority replication of the current-term entry can advance commit and protect the prefix. This table is not a generic quorum proof.

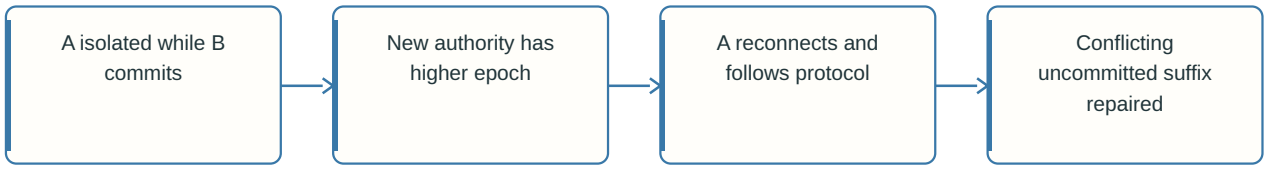
Author-created illustration. Numbers are illustrative unless stated otherwise.

Majority	Members	Shared witness
Election quorum 1	A, B	B
Election quorum 2	B, C	B
Required rules	Voting and durable log evidence	Intersection alone is insufficient

FIG. 15.0.4 **quorum intersection**

Safety comes from protocol rules applied to intersecting decision sets.

Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 15.0.5 **network partition recovery**

Recovery does not authorize an old leader to resume independent writes or ignore current term and committed history.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Choose a replication model

Prerequisites: Distributed Systems Fundamentals 6 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A primary replies before a replica catches up. Compare replication models by write authority, acknowledgment milestones, conflicts, and the history available after failure.

15.1.1 · Primary/replica architectures

A primary accepts the writes for a dataset and replicas keep copies. One current authority can simplify conflict handling, but the topology needs rules for replacing that authority and preventing an old primary from returning as a competing writer.

State when a write is acknowledged and which failures preserve it. Replica reads may show earlier data. “One primary, two replicas” describes placement, not a complete consensus protocol. Persistence, election eligibility, membership, and recovery determine the actual guarantee. Ask what happens to an acknowledged write if the primary disappears before its newest changes reach another eligible owner.

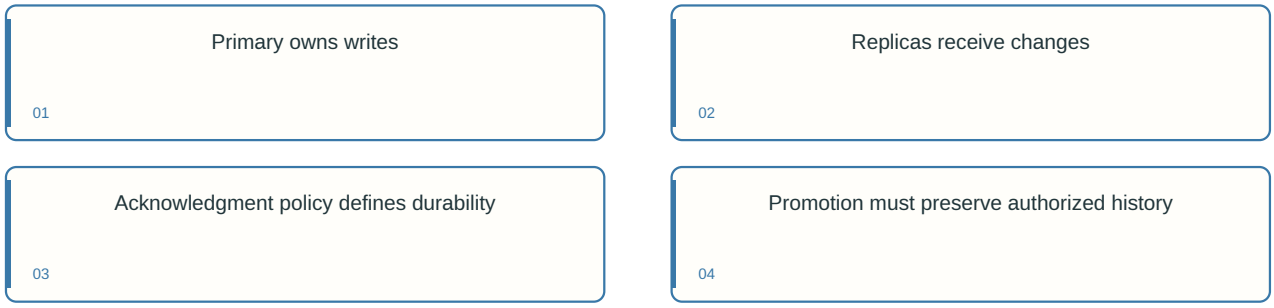


FIG. 15.1.1 **Primary/replica architectures · the example**

Primary/replica topology needs explicit write authority, acknowledgment, promotion, and stale-owner protection.

Author-created illustration. Numbers are illustrative unless stated otherwise.

15.1.2 · Leader/follower replication

A leader sends an ordered stream of updates to followers. A follower may receive bytes, persist them, and later apply them to its local state. These milestones are not necessarily simultaneous.

Track the positions relevant to the implementation. Receipt in memory may not provide crash-recovery protection. A follower serving reads may still be behind even after receiving recent records. Synchronous and asynchronous arrangements choose different acknowledgment points and waits. Promotion needs a rule about which history is safe to continue. Understanding these stages makes lag and durability claims more precise than saying that a follower “has the data.”

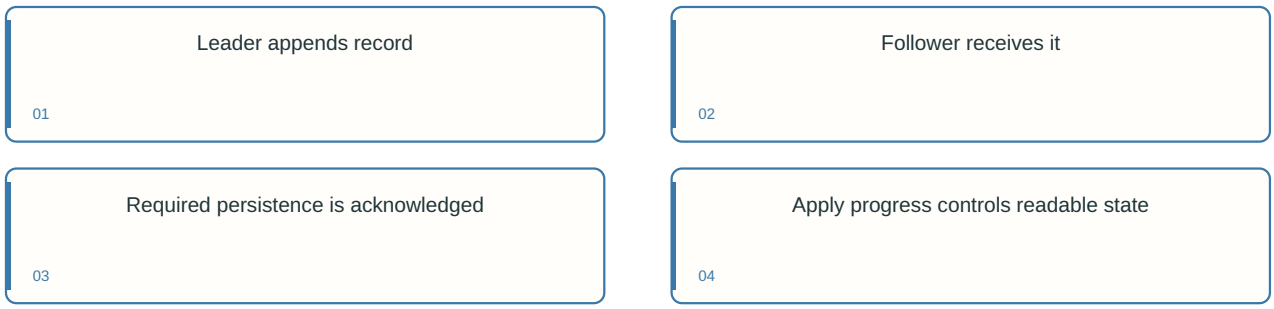


FIG. 15.1.2 **Leader/follower replication · the example**

Leader/follower replication orders updates, while receipt, persistence, commitment, and application can mark different progress.

Author-created illustration. Numbers are illustrative unless stated otherwise.

15.1.3 · Multi-leader replication

Two regions accept writes locally while their connection is interrupted. On reconnection, both may have changed the same record. Multi-leader replication needs a resolution rule for those concurrent updates.

Last-writer-wins can converge by choosing a timestamp, but clock differences or overwriting can discard meaningful work. Convergence alone does not establish a correct business merge. Use multiple writers where conflict semantics fit the domain, or add coordination for rules requiring one winner. Include deletes, prolonged disconnection, and replication-loop prevention in the design. Local write latency is a benefit only when the resulting observations and conflicts remain acceptable.

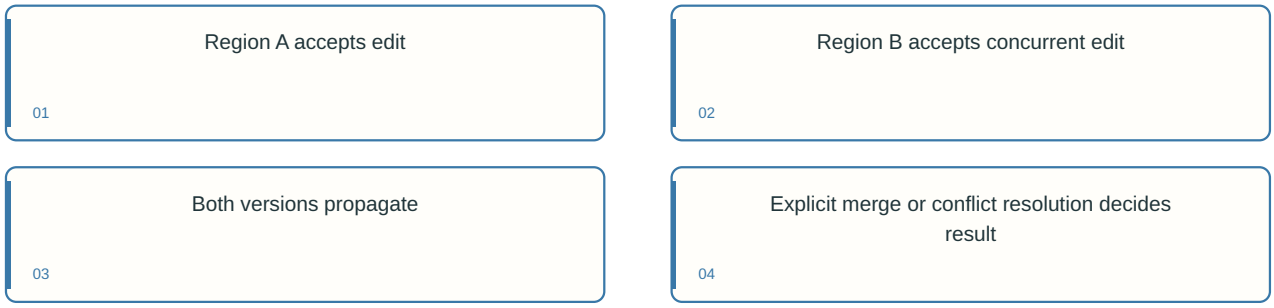


FIG. 15.1.3 **Multi-leader replication · the example**

Multiple write leaders need valid conflict semantics, not just deterministic convergence that discards business meaning.

Author-created illustration. Numbers are illustrative unless stated otherwise.

15.1.4 · Leaderless replication

A client contacts several replicas and compares their versions instead of sending every write through one permanent leader. Leaderless designs use protocols of this kind, with choices about read repair, hinted handoff, and background anti-entropy.

Each mechanism solves part of the problem. Quorum counts still need correct version interpretation, membership, and concurrency rules. Test two concurrent writes and a stale replica returning after a long failure. Choose the arrangement for its actual availability and conflict contract. Removing a permanent leader does not remove every need for coordination, nor does two replies out of three automatically establish linearizability.

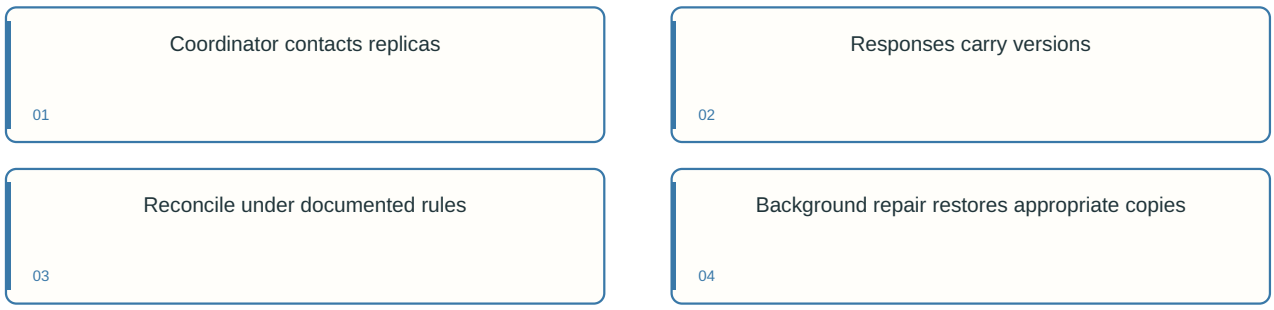


FIG. 15.1.4 **Leaderless replication · the example**

Leaderless protocols reconcile several replicas with implementation-specific version, repair, and membership rules.

Author-created illustration. Numbers are illustrative unless stated otherwise.

15.1.5 · Synchronous vs asynchronous replication

A primary can reply immediately and copy later, or wait for specified replica evidence. Asynchronous replication chooses the earlier reply. If the only current copy is then lost, a promoted replica may lack recent acknowledged writes.

Synchronous replication waits, but the word needs detail: does a replica acknowledge receipt, durable storage, or application? Waiting for independent durable copies can strengthen protection within the failure model, while adding latency and possible blocking. State which milestone success promises and test it during failover. The configuration is part of the application's data-loss contract, rather than an operating detail detached from users.

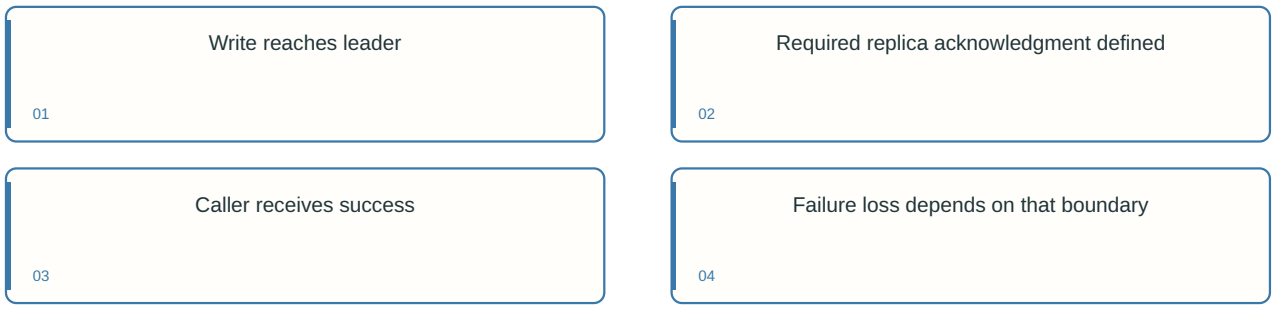


FIG. 15.1.5 **Synchronous vs asynchronous replication · the example**

Synchronous replication waits for named milestones; asynchronous acknowledgment can precede another durable copy.

Author-created illustration. Numbers are illustrative unless stated otherwise.

15.1.6 · Quorum-based systems

In a fixed three-node group, any two-member majority overlaps another two-member majority. This can carry evidence from one decision into a later one, provided the shared participant follows the protocol.

A quorum is an allowed decision set, not simply a vote count. Members may need to persist a vote or accepted record before replying and obey later restrictions based on it. Election eligibility and membership transitions also matter. Without these rules, intersecting sets can still acknowledge incompatible histories. Use quorum arithmetic to understand a reviewed protocol, rather than treating it as a complete algorithm for consistent storage.

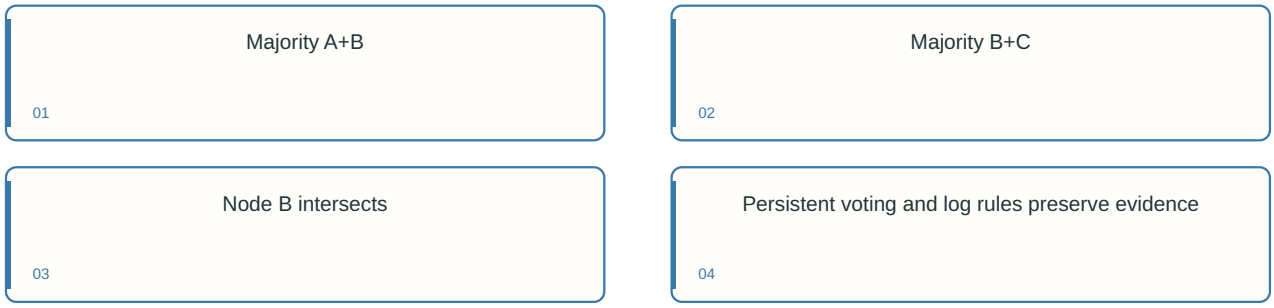


FIG. 15.1.6 **Quorum-based systems · the example**

A quorum is a permitted decision set within a protocol whose members preserve the required evidence.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · What must a repaired region establish before it accepts authoritative writes again?

It needs current authority, compatible committed history, required durable copies, valid routing, and sufficient capacity. An old leader may require rebuilding and fencing. Apply the supported rejoin protocol and reconcile uncertain external effects. A repaired machine being reachable is not evidence that its old state is safe to resume.

Remember: Replica count matters only with persistence, ownership, and recovery rules.

Try applying it: What must a synchronous acknowledgment mean if it is meant to survive process loss?

Unit 15 exercises → · Unit 15 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. MIT 6.5840: Distributed Systems, Spring 2026 ↗

   University course. Advanced distributed-systems reading.

   Access reviewed · 21 October 2026

2. In Search of an Understandable Consensus Algorithm — Ongaro & Ousterhout ↗

   Original research. Raft reference for Unit 15.

   Access reviewed · 21 October 2026

3. Paxos Made Simple — Leslie Lamport, 2001 ↗

   Original research. Conceptual consensus safety; atlas does not present toy code as production Paxos.

   Access reviewed · 21 October 2026

4. A Conflict-Free Replicated JSON Datatype — Kleppmann and Beresford ↗

   Original research · 2017 revision. One formal CRDT model; the miniature tie-break exercise is not this complete implementation.

   Access reviewed · 21 October 2026

Raft and replicated state machines

Prerequisites: 15.1 · Choose a replication model 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A new leader must preserve accepted history. Study deterministic command application and the election, log, commit, and membership rules that make replicated decisions safe.

15.2.1 · Consensus and replicated state machines

Suppose replicas agree on the sequence “create task 7, then complete task 7.” If each applies the same commands deterministically from the same state, they can maintain equivalent task state. This is the replicated state-machine approach.

Consensus supplies agreement under its stated fault and progress assumptions. Values such as random identifiers or current time must be part of an agreed command or otherwise handled consistently; recomputing them independently can make replicas differ. Also separate state-machine application from external effects. Applying one command on several replicas must not send several unprotected charges. External delivery needs explicit ownership, duplicate handling, and recovery outside the agreed log boundary.

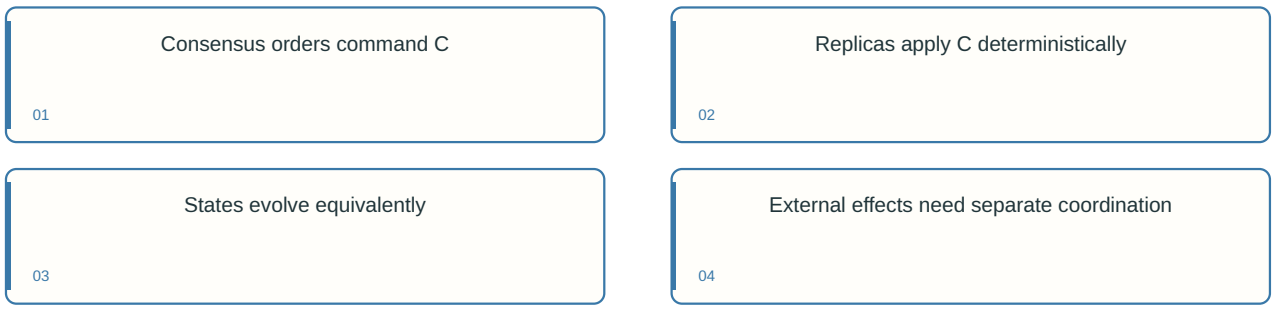


FIG. 15.2.1 **Consensus and replicated state machines · the example**

Replicated state machines apply agreed commands deterministically; external effects remain a separate boundary.

Author-created illustration. Numbers are illustrative unless stated otherwise.

15.2.2 · Raft: terms, elections, log replication, commit

Raft organises agreement around a leader and a replicated log. A term identifies an election generation. A candidate needs a majority of votes and a sufficiently up-to-date log under Raft's comparison rules. Votes and required log state are persisted as specified by the protocol.

The leader replicates entries and advances its commit position. It can directly use majority replication to commit a current-term entry; earlier entries become committed through the protocol's ordering rules. Counting copies of an arbitrary older entry is insufficient. This distinction preserves history across leadership changes. Reads also need a supported freshness mechanism. The paper's rules work together; this explanation is not a replacement implementation.

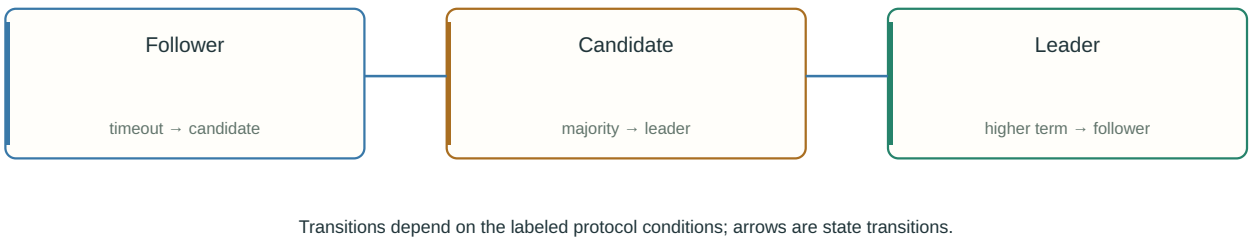


FIG. 15.2.2 **Raft election changes authority under protocol rules**

Election timeout can make a follower a candidate; a valid majority can elect an eligible leader. Observing a higher term returns a participant to follower. This is the role state machine, not the complete protocol.

Author-created illustration. Numbers are illustrative unless stated otherwise.

15.2.3 · Paxos: conceptual foundations

Paxos uses numbered proposals and intersecting groups to preserve an agreed choice. During preparation, a proposer gathers promises and any relevant earlier accepted value. It must carry the required value forward when it asks for acceptance.

The key idea is preserving prior evidence, rather than allowing each new leader to choose freely. Multi-Paxos uses related decisions to build an ordered sequence, often reducing repeated preparation around a stable leader. This is a conceptual introduction. Persistence, changing membership, retries, and progress under contention need precise rules. Use a reviewed implementation for real coordination instead of turning the introductory voting picture into production lock code.

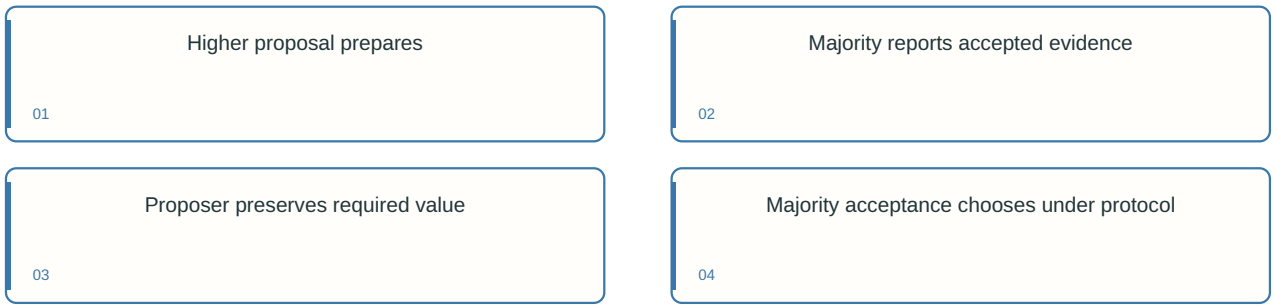


FIG. 15.2.3 **Paxos: conceptual foundations · the example**

Paxos preserves chosen decisions through numbered proposals and required prior evidence across intersecting majorities.

Author-created illustration. Numbers are illustrative unless stated otherwise.

15.2.4 · Log ordering and durability

A record can appear in one replica's log without being committed. A node can also have committed records it has not applied yet. Ordering, commitment, durable storage, and application describe different progress.

Recovery must retain the committed history while handling incompatible uncommitted suffixes. Reply according to the protocol's actual acceptance rule, rather than because bytes reached some follower. Monitor commit and apply progress, and ensure snapshot installation represents a compatible history. These distinctions explain why a local log inspection alone may not establish what the application is entitled to expose as an accepted result.

Participant	Durable log suffix	Meaning
Leader term 5	8:t4, 9:t5	current-term entry 9
Follower A	8:t4, 9:t5	required durable replica
Follower B	8:t4	lagging replica

FIG. 15.2.4 **Presence in a log is not the same as commitment**

Within the stated fixed-membership Raft protocol, majority replication of the current-term entry can advance commit and protect the prefix. This table is not a generic quorum proof.

Author-created illustration. Numbers are illustrative unless stated otherwise.

15.2.5 · Membership changes

Suppose an old cluster and a new cluster have disjoint decision majorities. Replacing membership abruptly can let both make incompatible decisions. A supported membership transition prevents that gap.

Raft's joint-consensus approach temporarily requires the relevant majorities from both configurations. New nodes can first catch up without voting. Use the implementation's reconfiguration procedure, preserving persistent identities and testing interruption during the change. Increasing a cluster-size setting is not merely adding capacity: it changes which groups have authority. The safety argument must remain valid at the transition, not only before and after it.

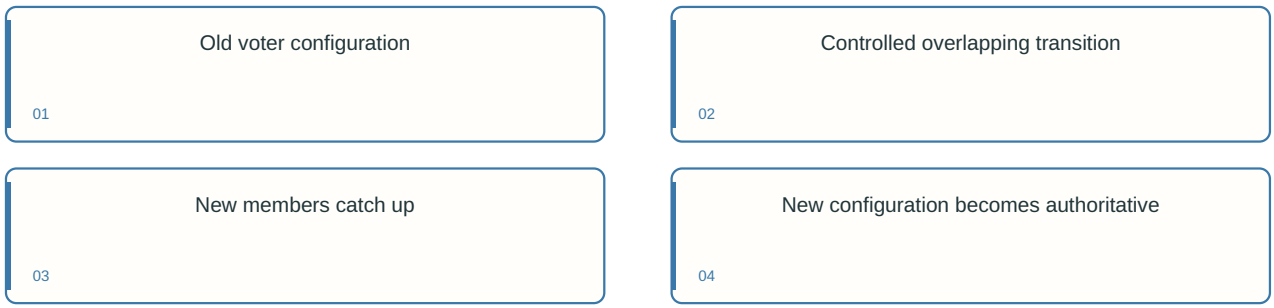


FIG. 15.2.5 **Membership changes · the example**

Membership changes need a supported transition preserving decision safety between old and new configurations.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · What must a repaired region establish before it accepts authoritative writes again?

It needs current authority, compatible committed history, required durable copies, valid routing, and sufficient capacity. An old leader may require rebuilding and fencing. Apply the supported rejoin protocol and reconcile uncertain external effects. A repaired machine being reachable is not evidence that its old state is safe to resume.

Remember: Consensus rules work together to preserve a decision history; a copy count is insufficient.

Try applying it: Why can an arbitrary old log entry on a majority still need more reasoning before commitment?

Unit 15 exercises → · Unit 15 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. MIT 6.5840: Distributed Systems, Spring 2026 ↗

   University course. Advanced distributed-systems reading.

   Access reviewed · 21 October 2026

2. In Search of an Understandable Consensus Algorithm — Ongaro & Ousterhout ↗

   Original research. Raft reference for Unit 15.

   Access reviewed · 21 October 2026

3. Paxos Made Simple — Leslie Lamport, 2001 ↗

   Original research. Conceptual consensus safety; atlas does not present toy code as production Paxos.

   Access reviewed · 21 October 2026

4. A Conflict-Free Replicated JSON Datatype — Kleppmann and Beresford ↗

   Original research · 2017 revision. One formal CRDT model; the miniature tie-break exercise is not this complete implementation.

   Access reviewed · 21 October 2026

Fences, leases & failure detection

Prerequisites: 15.2 · Raft and replicated state machines 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A paused worker wakes after its lease expired. Separate suspicion, ownership transfer, and resource-side enforcement to see where stale actions must be rejected.

15.3.1 · Split-brain prevention and fencing tokens

Worker A has token 7 and pauses. Its lease expires; B obtains token 8. A later resumes and sends an old write. The receiving resource must reject that stale authority.

A fencing token is an ordered ownership epoch checked at the effect boundary. The receiver must compare the token and apply the protected action atomically under its rule. Issuing a token that storage never checks does not fence a worker. Also account for effects already accepted before the newer owner took over. Fencing blocks later stale actions; it does not undo earlier accepted ones or automatically protect an unrelated external API.

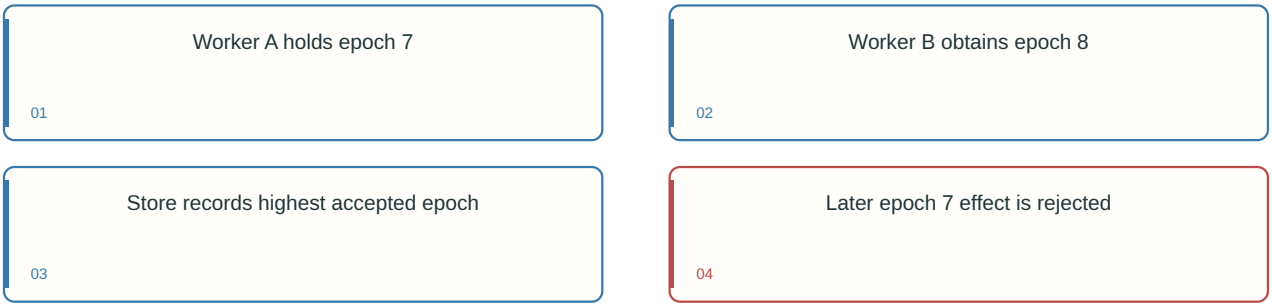


FIG. 15.3.1 **Split-brain prevention and fencing tokens · the example**

Fencing epochs reject stale authority only when receivers enforce them atomically with protected effects.

Author-created illustration. Numbers are illustrative unless stated otherwise.

15.3.2 · Failure detection

Missing heartbeats suggest failure, but a long pause or congested network can produce the same observation. Failure detectors trade responsiveness against false suspicions according to timing assumptions.

Shorter intervals can begin recovery sooner while increasing unnecessary suspicion. Keep this tuning separate from the protocol that transfers authority and fences stale owners. Safety should hold even if a suspicion is wrong. A received heartbeat also does not establish that every request can meet its deadline, so readiness and load signals can still be useful. Detection provides evidence; ownership rules determine which participant may act on it.

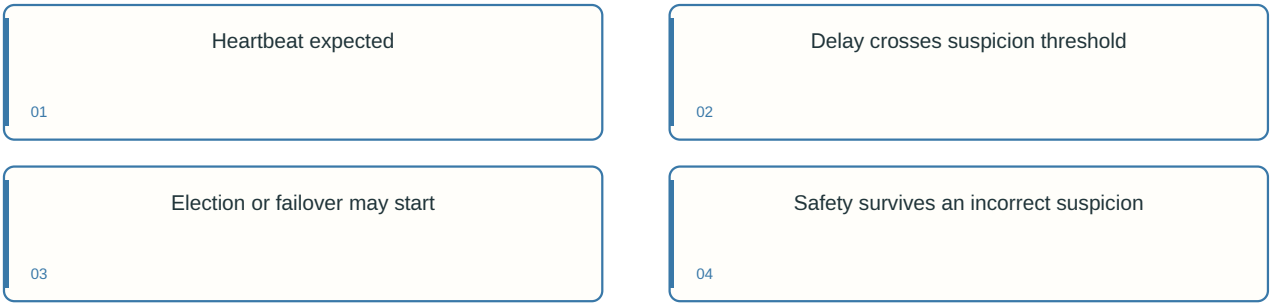


FIG. 15.3.2 **Failure detection · the example**

Failure detection produces suspicions under timing assumptions; authority safety must tolerate false suspicion.

Author-created illustration. Numbers are illustrative unless stated otherwise.

15.3.3 · Strongly consistent metadata services

A metadata service can maintain current ownership epochs or configuration through a strongly consistent protocol. These are usually small control decisions, rather than the full high-volume application payload.

Identify which actions need a fresh decision and which can use safe cached information. During a metadata outage, existing traffic might continue while new leases or deployments must wait. Drawing this control/data distinction makes the failure policy clearer. Avoid placing every request behind a synchronous control-plane lookup without a reason. The control service's own capacity, fault assumptions, and recovery are still dependencies of the decisions it owns.

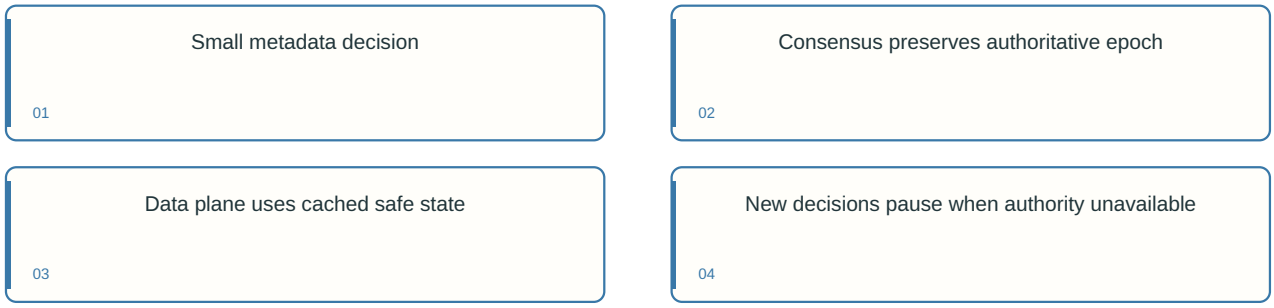


FIG. 15.3.3 **Strongly consistent metadata services · the example**

Consistent metadata services own small control decisions, with explicit consequences when new decisions cannot proceed.

Author-created illustration. Numbers are illustrative unless stated otherwise.

15.3.4 · Distributed locks and leases

A lock grants permission to a named owner. A lease makes that permission time-bounded under particular timing rules. The holder can pause past expiry and later continue unless the protected resource recognises that its authority is stale.

Use resource-side fencing where that history threatens correctness. Release operations must check ownership, so an old holder cannot delete a newer owner's lock. Acquisition and the later protected action are separate events with a gap between them. For an invariant entirely inside one database, a local transaction may avoid unnecessary distributed locking. Choose coordination for the actual shared resource and failure history.

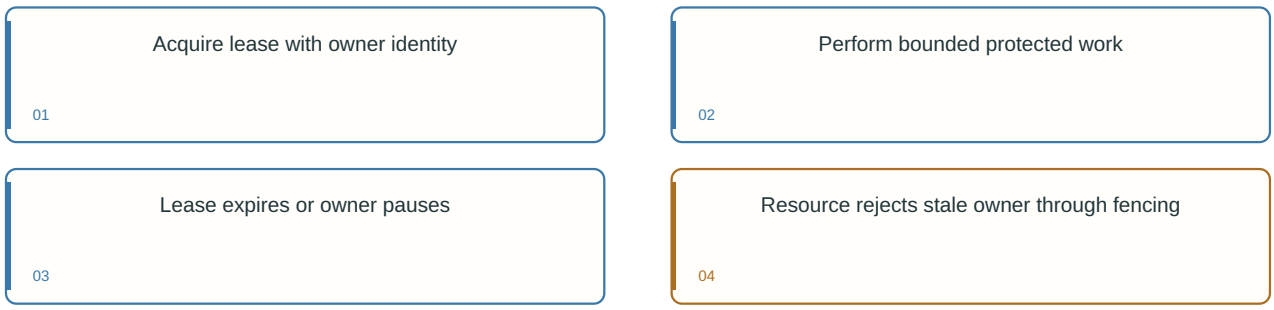


FIG. 15.3.4 **Distributed locks and leases · the example**

Locks and leases grant permission with identity and expiry rules, while resumed stale effects may still need fencing.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · What must a repaired region establish before it accepts authoritative writes again?

It needs current authority, compatible committed history, required durable copies, valid routing, and sufficient capacity. An old leader may require rebuilding and fencing. Apply the supported rejoin protocol and reconcile uncertain external effects. A repaired machine being reachable is not evidence that its old state is safe to resume.

Remember: Failure detection does not stop a process; the protected effect needs valid authority enforcement.

Try applying it: Where must token 7 be rejected after token 8 has become current?

Unit 15 exercises → · Unit 15 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. MIT 6.5840: Distributed Systems, Spring 2026 ↗

   University course. Advanced distributed-systems reading.

   Access reviewed · 21 October 2026

2. In Search of an Understandable Consensus Algorithm — Ongaro & Ousterhout ↗

   Original research. Raft reference for Unit 15.

   Access reviewed · 21 October 2026

3. Paxos Made Simple — Leslie Lamport, 2001 ↗

   Original research. Conceptual consensus safety; atlas does not present toy code as production Paxos.

   Access reviewed · 21 October 2026

4. A Conflict-Free Replicated JSON Datatype — Kleppmann and Beresford ↗

   Original research · 2017 revision. One formal CRDT model; the miniature tie-break exercise is not this complete implementation.

   Access reviewed · 21 October 2026

Conflicts, CRDTs & regional trade-offs

Prerequisites: 15.3 · Fences, leases & failure detection 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Replicas must merge concurrent changes or choose which authority may proceed across regions. Evaluate convergence and recovery against the business invariant rather than as ends in themselves.

15.4.1 · Conflict resolution and CRDT introduction

A grow-only set can merge copies by union: each addition remains present, and merging in different orders converges. CRDTs design types around update and merge rules with suitable algebraic and delivery assumptions.

More complex types need metadata. An observed-remove set distinguishes additions known at removal from concurrent additions. Convergence does not enforce an arbitrary rule such as nonnegative stock. Choose semantics that match the domain, and preserve the metadata needed for safe merge and cleanup. Some business edits are better handled by explicit conflict review. An automatic convergent answer is useful only if that answer means the right thing.

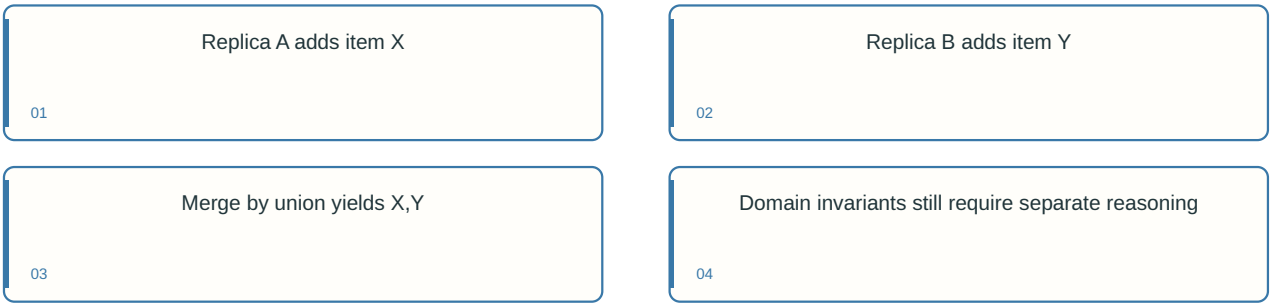


FIG. 15.4.1 **Conflict resolution and CRDT introduction · the example**

CRDTs converge through specified update/merge rules but do not enforce arbitrary business invariants.

Author-created illustration. Numbers are illustrative unless stated otherwise.

15.4.2 · Multi-region replication trade-offs

A local asynchronous reply and a cross-region durable quorum expose different latency and possible-loss behaviour. Regional replication must state where writes are authoritative, which copies acknowledgment requires, and how conflicts or promotion work.

During a partition, some operations may reject while permitted stale reads continue. On reconnection, an old primary must catch up or reconcile before it is trusted to write again. Test failback as well as failover. Restoring traffic quickly can conceal a second failure if stale authority or history returns after the visible outage. The regional design includes that return path, not merely the initial switch.

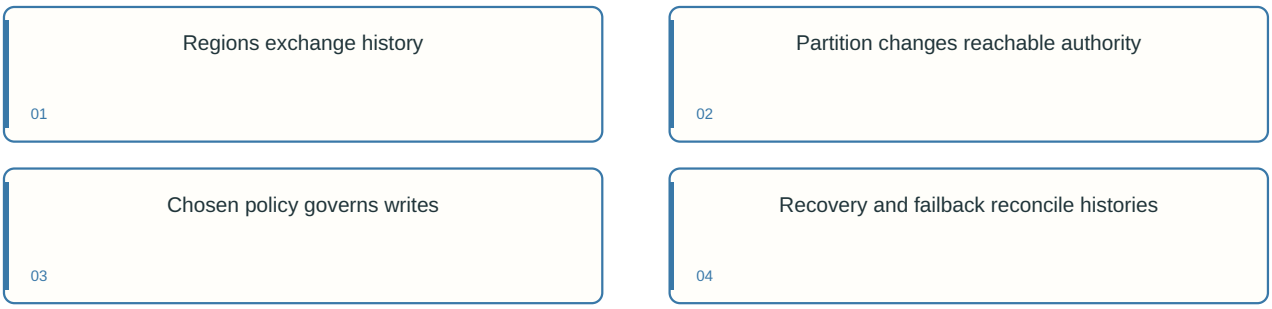


FIG. 15.4.2 **Multi-region replication trade-offs · the example**

Regional replication needs authority, acknowledgment geography, conflict, failover, and safe return-to-service rules.

Author-created illustration. Numbers are illustrative unless stated otherwise.

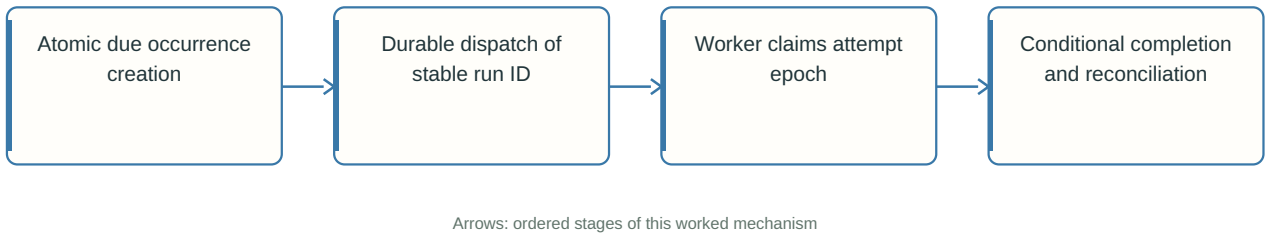


FIG. 15.4.3 **Follow the related complete design**

Create due work durably before a current worker attempts its effect.

Author-created illustration. Numbers are illustrative unless stated otherwise.

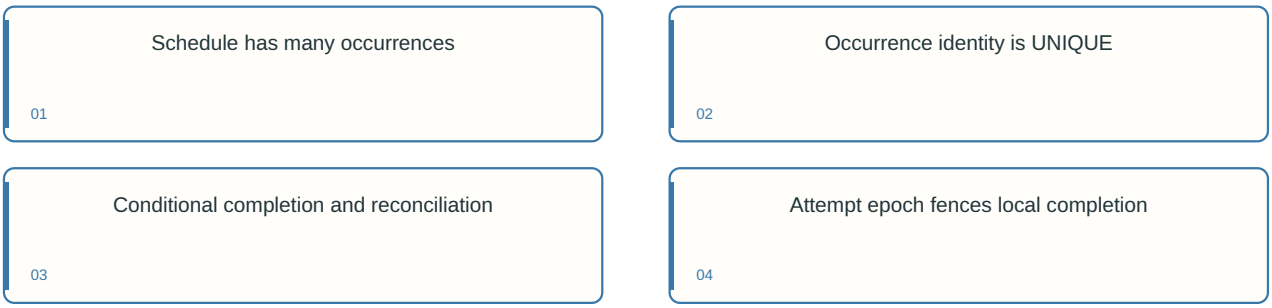


FIG. 15.4.4 **State and ownership in the related design**

One scheduled occurrence can have several attempts, while retaining one logical run identity.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Try the mechanism in code

PSEUDOCODE · THREE REPLICAS, ONE PARTITION

```
A,B,C contain committed index 8
partition isolates old leader A
B and C elect eligible B in term 5
B appends index 9, term 5
B and C durably replicate index 9
Raft commit rule establishes committed prefix
A cannot acknowledge a new quorum-committed write alone
rejoin: A follows term 5 and repairs uncommitted suffix
```

The example assumes correct non-Byzantine participants, durable protocol state, and three fixed voters. B's current-term entry reaches a valid majority while A is isolated.

Follow the full Raft rules for election, log repair, commitment, reads, and membership. This trace explains one history and is not an executable replacement protocol.

Download the executable mechanism collection ↓ · Follow the connected complete design →

▼ Think further · What must a repaired region establish before it accepts authoritative writes again?

It needs current authority, compatible committed history, required durable copies, valid routing, and sufficient capacity. An old leader may require rebuilding and fencing. Apply the supported rejoin protocol and reconcile uncertain external effects. A repaired machine being reachable is not evidence that its old state is safe to resume.

Remember: A deterministic merge is useful only when its meaning and failure assumptions fit the domain.

Try applying it: Why can a convergent counter still be unsuitable for allocating the last item?

Unit 15 exercises → · Unit 15 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. MIT 6.5840: Distributed Systems, Spring 2026 ↗

   University course. Advanced distributed-systems reading.

   Access reviewed · 21 October 2026

2. In Search of an Understandable Consensus Algorithm — Ongaro & Ousterhout ↗

   Original research. Raft reference for Unit 15.

   Access reviewed · 21 October 2026

3. Paxos Made Simple — Leslie Lamport, 2001 ↗

   Original research. Conceptual consensus safety; atlas does not present toy code as production Paxos.

   Access reviewed · 21 October 2026

4. A Conflict-Free Replicated JSON Datatype — Kleppmann and Beresford ↗

   Original research · 2017 revision. One formal CRDT model; the miniature tie-break exercise is not this complete implementation.

   Access reviewed · 21 October 2026

PART 4 / UNIT 16

Message Queues, Events, and Distributed Workflows

A message is accepted, delivered, executed, and acknowledged at different times. Trace those boundaries to understand queues, logs, repetition, and the scope of delivery guarantees.

The running example

Design an e-commerce checkout that coordinates payment, inventory reservation, shipment creation, and refund compensation.

Unit 16 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

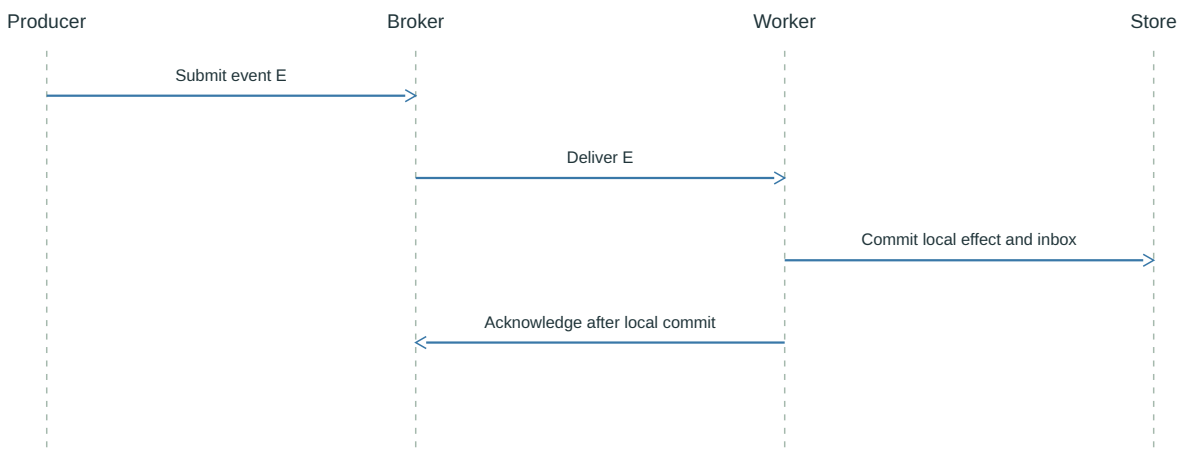


FIG. 16.0.1 **Queue lifecycle**

A lost final acknowledgment can cause redelivery; the local effect contract must handle it.

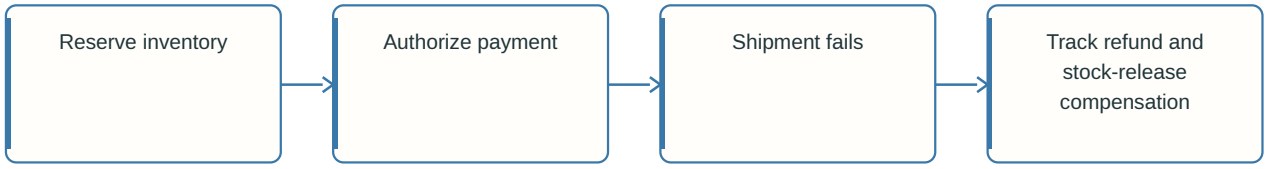
Author-created illustration. Numbers are illustrative unless stated otherwise.

Partition	Owner before	Owner after rebalance
P0	Consumer A	Consumer B
P1	Consumer B	Consumer B
Progress	Last committed offset	In-flight work may replay

FIG. 16.0.2 **consumer partition assignment**

Ordering is per partition under this model, and rebalancing does not make in-flight side effects disappear.

Author-created illustration. Numbers are illustrative unless stated otherwise.

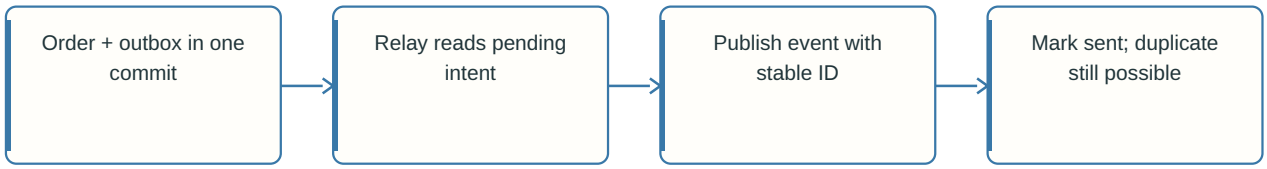


Arrows: ordered stages of this worked mechanism

FIG. 16.0.3 **Saga happy/failure paths**

Compensation has its own retry, uncertainty, and final-state obligations.

Author-created illustration. Numbers are illustrative unless stated otherwise.

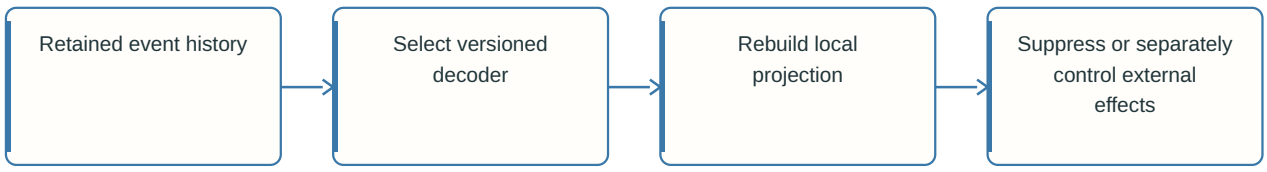


Arrows: ordered stages of this worked mechanism

FIG. 16.0.4 **The outbox closes a local publication gap**

The relay can crash after publish but before marking. A consumer must handle the repeated event; the database transaction does not include the entire network.

Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 16.0.5 **event replay**

Replaying state must not blindly repeat emails, charges, or other external business effects.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Queues, logs & delivery boundaries

Prerequisites: API Design and Communication Contracts, Database Internals, Transactions, and Concurrency Control, Distributed Systems Fundamentals, Replication, Leader Election, and Consensus

9 original figures

Blue: normal flow · Green: healthy · Amber: caution · Red: failure · Purple: control / shared infrastructure · Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A message is accepted, delivered, executed, and acknowledged at different times. Trace those boundaries to understand queues, logs, repetition, and the scope of delivery guarantees.

16.1.1 · Synchronous vs asynchronous communication

Checkout can save an order and let receipt delivery happen later. Synchronous communication makes the caller wait for a result; asynchronous communication separates submission from observing completion.

The delayed work still needs an owner. If accepting an order promises a receipt workflow, record the required intent durably instead of leaving it only in the request's memory. A queue can absorb a burst, but continued arrivals above worker capacity still build a backlog. State what the initial response means and where clients or operators can see progress after the initiating request ends.

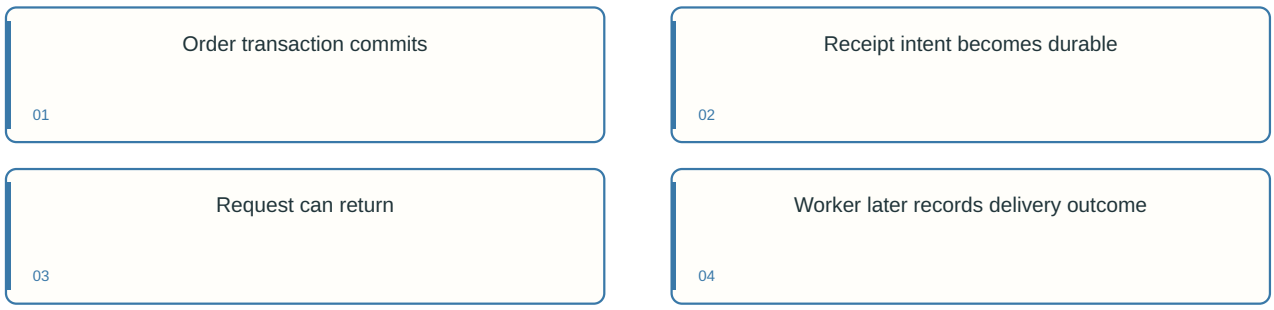


FIG. 16.1.1 **Synchronous vs asynchronous communication · the example**

Asynchronous submission separates acceptance from completion and needs an owner for the intervening work.

Author-created illustration. Numbers are illustrative unless stated otherwise.

16.1.2 · Queues vs logs vs pub/sub

A queue commonly distributes jobs among consumers. A retained log keeps an append history that consumers can replay. Pub/sub delivers publications to subscribers according to a particular retention and acknowledgment policy.

Products combine these behaviours, so read their actual contract. A subscriber absent during an ephemeral publication may never receive it. A consumer that must recover after a long outage needs enough retention and a stored progress position. Choose by recovery and ordering requirements, rather than assuming that every broker is a durable history. The important question is which records remain and how a returning consumer finds its next work.

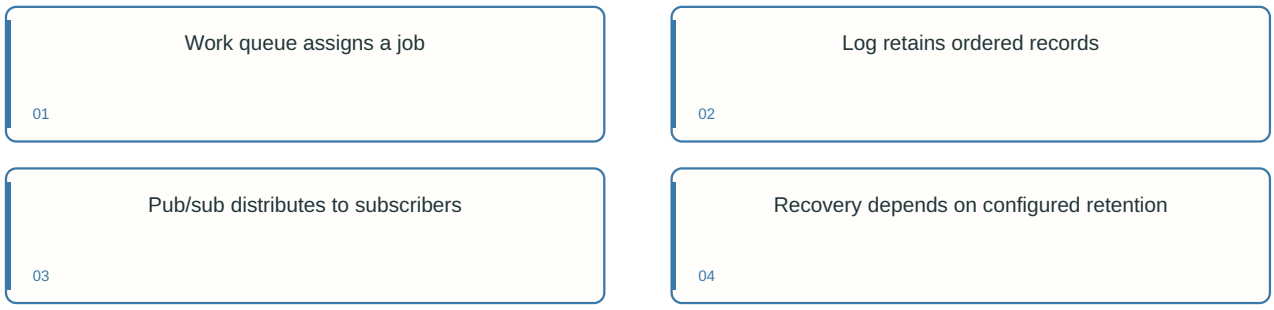


FIG. 16.1.2 **Queues vs logs vs pub/sub · the example**

Queues distribute work, logs retain ordered history, and pub/sub follows its stated subscription and retention contract.

Author-created illustration. Numbers are illustrative unless stated otherwise.

16.1.3 · Producers, consumers, brokers

A producer submits a message, the broker stores or routes it, and a consumer performs work. “Acknowledged” can refer to several points: broker receipt, durable replication, consumer receipt, or completion of the business effect.

Name the point that matters. Include stable message identity, schema version, ownership, and correlation information. A broker may be healthy while every consumer rejects the payload. Observe end-to-end age and outcomes as well as enqueue rate and broker uptime. Following one message through these stages reveals which component has actually accepted responsibility at each step.

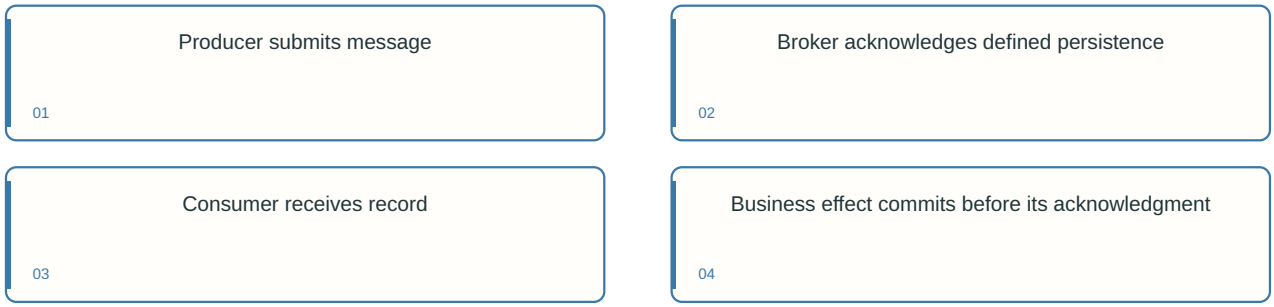


FIG. 16.1.3 **Producers, consumers, brokers · the example**

Producers, brokers, and consumers acknowledge different stages that must be named to reason about ownership.

Author-created illustration. Numbers are illustrative unless stated otherwise.

16.1.4 · Partitions, consumer groups, offsets

Kafka-style partitions provide ordered lanes. An offset identifies progress in one lane, and a consumer group distributes partition work among members. Order within a partition does not establish order across all partitions.

When group membership changes, a partition can move to another consumer. Work near the progress-commit boundary may repeat. Choose keys for the ordering the domain needs and commit progress according to the effect contract. One very popular partition can limit the group while other consumers are idle. Adding workers does not divide that lane automatically. Separate lane ownership, processing concurrency, and completed effects when reasoning about throughput.

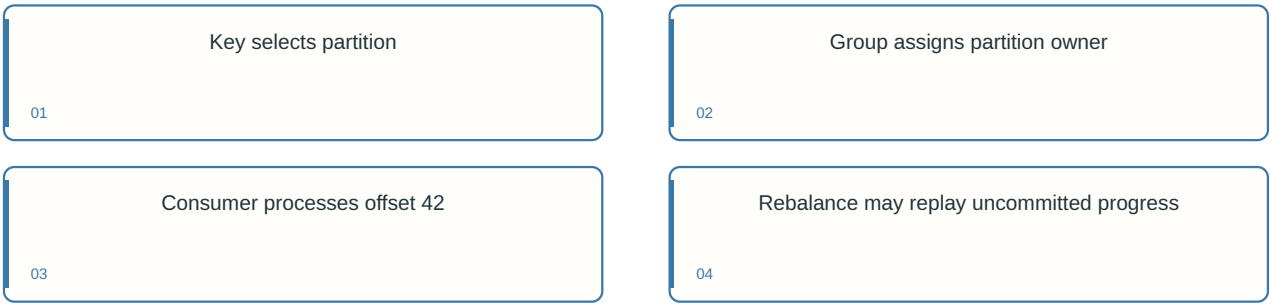


FIG. 16.1.4 **Partitions, consumer groups, offsets · the example**

Partitions provide ordered lanes; consumer groups and offsets track ownership and progress within those lanes.

Author-created illustration. Numbers are illustrative unless stated otherwise.

16.1.5 · Delivery guarantees

A broker delivers the same record twice. A consumer can still make one local database change if it recognises the event identity and commits that recognition with the change. A later external call may have different repetition behaviour.

Delivery guarantees need a boundary: are we counting broker deliveries, execution starts, or committed effects? Trace source acceptance, retained records, retries, effect commit, and progress acknowledgment. A guarantee at one link does not automatically extend across the whole path. This vocabulary becomes useful when each stage states exactly what survives a crash and what can be repeated.

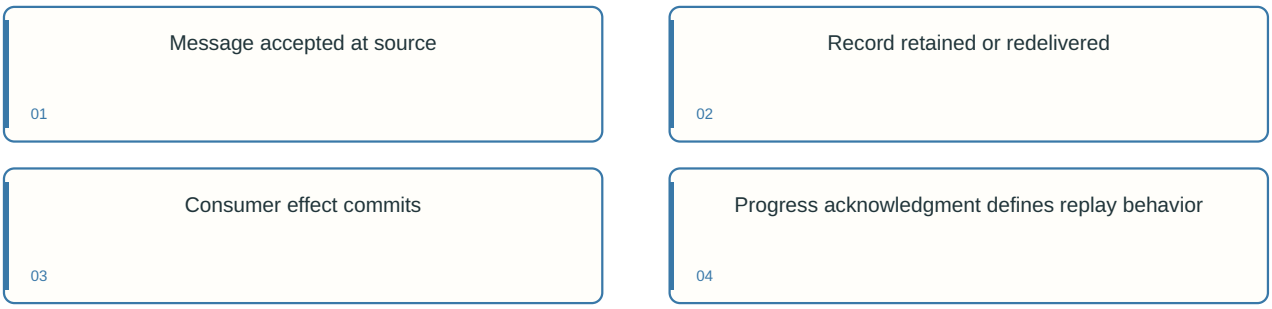


FIG. 16.1.5 **Delivery guarantees · the example**

Delivery guarantees need a stated record or effect boundary and do not automatically compose across external systems.

Author-created illustration. Numbers are illustrative unless stated otherwise.

16.1.6 · At-most-once and at-least-once processing

A consumer acknowledges before applying an effect, then crashes. The broker may not redeliver, so the effect is missing. This placement can support at-most-once handling at the chosen boundary: zero outcomes remain possible.

If the consumer applies the effect before acknowledging, a crash in between can cause redelivery. This supports at-least-once handling under the retention and recovery assumptions, but repetition must be safe. For one local database effect, store the processed event identifier and effect in the same transaction. Doing those steps separately creates another failure gap. Neither placement alone makes an unrelated external call atomic with broker progress.

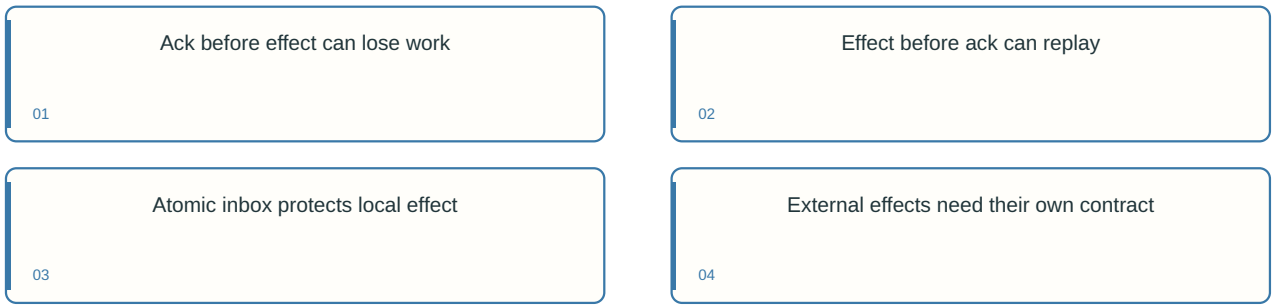


FIG. 16.1.6 **At-most-once and at-least-once processing · the example**

At-most-once can lose required effects; at-least-once can repeat them and therefore needs safe effect handling.

Author-created illustration. Numbers are illustrative unless stated otherwise.

16.1.7 · Exactly-once processing within defined boundaries

A stream-processing transaction can couple consumed progress and produced records inside a system that supports that operation. An exactly-once claim can be meaningful within that stated boundary and fault model.

Now substitute an external email or payment provider for the produced record. Unless another protocol covers it, the transaction cannot establish one external effect. Idempotency may make repeated attempts have one relevant outcome under a key and retention policy, while executions can still start more than once. Name the records, effects, failures, and recovery covered by the guarantee. Avoid treating “exactly once” as a property that automatically crosses every connected service.

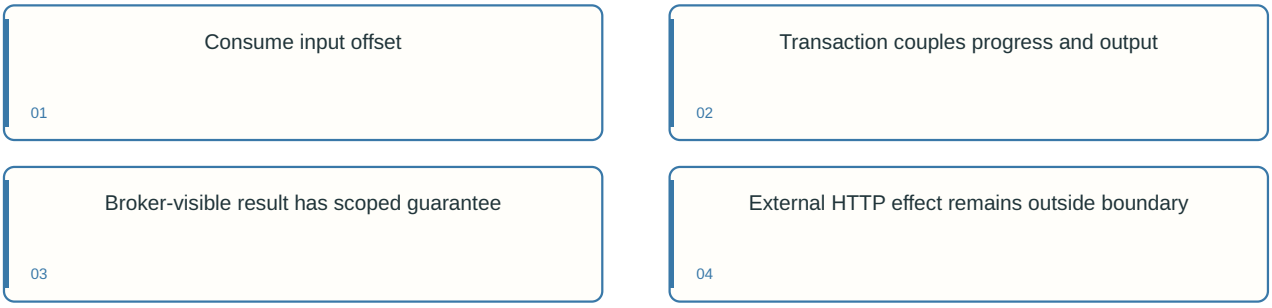


FIG. 16.1.7 **Exactly-once processing within defined boundaries · the example**

Exactly-once guarantees cover only the supported atomic boundary and its identity, retention, and failure assumptions.

Author-created illustration. Numbers are illustrative unless stated otherwise.

16.1.8 · Ordering guarantees

Messages can be published in order and still commit effects out of order when consumers process them in parallel. Distinguish log order, delivery order, start order, and effect-completion order.

If account changes require sequential application, coordinate by account or an equivalent ordering boundary. A later message should not be treated as applied while an earlier required change remains unresolved. Poison records need a deliberate policy: skipping restores movement but may break the sequence. Sequence numbers and gap detection can help expose that choice. State the business ordering requirement before selecting a partitioning and concurrency arrangement.

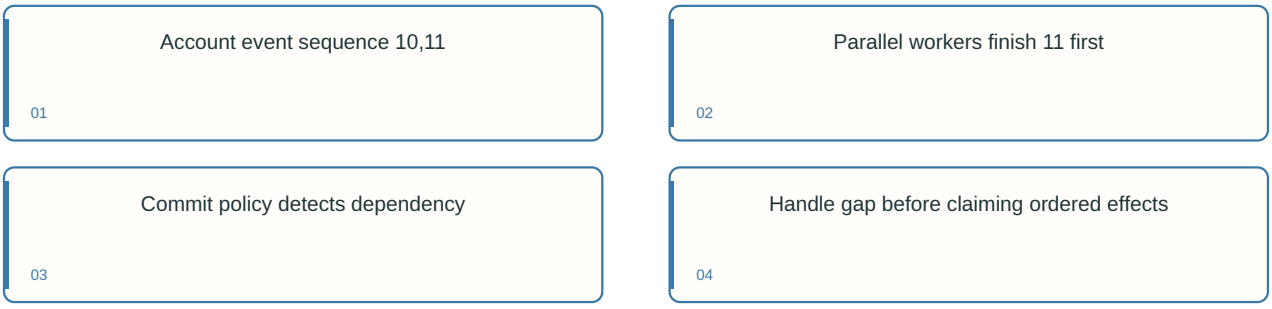


FIG. 16.1.8 **Ordering guarantees · the example**

Publication, delivery, execution, and effect order can differ when work proceeds concurrently.

Author-created illustration. Numbers are illustrative unless stated otherwise.

16.1.9 · Consumer acknowledgements

A consumer finishes its local update, but its acknowledgment is lost. The broker can redeliver the record. If acknowledgment were earlier, a crash could instead lose the update.

The acknowledgment tells the broker what progress it may record under its protocol. No placement makes it automatically atomic with a separate external system. Use appropriate local transactions, inbox/outbox records, provider keys, and reconciliation. Also match acknowledgment deadlines or lease extensions to expected processing time. Otherwise, healthy long-running work may be reassigned repeatedly. Identify the effect boundary first, then choose where progress can safely advance.

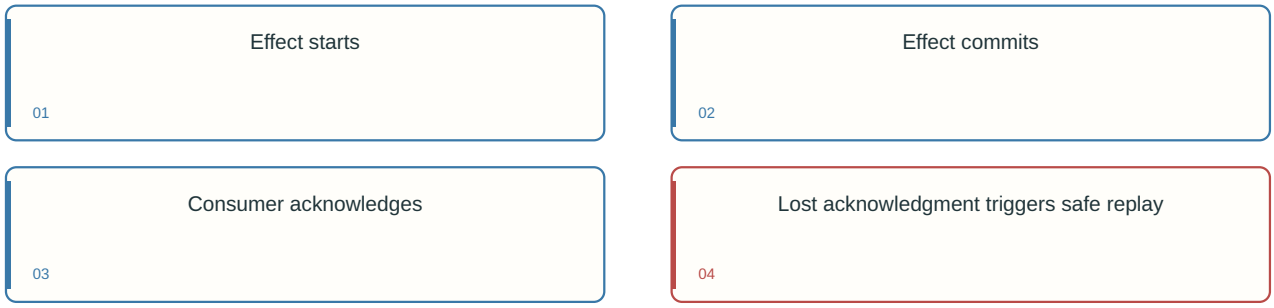


FIG. 16.1.9 **Consumer acknowledgements · the example**

Consumer acknowledgment advances broker progress, with loss or repetition gaps around separately committed effects.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · How could one broker transaction be exactly-once while two emails are sent?

The broker may atomically couple consumed progress and produced records while email lies outside that transaction. A consumer can send, lose its completion reply, and send again on recovery. Use the provider contract, stable identity where supported, and reconciliation; do not extend the broker guarantee to an unrelated effect.

Remember: Name the accepted record or effect before claiming at-most-once, at-least-once, or exactly-once behaviour.

Try applying it: What changes if the consumer replaces its local counter update with an external email call?

Unit 16 exercises → · Unit 16 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Saga distributed transactions pattern — Microsoft ↗

   Official engineering guidance. Local transactions and compensation; compensation is not always a perfect inverse.

   `Access reviewed · 21 October 2026`

2. Transactional Outbox — Chris Richardson ↗

   Pattern author's explanation. Publication intent and duplicate delivery.

   `Access reviewed · 21 October 2026`

3. Apache Kafka 4.1: Design ↗

   Official versioned documentation. Partitioning and scoped transactional/processing behavior; external effects are separate.

   `Access reviewed · 21 October 2026`

Retries, consumers & event history

Prerequisites: 16.1 · Queues, logs & delivery boundaries 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A consumer crashes after applying an event, while another record fails on every retry. Design repeat-safe effects and a repair path, then distinguish delivery history from authoritative domain history.

16.2.1 · Retries and dead-letter queues

A temporary provider outage may deserve a delayed retry. A permanently invalid address will not improve because the same payload is sent a thousand times. Classify failures and bound the attempts and waiting time.

A dead-letter queue holds work needing another path, such as repair or review. It does not mean the business task is complete. Retain original identity, source version, attempt information, and a safe error reason. Give important unresolved work an owner and alerting. Replay must preserve duplicate and ordering rules. A repair process should restore valid progress rather than turn rejected records into uncontrolled new operations.

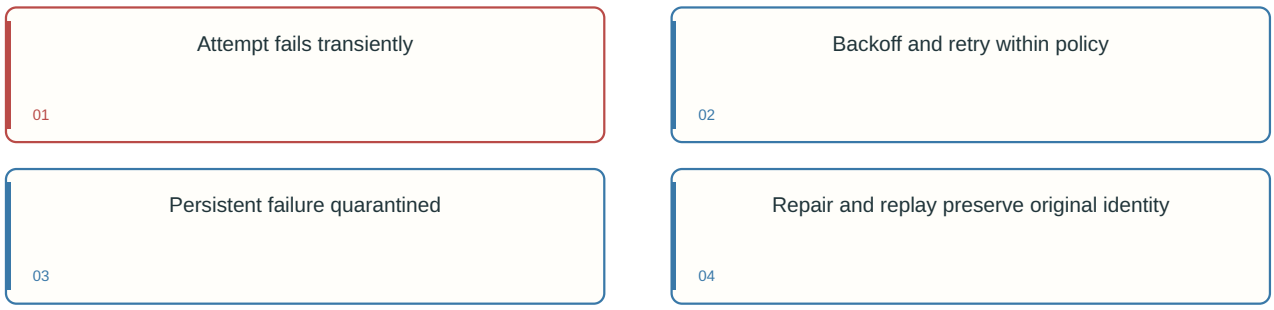


FIG. 16.2.1 **Retries and dead-letter queues · the example**

Retry and dead-letter policies need bounds, useful error context, ownership, and a safe repair/replay path.

Author-created illustration. Numbers are illustrative unless stated otherwise.

16.2.2 · Poison messages

A particular message fails every time while neighbouring messages work. It may have malformed data, an unsupported schema, or a rule violation. Such a poison message needs a different response from a transient network fault.

Quarantine enough evidence to diagnose it and decide whether dependent records must wait. Avoid retaining unnecessary sensitive payloads. A software bug can also reject an entire legitimate historical version, so inspect failure patterns and samples before discarding data. Bounded retries protect resources while a review path protects the business work. Both are needed when skipping a record could leave an important operation unresolved.

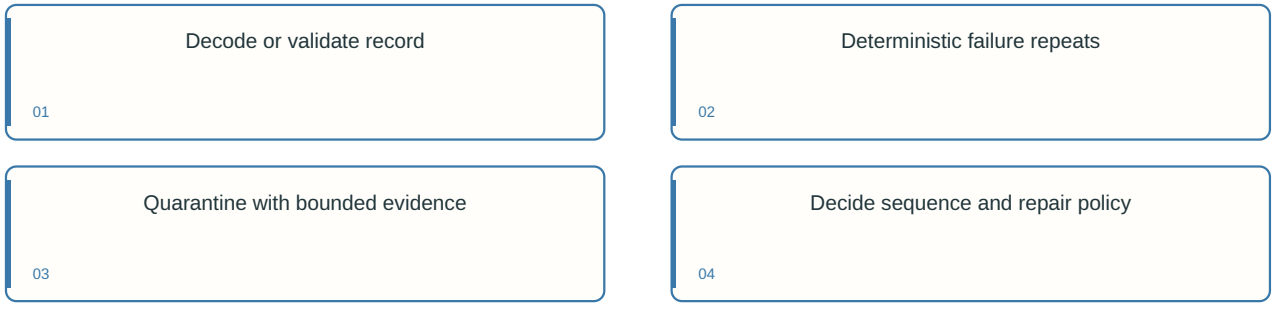


FIG. 16.2.2 **Poison messages · the example**

Poison messages need diagnosis and controlled quarantine rather than unlimited identical retries.

Author-created illustration. Numbers are illustrative unless stated otherwise.

16.2.3 · Idempotent consumers

The first delivery inserts processed(event_id) and increments a counter in one transaction. A second delivery encounters the same unique identity and does not repeat the increment. This is an idempotent consumer for that local effect.

Keep the identity long enough for the supported replay and delivery window. If recognition is only in memory, restart loses it. If recognition commits separately from the effect, a crash can either lose or repeat the work. External providers remain outside this local transaction, so they need their own stable key or reconciliation. State what outcome is protected rather than assuming the consumer's name establishes it.

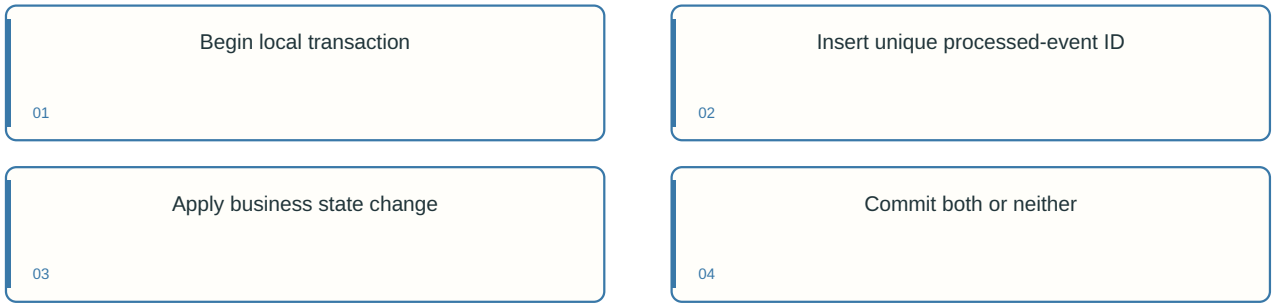


FIG. 16.2.3 **Idempotent consumers · the example**

Idempotent consumers commit recognition and the protected local effect together, retaining identity for supported replay.

Author-created illustration. Numbers are illustrative unless stated otherwise.

16.2.4 · Event sourcing

Instead of storing only the latest order state, event sourcing keeps the accepted domain events as its authoritative history. Applying them in order reconstructs the current state. A snapshot can speed this work if it records the event position it represents.

The history must remain interpretable as schemas change. Define corrections, replay, privacy, and retention. Rebuilding a projection must not resend real charges or emails merely because their events are read again. Auditability also depends on domain and access rules; append-only storage is not automatically a sufficient financial ledger. Choose the approach for a real need to preserve and interpret that history.

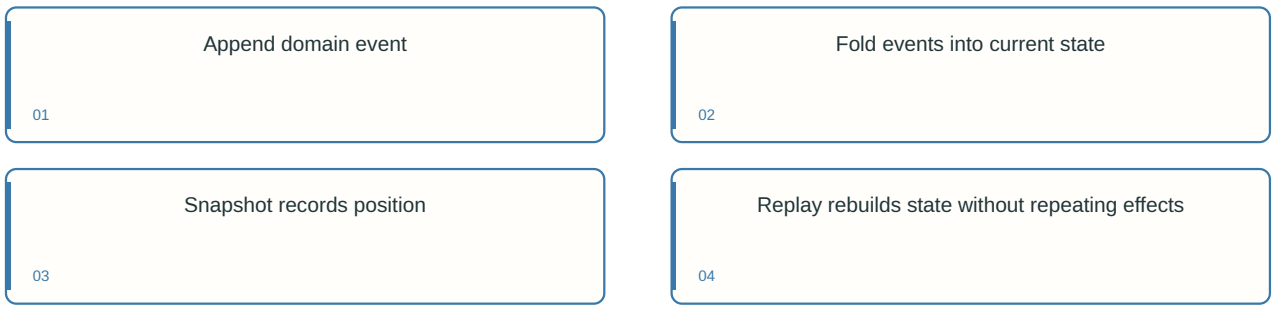


FIG. 16.2.4 **Event sourcing · the example**

Event sourcing makes interpretable accepted events authoritative, with snapshot, correction, privacy, and replay obligations.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · How could one broker transaction be exactly-once while two emails are sent?

The broker may atomically couple consumed progress and produced records while email lies outside that transaction. A consumer can send, lose its completion reply, and send again on recovery. Use the provider contract, stable identity where supported, and reconciliation; do not extend the broker guarantee to an unrelated effect.

Remember: Retrying, deduplicating, and preserving events serve different recovery needs.

Try applying it: How should a repaired poison record be replayed without duplicating its established effects?

Unit 16 exercises → · Unit 16 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Saga distributed transactions pattern — Microsoft ↗

   Official engineering guidance. Local transactions and compensation; compensation is not always a perfect inverse.

   Access reviewed · 21 October 2026

2. Transactional Outbox — Chris Richardson ↗

   Pattern author’s explanation. Publication intent and duplicate delivery.

   Access reviewed · 21 October 2026

3. Apache Kafka 4.1: Design ↗

   Official versioned documentation. Partitioning and scoped transactional/processing behavior; external effects are separate.

   Access reviewed · 21 October 2026

Read models, the outbox & sagas

Prerequisites: 16.2 · Retries, consumers & event history 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

An order commits but still needs publication and later compensation. Connect read models, outbox delivery, and sagas to make the obligations between local owners explicit.

16.3.1 · CQRS

An order command model checks legal transitions. A customer-history view arranges the same accepted information for convenient reading. CQRS separates these responsibilities or models when their requirements differ.

A projection may update the read model later, so define what a user sees immediately after a command. It may need a current-owner read, a progress indicator, or an accepted delay. CQRS can use one database and process. Event sourcing and separate services are additional choices. The extra models and synchronisation are worthwhile when they simplify actual workloads, not merely because reads and writes have different names.

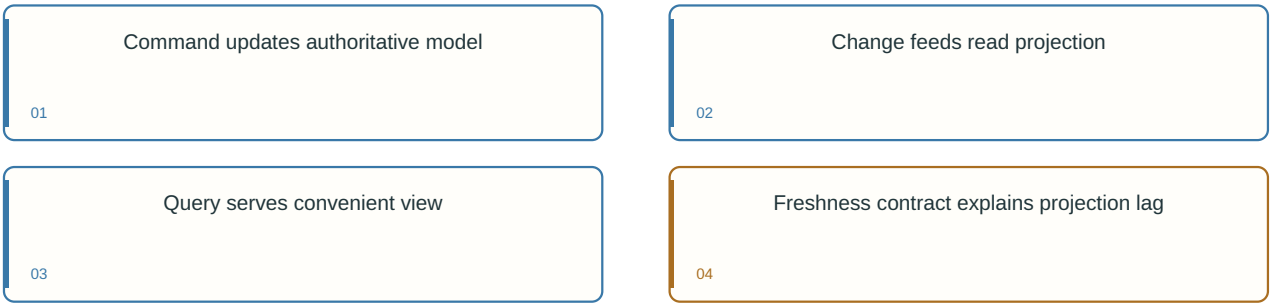


FIG. 16.3.1 **CQRS · the example**

CQRS separates command and query models where their different needs justify projection and freshness work.

Author-created illustration. Numbers are illustrative unless stated otherwise.

16.3.2 · Transactional outbox

An order commits, then the process dies before publishing OrderPlaced. A transactional outbox closes this local gap by writing the order change and publication intent in the same database transaction.

A relay later publishes the intent and records its progress. If it crashes after publication but before recording progress, publication can repeat. Use stable event identities and consumer duplicate handling. Monitor old pending intents and retain an appropriate recovery history. The pattern coordinates local state with the obligation to publish; it does not make every subscriber part of the original transaction or guarantee one external effect everywhere.

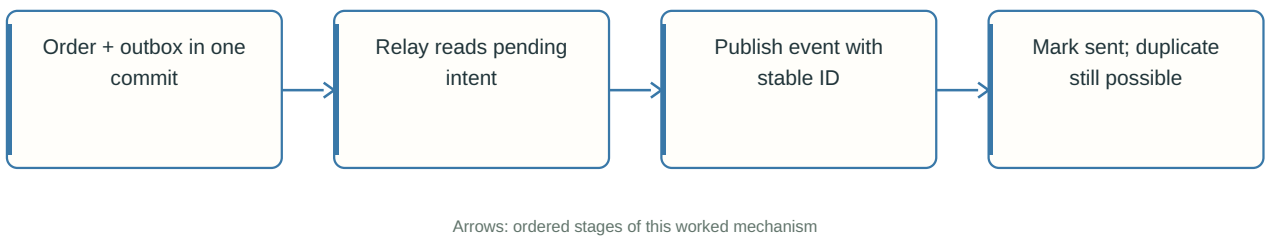


FIG. 16.3.2 **The outbox closes a local publication gap**

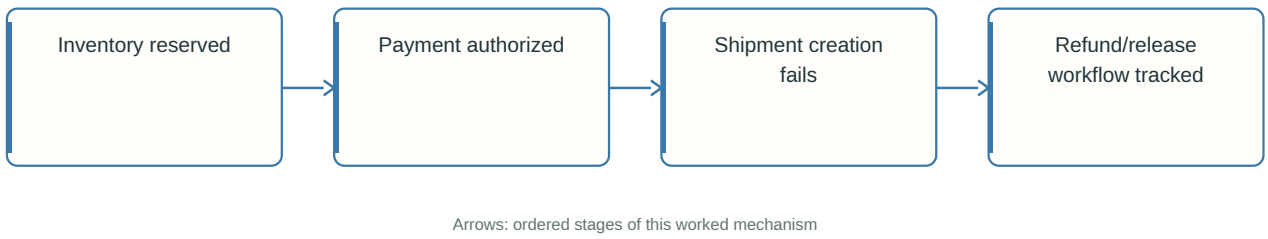
The relay can crash after publish but before marking. A consumer must handle the repeated event; the database transaction does not include the entire network.

Author-created illustration. Numbers are illustrative unless stated otherwise.

16.3.3 · Sagas and compensating transactions

Checkout reserves stock, authorises payment, and arranges shipment across separate owners. A saga records these local steps and the actions needed if the overall workflow cannot continue.

Compensation might release a reservation or request a refund. It may itself fail and need retries, and it is not always a perfect inverse: a refund can occur after costs or user-visible effects. Persist both forward and compensating progress. Distinguish uncertain, recoverable intermediate states from final outcomes. When replies are lost, reconcile with the relevant authority before assuming a step did not happen or starting it again.



Arrows: ordered stages of this worked mechanism

FIG. 16.3.3 **A failed shipment starts tracked compensation**

Each step is a local outcome. Compensation can fail or remain uncertain and must have durable progress and reconciliation.

Author-created illustration. Numbers are illustrative unless stated otherwise.

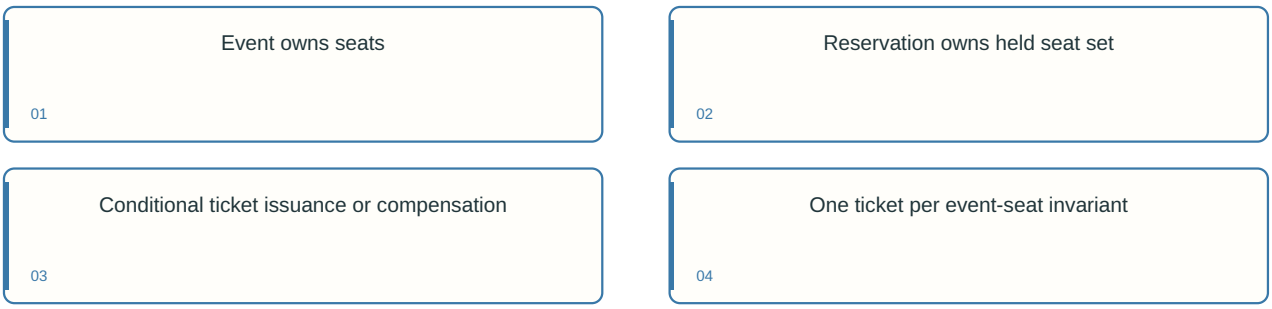


FIG. 16.3.4 **State and ownership in the related design**

Seat, hold, payment, and ticket records must preserve their defined ownership relationships.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · How could one broker transaction be exactly-once while two emails are sent?

The broker may atomically couple consumed progress and produced records while email lies outside that transaction. A consumer can send, lose its completion reply, and send again on recovery. Use the provider contract, stable identity where supported, and reconciliation; do not extend the broker guarantee to an unrelated effect.

Remember: Local atomic progress closes specific gaps while external steps retain their own failure and recovery rules.

Try applying it: Why can an outbox relay still publish one event twice?

Unit 16 exercises → · Unit 16 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Saga distributed transactions pattern — Microsoft ↗

   Official engineering guidance. Local transactions and compensation; compensation is not always a perfect inverse.

   `Access reviewed · 21 October 2026`

2. Transactional Outbox — Chris Richardson ↗

   Pattern author's explanation. Publication intent and duplicate delivery.

   `Access reviewed · 21 October 2026`

3. Apache Kafka 4.1: Design ↗

   Official versioned documentation. Partitioning and scoped transactional/processing behavior; external effects are separate.

   `Access reviewed · 21 October 2026`

Coordinate durable workflows

Prerequisites: 16.3 · Read models, the outbox & sagas 6 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A durable job outlives its request, scheduler, and first worker. Give each occurrence and attempt an identity, limit replay, and reconcile outcomes that remain uncertain.

16.4.1 · Choreography vs orchestration

With choreography, participants react to each other's events. With orchestration, an explicit coordinator records workflow progress and chooses the next action. Both can implement the same business sequence with different ownership and visibility.

Choreography can avoid one central controller but make end-to-end state harder to see. Orchestration makes retries and compensation explicit while adding a coordinator and evolving workflow definition. Neither removes participant failure. Choose according to the complexity, audit needs, and teams changing the process. Carry a shared correlation identity through either design so an operator can follow one business operation across its steps.

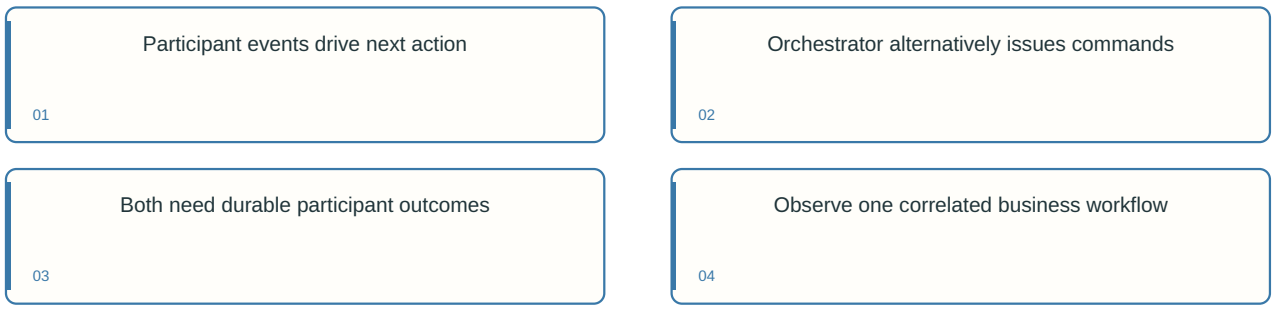


FIG. 16.4.1 **Choreography vs orchestration · the example**

Choreography distributes event reactions; orchestration gives workflow direction and progress an explicit owner.

Author-created illustration. Numbers are illustrative unless stated otherwise.

16.4.2 · Workflow engines

A workflow pauses for human approval, then resumes after a process restart. A workflow engine can persist progress, timers, and retry decisions under its supported programming model.

Activities still call outside systems. If an activity's effect succeeds but its completion record is lost, it may run again. Understand the engine's replay, determinism, versioning, activity identity, and cancellation rules. Use stable effect identities and reconciliation where necessary. The engine makes durable coordination easier; the application must still define correct transitions and safe external actions. Persistent orchestration is a useful mechanism with a scope, rather than a universal exactly-once guarantee.



FIG. 16.4.2 **Workflow engines · the example**

Workflow engines persist coordination under a programming model while external activities still need safe effect contracts.

Author-created illustration. Numbers are illustrative unless stated otherwise.

16.4.3 · Durable job execution

A durable job has an identity, input reference, progress state, attempt history, and an owner that can be replaced. A worker completes only if its current attempt or fencing epoch is still valid.

Store large inputs separately and verify their version before processing. An old worker must not overwrite a newer attempt's output. Bound retries and expose terminal or review states when work cannot finish automatically. A process-local background task has none of these recovery properties merely because the request returned 202. The response and durable acceptance contract must agree about who will continue the work after a crash.

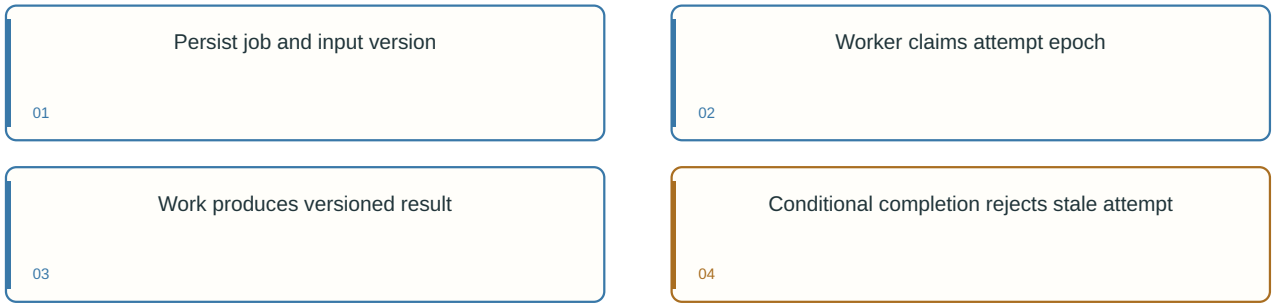


FIG. 16.4.3 **Durable job execution · the example**

Durable jobs retain identity, input, state, and attempt authority beyond the initiating process.

Author-created illustration. Numbers are illustrative unless stated otherwise.

16.4.4 · Scheduling, leases, and worker coordination

A schedule says a report is due at noon. The scheduler needs a stable occurrence identity, such as (schedule_id,occurrence), and an atomic way to claim or create that run.

Define missed periods, clock changes, overlap, and failover. A lease can expire while its worker still runs, so protect effects with an appropriate idempotency or fencing protocol. A single scheduler process does not by itself make runs recoverable or unique across restarts. Separate deciding that an occurrence is due from safely executing its external effect. Both stages need durable identity and recovery rules.

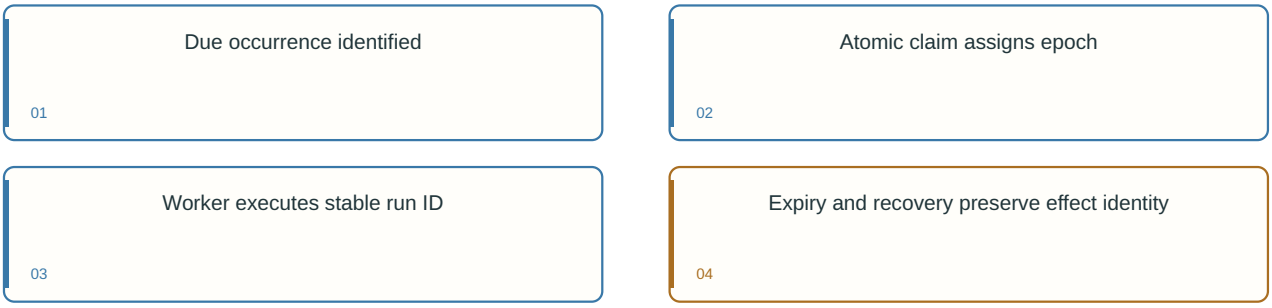


FIG. 16.4.4 **Scheduling, leases, and worker coordination · the example**

Scheduling separates stable due occurrences from attempts, with atomic creation, time, overlap, and stale-worker rules.

Author-created illustration. Numbers are illustrative unless stated otherwise.

16.4.5 · Backpressure and replay

A consumer falls behind and later replays retained input. Replaying as fast as possible can overwhelm the same live dependencies currently serving users. Backpressure and replay need coordinated capacity limits.

Give catch-up work its own rate budget, preserve source identities, and suppress unintended repeated side effects. Retention must cover the required recovery interval. Monitor oldest age and per-key blockage as well as queue length and throughput. A processor can be busy while one important operation remains stuck. Useful progress means business work reaching its defined outcome, rather than merely reading many historical messages.

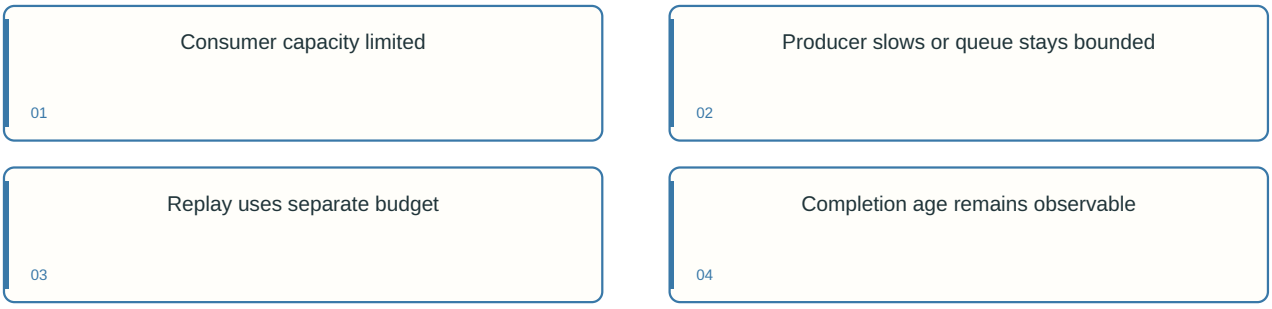


FIG. 16.4.5 **Backpressure and replay · the example**

Backpressure and replay need capacity budgets, retained identity, and useful completion indicators.

Author-created illustration. Numbers are illustrative unless stated otherwise.

16.4.6 · Reconciliation after partial failure

A local payment remains pending while the provider shows it charged. Reconciliation compares these intended and observed states and repairs the mismatch using the stable operation identity.

Query and update safely instead of blindly submitting another charge. The reconciler must also be authorised, idempotent, and observable. Run it periodically and during recovery, and track unresolved discrepancies by age and owner. It is part of the normal protocol for partial failures. It supplements correct persistence and validation on the main path rather than excusing missing records that make later investigation impossible.

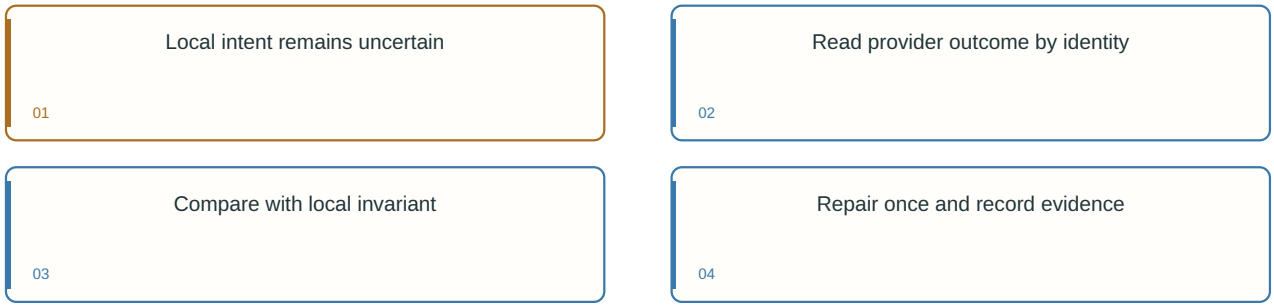


FIG. 16.4.6 **Reconciliation after partial failure · the example**

Reconciliation compares intended and observed outcomes and repairs partial failure through safe identified operations.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Try the mechanism in code

PYTHON · DUPLICATE DELIVERY, ONE LOCAL EFFECT

```
from mechanisms import open_demo_store, consume_local_event
db = open_demo_store()
assert consume_local_event(db, "E1", "orders")
assert not consume_local_event(db, "E1", "orders")
assert db.execute("SELECT value FROM counters").fetchone()[0] == 1
db.close()
```

The first transaction records E1 and increments the counter. The second finds E1 already processed and leaves the counter at one.

Event recognition and the local effect share a transaction. Replacing the counter update with an external email or payment call requires another protocol; it changes the boundary this example protects.

Download the executable mechanism collection ↓ · Follow the connected complete design →

▼ Think further · How could one broker transaction be exactly-once while two emails are sent?

The broker may atomically couple consumed progress and produced records while email lies outside that transaction. A consumer can send, lose its completion reply, and send again on recovery. Use the provider contract, stable identity where supported, and reconciliation; do not extend the broker guarantee to an unrelated effect.

Remember: Persistent ownership and safe effect identity let accepted work recover without inventing new intentions.

Try applying it: What should recovery do when a provider shows success but the local workflow remains pending?

Unit 16 exercises → · Unit 16 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Saga distributed transactions pattern — Microsoft ↗

   Official engineering guidance. Local transactions and compensation; compensation is not always a perfect inverse.

   `Access reviewed · 21 October 2026`

2. Transactional Outbox — Chris Richardson ↗

   Pattern author's explanation. Publication intent and duplicate delivery.

   `Access reviewed · 21 October 2026`

3. Apache Kafka 4.1: Design ↗

   Official versioned documentation. Partitioning and scoped transactional/processing behavior; external effects are separate.

   `Access reviewed · 21 October 2026`

PART 4 / UNIT 17

Reliability, Fault Tolerance, and Disaster Recovery

Several replicas share a hidden dependency and fail together. Review the serving and data goals, then calculate what independent capacity really remains after a failure.

The running example

A downstream payment provider becomes unavailable during peak traffic. Demonstrate the failure cascade and progressively introduce resilience mechanisms.

Unit 17 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

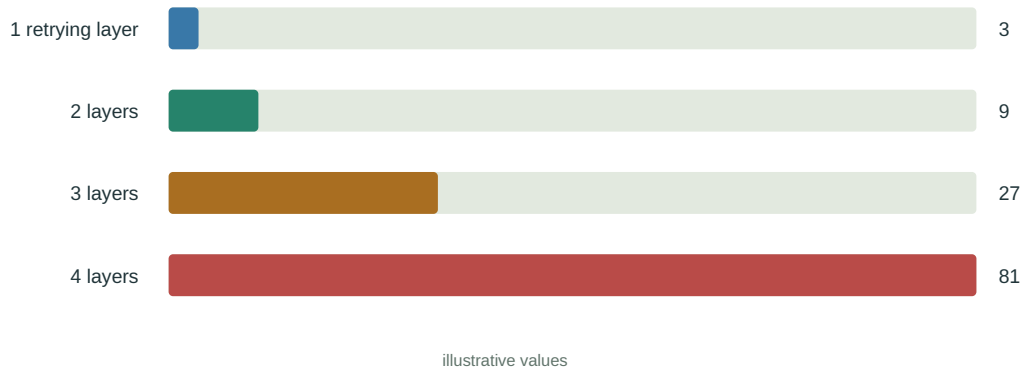


FIG. 17.0.1 **Retry amplification timeline**

Three attempts at every independent layer can amplify an intent geometrically; jitter alone does not reduce the attempt budget.

Author-created illustration. Numbers are illustrative unless stated otherwise.

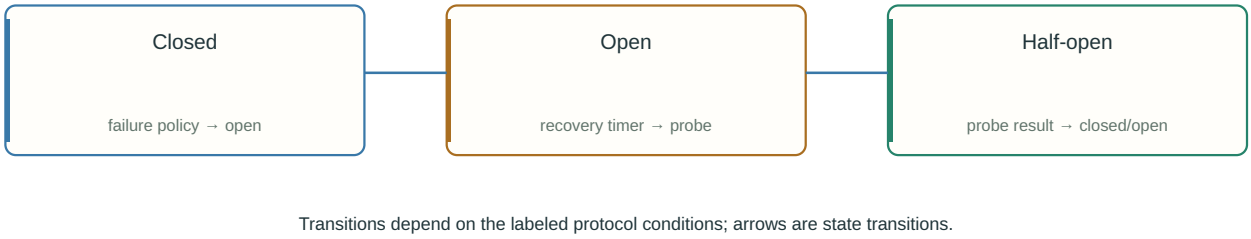


FIG. 17.0.2 **A circuit breaker bounds futile calls**

Closed permits calls, an observed failure policy opens the breaker, and bounded half-open probes test recovery. The breaker does not fabricate a successful business result.

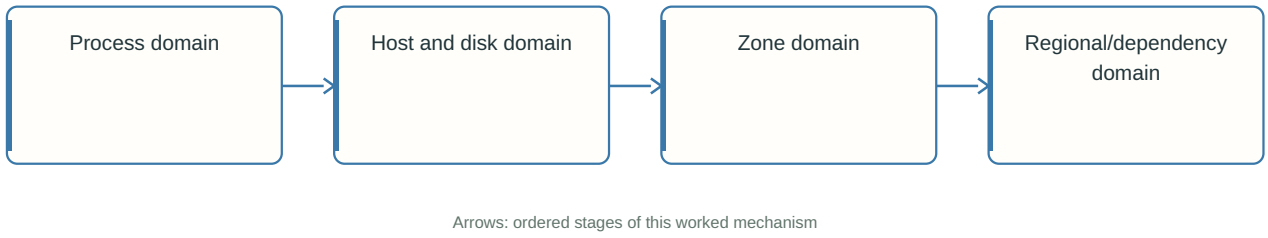
Author-created illustration. Numbers are illustrative unless stated otherwise.

Pool	Criticality	Failure isolation
Payments	Critical	Reserved bounded capacity
Receipts	Optional	Own queue and workers
Shared limits	DB, memory, network	Still require aggregate budgets

FIG. 17.0.3 **bulkhead isolation**

Separate pools limit some failure propagation, but shared resources remain correlated.

Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 17.0.4 **failure-domain map**

Each larger failure domain can remove several smaller replicas at once; independent placement is deliberate.

Author-created illustration. Numbers are illustrative unless stated otherwise.



FIG. 17.0.5 **Recovery time and recoverable data are different**

Last recoverable data at 11:55, incident at 12:00, service restored at 12:20: five minutes of data exposure and twenty minutes of recovery in this example.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Failure domains & redundancy

Prerequisites: Requirements, Estimation, and Engineering Trade-offs, Scalability, Load Balancing, and Data Distribution, Replication, Leader Election, and Consensus, Message Queues, Events, and Distributed Workflows

4 original figures

Blue: normal flow · Green: healthy · Amber: caution · Red: failure · Purple: control / shared infrastructure · Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Several replicas share a hidden dependency and fail together. Review the serving and data goals, then calculate what independent capacity really remains after a failure.

17.1.1 · Failure domains and fault isolation

Two workers use the same disk and credential source. Their processes are separate, but one disk failure or credential change can stop both. A failure domain groups parts exposed to a common cause.

Map process, host, zone, dependency, and administrative correlations. Fault isolation limits how one failure spreads, for example by separating critical resource pools. The separation costs capacity and operating work, so choose it according to the harm and recovery objective. Test the shared dependencies as well as individual processes. Counting replicas without their common causes can greatly overstate the protection the design provides.

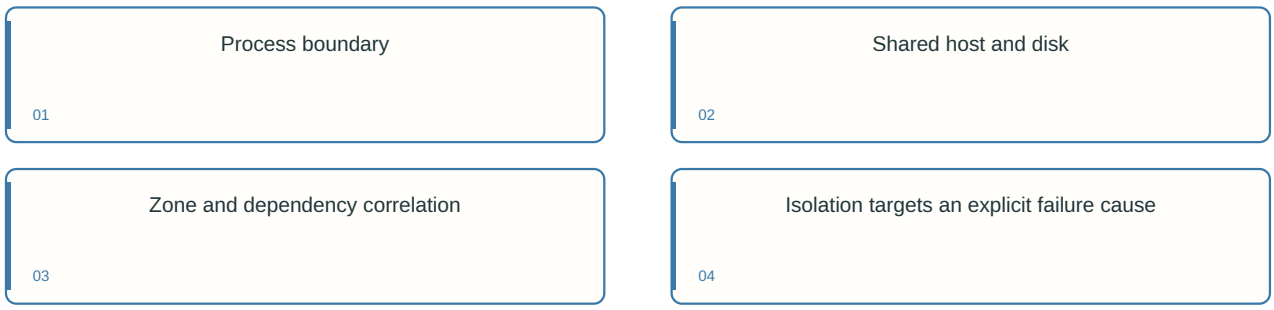


FIG. 17.1.1 **Failure domains and fault isolation · the example**

Failure domains identify shared causes; isolation limits their spread through resources and dependencies.

Author-created illustration. Numbers are illustrative unless stated otherwise.

17.1.2 · Availability vs durability

A database refuses requests during recovery but keeps all committed rows. This is a period of unavailability without demonstrated committed-data loss. A web application can show the opposite problem: it keeps replying successfully but drops writes stored only in memory.

Define availability and durability independently. Specify useful eligible responses, data-loss tolerance, and the failures being covered. Timeouts, traffic failover, backups, and replication influence different parts of that contract. A general promise of reliability needs these details before it can be measured. Choose checks that establish serving behaviour and retained state rather than treating either one as evidence of both.

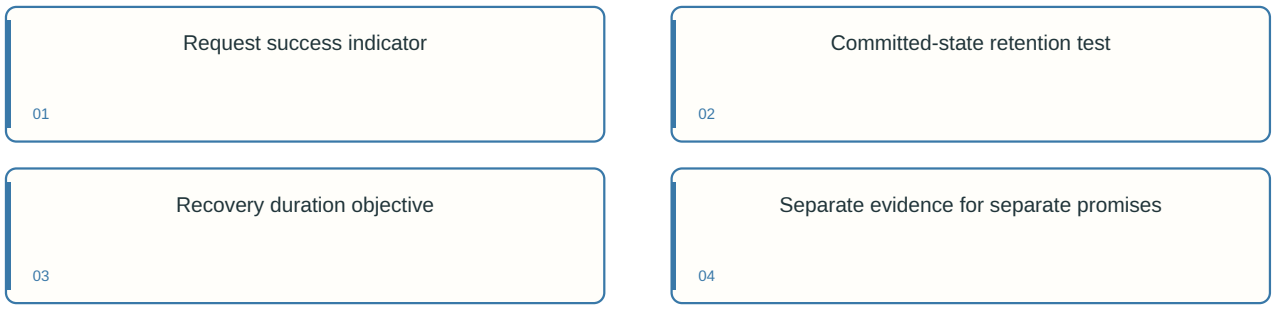


FIG. 17.1.2 **Availability vs durability · the example**

Availability concerns useful serving; durability concerns committed data surviving the specified failure model.

Author-created illustration. Numbers are illustrative unless stated otherwise.

17.1.3 · Redundancy

Two instances each operate at 70% of their individual capacity. If one fails, the remaining instance cannot absorb both workloads at the same rate. Redundancy needs enough surviving capacity or an explicit reduced-service policy.

It also needs independent failure placement, usable state, detection, and traffic movement. Shared bad configuration can disable every replica at once. Test takeover and restoration under the actual load instead of treating the second box as automatic protection. Spare resources are useful when the system can find them, direct work to them, and preserve the required state during the transition.

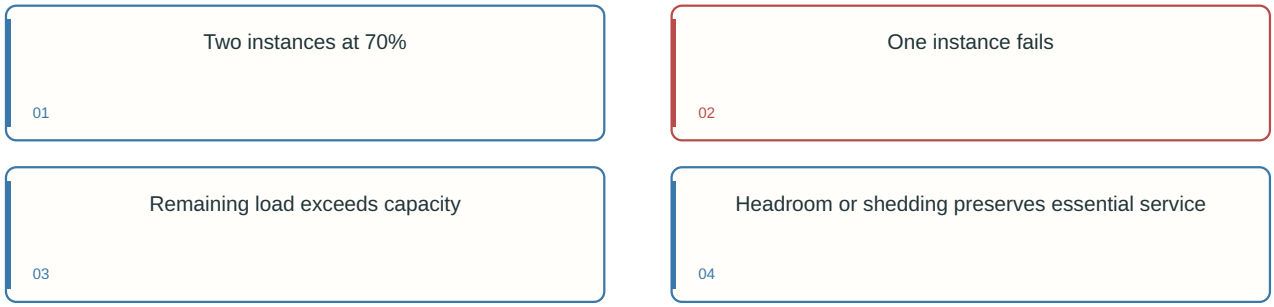


FIG. 17.1.3 **Redundancy · the example**

Redundancy requires independent failure exposure, valid state, usable takeover, and enough surviving capacity.

Author-created illustration. Numbers are illustrative unless stated otherwise.

17.1.4 · Active-active vs active-passive

In active-active operation, several locations or owners serve the selected function. Active-passive keeps a standby ready for takeover. The choice can differ by layer: application instances may all serve while one database leader owns writes.

A passive standby can have drift or warmup problems that appear only during takeover. Multiple active owners need routing and suitable coordination or conflict rules. For either arrangement, define detection, promotion, fencing, surviving capacity, and the return after repair. A standby that has never reconstructed usable state is an untested assumption. The operating model is a complete transition procedure, not just a traffic label.

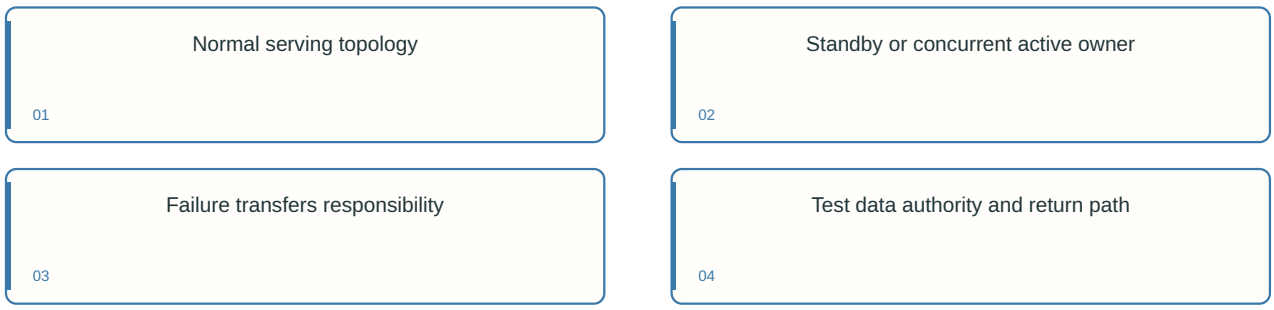


FIG. 17.1.4 **Active-active vs active-passive · the example**

Active-active and active-passive assign serving roles for a particular function, each needing tested transitions.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · How can a disaster drill restore infrastructure yet fail the business recovery requirement?

Machines can restart while the recovered data is too old, encrypted backups are unreadable, active authority conflicts, or accepted jobs are missing. Check the complete business journey and its RTO/RPO, including keys, state, identity, validation, and useful traffic return. A drill proves only the scenarios it actually exercises.

Remember: Redundancy requires independence, valid state, and sufficient surviving capacity.

Try applying it: Can two instances each at 70% load absorb the loss of either one?

Unit 17 exercises → · Unit 17 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Handling Overload — Google SRE ↗

   Engineering book. Admission, capacity, and overload response.

   Access reviewed · 21 October 2026

2. Disaster recovery planning guide — Google Cloud ↗

   Official guidance · accessed 21 Oct 2026. Recovery planning, distinct from a measured achievement of an objective.

   Access reviewed · 21 October 2026

3. Reliability Pillar — AWS Well-Architected ↗

   Official guidance · 2024 revision at access. Reliability review lenses and operations; provider-neutral teaching uses the concepts.

   Access reviewed · 21 October 2026

Retries, circuit breakers & isolation

Prerequisites: 17.1 · Failure domains & redundancy 6 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A slow provider consumes every connection and triggers retries. Use deadlines, budgets, breakers, and resource isolation to limit the pressure rather than merely repeat the same work faster.

17.2.1 · Timeouts

A client permits two seconds for an operation. It would be wasteful to spend three seconds waiting for a database connection before starting the query. An overall deadline bounds the useful lifetime of the request.

Stage limits can cover connection setup, pool acquisition, and execution, but should fit the remaining overall time. Propagate that remaining time downstream and clean up work when possible. A timeout means the caller stopped waiting; an external write may already have occurred. Record which stage timed out so diagnosis distinguishes waiting from computation. Independent full timeouts at every layer can keep useless work alive far beyond the caller's limit.

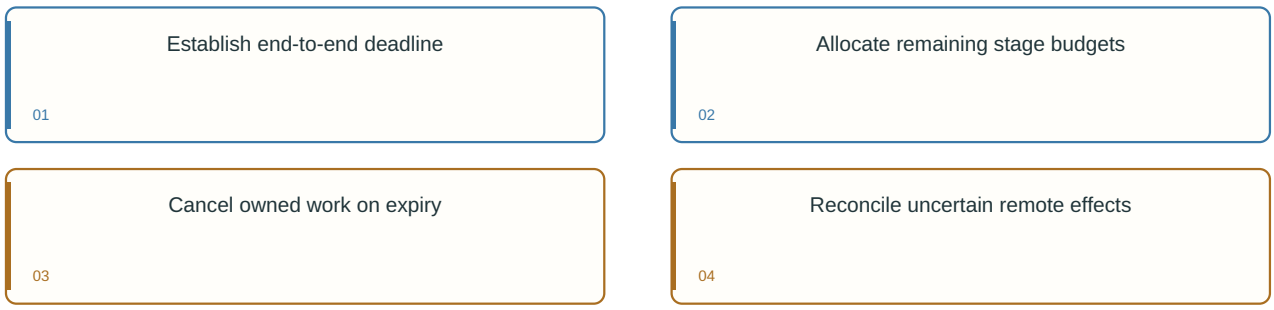


FIG. 17.2.1 **Timeouts · the example**

Timeouts bound waiting; overall deadlines include acquisition and stage waits without proving remote nonexecution.

Author-created illustration. Numbers are illustrative unless stated otherwise.

17.2.2 · Retries, exponential backoff, jitter

A dropped connection before a safe lookup may justify another attempt. An invalid payment amount will not improve if the same request is repeated. Classify the error before retrying.

Use bounded attempts, increasing backoff, jitter, and a deadline. Repeated effects need a stable operation identity. Respect relevant retry guidance while keeping the whole business operation within its intended lifetime. Retry ownership should be clear across layers; several independently aggressive policies can multiply load. Retries are additional work spent to recover from a particular failure, so include that work in capacity and recovery planning.

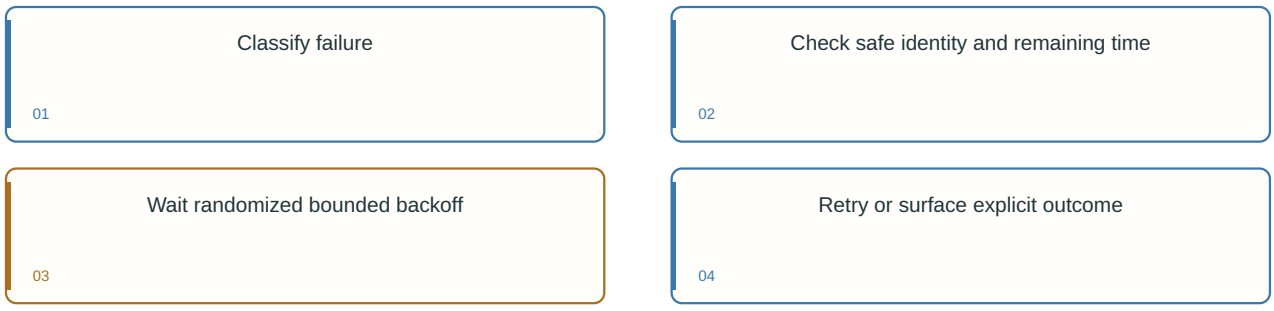


FIG. 17.2.2 **Retries, exponential backoff, jitter · the example**

Retries spend bounded capacity and time, with backoff, jitter, classification, and safe effect identity.

Author-created illustration. Numbers are illustrative unless stated otherwise.

17.2.3 · Retry storms

Four nested layers each allow three attempts. One logical request can then create up to $3^4 = 81$ downstream attempts. During a dependency outage, this amplification can turn partial capacity into a complete collapse.

Jitter spreads attempts in time but does not reduce their total by itself. Use an overall attempt budget, admission controls, and suppression of redundant work. Measure original intents separately from attempts and useful completions. An outage-related rise in requests may come from retries rather than new users. Protect capacity for successful work and recovery instead of letting repeated failures consume it all.

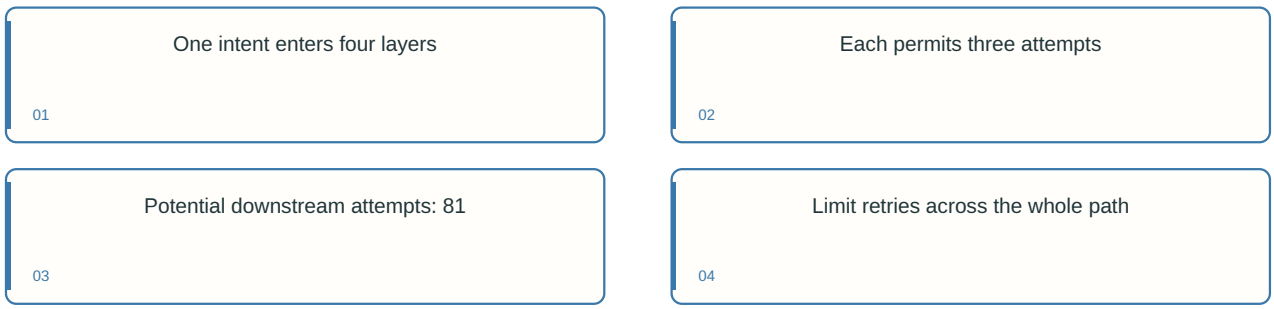


FIG. 17.2.3 **Retry storms · the example**

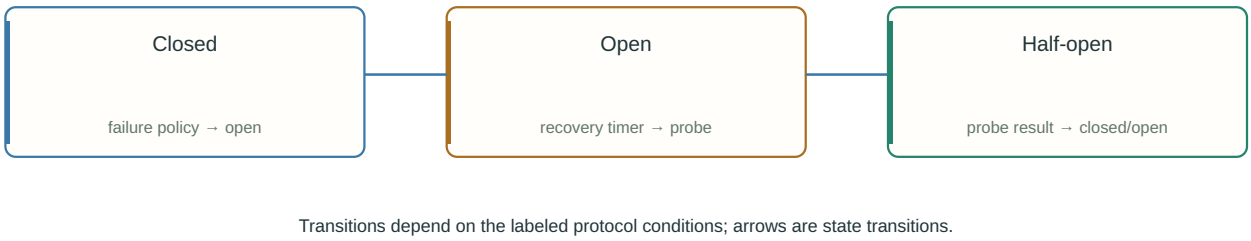
Retry storms amplify failure traffic; timing randomisation alone does not bound total attempts.

Author-created illustration. Numbers are illustrative unless stated otherwise.

17.2.4 · Circuit breakers

A dependency repeatedly fails. A circuit breaker can temporarily reject new calls rather than making every caller wait for another likely failure. Common states are closed, open, and half-open, where limited probes test recovery.

Define which failures count and how probes are bounded. Avoid all instances probing simultaneously after a shared timer. The breaker does not repair the dependency or decide a safe business fallback. It must not turn a failed authorisation call into a fabricated payment approval. Its useful role is controlling further resource expenditure while the application returns or performs a deliberately chosen degraded behaviour.



Transitions depend on the labeled protocol conditions; arrows are state transitions.

FIG. 17.2.4 **A circuit breaker bounds futile calls**

Closed permits calls, an observed failure policy opens the breaker, and bounded half-open probes test recovery. The breaker does not fabricate a successful business result.

Author-created illustration. Numbers are illustrative unless stated otherwise.

17.2.5 · Bulkheads

Receipt email stalls and occupies every worker. Payment authorisation now cannot obtain resources even though its provider is healthy. Bulkheads reserve or separate resources for work with different failure priorities.

Dedicated pools can keep optional delivery from consuming checkout's whole allowance. Check shared limits as well: two pools may still exhaust one database or host memory. Too much partitioning can leave spare resources unused, so choose boundaries by harm and workload. Observe saturation and rejection in each pool. Isolation is effective when it covers the resource through which failure would spread, not merely when two thread-pool names exist.

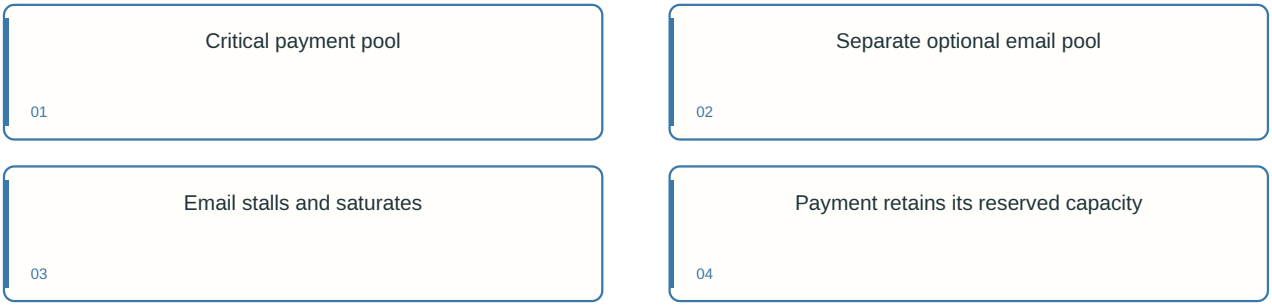


FIG. 17.2.5 **Bulkheads · the example**

Bulkheads partition relevant resources so one workload cannot consume another's entire budget.

Author-created illustration. Numbers are illustrative unless stated otherwise.

17.2.6 · Rate limiting

A token bucket accumulates tokens at a configured rate up to a capacity. Each admitted request spends tokens. It permits a bounded burst while controlling continuing admission. A shaper may instead delay work to change its pace.

For several tenants, combine fairness with protection of total service capacity. Define the identity, cost unit, burst, and failure policy. A limiter whose remote state is unavailable must reject, allow, or use a previously allocated local budget according to an explicit contract. These choices change both availability and abuse exposure. Test that path rather than assuming the limiter always has a healthy backing store.

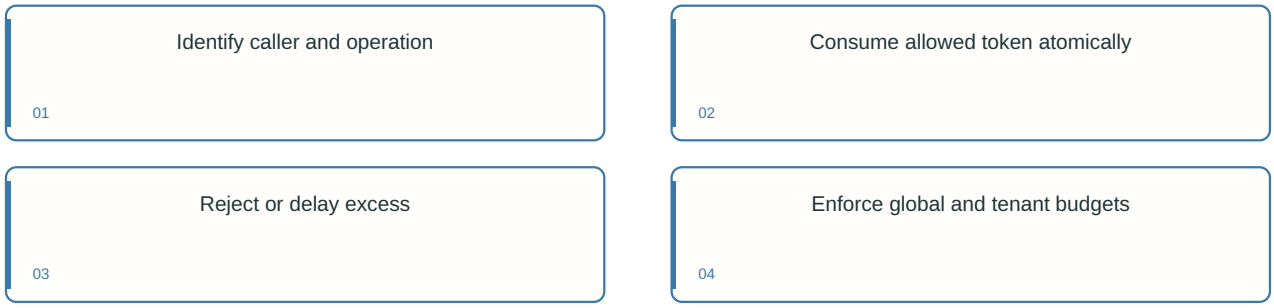


FIG. 17.2.6 **Rate limiting · the example**

Rate limiting applies a scoped pace and burst rule with an explicit state-failure policy.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · How can a disaster drill restore infrastructure yet fail the business recovery requirement?

Machines can restart while the recovered data is too old, encrypted backups are unreadable, active authority conflicts, or accepted jobs are missing. Check the complete business journey and its RTO/RPO, including keys, state, identity, validation, and useful traffic return. A drill proves only the scenarios it actually exercises.

Remember: Bound both useful work and recovery attempts at the resource boundaries that can saturate.

Try applying it: Why can jitter reduce synchronisation while leaving total retry work excessive?

Unit 17 exercises → · Unit 17 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Handling Overload — Google SRE ↗

   Engineering book. Admission, capacity, and overload response.

   Access reviewed · 21 October 2026

2. Disaster recovery planning guide — Google Cloud ↗

   Official guidance · accessed 21 Oct 2026. Recovery planning, distinct from a measured achievement of an objective.

   Access reviewed · 21 October 2026

3. Reliability Pillar — AWS Well-Architected ↗

   Official guidance · 2024 revision at access. Reliability review lenses and operations; provider-neutral teaching uses the concepts.

   Access reviewed · 21 October 2026

Shed load and degrade deliberately

Prerequisites: 17.2 · Retries, circuit breakers & isolation 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Some features can continue with reduced data; others must stop when their authority is unavailable. Define shedding, degradation, fallbacks, and probes from those meanings.

17.3.1 · Load shedding

Under overload, the service may stop generating recommendations while preserving order status and reconciliation. Load shedding rejects selected work early enough to keep resources available for higher-priority outcomes.

Choose priorities from product and correctness requirements. Rejecting after expensive computation saves little. Return a clear overload result and bound subsequent retries. Shedding can improve useful completions while demand exceeds capacity; it cannot sustain an impossible workload forever. More capacity, less work, or a changed target may still be needed. Measure useful accepted outcomes so a drop in raw throughput is not mistaken for failure when deliberate shedding protects essential work.

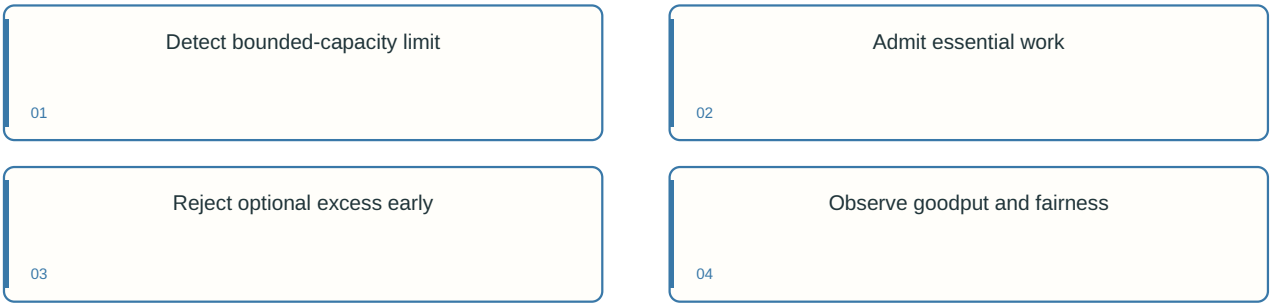


FIG. 17.3.1 **Load shedding · the example**

Load shedding rejects selected work early to preserve useful higher-priority outcomes under overload.

Author-created illustration. Numbers are illustrative unless stated otherwise.

17.3.2 · Graceful degradation

A product page can remain useful without recommendations. A sale cannot safely proceed merely because an old cached count says stock exists. Graceful degradation preserves an agreed subset of useful behaviour while respecting the rules that still matter.

Specify which data may be stale, which actions pause, and what the user sees. Test rarely exercised degraded paths before relying on them during an incident. Catching every error and returning a default does not establish safety. The alternative outcome needs the same clarity about permissions, freshness, and meaning as the ordinary one.

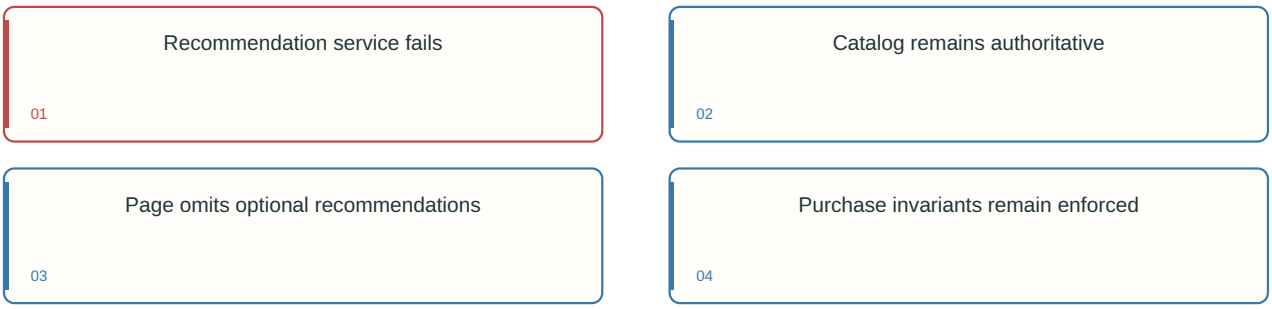


FIG. 17.3.2 **Graceful degradation · the example**

Graceful degradation preserves a permitted subset of behaviour with visible freshness and action limits.

Author-created illustration. Numbers are illustrative unless stated otherwise.

17.3.3 · Fallbacks

A cached help page may substitute for an unavailable content service. A default account balance cannot substitute for an unavailable ledger. A fallback is another behaviour whose meaning must be acceptable for this operation.

Check its freshness, duration, access scope, and extra load. A fallback can itself overload another dependency or hide a long incident. Switching providers also changes operation identity and recovery unless their semantics are deliberately aligned. Make degraded operation visible where needed. Evaluate the substituted result as a business outcome rather than treating any returned response as successful recovery.

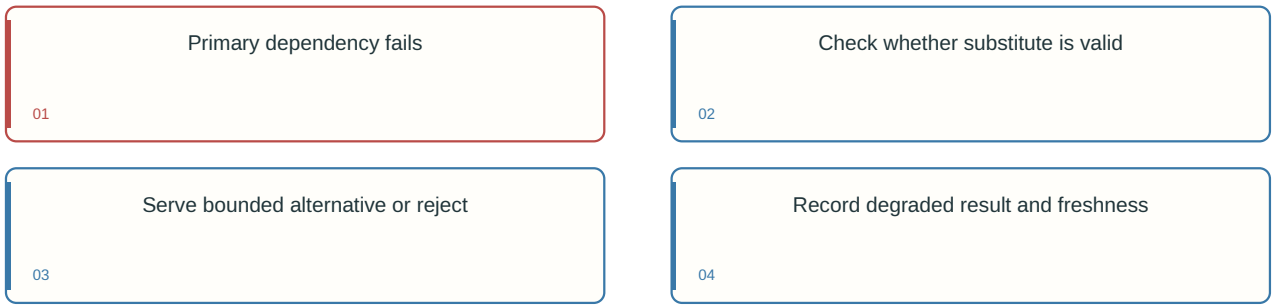


FIG. 17.3.3 **Fallbacks · the example**

Fallbacks substitute defined business outcomes, with their own access, duration, capacity, and recovery risks.

Author-created illustration. Numbers are illustrative unless stated otherwise.

17.3.4 · Health checking

A process keeps restarting because its liveness check calls an unavailable shared provider. Restarting cannot repair that provider and may make recovery harder. Keep probes focused on the decision they control.

Liveness asks whether the process needs restarting. Readiness can stop new traffic while ongoing work finishes. Startup probes or allowances can cover warmup. Bound checks and avoid coupling every process restart to independent external failures. A probe response is evidence about that probe, not proof that every critical operation meets its objective. Monitor real useful operations alongside these local signals.

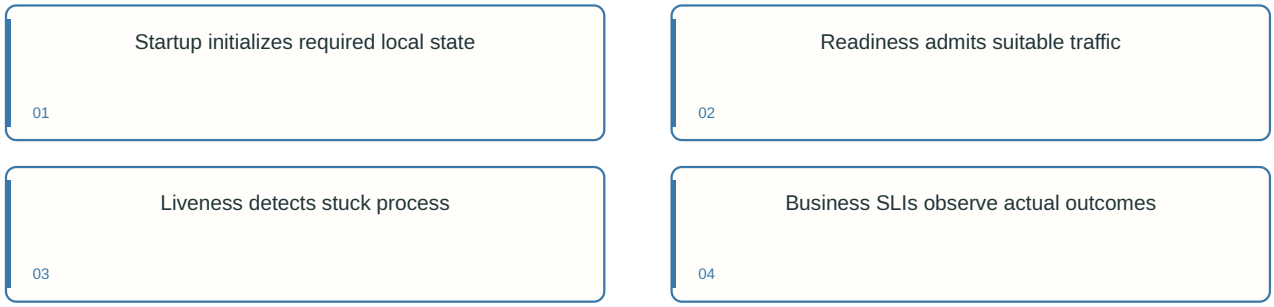


FIG. 17.3.4 **Health checking · the example**

Health probes support bounded local decisions rather than establishing the complete business service guarantee.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · How can a disaster drill restore infrastructure yet fail the business recovery requirement?

Machines can restart while the recovered data is too old, encrypted backups are unreadable, active authority conflicts, or accepted jobs are missing. Check the complete business journey and its RTO/RPO, including keys, state, identity, validation, and useful traffic return. A drill proves only the scenarios it actually exercises.

Remember: A degraded result must remain a permitted business outcome.

Try applying it: Why is a cached help page a different fallback from a guessed account balance?

Unit 17 exercises → · Unit 17 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Handling Overload — Google SRE ↗

   Engineering book. Admission, capacity, and overload response.

   Access reviewed · 21 October 2026

2. Disaster recovery planning guide — Google Cloud ↗

   Official guidance · accessed 21 Oct 2026. Recovery planning, distinct from a measured achievement of an objective.

   Access reviewed · 21 October 2026

3. Reliability Pillar — AWS Well-Architected ↗

   Official guidance · 2024 revision at access. Reliability review lenses and operations; provider-neutral teaching uses the concepts.

   Access reviewed · 21 October 2026

Recover, test & learn from failure

Prerequisites: 17.3 · Shed load and degrade deliberately 8 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

The outage ends when useful service and required state recover. Set recovery objectives, test restores and authority changes, and learn from correlated failure rather than only process restarts.

17.4.1 · RTO and RPO

An incident begins at noon. Service returns at 12:20, so the interruption lasted twenty minutes. If recovery restores data only through 11:55, there is five minutes of data-loss exposure relative to the incident point.

RTO is the recovery time objective. RPO is the recovery point objective, expressing acceptable loss exposure in time. Set them for the business function, then check the whole procedure: detection, decision, restoration, validation, and traffic return. Timing only a file copy misses much of the outage. A faster restore may also use older data, so evaluate the two objectives separately.



FIG. 17.4.1 **Recovery time and recoverable data are different**

Last recoverable data at 11:55, incident at 12:00, service restored at 12:20: five minutes of data exposure and twenty minutes of recovery in this example.

Author-created illustration. Numbers are illustrative unless stated otherwise.

17.4.2 · Backups and restore testing

A backup survives the lost host, but its only decryption key was on that host. The bytes exist, yet the application cannot recover them. A useful backup needs the surrounding restoration resources too.

Document credentials, keys, schema versions, retained logs, and the usable recovery environment. Restore actual backups and check business invariants, rather than relying only on job status or checksums. Keep copies separate from the failure they protect against. Live replication can repeat corruption or deletion, while historical backups can support a different recovery point. Specify which role each copy serves and test the corresponding procedure.

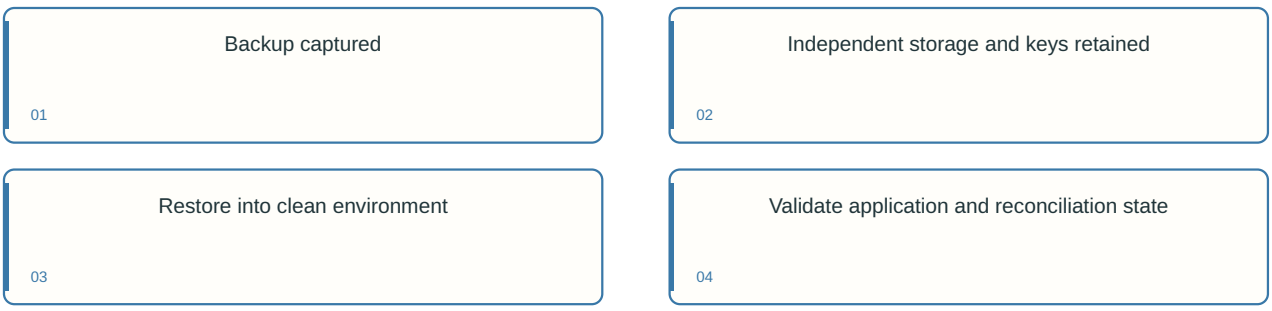


FIG. 17.4.2 **Backups and restore testing · the example**

Usable backups require independent storage, keys, compatible state, and tested application restoration.

Author-created illustration. Numbers are illustrative unless stated otherwise.

17.4.3 · Failover and failback

Failover moves service or write authority to a replacement. Failback returns it after repair. Both transitions need valid history, sufficient capacity, routing, and protection from conflicting owners.

An old primary may need rebuilding from the new history before rejoining. Promote according to the promised data-loss policy, and fence stale writers. Client connections, DNS caches, and retries can continue using old paths after a database promotion succeeds. Test the full user-visible sequence, including return after repair. Recovery is incomplete if traffic looks restored but a later old owner can reintroduce incompatible writes.

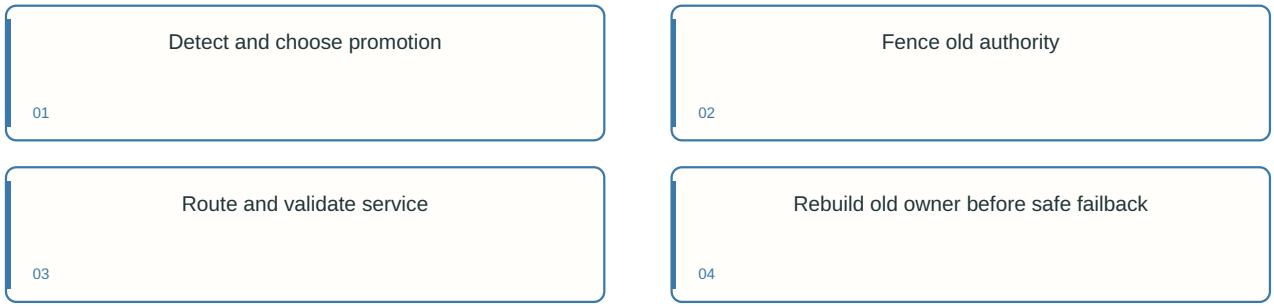


FIG. 17.4.3 **Failover and failback · the example**

Failover and failback transfer traffic or authority while preserving current history, fencing, and capacity.

Author-created illustration. Numbers are illustrative unless stated otherwise.

17.4.4 · Multi-zone and multi-region deployments

Copies in separate zones can survive some local incidents. Separate regions cover other failures but add propagation delay, transfer cost, data-placement concerns, and recovery work.

Check dependencies that remain shared. A global credential change or configuration fault can still affect all regions. Plan capacity for surviving traffic and catch-up work, not just the normal split. Select placement according to named failure and recovery goals. Geographic distance is one part of independence; the authority protocol and operational dependencies determine whether the locations can actually provide the promised protection.

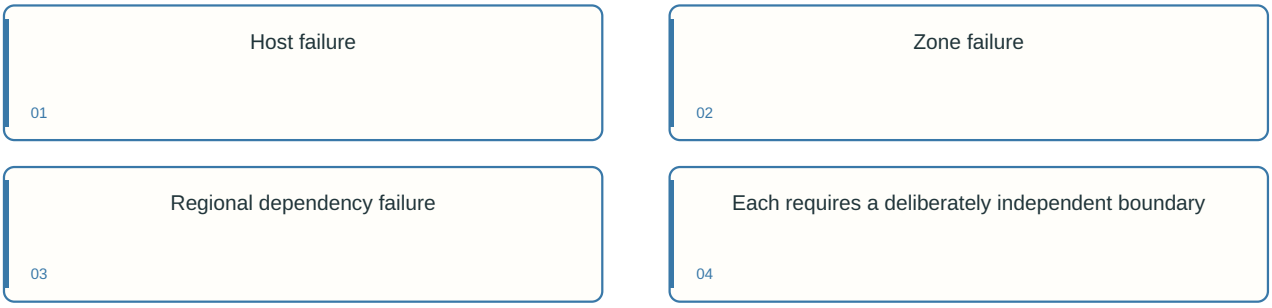


FIG. 17.4.4 **Multi-zone and multi-region deployments · the example**

Zone and region placement address different correlations while adding propagation, cost, and recovery dependencies.

Author-created illustration. Numbers are illustrative unless stated otherwise.

17.4.5 · Chaos engineering

Begin a fault exercise with a question: if a worker pauses past its lease, can its old completion overwrite a new result? Then choose a controlled pause and observe the specified outcome.

Chaos engineering tests such hypotheses under bounded faults. Define the affected scope, stop conditions, observations, recovery owner, and environment appropriate to the risk. Random process killing without a hypothesis can cause harm without clarifying a mechanism. Increase scope only when justified. A passing drill provides evidence for that scenario and configuration; it does not prove every future failure harmless.

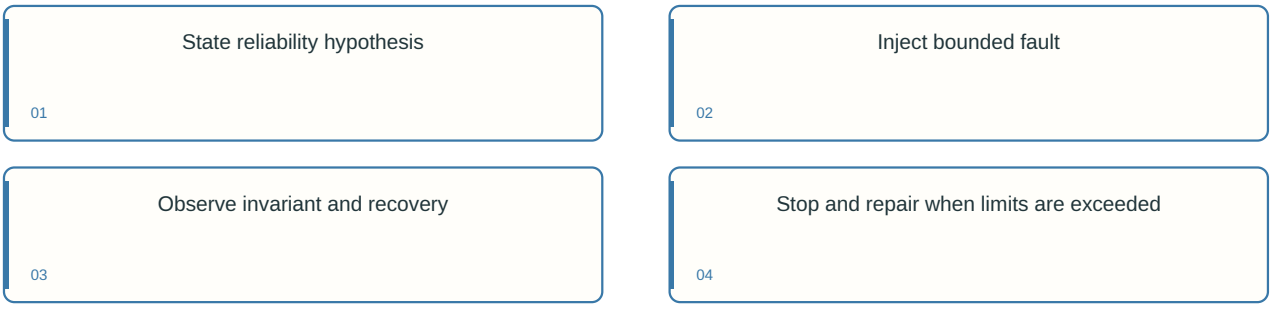


FIG. 17.4.5 **Chaos engineering · the example**

Chaos exercises test explicit hypotheses with bounded scope, observations, stop conditions, and recovery ownership.

Author-created illustration. Numbers are illustrative unless stated otherwise.

17.4.6 · SLO-based reliability decisions

Suppose optional calls consume most of a request's deadline and cause most failures. Removing them from the critical path may improve users' experience more directly than adding another region.

Use SLO and error-budget evidence to choose reliability work. Keep important low-volume operations visible: their failures can disappear inside a large denominator dominated by cheap successful reads. Segment indicators where the harm differs. More redundancy has both cost and complexity, so evaluate the mechanism it improves. An error budget is a decision aid, rather than permission to ignore a severe failure merely because it affects few events.

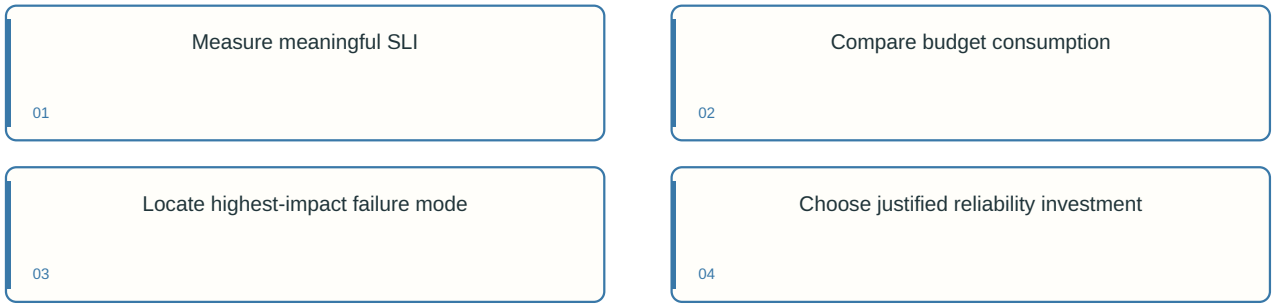


FIG. 17.4.6 **SLO-based reliability decisions · the example**

SLO evidence guides reliability investment without hiding critical low-volume harm inside a broad denominator.

Author-created illustration. Numbers are illustrative unless stated otherwise.

17.4.7 · Cascading failures

A slow provider holds connections. Waiting requests accumulate, callers retry, and the extra work overloads previously healthy components. This feedback creates a cascading failure.

Break the actual resource path with deadlines, bounded admission, retry budgets, isolation, and meaningful degradation. Keep capacity for recovery. A breaker that sends every failed request to one fallback can start another cascade. Trace both the original path and the recovery path under correlated failure. Testing each dependency in isolation can miss the interaction that makes a modest initial slowdown become a system-wide outage.

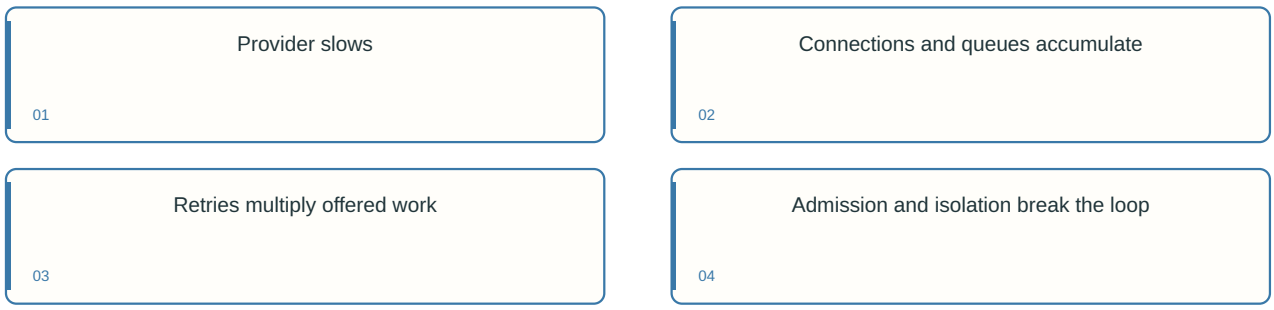


FIG. 17.4.7 **Cascading failures · the example**

Cascading failures spread through resource pressure and retries, including potentially overloaded fallback paths.

Author-created illustration. Numbers are illustrative unless stated otherwise.

17.4.8 · Incident analysis and prevention

After an incident, reconstruct what happened, which conditions allowed it, how it was detected, and what helped or delayed recovery. Stopping at “human error” leaves unanswered why one action could affect every replica.

A useful analysis connects corrective work to observed mechanisms. Scoped credentials can reduce exposure; tested restores can improve recovery; bounded retries can limit amplification. Give each action an owner and evidence of completion. Avoid adding alerts without a useful response. The aim is to reduce recurrence or harm by changing the system and its operating conditions, rather than attaching blame to the person closest to the trigger.

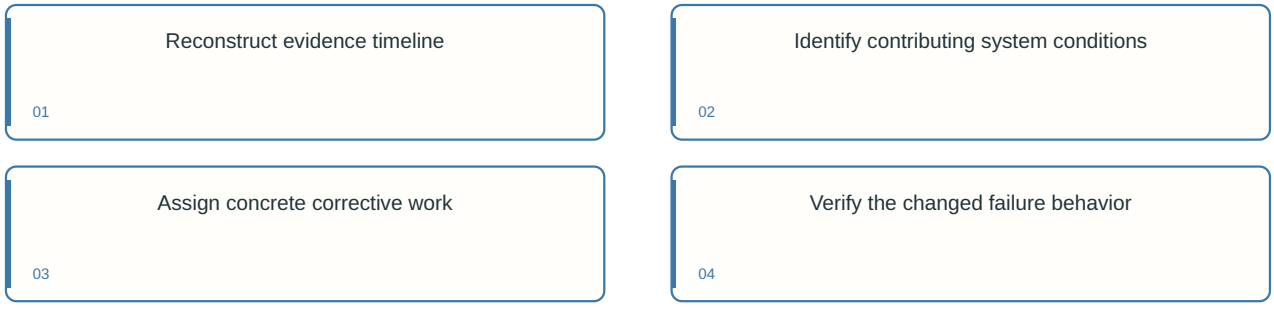


FIG. 17.4.8 **Incident analysis and prevention · the example**

Incident analysis links contributing mechanisms to verified corrective changes without substituting blame for explanation.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Try the mechanism in code

PYTHON · RETRY AMPLIFICATION

```
attempts_per_layer = 3
independent_retry_layers = 4
assert attempts_per_layer ** independent_retry_layers == 81
```

The arithmetic gives 81 permitted downstream attempts in the nested worst case. Actual execution can stop sooner, but failures may activate many layers at once.

Use an overall retry budget and deadline, protect useful work, and keep operation identity stable. Jitter spreads attempt timing without reducing the configured total by itself.

Download the executable mechanism collection ↓ · Follow the connected complete design →

▼ Think further · How can a disaster drill restore infrastructure yet fail the business recovery requirement?

Machines can restart while the recovered data is too old, encrypted backups are unreadable, active authority conflicts, or accepted jobs are missing. Check the complete business journey and its RTO/RPO, including keys, state, identity, validation, and useful traffic return. A drill proves only the scenarios it actually exercises.

Remember: Recovery evidence covers the tested journey and must account for unresolved business effects.

Try applying it: What could make a fast infrastructure recovery fail the original data-loss requirement?

Unit 17 exercises → · Unit 17 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Handling Overload — Google SRE ↗

   Engineering book. Admission, capacity, and overload response.

   Access reviewed · 21 October 2026

2. Disaster recovery planning guide — Google Cloud ↗

   Official guidance · accessed 21 Oct 2026. Recovery planning, distinct from a measured achievement of an objective.

   Access reviewed · 21 October 2026

3. Reliability Pillar — AWS Well-Architected ↗

   Official guidance · 2024 revision at access. Reliability review lenses and operations; provider-neutral teaching uses the concepts.

   Access reviewed · 21 October 2026

PART 5 / UNIT 18

Security Architecture and Privacy

A tenant administrator guesses another tenant's document ID. Follow identity, permissions, roles, and the trust boundaries the request crosses before deciding which controls are needed.

The running example

Design a multi-tenant SaaS application where an administrator must never gain access to another organization's data.

Unit 18 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

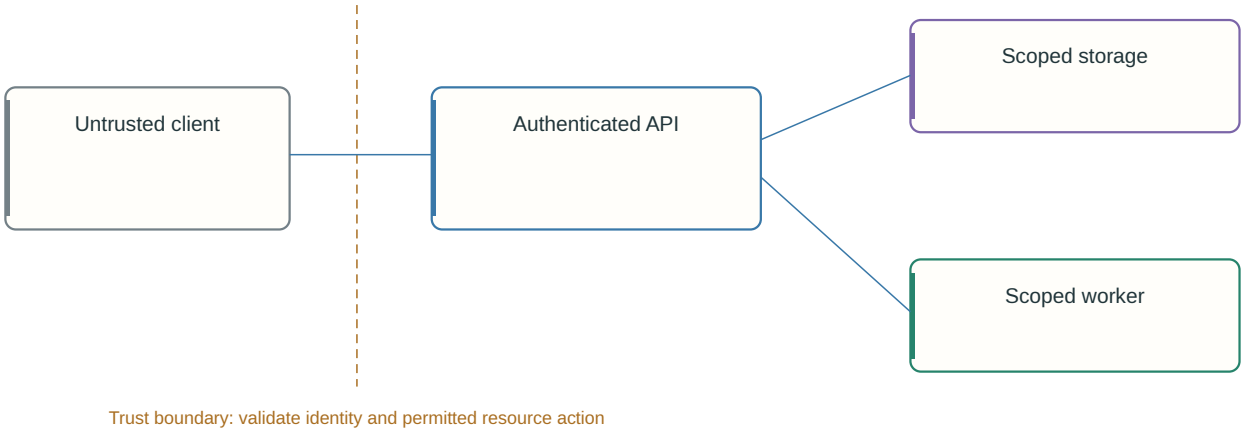


FIG. 18.0.1 **Tenant identity must survive every trust boundary**

An authenticated tenant context scopes storage queries, jobs, object capabilities, and search. Client-provided tenant labels are not independent authority.

Author-created illustration. Numbers are illustrative unless stated otherwise.

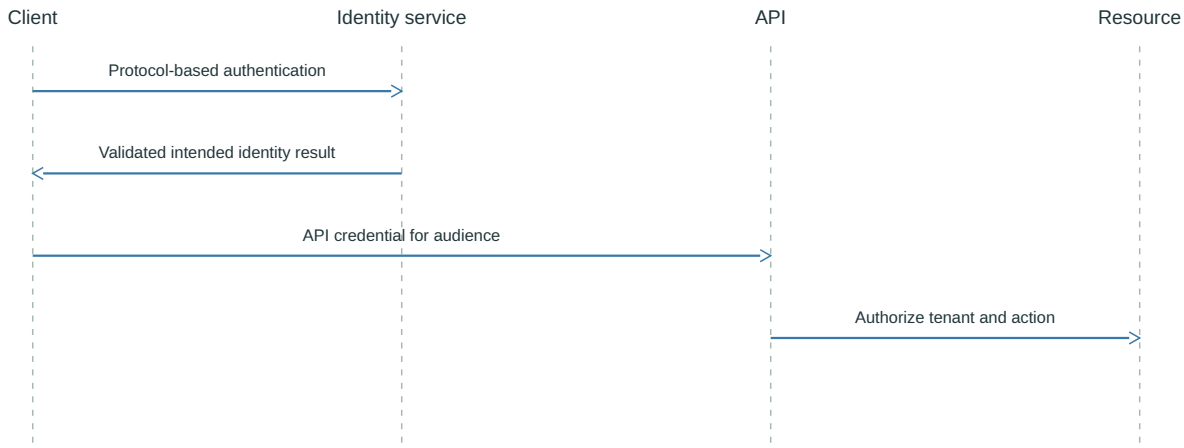


FIG. 18.0.2 **authentication sequence**

Authentication, token validation, and resource authorization are distinct protocol and enforcement decisions.

Author-created illustration. Numbers are illustrative unless stated otherwise.

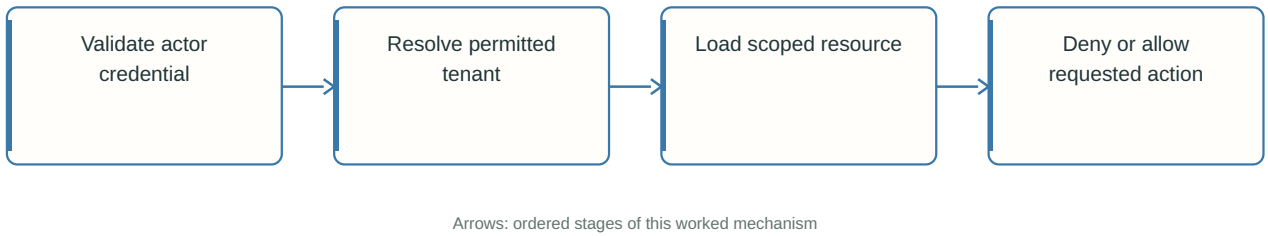


FIG. 18.0.3 **authorization decision flow**

A valid token never grants arbitrary access to a guessed resource identifier.

Author-created illustration. Numbers are illustrative unless stated otherwise.

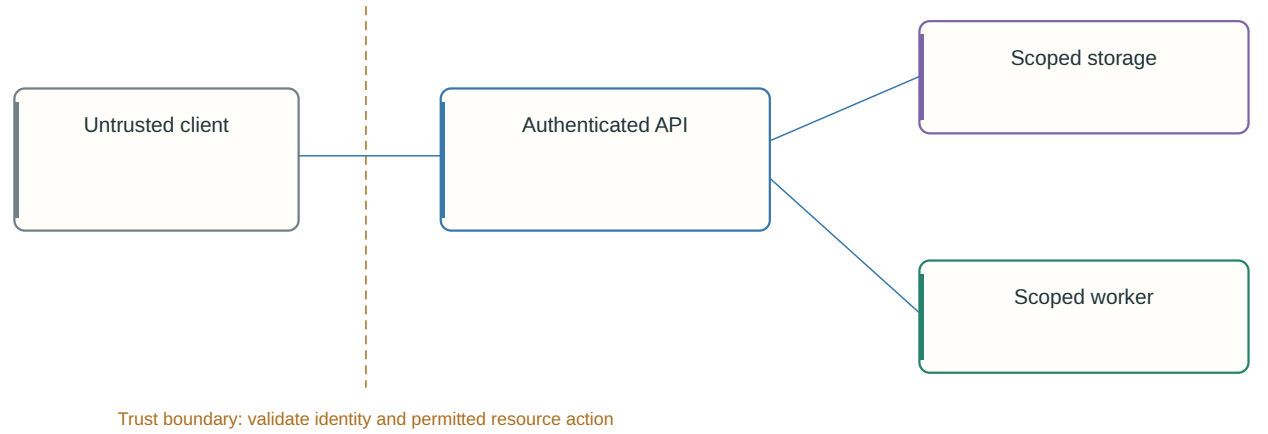


FIG. 18.0.4 **Tenant identity must survive every trust boundary**

An authenticated tenant context scopes storage queries, jobs, object capabilities, and search. Client-provided tenant labels are not independent authority.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Threats, identity & authorization

Prerequisites: Networking Foundations for System Design, Application Architecture and Domain-Driven Design, API Design and Communication Contracts, Data Models and Storage Technologies

7 original figures

Blue: normal flow · Green: healthy · Amber: caution · Red: failure · Purple: control / shared infrastructure · Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A tenant administrator guesses another tenant's document ID. Follow identity, permissions, roles, and the trust boundaries the request crosses before deciding which controls are needed.

18.1.1 · Threat modeling

Begin with a document service's valuable assets: private files, credentials, processing capacity, and trustworthy audit records. Identify users, workers, administrators, and possible attackers, then ask how each asset could be misused.

For example, could one tenant guess another's file identifier, forge a processing callback, or submit files that exhaust extraction workers? Threat modelling connects such paths to their impact, controls, and tests. Draw data flows and trust boundaries, but also record the actors' real privileges. Prioritise plausible harmful paths. A diagram alone is incomplete if it leaves unclear what must be protected and who may reach it.

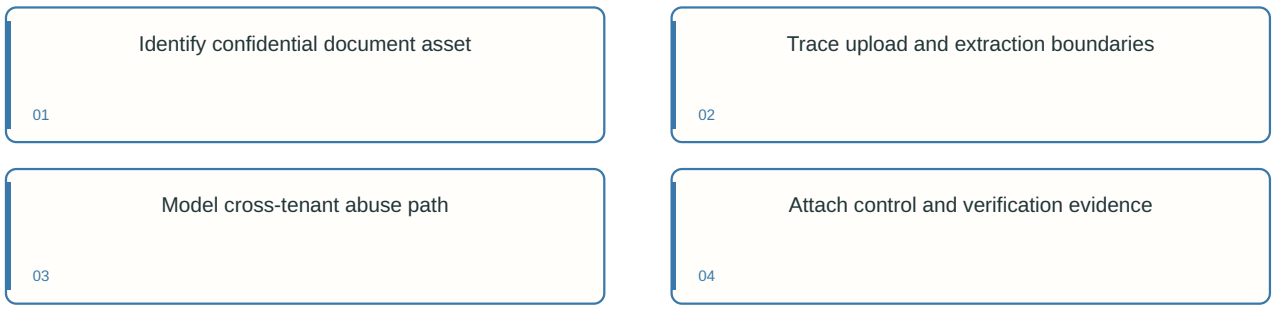


FIG. 18.1.1 **Threat modeling · the example**

Threat modelling connects assets, actors, abuse paths, and impact to controls and concrete tests.

Author-created illustration. Numbers are illustrative unless stated otherwise.

18.1.2 · Trust boundaries

A browser submits a document, a worker reads it from storage, and an operator changes production settings. Each crosses a boundary with different expectations about identity, data, and authority.

A trust boundary marks where those expectations change. Validate the relevant evidence at the boundary rather than assuming a request is legitimate because it came from an internal network. Forwarded identity headers need a configured trusted source. Stored uploads remain untrusted content and may require restricted execution and resource budgets when parsed. Saving bytes does not make their instructions or structure safe for a privileged worker.

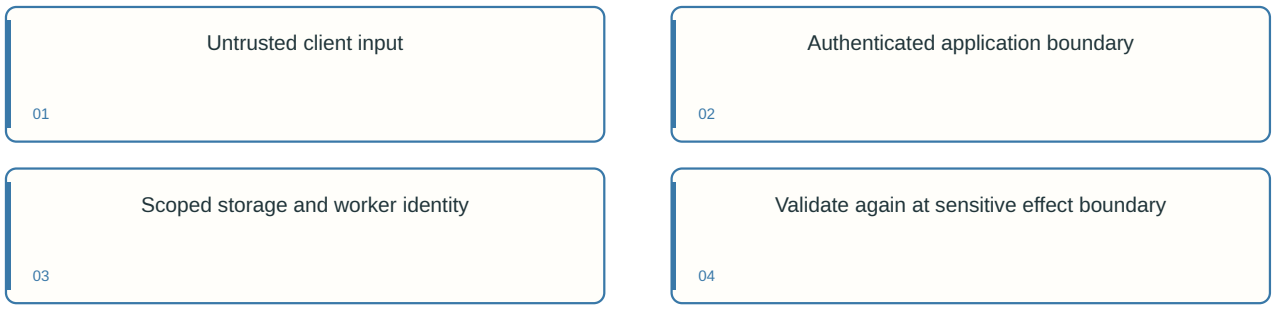


FIG. 18.1.2 **Trust boundaries · the example**

Trust boundaries mark changes in identity, authority, and confidence that require appropriate validation.

Author-created illustration. Numbers are illustrative unless stated otherwise.

18.1.3 · Authentication vs authorization

An administrator of tenant A may manage A's documents while having no permission to tenant B's files. Authentication establishes who the actor is. Authorisation checks what that actor may do to the particular resource in context.

Derive tenant membership from trusted identity evidence and enforce it on reads, edits, exports, and background work. Deny access unless the policy allows it. Difficult-to-guess identifiers can reduce enumeration but cannot replace ownership checks. Test permitted and forbidden paths with two tenants, including overlapping local identifiers. A correct login system does not establish that every later resource lookup is correctly authorised.

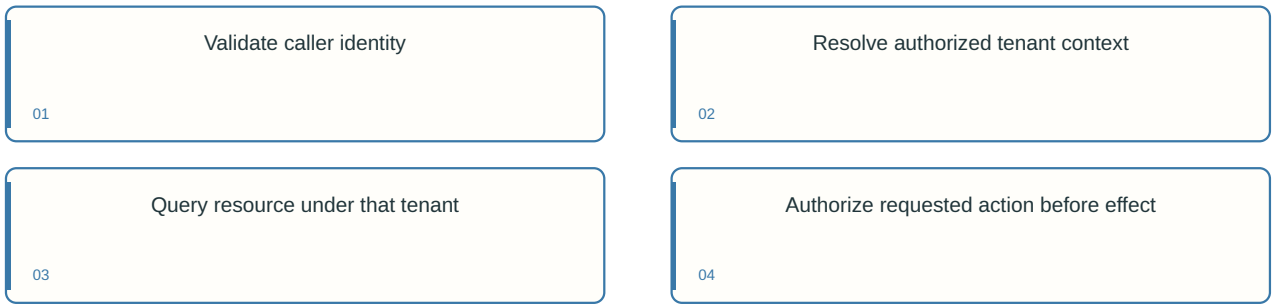


FIG. 18.1.3 **Authentication vs authorization · the example**

Authentication establishes identity; authorisation grants a particular action on a resource under policy.

Author-created illustration. Numbers are illustrative unless stated otherwise.

18.1.4 · Sessions, cookies, tokens

A browser can send a session credential in a cookie. The server may use it to find session state. Another design may verify claims in a token. Cookies describe a browser storage and transport mechanism; they do not alone define the authentication scheme.

Protect credentials using the appropriate secure transport, cookie attributes, validation, and CSRF controls. Decide how logout, expiry, and revocation work. A signed token may still be expired, intended for another audience, or no longer sufficient for a resource action. Choose a reviewed protocol and library, and validate the fields required by that protocol rather than treating a valid signature as the whole decision.

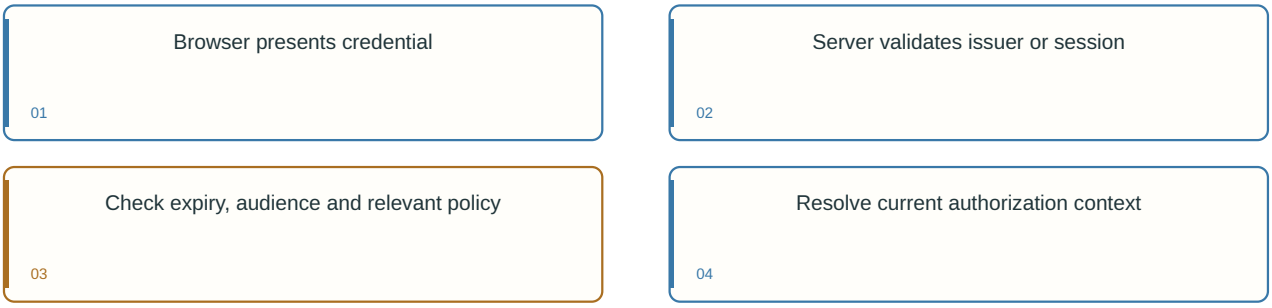


FIG. 18.1.4 **Sessions, cookies, tokens · the example**

Sessions, tokens, and cookies have different state and transport roles with explicit validation and revocation rules.

Author-created illustration. Numbers are illustrative unless stated otherwise.

18.1.5 · OAuth 2.0 and OpenID Connect

An application may ask another service for delegated access using OAuth 2.0. To establish a login identity, OpenID Connect adds an identity layer with its own assertions and validation rules.

Do not treat an arbitrary access token as proof of identity. Use the appropriate reviewed flow and maintained library, following current guidance such as authorisation code with PKCE where applicable. Validate the issuer, audience, token type, redirects, and required state or nonce evidence for that flow. These requirements differ by role and protocol. Keeping identity and delegated permission distinct prevents a convenient login shortcut from becoming permission to call the wrong API.

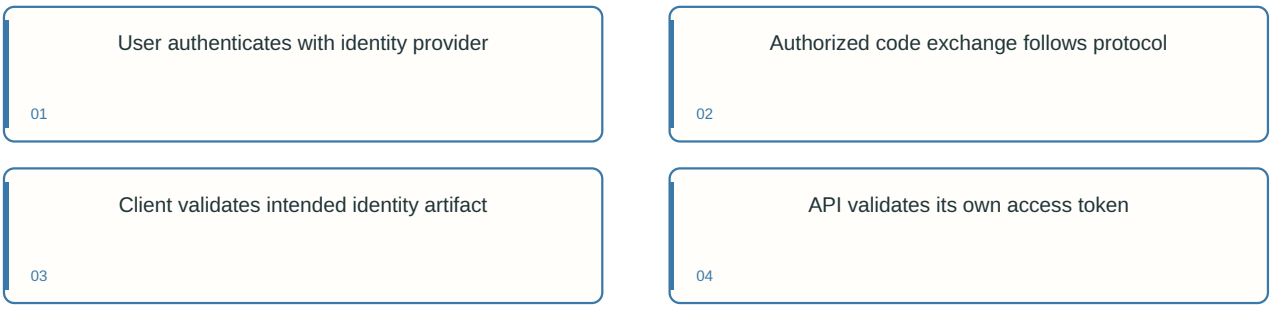


FIG. 18.1.5 **OAuth 2.0 and OpenID Connect · the example**

OAuth delegates access; OpenID Connect adds identity assertions with flow-specific validation.

Author-created illustration. Numbers are illustrative unless stated otherwise.

18.1.6 · Role-based and attribute-based access control

Role-based access control assigns permissions through roles. Attribute-based access control evaluates trusted facts such as tenant, owner, classification, and request context. A service can use both.

For example, “tenant administrator” must be tied to a tenant. Treating that role as global grants far too much. Roles can become broad, while detailed attribute rules can be harder to explain and test. Keep policy inputs trustworthy and check revocation and queued work. A central policy engine helps only when every required enforcement point calls it correctly. Evaluate the whole path rather than merely choosing a policy language.

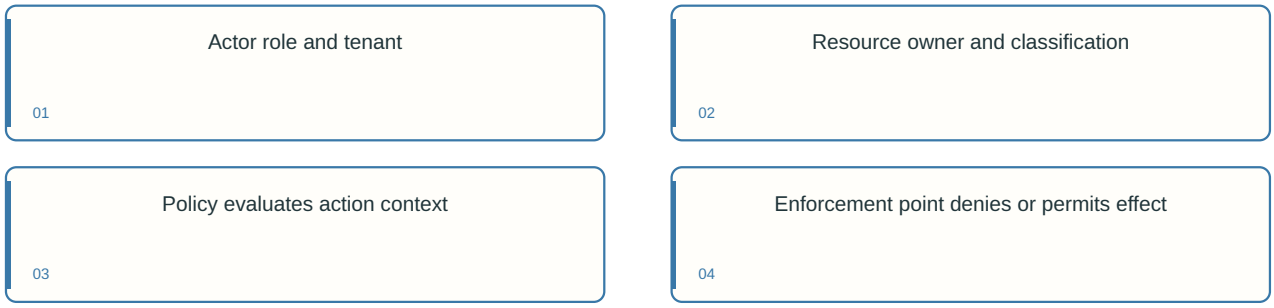


FIG. 18.1.6 **Role-based and attribute-based access control · the example**

RBAC assigns scoped role permissions; ABAC evaluates trusted attributes, with enforcement and testing required for either.

Author-created illustration. Numbers are illustrative unless stated otherwise.

18.1.7 · Least privilege

An extraction worker needs to read one input and write its output. It does not ordinarily need to list every tenant's files or alter deployment settings. Least privilege gives it the operations and scope required for that job.

Use separate runtime, deployment, and administrative identities where appropriate. Short-lived scoped credentials can reduce exposure, but need workable refresh and failure behaviour. Review effective inherited permissions as well as the intended role. Emergency access needs its own limits and audit. The useful question is what this identity can actually do now, not only what its description says it should do.

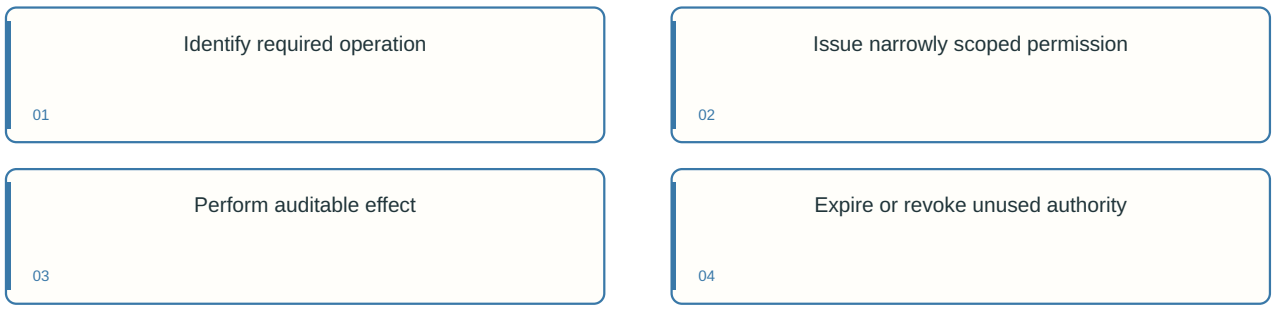


FIG. 18.1.7 **Least privilege · the example**

Least privilege limits effective identity capabilities to the work and resources actually needed.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · What should revocation mean for an issued object capability and a queued export?

Specify both boundaries. An already issued capability may remain valid until expiry unless the delivery system supports revocation. A queued export may need a current policy check before execution or publication. Keep users informed about the chosen lifetime and ensure workers cannot bypass it using an old submission snapshot.

Remember: Authenticated identity is an input to resource-specific permission, not permission by itself.

Try applying it: Which paths must reject a tenant A administrator requesting tenant B data?

Unit 18 exercises → · Unit 18 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. OWASP: Authorization ↗

   Official project guidance · accessed 21 Oct 2026. Least privilege, deny by default, and per-resource enforcement.

   Access reviewed · 21 October 2026

2. OWASP: Password Storage ↗

   Official project guidance · accessed 21 Oct 2026. Password-specific hashing and current deployment parameter guidance.

   Access reviewed · 21 October 2026

3. RFC 9700: OAuth 2.0 Security Best Current Practice, 2025 ↗

   Original protocol security guidance. OAuth threat and flow guidance; implementation should use maintained protocol libraries.

   Access reviewed · 21 October 2026

4. OpenID Connect Core 1.0, errata set 2 ↗

   Original specification · December 2023. Identity layer and validation requirements; distinct from API authorization.

   Access reviewed · 21 October 2026

Secrets, encryption & trust boundaries

Prerequisites: 18.1 · Threats, identity & authorization 7 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A stored file is encrypted, but the processing worker can read every file and every key. Examine credentials, cryptography, segmentation, and control placement against the actual threat.

18.2.1 · Defense in depth

Tenant-scoped queries, storage permissions, limited object capabilities, worker isolation, and audit records can provide complementary protection for a private document. Defense in depth makes a single control failure less likely to cause the full harm.

Check independence. If every layer trusts the same client-supplied tenant field without validation, repeating the check adds little. Also examine failure behaviour: a broken policy dependency should not silently disable access controls. Trace which control stops each misuse path and how its result is enforced. Several control names do not necessarily provide several effective barriers.

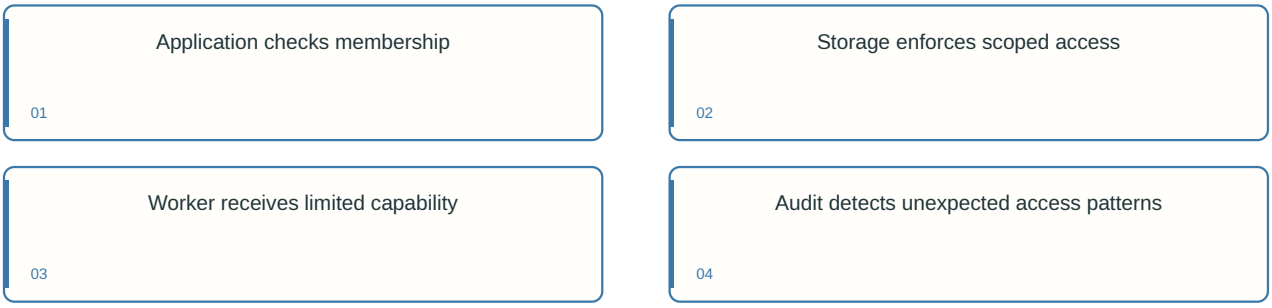


FIG. 18.2.1 **Defense in depth · the example**

Defense in depth combines complementary controls rather than repeating one unverified assumption at several layers.

Author-created illustration. Numbers are illustrative unless stated otherwise.

18.2.2 · Secrets management

A secret has a lifecycle: creation, distribution, use, rotation, and revocation. Assign an owner and limit who can read or replace it. Keep credentials out of source, logs, client bundles, and command history.

Managed identities or short-lived credentials may reduce stored secrets, while still needing bootstrap and outage plans. Rotation can require a period when both old and new versions work, followed by verified retirement. Encrypting a secret file offers little separation if every reader also has its key. The relevant protection comes from actual identity, access, key custody, and operating boundaries.

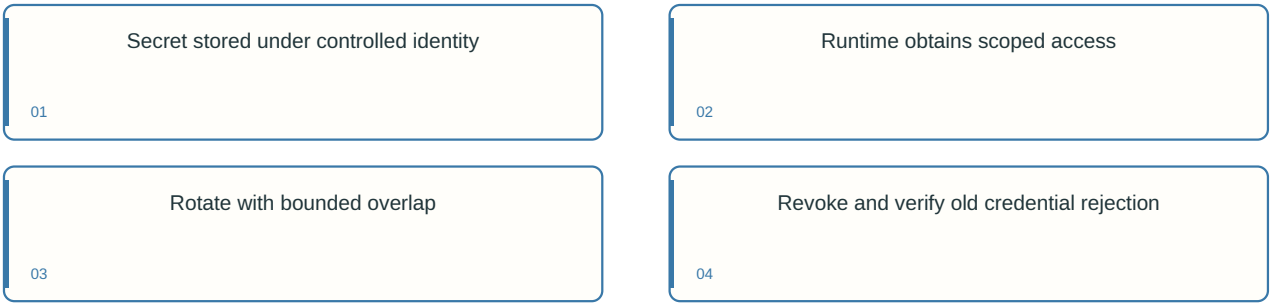


FIG. 18.2.2 **Secrets management · the example**

Secrets need controlled distribution, ownership, rotation, revocation, and protection from accidental exposure.

Author-created illustration. Numbers are illustrative unless stated otherwise.

18.2.3 · Encryption in transit and at rest

TLS protects traffic while it travels under its protocol. Encryption at rest protects stored bytes under particular key and access assumptions. Trace where each protection begins and ends.

A compromised application with legitimate decryption rights may still read the data it is allowed to access. Backups, logs, extracted text, and exports also need appropriate protection; encrypting the original storage does not automatically protect a CSV sent elsewhere. Choose controls for the identified threat and use reviewed cryptographic protocols and libraries. Encryption is one part of access and key design, rather than a replacement for it.

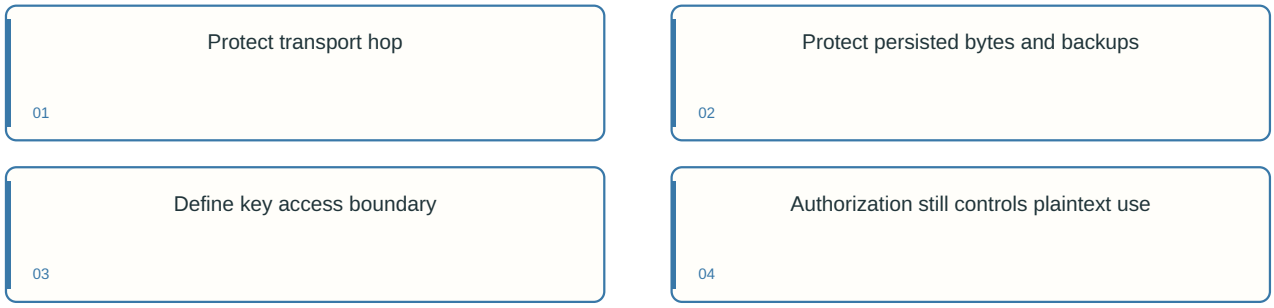


FIG. 18.2.3 **Encryption in transit and at rest · the example**

Transit and storage encryption protect different boundaries under key/access assumptions and do not replace authorisation.

Author-created illustration. Numbers are illustrative unless stated otherwise.

18.2.4 · Password hashing

A stolen password database should make each guess expensive. Purpose-built password-hashing algorithms use salts and configurable work, unlike fast general-purpose hashes. Reversible encryption creates a different risk because someone holding the key can recover the passwords.

Use a maintained implementation and current OWASP guidance. Select parameters for the deployment's hardware and capacity rather than copying one universal textbook value. Store algorithm and parameter information so hashes can be upgraded, and rehash when appropriate. Protect verification and recovery against abuse as well. Password hashing addresses offline guessing; the login and account-recovery paths still need their own controls.

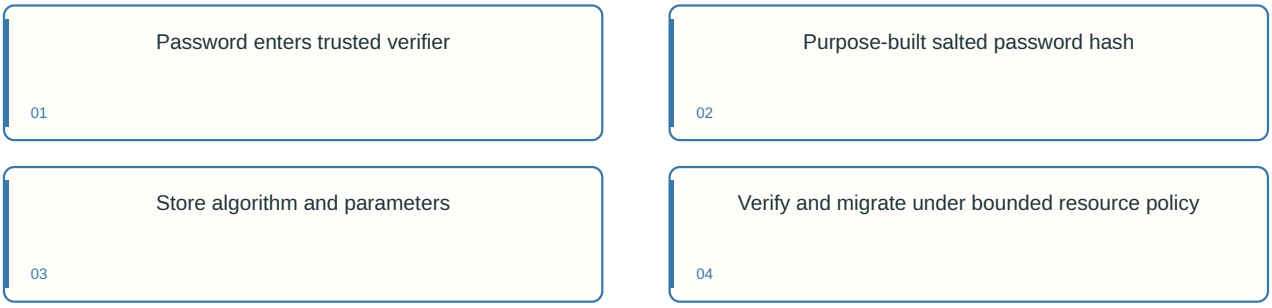


FIG. 18.2.4 **Password hashing · the example**

Password hashing uses a purpose-built salted work mechanism with maintained implementations and deployment-appropriate parameters.

Author-created illustration. Numbers are illustrative unless stated otherwise.

18.2.5 · Key management and rotation

Data encrypted with key version one may still be needed after version two becomes active. Key rotation therefore needs a mapping from data to the key version that can decrypt it.

Key management covers generation, storage, access, rotation, backup, and destruction. Envelope encryption can separate data keys from a key-encryption authority. Verify historical recovery before deleting old versions; retaining every key indefinitely can also conflict with deletion goals. Record the services and datasets depending on each version and test rotation and restore together. Changing the active key is only one step in the lifecycle.

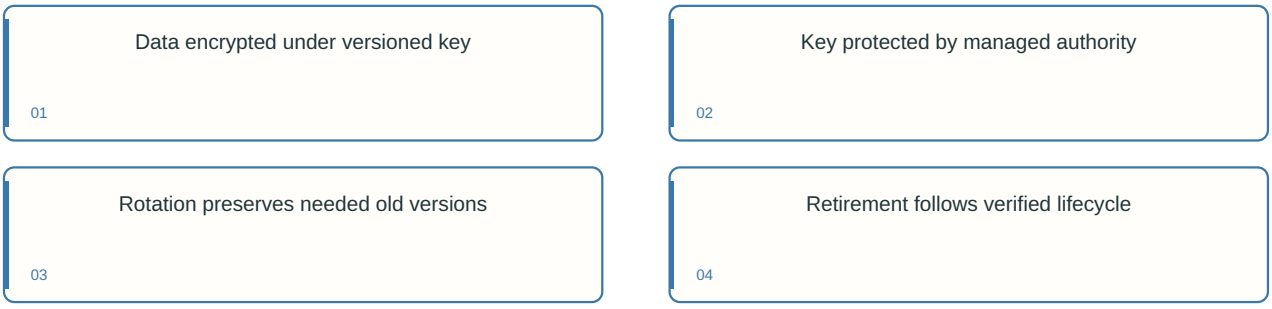


FIG. 18.2.5 **Key management and rotation · the example**

Key management tracks data dependencies through creation, rotation, historical recovery, and destruction.

Author-created illustration. Numbers are illustrative unless stated otherwise.

18.2.6 · Network segmentation

An upload worker may need storage access but no route to the production administration interface. Network segmentation limits reachable paths and can reduce lateral movement after compromise.

Outbound rules matter too. A service fetching a user-provided URL might otherwise reach private internal endpoints. Restrict and validate the relevant destinations and behaviours. An allowed network connection still needs appropriate authentication and operation-level permission. Test actual reachability, name resolution, and return paths rather than only reading a firewall-rule description. Network placement supplies one barrier in a wider access design.

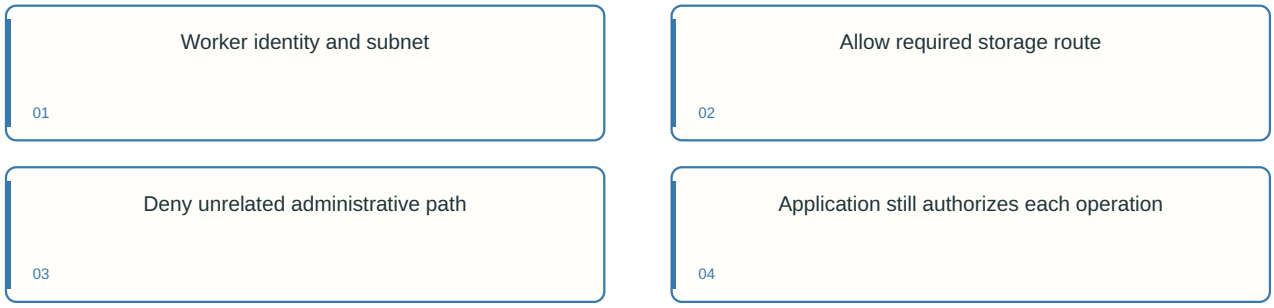


FIG. 18.2.6 **Network segmentation · the example**

Segmentation restricts network reachability while application identity and resource permissions remain necessary.

Author-created illustration. Numbers are illustrative unless stated otherwise.

18.2.7 · Zero-trust principles

A request from an internal address is not automatically trustworthy. Zero-trust principles make identity, resource, action, and context explicit rather than granting implicit access solely from network location.

Apply limited privileges and inspectable policy at appropriate boundaries. The approach is not one product, and it does not require a new central lookup for every packet. Cached credentials and policy need clear expiry and revocation behaviour. A mesh or VPN can provide useful transport and identity mechanisms while leaving business authorisation to be designed. State what evidence permits each action so the trust assumptions can be reviewed.

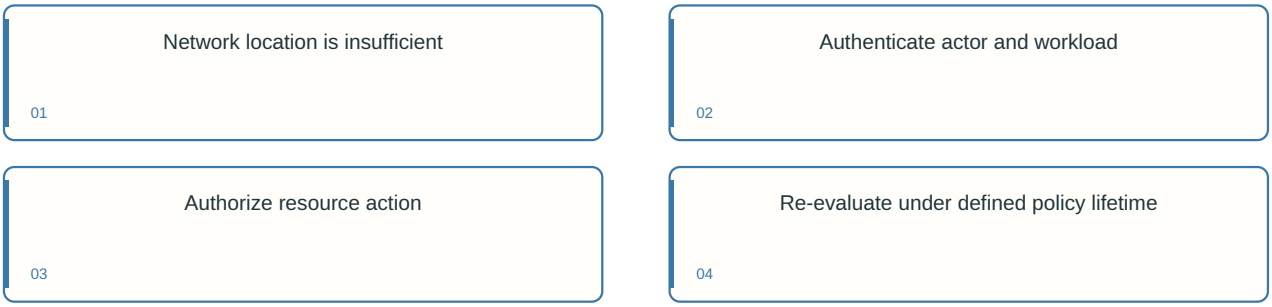


FIG. 18.2.7 **Zero-trust principles · the example**

Zero-trust principles avoid implicit permission from location alone and make trust evidence and limits explicit.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · What should revocation mean for an issued object capability and a queued export?

Specify both boundaries. An already issued capability may remain valid until expiry unless the delivery system supports revocation. A queued export may need a current policy check before execution or publication. Keep users informed about the chosen lifetime and ensure workers cannot bypass it using an old submission snapshot.

Remember: Security depends on real access and key boundaries, including how they fail and rotate.

Try applying it: Which recovery assets become useless if their only decryption key is lost?

Unit 18 exercises → · Unit 18 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. OWASP: Authorization ↗

   Official project guidance · accessed 21 Oct 2026. Least privilege, deny by default, and per-resource enforcement.

   Access reviewed · 21 October 2026

2. OWASP: Password Storage ↗

   Official project guidance · accessed 21 Oct 2026. Password-specific hashing and current deployment parameter guidance.

   Access reviewed · 21 October 2026

3. RFC 9700: OAuth 2.0 Security Best Current Practice, 2025 ↗

   Original protocol security guidance. OAuth threat and flow guidance; implementation should use maintained protocol libraries.

   Access reviewed · 21 October 2026

4. OpenID Connect Core 1.0, errata set 2 ↗

   Original specification · December 2023. Identity layer and validation requirements; distinct from API authorization.

   Access reviewed · 21 October 2026

Tenant isolation, audit & deletion

Prerequisites: 18.2 · Secrets, encryption & trust boundaries 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A correct API query is followed by a badly scoped cache or export. Carry tenant policy and minimal data through every copy, audit event, and deletion stage.

18.3.1 · API security and common vulnerabilities

A parameterised query can prevent a value from becoming SQL syntax. It does not establish whether the actor may read the selected row. Input safety and object authorisation address different risks.

Review excessive data exposure, unsafe resource use, injection, and server-side requests alongside ownership checks. Bound expensive exports and validate destinations used for callbacks or fetches. Test attempts to change protected properties, cross tenant boundaries, bypass quotas, or call privileged operations. A syntactically valid request can still be harmful. Follow how accepted input is used, rather than stopping the review after parsing.

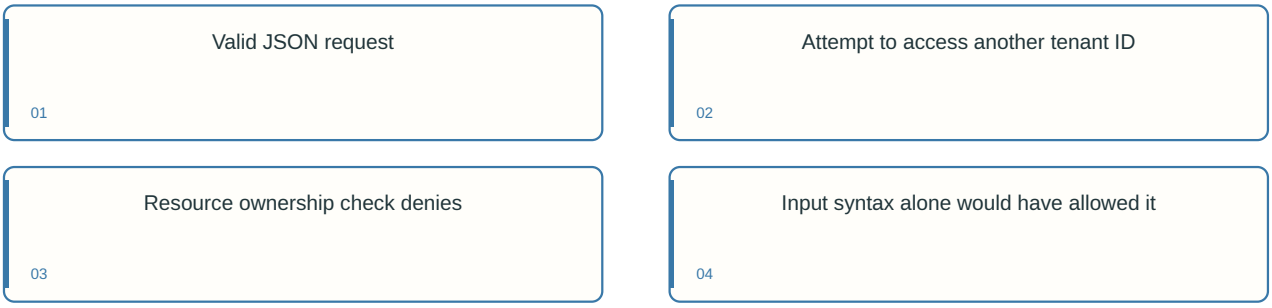


FIG. 18.3.1 **API security and common vulnerabilities · the example**

API security follows how accepted input is used, including ownership, injection, resource, and destination risks.

Author-created illustration. Numbers are illustrative unless stated otherwise.

18.3.2 · Multi-tenant isolation

A document lookup checks tenant membership correctly, but a shared cache uses only document_id as its key. A result saved for tenant A can now be returned on tenant B's path. Isolation failed outside the database query.

Carry trusted tenant context through caches, queues, object URLs, search, exports, telemetry, and administration. Recheck resource relationships instead of trusting arbitrary tenant values in job messages. Composite keys and database policies can add enforcement when configured correctly. Test two tenants and every route by which their data travels. Isolation is an end-to-end property of those paths, not a single filter.

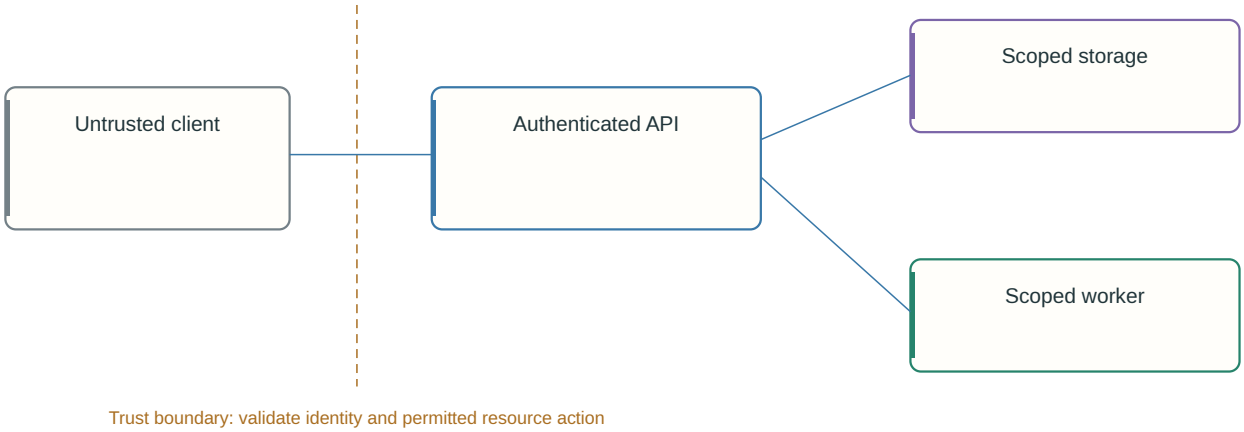


FIG. 18.3.2 **Tenant identity must survive every trust boundary**

An authenticated tenant context scopes storage queries, jobs, object capabilities, and search. Client-provided tenant labels are not independent authority.

Author-created illustration. Numbers are illustrative unless stated otherwise.

18.3.3 · Data minimization

A diagnostic event may need an operation identifier and error class, but not the full document or access token. Data minimisation asks which information is necessary for a defined purpose.

Collecting less sensitive data reduces exposure and makes retention and deletion easier. Avoid storing fields just because they might be interesting later. Record the intended use, permitted readers, retention, and derived uses. This is an architectural choice made when designing messages, logs, and storage—not merely a later policy document. Retain enough to operate the service while deliberately excluding unnecessary private content.

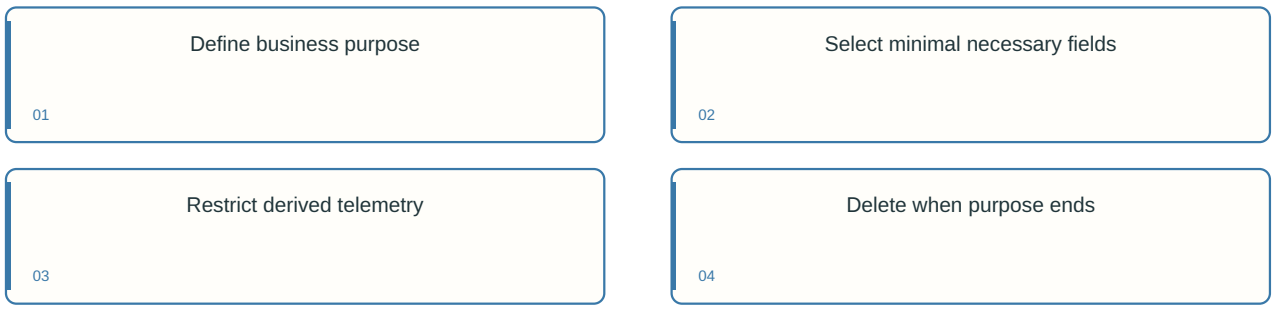


FIG. 18.3.3 **Data minimization · the example**

Data minimisation retains information justified by a stated purpose and lifecycle.

Author-created illustration. Numbers are illustrative unless stated otherwise.

18.3.4 · Audit trails

An administrator exports a private dataset. An audit trail should identify the actor, scope, target, action, and outcome, with useful time or sequence information. It should also resist inappropriate alteration by the people whose actions it records.

Store necessary identifiers and context rather than complete sensitive payloads. A “request received” record does not prove the export completed or a change committed. Record the relevant outcome boundary and reconcile interrupted privileged operations. Protect access and retention of audit data itself. Trustworthy evidence requires clear event meanings as well as an append path.

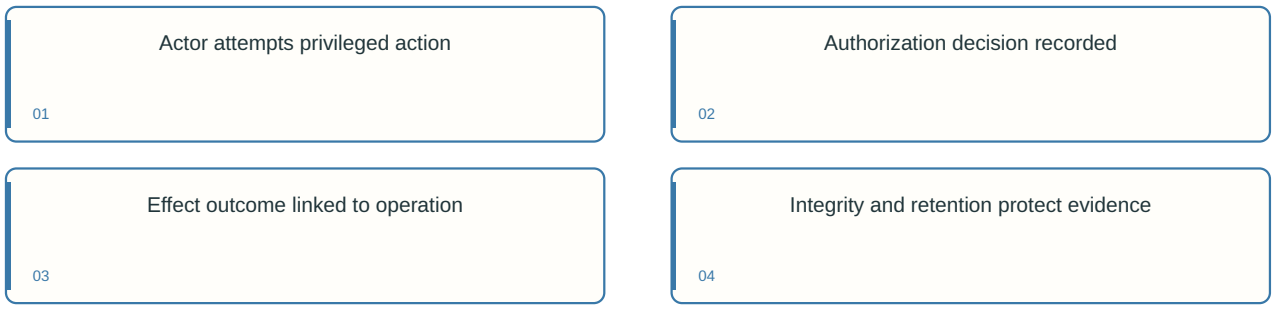


FIG. 18.3.4 **Audit trails · the example**

Audit trails record scoped actors, actions, and actual outcomes while protecting integrity and sensitive context.

Author-created illustration. Numbers are illustrative unless stated otherwise.

18.3.5 · Privacy, retention, and deletion

Deleting a record can require work in objects, indexes, caches, and recoverable copies. If the API reports completion, specify whether all required work finished or whether a durable deletion request has merely been accepted.

Track multi-store deletion progress and recovery. Test that old snapshots and replay do not recreate removed content without applying the deletion history. Privacy requirements vary by organisation and jurisdiction and need the appropriate review; this architecture lesson does not supply legal rules. It explains how collection, access, correction, retention, and deletion become concrete state transitions across the system's copies.

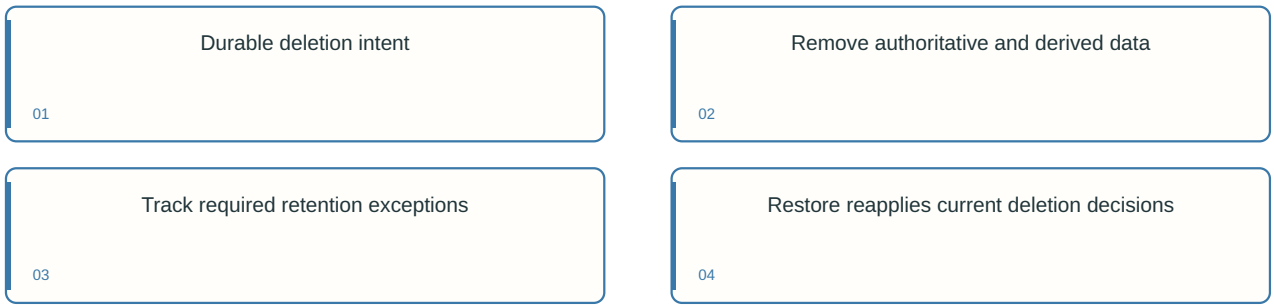


FIG. 18.3.5 **Privacy, retention, and deletion · the example**

Privacy lifecycle spans collection, access, correction, retention, deletion, and historical recovery of copies.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · What should revocation mean for an issued object capability and a queued export?

Specify both boundaries. An already issued capability may remain valid until expiry unless the delivery system supports revocation. A queued export may need a current policy check before execution or publication. Keep users informed about the chosen lifetime and ensure workers cannot bypass it using an old submission snapshot.

Remember: Isolation and privacy are lifecycle properties across the whole data path.

Try applying it: Which derived records could survive a delete that removes only the original file?

Unit 18 exercises → · Unit 18 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. OWASP: Authorization ↗

   Official project guidance · accessed 21 Oct 2026. Least privilege, deny by default, and per-resource enforcement.

   Access reviewed · 21 October 2026

2. OWASP: Password Storage ↗

   Official project guidance · accessed 21 Oct 2026. Password-specific hashing and current deployment parameter guidance.

   Access reviewed · 21 October 2026

3. RFC 9700: OAuth 2.0 Security Best Current Practice, 2025 ↗

   Original protocol security guidance. OAuth threat and flow guidance; implementation should use maintained protocol libraries.

   Access reviewed · 21 October 2026

4. OpenID Connect Core 1.0, errata set 2 ↗

   Original specification · December 2023. Identity layer and validation requirements; distinct from API authorization.

   Access reviewed · 21 October 2026

Supply chains and security reviews

Prerequisites: 18.3 · Tenant isolation, audit & deletion 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A trusted artifact still needs a controlled route to production, and users need workable safe recovery. Review the complete sensitive journey and the assumptions its controls depend on.

18.4.1 · Supply-chain security

A trusted release depends on more than application source. Build tools, dependencies, artifacts, deployment credentials, and update mechanisms form its supply chain.

Control dependency resolution, verify artifact identity, and limit build and runtime privileges. A signature proves a signing relationship, not that the signed code is harmless. Keep provenance and a way to respond to vulnerable or compromised releases. Untrusted contribution builds should not gain deployment secrets merely because trusted branch builds use them. Trace how code becomes a running artifact, identifying who can change each step and what evidence the next step checks.

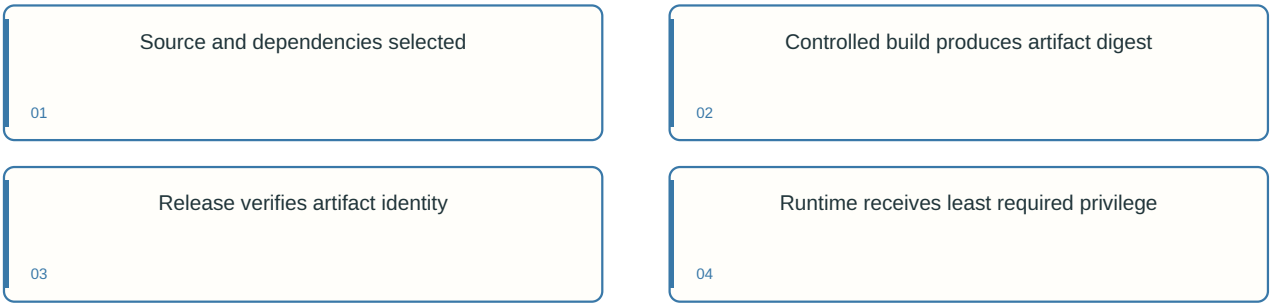


FIG. 18.4.1 **Supply-chain security · the example**

Supply-chain controls protect inputs, builds, artifacts, credentials, deployment, and compromised-release response.

Author-created illustration. Numbers are illustrative unless stated otherwise.

18.4.2 · Security vs usability trade-offs

An account-recovery rule can be so difficult that legitimate users invent unsafe workarounds. Conversely, a convenient shortcut granting every administrator global access can break tenant isolation. Evaluate the actual user journey and threat.

Safe defaults, scoped administration, understandable recovery, and proportionate extra verification can improve both protection and usability. Keep mandatory constraints explicit: convenience scores cannot justify cross-tenant disclosure. Examine fallback behaviour as carefully as normal login. A security control succeeds when people can use the supported safe path and the system continues to enforce the required boundaries.

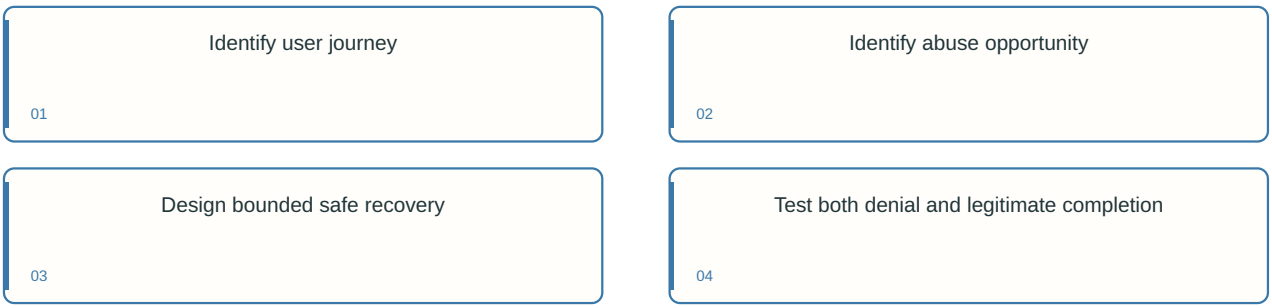


FIG. 18.4.2 **Security vs usability trade-offs · the example**

Security and usability need a workable safe journey without trading away mandatory isolation or correctness.

Author-created illustration. Numbers are illustrative unless stated otherwise.

18.4.3 · Secure-by-design reviews

Follow a sensitive export from request to download. Check the caller and tenant, filtered resources, queued context, worker permissions, output object, download capability lifetime, and audit result.

Then change the conditions: membership is revoked after enqueue, a worker retries after cancellation, or deletion starts before publication. The chosen policy must determine what happens at each later boundary. Record who owns each control and how it is checked. A review checklist aids coverage, while tests and configurations provide evidence. “Secure by design” should describe this reasoning and enforcement, rather than an unsupported label attached to the architecture.

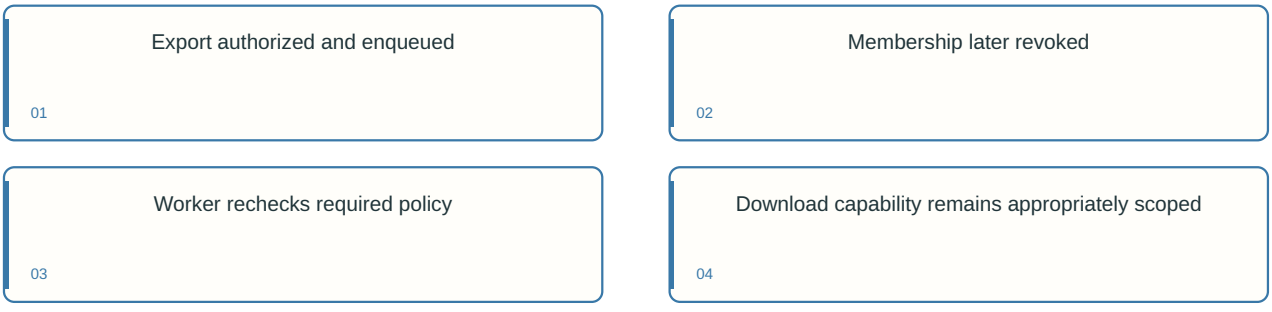


FIG. 18.4.3 **Secure-by-design reviews · the example**

Secure reviews follow sensitive operations and changed conditions through every required enforcement boundary.

Author-created illustration. Numbers are illustrative unless stated otherwise.

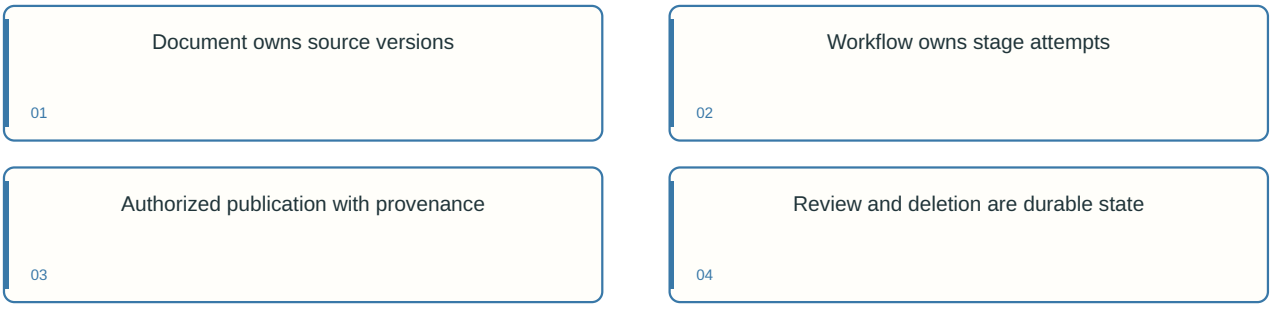


FIG. 18.4.4 **State and ownership in the related design**

Fields and decisions retain tenant, input, transform, and evidence relationships.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Try the mechanism in code

PYTHON · SYNTHETIC TENANT POLICY

```
from mechanisms import permit_demo
actor = {"tenant":"A", "active":True, "actions":["read"]}
assert permit_demo(actor, {"tenant":"A"}, "read")
assert not permit_demo(actor, {"tenant":"B"}, "read")
```

The predicate permits the same-tenant read and rejects tenant A reading tenant B. It assumes the supplied actor and resource attributes are already trustworthy.

This is authorisation logic, not credential validation. Every object, queue, cache, and export path must enforce the relevant rule with valid context.

Download the executable mechanism collection ↓ · Follow the connected complete design →

▼ Think further · What should revocation mean for an issued object capability and a queued export?

Specify both boundaries. An already issued capability may remain valid until expiry unless the delivery system supports revocation. A queued export may need a current policy check before execution or publication. Keep users informed about the chosen lifetime and ensure workers cannot bypass it using an old submission snapshot.

Remember: Secure design joins threat-specific controls with actual enforcement and testable outcomes.

Try applying it: What must happen if membership is revoked after a private export is queued?

Unit 18 exercises → · Unit 18 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. OWASP: Authorization ↗

   Official project guidance · accessed 21 Oct 2026. Least privilege, deny by default, and per-resource enforcement.

   Access reviewed · 21 October 2026

2. OWASP: Password Storage ↗

   Official project guidance · accessed 21 Oct 2026. Password-specific hashing and current deployment parameter guidance.

   Access reviewed · 21 October 2026

3. RFC 9700: OAuth 2.0 Security Best Current Practice, 2025 ↗

   Original protocol security guidance. OAuth threat and flow guidance; implementation should use maintained protocol libraries.

   Access reviewed · 21 October 2026

4. OpenID Connect Core 1.0, errata set 2 ↗

   Original specification · December 2023. Identity layer and validation requirements; distinct from API authorization.

   Access reviewed · 21 October 2026

PART 5 / UNIT 19

Observability, Monitoring, and Incident Engineering

A user waits while CPU looks normal. Combine population measurements, individual timelines, and structured events to find the work or wait the dashboard missed.

The running example

An API's p99 latency increases from 200 ms to 5 seconds. Use traces, metrics, logs, and database analysis to localize the bottleneck.

Unit 19 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

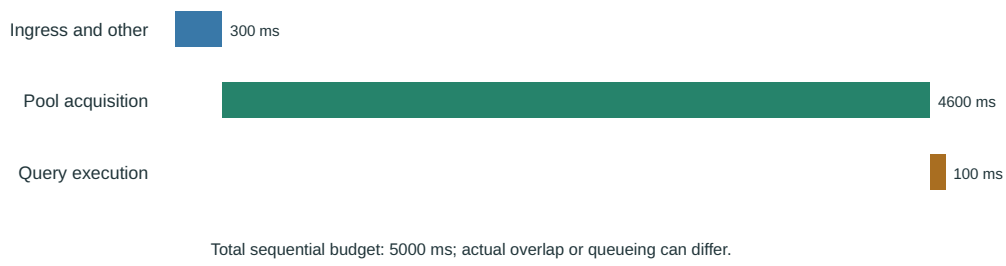


FIG. 19.0.1 **This slow request is dominated by waiting**

The synthetic five-second path spends 92% in pool acquisition. More CPU is not an evidence-based first fix for this execution.

Author-created illustration. Numbers are illustrative unless stated otherwise.

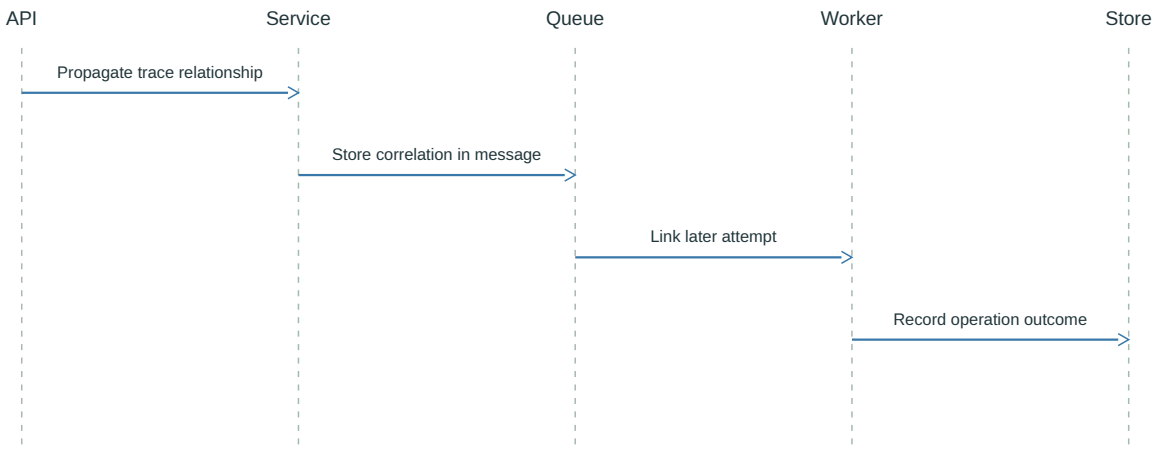


FIG. 19.0.2 **request ID propagation**

Trace IDs relate evidence; they are not authorization or proof that the entire workflow is complete.

Author-created illustration. Numbers are illustrative unless stated otherwise.

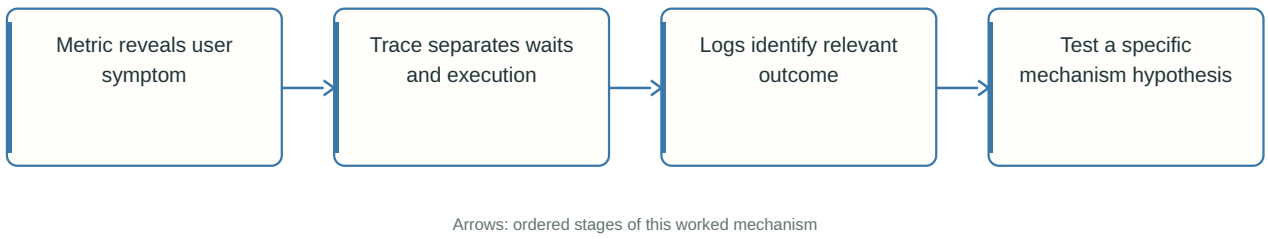


FIG. 19.0.3 **metrics-to-root-cause workflow**

No one signal substitutes for a causal investigation with explicit counterevidence.

Author-created illustration. Numbers are illustrative unless stated otherwise.



FIG. 19.0.4 **error-budget burn chart**

Percentages: at a 99.9% event objective, 1% bad events burns the 0.1% allowance ten times as fast.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Observe behavior with three signals

Prerequisites: Requirements, Estimation, and Engineering Trade-offs, Networking Foundations for System Design, Runtime, Processes, Threads, and Concurrency, Database Internals, Transactions, and Concurrency Control

6 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A user waits while CPU looks normal. Combine population measurements, individual timelines, and structured events to find the work or wait the dashboard missed.

19.1.1 · Monitoring vs observability

A user waits five seconds for a payment request while the CPU dashboard looks normal. Monitoring the CPU told us about one known signal; it did not explain the user's wait.

Monitoring tracks selected conditions. Observability emphasises being able to explain behaviour using the system's outputs and instrumentation. These are useful emphases rather than universal product boundaries. Start from an operation and connect its work, waiting, dependencies, and state changes. Collect evidence that supports questions and actions. More telemetry is not automatically more understanding, particularly when it is uncorrelated or hides the business outcome.

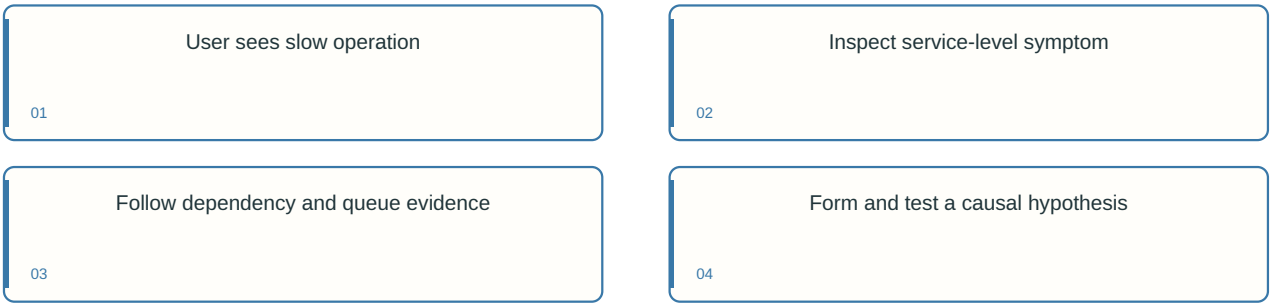


FIG. 19.1.1 **Monitoring vs observability · the example**

Monitoring follows known signals; observability helps explain behaviour through correlated outputs and instrumentation.

Author-created illustration. Numbers are illustrative unless stated otherwise.

19.1.2 · Logs, metrics, and traces

A latency histogram shows that many requests are slow. A trace reveals where one of them waited. A structured log explains the error returned by a particular attempt. Metrics, traces, and logs answer complementary questions.

Logs record events; metrics aggregate measurements; traces connect work along an execution path. Sampling, dropped records, clock differences, and privacy limits affect what each can establish. Use stable operation identities where appropriate to connect detailed evidence. Avoid turning every unique identifier into a metric label. Combine population-level measurements with individual histories rather than expecting one trace or log line to describe every user.

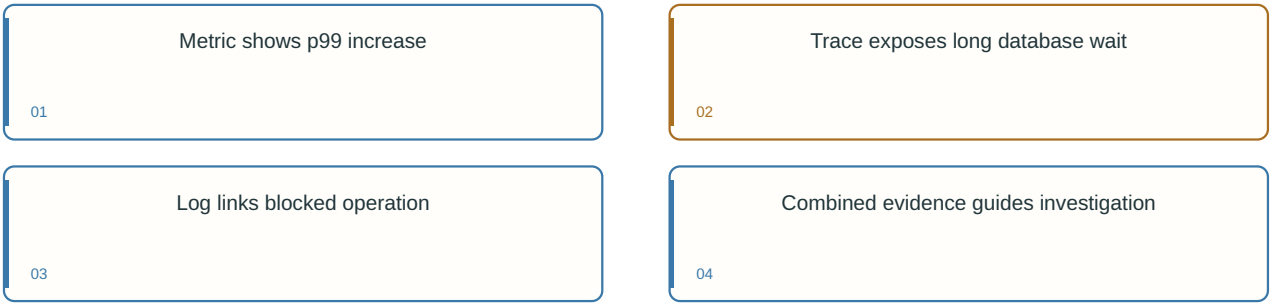


FIG. 19.1.2 **Logs, metrics, and traces · the example**

Logs record events, metrics aggregate populations, and traces connect execution work and waits.

Author-created illustration. Numbers are illustrative unless stated otherwise.

19.1.3 · Structured logging

Instead of a free-form line such as “payment failed,” record named fields: event type, operation identity, outcome class, and relevant duration or safe scope. Structured logging makes their meanings easier to query consistently.

A start event and a committed outcome are different observations. If a finish event is missing, the process might have stopped, but that absence does not identify the payment's remote result. Keep credentials and raw private documents out of routine logs. Bound message sizes and volume so an error burst cannot turn logging itself into the next outage. Record enough to investigate with a clear purpose and retention policy.

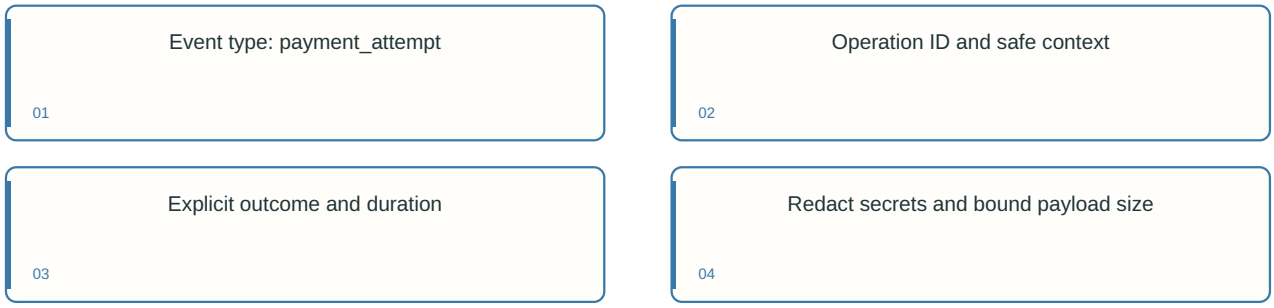


FIG. 19.1.3 **Structured logging · the example**

Structured logs give fields stable meanings, bounded size, safe content, and explicit event boundaries.

Author-created illustration. Numbers are illustrative unless stated otherwise.

19.1.4 · Distributed tracing

A five-second trace shows 4.6 seconds acquiring a database connection. Optimising SQL execution is unlikely to remove most of that wait. Instrument the acquisition and queue intervals so the trace exposes them separately.

Distributed tracing connects spans for work and waits across components. Asynchronous jobs may need links rather than a parent relationship implying they finished within the original request. A trace is one execution, possibly sampled. Compare it with normal traces, errors, and distributions before concluding that the same explanation covers the population. The useful result is a supported location for delay, not just a coloured timeline.

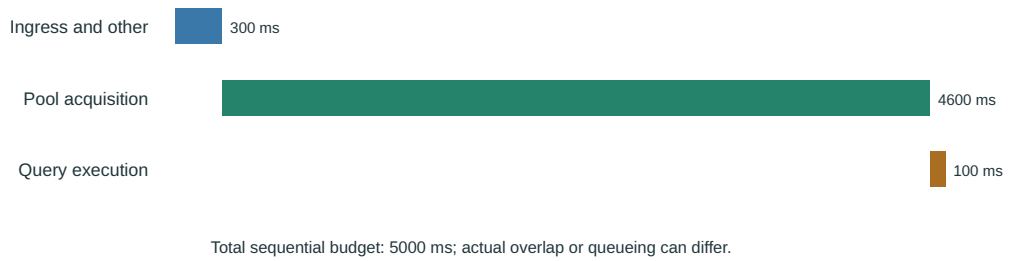


FIG. 19.1.4 **This slow request is dominated by waiting**

The synthetic five-second path spends 92% in pool acquisition. More CPU is not an evidence-based first fix for this execution.

Author-created illustration. Numbers are illustrative unless stated otherwise.

19.1.5 · Trace IDs, span IDs, context propagation

A trace identifier connects related spans; each span identifier names a particular piece of work. Context propagation carries the needed relationships over calls and messages.

Incoming context can be untrusted. Validate or replace it according to policy, and never treat a trace identifier as permission to access a resource. For queued work, preserve appropriate correlation or links across the request and later attempts. Standard formats aid interoperability but do not instrument every library automatically. Check that each transport actually carries the context and that sensitive user data is not smuggled into it as a convenient correlation field.

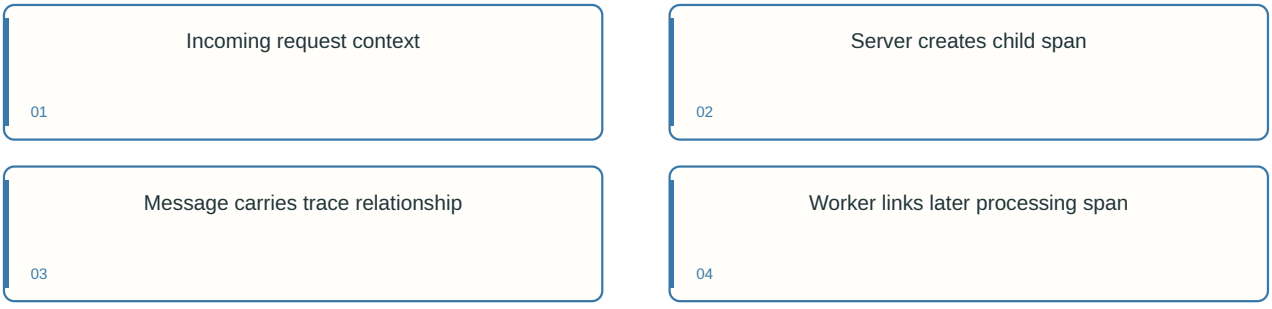


FIG. 19.1.5 **Trace IDs, span IDs, context propagation · the example**

Trace context links observations across calls or messages and must not be treated as authorisation.

Author-created illustration. Numbers are illustrative unless stated otherwise.

19.1.6 · Golden signals

Traffic rises, queues grow, and responses become slower while errors initially remain low. Looking only at failure counts would miss the developing problem. Golden signals commonly combine latency, traffic, errors, and saturation.

Define useful responses and eligible work, rather than counting every HTTP 200 as business success. Break down important operations where high-volume cheap calls could hide low-volume critical failures. Interpret resource pressure alongside user outcomes. These signals organise investigation, while workload-specific measures such as oldest unresolved payment age can reveal harm outside the ordinary request-response path.

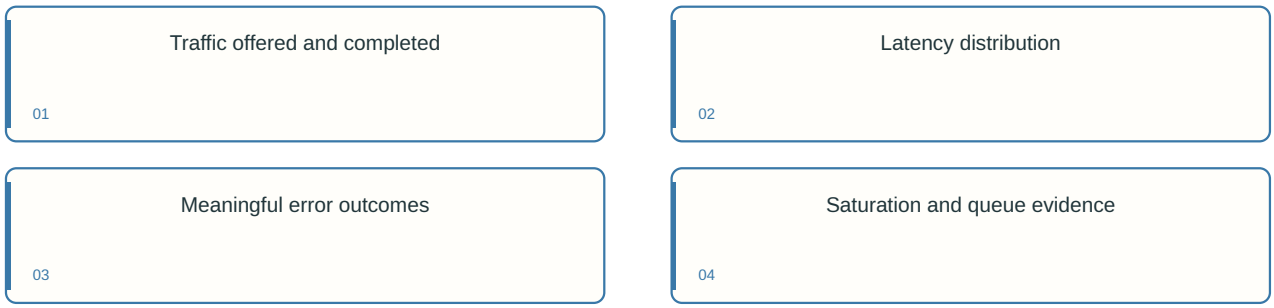


FIG. 19.1.6 **Golden signals · the example**

Golden signals connect latency, traffic, errors, and saturation using meaningful eligible outcomes.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · What evidence would support a latency explanation beyond correlation with a release?

Locate the extra wait or execution on affected paths, compare normal and slow populations, inspect resource saturation and changed behaviour, and test a focused intervention. A release marker is a clue. Evidence is stronger when the mechanism predicts the symptom and the correction changes that mechanism under comparable conditions.

Remember: Useful telemetry connects a user-visible outcome to its resource and state path.

Try applying it: What can a trace show that a latency histogram cannot, and what can the histogram show that one trace cannot?

Unit 19 exercises → · Unit 19 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. OpenTelemetry: Signals ↗

   Official documentation · accessed 21 Oct 2026. Complementary observability signals.

   `Access reviewed · 21 October 2026`

2. OpenTelemetry: Context propagation ↗

   Official documentation · accessed 21 Oct 2026. Trace context relationships; trace identity does not authorize access.

   `Access reviewed · 21 October 2026`

3. Alerting on SLOs — The SRE Workbook ↗

   Engineering book. Budget burn and actionable alerts.

   `Access reviewed · 21 October 2026`

Choose indicators, budgets & alerts

Prerequisites: 19.1 · Observe behavior with three signals 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A full pool creates long waits, but paging on every CPU fluctuation overwhelms responders. Choose bounded evidence, indicators, and alerts that support a particular operating decision.

19.2.1 · RED and USE methods

RED examines rate, errors, and duration for request-serving components. USE examines utilisation, saturation, and errors for resources. Apply the perspective that fits the question being asked.

A full connection pool can delay requests while CPU stays low. USE points attention to the saturated pool; RED shows the resulting duration and error behaviour. Use sufficient time detail and per-owner views to find bursts or hot shards hidden by averages. Neither method replaces domain indicators. A queue of uncertain charges can need action even while the general web service looks healthy.

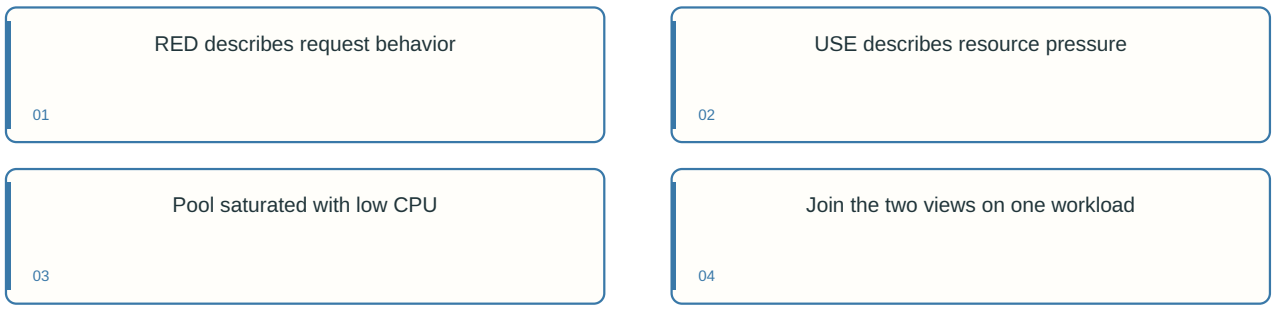


FIG. 19.2.1 **RED and USE methods · the example**

RED focuses on serving rate, errors, and duration; USE examines resource utilisation, saturation, and errors.

Author-created illustration. Numbers are illustrative unless stated otherwise.

19.2.2 · SLIs, SLOs, error budgets

For a 99.9% success target, the allowed bad-event rate is 0.1%. If 1% of eligible events fail during an interval, the service spends that budget at ten times the allowed rate. This is tenfold error-budget burn.

An SLI measures the behaviour; an SLO sets its target and window; the error budget expresses the allowed shortfall. Define both good and eligible events, including relevant failures. Segment critical operations when one denominator hides their harm. Handle low-volume indicators thoughtfully. The objective guides service decisions over its population and window; it is not a promise that every individual request succeeds.

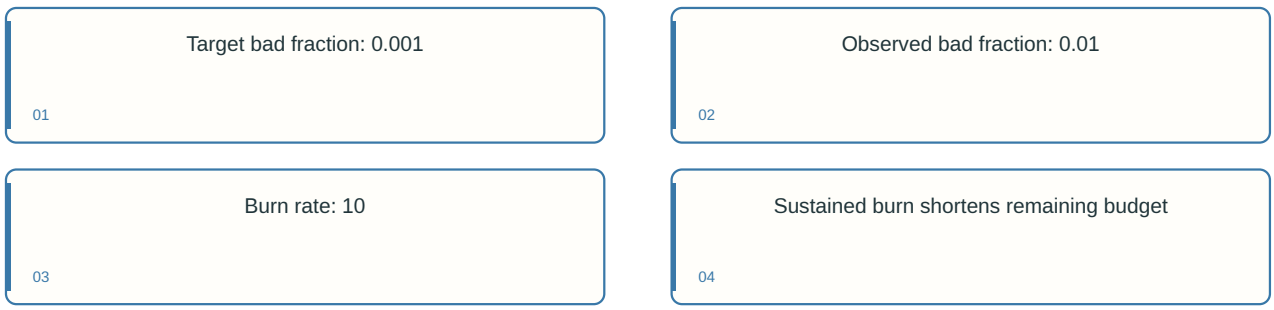


FIG. 19.2.2 **SLIs, SLOs, error budgets · the example**

SLIs measure behaviour; SLOs target it; error-budget burn compares observed shortfall with its allowance.

Author-created illustration. Numbers are illustrative unless stated otherwise.

19.2.3 · Alert design and alert fatigue

An alert should lead someone to a useful action. Repeated pages about isolated CPU spikes can distract responders, while sustained rapid budget consumption may indicate immediate user harm.

Choose windows and thresholds suited to the response time and objective. Assign an owner, include context, and provide a runbook for urgent cases. Avoid a series of alerts that all demand the same action during known degradation. Review missed incidents and unnecessary pages, not just the number configured. Alerting is an operating workload, so its value includes helping people recognise and mitigate the right problem promptly.

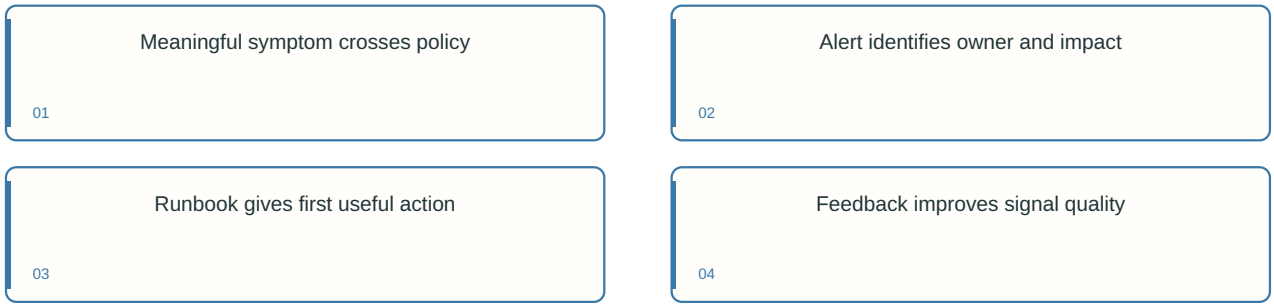


FIG. 19.2.3 **Alert design and alert fatigue · the example**

Useful alerts identify actionable harm or impending objective loss with an owner and response.

Author-created illustration. Numbers are illustrative unless stated otherwise.

19.2.4 · Sampling and telemetry cost

Keeping every trace forever may cost too much. Head sampling chooses early; tail sampling can use later observations such as duration or failure, but needs buffering and processing capacity.

Sampling and retention change which questions can be answered. Unique request IDs as ordinary metric labels can create unbounded numbers of series. Put detailed identities in suitable logs or traces and use bounded metric dimensions. Protect telemetry ingestion during failure bursts, when both volume and need can rise. Select a strategy according to diagnostic purpose, cost, and privacy instead of treating either maximum collection or maximum reduction as the default goal.

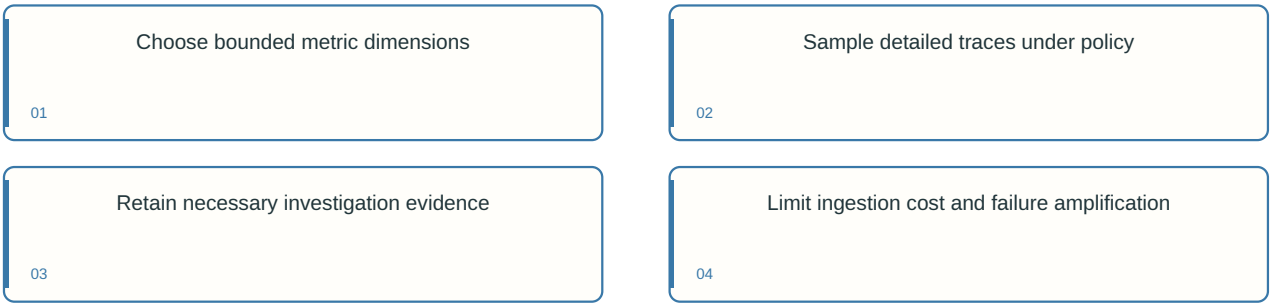


FIG. 19.2.4 **Sampling and telemetry cost · the example**

Sampling and retention balance diagnostic questions, privacy, cardinality, and telemetry capacity.

Author-created illustration. Numbers are illustrative unless stated otherwise.

19.2.5 · Dashboards

Begin a dashboard with a decision: is the service meeting its objective, what changed, or which resource needs capacity? Show the relevant user outcomes before unrelated infrastructure measurements.

Include units, denominators, windows, deployment markers, and data gaps. A line without a stable definition can mislead during an incident. Provide useful drill-downs and ownership rather than hundreds of equal-priority panels. Check the dashboard against known examples or earlier incidents. The purpose is to shorten a reasoning path from symptom to action, not simply display all available measurements on one screen.

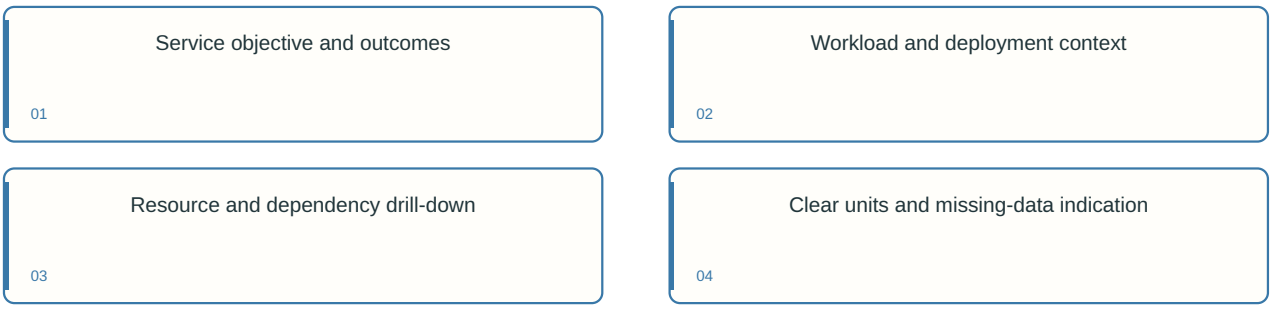


FIG. 19.2.5 **Dashboards · the example**

Dashboards organise evidence for decisions with clear units, denominators, windows, gaps, and ownership.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · What evidence would support a latency explanation beyond correlation with a release?

Locate the extra wait or execution on affected paths, compare normal and slow populations, inspect resource saturation and changed behaviour, and test a focused intervention. A release marker is a clue. Evidence is stronger when the mechanism predicts the symptom and the correction changes that mechanism under comparable conditions.

Remember: Measure the relevant denominator and make each urgent signal actionable.

Try applying it: Why should a unique request ID usually stay out of a general metric label?

Unit 19 exercises → · Unit 19 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. OpenTelemetry: Signals ↗

   Official documentation · accessed 21 Oct 2026. Complementary observability signals.

   `Access reviewed · 21 October 2026`

2. OpenTelemetry: Context propagation ↗

   Official documentation · accessed 21 Oct 2026. Trace context relationships; trace identity does not authorize access.

   `Access reviewed · 21 October 2026`

3. Alerting on SLOs — The SRE Workbook ↗

   Engineering book. Budget burn and actionable alerts.

   `Access reviewed · 21 October 2026`

Profile and investigate production

Prerequisites: 19.2 · Choose indicators, budgets & alerts 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A deployment precedes a slowdown, but correlation is only a clue. Use profiles, traces, query evidence, and a coordinated response to test explanations and restore useful service.

19.3.1 · Profiling

A profiler can show where CPU samples occur, where allocations grow, or where threads block. Under a representative workload, this can reveal repeated parsing or lock contention missed by coarse request metrics.

Choose a method with acceptable overhead and privacy behaviour. A function consuming most CPU is not necessarily responsible for most response time if requests spend longer waiting on I/O. Compare profiles with traces and completion rates. Make a focused change, then measure again. Profiling is most useful when it tests an explanation about an actual bottleneck rather than producing a ranking to optimise blindly.

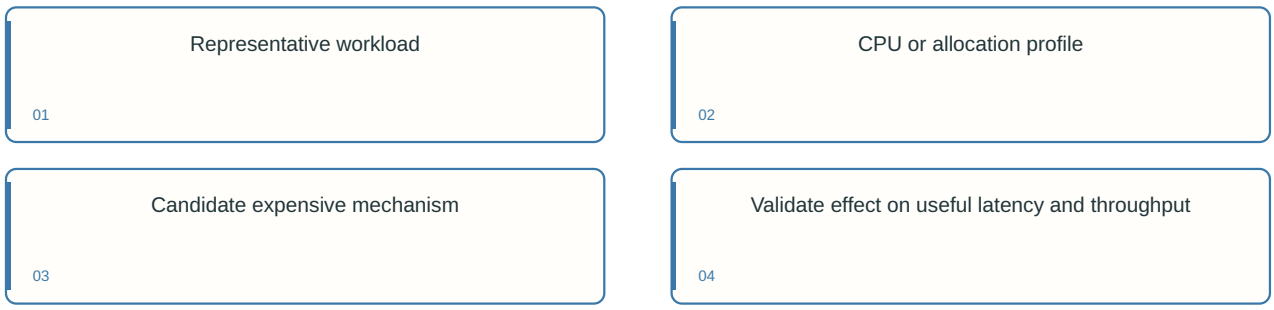


FIG. 19.3.1 **Profiling · the example**

Profiling locates runtime work or waits under measured conditions, which must be related to user outcomes.

Author-created illustration. Numbers are illustrative unless stated otherwise.

19.3.2 · Query and application performance diagnosis

Split a slow request into queue wait, connection acquisition, lock wait, query execution, network transfer, and encoding. This gives plausible causes a place in the path.

If p99 grows from 200 ms to five seconds with modest CPU, a full pool or lock may be responsible, but evidence must distinguish them. Compare slow and normal requests and recent changes. Preserve useful evidence before restarting components, and assess whether a restart would add load or erase diagnostic state. Diagnosis narrows a hypothesis with observations; a familiar symptom alone does not establish its cause.

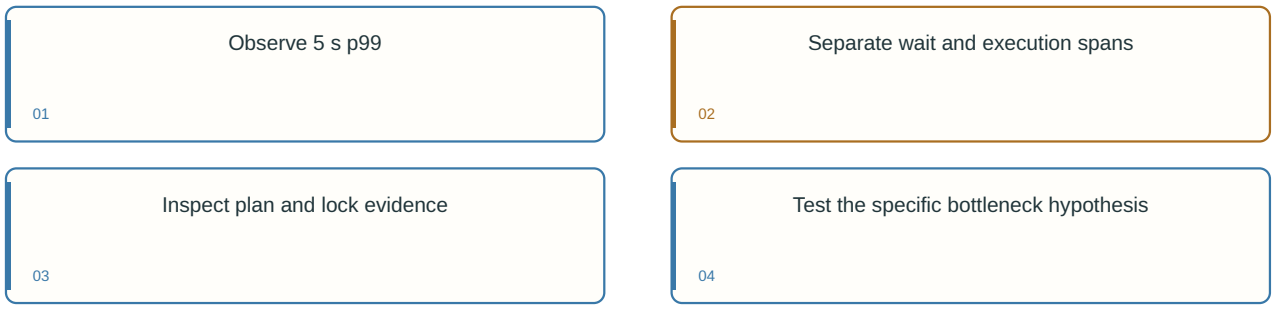


FIG. 19.3.2 **Query and application performance diagnosis · the example**

Diagnosis separates waiting from execution and tests proposed bottleneck mechanisms against evidence.

Author-created illustration. Numbers are illustrative unless stated otherwise.

19.3.3 · Incident response

During a complex incident, coordinate mitigation, communication, evidence, and decision ownership. Keep a timeline and limit unrelated changes so the situation remains understandable.

Choose an intervention that restores useful service while protecting state. Code rollback may be unsafe after an incompatible data migration. Track pending business outcomes after infrastructure recovers: unresolved provider charges still require reconciliation. A stopped alert does not by itself close those outcomes. Scale incident leadership and communication to the event's complexity, keeping a clear owner for decisions and follow-up work.

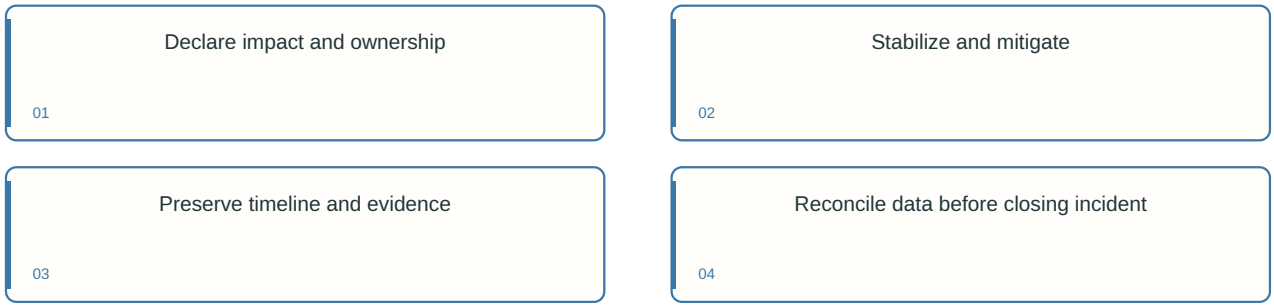


FIG. 19.3.3 **Incident response · the example**

Incident response coordinates mitigation, decisions, communication, and remaining uncertain business outcomes.

Author-created illustration. Numbers are illustrative unless stated otherwise.

19.3.4 · On-call and runbooks

A runbook saying only “restart the service” leaves diagnosis, risk, and verification to the responder. A useful runbook explains what to inspect, which actions are supported, when to stop, and how to confirm recovery.

On-call also needs access, escalation, rehearsal, and sustainable ownership. Audit emergency access. Let service owners reduce recurring failure mechanisms instead of merely extending instructions for repeated manual repair. If nightly reconciliation or cache rebuilding is constantly needed, that operating burden is evidence about the architecture. Good instructions help people operate it; persistent toil may require changing the system itself.

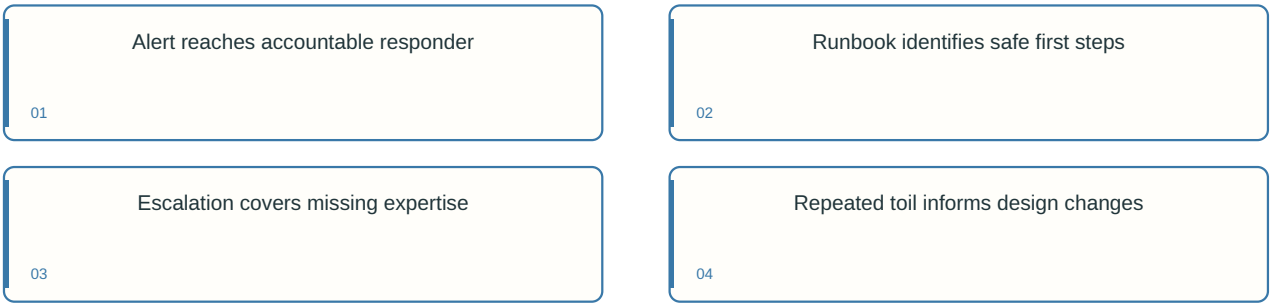


FIG. 19.3.4 **On-call and runbooks · the example**

On-call needs useful alerts, access, escalation, rehearsed runbooks, and service ownership that reduces recurring toil.

Author-created illustration. Numbers are illustrative unless stated otherwise.

19.3.5 · Root-cause investigation

A failure follows a deployment, but the deployment may only be the trigger. An unbounded queue or incompatible schema can explain why that change caused widespread harm.

Test explanations against the timeline and affected requests. Ask which resource or state transition connects the suspected cause to the observation, and whether the proposed fix changes that mechanism. Correlation starts an investigation; it does not complete it. Several contributing conditions are common. Keeping them visible leads to stronger prevention than choosing one short label that hides why detection, isolation, or recovery also failed.

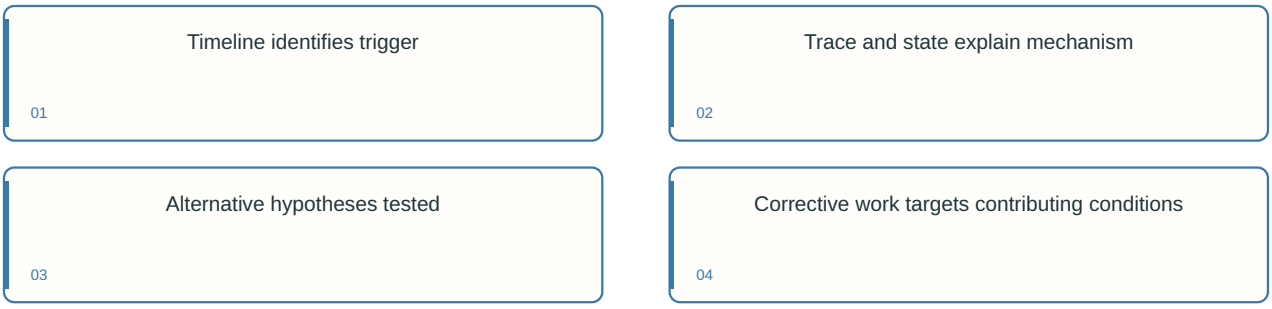


FIG. 19.3.5 **Root-cause investigation · the example**

Cause investigation distinguishes triggers from contributing mechanisms and tests explanations beyond correlation.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · What evidence would support a latency explanation beyond correlation with a release?

Locate the extra wait or execution on affected paths, compare normal and slow populations, inspect resource saturation and changed behaviour, and test a focused intervention. A release marker is a clue. Evidence is stronger when the mechanism predicts the symptom and the correction changes that mechanism under comparable conditions.

Remember: Separate triggers from mechanisms and preserve correctness while mitigating harm.

Try applying it: Which evidence would distinguish a slow query from a long wait to acquire its connection?

Unit 19 exercises → · Unit 19 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. OpenTelemetry: Signals ↗

   Official documentation · accessed 21 Oct 2026. Complementary observability signals.

   Access reviewed · 21 October 2026

2. OpenTelemetry: Context propagation ↗

   Official documentation · accessed 21 Oct 2026. Trace context relationships; trace identity does not authorize access.

   Access reviewed · 21 October 2026

3. Alerting on SLOs — The SRE Workbook ↗

   Engineering book. Budget burn and actionable alerts.

   Access reviewed · 21 October 2026

Learn from incidents and trace async work

Prerequisites: 19.3 · Profile and investigate production 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

An asynchronous job remains unresolved after HTTP serving recovers. Follow the accepted operation through its later attempts, and use incident evidence to improve the system that owns it.

19.4.1 · Blameless postmortems

A blameless postmortem records impact, timeline, detection, mitigation, and contributing conditions using what people knew at the time. Include the measures that helped as well as the gaps.

“Be more careful” supplies no change to the failure mechanism. Assign specific corrective work with an owner and evidence that it changes future behaviour. Explain why safeguards failed to prevent or limit harm, and avoid exposing unnecessary customer or employee information. The document should help the team reason about recurrence and recovery, rather than turn hindsight into a judgment of the nearest individual.

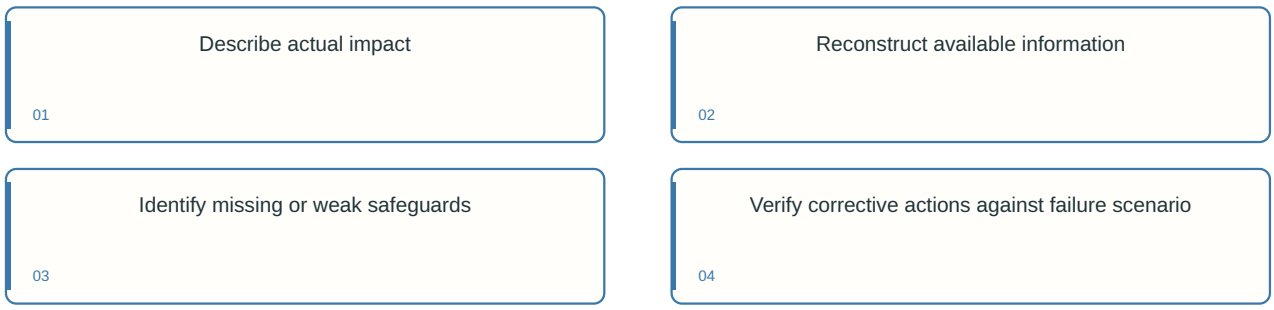


FIG. 19.4.1 **Blameless postmortems · the example**

Blameless postmortems reconstruct impact and conditions, with specific owned improvements and verification evidence.

Author-created illustration. Numbers are illustrative unless stated otherwise.

19.4.2 · Capacity monitoring

A fleet's average CPU is low while one shard fills its disk and a connection pool stays saturated. Capacity monitoring needs the resources and owners that can actually limit service.

Track rates, queue age, growth, and surviving capacity under failure. Include background costs such as compaction, backup, and migration. Alert early enough for the available response: a disk growing faster than procurement is a different risk from a short CPU burst. Keep uncertainty in forecasts and revise them when traffic patterns or releases change. A useful forecast includes both the approaching limit and the time needed to act.

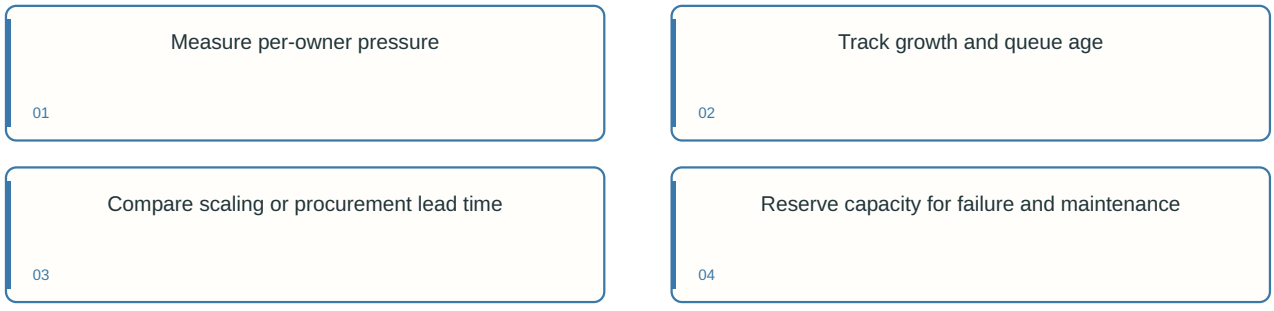


FIG. 19.4.2 **Capacity monitoring · the example**

Capacity monitoring tracks meaningful approaching limits, growth uncertainty, and the time available to respond.

Author-created illustration. Numbers are illustrative unless stated otherwise.

19.4.3 · Observability across asynchronous workflows

An HTTP request returns 202 in a few milliseconds, but its extraction job waits for hours. Measuring acceptance latency alone misses the service the user actually requested.

Follow the stable operation identity through submission, delivery, attempts, stage commits, retries, and final outcome. Separate queue delay from processing duration and record stale or duplicate attempt rejection. Define completion age and success at the business boundary. Provide a way to locate stuck work across brokers, workers, stores, and providers. Asynchronous observability extends the history beyond the original connection so accepted work does not disappear from view.

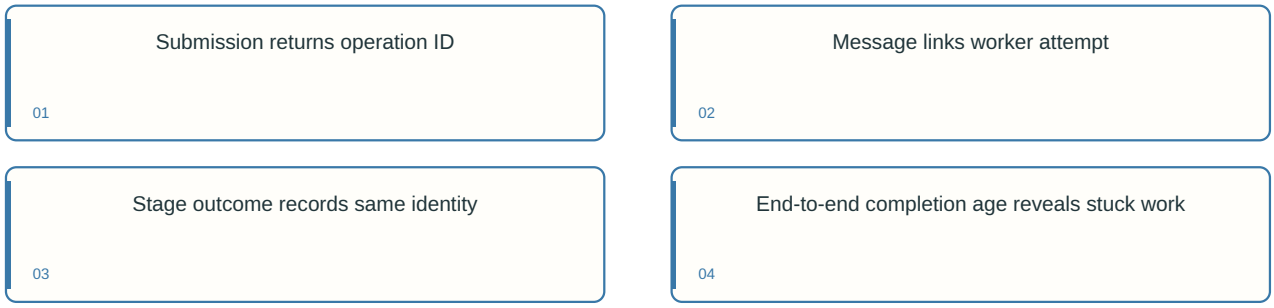


FIG. 19.4.3 **Observability across asynchronous workflows · the example**

Asynchronous observability follows accepted work through later attempts, stage waits, and required final outcomes.

Author-created illustration. Numbers are illustrative unless stated otherwise.

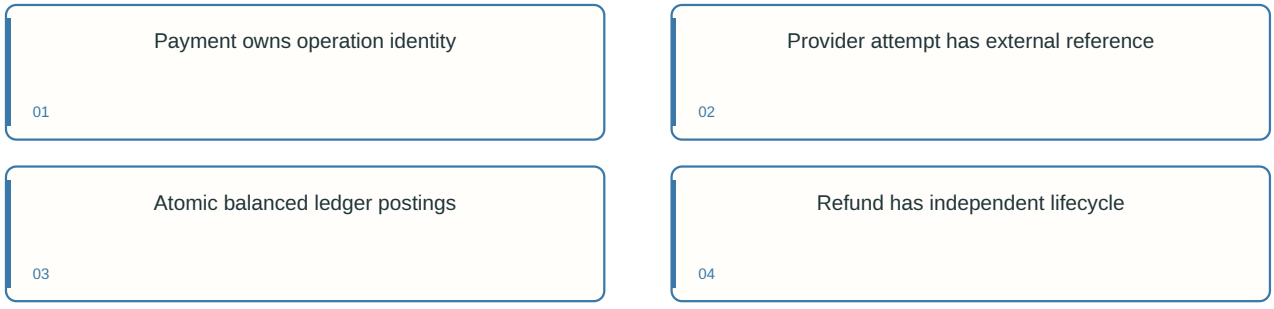


FIG. 19.4.4 **State and ownership in the related design**

Operation identity links provider evidence to payment state and balanced accounting entries.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Try the mechanism in code

PYTHON · ONE TRACE ACCOUNTING MODEL

```
stages_ms = {"other":300, "pool_wait":4600, "query":100}
assert sum(stages_ms.values()) == 5000
assert stages_ms["pool_wait"] / sum(stages_ms.values()) == .92
```

Pool acquisition takes 4,600 of 5,000 ms, or 92% of this request's elapsed time. The query itself accounts for only 100 ms.

Investigate hold times, locks, and the wider population before attributing the whole incident to one trace. The arithmetic locates the wait but does not establish why the pool is full.

Download the executable mechanism collection ↓ · Follow the connected complete design →

▼ Think further · What evidence would support a latency explanation beyond correlation with a release?

Locate the extra wait or execution on affected paths, compare normal and slow populations, inspect resource saturation and changed behaviour, and test a focused intervention. A release marker is a clue. Evidence is stronger when the mechanism predicts the symptom and the correction changes that mechanism under comparable conditions.

Remember: Operation age, outcomes, and recovery conditions extend beyond the original request.

Try applying it: What should an on-call engineer inspect when acceptance is healthy but document completion has stopped?

Unit 19 exercises → · Unit 19 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. OpenTelemetry: Signals ↗

   Official documentation · accessed 21 Oct 2026. Complementary observability signals.

   Access reviewed · 21 October 2026

2. OpenTelemetry: Context propagation ↗

   Official documentation · accessed 21 Oct 2026. Trace context relationships; trace identity does not authorize access.

   Access reviewed · 21 October 2026

3. Alerting on SLOs — The SRE Workbook ↗

   Engineering book. Budget burn and actionable alerts.

   Access reviewed · 21 October 2026

PART 5 / UNIT 20

Deployment, Cloud Infrastructure, and Platform Architecture

The same image works locally and fails in production. Compare environments and runtime isolation, keeping persistent state, resources, and platform duties explicit.

The running example

Take a single-process application and deploy it into a reliable, observable, multi-instance production environment with safe releases.

Unit 20 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

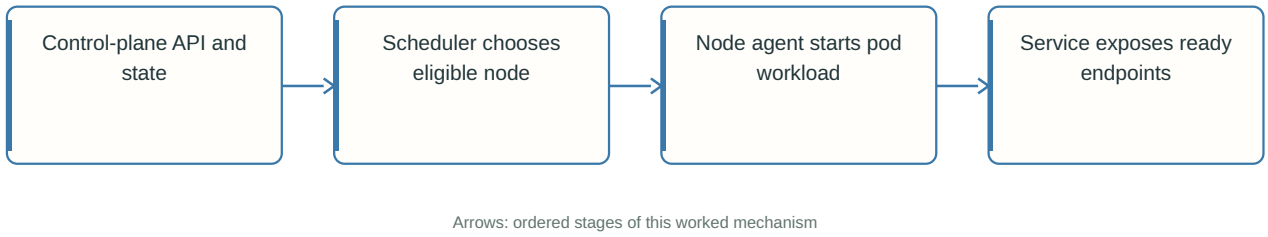
Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

Layer	Virtual machine	Container
Operating environment	Guest OS per VM	Shared host kernel
Process resources	Guest-managed processes	Runtime constraints
Persistent state	Explicit attached owner	Not ephemeral writable layer

FIG. 20.0.1 **VMs and containers isolate at different layers**

These are common architecture distinctions, not a claim that one option universally provides sufficient security or reliability.

Author-created illustration. Numbers are illustrative unless stated otherwise.

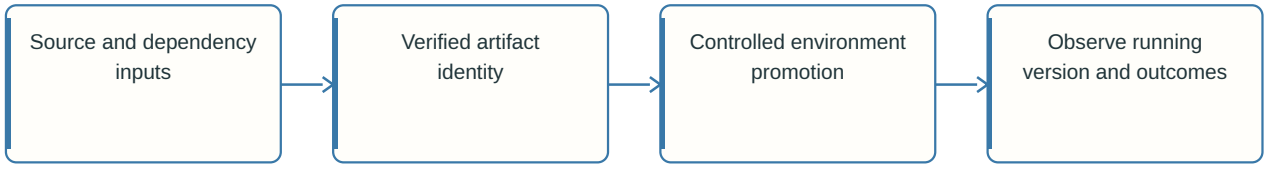


Arrows: ordered stages of this worked mechanism

FIG. 20.0.2 **Kubernetes cluster**

A simplified cluster control/data relationship; scheduling does not supply application data consistency.

Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 20.0.3 **CI/CD pipeline**

CI/CD preserves evidence and identity rather than merely copying files.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Stage	Blue	Green
Before switch	Serving existing version	Candidate verified
During switch	Drain under policy	Receives new traffic
Data	Shared or migrated state	Rollback compatibility required

FIG. 20.0.4 **blue-green deployment**

Parallel runtime environments do not automatically make shared schema changes reversible.

Author-created illustration. Numbers are illustrative unless stated otherwise.

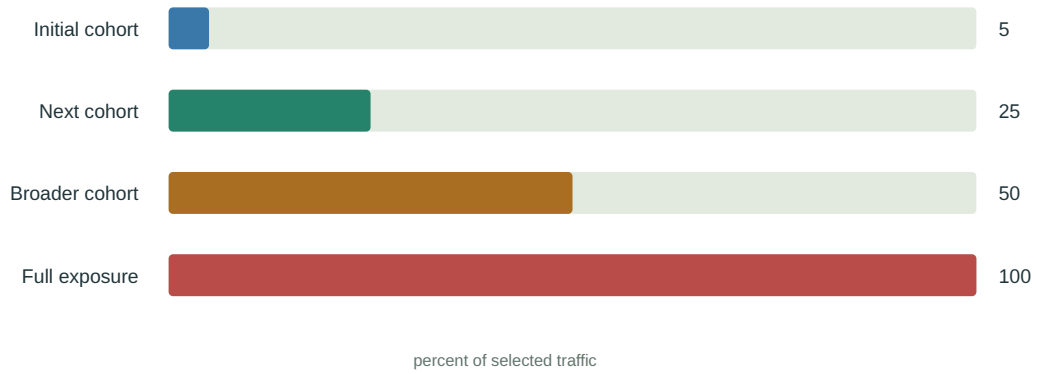


FIG. 20.0.5 **Canary exposure expands only after verification**

Percentages are an illustrative rollout plan, not an automatic safety rule. Each increase depends on meaningful representative checks and stop conditions.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Environments, VMs & containers

Prerequisites: Networking Foundations for System Design, Application Architecture and Domain-Driven Design, Reliability, Fault Tolerance, and Disaster Recovery, Security Architecture and Privacy, Observability, Monitoring, and Incident Engineering

4 original figures

Blue: normal flow · Green: healthy · Amber: caution · Red: failure · Purple: control / shared infrastructure · Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

The same image works locally and fails in production. Compare environments and runtime isolation, keeping persistent state, resources, and platform duties explicit.

20.1.1 · Development vs staging vs production

A service passes development tests but fails in production because its network permissions and connection limits differ. Development, staging, and production have different purposes, yet the relevant behaviours need deliberate compatibility.

Use staging to exercise protocols, identity, migrations, and failure paths with appropriate test data and secrets. Exact production scale may be impractical, so record what staging cannot establish. Separate environment values from business meaning. A local pass is evidence for that environment; it does not prove storage, network, or load behaviour elsewhere. Make the differences visible enough that release checks can address the important ones.

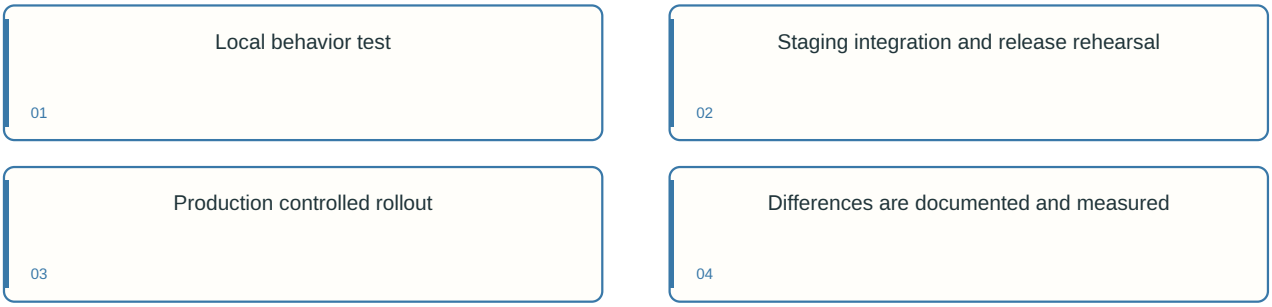


FIG. 20.1.1 **Development vs staging vs production · the example**

Environments have different purposes while relevant protocols, state transitions, and configuration meanings need deliberate compatibility.

Author-created illustration. Numbers are illustrative unless stated otherwise.

20.1.2 · Virtual machines vs containers

A virtual machine presents virtualised hardware and a guest operating system. A container commonly isolates processes while sharing the host kernel. Both provide useful boundaries, but their isolation and operating responsibilities differ.

Choose using workload, expertise, startup behaviour, and required separation. Images, identity, networking, storage, and resource limits need design in either case. Several containers on one host still share its failure and capacity. VM boundaries also sit on shared physical and administrative infrastructure. Identify the actual threat and failure model rather than treating either packaging label as complete isolation.

Layer	Virtual machine	Container
Operating environment	Guest OS per VM	Shared host kernel
Process resources	Guest-managed processes	Runtime constraints
Persistent state	Explicit attached owner	Not ephemeral writable layer

FIG. 20.1.2 **VMs and containers isolate at different layers**

These are common architecture distinctions, not a claim that one option universally provides sufficient security or reliability.

Author-created illustration. Numbers are illustrative unless stated otherwise.

20.1.3 · Container fundamentals

A container packages an executable environment. Runtime settings then restrict its CPU, memory, files, and network access. Packaging helps repeatability when image and configuration are controlled.

Required persistent data should not live only in an ephemeral writable layer that disappears on replacement. Resource limits can also throttle or terminate work, so preserve necessary progress and handle shutdown. Broad host mounts or privileged execution can weaken the intended isolation. Inspect the actual runtime permissions and persistence path. A reproducible image is one input to a predictable service, alongside configuration, state, and operating policy.

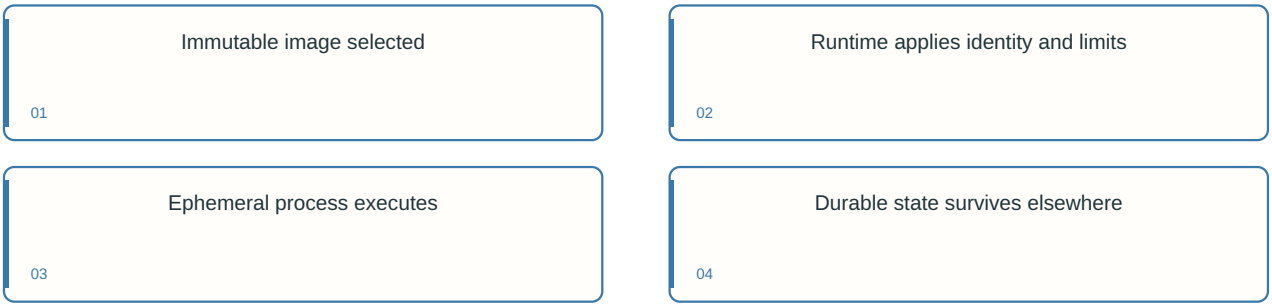


FIG. 20.1.3 **Container fundamentals · the example**

Containers package execution while runtime controls, persistent-state placement, identity, and shutdown still need design.

Author-created illustration. Numbers are illustrative unless stated otherwise.

20.1.4 · Kubernetes fundamentals

Kubernetes controllers compare declared objects with observed cluster state and work to reconcile differences. Pods, scheduling, services, and node agents provide primitives for running and locating workloads.

The platform does not infer an application's transaction invariants or create its tested backups and access model automatically. Identify which cluster responsibilities the team operates and which a provider manages. An orchestrator adds a control plane and a substantial configuration surface. If simpler deployment machinery meets the requirements, that can be a reasonable choice. Adopt platform facilities for their concrete benefits while retaining ownership of application behaviour and recovery.

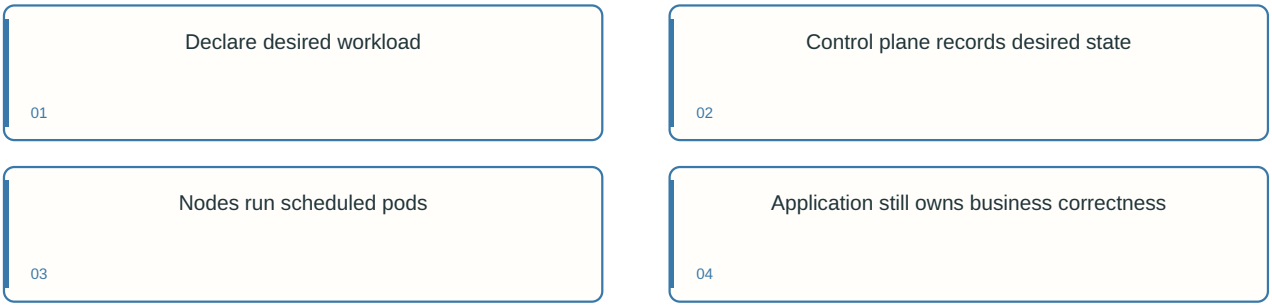


FIG. 20.1.4 **Kubernetes fundamentals · the example**

Kubernetes reconciles workload resources; application transactions, access, backups, and service objectives remain explicit responsibilities.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · How can rollback restore old code while leaving incompatible data or effects?

Old code can return while new schemas, values, provider calls, and published messages remain. Maintain compatibility during overlap and identify the point where contraction ends safe rollback. Some outcomes need forward repair or compensation. Restoring an artifact cannot reverse every state transition it previously performed.

Remember: Packaging improves repeatability only with controlled configuration and appropriate state ownership.

Try applying it: Which data would disappear if a container's writable layer were replaced?

Unit 20 exercises → · Unit 20 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. The Twelve-Factor App ↗

   Original methodology. Application deployment conventions.

   Access reviewed · 21 October 2026

2. Kubernetes components ↗

   Official documentation · accessed 21 Oct 2026. Platform reference for Unit 20.

   Access reviewed · 21 October 2026

3. Canarying Releases — The SRE Workbook ↗

   Engineering book. Representative rollout evaluation and stop conditions.

   Access reviewed · 21 October 2026

Control planes, scheduling & configuration

Prerequisites: 20.1 · Environments, VMs & containers 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

The scheduler cannot place new work, but existing requests still succeed. Separate control and serving paths, then examine startup, scaling, configuration, and replacement dependencies.

20.2.1 · Control plane vs data plane

The control plane decides placement and configuration. The data plane carries the running application's traffic. Under some conditions, a control-plane outage stops new scheduling while already running workloads continue.

Draw both paths. A request that unnecessarily performs a synchronous configuration lookup can turn a control-plane problem into a serving outage. Cached values need validity and update rules, particularly for permission changes. Recovery also may need a configuration or credential service before replacements can start. A redundant data path is useful only when its required startup and authority dependencies are understood.

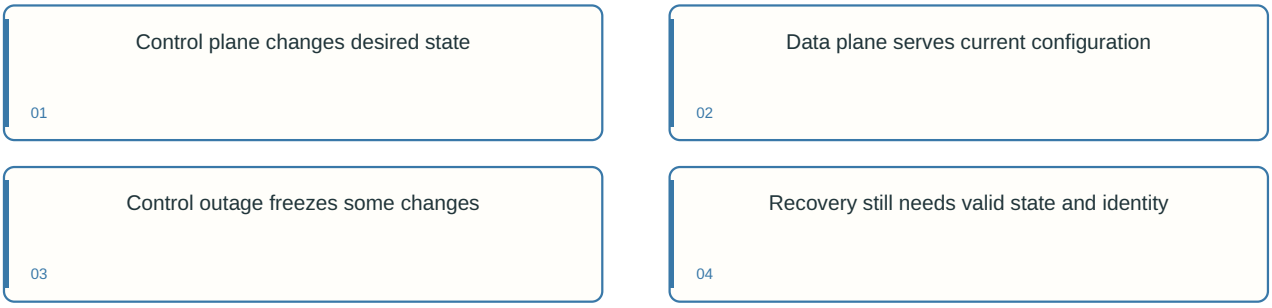


FIG. 20.2.1 **Control plane vs data plane · the example**

Control planes decide configuration and placement; data planes serve traffic with different startup and failure dependencies.

Author-created illustration. Numbers are illustrative unless stated otherwise.

20.2.2 · Scheduling and service discovery

A scheduler assigns a workload to eligible resources. Its decisions can consider capacity, affinity, anti-affinity, and failure domains. Service discovery later helps callers locate the running endpoints.

A scheduled process may still be starting and not ready for traffic. Discovery can lag both startup and termination, so clients need bounded handling of stale endpoints. Placement also says nothing about shared-data correctness: two healthy replicas can each point to an unrelated local database. State the storage and readiness contracts alongside the scheduling rule. Successful placement establishes where work runs, rather than that the whole application state is coordinated.

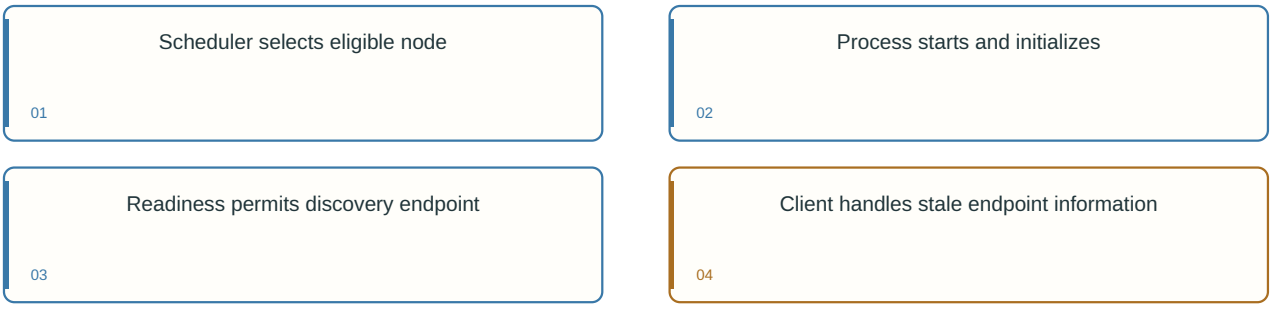


FIG. 20.2.2 **Scheduling and service discovery · the example**

Scheduling selects eligible resources; discovery advertises endpoints whose readiness and shared-state behaviour still need handling.

Author-created illustration. Numbers are illustrative unless stated otherwise.

20.2.3 · Autoscaling

Scaling up creates more workers, but each may open additional database connections. If the database is already saturated, the extra concurrency can worsen the queue. Scale useful capacity while respecting downstream limits.

Choose signals appropriate to the workload and include warmup, minimum capacity, and maximum spend. Scaling down needs draining: stop accepting new work, finish or transfer durable jobs, and release resources. Killing an unrecorded background task loses it. Autoscaling during retries also needs care, because attempt traffic can rise while useful demand is unchanged. The platform and application admission rules should act together.

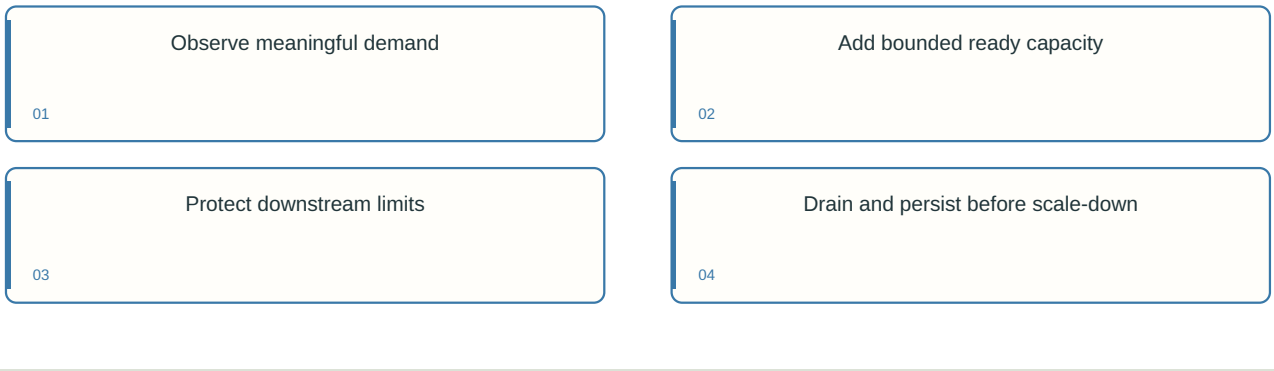


FIG. 20.2.3 **Autoscaling · the example**

Autoscaling must include useful signals, startup, draining, durable work, downstream limits, and spending bounds.

Author-created illustration. Numbers are illustrative unless stated otherwise.

20.2.4 · Configuration and secret delivery

A bad configuration can disable every otherwise healthy replica without changing application code. Treat configuration delivery as a versioned, validated operation with a recovery path.

Roll out risky changes in a controlled way and record the effective version in telemetry. Some settings can reload safely; others require replacing connections or processes. Secrets need stricter access and logging controls than ordinary values. Verify that the application actually uses the intended version. This gives an incident investigation something concrete to compare with the time the behaviour changed.

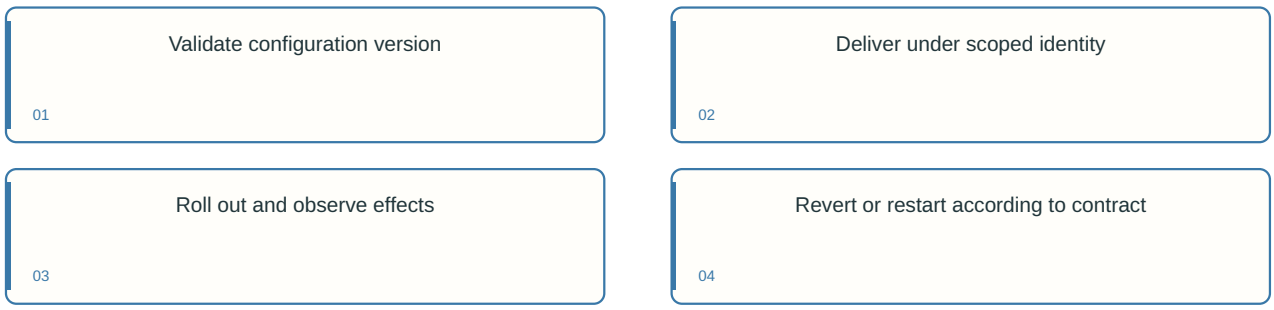


FIG. 20.2.4 **Configuration and secret delivery · the example**

Configuration and secret delivery need validation, versions, rollout/reload semantics, and a recovery path.

Author-created illustration. Numbers are illustrative unless stated otherwise.

20.2.5 · Immutable infrastructure

Instead of manually changing a server until it works, immutable infrastructure replaces it from a defined artifact and configuration. Repeatability improves when provisioning and inputs are controlled.

Runtime data still changes, and rollback still depends on compatible state. Keep persistence and migrations explicit. If an emergency edit is needed, capture it in the maintained configuration so replacement does not lose it or recreate drift. Check the running artifact and values as well as the deployment's intention. Rebuilding the same machine definition does not automatically restore its lost data or reverse an external effect.

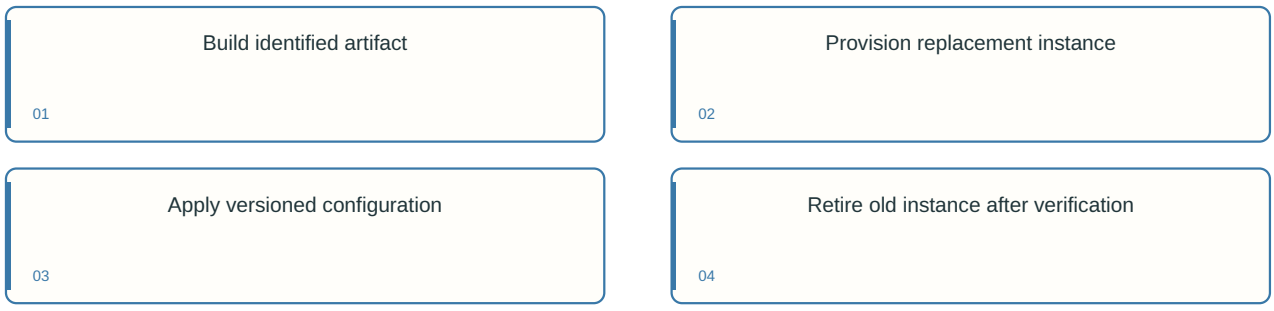


FIG. 20.2.5 **Immutable infrastructure · the example**

Immutable infrastructure replaces defined instances, while runtime state, migrations, and emergency drift remain managed concerns.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · How can rollback restore old code while leaving incompatible data or effects?

Old code can return while new schemas, values, provider calls, and published messages remain. Maintain compatibility during overlap and identify the point where contraction ends safe rollback. Some outcomes need forward repair or compensation. Restoring an artifact cannot reverse every state transition it previously performed.

Remember: Running traffic and creating replacement capacity can depend on different paths.

Try applying it: Which dependency could let current instances serve while preventing every replacement from starting?

Unit 20 exercises → · Unit 20 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. The Twelve-Factor App ↗

   Original methodology. Application deployment conventions.

   Access reviewed · 21 October 2026

2. Kubernetes components ↗

   Official documentation · accessed 21 Oct 2026. Platform reference for Unit 20.

   Access reviewed · 21 October 2026

3. Canarying Releases — The SRE Workbook ↗

   Engineering book. Representative rollout evaluation and stop conditions.

   Access reviewed · 21 October 2026

Releases, flags & schema migrations

Prerequisites: 20.2 · Control planes, scheduling & configuration 6 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A release writes a new field, then a rollback restores a reader that cannot understand it. Keep artifacts, flags, configuration, and migration stages compatible while versions overlap.

20.3.1 · Infrastructure as Code

Infrastructure as Code makes resource definitions and policies reviewable and repeatable. A change plan can reveal proposed replacements, but the real effect depends on provider behaviour, stored state, privileges, and drift.

A resource replacement can be destructive for a stateful service. Understand lifecycle and recovery before applying it. Coordinate concurrent updates and protect secrets outside ordinary tracked values. Verify the resulting state. Also preserve the state information and credentials needed to recover the infrastructure definition itself. Code is a useful source of desired configuration, while restoration remains an operating procedure with dependencies.

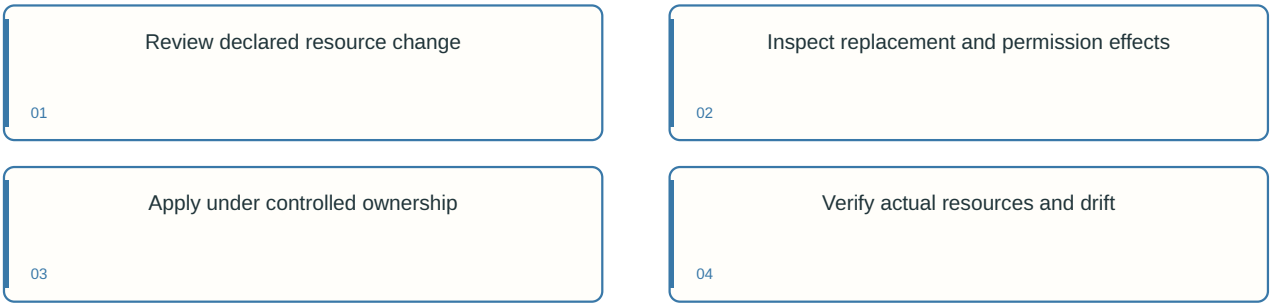


FIG. 20.3.1 **Infrastructure as Code · the example**

Infrastructure as Code makes desired resources reviewable without eliminating state, privileges, drift, or destructive-change risks.

Author-created illustration. Numbers are illustrative unless stated otherwise.

20.3.2 · CI/CD

Continuous integration checks changes. Delivery prepares a releasable artifact, and deployment places that artifact into an environment. Preserve identity from the source through the built artifact to the version actually running.

Use scoped pipeline permissions and meaningful checks for the release risks. A test in one environment may miss mixed-version traffic during a database migration. Keep untrusted build input away from release secrets. Automation reduces manual variation, but its own partial failures need a supported recovery path. A green pipeline is evidence of its defined checks, rather than proof of every production assumption.

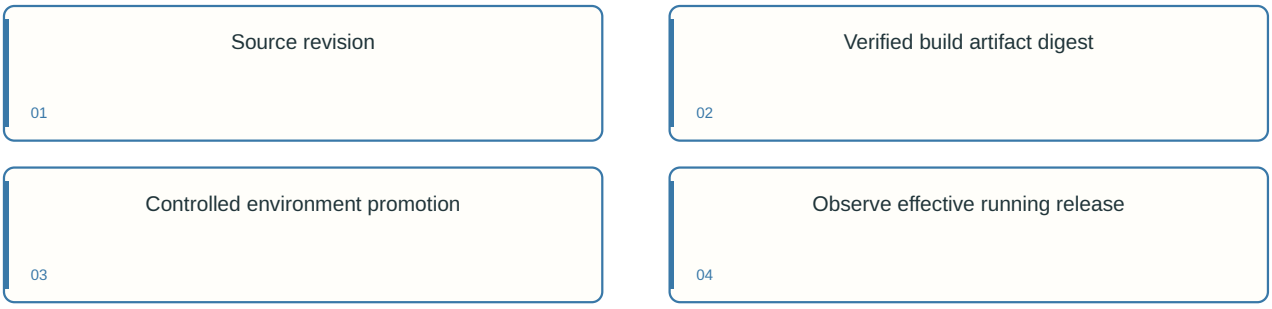


FIG. 20.3.2 **CI/CD · the example**

CI, delivery, and deployment preserve artifact identity and run checks appropriate to actual release risks.

Author-created illustration. Numbers are illustrative unless stated otherwise.

20.3.3 · Rolling, blue-green, and canary deployments

Rolling deployment replaces instances gradually. Blue-green prepares another environment before switching traffic. Canary sends a controlled traffic subset to the new version before wider exposure.

Each needs compatible state, draining, spare capacity, and verification. A canary receiving only cheap requests can miss an expensive-path regression. Choose cohorts and stop conditions using the meaningful outcomes. Blue-green can duplicate application capacity while both versions still share one mutable database; switching traffic back does not reverse its changes. Select the release arrangement according to the risk and operating cost, including the data transition.

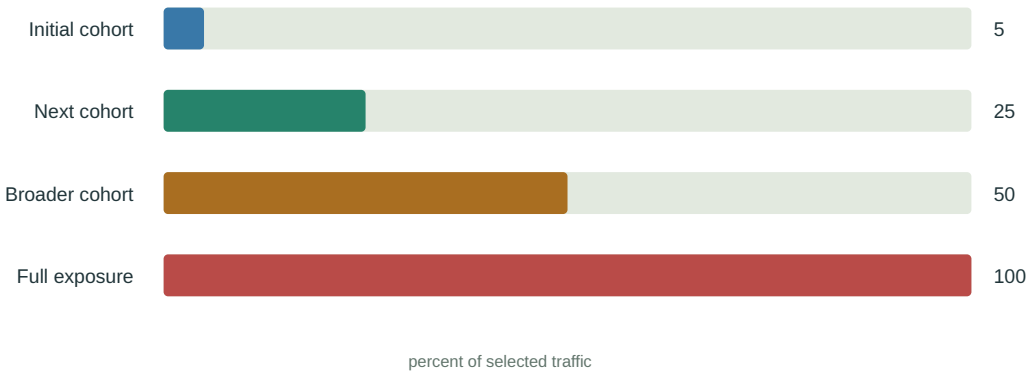


FIG. 20.3.3 **Canary exposure expands only after verification**

Percentages are an illustrative rollout plan, not an automatic safety rule. Each increase depends on meaningful representative checks and stop conditions.

Author-created illustration. Numbers are illustrative unless stated otherwise.

20.3.4 · Feature flags

A feature flag can release code separately from exposing a behaviour. Give the flag an owner, scope, safe default, observable state, and removal plan. Combinations of flags also create combinations to test.

The flag is not authorisation. A browser-controlled setting must not grant a privileged server operation. Consider disabling after the feature has written a new data format: older code may still need to understand it. Keep state compatibility and migration separate from the exposure switch. Flags offer control over some transitions, while stale flags and hidden configuration can accumulate their own complexity.

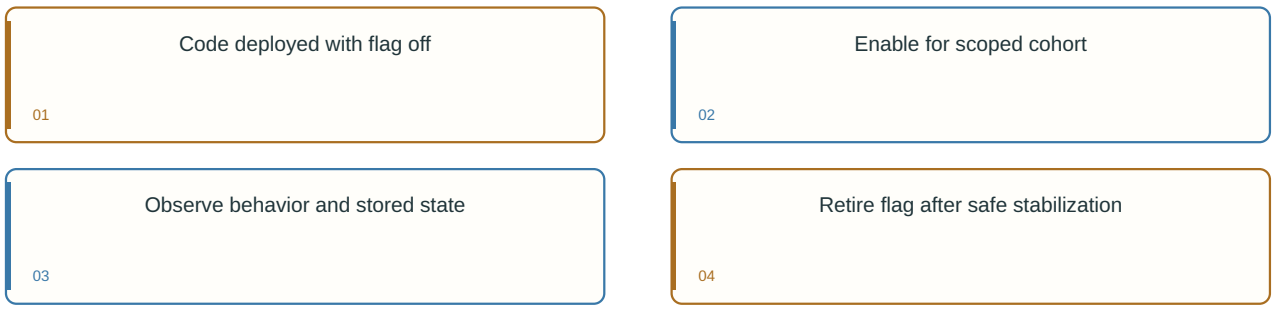


FIG. 20.3.4 **Feature flags · the example**

Feature flags control exposure with ownership and safe defaults, but do not reverse stored effects or replace permission.

Author-created illustration. Numbers are illustrative unless stated otherwise.

20.3.5 · Rollbacks and database migrations

A release removes a database column. Rolling back its code now restores a reader that needs the missing column. The old artifact exists, but the state is no longer compatible with it.

Expand-and-contract avoids this abrupt transition. Add compatible structures, deploy code that tolerates the overlap, backfill with bounded resumable work, and verify. Remove old structures only after old readers are gone. Mark when rollback stops being supported. Code, configuration, schema, and external effects have different reversal rules, so a rollback procedure must name the assumptions it relies on.

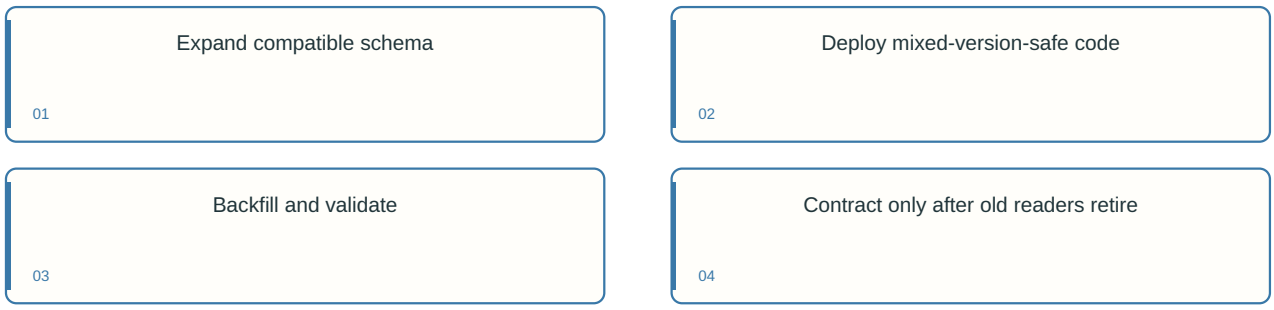


FIG. 20.3.5 **Rollbacks and database migrations · the example**

Rollback depends on compatible state; expand-and-contract maintains a supported overlap before removing old structures.

Author-created illustration. Numbers are illustrative unless stated otherwise.

20.3.6 · Multi-environment configuration

Production uses different endpoints and credentials from staging. Those are environment values. The meaning of a payment amount should remain the same unless a deliberate business policy says otherwise.

Validate required fields and types at startup or controlled reload. Use explicit versions and careful defaults for security and retention. An absent value should fail clearly when guessing could route data to the wrong region or tenant. Avoid scattered environment-specific conditionals that no test setup reproduces. Keep the effective configuration understandable so differences between environments are intentional rather than accidental business semantics.

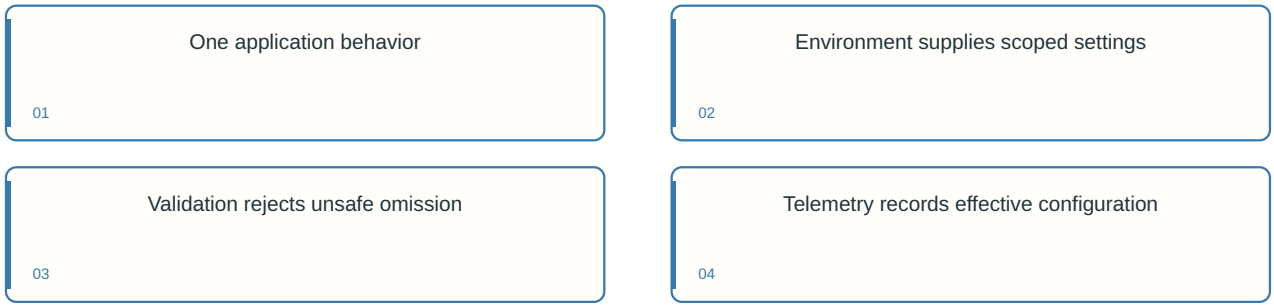


FIG. 20.3.6 **Multi-environment configuration · the example**

Environment values should be validated and explicit without silently changing business meanings.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · How can rollback restore old code while leaving incompatible data or effects?

Old code can return while new schemas, values, provider calls, and published messages remain. Maintain compatibility during overlap and identify the point where contraction ends safe rollback. Some outcomes need forward repair or compensation. Restoring an artifact cannot reverse every state transition it previously performed.

Remember: Release safety includes data and protocol transitions, not only selecting an old executable.

Try applying it: At what point in expand-and-contract does rollback to the old reader become unsafe?

Unit 20 exercises → · Unit 20 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. The Twelve-Factor App ↗

   Original methodology. Application deployment conventions.

   Access reviewed · 21 October 2026

2. Kubernetes components ↗

   Official documentation · accessed 21 Oct 2026. Platform reference for Unit 20.

   Access reviewed · 21 October 2026

3. Canarying Releases — The SRE Workbook ↗

   Engineering book. Representative rollout evaluation and stop conditions.

   Access reviewed · 21 October 2026

Platforms, serverless & ownership

Prerequisites: 20.3 · Releases, flags & schema migrations 6 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A managed platform offers useful capabilities while leaving application choices in your hands. Compare delegated work, limits, topologies, and internal developer needs under the same service requirements.

20.4.1 · Twelve-Factor App principles

The Twelve-Factor App describes operational practices such as explicit dependencies and configuration, backing-service treatment, process discipline, and logs as event streams. These ideas can improve repeatability for suitable service applications.

They are methodology rather than a universal proof of quality. Stateful work and current platforms still need explicit persistence, identity, limits, and recovery. Apply each practice's purpose to the actual workload. Checking twelve labels does not establish authorisation, transaction correctness, or usable backups. A design remains accountable to its requirements and observed behaviour, even when it follows a well-known set of conventions.

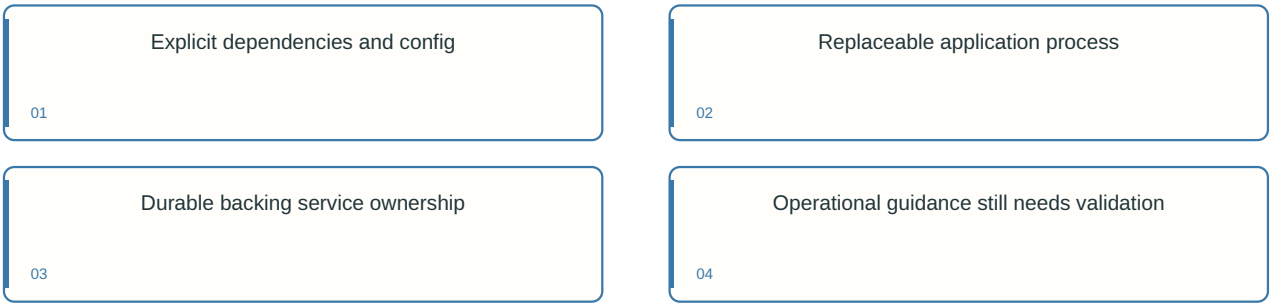


FIG. 20.4.1 **Twelve-Factor App principles · the example**

Twelve-Factor practices guide repeatable service operation while workload-specific correctness and recovery remain required.

Author-created illustration. Numbers are illustrative unless stated otherwise.

20.4.2 · Serverless compute

A function platform manages some provisioning and invocation work. An activity can still time out, restart, or be invoked again. Rapid scaling may also create too many concurrent calls to one database.

Externalise required state, bound downstream concurrency, and use operation identities for effects. Compare startup delay, sustained cost, portability, and remaining operating work with container or VM alternatives. The application still needs monitoring and secure access. “Serverless” describes a delegation arrangement, not unlimited capacity or absence of failures. Use it where its execution model suits the work and its recovery requirements.

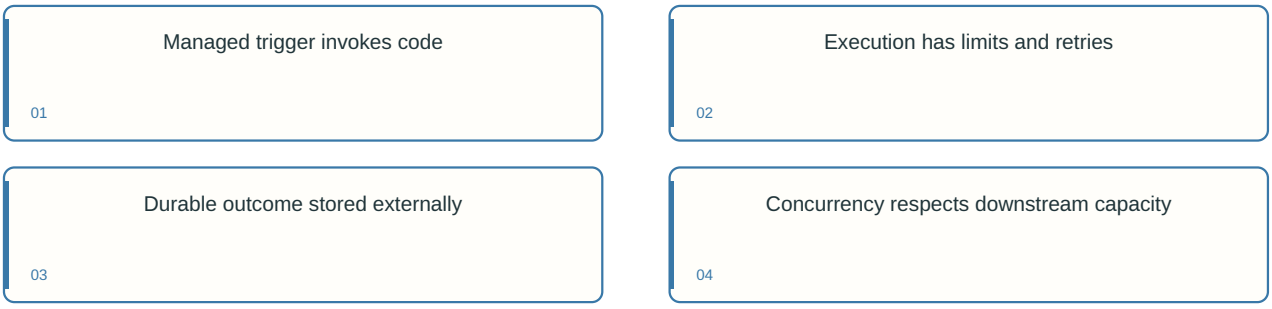


FIG. 20.4.2 **Serverless compute · the example**

Serverless delegates selected platform duties while retaining limits, durable state, identity, and safe retries.

Author-created illustration. Numbers are illustrative unless stated otherwise.

20.4.3 · Cloud vs on-premises trade-offs

Cloud and on-premises operation place responsibilities in different hands. Compare procurement delay, expertise, networking, data placement, managed capabilities, recovery, and exit costs for the same workload.

Cloud capacity remains subject to quotas and provisioning time. Predictable owned capacity can be efficient in some circumstances but requires equipment and disaster management. Avoid generic cheapest-provider claims without current rates and full assumptions. The atlas uses hypothetical cost units for arithmetic; a procurement choice needs actual quotations and organisational requirements. The useful comparison is the complete operating arrangement rather than one machine price.

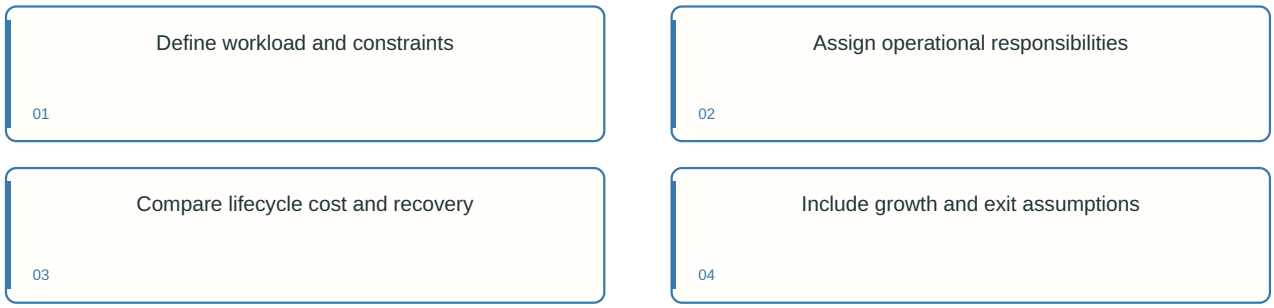


FIG. 20.4.3 **Cloud vs on-premises trade-offs · the example**

Cloud/on-premises comparisons need equal workload, ownership, procurement, recovery, and verified cost assumptions.

Author-created illustration. Numbers are illustrative unless stated otherwise.

20.4.4 · Infrastructure ownership and managed services

A provider manages database patching and replication. The application team still chooses data access, schema, acknowledgment settings, retention, and how recovery is validated. Managed service means selected responsibilities have moved, not that business responsibility has disappeared.

Read the actual contract and failure boundaries. Self-management offers control while requiring expertise and on-call work. In either arrangement, test client behaviour during maintenance, throttling, failover, and credential changes. List what the team still owns. That makes cost and staffing comparisons more accurate than describing an entire system as simply managed or self-hosted.

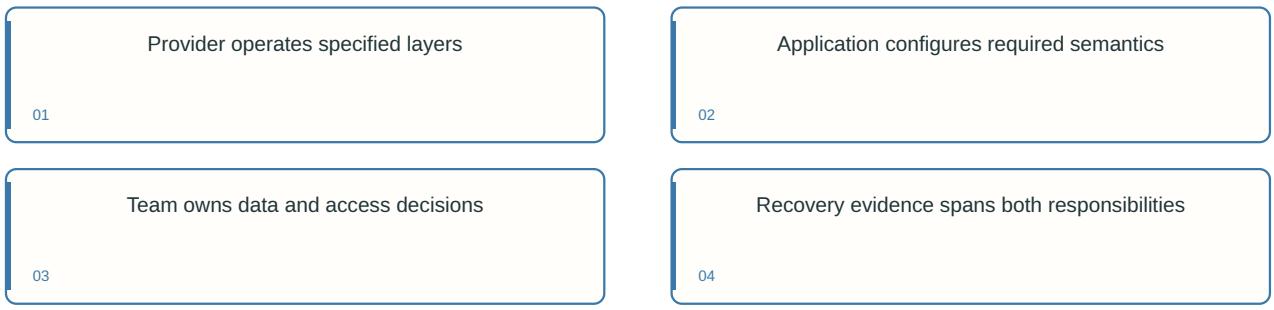


FIG. 20.4.4 **Infrastructure ownership and managed services · the example**

Managed services transfer selected duties while clients retain responsibilities defined by their actual contract.

Author-created illustration. Numbers are illustrative unless stated otherwise.

20.4.5 · Deployment topologies and networking

A module diagram shows application responsibilities. A deployment topology shows their processes, machines, zones, networks, data, and trust boundaries. It reveals whether two replicas share the same host or storage dependency.

Include relevant ingress, egress, encryption termination, private routes, and operating access. Follow the loss of one placement: how does a replacement obtain its configuration and state? Label whether arrows represent traffic, control, or another dependency. The purpose is to explain actual communication and failure relationships, rather than redraw logical boxes with infrastructure logos.

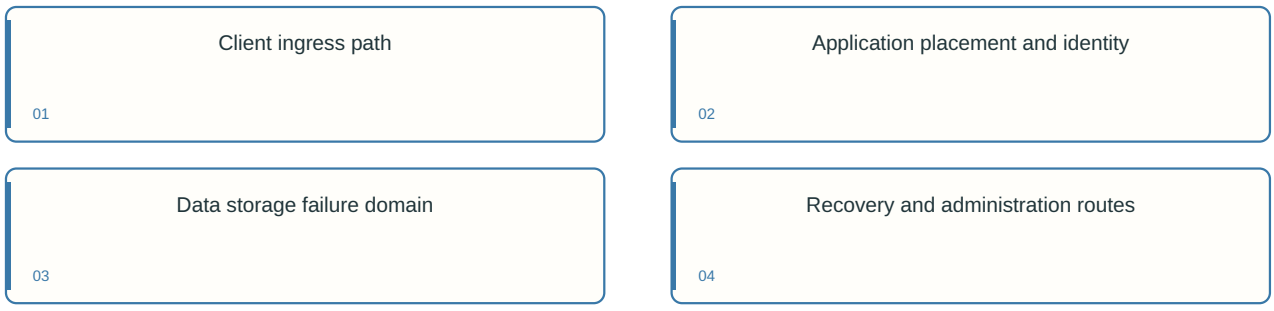


FIG. 20.4.5 **Deployment topologies and networking · the example**

Deployment topology places execution, state, routes, trust, and failure domains on real infrastructure.

Author-created illustration. Numbers are illustrative unless stated otherwise.

20.4.6 · Platform engineering basics

A platform team can offer a reusable way to build artifacts, obtain identity, emit telemetry, and deploy with tested defaults. Platform engineering treats development teams as users of those capabilities.

Provide understandable limits, ownership, and supported escape paths for different workloads. An opaque mandatory framework can hide decisions that application teams still need to make. Measure useful adoption, reliability, and operating burden rather than only the number of internal tools created. A platform abstraction earns its place when it reduces repeated work while exposing the failure and resource behaviour relevant to its users.

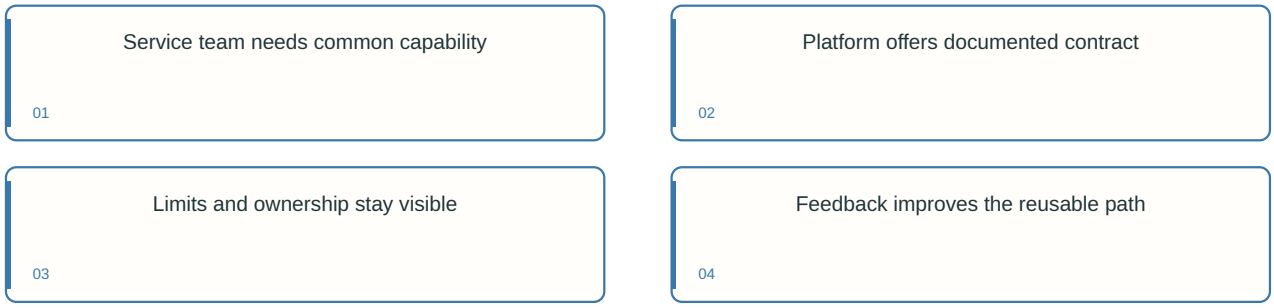


FIG. 20.4.6 **Platform engineering basics · the example**

Platform engineering offers reusable operating capabilities whose limits and ownership remain visible to developer users.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Try the mechanism in code

PSEUDOCODE · SAFE RELEASE PROGRESSION

```
add_compatible_schema_field()
deploy_mixed_version_safe_code()
backfill_with_bounded_progress()
verify_old_and_new_readers()
retire_old_readers()
remove_obsolete_schema_only_when_safe()
```

Old and new readers coexist during compatible expansion and backfill. Retire old readers before removing the structures they need.

Rollback is safe only while its state and protocol assumptions still hold. Destructive contraction or external effects may require forward repair. Verify the actual running artifact and configuration too.

Download the executable mechanism collection ↓ · Follow the connected complete design →

▼ Think further · How can rollback restore old code while leaving incompatible data or effects?

Old code can return while new schemas, values, provider calls, and published messages remain. Maintain compatibility during overlap and identify the point where contraction ends safe rollback. Some outcomes need forward repair or compensation. Restoring an artifact cannot reverse every state transition it previously performed.

Remember: A platform abstraction should simplify repeated work while exposing the limits its users must manage.

Try applying it: Which correctness and recovery responsibilities remain with a team using a managed database?

Unit 20 exercises → · Unit 20 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. The Twelve-Factor App ↗

   Original methodology. Application deployment conventions.

   `Access reviewed · 21 October 2026`

2. Kubernetes components ↗

   Official documentation · accessed 21 Oct 2026. Platform reference for Unit 20.

   `Access reviewed · 21 October 2026`

3. Canarying Releases — The SRE Workbook ↗

   Engineering book. Representative rollout evaluation and stop conditions.

   `Access reviewed · 21 October 2026`

PART 5 / UNIT 21

Performance, Cost Optimization, and System Validation

A service misses its response-time goal. Budget the path, locate the limit, and design a benchmark that can test the proposed explanation without hiding waiting or errors.

The running example

Compare three production architectures for a service with variable traffic. Model monthly costs, p99 latency, availability, and operational complexity under explicit assumptions.

Unit 21 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

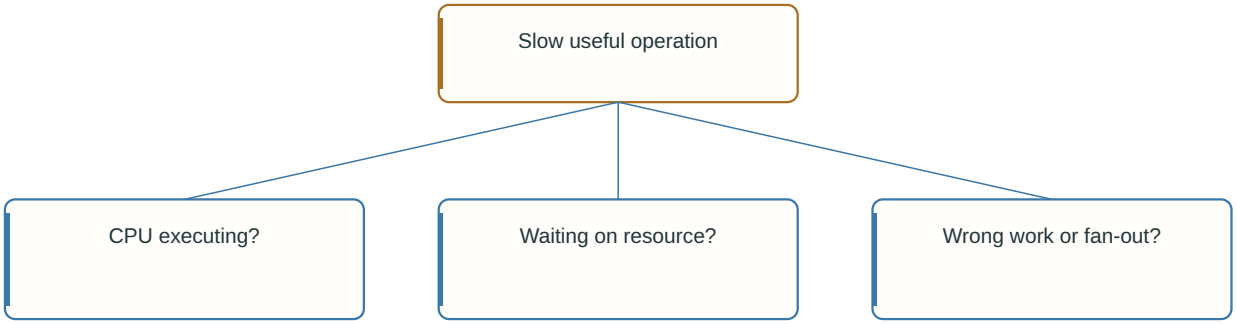


FIG. 21.0.1 **Performance bottleneck tree**

Start with a symptom, distinguish execution from waiting, then inspect the resource on the critical path.

Author-created illustration. Numbers are illustrative unless stated otherwise.

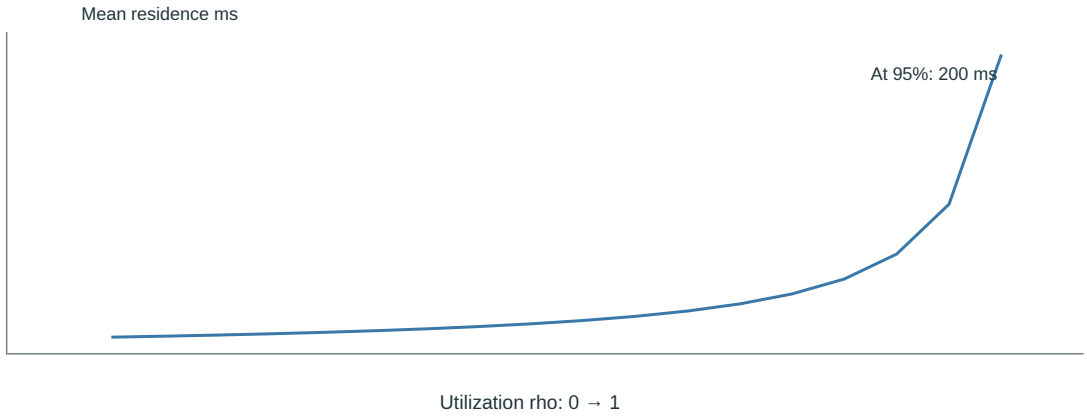


FIG. 21.0.2 **throughput-latency curve**

In this single-server M/M/1 model, mean waiting plus service time grows near full utilization. The model assumes Poisson arrivals and independent exponential service times. Measure real workloads and p99 separately.

Author-created illustration. Numbers are illustrative unless stated otherwise.

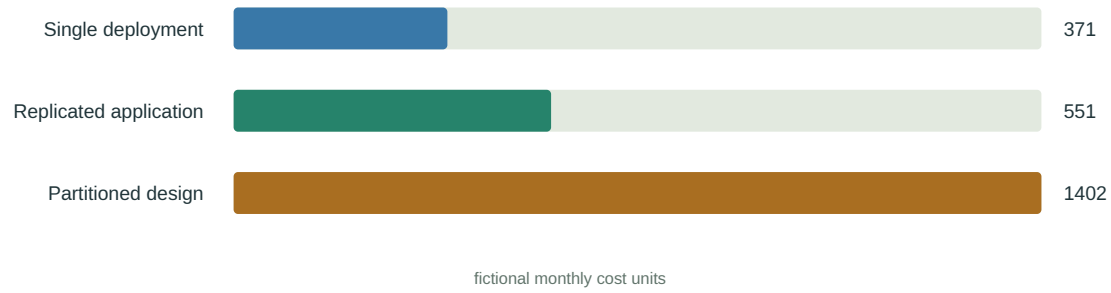


FIG. 21.0.3 **Architecture costs are not capacity guarantees**

The complete cost worksheet uses fictional monthly units. Each candidate still needs independent checks for correctness, performance, and failure requirements.

Author-created illustration. Numbers are illustrative unless stated otherwise.

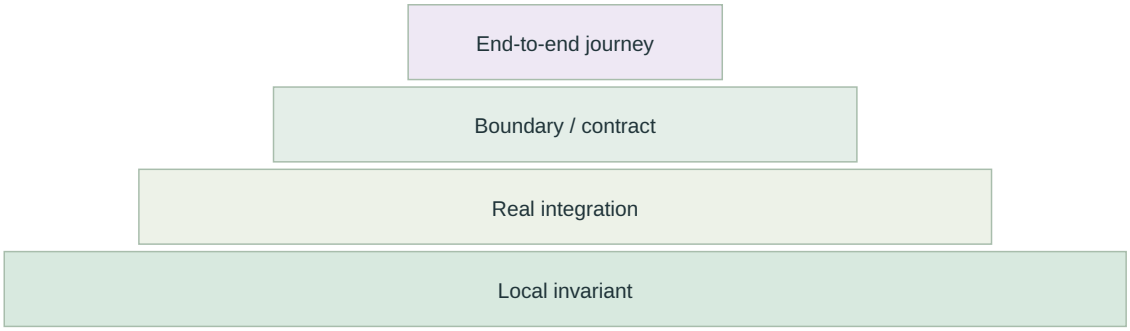


FIG. 21.0.4 **test pyramid**

The conceptual pyramid distinguishes feedback scope and cost, not a mandatory numerical ratio of tests.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Criterion	Hard constraint?	Evidence
No duplicate local posting	Yes	Transactional invariant test
p99 target	Required objective	Representative load distribution
Recovery target	Required objective	Restore/failover drill
Monthly cost	Preference or budget	Explicit unit-cost model

FIG. 21.0.5 **design decision matrix**

Feasibility precedes preference scoring. An unmeasured objective must not be reported as achieved.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Budgets, bottlenecks & honest benchmarks

Prerequisites: Requirements, Estimation, and Engineering Trade-offs, Runtime, Processes, Threads, and Concurrency, Database Internals, Transactions, and Concurrency Control, Scalability, Load Balancing, and Data Distribution, Reliability, Fault Tolerance, and Disaster Recovery, Observability, Monitoring, and Incident Engineering, Deployment, Cloud Infrastructure, and Platform Architecture

6 original figures

Blue: normal flow · Green: healthy · Amber: caution · Red: failure · Purple: control / shared infrastructure · Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A service misses its response-time goal. Budget the path, locate the limit, and design a benchmark that can test the proposed explanation without hiding waiting or errors.

21.1.1 · Performance budgets

Assume a request has a 200 ms budget: 30 ms for edge and network, 20 ms for queues, 100 ms for data access, and 50 ms for computation and response. These allocations add to the end-to-end target.

A performance budget makes the expected costs visible before implementation. Validate the assumed allocations with measurements under the stated workload. Include connection waits, encoding, tails, and failure paths, rather than budgeting only the main query. Work that exhausts the budget before reaching the query still matters to the user. A proposed allocation is an objective to check, not a measured result.

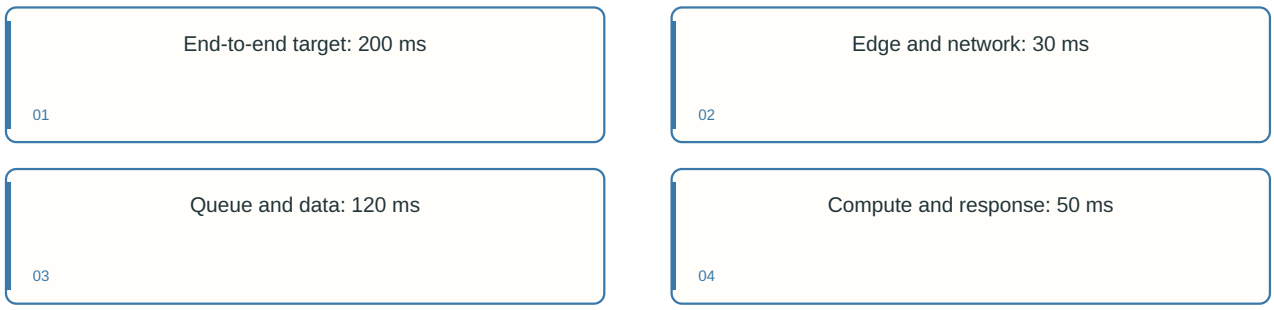


FIG. 21.1.1 **Performance budgets · the example**

Performance budgets allocate acceptable end-to-end costs under a workload and remain targets until measured.

Author-created illustration. Numbers are illustrative unless stated otherwise.

21.1.2 · CPU, memory, disk, network, and database bottlenecks

A slow request may compute heavily, wait on disk, wait for a lock, or queue for a database connection. High CPU suggests one set of hypotheses; low CPU does not rule out a severe delay.

Trace the critical path and resource queues. Memory pressure can cause allocation stalls or termination, while transfer volume can saturate a network. A bottleneck tree organises possible explanations so measurements can distinguish them. Change the component implicated by the evidence rather than adjusting everything at once. An improvement can reveal another limit, so repeat the observation after the change.

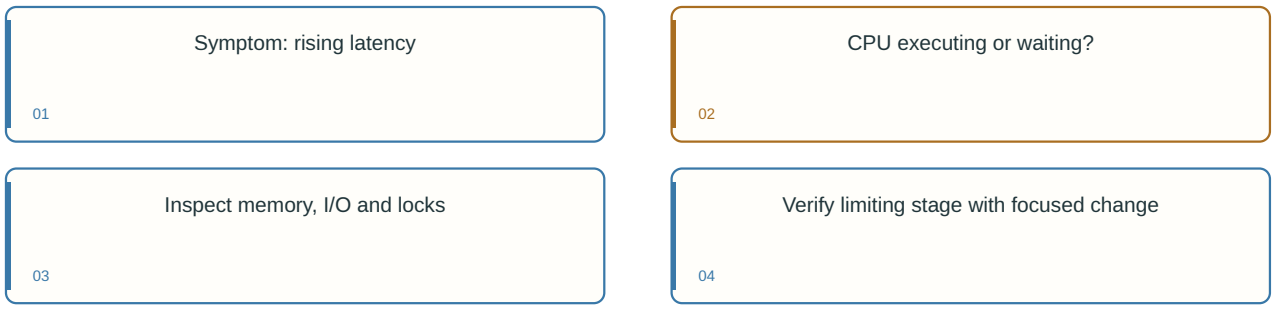


FIG. 21.1.2 **CPU, memory, disk, network, and database bottlenecks · the example**

Bottlenecks can be computation or waiting at CPU, memory, storage, network, locks, or data services.

Author-created illustration. Numbers are illustrative unless stated otherwise.

21.1.3 · Profiling and benchmarking

A profile shows where runtime work occurs. A benchmark compares behaviour under chosen conditions. To make a result useful, record the workload, data distribution, capacity, versions, warmup, concurrency, duration, and success criteria.

A fast isolated function does not establish application throughput. Include variability, failures, and resource cost, and compare equivalent service promises. Preserve enough setup to reproduce a change in behaviour after a dependency or schema update. The value of a benchmark is the question it answers under its conditions, rather than the size of its most impressive number.

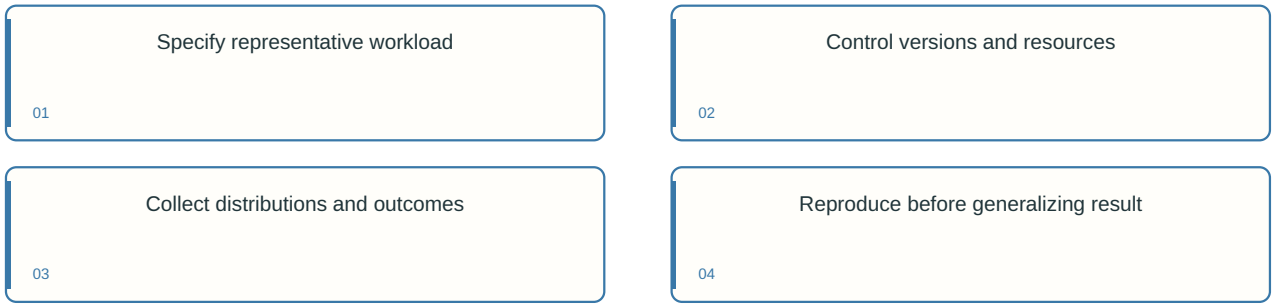


FIG. 21.1.3 **Profiling and benchmarking · the example**

Profiles locate runtime work; benchmarks compare controlled conditions with disclosed workloads, versions, variation, and outcomes.

Author-created illustration. Numbers are illustrative unless stated otherwise.

21.1.4 · Load, stress, spike, and soak testing

A load test checks expected demand. A stress test explores limits. A spike test examines abrupt changes, while a soak test looks for long-duration effects such as leaks or compaction backlog.

Choose the test for the uncertainty. Also inspect the generator: one that waits for each slow reply may stop issuing traffic that real users would continue to send. Check its own capacity and record errors, abandonment, waiting, and correct business results. High request throughput with lost reservations is a failed correctness test. Each measurement needs a workload and an outcome definition that match the claim being investigated.

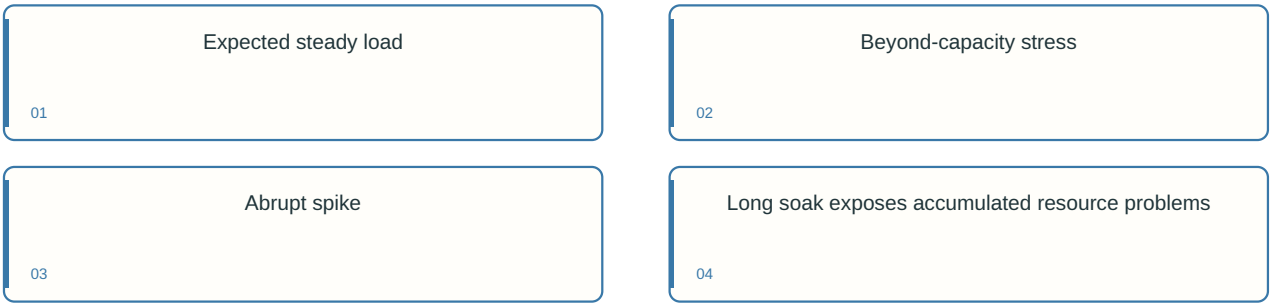


FIG. 21.1.4 **Load, stress, spike, and soak testing · the example**

Load, stress, spike, and soak tests answer different expected-load, limit, abrupt-change, and long-duration questions.

Author-created illustration. Numbers are illustrative unless stated otherwise.

21.1.5 · Benchmark methodology and statistical pitfalls

A generator waits for a slow response before sending the next request, although real arrivals would continue independently. The test omits some waiting that users would experience. This is a common form of coordinated omission.

Other pitfalls include tiny samples, unrealistic data, undisclosed warm caches, omitted failures, and incompatible versions. Report the method and distributions with appropriate precision. Change one factor or use a justified experimental design. A result supports the tested conditions; a different traffic mix, failure state, or data distribution may produce another outcome. Preserve the context rather than presenting the number as a general property of the system.

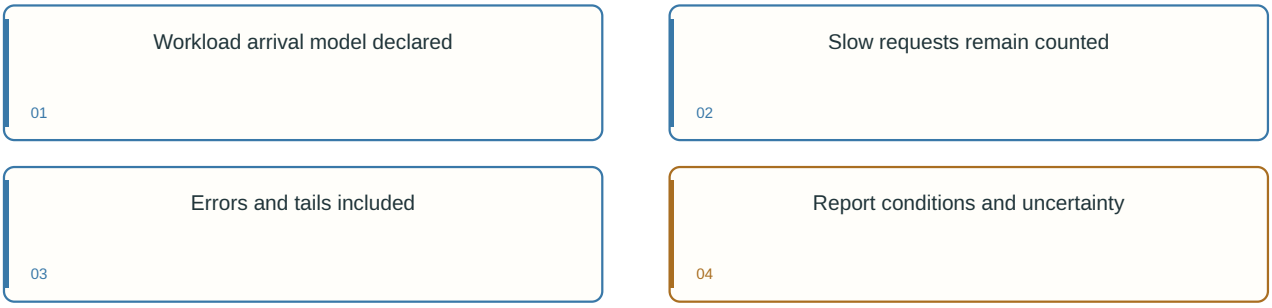


FIG. 21.1.5 **Benchmark methodology and statistical pitfalls · the example**

Benchmark validity depends on realistic input, sufficient observations, honest failures, and a generator that represents arrivals correctly.

Author-created illustration. Numbers are illustrative unless stated otherwise.

21.1.6 · Tail latency analysis

A service's p99 rises from 200 ms to five seconds. First ask whether a rare expensive operation changed or whether many operations slowed. Split queueing, execution, network, and required fan-out.

Use population distributions and representative traces together. Averaging individual machines' percentiles does not reconstruct a fleet percentile, and very high percentiles need enough observations to estimate usefully. One slow trace suggests a path to investigate, not a universal explanation. Compare normal and slow cases under the same workload. Tail analysis connects the distribution to its causes instead of assuming the mean describes all users.

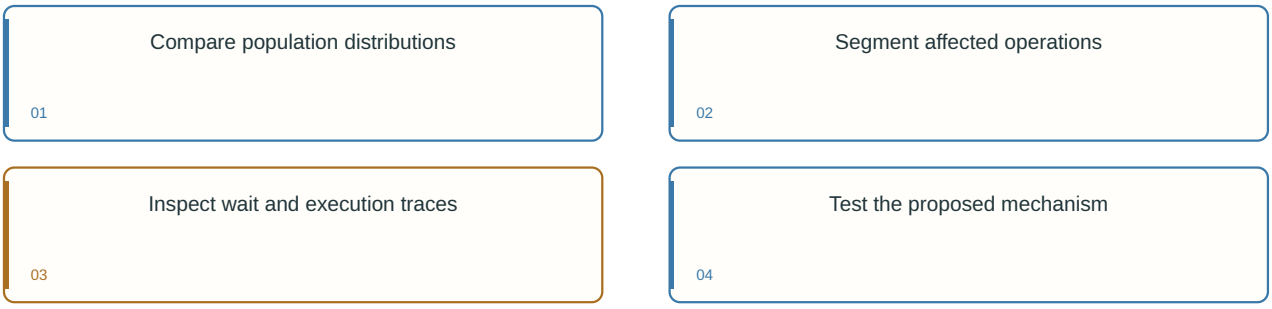


FIG. 21.1.6 **Tail latency analysis · the example**

Tail analysis connects timing distributions and representative paths without incorrectly averaging percentile summaries.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · When could a more expensive architecture be simpler to operate overall?

Compare total work for people as well as the invoice. A managed service may reduce patching, backup, or recovery burden while adding charges and leaving application responsibilities. Evaluate the same correctness and service goals with explicit staffing assumptions. Choose the design the team can understand, change, and recover reliably.

Remember: Performance evidence is meaningful only with its workload, conditions, and outcome definition.

Try applying it: How could a load generator accidentally omit the very delays users would experience?

Unit 21 exercises → · Unit 21 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. The Practical Test Pyramid — Ham Vocke ↗
   Original engineering article. Test scope and feedback trade-offs, not a mandatory test-count ratio.
   `Access reviewed · 21 October 2026`

2. Handling Overload — Google SRE ↗
   Engineering book. Admission, capacity, and overload response.
   `Access reviewed · 21 October 2026`

3. The Tail at Scale — Dean and Barroso, 2013 ↗
   Original research. Tail-latency considerations; atlas probabilities and latency examples are author-calculated.
   `Access reviewed · 21 October 2026`

Capacity, architecture & cost per outcome

Prerequisites: 21.1 · Budgets, bottlenecks & honest benchmarks 6 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Extra nodes, caches, and transfer can change both capacity and cost. Compare complete architectures using explicit inputs and useful outcomes rather than one attractive infrastructure rate.

21.2.1 · Queue saturation and throughput ceilings

As offered traffic rises, useful completions eventually stop increasing. Extra work then waits or is rejected. A queue stores that mismatch; it supplies no additional service capacity.

Observe successful outcomes, latency, and saturation to locate the throughput knee, where additional demand sharply worsens experience. Reserve margin for variability, background work, maintenance, and the failures the system promises to tolerate. A short benchmark maximum is not automatically a safe production target. State which amount of useful traffic remains supportable after a node or zone is lost, rather than sizing only the healthy fleet.

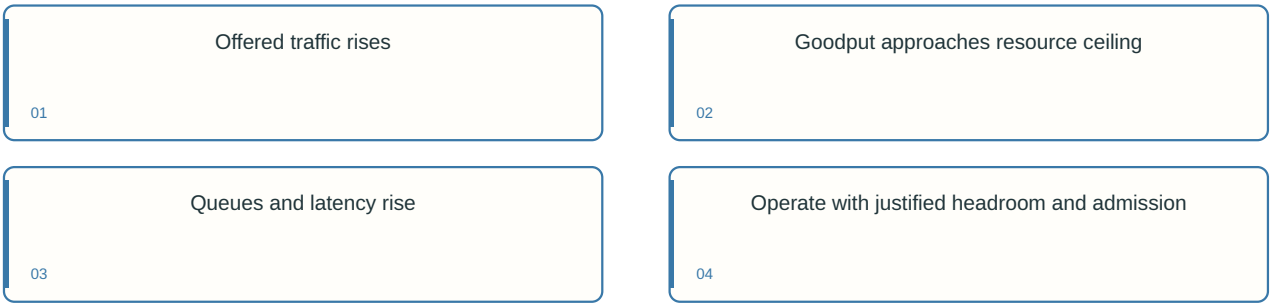


FIG. 21.2.1 **Queue saturation and throughput ceilings · the example**

Saturation limits useful completion; added arrivals become waiting or rejection, requiring operating margin.

Author-created illustration. Numbers are illustrative unless stated otherwise.

21.2.2 · Performance tuning vs architecture changes

A query spends most of its time scanning rows. An index may solve the problem without changing data ownership. Sharding that database would introduce routing and migration while possibly leaving the expensive query design intact.

Performance tuning changes an implementation within its arrangement; an architecture change alters major placement, ownership, or interaction. Either may be justified. Measure before and after, and include correctness, operating work, and migration risk. Some central invariants really require a different model to scale. Choose the intervention by the demonstrated limit, rather than assuming the larger architectural change is the more effective one.

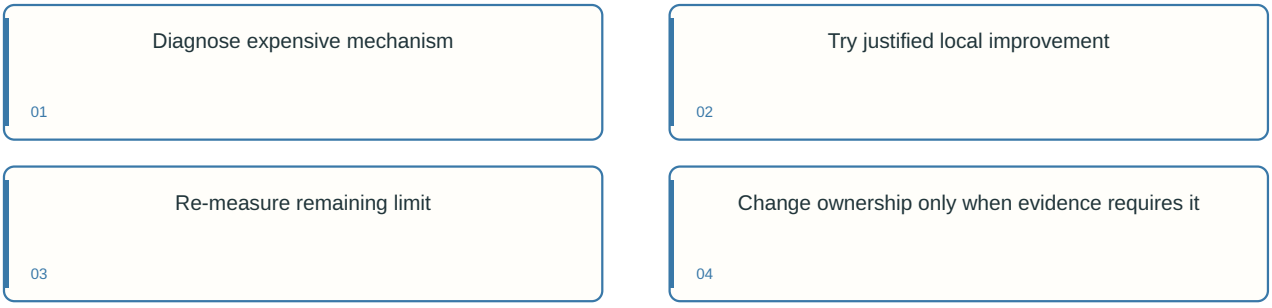


FIG. 21.2.2 **Performance tuning vs architecture changes · the example**

Tuning changes implementation costs; architecture changes alter major ownership or placement and need proportionate evidence.

Author-created illustration. Numbers are illustrative unless stated otherwise.

21.2.3 · Horizontal scaling economics

Compare one larger deployment, replicated application instances sharing a data owner, and partitioned storage. More nodes add baseline memory, discovery, coordination, and network work. Their advantage depends on which load can actually be divided.

Use equal requirements and failure headroom. A partitioned design may cost more at small scale while extending a later ceiling, but that is a hypothesis to test. Keep price, operating labour, and traffic assumptions explicit. Hypothetical rates can teach arithmetic; actual spending decisions need current prices. Node count alone cannot establish useful capacity or a lower cost per successful outcome.

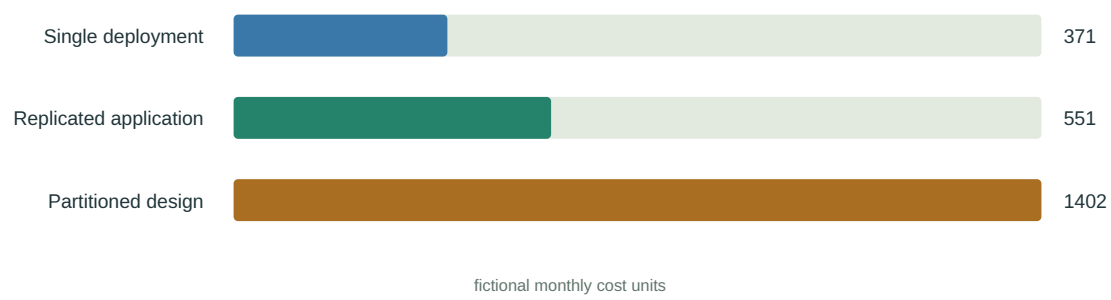


FIG. 21.2.3 **Architecture costs are not capacity guarantees**

The complete cost worksheet uses fictional monthly units. Each candidate still needs independent checks for correctness, performance, and failure requirements.

Author-created illustration. Numbers are illustrative unless stated otherwise.

21.2.4 · Compute, storage, networking, and telemetry costs

A cheap computation can produce a large response sent millions of times. Transfer may dominate its total cost. Follow the cost path through compute, requests, storage, indexes, replicas, backups, networking, and telemetry.

Include operational work where relevant. A cache can save source computation while adding memory and invalidation cost. Compression can save transfer while adding CPU. The worked prices in this atlas are fictional inputs, not current provider quotations. Keep the units and accounting period visible so the reader can replace the inputs and understand which part of the cost changes.

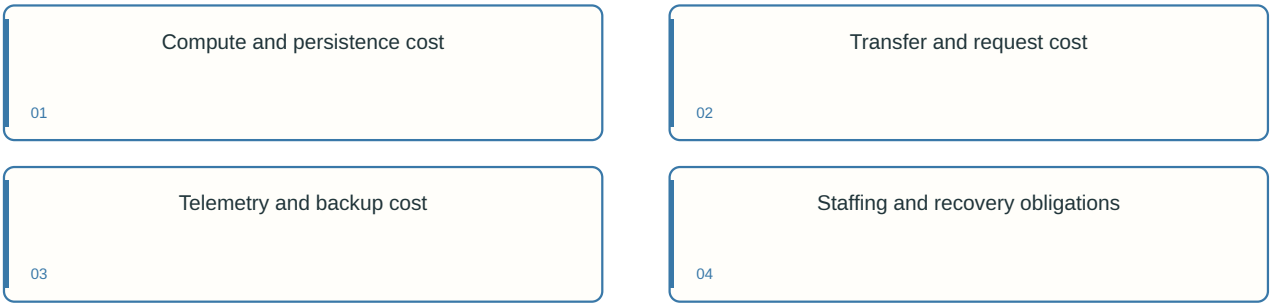


FIG. 21.2.4 **Compute, storage, networking, and telemetry costs · the example**

Total cost follows compute, requests, data, copies, transfer, telemetry, and operating work under explicit rates.

Author-created illustration. Numbers are illustrative unless stated otherwise.

21.2.5 · Cost per request or transaction

Assume monthly cost is 3,000 units and the system completes 150,000 useful business outcomes. The unit cost is 3,000/150,000 = 0.02 units per outcome.

Dividing by every attempt can conceal waste if many attempts fail or repeat. Choose a meaningful denominator such as completed orders or delivered documents, while retaining technical breakdowns for diagnosis. State the period and included expenses. Compare successful work under equivalent requirements; an operation that returns quickly but loses its result is not the same outcome. This keeps cost optimisation tied to the service users actually receive.

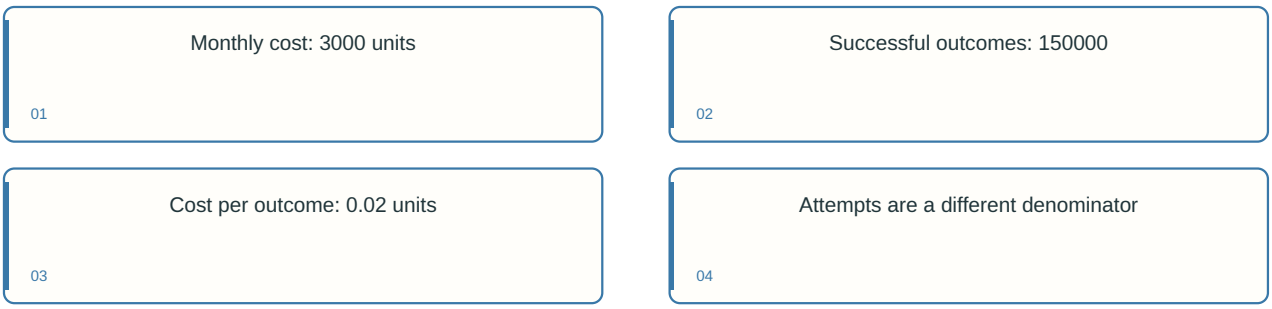


FIG. 21.2.5 **Cost per request or transaction · the example**

Cost per useful outcome avoids denominators inflated by failed or repeated attempts.

Author-created illustration. Numbers are illustrative unless stated otherwise.

21.2.6 · Capacity reservation vs elastic capacity

A steady baseline can suit committed capacity, while occasional bursts may favour capacity added when needed. Reservations trade commitment and idle-risk for particular commercial terms. Elastic resources still need provisioning time and may have different rates.

Compare baseline utilisation, burst duration, growth uncertainty, and failure needs. Buying for a rare peak can leave resources idle; waiting to scale after a short burst begins can arrive too late. A mixed approach may fit, but the balance needs measured traffic and current terms. Keep these commercial assumptions separate from the technical proof that surviving capacity meets the workload.

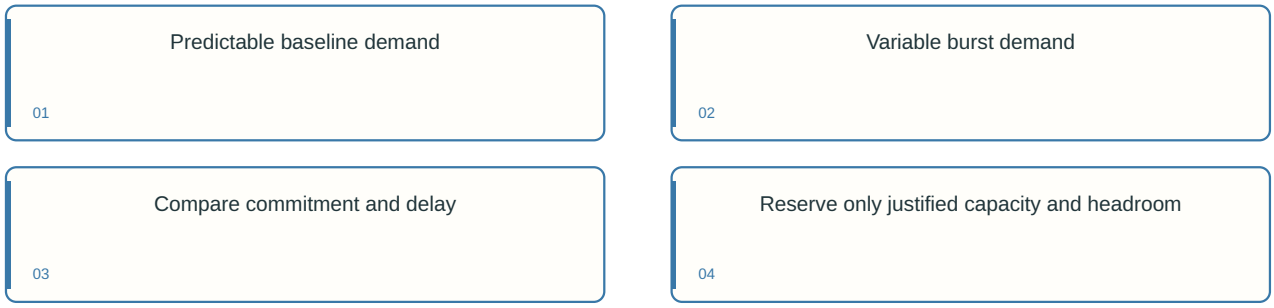


FIG. 21.2.6 **Capacity reservation vs elastic capacity · the example**

Reserved and elastic capacity trade commitment, idle risk, startup delay, and commercial terms against measured demand.

Author-created illustration. Numbers are illustrative unless stated otherwise.

A complete cost worksheet with fictional inputs

Monthly rates below are invented teaching inputs, not provider quotations. Capacity, p99, and availability are evaluation dimensions whose results must be measured; this cost table cannot establish them.

Fictional monthly cost units; compare only after checking requirements

Architecture	Compute	Data	Cache	Storage	Transfer	Telemetry	Total
Single deployment	40	300	0	1	20	10	371
Replicated application	160	300	50	1	20	20	551

Architecture	Compute	Data	Cache	Storage	Transfer	Telemetry	Total
Partitioned design	320	900	100	2	40	40	1402

Compare latency, availability, and operating work under stated assumptions

Construct a 100-request trace for each option: single deployment has 98 observations at 190 ms, one at 900 ms, and one at 1,400 ms; replicated application has 98 at 140 ms, one at 450 ms, and one at 800 ms; partitioned design has 98 at 110 ms, one at 600 ms, and one at 1,600 ms. Nearest-rank p99 is observation 99 after sorting. These synthetic traces deliberately show that lower common-case latency can coexist with worse tail latency. They predict no real deployment.

For a separate steady-state availability model, assume independent app-instance availability of 0.99 and data-owner availability of 0.999. An interchangeable pool of k apps is available with probability 1 − 0.01^k. The single and replicated options require one data owner. The partitioned operation in this example requires all three independent owners, so multiply by 0.999^3. Shared outages, overload, failover time, repair, and correlated failures are omitted; these probabilities are not SLO commitments.

```
Illustrative comparison; latency traces, availability probabilities, and operating judgments are assumptions
```

Architecture	Monthly cost units	Constructed p99	Modeled availability	Operational complexity judgment
Single: 1 app, 1 data owner	371	900 ms	98.901%	Lower: one app release and one data owner
Replicated: 4 apps, 1 data owner	551	450 ms	99.899999%	Medium: rolling releases, shared cache, pool capacity and failover
Partitioned: 8 apps, 3 required data owners	1,402	600 ms	99.700300%	Higher: routing, rebalancing, multiple owners and partial recovery

For a per-key request touching only one shard, the all-three-owners availability expression does not apply. Replication with an actual quorum or failover protocol also needs a different model. The calculation is executable as architecture_comparison() in the mechanism collection. Choose a candidate against the

workload and targets, then test tail latency, degraded capacity, restoration, and operating burden. Cost arithmetic alone cannot establish those properties.

▼ Think further · When could a more expensive architecture be simpler to operate overall?

Compare total work for people as well as the invoice. A managed service may reduce patching, backup, or recovery burden while adding charges and leaving application responsibilities. Evaluate the same correctness and service goals with explicit staffing assumptions. Choose the design the team can understand, change, and recover reliably.

Remember: Filter by required behaviour before ranking cost, and keep modelled quantities distinct from measured results.

Try applying it: Why can a lower cost per attempt coexist with a higher cost per completed order?

Unit 21 exercises → · Unit 21 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. The Practical Test Pyramid — Ham Vocke ↗
   Original engineering article. Test scope and feedback trade-offs, not a mandatory test-count ratio.
   Access reviewed · 21 October 2026

2. Handling Overload — Google SRE ↗
   Engineering book. Admission, capacity, and overload response.
   Access reviewed · 21 October 2026

3. The Tail at Scale — Dean and Barroso, 2013 ↗
   Original research. Tail-latency considerations; atlas probabilities and latency examples are author-calculated.
   Access reviewed · 21 October 2026

Efficiency and a useful test portfolio

Prerequisites: 21.2 · Capacity, architecture & cost per outcome 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

An optimisation saves CPU while adding transfer, or a test suite adds checks without covering failures. Evaluate the whole effect and choose evidence for the particular risk.

21.3.1 · Cost-performance optimization

A smaller instance is cheaper but misses the latency target under the same traffic. It is not a comparable successful design. Cost-performance optimisation first requires feasible correctness, service, security, and maintenance behaviour.

Then vary factors such as cache size, batching, compression, placement, or capacity shape with measurements. Check downstream effects: saving write bytes may add repeated expensive decompression on reads. Record the workload and objective that make the choice useful. An optimisation can become unsuitable after the traffic mix changes, so retain the conditions and a way to reconsider it.

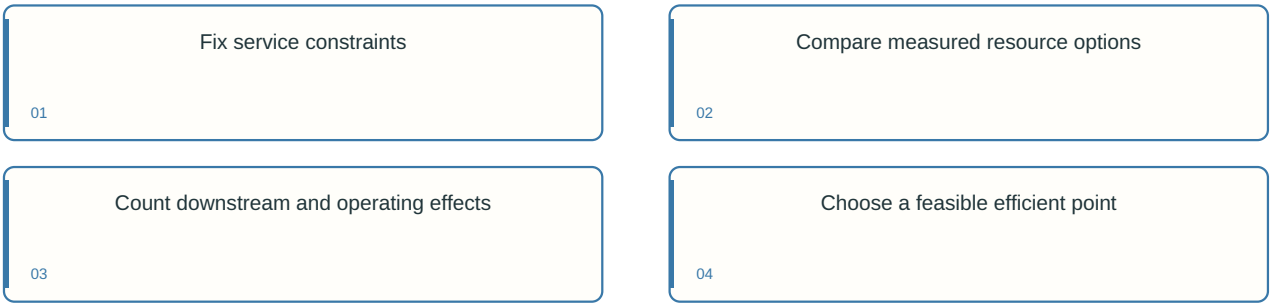


FIG. 21.3.1 **Cost-performance optimization · the example**

Cost-performance optimisation compares feasible equivalent service outcomes and checks downstream effects.

Author-created illustration. Numbers are illustrative unless stated otherwise.

21.3.2 · Sustainability and resource efficiency

Removing unused retained data saves storage work. Bounding retries avoids repeated computation and transfer. Resource efficiency begins by finding work the service does not need to perform.

Right-size capacity and batch suitable background operations without breaking deadlines. Evaluate complete effects: less CPU in one component can cause much more network or storage work elsewhere. Environmental claims need actual evidence about the relevant hardware, place, time, and lifecycle. Do not invent energy or carbon measurements. Useful engineering improvements can be explained by the resources they reduce while keeping broader sustainability conclusions within the available evidence.

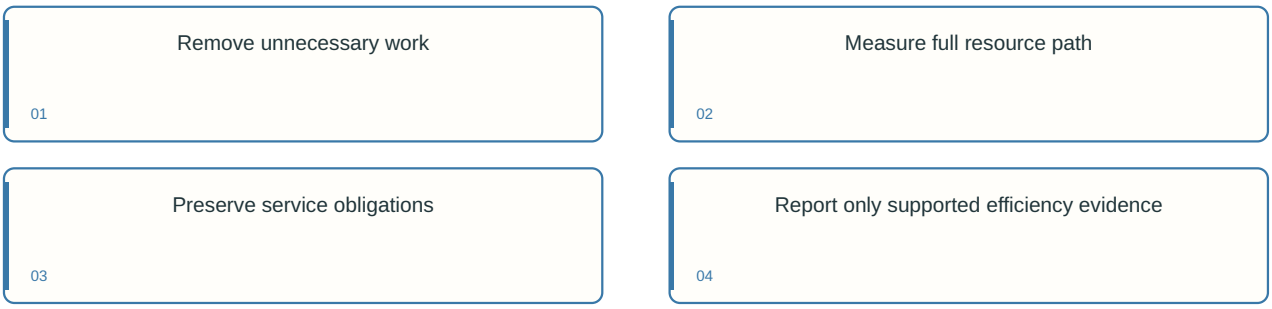


FIG. 21.3.2 **Sustainability and resource efficiency · the example**

Resource efficiency removes unnecessary work and transfer while broader sustainability claims require actual supporting evidence.

Author-created illustration. Numbers are illustrative unless stated otherwise.

21.3.3 · Unit, integration, contract, end-to-end, and performance tests

A unit test can check fee arithmetic with controlled inputs. A database integration test can check a transaction. A contract test checks boundary expectations, and an end-to-end test follows a complete journey. Performance tests examine behaviour under load.

Choose each test for the failure it can detect. A fake provider helps exercise declines, but cannot establish real serialization or commit behaviour. A suite that asserts every private call may hinder harmless refactoring while missing a lost reply. The test pyramid is guidance about feedback cost and scope, not a mandatory numerical ratio. Useful evidence matters more than raw test count.

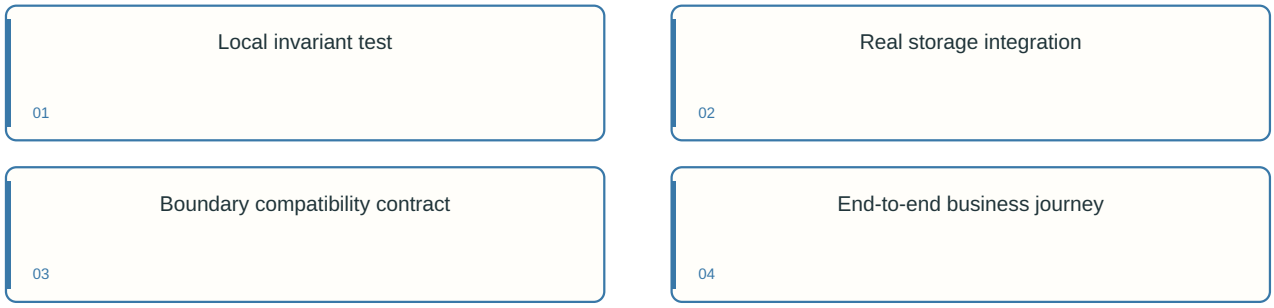


FIG. 21.3.3 **Unit, integration, contract, end-to-end, and performance tests · the example**

Test scopes provide different evidence; choose checks by failures they can detect rather than a mandatory count ratio.

Author-created illustration. Numbers are illustrative unless stated otherwise.

21.3.4 · Fault-injection tests

A process killed before it starts work says little about a crash after commit but before acknowledgment. Place faults at the boundary whose guarantee is in question.

Inject delays, unavailability, interruptions, or invalid responses with explicit acceptable outcomes and stop conditions. Test a resumed worker after lease expiry and an interrupted data move where relevant. Combine serving observations with data checks: the application can start responding again while a promised job remains lost. A fault test passes the business contract only when both the recovered service and its required state are accounted for.

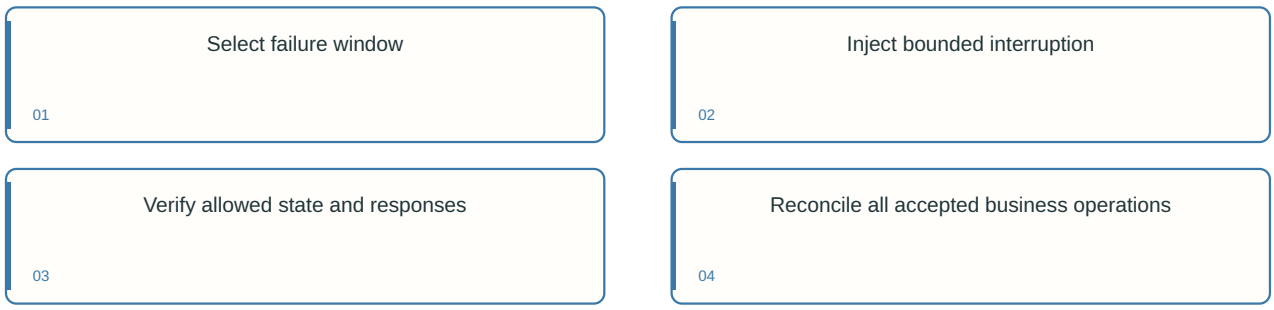


FIG. 21.3.4 **Fault-injection tests · the example**

Fault injection targets meaningful interruptions and validates both useful recovery and required state.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · When could a more expensive architecture be simpler to operate overall?

Compare total work for people as well as the invoice. A managed service may reduce patching, backup, or recovery burden while adding charges and leaving application responsibilities. Evaluate the same correctness and service goals with explicit staffing assumptions. Choose the design the team can understand, change, and recover reliably.

Remember: Efficiency and validation should reduce real work or uncertainty without weakening the required contract.

Try applying it: Which failure window would a process kill before work begins leave untested?

Unit 21 exercises → · Unit 21 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. The Practical Test Pyramid — Ham Vocke ↗

   Original engineering article. Test scope and feedback trade-offs, not a mandatory test-count ratio.

   Access reviewed · 21 October 2026

2. Handling Overload — Google SRE ↗

   Engineering book. Admission, capacity, and overload response.

   Access reviewed · 21 October 2026

3. The Tail at Scale — Dean and Barroso, 2013 ↗

   Original research. Tail-latency considerations; atlas probabilities and latency examples are author-calculated.

   Access reviewed · 21 October 2026

Fitness functions and design validation

Prerequisites: 21.3 · Efficiency and a useful test portfolio 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A migration has matching row counts but wrong tenant ownership. Validate identities and invariants, preserve important regressions, and return to the requirements that justified the design.

21.4.1 · Data consistency validation

Source and destination each contain a thousand rows, but some rows belong to the wrong tenant. Equal counts missed the error. Data consistency validation must compare identities, versions, and domain rules as well as totals.

For a ledger, check balanced postings and operation uniqueness. For a search index, check the source version and deletion state. Run suitable checks after migration, replay, and restoration. Sampling can reduce cost but supports a weaker claim than examining every relevant record. State the coverage of the evidence so a useful sample is not presented as complete equivalence.

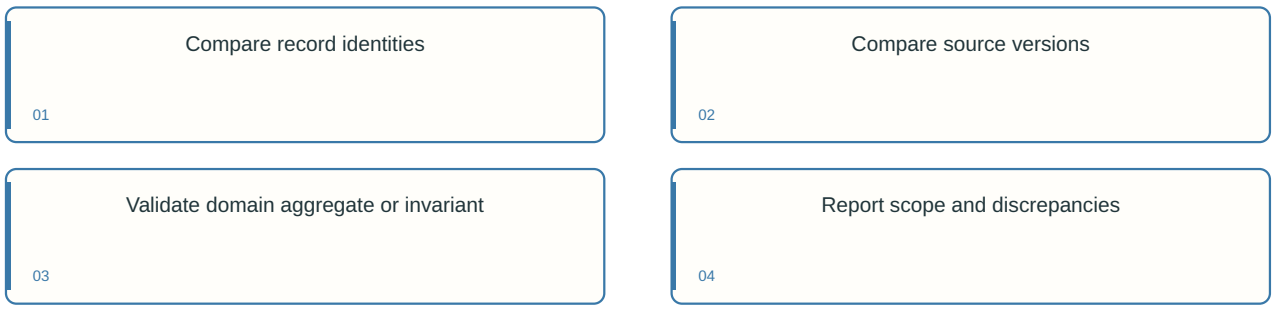


FIG. 21.4.1 **Data consistency validation · the example**

Data consistency checks compare identities, versions, relationships, and invariants beyond equal counts.

Author-created illustration. Numbers are illustrative unless stated otherwise.

21.4.2 · Architecture fitness functions

An import rule can prevent the orders module from depending on private payment storage. A runtime test can check tenant-scoped queries. These are architecture fitness functions: repeatable checks for properties the architecture needs to preserve.

Protect the reason for the boundary instead of freezing every incidental class relationship. Some qualities require drills or manual review rather than static inspection. Update a check when a legitimate decision changes the required property. Passing the set provides evidence for its covered behaviours, not a general certificate of architectural correctness. Choose checks that can reveal a meaningful erosion of the intended design.

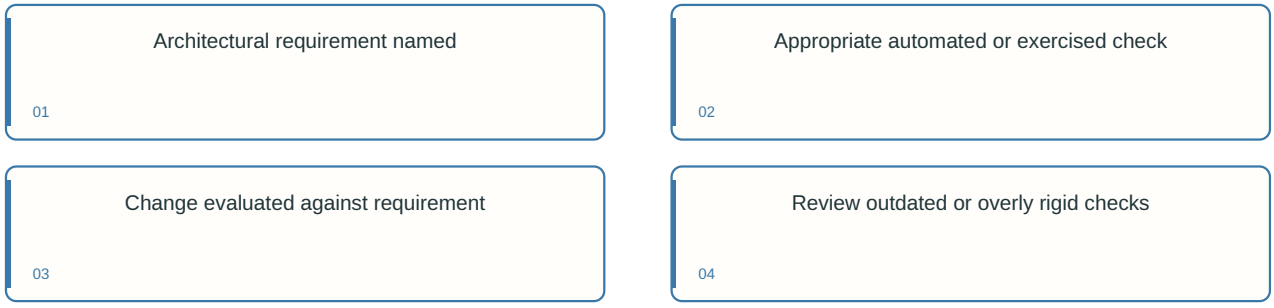


FIG. 21.4.2 **Architecture fitness functions · the example**

Architecture fitness checks protect required properties without freezing irrelevant implementation details.

Author-created illustration. Numbers are illustrative unless stated otherwise.

21.4.3 · Regression testing

A regression test keeps an important behaviour visible while the implementation changes. Include relevant errors, concurrency, compatibility, and recovery sequences, especially those that previously caused harm.

Use representative data and preserve the test's purpose through refactoring. Performance checks need sufficiently stable conditions and tolerances to avoid treating noise as a defect. If a requirement changes, revise the expected outcome explicitly. Deleting a failing test without naming the changed contract loses the reason it existed. The suite should preserve useful obligations while allowing the code that fulfils them to improve.

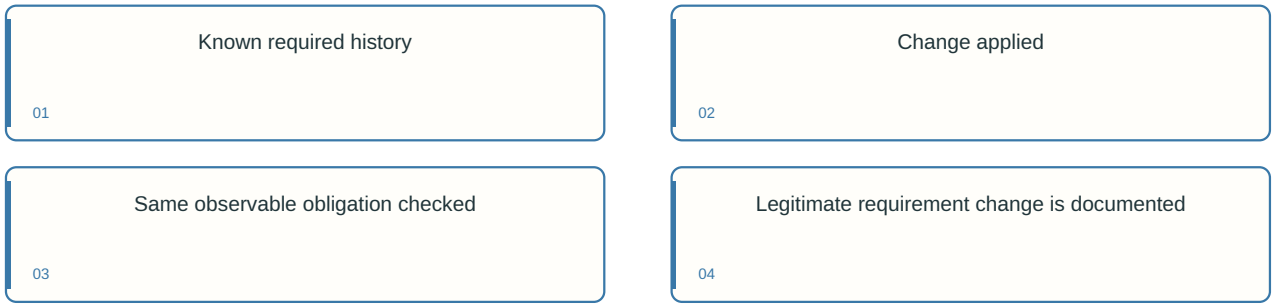


FIG. 21.4.3 **Regression testing · the example**

Regression tests preserve important behaviour while allowing deliberate contract changes and tolerating measured noise appropriately.

Author-created illustration. Numbers are illustrative unless stated otherwise.

21.4.4 · Evaluating a design against its original requirements

Return to the original brief. For each promised behaviour, identify its test, measurement, review, or still-unresolved assumption. This creates a traceable link from the design choice to its evidence.

Check function, timing distributions, capacity, recovery, access, cost, and operating burden under the stated workload. A diagram review and a load test answer different questions. If a target has not been measured, keep calling it a target. Record the remaining uncertainty rather than treating completed coverage as completed validation. The final evaluation asks whether the chosen system meets the needs that justified building it.

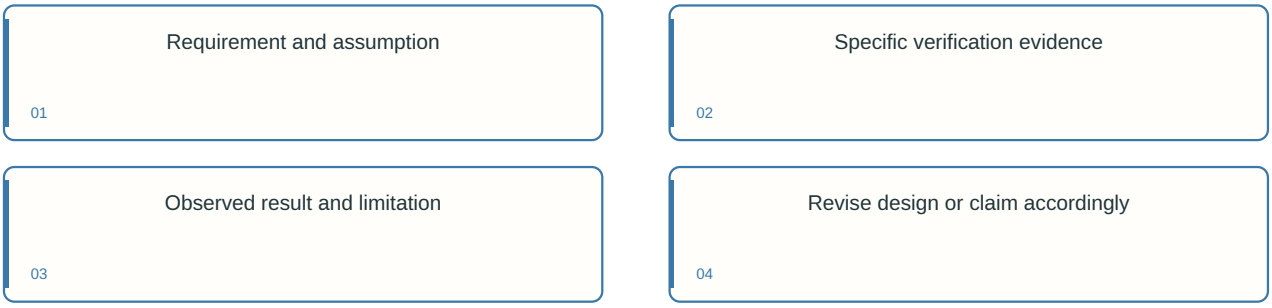


FIG. 21.4.4 **Evaluating a design against its original requirements · the example**

Design validation traces original requirements to tests, measurements, reviews, and clearly unresolved assumptions.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Try the mechanism in code

PYTHON · FICTIONAL COMPARISON

```
from mechanisms import cost_options
totals = {name:sum(parts.values()) for name,parts in cost_options().items()}
assert totals == {"single":371, "replicated_app":551, "partitioned":1402}
```

The monthly totals are 371, 551, and 1,402 fictional units. They show how component costs add, rather than how the deployments actually perform.

Check equal requirements before ranking price. The additional latency and availability models have explicit synthetic inputs; real capacity, tails, and recovery require corresponding tests.

Download the executable mechanism collection ↓ · Follow the connected complete design →

THE WORKBENCH · TRY CHANGING THE INPUTS

Calculate the cost of one successful outcome

- Single deployment: 371 monthly units / 150,000 successful outcomes = **0.002473** units/outcome.
- Replicated application: 551 monthly units / 150,000 successful outcomes = **0.003673** units/outcome.
- Partitioned design: 1402 monthly units / 150,000 successful outcomes = **0.009347** units/outcome.

Compare costs only after checking that the options meet the same requirements.

These invented monthly costs are 371, 551, and 1,402 units. Change the number of successful outcomes to see the cost per outcome. Before comparing the options, test whether each can meet the required capacity, latency, availability, and correctness. These figures are not provider prices.

▼ Think further · When could a more expensive architecture be simpler to operate overall?

Compare total work for people as well as the invoice. A managed service may reduce patching, backup, or recovery burden while adding charges and leaving application responsibilities. Evaluate the same correctness and service goals with explicit staffing assumptions. Choose the design the team can understand, change, and recover reliably.

Remember: Pair each requirement with evidence and keep unresolved targets clearly labelled.

Try applying it: What additional evidence would show that an equal-count migration preserved the right data?

Unit 21 exercises → · Unit 21 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. The Practical Test Pyramid — Ham Vocke ↗

   Original engineering article. Test scope and feedback trade-offs, not a mandatory test-count ratio.

   Access reviewed · 21 October 2026

2. Handling Overload — Google SRE ↗

   Engineering book. Admission, capacity, and overload response.

   Access reviewed · 21 October 2026

3. The Tail at Scale — Dean and Barroso, 2013 ↗

   Original research. Tail-latency considerations; atlas probabilities and latency examples are author-calculated.

   Access reviewed · 21 October 2026

PART 6 / UNIT 22

End-to-End Design, Architecture Reviews, and Evolution

We can now bring the earlier ideas together. Each design starts with a concrete workload, follows its main operation, and examines what happens when part of the system fails.

The running example

Twelve complete architectural designs with a common 22-point review protocol.

Unit 22 assessment → Explained solutions →

The complete review protocol

1. Functional requirements
2. Nonfunctional requirements
3. Explicit assumptions and workload numbers
4. Naive initial architecture
5. Core API contracts
6. Data model and schema
7. Low-level component organization
8. High-level system architecture
9. Data and control flows
10. Capacity estimates
11. End-to-end latency estimates
12. Failure scenarios
13. Scalability improvements
14. Consistency requirements
15. Security model
16. Deployment approach
17. Observability design
18. Infrastructure cost considerations
19. Alternative architectures and trade-offs
20. Architecture evolution from small scale to large scale
21. Design review identifying weaknesses and improvements
22. A short implementation exercise

Each of the twelve designs above contains every section, including the implementation challenge. Optional next subjects: Search engine. · Cloud storage service. · Distributed cache. · API gateway. · Metrics collection platform. · Multi-tenant SaaS. · Real-time fraud detection. · IoT ingestion platform..

URL Shortener

Prerequisites: API Design and Communication Contracts, Data Models and Storage Technologies, Caching and Content Delivery, Scalability, Load Balancing, and Data Distribution

4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A short link hides a lookup, an ownership rule, and a cache-freshness decision. Follow one mapping from creation through redirect and revocation.

The workload, timing, and prices in this design are teaching assumptions. They are not measured benchmarks or a description of a particular company.

22.1.1 · Functional requirements

A user submits a long URL and receives a short code. Later, a visitor opens that code and is redirected to the original target. The owner can expire or revoke the link and manage it through an authenticated interface.

Keep the original target and creation identity. Click analytics is secondary work: a failed analytics database should not make an otherwise valid redirect wait indefinitely. Abuse handling and the correctness of the redirect remain part of the main product.

22.1.2 · Nonfunctional requirements

For this example, target redirect p99 below 100 ms under the supported peak. An acknowledged mapping must survive the selected single-storage-node failure, and management operations must preserve ownership.

Specify how soon revocation and expiry take effect. A long-lived cached redirect can remain usable after the source changes unless another mechanism prevents it. The latency target, storage acknowledgment

policy, and cache freshness rule therefore need to be checked together rather than promised independently.

22.1.3 · Explicit assumptions and workload numbers

Assume 100 creations/s and 10,000 redirects/s on average, with a tenfold redirect peak. Assume 200 stored bytes per mapping before indexes and extra copies. Daily raw growth is 100×86,400×200 = 1.728 GB.

For transfer calculations, assume a 500-byte redirect response. These quantities are teaching inputs. Actual record size, traffic duration, and transport overhead must be measured before using them to provision a service.

22.1.4 · Naive initial architecture

Begin with one web application, one relational mapping table, and a unique code constraint. A background job handles analytics. Local uniqueness is straightforward, and this arrangement is suitable for an initial small workload.

Its weaknesses are visible: the application host can fail, and every miss reaches the one data path. Measure lookup and write behaviour before distributing data. A sharded design adds routing and uniqueness work that may not yet solve the actual bottleneck.

22.1.5 · Core API contracts

POST /links accepts a target, expiry, and idempotency key. GET /{code} returns the permitted redirect or the defined expired/revoked result. DELETE /links/{id} checks the owner before changing state.

Validate supported URL schemes and syntax without fetching arbitrary destinations. Reusing a key with different content conflicts; retrying an identical creation can recover its original result. State whether redirects are permanent or temporary, since that affects how clients may keep copies after later changes.

22.1.6 · Data model and schema

A link record contains id, unique code, target, owner_id, creation time, expiry, state, and version. An operation record binds the owner and operation key to a request fingerprint and result link. The key is unique within the owner scope.

The code must be unique across its intended namespace. If mappings are later sharded, routing or allocation must preserve that rule across owners. A uniqueness constraint applied independently to

unrelated shards is insufficient. Keep revocation and expiry attached to authoritative versioned state.

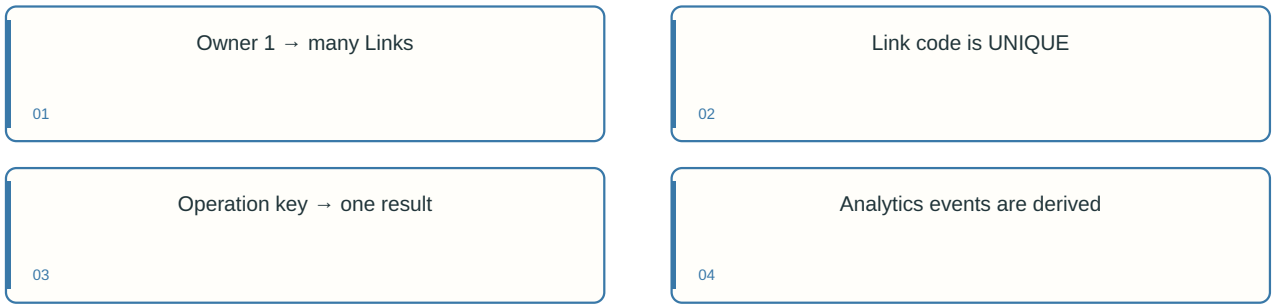


FIG. 22.1.1 **Data ownership and relationships**

A link has one owner and a unique code. The stored operation identifies its creation result.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.1.7 · Low-level component organization

LinkService checks the target and caller. CodeAllocator proposes a code. LinkRepository commits the mapping and its operation result together. RedirectResolver decides whether the link is usable, and AnalyticsPublisher records secondary events.

The allocator can encode an appropriately unique integer in base62 or propose random codes and retry collisions. The repository's uniqueness check is the decisive boundary. A random proposal is not proof of uniqueness, and a provider-specific storage error should not leak through every application caller.

22.1.8 · High-level system architecture

Put an edge or reverse proxy before replaceable application instances. A cache serves representations whose expiry and revocation policy permits reuse. The authoritative replicated store owns mappings and recovery; a queue or log carries analytics separately.

The cache can improve reads but must not become the only recoverable copy. Bound fallback traffic when it is unavailable. Extra application instances protect some serving capacity, while storage placement and acknowledgment determine which mapping history survives a failure.

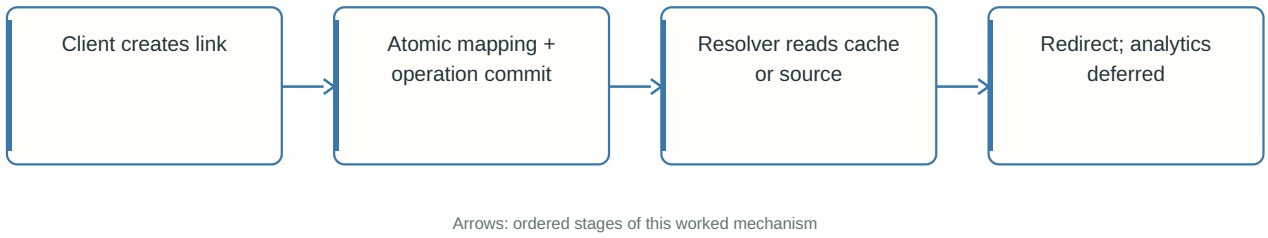


FIG. 22.1.2 **Follow the main operation**

Creation and redirect use authoritative mappings; analytics is a separate obligation.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.1.9 · Data and control flows

For creation, validate the request, atomically commit the mapping and deduplication result, then reply. For redirect, check usable state, obtain a permitted cached or source value, and send the response. Record analytics independently of the visitor's required result.

Combine concurrent misses for a code where useful. Keep private management representations out of the public redirect cache. The write, read, and analytics paths have different responsibilities and should remain distinguishable in the implementation.

22.1.10 · Capacity estimates

With an assumed 90% hit ratio, 10,000 redirects/s cause 1,000 source reads/s before retries or coalescing. At 100,000 redirects/s, the same ratio implies 10,000 source reads/s. Response traffic is 100,000×500 = 50 MB/s before transport overhead.

Also test a cold cache, when the nominal miss calculation no longer applies. The source and admission policy need a defined response to that workload; ordinary warm-cache success does not establish it.

22.1.11 · End-to-end latency estimates

Assume a warm path spends 20 ms at the edge/network, 10 ms in the application, 5 ms in cache access, and 20 ms returning the result. The sum is 55 ms.

This is a proposed budget, not measured p99. A miss adds source lookup and possible queueing. Measure the tail under hot links, cache loss, storage failover, and required revocation checks before claiming the 100

ms objective. Adding average stage times does not by itself establish an end-to-end percentile.

22.1.12 · Failure scenarios

Two creations propose the same code. The second insert must fail atomically and retry, preserving the first target. A lost creation reply should recover the stored operation result instead of creating another mapping.

Cache outage needs origin admission controls. A stale cache can redirect a revoked link unless the freshness rule prevents it. Analytics failure should affect analytics progress rather than valid redirect service. Test these separate histories; one successful create-and-open demonstration leaves all of them unexamined.

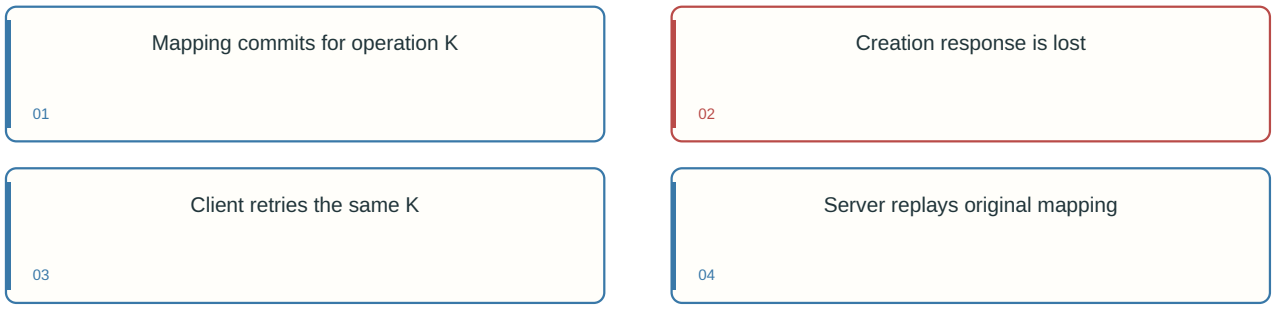


FIG. 22.1.3 **Follow the failure and its correction**

Test a collision, a lost reply, and a stale cached target at their different boundaries.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.1.13 · Scalability improvements

Add application capacity when independent application work is limiting. Add shared or edge caching when measured reads justify it. Partition mappings only when the authoritative storage path needs division.

A globally popular code may need permitted read copies instead of more cold partitions. Every added copy retains the expiry and revocation obligation. Data movement needs routing, version, and recovery rules, so expansion should preserve both global code ownership and the supported state of existing operations.

22.1.14 · Consistency requirements

Creating a code needs one atomic uniqueness decision. The owner-facing read should meet the chosen read-after-write rule. Public redirects may tolerate delayed target changes only within the stated expiry and

revocation contract.

Analytics can be processed later with stable event identities. These operations therefore have different consistency needs. Calling the whole system “eventually consistent” hides the exclusive code insertion and the permitted delay of a click count. State the observable rule for each path.

22.1.15 · Security model

Authorise owner management and limit creation abuse. Accept only supported target schemes. If previews fetch destinations, restrict egress and validate redirect hops so a public URL field does not become access to private services.

Protect credentials and avoid leaking private targets through logs or analytics. A short code that is difficult to guess is still a link, not a complete permission system for confidential content. If restricted destinations are supported, define authentication and access checking for those redirects separately.

22.1.16 · Deployment approach

Start with a region and a clear failure contract. Use compatible rolling application releases, a suitably replicated authoritative store, and tested backups. Preserve operation identities through replacement and drain connections as needed.

New code must understand existing mapping versions. A later second region needs a deliberate write-authority and possible-loss policy. Add it for a named locality or recovery requirement, rather than assuming that more geography automatically preserves every acknowledged mapping.

22.1.17 · Observability design

Measure redirect outcomes and latency, separating hits from misses. Track origin admission, hot codes, code conflicts, storage progress, and how long revocation takes to reach permitted readers.

Observe analytics lag separately. A good cache hit ratio does not show whether a newly created or revoked link is correct. Alert on useful request failures and rapid budget consumption, and use traces or sampled histories to locate the affected path without exposing private target data unnecessarily.

22.1.18 · Infrastructure cost considerations

Model compute as instance-hours times unit rate, storage as retained bytes times copy factor and rate, and transfer as response bytes times rate. Keep the duration assumption explicit.

For example, sustaining 100,000 redirects/s for 30 days at 500 bytes produces 129.6 TB before overhead. That is a different workload from a brief peak of the same height. Use actual traffic durations and current rates for spending decisions; these formulas only expose the inputs the estimate requires.

22.1.19 · Alternative architectures and trade-offs

Allocated sequential identifiers simplify code generation but need a valid allocator and can reveal order. Random codes reduce that predictability but still need collision detection. Neither option grants access control.

Permanent redirects can let clients reuse responses more aggressively, making later changes harder to enforce. Temporary redirects retain more server control at a traffic cost. Compare the alternatives using revocation, mutation, privacy, and load requirements. The right choice is the smallest mechanism that preserves those meanings, not the most familiar coding trick.

22.1.20 · Architecture evolution from small scale to large scale

Use traffic levels as teaching checkpoints rather than hardware promises. Around 100 requests/s, begin with one application and transactional store. Around 1,000, inspect plans and indexes. Around 10,000, consider measured application and cache needs.

At 100,000, expand the actual bottleneck and verify cold-cache and degraded operation. A given machine may reach a limit earlier or later. Preserve stable code and operation identities at every stage so scaling does not change the meaning of existing links or retries.

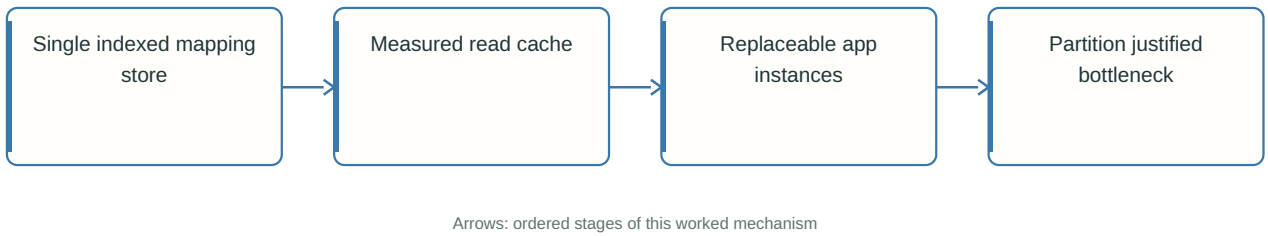


FIG. 22.1.4 **How the design can grow**

Add capacity for a measured limit while preserving code uniqueness and revocation.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.1.21 · Design review identifying weaknesses and improvements

The main uncertainties are revocation freshness, source capacity after cache loss, and global uniqueness during partitioning. Pair each with a specific check: revoke under load, empty or disable the cache, and force a collision across the real ownership boundary.

Record acceptable degraded outcomes and unresolved evidence. Replicating the cache alone does not answer these questions. A useful review ends with the failure behaviour the design supports and the test that could show its assumption wrong.

22.1.22 · A short implementation exercise

Implement base62 encoding and a repository with an atomic unique-code constraint. Force two proposals to use one code and check that the original target remains unchanged.

Then stop execution after commit but before returning the creation result. Retry with the same owner and operation key. The expected result is the existing mapping, with no second logical creation. Also check that changing the target under that key returns the documented conflict.

A small implementation of the key rule

PSEUDOCODE · ONE PART OF THE DESIGN

```
def allocate(target, operation_key):
    prior = operations.find(operation_key)
    if prior: return prior
    while attempts_remaining():
        code = propose_code()
        try:
            with transaction():
                link = insert_unique_code(code, target)
                save_operation_result(operation_key, link.id)
            return link
        except CodeConflict:
            continue
        except OperationConflict:
            return load_and_validate_prior_result(operation_key, target)
    raise AllocationUnavailable
```

Use the implementation exercise to test the rule and the failure described above. You can also run local examples of seat claims, duplicate handling, ledger balance, and workflow epochs in the mechanism collection.

Unit 22 exercises → · Unit 22 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. RFC 9110: HTTP Semantics, June 2022 ↗

   Original standard. HTTP semantics; application commit policy is separate.

   `Access reviewed · 21 October 2026`

2. RFC 9111: HTTP Caching, 2022 ↗

   Original standard. HTTP cache semantics; application cache races are separate examples.

   `Access reviewed · 21 October 2026`

3. Application best practices — Microsoft Architecture Center ↗

   Official guidance · accessed 21 Oct 2026. Scaling and application boundary review.

   `Access reviewed · 21 October 2026`

Rate Limiter

Prerequisites: Requirements, Estimation, and Engineering Trade-offs, Runtime, Processes, Threads, and Concurrency, Scalability, Load Balancing, and Data Distribution, Reliability, Fault Tolerance, and Disaster Recovery

4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Three application instances each grant a full bucket of tokens. We need to decide whether the policy is local, globally strict, or a bounded allocation shared among regions.

The workload, timing, and prices in this design are teaching assumptions. They are not measured benchmarks or a description of a particular company.

22.2.1 · Functional requirements

A request arrives from a caller and asks to perform work with a known cost. The limiter decides whether the current policy permits it and returns useful guidance if it must wait or be rejected.

Distinguish continuing rate, permitted burst, and cumulative quota. For a distributed deployment, state whether the limit is exact globally or a bounded regional approximation. Giving every region a full independent global bucket silently creates a different policy.

22.2.2 · Nonfunctional requirements

Admission should add little delay while keeping protected resources within their allowance. Preserve tenant fairness and specify how the limiter behaves when its state is unavailable.

An exact global cap can require rejection or waiting during a partition. Failing open cannot preserve that cap unless previously allocated safe budgets cover the admitted work. Choose the failure rule alongside the availability goal. “Always fast and always exact” needs a protocol and assumptions, rather than two independent targets.

22.2.3 · Explicit assumptions and workload numbers

Assume 100,000 checks/s and one million active keys. A sample policy holds ten tokens and refills two per second. At an assumed 64 bytes of raw state per key, the raw total is 64 MB.

Engine allocation, expiry metadata, replication, and network requests add to this. These are teaching inputs, not product benchmarks. State size is only one capacity dimension; the update rate and contention of a popular key may become limiting sooner.

22.2.4 · Naive initial architecture

Begin with a dictionary in one process and protect each bucket transition with a suitable lock. This provides one local atomic decision for the callers sharing that process.

Starting a second unrelated process gives it another full allowance. Restart may reset the bucket too. The simple version is appropriate only if this scope and reset behaviour meet the requirement. Demonstrating correct arithmetic locally does not establish a shared cap across instances.

22.2.5 · Core API contracts

check(key,cost,policy_version) returns the admission decision, remaining allowance, and meaningful retry delay. Derive the key from trusted caller and resource identity; a caller must not evade its policy by inventing another key.

Require a positive bounded cost. Policy administration needs authorisation and an effective-version rule. Distinguish overload from invalid business input, and state whether rejected attempts consume a separate abuse budget. These meanings let callers react without immediately recreating the same excessive demand.

22.2.6 · Data model and schema

Keep a bucket record with scope_key, policy_version, tokens, last_refill, and expiry. Refill and spending share one atomic update. A cumulative daily quota can require separate state and rules.

Fixed-point or integer units can avoid unnecessary numeric drift. A policy change needs defined treatment of existing tokens and capacity. The elapsed-time basis must be trusted; client timestamps must not let a caller refill early. Expiring state also changes allowance, so it belongs in the contract.

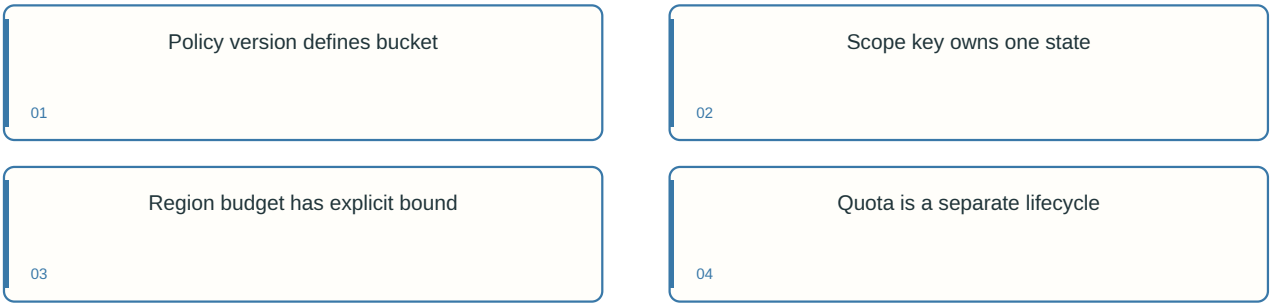


FIG. 22.2.1 **Data ownership and relationships**

A bucket groups scope, policy, tokens, and refill time in one coordinated record.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.2.7 · Low-level component organization

KeyResolver derives trusted scope, PolicyResolver supplies validated policy, and Clock supplies the time basis. BucketStore performs the coordinated transition. AdmissionService translates the result for the caller.

These roles make deterministic tests practical, but do not replace atomic state access. Two pure calculations using the same old token count can both admit work. Keep the refill/check/debit decision under one owner or supported transaction. The decisive property concerns the combined operation, not merely each helper's correctness.

22.2.8 · High-level system architecture

Use coarse admission near ingress and more specific protection before expensive resources. Independent keys can have different owners. A strict shared bucket needs one suitable atomic authority or a reviewed distributed protocol.

Regional budgets can avoid a remote check on each request, but need bounded allocation and replenishment. State the allowed overshoot or outstanding-budget rule. Protect the limiter's own capacity and define emergency controls. A protection service can otherwise become the first component overloaded by the traffic it was meant to regulate.

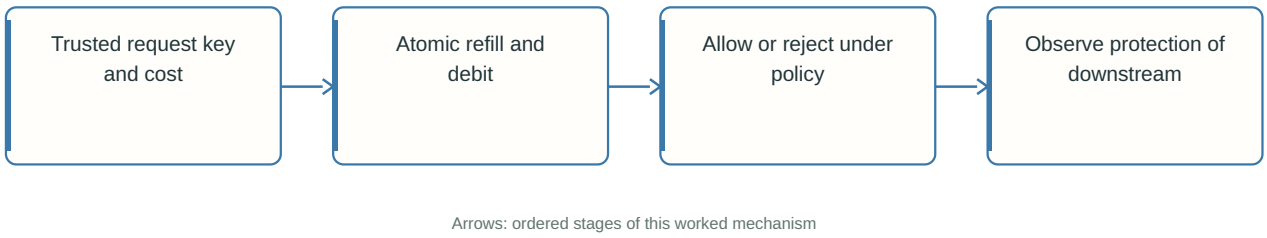


FIG. 22.2.2 **Follow the main operation**

Derive trusted scope before making the refill-and-spend decision.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.2.9 · Data and control flows

Derive the request's scope and current policy. Under coordinated access, load its bucket, calculate nonnegative elapsed time, refill no further than capacity, compare the cost, and debit only if allowed.

Compute retry delay from the deficit and refill rate when that advice is meaningful. Record bounded metrics. Splitting the read and final write into unprotected calls lets two requests spend the same last token. The atomic transition is what connects correct arithmetic to a correct concurrent decision.

22.2.10 · Capacity estimates

One million 64-byte records require 64 MB raw. A remote check per incoming request also means 100,000 limiter requests/s under the assumed workload, before replica work and retries.

A single hot key can serialise many updates despite evenly distributed storage. Measure check rate, active state, contention, and replicated write cost. Batching or local budgets may reduce traffic, but their changed timing and precision must be stated. A memory estimate alone is not a complete limiter capacity plan.

22.2.11 · End-to-end latency estimates

Assume a 100 ms application target and allocate 5 ms to admission. This is a budget to validate under contention, not a measured guarantee.

A strict cross-region decision adds propagation and queueing and may not fit it. Local preallocated budgets can reduce that delay while adding an allocation protocol. Measure the complete path, including store

failover and policy retrieval, against the required precision. Changing the consistency contract can change latency, so present those choices together.

22.2.12 · Failure scenarios

Two requests read one token and both pass. Repair that race by combining the read, refill, and debit atomically. Backward time must not create extra refill, and state expiry must not grant an unintended unsafe burst.

Test key spoofing and policy disagreement as well as arithmetic. During owner failure, apply the documented reject, safe-local-budget, or bounded fail-open policy. A mathematically correct bucket can still fail to protect the resource if identities or deployment scopes are wrong.

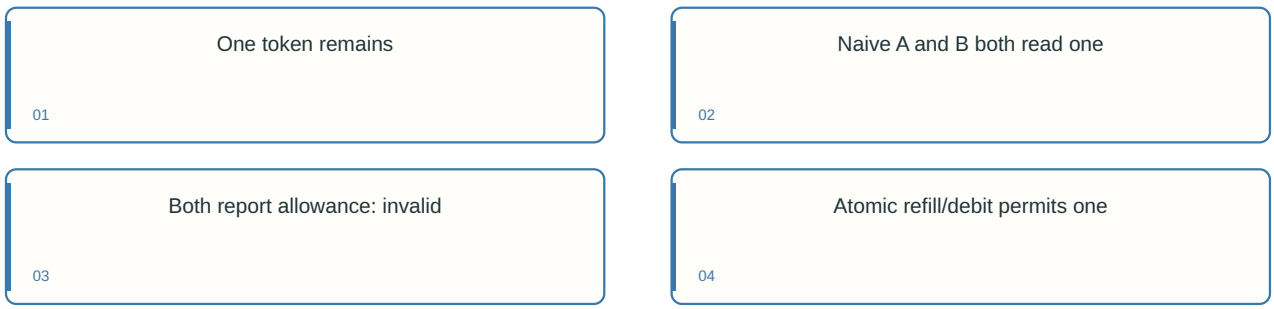


FIG. 22.2.3 **Follow the failure and its correction**

Two callers must not independently spend the same last token.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.2.13 · Scalability improvements

Partition independent keys while keeping a bucket's decision coordinated. Cache versioned policy definitions where suitable, while maintaining the authority needed for spending.

One strict hot bucket cannot be divided freely without a budget-allocation protocol. Hierarchical limits or local allowances can help when their semantics fit the resource. Different operations may deserve different costs. Scale according to protected work and measured limiter contention, rather than increasing the key count while leaving the hot decision unchanged.

22.2.14 · Consistency requirements

A strict bucket preserves one policy's refill-and-debit rule under its time model. An approximate distributed version needs a stated bound, such as the total independently outstanding regional allowance.

Token buckets permit bursts. Fixed windows have boundary effects, while sliding windows model recent history differently. These are different admission contracts. Converging usage metrics later cannot repair requests already admitted beyond a mandatory cap. Name the invariant the live decision must preserve instead of treating eventual counter agreement as equivalent protection.

22.2.15 · Security model

Bind scope keys to authenticated tenant or service identity. Guard policy changes and cap key creation so arbitrary callers cannot exhaust state memory. Do not trust client time, policy names, or tenant headers without validation.

Limit what usage information responses reveal about other callers. Admission and permission remain separate: possessing tokens does not authorise a read of another tenant's document. Apply the resource access rule after, or as part of, the relevant protected operation.

22.2.16 · Deployment approach

Deploy the policy and bucket owners with specified state-loss and failover behaviour. A restart that grants every caller a new full bucket changes the protection contract.

Roll out policy versions compatibly and test time behaviour, old clients, and shared-owner loss. A local fallback must have an acceptable bound. Preserve state or allocate safe replacement allowance according to the chosen design, rather than relying on the assumption that protection state is disposable merely because individual records are small.

22.2.17 · Observability design

Measure allowed and rejected cost, check latency, owner errors, active keys, hot-key pressure, and policy-version differences. Compare these with the resource the limiter protects.

Many rejections can indicate effective protection or a bad configuration blocking legitimate users. Few rejections can hide fail-open behaviour during store failure. Observe both protection failures and unnecessary denial. Use bounded dimensions for metrics so tenant or key cardinality does not make the monitoring system the next capacity problem.

22.2.18 · Infrastructure cost considerations

The cost includes state memory, check traffic, replicated updates, transfer, and operating work. A remote strict decision on every cheap business request can cost more than the operation being protected.

Compare strict and local-budget designs with their guarantees stated. Use symbolic or explicitly fictional rates for teaching. If the cheaper choice permits more overshoot or fails differently, show that difference. A reduced bill does not establish equal protection when the policy has changed.

22.2.19 · Alternative architectures and trade-offs

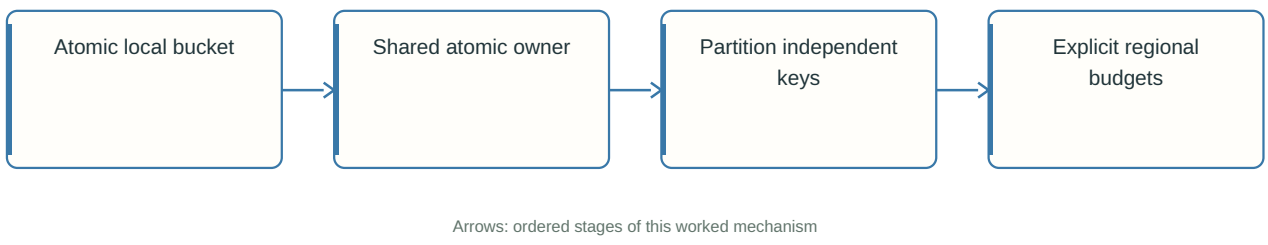
Token bucket admits bounded bursts. A leaky-bucket shaper smooths output by adding waiting. Fixed-window counters are simple but permit bursts around boundaries. Sliding approaches model recent traffic at additional state or computation cost.

Regional budgets trade precision and allocation work for local delay and availability. Choose according to the protected resource and acceptable behaviour, not a universally preferred algorithm name. Include what a caller sees when rejected and how its retries interact with the limit.

22.2.20 · Architecture evolution from small scale to large scale

First build a deterministic local bucket with an atomic owner. Move to shared state when several processes need one cap. Partition keys only when measured load requires it.

Introduce regional allocation for a named latency or failure need, with an explicit bound. Preserve the original policy meaning through each step or version the changed contract. Evolution is safest when each addition answers a demonstrated scope or capacity problem rather than silently multiplying the allowance.



Arrows: ordered stages of this worked mechanism

FIG. 22.2.4 **How the design can grow**

Move from local to shared admission only with an explicit scope and allocation rule.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.2.21 · Design review identifying weaknesses and improvements

Ask whether the advertised scope matches enforcement, whether a policy update creates unintended tokens, and what happens to the hottest key. These are protocol questions beyond the basic refill formula.

Test simultaneous spending of the last token, backward time, expiry, policy replacement, owner failure, and regional reconnection. Reject an exact-global claim implemented as full independent buckets. Record unresolved evidence instead of assuming a shared product name establishes all these behaviours.

22.2.22 · A short implementation exercise

Use an injected monotonic clock to build the sample bucket. Spend its ten initial tokens, reject an immediate eleventh spend, advance three seconds, and verify that six tokens have returned under capacity ten.

Reject invalid costs and ensure rollback in elapsed time adds no allowance. Then reproduce two unprotected readers spending one token and fix it with a coordinated transition. The test should show both the incorrect history and why the corrected ownership permits only one winner.

A small implementation of the key rule

PSEUDOCODE · ONE PART OF THE DESIGN

```
def spend(bucket, now, cost):
    require(cost > 0)
    elapsed = max(0, now - bucket.last)
    available = min(bucket.capacity,
                    bucket.tokens + elapsed * bucket.rate)
    allowed = available >= cost
    bucket.tokens = available - cost if allowed else available
    bucket.last = max(bucket.last, now)
    return allowed
# Entire transition executes atomically for this bucket.
```

Use the implementation exercise to test the rule and the failure described above. You can also run local examples of seat claims, duplicate handling, ledger balance, and workflow epochs in the mechanism collection.

THE WORKBENCH · TRY CHANGING THE INPUTS

Use a burst of tokens, then let the bucket refill

Virtual time: 0s · tokens: **10** of 10 · admitted: 0 · rejected: 0.

Each second adds two tokens, up to ten. Advancing three seconds adds at most six tokens.

The bucket holds at most ten tokens and gains two per second. Spend tokens, then advance the simulated clock. One browser owns this bucket. A shared implementation must update the balance atomically; several independent owners need a protocol to enforce a global limit.

Unit 22 exercises → · Unit 22 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. RFC 2697: A Single Rate Three Color Marker, 1999 ↗

   Original protocol description · Informational RFC. Token-based traffic metering example; atlas admission bucket is a simpler model.

   Access reviewed · 21 October 2026

2. Handling Overload — Google SRE ↗

   Engineering book. Admission, capacity, and overload response.

   Access reviewed · 21 October 2026

Chat Application

Prerequisites: Networking Foundations for System Design, Data Models and Storage Technologies, Distributed Systems Fundamentals, Message Queues, Events, and Distributed Workflows

4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A message can be safely stored while its live frame never reaches a device. Build chat around that distinction so reconnecting repairs delivery without creating another message.

The workload, timing, and prices in this design are teaching assumptions. They are not measured benchmarks or a description of a particular company.

22.3.1 · Functional requirements

Two people send messages in an authorised conversation. Online devices should receive updates, and a disconnected device should later fetch what it missed. The system also supports bounded groups and a defined retention policy.

Name the receipts carefully. A message can be accepted, persisted, delivered to a device, and read by a person at different times. Presence is only an approximate indication of connectivity. Opening a WebSocket supplies a live path, not evidence of human reading or durable history.

22.3.2 · Nonfunctional requirements

Retain acknowledged messages under the chosen storage-failure contract and protect conversation privacy. Define ordering per conversation or state a deliberately weaker alternative. Target online delivery below one second for the assumed supported workload.

A disconnected client catches up later, so its delivery delay differs from that target. Presence can expire approximately without being treated as a strict allocation invariant. Moderation, deletion, and offline history need their own retention and visibility rules, including what returning devices may fetch.

22.3.3 · Explicit assumptions and workload numbers

Assume 100,000 concurrent connections and 10,000 accepted messages/s. Each stored message averages 500 bytes and produces five recipient deliveries in the simplified workload.

Raw ingestion is 5 MB/s, or 432 GB/day. Five deliveries per message produce about 25 MB/s of body traffic before framing and overhead. Model device multiplicity, large groups, connection memory, and reconnect bursts separately. These assumptions do not establish the capacity of a particular gateway or database.

22.3.4 · Naive initial architecture

Begin with one connection server and one transactional history store. Save a message before returning its persistence receipt, then attempt delivery to online clients.

A gateway crash loses live connections but need not lose committed history. Clients reconnect and request missing messages. This first design makes ownership clear, while exposing the server's availability and capacity limits. A database commit establishes persistence under its contract; it does not establish that any particular device received the message.

22.3.5 · Core API contracts

A send operation includes conversation_id, client_message_id, and bounded content. A history request continues after a sequence, for example GET /conversations/{id}/messages?after=sequence.

Receipts identify a device or user and the highest relevant delivered or read sequence under the product's policy. Check membership on reads and writes. Reusing a client message identifier within its sender scope returns the earlier message. This lets a client recover a lost send reply without adding another history record for the same logical message.

22.3.6 · Data model and schema

Keep conversations and membership separately from message records. A message includes conversation_id, sequence, sender, client_message_id, body, and version. Receipt records track delivered and read progress for the appropriate user or device.

Enforce the intended client-identity uniqueness scope. Presence uses a separate expiring heartbeat record. Conversation order needs a sequence authority or a supported conflict model; timestamps from client clocks do not automatically provide a stable total order. State which identity remains valid through reconnect and retry.

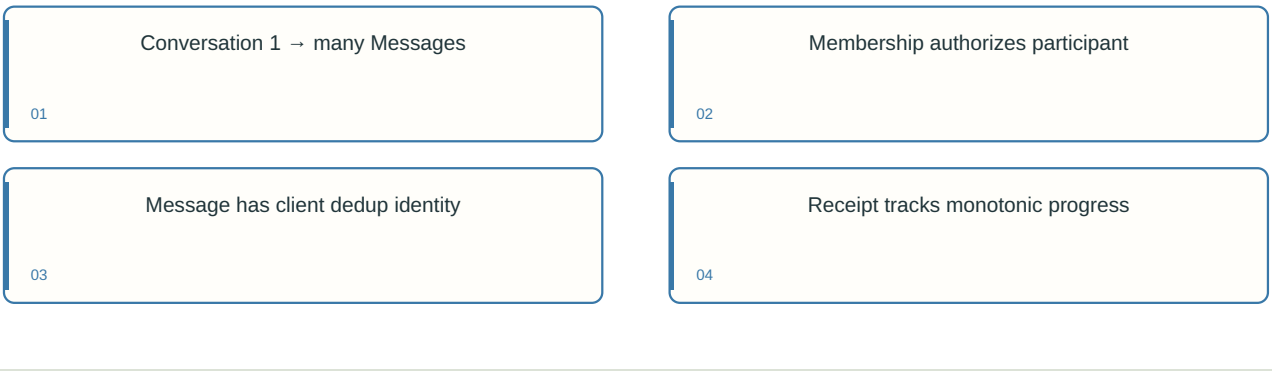


FIG. 22.3.1 **Data ownership and relationships**

History, membership, and receipt progress have different owners and lifetimes.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.3.7 · Low-level component organization

ConnectionGateway manages connections. MembershipPolicy checks access. MessageService coordinates identity and persistence, SequenceOwner establishes order, and HistoryRepository serves bounded pages. DeliveryWorker handles committed fan-out.

ReceiptAggregator combines progress without moving backward under the chosen device policy. Large-group work belongs in an explicit bounded operation instead of a long synchronous loop inside the message transaction. These components can begin together in one deployment. Their contracts should make clear which state is durable and which routing information may disappear.

22.3.8 · High-level system architecture

Use replaceable connection gateways and a conversation-routed durable message path. A log or outbox carries delivery intent to workers. Routing metadata identifies currently connected recipient devices, while history storage remains authoritative.

A route can be stale, so clients recover missing frames from their last known sequence. If publication intent commits with the message, a broker outage can delay live delivery without discarding acknowledged history. Gateway state is then a recoverable convenience for reaching a device, rather than the only record of what should be delivered.

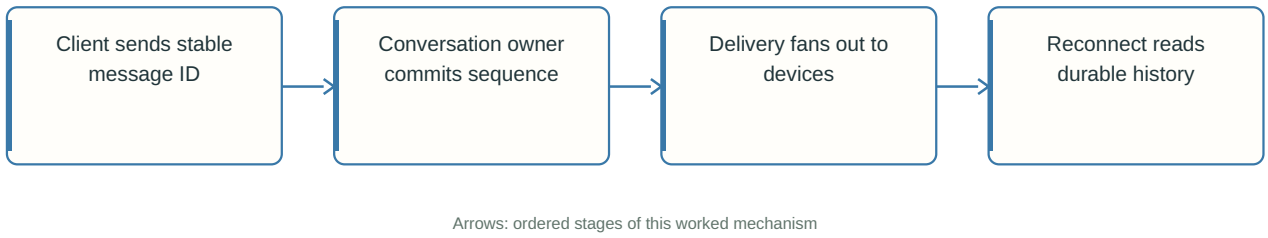


FIG. 22.3.2 **Follow the main operation**

Store the message before reporting persistence, then deliver and recover by identity.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.3.9 · Data and control flows

The client sends a stable message identity. The service checks membership, assigns order, and commits the message with its delivery intent. It then returns the persistence result.

Workers send live frames and process valid receipts. On reconnect, the client supplies its last observed sequence, fetches missing pages, and detects gaps. Repeated frames are recognised by message identity. This separates two questions: what the service has safely retained and what this device currently displays. Losing a live frame is repaired through history rather than assumed impossible.

22.3.10 · Capacity estimates

At 10,000 messages/s and 500 bytes each, raw storage grows by 5 MB/s, or 432 GB/day. Five recipients imply 50,000 deliveries/s and 25 MB/s of bodies.

Connection memory needs measurement. At an illustrative 20 kB per connection, 100,000 connections require 2 GB before other process memory and queued content. Bound outbound buffers for slow devices. A sender can otherwise make an idle-looking connection server run out of memory while the message database remains within its own limits.

22.3.11 · End-to-end latency estimates

Assume 20 ms ingress, 30 ms for commit coordination, 20 ms in delivery queues, and 80 ms on the recipient path. The sum is 150 ms before rendering.

This budget is not measured online p99. Test hot conversations, congestion, failover, and slow-client buffers. Offline delivery has a different timeline. Keep persistence-receipt delay, device-delivery delay, and human-read delay as distinct indicators. One successful transport response cannot establish all three outcomes.

22.3.12 · Failure scenarios

If the send reply is lost after commit, recognise the client's repeated identity. If a gateway stops, reconnect and recover from history. If a client is too slow, disconnect or reduce its service before buffers consume the gateway's resources.

A revoked membership can race with cached access information, so enforce the defined policy for delivery and history. Reconnect storms need admission and staggered retries. Test these event sequences directly rather than only testing two clients that stay connected throughout a normal exchange.

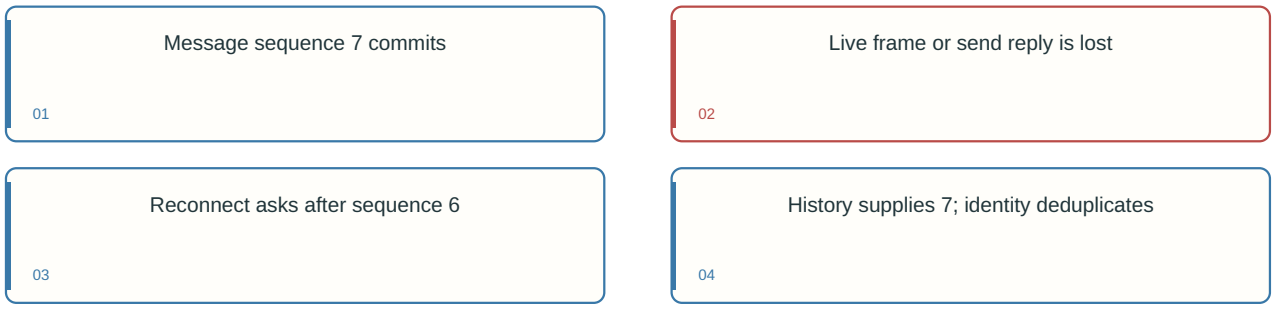


FIG. 22.3.3 **Follow the failure and its correction**

A missed live frame is recovered from history; repeating a send retains its original identity.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.3.13 · Scalability improvements

Distribute independent gateway connections and partition history and ordering by conversation where suitable. Use bounded fan-out workers for groups.

One extremely popular conversation can still concentrate ordering or distribution work. It may need different large-channel semantics rather than forcing every recipient into a low-latency acknowledgment chain. Preserve sequence and deduplication identity during ownership movement. Add capacity for the measured limit, with history catch-up included in the workload. Ordinary message averages do not account for every device returning after a gateway event.

22.3.14 · Consistency requirements

Provide a defined per-conversation order and a sender confirmation that agrees with the stored message. History catch-up must allow a client to resume without silently skipping required records.

Cross-conversation total order is unnecessary unless the product requires it. Frames can repeat, so devices deduplicate. Presence and receipt updates can propagate later, while delivered/read progress follows the chosen monotonic model. State the guarantees at these different boundaries instead of describing WebSocket delivery as exactly-once messaging.

22.3.15 · Security model

Check membership on send, history, fan-out, search, and media access. Limit abusive sending, group size, and payloads. Avoid putting message bodies into routine telemetry.

If end-to-end encryption is required, design device keys, recovery, and membership changes, and explain the resulting limits on server features. TLS protects a connection but does not implement that product guarantee. File attachments need access and retention rules too. Treat each content path as part of conversation privacy rather than assuming an authorised chat page protects every derived URL.

22.3.16 · Deployment approach

Drain gateways during rollout and support reconnecting old clients. Evolve message and receipt formats compatibly, and preserve history according to the selected replication and promotion rules.

Load-test reconnect and catch-up alongside normal sending. Restarting every gateway together can generate a larger burst than ordinary chat traffic. A new region also needs explicit ordering and membership behaviour. State how existing client identities and progress survive replacement so deployment does not turn uncertain sends into duplicated messages.

22.3.17 · Observability design

Measure persistence delay, live-delivery lag, missing sequence recovery, reconnect rate, fan-out backlog, and hot conversations. Observe slow-client limits and membership denials.

Correlate client and server message identities in suitable detailed evidence, without creating an unbounded metric label for every message. Synthetic clients can send, disconnect, reconnect, and check history and forbidden access. This tests the complete journey. A healthy connection count alone cannot show that an accepted message reached history or that a revoked device is blocked.

22.3.18 · Infrastructure cost considerations

Separate connection capacity, message history, fan-out transfer, indexes, replicas, media, and telemetry. Under the assumptions, thirty days of raw messages need 432 GB/day×30 = 12.96 TB before indexes and copies.

Media can dominate storage and transfer and needs its own quota and lifecycle. Use explicit rates or symbolic calculations rather than assumed provider prices. Cost per retained message and cost per delivered body explain different work, particularly when many recipients or devices multiply the live path.

22.3.19 · Alternative architectures and trade-offs

WebSockets supports bidirectional low-latency messaging with connection-management work. SSE with HTTP sends may fit some clients; polling is simpler but creates requests during idle periods.

A single conversation sequence authority simplifies order while creating a hot-key boundary. Conflict-tolerant regional writing changes that contract and needs a suitable merge and ordering model. Compare alternatives against the actual offline, receipt, and privacy requirements. The least complex transport that meets those behaviours can be preferable to one offering features the product never uses.

22.3.20 · Architecture evolution from small scale to large scale

First make durable history and reconnect recovery correct. Add gateway replicas with explicit connection routing. Introduce durable fan-out for higher volume or groups, then partition conversation ownership when storage or ordering measurements justify it.

Regional placement comes with another authority and failure decision. Preserve message identity at every step. A client retry should retain the same logical meaning after a shard move or gateway replacement. Evolution that loses this identity can duplicate messages even while increasing connection capacity.

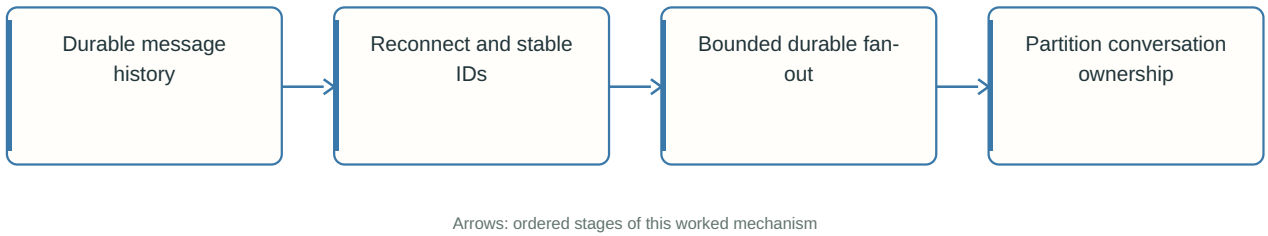


FIG. 22.3.4 **How the design can grow**

Scale connections and fan-out without losing per-conversation progress.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.3.21 · Design review identifying weaknesses and improvements

Review the meanings of “sent” and “delivered,” the hottest conversation, slow-client memory, and revocation. Each needs a specific supported history.

Test commit followed by a lost send reply, a dropped frame, gateway loss, a stale route, and a history read after membership is removed. Confirm the resulting list and receipts. A demo with two permanent online connections proves little about catch-up, privacy during change, or the storage contract that makes messages durable.

22.3.22 · A short implementation exercise

Build a client model that recognises message identities and detects sequence gaps. Save a message but suppress its live frame. Reconnect from the earlier sequence and verify that history supplies it once in the visible list.

Repeat the same client_message_id and check that storage does not add another logical message. Distinguish deduplicating displayed frames from deduplicating history creation: both are useful, and each protects a different boundary.

A small implementation of the key rule

PSEUDOCODE · ONE PART OF THE DESIGN

```
with transaction():
    existing = find_by_client_id(sender, client_message_id)
    if existing: return existing
    require_member(sender, conversation)
    seq = allocate_conversation_sequence(conversation)
    msg = insert_message(conversation, seq, body)
    insert_outbox(msg.id, "deliver")
return persisted_receipt(msg)
# Device delivery and read receipt happen later.
```

Use the implementation exercise to test the rule and the failure described above. You can also run local examples of seat claims, duplicate handling, ledger balance, and workflow epochs in the mechanism collection.

Unit 22 exercises → · Unit 22 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. RFC 6455: The WebSocket Protocol, 2011 ↗

   Original standard. Bidirectional transport framing; not durable chat delivery.

   Access reviewed · 21 October 2026

2. Apache Kafka 4.1: Design ↗

   Official versioned documentation. Partitioning and scoped transactional/processing behavior; external effects are separate.

   Access reviewed · 21 October 2026

3. PostgreSQL 18: Transaction Isolation ↗

   Official versioned documentation. Specific PostgreSQL isolation behavior; no all-database equivalence claim.

   Access reviewed · 21 October 2026

Notification Platform

Prerequisites: Design Patterns, Dependency Injection, and Object Lifecycles, API Design and Communication Contracts, Message Queues, Events, and Distributed Workflows, Reliability, Fault Tolerance, and Disaster Recovery

4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

The notification provider may accept a message just before its reply disappears. Track intent, attempts, and evidence so retries do not become unsupported delivery claims.

The workload, timing, and prices in this design are teaching assumptions. They are not measured benchmarks or a description of a particular company.

22.4.1 · Functional requirements

An application submits a notification intent. The service selects permitted channels, renders a versioned template, sends through email, SMS, or push, and tracks what happened. Some messages may be scheduled or cancelled within a defined window.

Provider acceptance, device delivery, and human viewing are separate outcomes. Preference rules can also differ by message purpose. Keep that policy explicit rather than letting a channel adapter decide whether a particular user may receive an operational or product message.

22.4.2 · Nonfunctional requirements

Preserve accepted intents and make delayed, expired, or permanently failed work visible. Define a business expiry: a password-reset message delivered hours late may be useless or harmful.

Isolate critical and bulk work so a marketing batch does not consume every delivery resource. Avoid duplicate relevant effects where provider contracts support that promise, and expose residual uncertainty

where they do not. The service should state the outcome it can establish rather than equating a submitted request with a delivered message.

22.4.3 · Explicit assumptions and workload numbers

Assume ten million intents/day. Dividing by 86,400 gives about 115.7/s average. An assumed tenfold peak is about 1,157/s. At 1 kB of initial intent and attempt metadata per item, raw growth is 10 GB/day.

Repeated attempts, indexes, copies, and retained content add costs. Channel quotas are separate configured inputs that must be checked with the actual provider. An overall average cannot establish the submission capacity or delay of a specific route.

22.4.4 · Naive initial architecture

A first version calls one provider directly after a user action. It is simple, but the user's request now waits for that provider. If the business action commits and the process dies before sending, the notification intent can disappear.

Persist required intent in the business transaction or an outbox, then let a worker send it. This separates accepted work from a temporary provider problem. It adds queued state and recovery rules, which should be visible rather than hidden inside an untracked background call.

22.4.5 · Core API contracts

POST /notifications accepts the recipient, template version, variables, channel policy, expiry, and idempotency key. GET /notifications/{id} reports a state such as pending, submitted, delivered, expired, failed, or cancelled.

Define cancellation's boundary: it cannot recall a message already accepted by a provider. Callback endpoints verify authenticity and recognise repeated events. State transitions must reflect provider evidence. A delivered label should not merely mean that a worker returned without an exception.

22.4.6 · Data model and schema

Store the logical intent, template version, relevant preference information, channel/recipient delivery tasks, attempts, and provider-event inbox. Uniqueness binds each intended effect to its stable identity under the chosen policy.

Keep provider references and uncertain outcomes. Retain sensitive message content separately from minimal progress or audit metadata. Decide what a new preference setting means for already queued tasks. Consent or cancellation rules may require a current check at send time rather than relying only on the submission snapshot.

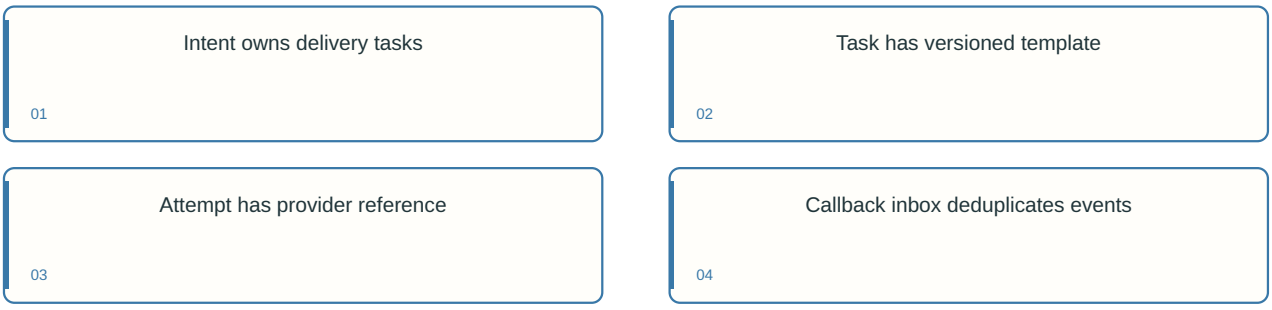


FIG. 22.4.1 **Data ownership and relationships**

Intent, delivery tasks, attempts, and callbacks record different stages of the notification.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.4.7 · Low-level component organization

NotificationService records intent. PreferencePolicy selects allowed channels. TemplateRenderer builds the message, SenderStrategy chooses channel behaviour, and ProviderAdapter maps its protocol and outcomes. DeliveryRepository owns durable attempts and progress.

Metrics decorators and factories can support genuine variation without changing result meanings. Keep construction and resource lifetimes clear. A retry wrapper needs more than the same method name: it must preserve operation identity and account for uncertainty about a prior submission.

22.4.8 · High-level system architecture

Use durable ingress, preference and template access, channel-specific queues, bounded workers, callback inboxes, and an outcome store. Separate resources for critical and bulk traffic where their priorities require it.

A failed provider should occupy only its allocated capacity. A queue helps recovery only when intent, attempts, acknowledgment, and progress records agree about ownership. Draw these state boundaries alongside the normal sending path so it is clear what survives a worker or broker interruption.

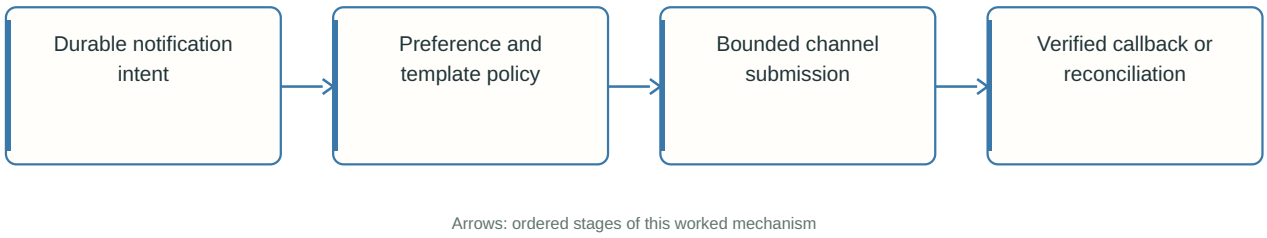


FIG. 22.4.2 **Follow the main operation**

Durable intent reaches a bounded channel worker and a provider with explicit outcome meanings.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.4.9 · Data and control flows

Accept and deduplicate the intent, choose permitted channel and template, and persist the delivery task. A worker checks its current authority, expiry, and required preference policy before submission.

Use a stable provider operation identity where supported. Save the returned result or an uncertain state, then acknowledge according to the task protocol. Apply verified callbacks idempotently, with rules preventing an older event from incorrectly reversing progress. An attempt and a callback can arrive out of order, so state transitions need evidence and identity rather than arrival order alone.

22.4.10 · Capacity estimates

Overall average traffic is about 115.7 intents/s, but each channel has its own capacity. Suppose SMS arrivals are 100/s and its route submits 40/s. Backlog grows by 60/s.

Over ten minutes that adds 60×600 = 36,000 waiting submissions if the rates persist. Model retries, recipient fan-out, and expiry too. The queued work may become useless before capacity catches up. Admission, priority, or supported alternate routes must respond to the channel's real limit rather than the service-wide daily average.

22.4.11 · End-to-end latency estimates

Assume acceptance takes 50 ms, queue wait 2 seconds, and provider submission 500 ms. Provider acceptance then occurs around 2.55 seconds after submission to our API.

The short API latency and delivery journey are different measurements. These inputs are illustrative budgets, not measured percentiles. Actual delivery and human viewing depend on further systems and evidence. Do not advertise a 50 ms notification outcome merely because durable acceptance was quick. Track the completion event that matters for the message's purpose.

22.4.12 · Failure scenarios

A worker sends a message, then dies before recording the provider reply. The provider may already have accepted it. Query or use provider idempotency where available; otherwise state the remaining duplicate risk.

Malformed templates need review, outages need bounded retries and expiry, and callbacks need authenticity checks. Reconcile old submitted or uncertain tasks. A second provider does not automatically resolve uncertainty: sending there can create another message if the first provider's effect already occurred.

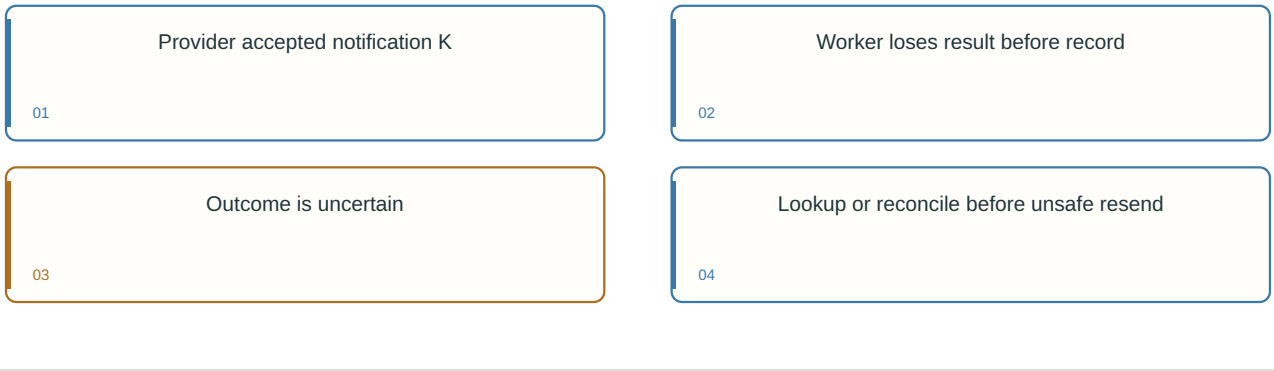


FIG. 22.4.3 **Follow the failure and its correction**

A sent message with a lost reply is uncertain, so recovery needs provider evidence or a stated duplicate policy.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.4.13 · Scalability improvements

Distribute independent recipients and channel tasks, with bounds and any required per-recipient order. Add routes only with a deliberate selection, rate, and uncertainty policy.

Extra providers can improve some outage paths while increasing content and reconciliation work. A timeout followed by route switching can produce two messages. Measure the actual channel bottleneck before adding workers, and keep downstream quotas and critical-message capacity visible. Safe progress is the goal, rather than maximum parallel submissions regardless of expiry or duplicate effects.

22.4.14 · Consistency requirements

Persisting intent and the deduplication result needs an atomic local boundary. Task deliveries may repeat, while local transitions recognise stable identities. Provider acceptance and final delivery have separate evidence.

Retain the template version so a retry does not silently send different content. Apply the chosen current-preference policy at the correct boundary. A general exactly-once label would hide the different guarantees supported by email, SMS, and push providers. State which local record or external effect the specific mechanism protects.

22.4.15 · Security model

Authorise which actor may send which template to which recipients. Validate variables and escape or encode them appropriately for the target channel. Protect recipient data, provider credentials, and callback authenticity.

Avoid leaking tokens or message bodies through logs. Limit intent creation to prevent abuse. Signed links inside messages need an appropriate audience and lifetime; the delivery system should not broaden their capability accidentally. Include adapter-level protections such as preventing unsafe header or content construction.

22.4.16 · Deployment approach

Deploy workers with durable attempt state and graceful draining. Scale-down must not abandon unrecorded work. Template edits create new versions rather than changing content under an existing retry identity.

Roll task schemas out compatibly and plan credential rotation. Test deployment during a provider outage so accumulated work does not overwhelm the recovering route. Verify that critical-channel and acceptance policies still work while another channel is saturated or being replaced.

22.4.17 · Observability design

Track accepted intents, queue age by channel, expiry, provider submission delay, uncertain outcomes, callbacks, and duplicate recognition. Separate critical and bulk outcomes so an apparently good total cannot hide important delayed messages.

Alert on old unresolved tasks and rapid budget consumption. A growing queue without worker exceptions can still be a delivery incident. Correlate attempts and provider evidence while keeping private content out of general telemetry. Enqueue success is one stage of the journey, not its final outcome.

22.4.18 · Infrastructure cost considerations

Include intent and attempt storage, provider-submitted units, workers, callbacks, transfer, and operating effort. Retries can create billable work even when they do not produce a useful delivery.

Model channel_unit_price×submitted_units using explicit inputs. Content length and character encoding may change SMS billing under the provider contract, which must be checked rather than assumed. Compare cost per accepted intent and per relevant successful delivery. They explain different parts of the service and should not be mixed in one denominator.

22.4.19 · Alternative architectures and trade-offs

Direct synchronous sending has little infrastructure but ties the user path to the provider. Durable queued delivery adds recoverable state and a separate delay. Orchestration can make multi-channel escalation explicit; event reactions may distribute coordination but make total progress harder to see.

A backup provider adds another recovery option and another uncertainty boundary. Choose using message expiry, criticality, and the evidence each provider supplies. A solution that eventually sends something is insufficient when the message must be timely, permitted, and meaningfully tracked.

22.4.20 · Architecture evolution from small scale to large scale

Start with one sender behind a contract. Add durable intent and a worker when required work must survive failure or leave the user's latency path. Add channel strategies and factories for real variation.

Separate critical and bulk resources as volume and harm justify it. Provider failover and escalation come after the uncertain-outcome policy is explicit. Preserve stable task, template, and attempt identities through these changes, rather than treating scaling as permission to create fresh effects for the same intent.

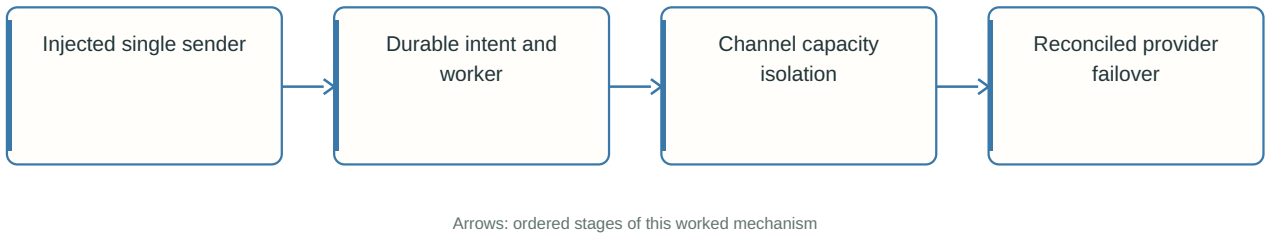


FIG. 22.4.4 **How the design can grow**

Add channels and routes after defining their expiry, priority, and retry behaviour.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.4.21 · Design review identifying weaknesses and improvements

Review whether “delivered” overstates the evidence, whether expiry and current preferences are checked, and whether failover can duplicate a message. These are product and protocol questions.

Test send-then-crash, forged or repeated callbacks, preference revocation after enqueue, template updates, and channel saturation. Confirm terminal progress does not regress under late events. A dashboard of successful submission requests cannot establish that the intended people received timely messages under the chosen policy.

22.4.22 · A short implementation exercise

Build a recording sender that stores a submission but returns uncertainty. Process the same task again and inspect the local attempt identities and resulting state. It must not invent evidence of delivery.

Add expiry and current-preference checks. Then switch provider after the uncertain attempt and show why this can produce a second external effect. Implement only the recovery behaviour your mock provider contract supports, and make any residual duplicate risk explicit in the result.

A small implementation of the key rule

PSEUDOCODE · ONE PART OF THE DESIGN

```
def deliver(task, now):
    require_current_attempt(task)
    if now >= task.expires_at: return mark_expired(task)
    if not permitted_now(task): return mark_cancelled(task)
    sender = sender_factory.for_channel(task.channel)
    result = sender.send(task.stable_provider_key, render(task))
    if result.uncertain:
        return persist_uncertain(task, result.reference)
    return persist_submission_result(task, result)
# A provider acknowledgment is not a human-read receipt.
```

Use the implementation exercise to test the rule and the failure described above. You can also run local examples of seat claims, duplicate handling, ledger balance, and workflow epochs in the mechanism collection.

Unit 22 exercises → · Unit 22 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Inversion of Control Containers and the Dependency Injection pattern — Martin Fowler, 2004 ↗

   Original engineering article. Dependency injection and wiring.

   Access reviewed · 21 October 2026

2. Transactional Outbox — Chris Richardson ↗

   Pattern author's explanation. Publication intent and duplicate delivery.

   Access reviewed · 21 October 2026

3. Apache Kafka 4.1: Design ↗

   Official versioned documentation. Partitioning and scoped transactional/processing behavior; external effects are separate.

   Access reviewed · 21 October 2026

Social Media Feed

Prerequisites: Data Models and Storage Technologies, Caching and Content Delivery, Scalability, Load Balancing, and Data Distribution, Message Queues, Events, and Distributed Workflows

4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Preparing every follower feed makes reads cheaper but multiplies writes. Compare where the work belongs while keeping private and deleted posts out of every served page.

The workload, timing, and prices in this design are teaching assumptions. They are not measured benchmarks or a description of a particular company.

22.5.1 · Functional requirements

Users publish posts, follow accounts, and read a ranked home feed. The feed must support bounded continuation and obey privacy, blocking, moderation, and deletion rules.

Ranking chooses among posts the viewer is allowed to see. A high score cannot override denial. Engagement counts and candidate lists can update later than the original post, but the product needs to state what freshness those derived views permit. Separate finding interesting content from deciding which content is authorised.

22.5.2 · Nonfunctional requirements

Assume a 300 ms p99 feed target under the supported workload. Retain accepted posts and define how quickly access changes must take effect. Pagination should have understandable duplicate and omission behaviour.

Ranking can change between refreshes. Decide whether each refresh begins a new generation and what a continuation cursor preserves. Cached candidates may be old, so specify which properties are rechecked

at serving time. These conditions are part of a useful feed response alongside its latency.

22.5.3 · Explicit assumptions and workload numbers

Assume ten million daily readers, twenty feed reads each, and one million posts/day. The read total is 200 million/day, or about 2,315/s on average.

For a simplified ordinary-author model, assume one hundred followers per author and 32 bytes per materialised candidate reference. Fan-out creates one hundred million references/day, or 3.2 GB raw. A celebrity with millions of followers is a different traffic class. The average cannot describe that author's burst or inactive followers' unused work.

22.5.4 · Naive initial architecture

Begin with indexed posts and follow relationships, building the feed when requested. This avoids writing a copy for every follower and keeps the initial system small.

As users follow more authors, candidate retrieval and ranking can require too much work per read. Measure those costs and the follower distribution. Materialising every feed immediately can replace read pressure with large unused write pressure. Introduce that trade-off only when the repeated read work justifies maintaining another representation.

22.5.5 · Core API contracts

POST /posts creates a versioned post; POST /follows changes the relationship. GET /feed returns a bounded page, continuation cursor, and relevant ranking generation. DELETE /posts/{id} records authoritative removal.

Bind cursors to the viewer and applicable filters, and explain their snapshot or continuation semantics. A cursor does not grant continuing permission to content after access changes. Validate it and reapply the required visibility policy whenever the next page is served.

22.5.6 · Data model and schema

Post records include identity, author, version, visibility, state, and creation time. Follow records connect follower and author. Optional feed candidates connect a reader to a post and source version with ranking information.

Keep model version or generation where reproducibility needs it. Candidates are derived references, while posts and access relationships remain authoritative. Stable source identity lets workers recognise repeated work, remove or suppress old results, and rebuild views. State whether copied features are snapshots or values expected to follow updates.

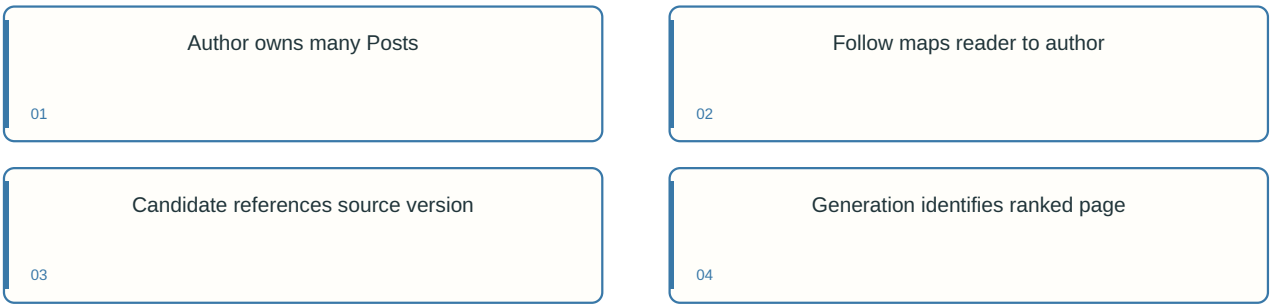


FIG. 22.5.1 **Data ownership and relationships**

The post and access relationships are authoritative; feed candidates are derived references.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.5.7 · Low-level component organization

PostService owns publication and FollowService owns relationship changes. CandidateGenerator retrieves or prepares references. FeatureFetcher supplies bounded ranking inputs, Ranker scores them, and VisibilityFilter enforces access. FeedAssembler combines results, removes duplicates, and produces pages.

Keep the access decision independent of desirability. Missing features can change order; they must not broaden visibility. These roles let performance work focus on candidate or feature costs while preserving a separate rule for what the viewer may receive.

22.5.8 · High-level system architecture

Use a durable publication path with an outbox. Workers prepare candidates for ordinary authors; high-fan-out authors can be pulled at read time. Candidate storage, ranking compute, and permitted caches complete the serving path.

This hybrid avoids immediately writing to every celebrity follower while making common reads cheaper. It adds strategy-selection and reconciliation work. Separate model deployment decisions from the live data path, and preserve source state and access checks when cached or materialised results lag.

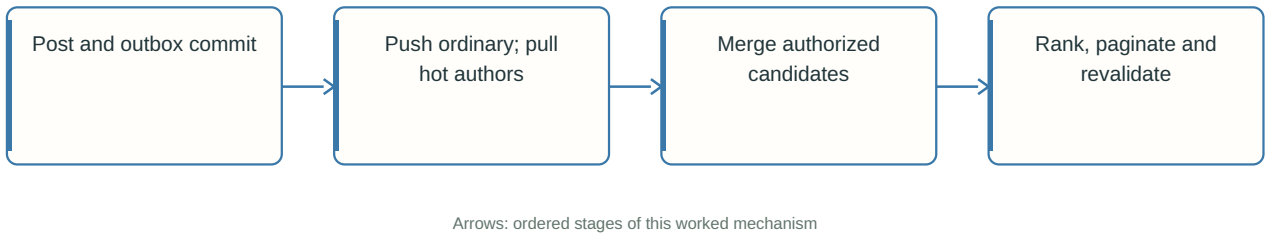


FIG. 22.5.2 **Follow the main operation**

Combine candidates, enforce visibility, rank, and return a bounded page.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.5.9 · Data and control flows

Publication commits the post and fan-out intent. Workers add candidate references under stable identity. A read combines prepared and pulled candidates, checks current required permissions and post state, obtains bounded features, ranks, deduplicates, and returns a continuation.

Track the generation involved. A deleted candidate may physically remain while serving correctly rejects it. That is acceptable only under the chosen deletion policy. Storage cleanup and permission to expose the reference are separate events with different completion requirements.

22.5.10 · Capacity estimates

The assumed 200 million daily reads average about 2,315/s. With an illustrative fivefold peak, the rate becomes about 11,574/s. One million posts times one hundred followers produces one hundred million candidate writes/day.

A single post by an author with ten million followers creates a separate burst. Also estimate candidates ranked per read and feature-lookup amplification. API rates alone miss much of the storage and compute work, especially when an apparently small feed request fans out to many dependencies.

22.5.11 · End-to-end latency estimates

Assume 20 ms at the edge, 50 ms retrieving candidates, 80 ms fetching features, 70 ms ranking, and 30 ms preparing the response. The proposed path sums to 250 ms before additional tail and queue allowance.

Measure against the 300 ms objective rather than calling this sum p99. A required slow feature call can dominate the request. Define a safe missing-feature response, and measure publication freshness separately from read latency. Fast reads can still show an outdated or empty feed.

22.5.12 · Failure scenarios

A worker can repeat fan-out after a crash, so candidate identity must suppress duplicate effects. A celebrity burst needs the selected pull or controlled distribution path rather than unlimited queue growth.

Candidate cleanup can lag privacy changes, making current serving checks important. If ranking fails, an authorised chronological feed may be an acceptable fallback. It must still reject deleted or forbidden content. Test the fallback's semantics as well as its speed; returning something quickly does not make it a valid feed.

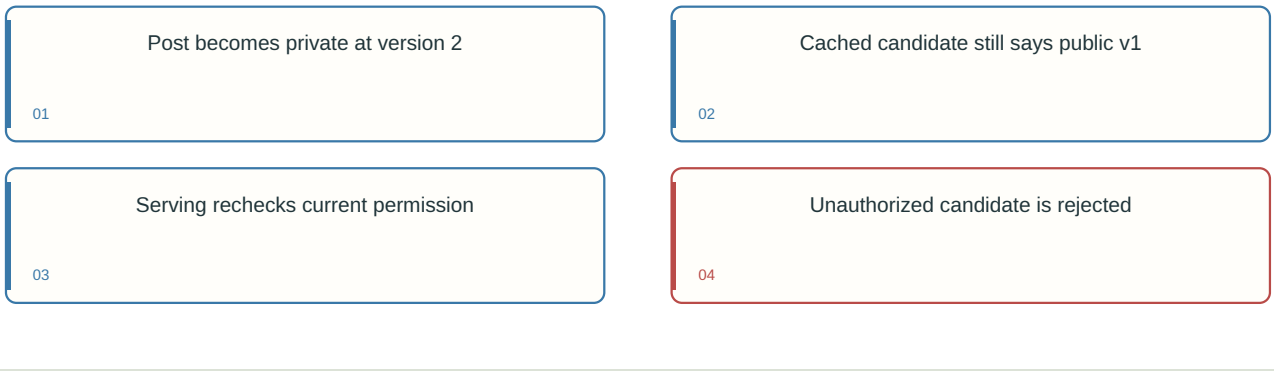


FIG. 22.5.3 **Follow the failure and its correction**

Repeated fan-out and stale candidates must not create duplicate or forbidden visible posts.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.5.13 · Scalability improvements

Partition candidate work by a measured ownership key, batch feature reads, and cache values within their freshness scope. Bound how many candidates a read retrieves and ranks. Give rebuilds separate budgets so they do not consume all live-serving capacity.

Do not increase workers past storage capacity. Choose the hybrid threshold from observed write amplification, reader activity, and read cost. Moving an author between push and pull needs a transition that prevents missing or duplicated candidates. A threshold is a design parameter to revisit, not a universal follower count.

22.5.14 · Consistency requirements

Post identity, updates, and deletion need a suitable local transactional authority. Feed projections and engagement totals may propagate later. An author expecting to see their own new post may need a direct-source overlay or another supported path.

Access policy must still meet its own freshness rule when candidates are stale. Specify which parts of a page belong to a ranking snapshot and which are revalidated. The feed has several consistency boundaries, so one label cannot describe all its accepted observations.

22.5.15 · Security model

Enforce author permissions, viewer visibility, blocks, and moderation on each relevant path. Validate cursors, bound scraping and feature-query amplification, and keep client-provided scores from influencing permission.

Ranking logs can contain private behaviour, so retain only justified information with controlled access. Propagate required deletion into feature and analytical copies as well as post storage. The model's input and output paths need the same tenant and content-scope discipline as the main feed response.

22.5.16 · Deployment approach

Version rankers and use representative canary cohorts, checking rollback compatibility with stored features. Roll fan-out formats out so mixed-version readers remain valid.

Keep rebuilding capacity separate from live publication and reads. A region-local feed needs a defined route for source and access-policy changes before it can promise both locality and the required privacy behaviour. Model release is another kind of serving change, so observe it with the same care as application code and schema deployment.

22.5.17 · Observability design

Track publication-to-candidate delay, candidate counts, denied visibility, read latency, feature tails, model version, duplication, and hot-author backlog. Measure relevance and useful content separately from infrastructure timing.

A low p99 does not establish that pages contain correct, varied, current results. Pair operational indicators with suitable content and privacy checks. Detailed evidence should identify generations and source versions without making every post or user an unbounded metric label.

22.5.18 · Infrastructure cost considerations

Costs include candidate-write amplification, feature storage and reads, ranking CPU, responses, and rebuilds. The simplified 3.2 GB/day of raw references excludes indexes and other metadata.

Compare push and pull using how many followers actually read. Preparing feeds for inactive users can spend work that is never used. Use explicit fictional rates or current verified quotes. A cost comparison is meaningful only when visibility, retention, freshness, and recovery requirements are equal.

22.5.19 · Alternative architectures and trade-offs

Fan-out-on-write spends more when posts arrive to make reads cheaper. Fan-out-on-read avoids unused preparation but adds work to each read. A hybrid targets different traffic classes while adding policy and transition complexity.

Precomputed ranking can reduce serving CPU and introduce another freshness delay. Compare these choices against actual active-reader ratios and author skew. A lower average read time obtained by weakening deletion or privacy is a different contract. Keep that distinction visible when evaluating the trade-off.

22.5.20 · Architecture evolution from small scale to large scale

Start with indexed pull reads, then add bounded ranking and useful caching. Materialise ordinary-author candidates when repeated read work becomes the limiting cost. Introduce a separate high-fan-out path after observing skew.

Regional copies require source-version and privacy-update rules. Revisit thresholds as posting, reading, and follow distributions change. Stable candidate identity and explicit generation boundaries should survive each stage so the new arrangement preserves continuation and duplicate behaviour.

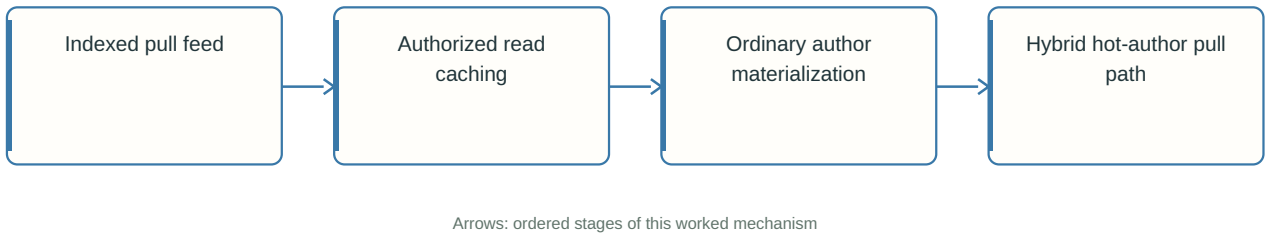


FIG. 22.5.4 **How the design can grow**

Prepare common feeds only where measured reading saves more work than publication adds.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.5.21 · Design review identifying weaknesses and improvements

Test the assumptions about inactive readers, high-degree authors, and revoked access. Place a post in cached candidate pages, then delete it or make it private. Verify that later serving follows the required rule.

Replay a fan-out event, pause a worker, and disable a feature dependency. Check both the resulting feed and fallback access. A drained queue does not prove that every intended reader received one usable authorised candidate; validate the identities and source versions involved.

22.5.22 · A short implementation exercise

Build push and pull candidate generation over a small follow graph with one very high-degree author. Count writes and reads for explicit user journeys rather than relying only on averages.

After materialising candidates, delete a post and check the serving filter. Replay a fan-out event and verify there is no visible duplicate. Explain which strategy spends work for inactive readers and which moves it into active reads, keeping privacy checks identical in both.

A small implementation of the key rule

PSEUDOCODE · ONE PART OF THE DESIGN

```
def feed(reader, cursor):
    candidates = materialized(reader) + pull_hot_authors(reader)
    unique = deduplicate_by_post_id(candidates)
    permitted = [p for p in unique if visible_now(reader, p)]
    features = fetch_bounded_features(permitted)
    ranked = rank(permitted, features, model_version)
    return page(ranked, validated_cursor(cursor, reader))
```

Use the implementation exercise to test the rule and the failure described above. You can also run local examples of seat claims, duplicate handling, ledger balance, and workflow epochs in the mechanism collection.

Unit 22 exercises → · Unit 22 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. RFC 9111: HTTP Caching, 2022 ↗

   Original standard. HTTP cache semantics; application cache races are separate examples.

   Access reviewed · 21 October 2026

2. Application best practices — Microsoft Architecture Center ↗

   Official guidance · accessed 21 Oct 2026. Scaling and application boundary review.

   Access reviewed · 21 October 2026

3. Apache Kafka 4.1: Design ↗

   Official versioned documentation. Partitioning and scoped transactional/processing behavior; external effects are separate.

   Access reviewed · 21 October 2026

Video Streaming Platform

Prerequisites: Networking Foundations for System Design, Caching and Content Delivery, File Systems, Search Infrastructure, and Data Pipelines, Deployment, Cloud Infrastructure, and Platform Architecture

4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

An encoder has written two of three required segments. Decide what makes the video ready, how a retry resumes, and which references a viewer is permitted to fetch.

The workload, timing, and prices in this design are teaching assumptions. They are not measured benchmarks or a description of a particular company.

22.6.1 · Functional requirements

An owner uploads a video, the service validates and converts it, and viewers receive an authorised playback manifest and media segments. Support resumable uploads, processing progress, and deletion.

Adaptive playback chooses among available renditions as conditions change. It cannot correct a missing segment or a manifest published before its required outputs exist. The product therefore needs both a byte-transformation workflow and a clear decision that a particular generation is ready to play.

22.6.2 · Nonfunctional requirements

Preserve original and required derived media according to stated durability and retention rules. Set playback-start and smoothness objectives for supported clients, measured through the delivery journey.

Asynchronous processing must expose failed and review states, not leave users waiting on an unexplained spinner. Protect private manifests, segments, thumbnails, and download links as well as the player page. A private metadata row cannot compensate for public media objects that anyone can fetch directly.

22.6.3 · Explicit assumptions and workload numbers

Assume 1,000 uploads/day, each averaging 1 GB and thirty minutes. Raw input is 1 TB/day and 500 source-hours/day. Assume the chosen renditions produce five output GB for each source GB, giving 5 TB/day derived data.

For 10,000 simultaneous viewers averaging 3 Mb/s each, delivery is 30 Gb/s, or 3.75 GB/s, before overhead. Encoding, retention, and playback therefore impose different capacity needs. These ratios are teaching assumptions rather than codec or infrastructure benchmarks.

22.6.4 · Naive initial architecture

A prototype can accept an upload and have one worker produce one playback file. It is simple but places large transfer and computation behind fragile local execution and offers little adaptation to changing networks.

Direct upload to storage removes much of the byte traffic from application workers. Durable encoding tasks preserve required progress. Begin with a small supported rendition set. Producing every possible format immediately adds compute and storage that may never be useful to the intended viewers.

22.6.5 · Core API contracts

POST /videos creates an upload intent. Scoped multipart authorisation permits the byte transfer. A completion operation validates the stored content, a status endpoint reports processing, and a playback endpoint returns an authorised manifest reference.

Deletion tracks the required cleanup. Define the scope, lifetime, and cache behaviour of playback capabilities. An old page URL should not silently grant indefinite access. The status contract must distinguish upload complete, processing, playable, and failed rather than treating every accepted byte as ready media.

22.6.6 · Data model and schema

Store video identity, owner, source version, and state. Upload sessions track unfinished transfers. Encoding jobs identify input digest, profile version, and attempt epoch. Renditions identify profile, version, object key, and checksum; a manifest generation names its required artifacts.

Publish only when that set is verified. Do not change bytes behind an immutable published reference without a version transition. Link derived objects to their source and owner so deletion, repair, and inventory

reconciliation can find the whole media lifecycle.

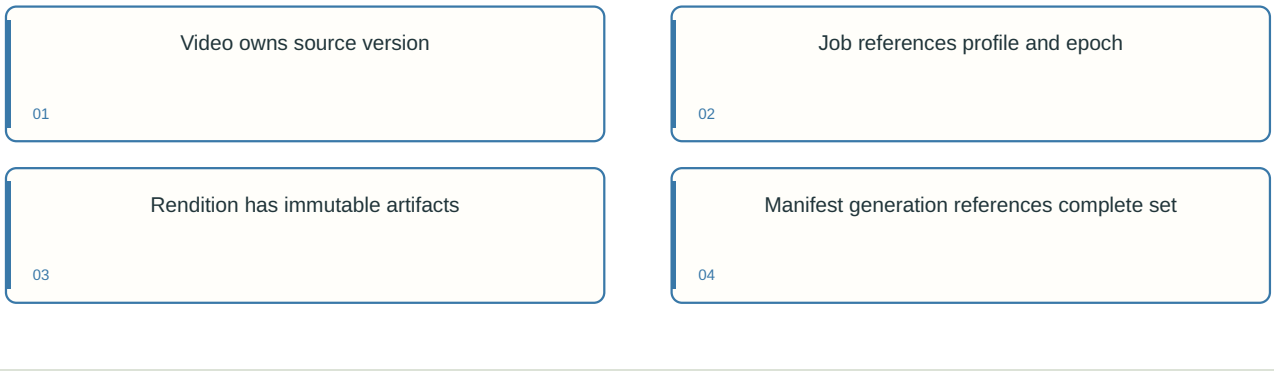


FIG. 22.6.1 **Data ownership and relationships**

A published manifest generation names identified, verified outputs from a particular input and profile.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.6.7 · Low-level component organization

UploadService owns upload permission. MediaValidator checks supported bounded input. EncodingPlanner chooses profiles, WorkerLease manages attempts, and TranscoderAdapter performs isolated conversion. ArtifactPublisher advances published metadata when the required outputs are verified.

PlaybackService checks access and issues scoped delivery references. Keep transformation separate from publication. A worker can finish writing one file without making the video's complete generation playable. The metadata transition needs evidence about the entire required artifact set.

22.6.8 · High-level system architecture

Use object storage for immutable sources and segments, a durable workflow and task queue, isolated transcoders, metadata storage, and an edge delivery path. The application handles control and access while large bytes flow directly through storage and CDN facilities.

Keep source and derived retention policies explicit. Untrusted media parsers need limited resources and suitable execution and egress permissions. The source store, workflow state, and published generation each have recovery responsibilities that a task queue alone does not supply.

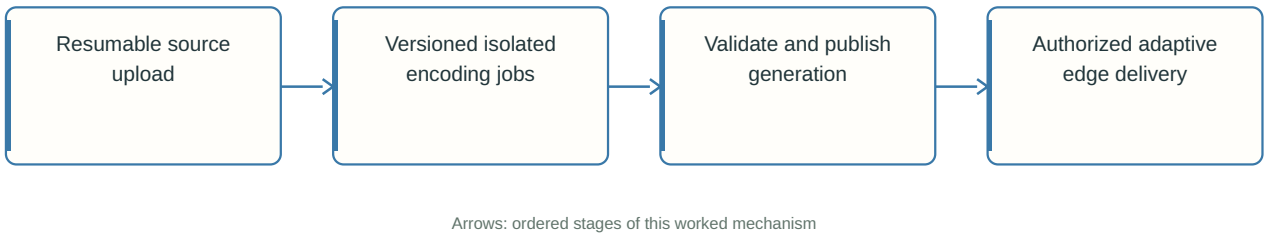


FIG. 22.6.2 **Follow the main operation**

Uploaded bytes become playable only after required transformation and publication checks.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.6.9 · Data and control flows

Validate completed upload identity, choose a versioned encoding plan, and produce artifacts under an attempt identity. Verify the required segment and manifest set, then atomically publish the playable metadata generation.

Failed attempts leave unpublished outputs for cleanup or safe reuse. A client obtains an authorised manifest and fetches supported renditions through the delivery path. Marking ready after the first output is valid only if progressive publication is an explicit supported contract. Otherwise it exposes a partially usable video as complete.

22.6.10 · Capacity estimates

Thirty days under the assumptions need 30 TB of original data plus 150 TB of renditions before extra copies and overhead. Viewer delivery at 30 Gb/s is far larger than the metadata API's transfer.

Encoding cost needs measured source-duration and codec ratios. If an assumed two compute-hours are needed per source-hour, 500 source-hours/day require 1,000 compute-hours/day. This is a model input, not a performance result. Use real content and profiles to measure the ratio before sizing workers.

22.6.11 · End-to-end latency estimates

Assume 50 ms for playback authorisation, 30 ms for manifest delivery, and 800 ms for initial buffering. That path totals 880 ms in the example.

A cache miss or a constrained client changes it. Measure startup, rebuffering, rendition switches, and failures independently of the metadata API's p99. The viewer's experience includes segment delivery and player behaviour. A fast permission endpoint establishes little about a missing or slowly delivered first segment.

22.6.12 · Failure scenarios

An upload stops halfway. An input is corrupt. A worker writes two segments and crashes before the third. A manifest points to a deleted object. Each needs a distinct state and supported repair path.

Keep unfinished generations unpublished and validate the final set. Protect the origin if the edge path fails; fallback can suddenly send enormous video traffic to it. Deletion and revoked access must affect playable references and copies according to the chosen capability and cache policy. Test through the actual media path.

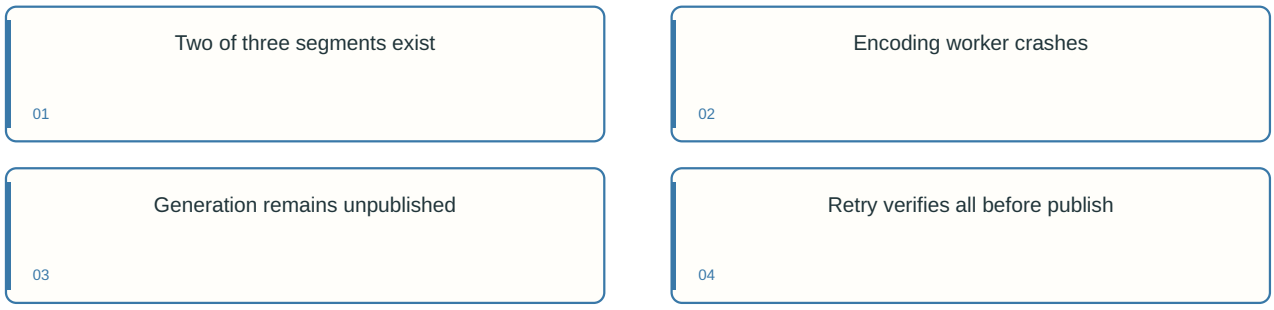


FIG. 22.6.3 **Follow the failure and its correction**

Partial segment output stays unpublished until the required generation is complete.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.6.13 · Scalability improvements

Scale upload bytes, encoding jobs, and viewer delivery separately. Bound worker concurrency and prioritise encoding according to deadlines. Immutable segments are suitable for caching where access policy permits it.

A viral video creates hot-object delivery pressure, not necessarily a metadata-sharding problem. Plan origin protection and regional copy cost during cache loss. Adding profiles or transcoders can also increase storage and backfill load. Improve the measured limiting path rather than expanding every component together.

22.6.14 · Consistency requirements

The published metadata generation must name a usable artifact set. Encoding attempts can repeat using stable output identities and conditional completion, so an old worker cannot replace a newer generation.

Playback caches have a declared reuse interval; private access and deletion have their own requirements. Before using regional copies for recovery, ensure the publication protocol establishes the required objects there. Merely copying some bytes does not make a region an authoritative complete failover source.

22.6.15 · Security model

Authorise upload and playback, limit size and supported formats, and isolate parsers and encoders. Give workers narrowly scoped storage access. Protect signed capabilities and keep them out of routine logs.

Check manifest, segment, and thumbnail delivery: private metadata can coexist accidentally with public objects. Test capability expiry, cache keys, membership revocation, and deletion end to end. The player's page is only one route to the content and should not be the only place enforcing its privacy.

22.6.16 · Deployment approach

Version encoding profiles and retain usable older outputs until a validated transition permits removal. Workers need durable claims and graceful shutdown. Expensive backfills should not consume playback or live-processing capacity.

Test recovery when source bytes survive but metadata is restored to an older point. Artifact inventories and source identities must let the application reconcile what exists. A rollout that preserves running containers but loses the link between metadata and media can still leave acknowledged videos unusable.

22.6.17 · Observability design

Observe upload completion, oldest encoding jobs, failed profiles, compute-to-media ratios, and ready-generation integrity. For playback, track edge hits, origin transfer, startup delay, rebuffering, and access outcomes.

Correlate source, profile, and playback generation in appropriate detail. Alert on missing segments and videos stuck processing, even if workers report few exceptions. A synthetic private-video journey should include permitted playback and later revocation, so monitoring covers content access as well as infrastructure availability.

22.6.18 · Infrastructure cost considerations

Include originals, renditions, repeated encoding, compute, delivery requests, transfer, metadata, and telemetry. Sustaining the assumed 3.75 GB/s for an hour delivers 13.5 TB before overhead.

The duration of viewer demand matters as much as a peak concurrency label. Compare profile count and retention with quality needs. Use fictional labelled rates for examples or verified current quotes for spending. A smaller rendition set can save costs while changing playback choices, so evaluate the user effect as well as the storage total.

22.6.19 · Alternative architectures and trade-offs

A single file is easy to manage but adapts poorly to varying delivery conditions. Segmented adaptive delivery adds manifests and a larger artifact lifecycle. Pre-encoding offers readiness while consuming work and storage for profiles that may never be used.

On-demand encoding can reduce unused work and create startup or burst risk. Managed conversion moves selected operations to a provider while leaving publication, access, and reconciliation decisions. Compare alternatives with the same supported-client, privacy, and recovery requirements.

22.6.20 · Architecture evolution from small scale to large scale

Start with resumable upload and durable processing identity. Add a small tested adaptive set and versioned publication. Then introduce suitable edge delivery and scale encoding separately from playback.

Add regional objects or more profiles for demonstrated locality, recovery, or quality needs. Keep exact source and profile identities so every result remains explainable and reproducible. More delivery capacity should not change what ready, private, or deleted means for an existing video.

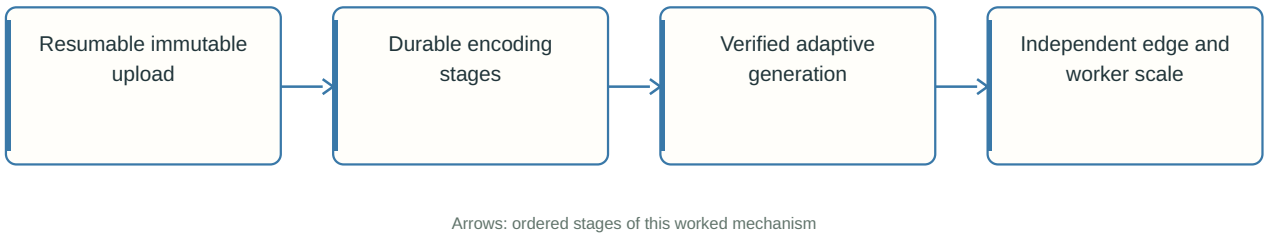


FIG. 22.6.4 **How the design can grow**

Scale encoding separately from delivery and preserve versioned playable sets.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.6.21 · Design review identifying weaknesses and improvements

Ask whether every ready generation is actually playable, whether its private segments remain protected, and whether origin survives edge loss. These are separate claims requiring separate evidence.

Test corrupt input, worker interruption between artifacts and publication, repeated completion, a missing segment, expired access, and deletion after caching. A video that played once through a warm public URL proves little about private recovery, partial encoding, or the failed-delivery path.

22.6.22 · A short implementation exercise

Build a toy publisher requiring three segments. Write two and stop. The manifest must remain unpublished. Resume using the same job identity, write the third, validate checksums, and advance one metadata generation atomically.

Then remove a segment deliberately and run an integrity check. The check should identify the broken published set. Explain which boundary establishes ready state and which later failure the integrity check detects, rather than using one success flag for both.

A small implementation of the key rule

PSEUDOCODE · ONE PART OF THE DESIGN

```
def publish(job):
    require_current_epoch(job)
    artifacts = expected_artifacts(job.source, job.profile)
    require(all(exists_and_valid(a) for a in artifacts))
    with transaction():
        assert_current_epoch(job)
        generation = record_complete_manifest(artifacts)
        mark_playable(job.video, generation)
# Partial worker output is never silently declared ready.
```

Use the implementation exercise to test the rule and the failure described above. You can also run local examples of seat claims, duplicate handling, ledger balance, and workflow epochs in the mechanism collection.

Unit 22 exercises → · Unit 22 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. RFC 8216: HTTP Live Streaming, 2017 ↗

   Original protocol description · Informational RFC. One adaptive streaming protocol; do not confuse Informational RFC with Standards Track.

   Access reviewed · 21 October 2026

2. RFC 9111: HTTP Caching, 2022 ↗

   Original standard. HTTP cache semantics; application cache races are separate examples.

   Access reviewed · 21 October 2026

3. OpenLineage: core model and documentation ↗

   Official specification documentation · 1.54.0 at access. Dataset, job, and run lineage concepts.

   Access reviewed · 21 October 2026

Ride-Hailing System

Prerequisites: Data Models and Storage Technologies, Scalability, Load Balancing, and Data Distribution, Distributed Systems Fundamentals, Message Queues, Events, and Distributed Workflows

4 original figures

Blue: normal flow · Green: healthy · Amber: caution · Red: failure · Purple: control / shared infrastructure · Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Two riders select the same nearby driver. Spatial search can find candidates; an authoritative claim must decide who actually receives the assignment.

The workload, timing, and prices in this design are teaching assumptions. They are not measured benchmarks or a description of a particular company.

22.7.1 · Functional requirements

A driver reports location and availability. A rider requests a trip; the system finds candidates, offers work, and records one accepted assignment. It also tracks trip progress and cancellation.

Being nearby makes a driver a candidate, not a confirmed owner of the trip. Define offer expiry and the rider's view while matching is unresolved. The same driver must not become actively assigned to incompatible trips merely because two matchers saw an old available state.

22.7.2 · Nonfunctional requirements

Set a bounded machine-side dispatch target for the assumed region and preserve exclusive active assignments. Protect precise location information. Treat the freshness and accuracy of reported positions as uncertainty, not ground truth.

Trip transitions must survive retries and temporary disconnection. A missing heartbeat does not establish that a trip ended. Human offer acceptance and provider payment status also have different timelines. Keep these states visible rather than collapsing every delay into one failed trip outcome.

22.7.3 · Explicit assumptions and workload numbers

Assume 100,000 online drivers report every five seconds. The rate is 20,000 updates/s. At 100 raw bytes per position, that is 2 MB/s, or 172.8 GB/day if every update is retained.

Assume 100 new trip requests/s. Map calls, candidate evaluation, regional skew, and location retention require separate estimates. The large location stream and the smaller authoritative assignment workload do not need the same storage or coordination mechanism.

22.7.4 · Naive initial architecture

Begin with one service that stores recent positions and selects a nearby available driver. It is straightforward while requests are handled sequentially, but concurrent matchers can select the same driver.

Separate the location projection used for search from authoritative assignment state. Preserve exclusive claims before optimising spatial queries. A better nearest-neighbour lookup improves candidates; it cannot resolve two buyers of the same driver's availability. Stale positions can also reduce match quality even when the assignment invariant remains correct.

22.7.5 · Core API contracts

A driver location update carries position, source sequence, and observed time. POST /trips creates a stable rider request. POST /offers/{id}/accept attempts a valid claim, while trip status and cancellation expose explicit transitions.

Derive driver identity from authentication and validate plausible input. An expired or superseded offer returns a defined conflict. A repeated accept can recover the existing result. The API must not permit a caller-supplied driver identifier to replace another assignment without passing the ownership rule.

22.7.6 · Data model and schema

Keep driver_state with availability, assignment_id, and version. Trip records contain rider, state, driver, and version. Offers identify the trip, driver, expiry, and status. Latest positions carry driver sequence, spatial cell, and observation time.

Retained location history has a separate restricted lifecycle. A conditional transition or appropriate uniqueness constraint protects active assignment. The spatial index is derived candidate state; it must not become a second independent authority for whether a driver is free.

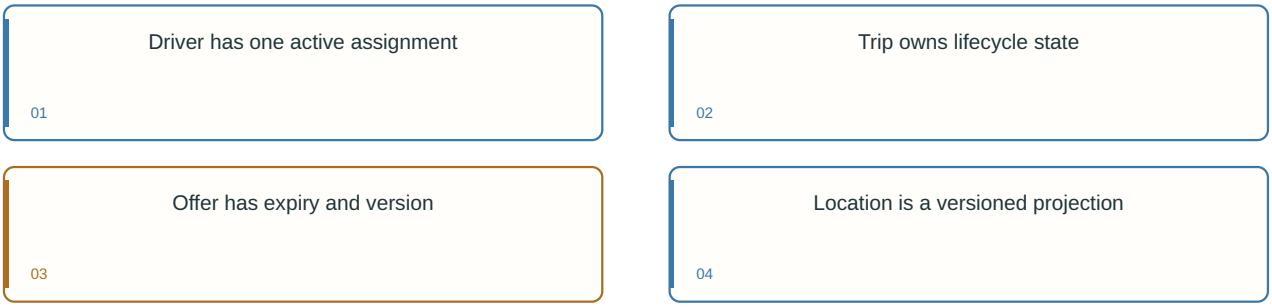


FIG. 22.7.1 **Data ownership and relationships**

Latest position helps search; driver and trip state decide exclusive assignment.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.7.7 · Low-level component organization

LocationIngestor validates updates and their order. SpatialIndex finds candidates, Matcher ranks a bounded set, and OfferService manages offer validity. AssignmentRepository performs the authoritative claim, while TripStateMachine controls later transitions.

Map and ETA adapters have their own deadlines. Their failure must not corrupt assignment. Stable trip and acceptance identities let clients retry without starting different logical work. Keep the availability decision at its atomic owner even when candidate selection runs on many workers.

22.7.8 · High-level system architecture

Use a regional location stream and spatial projection, a durable trip/assignment store, matching workers, offer delivery, and trip events. Partition geography for useful query locality while defining driver movement across boundaries.

Configuration can come from a control plane, but stale independent availability copies must not each accept the same driver. The placement needs one valid authority or transfer protocol for assignment. Region-local candidate reads and exclusive ownership are different paths and should appear separately in the design.

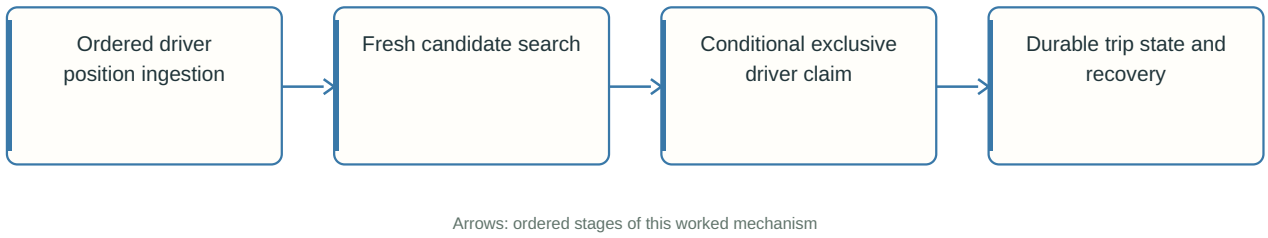


FIG. 22.7.2 **Follow the main operation**

Candidate search precedes a valid conditional offer acceptance.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.7.9 · Data and control flows

Commit one rider request identity. Search for sufficiently fresh candidates, recheck authoritative availability, and create a bounded-lifetime offer. Acceptance verifies that the offer and ownership conditions still hold.

Atomically bind the driver and trip under the chosen transaction model. Replay repeated accepts or reject stale ones. If the reply is lost, trip status resolves the client's uncertainty. Receiving an offer is not itself the assignment decision; the conditional acceptance is the point that prevents incompatible winners.

22.7.10 · Capacity estimates

The example ingests 20,000 locations/s and receives 100 dispatches/s. If each dispatch evaluates twenty candidates, that adds 2,000 candidate evaluations/s before retries or ETA calls.

Do not call a route service for every location update without justification. Busy centres and major events can concentrate work in one area while global averages look mild. Estimate regional peaks, hot cells, and movement across their edges. Candidate pruning can reduce map cost without changing the exclusive assignment rule.

22.7.11 · End-to-end latency estimates

Assume 30 ms ingress, 40 ms candidate lookup, 80 ms bounded ETA evaluation, and 30 ms assignment work. The machine-side planning budget sums to 180 ms.

Human acceptance is a separate interval. Missing candidates, offer expiry, communication delays, and retries can extend the matching journey. Measure these states rather than announcing one optimistic

dispatch latency. The budget is illustrative and needs validation for actual city shapes, traffic, and provider behaviour.

22.7.12 · Failure scenarios

Reject late location updates that would move the serving projection backward under its sequence rule. When two requests select one driver, only one authoritative claim may succeed. A paused worker cannot accept a superseded offer after it resumes.

Lost offers or replies need status recovery. A map outage may permit a justified simpler ranking or require pausing matching. It must not bypass exclusive assignment. Test each failure at the relevant state boundary so a candidate-search problem does not become a conflicting trip commitment.

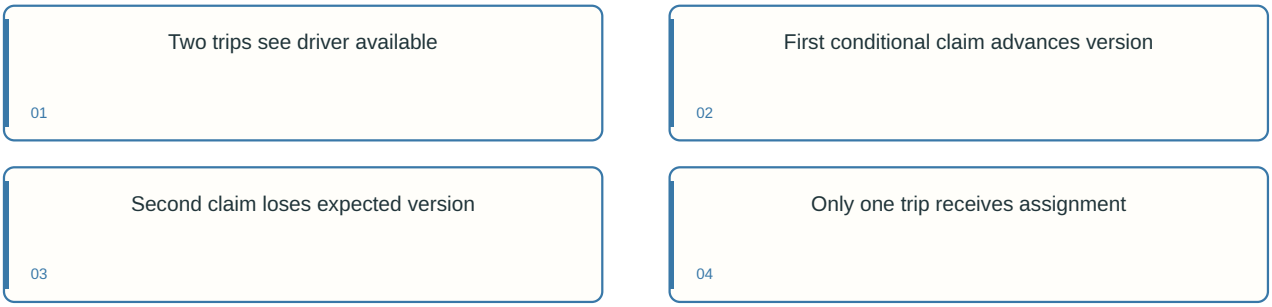


FIG. 22.7.3 **Follow the failure and its correction**

Two riders can choose one candidate, but only one authoritative claim may win.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.7.13 · Scalability improvements

Scale the location stream and spatial serving independently from assignment transactions. Batch or prune ETA calls. Partition regions or cells with an explicit handoff that preserves the current driver's authority.

Candidate caches need freshness information. More matchers do not divide a hot serialised claim owner automatically; changing that model must still preserve one active assignment. Observe the actual concentrated work before choosing finer partitions, since they also increase cross-cell queries and movement coordination.

22.7.14 · Consistency requirements

Location can update through a delayed projection with an explicit age policy. Driver and trip assignment require a coordinated conditional transition. Crossing a regional boundary needs authority transfer rather than independent acceptance in both regions.

Give trip status the required read-after-accept behaviour. Time helps judge whether a position is recent, but sequence and fencing rules handle stale updates and owners. These different forms of state should not all be given the same broad consistency label.

22.7.15 · Security model

Authenticate driver updates and restrict rider and driver views to the relevant trip context. Limit access to precise location history, including support and administrative paths. Retain only justified history and audit privileged use.

Bound geographic scraping and expensive map requests. A rider cannot select an arbitrary driver identity as authority to claim them. Keep raw coordinates out of broad telemetry where unnecessary. Privacy follows locations through streams, candidate caches, exports, and retained logs, not just the map displayed to the user.

22.7.16 · Deployment approach

Use regional placement with a stated failure model and durable assignment state. Roll releases compatibly with old clients and offers already in progress. Drain new work while preserving existing offer and trip records.

Disaster recovery must reconcile active assignments and uncertain provider outcomes. Restoring an old snapshot must not reoffer drivers still assigned elsewhere. The replacement needs valid history and fencing before it acts, even if the API has resumed responding. Test the recovery path with active trips rather than only an empty system.

22.7.17 · Observability design

Track location age, stale-update rejection, cell skew, candidate counts, claim conflicts, expiry, matching outcomes, and time spent in each dispatch state. Observe map-provider errors separately.

Correlate stable creation and acceptance identities across attempts. Alert on unresolved assignments and invariant violations, not only HTTP errors. Simulated drivers can move across cells, disconnect, and accept

near expiry while concurrent riders compete. These journeys test the distinction between fresh candidates and correct exclusive outcomes.

22.7.18 · Infrastructure cost considerations

Include ingestion, hot location state, maps/ETA calls, assignments, offer attempts, history retention, transfer, and operations. A route lookup for every position update could dominate the cost despite efficient state storage.

Model calls per candidate and dispatch, then compare pruning and batching. Use explicit fictional rates or verified commercial terms. The example does not claim a real company's internal bill. Keep the accuracy and latency effects of a cheaper candidate strategy visible alongside its savings.

22.7.19 · Alternative architectures and trade-offs

Straight-line proximity is cheap but may ignore road travel. Route-based ETA can improve ranking while adding delay and an external dependency. Sequential offers reduce simultaneous acceptance opportunities; multi-offer designs need a controlled single winner.

Fine spatial cells improve some lookup locality but add boundary queries and handoff work. Compare fairness and rider outcomes too. Candidate quality, human wait, and machine cost all matter, while none can justify weakening the active-assignment invariant.

22.7.20 · Architecture evolution from small scale to large scale

First establish a correct regional claim and basic proximity search. Add streamed positions and spatial indexing when lookup work requires it. Then distribute matching with bounded ETA evaluation.

Split geographic ownership after measuring hot areas, using a safe transfer procedure. Regional serving adds another partition and failure policy for active trips. Preserve stable identities and claims as the deployment grows. Replicating an apparently stateless matcher does not replicate the assignment authority it depends on.

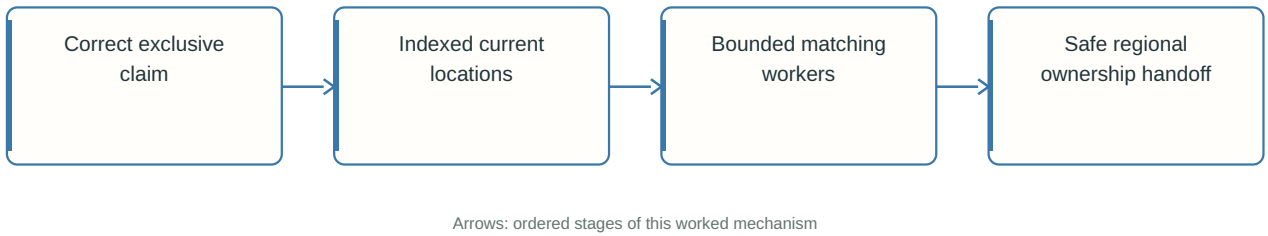


FIG. 22.7.4 **How the design can grow**

Partition location work without creating conflicting assignment owners.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.7.21 · Design review identifying weaknesses and improvements

Review the meanings of nearby, recently observed, available, offered, and assigned. The spatial index answers only some of those questions.

Test two riders choosing one driver, out-of-order positions, acceptance after expiry, paused workers, and ownership handoff. One claim should win and losers should receive a meaningful state. Confirm the search projection never acts as an independent writer of authoritative availability. A fast matching demo can otherwise hide the most consequential race.

22.7.22 · A short implementation exercise

Build an optimistic driver claim with expected version and offer expiry. Start two requests using the same initial version. Verify one claim succeeds locally and one returns conflict.

Then deliver an older location and a delayed acceptance after the winner commits. Neither should overwrite the active assignment. Explain the actual protected boundary: the local transaction can establish one local owner, while external trip or payment effects still need their own contracts and recovery evidence.

A small implementation of the key rule

PSEUDOCODE · ONE PART OF THE DESIGN

```
def accept_offer(offer, driver, now):
    with transaction():
        current = lock_offer(offer.id)
        require(current.pending and now < current.expires_at)
        won = claim_driver_if_available(driver, current.expected_version)
        if not won: raise AssignmentConflict
        bind_trip_and_offer(offer.trip, driver, offer.id)
    return current_trip_state(offer.trip)
```

Use the implementation exercise to test the rule and the failure described above. You can also run local examples of seat claims, duplicate handling, ledger balance, and workflow epochs in the mechanism collection.

Unit 22 exercises → · Unit 22 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. PostGIS: ST_DWithin ↗

   Official implementation documentation · accessed 21 Oct 2026. Coordinate-system-aware proximity predicate, not exclusive dispatch assignment.

   Access reviewed · 21 October 2026

2. PostgreSQL 18: Transaction Isolation ↗

   Official versioned documentation. Specific PostgreSQL isolation behavior; no all-database equivalence claim.

   Access reviewed · 21 October 2026

3. Saga distributed transactions pattern — Microsoft ↗

   Official engineering guidance. Local transactions and compensation; compensation is not always a perfect inverse.

   Access reviewed · 21 October 2026

Online Ticket Booking

Prerequisites: API Design and Communication Contracts, Database Internals, Transactions, and Concurrency Control, Message Queues, Events, and Distributed Workflows, Security Architecture and Privacy

4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A payment succeeds just after a seat hold expires. Follow the reservation, current seat owner, and provider evidence to find an outcome that does not sell the seat twice.

The workload, timing, and prices in this design are teaching assumptions. They are not measured benchmarks or a description of a particular company.

22.8.1 · Functional requirements

A buyer browses an event, holds selected seats, pays, and receives tickets. Holds last for a defined period; cancellation and refunds follow an explicit policy.

The displayed seat map is advisory. The authoritative reservation decides who owns the opportunity to buy a seat. Retries, expiry tasks, and payment callbacks may overlap, but must not confirm the same seat for two buyers. Design that invariant before optimising the map or launching a waiting room.

22.8.2 · Nonfunctional requirements

Prevent duplicate confirmed ownership and retain accepted payment and ticket records under the chosen failure contract. Pending reservations must resolve through supported confirmation, expiry, or recovery.

Set booking and waiting-room objectives for launches, including the intended fairness policy. Fixed inventory cannot satisfy every simultaneous request. Reject or queue excess demand clearly instead of promising universal acceptance and overselling. Separate an admitted transaction's latency from the user's total time waiting for admission and payment.

22.8.3 · Explicit assumptions and workload numbers

Assume 50,000 seats and a short peak of 100,000 reservation attempts/s. At 200 raw bytes per seat, seat records occupy about 10 MB before indexes.

The small byte total does not make the launch easy. Many callers compete for the same limited ownership transitions. Assume two-minute holds and a bounded number of seats per user. Retry volume, transaction conflicts, and expiry work need their own capacity estimates. These assumptions are a launch scenario, not a benchmark for a particular database.

22.8.4 · Naive initial architecture

Start with a transactional seat table. Claim a seat using a conditional change from available to reserved, and insert its reservation in the same transaction.

A read followed by an unconditional update can let both callers report success. The conditional claim must check its result. This first correct boundary handles simpler traffic; flash demand, payment uncertainty, and expiry races motivate further controls. Fixing concurrency first prevents a faster interface from merely producing incorrect tickets more quickly.

22.8.5 · Core API contracts

POST /reservations includes event, seat identities, and an idempotency key. The returned resource exposes status and expiry. A payment operation belongs to that reservation, and authorised ticket reads show committed issuance.

Repeat keys recover the established result rather than claiming another seat. Bind them to the trusted actor and request content. Payment states distinguish authorisation, capture, decline, and uncertainty according to the provider. An expired reservation and an uncertain payment are different conditions needing different next actions.

22.8.6 · Data model and schema

A seat identifies event and seat, state, reservation owner, and version. Reservation records hold owner, state, expiry, and version, with a relationship to selected seats. Payment operations and ticket records have their own identities.

Enforce one relevant confirmed active ticket per seat. Expiry and confirmation check the reservation identity and current state/version. An old expiry task must not release a seat now held by a later reservation.

These relationships need actual constraints or atomic checks, not just matching names in a diagram.

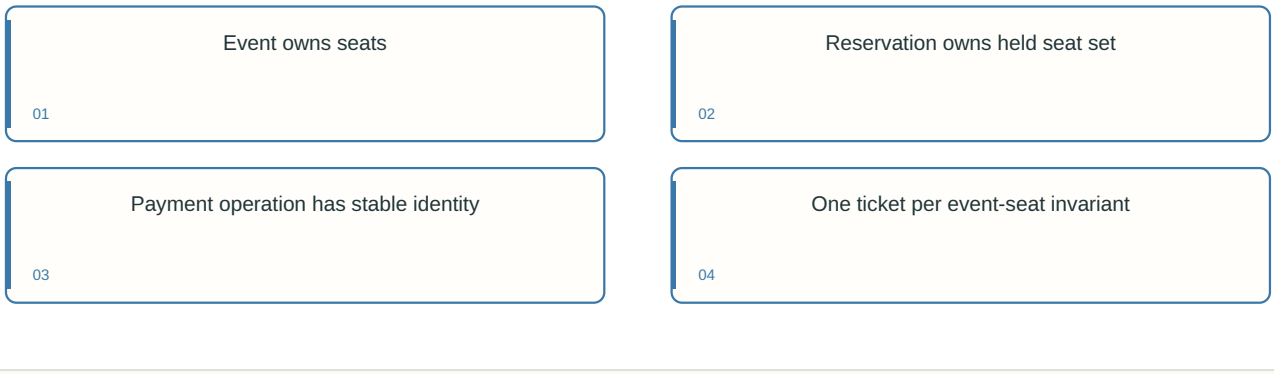


FIG. 22.8.1 **Data ownership and relationships**

Seat, hold, payment, and ticket records must preserve their defined ownership relationships.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.8.7 · Low-level component organization

SeatRepository performs conditional claims or locks. ReservationService manages selection and hold rules. PaymentWorkflow handles external effects, TicketIssuer commits valid issuance, and ExpiryWorker conditionally releases old holds.

Pricing and refund rules use explicit amounts and units. Enforce ownership at the shared state boundary so callbacks and workers cannot bypass a check implemented only in an HTTP handler. Each role should return enough state to distinguish conflict, expiry, confirmed success, and an uncertain provider result.

22.8.8 · High-level system architecture

Place bounded admission or a waiting room before the ownership store. Keep durable reservation state, the payment adapter, a workflow or outbox path, and a ticket read model. Cache descriptions and advisory maps separately.

Event partitioning can keep one event's seat operations local, but a popular event remains hot. Admission and contention control still matter. The cache answers what may be available; the transaction answers what this buyer actually reserved. Do not let those paths become competing allocation authorities.

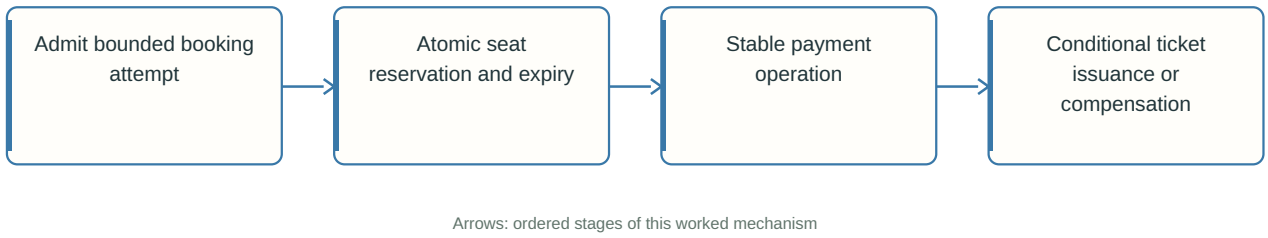


FIG. 22.8.2 **Follow the main operation**

A hold and payment workflow lead to a version-checked confirmation or supported recovery.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.8.9 · Data and control flows

Atomically claim seats and create the hold, then return its expiry after commit. Submit payment with stable provider identity. Confirmation verifies a still-valid transition and commits the required reservation, seat, and ticket changes.

If payment succeeds after expiry, follow a defined recovery policy: reacquire only if safe or refund according to the contract. Never issue against a seat already owned by someone else. Expiry checks current identity and version before release, so delayed tasks cannot undo a later valid outcome.

22.8.10 · Capacity estimates

Assume admission is configured to allow 2,000 reservation attempts/s out of a 100,000/s launch. The remaining callers wait or receive rejection under the stated policy. That 2,000 limit is an input to validate, not demonstrated capacity.

A two-minute hold is bounded by the number of seats, but the surrounding retry traffic is not. Estimate writes per attempt, repeated-key checks, conflicts, and expiry scans. Web concurrency can grow rapidly while the small inventory remains the same.

22.8.11 · End-to-end latency estimates

Assume 20 ms edge time, 30 ms acquiring the transaction resources, 40 ms for conditional seat/reservation work, and 20 ms returning the result. The admitted path totals 110 ms in this scenario.

Contention can exceed that budget, so bound waiting and stop work that is no longer useful. Report waiting-room time separately. Payment has its own expiry-sensitive timeline. Fast admitted requests do not establish a fast journey for users still queued outside the allocation boundary.

22.8.12 · Failure scenarios

Two buyers must not both win after reading available. A crash after reservation commit but before reply needs repeat-key recovery. Duplicate payment evidence must not issue another ticket.

Also test payment after hold expiry and stale expiry after confirmation or reassignment. Conditional current-state transitions and reconciliation protect these overlaps. If email fails after a ticket commits, the ticket still exists and needs another delivery path. Communication failure must not be mistaken for failure of the already accepted ownership transition.

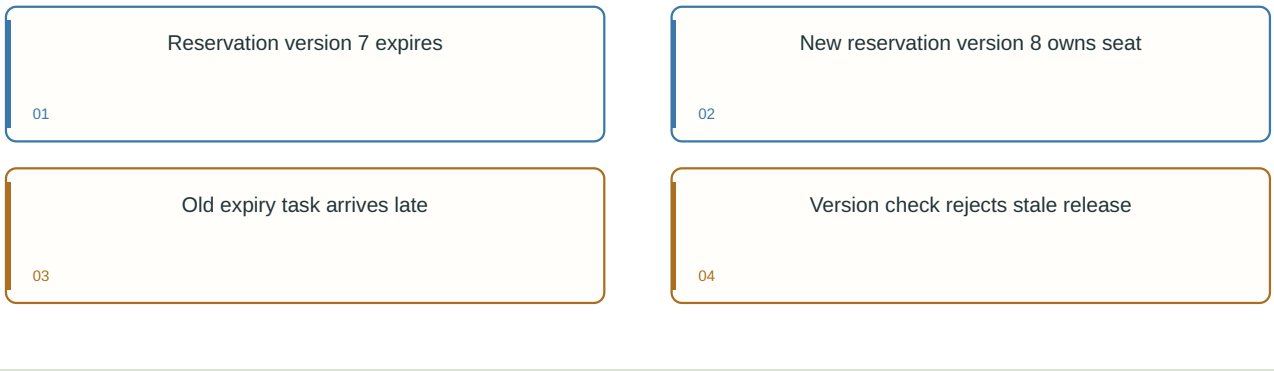


FIG. 22.8.3 **Follow the failure and its correction**

An old expiry or late payment must not overwrite a newer seat owner.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.8.13 · Scalability improvements

Cache advisory data, bound launch admission, and shorten the correct seat transaction. Partition independent events when measured load justifies it.

Multi-seat claims need a deliberate all-or-nothing rule and consistent lock order. Splitting an event across shards can make group bookings harder. Extra shards do not split contention over one particular seat. Scale the work around the invariant while preserving its atomic owner, and introduce a more complex allocation protocol only when its benefit exceeds the added transaction and recovery burden.

22.8.14 · Consistency requirements

Seat ownership and ticket issuance need an atomic authoritative rule. Availability displays may lag if that is clear to the buyer. Payment and email are external effects with stable identities and tracked workflow state.

A serializable local transaction can help with local concurrency but does not include the provider automatically. Handle retryable serialization failures with bounded whole-transaction retries and safe external-effect placement. State which records move together and which later outcomes are reconciled, rather than claiming the entire booking journey is one database transaction.

22.8.15 · Security model

Authorise holds, payments, tickets, transfers, and refunds. Calculate prices on the server, verify provider callbacks, and protect credentials. Limit automated and bulk abuse under a defined identity policy.

Scope idempotency keys so one actor cannot recover another's result. If tickets use download capabilities, set their scope and lifetime and audit important changes. A waiting room protects capacity; it is not proof of authorised identity or fairness. Those policies still need their own enforcement and tests.

22.8.16 · Deployment approach

Keep reservation states and expiry meanings compatible during rollout. Durable scheduled expiry survives worker replacement. Database promotion must respect the acknowledged-data policy.

After recovery, reconcile provider results before declaring all bookings resolved. When write authority is uncertain, temporarily pausing allocation can protect exclusive ownership while descriptions remain readable. Test these transitions with live holds and callbacks, not only an empty event. A newly running server must not treat an old snapshot as permission to sell seats already confirmed elsewhere.

22.8.17 · Observability design

Track admitted attempts, claim conflicts, transaction waits, hold age, expiry backlog, payment uncertainty, late success, and ticket uniqueness. Follow one logical booking through repeated requests and callbacks.

Run appropriate invariant checks over seats, reservations, and tickets, with reconciliation against payment evidence. Empty error logs cannot establish that the relationships agree. Keep user-visible waiting and completion times separate from raw API times so operational dashboards reflect the actual booking journey.

22.8.18 · Infrastructure cost considerations

Include burst admission, transactional work, abandoned holds, payment calls, delivery, retention, telemetry, and recovery capacity. Model the duration of a launch rather than charging its peak rate for every hour of the month.

More web workers cannot remove contention on one seat. Compare alternatives using the same no-double-ticket and recovery requirements. Use explicit fictional rates or verified prices, and include retry and expiry work that occurs even for buyers who never complete a purchase.

22.8.19 · Alternative architectures and trade-offs

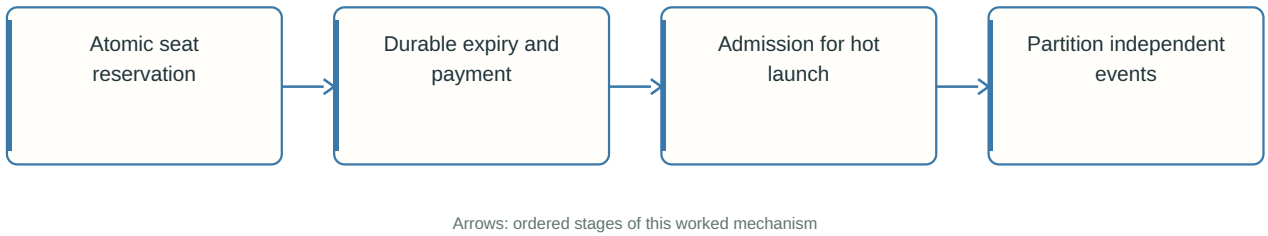
Pessimistic locking serialises access early but can create waiting and deadlocks. Conditional optimistic claims return conflicts quickly but require clear retry and user handling. One event owner can simplify order while imposing a capacity boundary.

Distributed allocation can reduce some central work and complicate multi-seat atomicity and fairness. Choose by the actual seat and hold contract. The mechanism must explain one winner under concurrent attempts, then integrate that winner with payment and expiry rather than stopping at a successful lock acquisition.

22.8.20 · Architecture evolution from small scale to large scale

First implement the seat transaction and repeat-key result. Add durable expiry and provider reconciliation. Then protect launches with admission and advisory caching.

Partition independent events after measuring the workload. Change one-event allocation only when its demonstrated constraints justify the added protocol. Recheck no-double-ticket behaviour at every stage. Increasing traffic capacity is not progress if an earlier safe expiry or retry becomes an uncoordinated write in the new arrangement.



Arrows: ordered stages of this worked mechanism

FIG. 22.8.4 **How the design can grow**

Add admission and partition independent events while protecting the same seat invariant.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.8.21 · Design review identifying weaknesses and improvements

Check that expiry and confirmation refer to the same reservation identity and current state. Review the recovery for a successful payment after the hold is gone, and confirm that display state never grants ownership.

Test competing buyers, repeated callbacks, commit-then-crash, stale expiry after reassignment, and late payment. A correct happy-path seat update answers only part of the workflow. The review must also account for money and tickets that remain uncertain after the visible request ends.

22.8.22 · A short implementation exercise

Reproduce the last-seat race using two connections or an explicit interleaving. Show both buyers reporting success under naive reads and writes. Replace it with a conditional claim and reservation transaction, checking affected rows.

Add versioned expiry. Confirm or reassign the seat, then deliver the old expiry task. It must leave the current owner's seat unchanged. Explain the condition that prevents the stale release, rather than relying on the test's usual timing.

A small implementation of the key rule

PSEUDOCODE · ONE PART OF THE DESIGN

```
with transaction():
    insert_reservation(reservation_id, owner, expires_at)
    updated = reserve_if_available(event, seat, reservation_id)
    if updated != 1: raise SeatUnavailable
# Expiry later releases only the same current reservation version.
# Provider payment is coordinated outside this local transaction.
```

Use the implementation exercise to test the rule and the failure described above. You can also run local examples of seat claims, duplicate handling, ledger balance, and workflow epochs in the mechanism collection.

Unit 22 exercises → · Unit 22 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. PostgreSQL 18: Transaction Isolation ↗

   Official versioned documentation. Specific PostgreSQL isolation behavior; no all-database equivalence claim.

   Access reviewed · 21 October 2026

2. Saga distributed transactions pattern — Microsoft ↗

   Official engineering guidance. Local transactions and compensation; compensation is not always a perfect inverse.

   Access reviewed · 21 October 2026

3. RFC 9110: HTTP Semantics, June 2022 ↗

   Original standard. HTTP semantics; application commit policy is separate.

   Access reviewed · 21 October 2026

Payment System

Prerequisites: API Design and Communication Contracts, Database Internals, Transactions, and Concurrency Control, Message Queues, Events, and Distributed Workflows, Reliability, Fault Tolerance, and Disaster Recovery, Security Architecture and Privacy

4 original figures

Blue: normal flow · Green: healthy · Amber: caution · Red: failure · Purple: control / shared infrastructure · Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A provider charges successfully, but the local service stops before recording the reply. Use stable identity and reconciliation to connect the external result to one valid local posting set.

The workload, timing, and prices in this design are teaching assumptions. They are not measured benchmarks or a description of a particular company.

22.9.1 · Functional requirements

A client creates a payment intent, the service requests authorisation or capture, and later reports the supported outcome. It records balanced accounting entries, reconciles provider evidence, and tracks refunds as separate operations.

An order's status, a provider's operation, and the ledger are related but different records. Represent money with currency and integer minor units plus an explicit rounding policy. A single paid flag cannot explain the sequence, uncertainty, or accounting consequences of moving money.

22.9.2 · Nonfunctional requirements

Preserve financial invariants and auditable state. Prevent duplicate relevant provider effects where its contract supports that guarantee, and resolve uncertain outcomes rather than inventing success or decline.

Set intent-acceptance timing separately from completion. Each settled amount should have unique balanced postings and authorised corrections under the chosen accounting model. A provider outage may leave work pending. Safe pending state is preferable to a fast unsupported final result that the system cannot later explain.

22.9.3 · Explicit assumptions and workload numbers

Assume one million intents/day, about 11.6/s on average, with an illustrative peak of 200/s. Suppose each initially produces six 1 kB operational/audit records: raw growth is 6 GB/day.

Ledger details, indexes, copies, callbacks, and retention add to this. The model uses synthetic amounts and does not assert actual provider prices or institutional architecture. Estimate provider concurrency and reconciliation traffic separately from the daily average, especially under long waits and retries.

22.9.4 · Naive initial architecture

A first service calls the provider, sets an order flag, and replies. If the provider charges and the process dies before saving the result, a retry can repeat the charge.

Before scaling, add a durable operation identity and explicit states including uncertainty. Local transactions protect related local changes. Provider keys, status queries, and reconciliation address the external gap. They solve different parts of the history. A larger deployment cannot repair an ambiguous operation for which no stable identity was recorded.

22.9.5 · Core API contracts

POST /payment-intents accepts amount_minor, currency, business reference, and idempotency key. The status resource reports explicitly supported states such as pending, authorised, captured, declined, or uncertain. A refund endpoint creates a separately keyed refund.

Bind keys to actor and financial payload; changed content under the same key conflicts. Verify provider webhooks rather than trusting client approval fields. Define how clients wait or query after uncertainty, so a missing reply does not automatically become another financial attempt.

22.9.6 · Data model and schema

Keep payment_intent and operation records with actor, key, fingerprint, progress, and result. Provider attempts retain stable keys, references, and outcomes. A verified-event inbox recognises repeated

evidence.

The ledger has transactions and entries identifying account, currency, and signed amount. Required entries balance within the relevant currency and accounting scope. Enforce external-operation linkage and correction identity. An authorised correction creates a new accounting event instead of silently rewriting past evidence. These constraints make the relationship between provider facts and local postings reviewable.

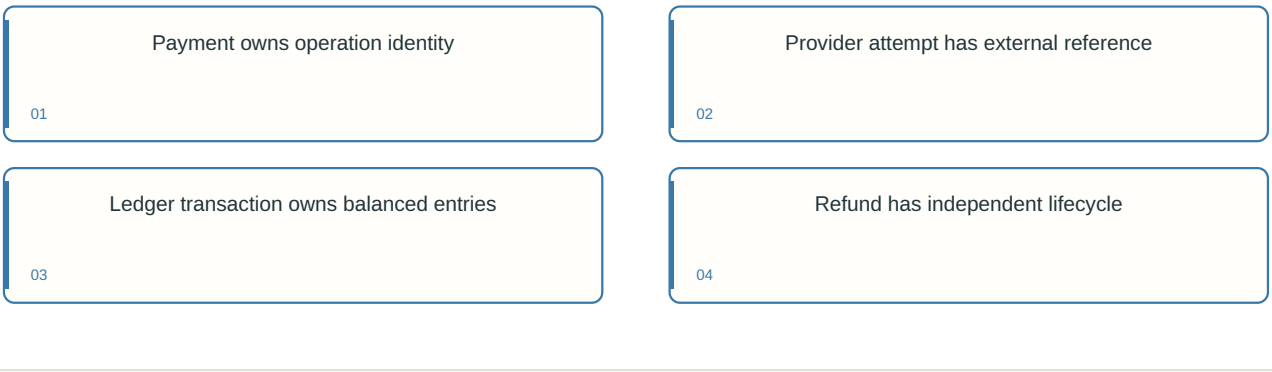


FIG. 22.9.1 **Data ownership and relationships**

Operation identity links provider evidence to payment state and balanced accounting entries.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.9.7 · Low-level component organization

PaymentService owns allowed transitions. Money and policy values check amounts. ProviderAdapter maps explicit outcomes, OperationRepository handles duplicate identity, and LedgerService validates and commits postings. Reconciler compares external evidence; RefundWorkflow owns refund progress.

These responsibilities can begin in one application. A timeout maps to uncertainty, not a definite decline. Keep provider-specific details at the adapter while exposing enough evidence for recovery. A component boundary is useful when it preserves meaning and ownership, rather than requiring a network service for every role.

22.9.8 · High-level system architecture

Use an authenticated API, durable intent and operation state, bounded provider workers, a verified callback inbox, a ledger boundary, and scheduled reconciliation. Where downstream updates are required, commit their publication intent with the relevant local state.

Reserve capacity for reconciliation rather than letting optional analytics consume it. Regional serving must preserve the chosen ledger and provider-operation authority. Draw external calls outside the local transaction boundary so the uncertainty and recovery path remain visible.

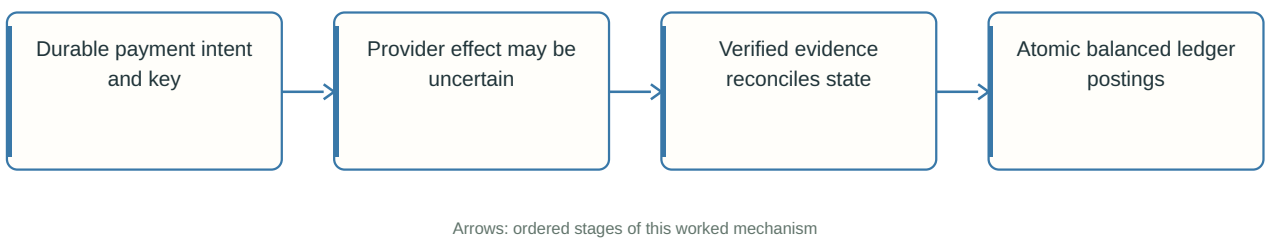


FIG. 22.9.2 **Follow the main operation**

Persist intent, obtain or reconcile provider evidence, and commit the required local result.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.9.9 · Data and control flows

Commit the intent and logical operation identity. Submit the stable provider key, then record either supported evidence or uncertainty. Capture or settlement evidence drives an idempotent transition creating the required balanced postings once.

Callbacks may arrive before or after a synchronous reply, so transitions must not regress incorrectly. Reconciliation queries the stable reference and repairs discrepancies. It must not blindly resend a charge merely because a local status is still pending. Trace evidence and local updates in order to see where a crash can leave work unresolved.

22.9.10 · Capacity estimates

At a stable 200 calls/s and an assumed 500 ms provider wait, Little's Law gives 200×0.5 = 100 in-flight provider calls in that boundary. The average daily rate of 11.6/s does not describe this peak.

The calculation is not a safe pool recommendation: allow for variance, deadlines, provider limits, and failures. Initial operational records total 6 GB/day before other storage. Callback and reconciliation work can continue after original traffic subsides and need capacity of their own.

22.9.11 · End-to-end latency estimates

Assume 20 ms ingress, 20 ms committing the intent, 500 ms with the provider, and 20 ms for the final local update. The successful synchronous path totals 560 ms.

These are assumed stage times, not p99 or guaranteed completion. Returning a durable pending intent after about 40 ms is a different API contract with later completion. A timeout requires status recovery or reconciliation. It supplies no reliable basis for converting an attempted payment into a definite failed business outcome.

22.9.12 · Failure scenarios

Test a lost reply after local commit, provider execution followed by process loss, repeated or reordered callbacks, refund uncertainty, and failure to commit postings.

Persist unknown outcomes and repeat only with a supported safe identity or established evidence. If the ledger is unavailable, do not bypass it by setting paid. Recovery must explain provider state and required postings before claiming the complete outcome resolved. A healthy API response after restart does not establish that the money histories now agree.

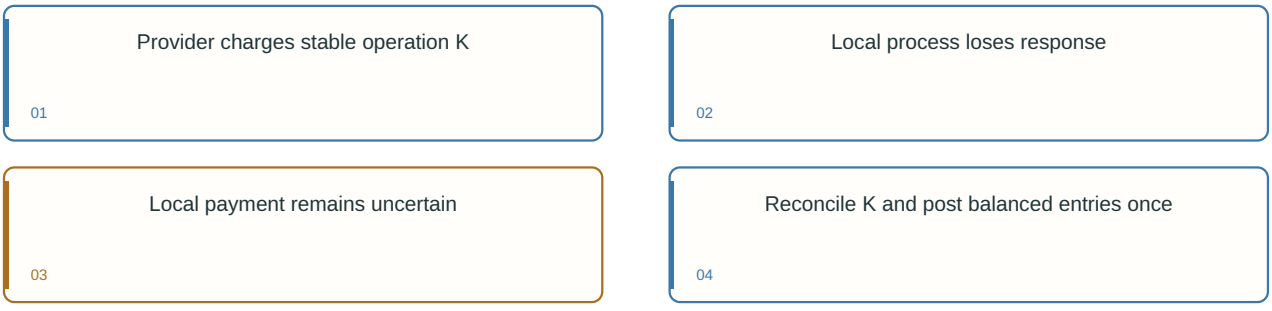


FIG. 22.9.3 **Follow the failure and its correction**

A lost provider reply leaves uncertainty rather than proof that no charge occurred.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.9.13 · Scalability improvements

Scale independent acceptance and operation workers, while bounding provider concurrency and retaining per-payment and relevant per-account invariants. Keep reconciliation resources isolated enough to repair uncertainty during an incident.

Status projections can serve reads with a stated freshness rule. Ledger partitioning needs a deliberate balance and transaction scope; one hot account can remain concentrated. A modest average daily rate alone is no reason to shard. Make the added ownership and recovery complexity answer an observed bottleneck.

22.9.14 · Consistency requirements

Local duplicate recognition, payment transitions, and required postings need appropriate atomic boundaries. A provider is outside those local transactions and requires its own identity and recovery contract.

Uniqueness and transaction semantics can protect one local posting set. They do not prove the provider executed exactly once. Refunds have separate states and may not reverse timing, fees, or downstream consequences perfectly. State what each accepted result means and preserve uncertainty until there is evidence for a final transition.

22.9.15 · Security model

Authorise payments and refunds, validate values server-side, verify callbacks, and protect credentials. Minimise sensitive data through reviewed provider tokenisation or integration where appropriate, rather than keeping unnecessary raw credentials.

Restrict and audit privileged corrections. An operator is not automatically permitted to rewrite accounting history. Bind every effect and query to its tenant and account scope. Logs should preserve relevant identity and outcomes without exposing secrets. These are architecture controls, not a claim of compliance with any particular financial or legal regime.

22.9.16 · Deployment approach

Roll states and event formats out compatibly. Preserve operation identities through releases and recovery, and control provider configuration and key changes.

An old database restore can forget keys for effects the provider already executed. Recovery therefore needs replay protection and reconciliation before resubmission. Test delayed callbacks from an older release and return after an outage. The new process's local view must be brought into agreement with external evidence instead of treating restored pending rows as untouched work.

22.9.17 · Observability design

Separate acceptance and completion delay. Track pending/uncertain age, provider attempts, repeated-event recognition, callback verification, imbalances, unmatched settlements, refund progress, and reconciliation outcomes.

Alert on invariant violations and old unresolved money movement, even when HTTP success rates are good. Evidence should distinguish attempted, provider-accepted, and locally posted states. Stable identities let an investigator reconstruct the operation without pretending that one log line or response code proves the complete financial journey.

22.9.18 · Infrastructure cost considerations

Costs include provider attempts, secure records, ledger work, retained audit evidence, telemetry, reconciliation, and operation. Use symbolic fees and labelled hypothetical infrastructure inputs unless current prices are verified.

A cheaper arrangement that cannot explain uncertain charges is a different and insufficient service. Cost per reconciled successful business payment is often more useful than cost per HTTP attempt. Retries can increase the attempt denominator precisely when useful outcomes decline. Keep the supported financial invariants equal when comparing alternatives.

22.9.19 · Alternative architectures and trade-offs

Synchronous completion can suit a client expecting an immediate result but exposes provider waits and uncertainty. Asynchronous intents make pending progress explicit while adding workflow state.

One local transactional application simplifies related invariants. Separating provider and ledger services can give organisational or isolation benefits and adds coordination. Append-only accounting history and mutable operational projections can complement each other. Do not force one paid record to serve every status, audit, and accounting need. Compare boundaries by the work they clarify and the recovery they require.

22.9.20 · Architecture evolution from small scale to large scale

Begin with explicit states, currency-aware integer values, local transactions, and stable provider operation identity. Add durable callback handling and reconciliation before introducing more traffic paths.

Then use projections and bounded workers for measured needs. Partition only when a demonstrated limit justifies a preserved invariant boundary. Every earlier payment must remain explainable through its provider evidence and postings after the topology changes. Scaling is successful when it expands supported work without losing that meaning.

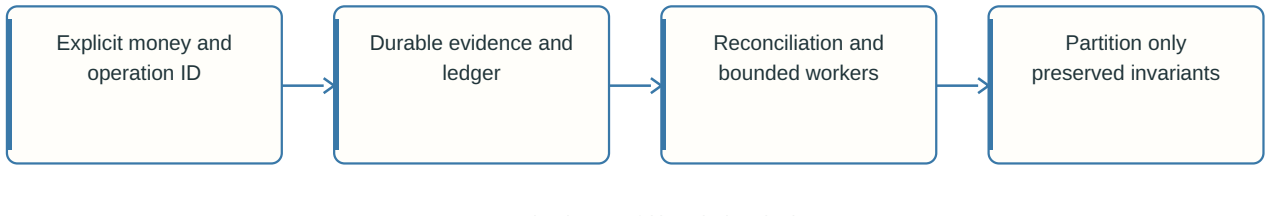


FIG. 22.9.4 **How the design can grow**

Scale useful independent work while keeping each prior operation and posting explainable.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.9.21 · Design review identifying weaknesses and improvements

Ask which evidence establishes financial success, who commits required postings, and how recovery handles an old deduplication snapshot beside newer provider effects.

Test charge-then-crash, callback-before-reply, duplicate evidence, partial refunds, and restore from earlier local state. Check for one valid local posting set and explicit unresolved states where evidence is missing. A provider timeout must not become a convenient decline merely to make the normal request path appear finished.

22.9.22 · A short implementation exercise

Build a synthetic provider that remembers stable operation keys and a local ledger that accepts only balanced entries. Stop after provider execution but before recording the result locally.

Reconcile by key and verify one logical operation leads to one balanced local posting set. Replay the provider event, and reject a changed financial payload under the old key. Explain that this demonstrates the synthetic contract and local boundary, rather than a guarantee for an arbitrary real provider.

A small implementation of the key rule

PSEUDOCODE · ONE PART OF THE DESIGN

```
def apply_settlement(event):
    with transaction():
        if inbox_contains(event.id): return prior_outcome(event.id)
        payment = lock_payment(event.payment_id)
        validate_transition_and_amount(payment, event)
        entries = [debit_entry(event), credit_entry(event)]
        require_balanced_by_currency(entries)
        insert_unique_posting_set(event.operation_id, entries)
        update_payment_and_insert_inbox(payment, event)
# Provider execution is reconciled separately from this transaction.
```

Use the implementation exercise to test the rule and the failure described above. You can also run local examples of seat claims, duplicate handling, ledger balance, and workflow epochs in the mechanism collection.

Unit 22 exercises → · Unit 22 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. PostgreSQL 18: Transaction Isolation ↗

   Official versioned documentation. Specific PostgreSQL isolation behavior; no all-database equivalence claim.

   Access reviewed · 21 October 2026

2. Transactional Outbox — Chris Richardson ↗

   Pattern author's explanation. Publication intent and duplicate delivery.

   Access reviewed · 21 October 2026

3. OWASP: Authorization ↗

   Official project guidance · accessed 21 Oct 2026. Least privilege, deny by default, and per-resource enforcement.

   Access reviewed · 21 October 2026

Distributed Job Scheduler

Prerequisites: Runtime, Processes, Threads, and Concurrency, Replication, Leader Election, and Consensus, Message Queues, Events, and Distributed Workflows, Reliability, Fault Tolerance, and Disaster Recovery

4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A scheduler and its worker can both restart around a due job. Give the schedule, occurrence, attempt, and effect separate identities so recovery knows what it is resuming.

The workload, timing, and prices in this design are teaching assumptions. They are not measured benchmarks or a description of a particular company.

22.10.1 · Functional requirements

A user creates a schedule, the service identifies due occurrences, and workers execute the jobs. The user can inspect history, configure retries, and cancel supported future work.

Define fixed times versus intervals, timezones, missed runs, and overlap. A schedule and one occurrence are different identities. Restarting a worker or scheduler must not invent a new logical occurrence for the same due time. Timing semantics need to be explicit before choosing a timer or queue implementation.

22.10.2 · Nonfunctional requirements

Keep accepted schedules and required runs recoverable under the supported failures. Bound scheduling delay and prevent stale attempts from overwriting newer results. Execution can start repeatedly around failures, so external effects need appropriate identity or fencing.

Cancellation also needs a boundary: future dispatch, pending work, and best-effort interruption of a running effect are different promises. A local cancellation flag cannot prove a remote effect was undone. Make these distinctions visible in run status.

22.10.3 · Explicit assumptions and workload numbers

Assume one million schedules and 100,000 occurrences/hour. The average rate is 100,000/3,600 ≈ 27.8 runs/s. At an initial 2 kB of run state each, raw growth is 200 MB/hour.

With ten-second mean execution at a stable rate, average active work is about 27.8×10 = 278 runs. Top-of-hour bursts, long jobs, retries, and retention add requirements. The number of saved schedules alone does not describe due-time concentration or required execution capacity.

22.10.4 · Naive initial architecture

One process scans due schedules and launches local functions. If it creates a run and advances the schedule in separate commits, a crash between them can repeat or miss the occurrence.

Create a stable occurrence record and advance due state transactionally. A leader can simplify scanning, but leadership alone does not make worker effects execute once. The first improvement is durable identity and atomic progress, not simply adding a second process to repeat the same unsafe loop.

22.10.5 · Core API contracts

POST /schedules accepts timing, timezone, payload reference, retry rules, and overlap policy. Cancellation changes the documented future behaviour. GET /runs/{id} reports attempts and outcome. A run-once operation has its own idempotency identity.

Bound timing-expression and payload complexity. Return durable acceptance separately from execution completion. Where an external effect remains uncertain, status should say so rather than treating a worker's missing reply as proof it never happened.

22.10.6 · Data model and schema

A schedule stores id, version, next_due, timing policy, and state. A run binds a unique (schedule_id,occurrence) to input version and progress. Attempts record epoch, lease deadline, worker, and result.

Preserve original due time separately from dispatch time. Runs and attempts are not interchangeable: another attempt does not create another logical occurrence. Conditional progress prevents a superseded attempt from replacing newer state. Use outbox or effect records where durable dispatch and external identity require them.

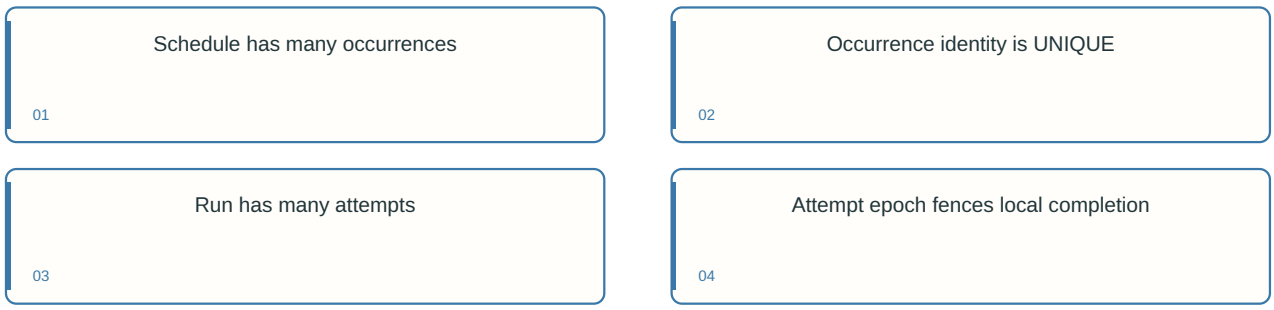


FIG. 22.10.1 **Data ownership and relationships**

One scheduled occurrence can have several attempts, while retaining one logical run identity.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.10.7 · Low-level component organization

ScheduleParser and TimePolicy calculate occurrences. DueScanner finds candidates. RunRepository inserts the run and advances schedule state atomically. Dispatcher routes work, LeaseManager grants attempts, and WorkerAdapter executes it.

Reconciler finds stuck or uncertain outcomes. The scheduler owns when work is due; workers execute under a bounded current claim. Each effect adapter needs a stated repetition or fencing contract. The component split helps test time decisions separately from executing a potentially irreversible external operation.

22.10.8 · High-level system architecture

Keep durable schedules, coordinated scanners or partition owners, dispatch records, bounded workers, attempt state, and reconciliation. Store large versioned payloads separately rather than copying them into every retry message.

Timing control and execution capacity are different paths. Losing the control plane may delay new dispatch while already running work continues. Losing a worker should leave a recoverable run. These behaviours require explicit state and ownership rules, not just a queue attached to a cron loop.

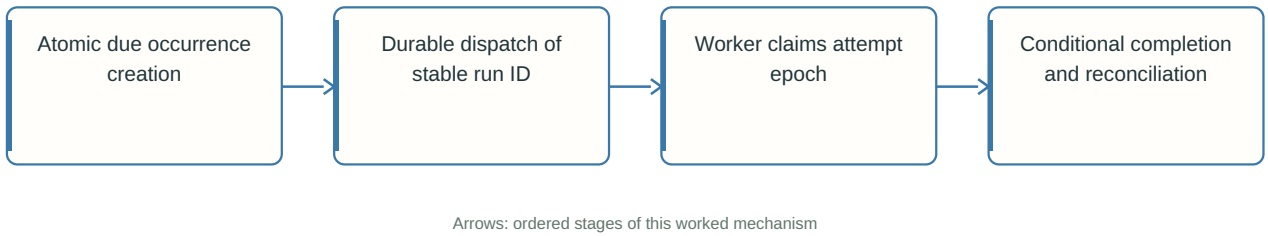


FIG. 22.10.2 **Follow the main operation**

Create due work durably before a current worker attempts its effect.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.10.9 · Data and control flows

A scanner locks or conditionally claims due state, creates the unique occurrence and dispatch intent, advances next_due, and commits. The worker then claims an attempt epoch and executes under the stable run identity.

It renews a lease where supported and records completion only if still current. A replacement can take over expired work, while effect-side identity or fencing limits harmful stale execution. Reconcile runs whose progress never reaches an acceptable terminal state. This closes the history beyond merely starting a callback.

22.10.10 · Capacity estimates

The assumed 27.8 runs/s and ten-second average yield about 278 active executions. If one worker can sustain four independent I/O jobs, the arithmetic minimum is roughly seventy workers before headroom and other limits.

CPU jobs need measured compute capacity instead. A round-hour burst can be far larger than the average, so estimate due-time distribution and queue delay. Include retries and catch-up. A worker count derived from steady averages is not evidence that launch or outage recovery meets its scheduling objective.

22.10.11 · End-to-end latency estimates

Scheduling delay measures due time to dispatch or start. Runtime measures how long the job takes afterward. An illustrative target is that 95% of eligible runs start within five seconds under supported load.

Scanning intervals, claim coordination, queue waits, cold starts, and concurrency limits all consume that budget. A ten-minute task can still meet a five-second start target, so completion deadlines need another rule. Measure both where the business requires them rather than describing every duration as “job latency.”

22.10.12 · Failure scenarios

Test crashes between occurrence creation and schedule advancement; the atomic boundary should prevent a partial accepted result. Lease expiry can overlap a resumed old worker with a replacement.

Clock changes, timezone transitions, malformed inputs, and poison jobs need policies. Reject stale local completions by epoch. External effects accepted before ownership changes remain a separate reconciliation concern. A timer can be correct about due time while the workflow is still incorrect about identity or repeated execution.

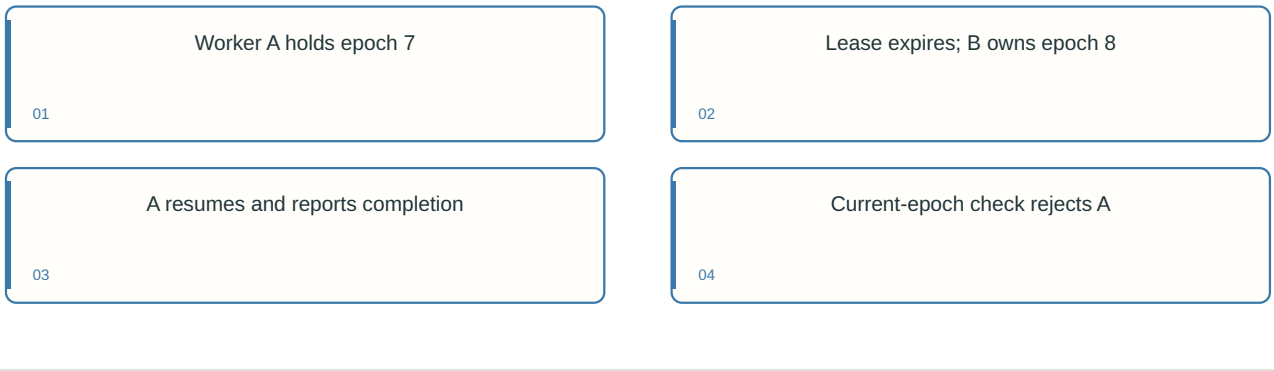


FIG. 22.10.3 **Follow the failure and its correction**

A replaced worker can resume, so its stale completion must be rejected at the protected boundary.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.10.13 · Scalability improvements

Index due time, claim bounded batches, and partition schedules with stable ownership when scan work becomes limiting. Scale independent workers by their actual CPU, memory, and I/O use.

Keep long, high-priority, and CPU-heavy work from consuming each other's whole budget. Catch-up after an outage also needs limits so millions of overdue runs do not starve current critical jobs. Replacing a global lock requires preserving atomic occurrence creation, not merely removing the contention point and hoping duplicates will be rare.

22.10.14 · Consistency requirements

Advancing the schedule and creating its occurrence need uniqueness and atomicity. Delivery to workers may repeat. Current-epoch completion protects local progress; side-effect identity or enforced fencing protects the relevant external effect.

A lease alone does not establish exactly-once execution. Define missed-run and overlap policy so timing races do not decide which logical runs exist. The accepted schedule, occurrence, attempt, and effect are separate scopes with connected but different consistency requirements.

22.10.15 · Security model

Authorise schedule ownership, payload references, run views, and cancellation. Validate supported executable work types rather than permitting arbitrary code or unrestricted URLs through the API.

Bind each run to tenant and immutable input. Give the worker only the required capabilities and audit administration. A global administrator credential in every worker would make one malformed or compromised job unnecessarily powerful. Follow access from stored schedule through dispatched task to the actual resource used by the job.

22.10.16 · Deployment approach

Roll schedule and run formats out compatibly, including lease and epoch meanings. Drain new claims and preserve progress before workers terminate. Reconcile queues against authoritative run state during recovery.

An older restored schedule table can lag external effects already executed. Stable occurrence identity and evidence are needed before replay. Test control-plane interruption separately from worker loss. Recovery must restore the meaning of due work and prior outcomes, rather than simply restart scanning.

22.10.17 · Observability design

Measure due-to-dispatch and due-to-start delay, oldest eligible work, repeated occurrence attempts, lease expiry, stale completion rejection, retries, duration, and tenant fairness.

Check that one occurrence maps to one logical run. CPU can stay low while a broken scanner creates no work, so monitor generation and completion directly. Alert on accepted jobs with no progress. Detailed histories should distinguish one run from its attempts, allowing operators to recognise repetition without interpreting it as new scheduled demand.

22.10.18 · Infrastructure cost considerations

Costs include scanning, indexes, history, dispatch, workers, spare capacity, external calls, and recovery. At 2 kB per run and 100,000 runs/hour, raw daily history is 4.8 GB before attempts and indexes.

Compare scan frequency and batching against the start-delay target. A cheaper design that silently misses due runs offers a different service. Use explicit rates and include catch-up and idle headroom so normal execution cost does not hide the work required after a failure.

22.10.19 · Alternative architectures and trade-offs

Polling a due-time index is simple and recoverable. Timer wheels or delayed-message facilities may improve some timing workloads while needing durable reconstruction. Central scanning simplifies ownership; partitioned scanning divides work and adds coordination.

Long-lived workers reduce startup costs, while managed invocation can fit bursts under different limits. Compare timing precision, job duration, recovery, and effects. The timer data structure answers only part of the design; a fast due lookup does not establish safe occurrence or worker semantics.

22.10.20 · Architecture evolution from small scale to large scale

First implement deterministic time policy and atomic occurrence creation. Add durable dispatch and bounded execution. Introduce leases and epochs together with a safe effect identity before claiming worker takeover.

Partition whichever scan or execution path measurements identify. Regional ownership requires a supported transfer and partition policy for due work. Preserve stable identities throughout so the next deployment does not reinterpret retries as new occurrences or lose the evidence needed to resolve old runs.

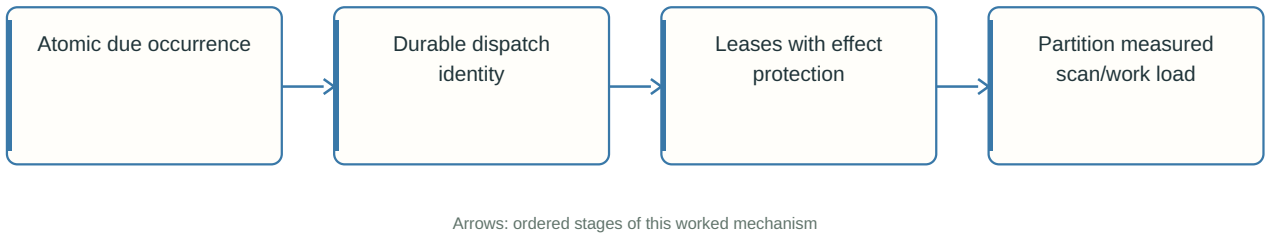


FIG. 22.10.4 **How the design can grow**

Separate scan capacity from execution while preserving occurrence and effect identities.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.10.21 · Design review identifying weaknesses and improvements

Review stable occurrence identity, atomic next_due advancement, stale workers, cancellation, and recovery from old state. Each is a separate potential failure gap.

Run competing scanners at commit boundaries, pause a worker past its lease, cancel during execution, cross a timezone transition, and recover after a long outage. Check which runs should exist and which effects remain uncertain. A callback firing on time once is evidence of timing for one run, not a durable scheduler's full contract.

22.10.22 · A short implementation exercise

Use an injected clock and transactional run table. Start two scanners against the same due occurrence and verify that the unique logical run is created once.

Pause an attempt at epoch 7, replace it with epoch 8, then submit the old completion. The local store must reject it. Identify the further key or fencing enforcement needed at an external effect receiver. This separates the local progress guarantee from unsupported claims about physically executing the job once.

A small implementation of the key rule

PSEUDOCODE · ONE PART OF THE DESIGN

```
with transaction():
    schedule = claim_due_schedule(now)
    run = insert_unique_run(schedule.id, schedule.next_due)
    insert_dispatch_outbox(run.id)
    advance_next_due(schedule)
# Worker completion compares the current attempt epoch atomically.
# A stale worker's external effects need resource-side protection.
```

Use the implementation exercise to test the rule and the failure described above. You can also run local examples of seat claims, duplicate handling, ledger balance, and workflow epochs in the mechanism collection.

Unit 22 exercises → · Unit 22 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Distributed Periodic Scheduling — Google SRE ↗

   Engineering book. Durable scheduling concerns; atlas schedule architecture is an original worked design.

   Access reviewed · 21 October 2026

2. In Search of an Understandable Consensus Algorithm — Ongaro & Ousterhout ↗

   Original research. Raft reference for Unit 15.

   Access reviewed · 21 October 2026

3. Apache Kafka 4.1: Design ↗

   Official versioned documentation. Partitioning and scoped transactional/processing behavior; external effects are separate.

   Access reviewed · 21 October 2026

Collaborative Document Editor

Prerequisites: Networking Foundations for System Design, Distributed Systems Fundamentals, Replication, Leader Election, and Consensus, Message Queues, Events, and Distributed Workflows

4 original figures

Blue: normal flow · Green: healthy · Amber: caution · Red: failure · Purple: control / shared infrastructure · Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Two collaborators insert text at the same position and receive the edits in different orders. The editing protocol must define a shared interpretation and preserve the context needed to reconnect.

The workload, timing, and prices in this design are teaching assumptions. They are not measured benchmarks or a description of a particular company.

22.11.1 · Functional requirements

Collaborators create a document, edit it together, reconnect after disconnection, and recover required history. They can also export or delete it under access rules.

Choose the editing model: plain text, structured rich text, and embedded objects have different valid operations and conflicts. Cursor position and presence are temporary collaboration hints, separate from durable document content. The first design decision is what an edit means, rather than which transport carries it.

22.11.2 · Nonfunctional requirements

Preserve acknowledged operations and enforce access under a defined revocation policy. Online users need responsive local editing and a bounded supported synchronization delay.

The selected protocol should converge under its assumptions, but equal final strings alone do not establish intended editing behaviour, Unicode correctness, or structural validity. Offline users may return after access

changes. Decide what happens to their unaccepted edits and how the interface distinguishes them from accepted history. A local optimistic display is not yet a durable server acknowledgment.

22.11.3 · Explicit assumptions and workload numbers

Assume 100,000 active documents, 10,000 edited simultaneously, and three active users per edited document. At two operations/s per user, ingestion is 60,000 operations/s.

Assume 200 retained bytes per operation. That is 12 MB/s, or about 1.037 TB/day if sustained. Actual editing duration, batching, snapshots, metadata, fan-out, and retention change the estimate. A peak concurrency count must not silently become a full-day activity assumption when estimating real storage.

22.11.4 · Naive initial architecture

Begin with whole-document updates carrying an expected version. Two conflicting writers cannot both replace the same version; one receives a conflict. This is a reasonable simple contract but an awkward real-time editing experience.

Operation-based synchronization addresses finer-grained concurrent changes. Use a reviewed OT or CRDT model suited to the document structure. Applying arbitrary text patches in arrival order is not enough to establish convergence. The algorithm's operation meaning, context, identity, and recovery rules must work together.

22.11.5 · Core API contracts

Document creation returns an identity. Sync messages carry document and replica identities, stable operation identifiers, known progress, and protocol version. A snapshot request returns a versioned base with explicit covered history.

Authorise membership changes and exports separately. Clients must not impersonate another replica to alter its operation history. Define acknowledgment and reconnect behaviour so a client knows which edits are accepted, pending, or rejected. A successful transport handshake grants no permission to every later edit or export.

22.11.6 · Data model and schema

Store document owner and protocol version, membership, accepted operation identities with ordering or causal context, and snapshots naming their covered positions. Compaction metadata records what history

can safely be retired.

Sequence CRDTs need stable element identifiers and ordering rules. OT needs the transform context required by its supported offline interval. A snapshot is usable only when the receiver knows which operations it already includes. Dropping that relationship can cause omissions or repeated application even when both snapshot and log files are individually intact.

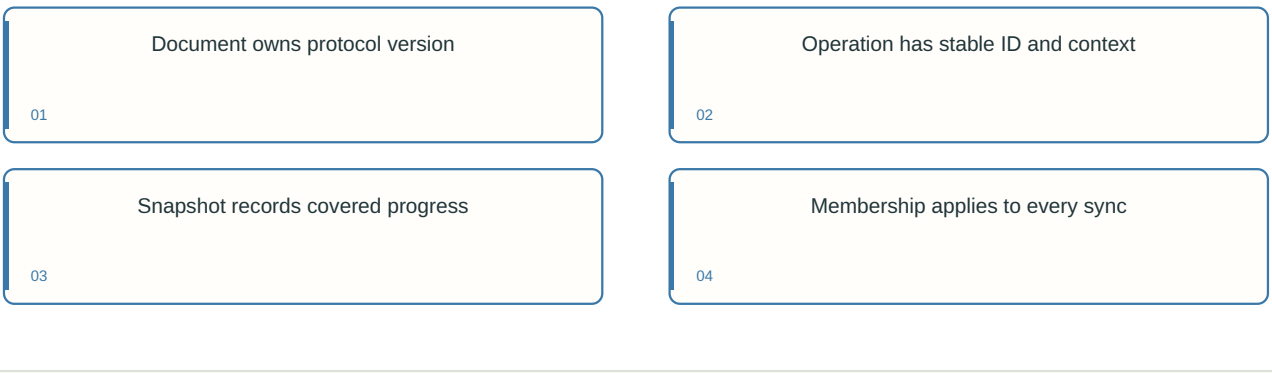


FIG. 22.11.1 **Data ownership and relationships**

Operations, snapshots, and retained context describe the same document history under one protocol.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.11.7 · Low-level component organization

DocumentService owns identity and membership. SyncCoordinator validates the protocol, EditModel applies its rules, and OperationLog persists accepted work. SnapshotBuilder prepares recovery bases, while Compactor retires only history the protocol permits.

Presence uses a separate temporary path. Keep the deterministic editing core apart from rendering and transport so tests can deliver operations in controlled orders. Each component should expose its progress and ownership rather than interpreting any gateway success as a committed edit.

22.11.8 · High-level system architecture

Use authenticated collaboration gateways, client replicas, durable operation storage, snapshots, and the coordination or merging arrangement required by the chosen algorithm.

OT commonly relies on ordered server context and transformation. CRDT approaches use their defined identities and merge semantics. These are different families, not interchangeable labels on one undefined box. Select one supported protocol per document version and plan any transition. The serving topology must preserve that protocol's persistence and context needs across gateway replacement.

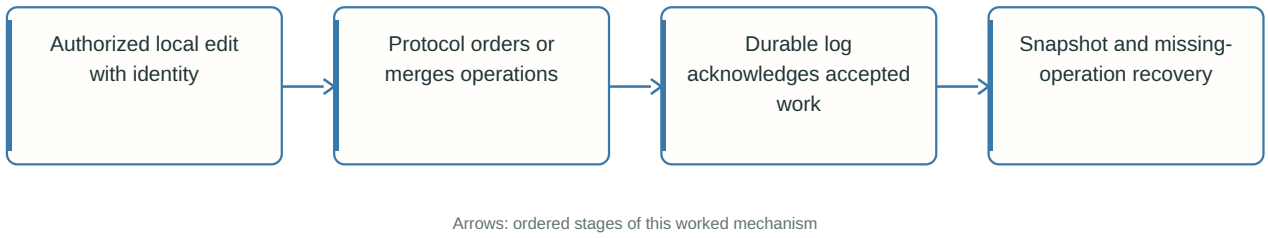


FIG. 22.11.2 **Follow the main operation**

Local edits become accepted operations and reconnectable history through explicit sync rules.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.11.9 · Data and control flows

A client applies an allowed local edit optimistically and assigns it stable identity. The server verifies access and protocol context, persists accepted work, and distributes or acknowledges it under the chosen contract.

On reconnect, exchange missing operations or a compatible snapshot and continuation. Ignore repeated operations by identity. Surface rejected offline changes rather than implying they were accepted earlier. This flow keeps local responsiveness separate from durable agreement and access decisions while still giving the user a clear way to recover progress.

22.11.10 · Capacity estimates

The assumed 60,000 operations/s at 200 bytes yields 12 MB/s of raw history. Distribution to collaborators and extra copies add work. One large popular document can concentrate transforms, merge metadata, and gateway traffic.

Snapshots trade replay work for storage and background computation. Measure actual batching, document sizes, offline duration, and tombstone growth. Do not use an average document to size the hottest editor. Independent documents are easy to distribute compared with one shared stream of strongly related operations.

22.11.11 · End-to-end latency estimates

Local optimistic editing can update the screen before a network round trip, while the interface still shows unsynchronised work. Assume 30 ms ingress, 20 ms durable acceptance, and 50 ms peer delivery: the

example synchronization path totals 100 ms before additional waiting.

This is not measured p99. Geographic distance, disconnection, and large documents change expectations. Track pending duration separately from client frame responsiveness. A smooth local animation does not establish that an acknowledged edit can be reconstructed after the client disappears.

22.11.12 · Failure scenarios

Two users insert at the same position. The protocol needs a deterministic interpretation. Repeated delivery must not insert the same character twice. Missing history, unsafe compaction, or incompatible snapshots can make reconnect fail.

Test stale membership and resource-heavy malformed operations too. A short online convergence demo does not cover a long partition or deleted metadata. Store enough failing-history context to reproduce problems without logging all confidential content. The recovery assumptions must match the offline behaviour the product advertises.

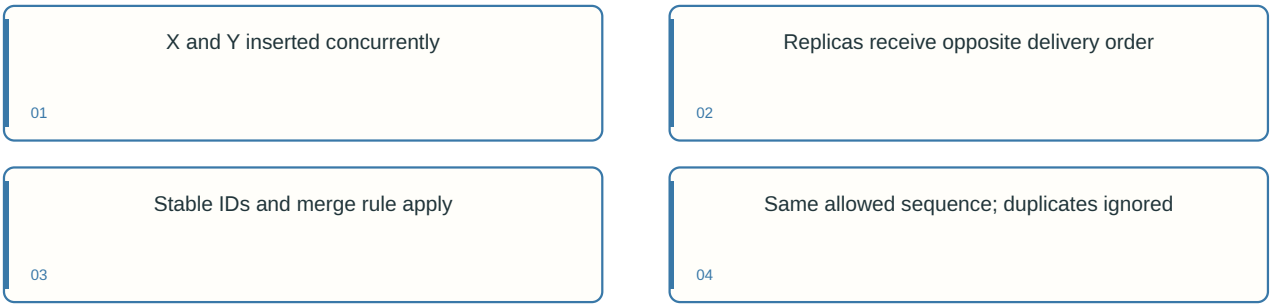


FIG. 22.11.3 **Follow the failure and its correction**

Concurrent or repeated operations need deterministic interpretation and stable identity.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.11.13 · Scalability improvements

Distribute independent documents, batch suitable operations, bound gateway buffers, and snapshot to shorten replay. A single hot document may need a specific coordination and fan-out design rather than ordinary hash partitioning.

Compaction must preserve the supported offline and merge horizon. Removing an old identifier because it looks unused can make a returning operation impossible to interpret safely. Scale cursor updates separately

from durable edits. They have different retention and correctness needs, even when both use the same live connection.

22.11.14 · Consistency requirements

OT transforms concurrent edits using their relevant ordered context; an insertion can shift another operation's position. A sequence CRDT can use stable element identifiers and a common tie-break rule for concurrent insertions.

If X and Y are inserted after the same element, replicas need compatible rules to reach the intended shared sequence. Neither acronym specifies all rich-text, undo, or user-intent behaviour. Authorisation and durable acknowledgment are separate properties. Choose and test the actual algorithm rather than assuming convergence supplies every document invariant.

22.11.15 · Security model

Authorise sync sessions and individual operations according to the revocation policy. Long-lived connections and offline clients need explicit membership checks at later boundaries. Restrict exports and snapshots as well as the editor page.

Validate protocol inputs and avoid leaking content through presence or logs. End-to-end encryption changes what the server can search, validate, retain, and revoke; design those consequences if required. TLS secures the transport but does not provide that document-level product guarantee.

22.11.16 · Deployment approach

Negotiate protocol versions and keep stored history interpretable during rollout. Never reuse an operation identity with a silently changed algorithm. Gateways need reconnect and durable recovery rather than reliance on one client's memory.

Validate snapshots before retiring their source history. Test restoration from persisted artifacts. Protocol migration and rollback affect stored documents, not just application containers, so maintain a supported transition for old clients and offline context. A code rollback that misreads accepted operations can damage otherwise durable content.

22.11.17 · Observability design

Measure pending and accepted edits, synchronization delay, repeated-operation recognition, reconnect gaps, snapshot age, replay duration, compaction growth, protocol rejection, and access denial.

Keep reproducible multi-client histories for failing interleavings. Test convergence alongside access and structural validity. Users' unsynchronised duration reveals harm that gateway latency may miss. Use privacy-safe identifiers and limited diagnostic artifacts rather than copying raw private documents into every trace or log.

22.11.18 · Infrastructure cost considerations

Costs include history and snapshots, connection serving, fan-out, protocol computation, compaction, and telemetry. The sustained-rate arithmetic needs replacement with actual activity duration when planning expenditure.

Longer offline support may require more context or metadata. Compare costs at the same supported recovery horizon. Deleting information needed by returning clients makes a cheaper but weaker service. Keep storage savings tied to a valid compaction rule rather than an arbitrary age threshold.

22.11.19 · Alternative architectures and trade-offs

Whole-document version checks suit low collaboration and simple explicit conflicts. OT supports particular ordering and transformation models. CRDTs support specified merging with identifier, metadata, and cleanup costs.

A well-tested editor library may be preferable to a custom algorithm. Evaluate Unicode graphemes, selections, undo, rich-text structure, and offline context as well as final convergence. No family is universally superior; choose the complete semantics required by this document model and the team's ability to verify and maintain them.

22.11.20 · Architecture evolution from small scale to large scale

Start with versioned documents and clear conflicts. Add a tested operation model, durable history, and reconnect. Then introduce snapshots and justified compaction before distributing independent documents.

Isolate hot workloads and add regional or offline merging only within the protocol's supported assumptions. Preserve stable identities, access policy, and deterministic regression histories. Each stage should retain

the meaning of accepted edits; a more distributed editor is not an improvement if old clients can no longer recover their supported context.

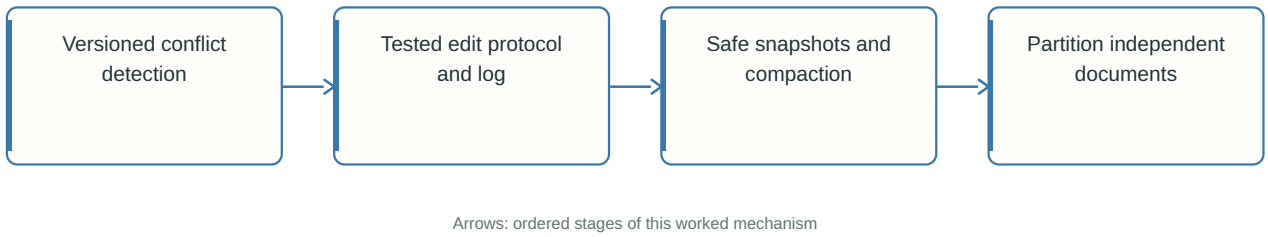


FIG. 22.11.4 **How the design can grow**

Distribute documents and compact history only within the supported offline and merge contract.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.11.21 · Design review identifying weaknesses and improvements

Review convergence, offline duration, history retirement, and revocation. Deliver concurrent inserts in different orders, repeat a delete, lose an acknowledgment, and reconnect from an old snapshot plus later operations.

Also reconnect a revoked user and inspect structural validity. Equal strings alone can conceal an unauthorised edit or invalid document structure. Check those properties separately. The review should name the protocol and assumptions under which each claimed result follows.

22.11.22 · A short implementation exercise

Implement a miniature merge of two insertions with stable IDs and an explicit tie-break rule. Deliver them in both orders and repeat each. Verify the same final sequence and no duplicate insertion.

Document the limited scope: this is not a complete rich-text CRDT. Add snapshot-context and revoked-client checks to expose other boundaries. The exercise teaches why identity and deterministic ordering matter, without treating a two-operation example as evidence for a full production editor.

A small implementation of the key rule

PSEUDOCODE · ONE PART OF THE DESIGN

```
def receive(operation, document):
    with transaction():
        current = lock_document(document.id)
        require_authorized(operation.actor, current)
        require_protocol_version(operation, current)
        if log.contains(operation.id): return prior_ack(operation.id)
        next_state = edit_model.apply(current.state, operation)
        append_unique_operation(operation)
        commit_state_and_progress(current, next_state)
    return durable_ack(operation.id)
# The edit model must implement a specified, tested algorithm.
```

Use the implementation exercise to test the rule and the failure described above. You can also run local examples of seat claims, duplicate handling, ledger balance, and workflow epochs in the mechanism collection.

Unit 22 exercises → · Unit 22 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. A Conflict-Free Replicated JSON Datatype — Kleppmann and Beresford ↗

   Original research · 2017 revision. One formal CRDT model; the miniature tie-break exercise is not this complete implementation.

   Access reviewed · 21 October 2026

2. RFC 6455: The WebSocket Protocol, 2011 ↗

   Original standard. Bidirectional transport framing; not durable chat delivery.

   Access reviewed · 21 October 2026

3. OWASP: Authorization ↗

   Official project guidance · accessed 21 Oct 2026. Least privilege, deny by default, and per-resource enforcement.

   Access reviewed · 21 October 2026

Large-Scale Document Processing Platform

Prerequisites: File Systems, Search Infrastructure, and Data Pipelines, Message Queues, Events, and Distributed Workflows, Reliability, Fault Tolerance, and Disaster Recovery, Security Architecture and Privacy, Observability, Monitoring, and Incident Engineering, Deployment, Cloud Infrastructure, and Platform Architecture

4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A document can finish extraction yet still need review, publication, or deletion. Model the whole lifecycle so each result is usable, authorised, and traceable to its input.

The workload, timing, and prices in this design are teaching assumptions. They are not measured benchmarks or a description of a particular company.

22.12.1 · Functional requirements

A tenant uploads a document, the system classifies it and extracts fields, and uncertain results go to human review. Validated output can be published for authorised search or export. The user can inspect progress, correct results, reprocess versions, and request deletion.

Successful upload means accepted bytes under the upload contract. It does not mean extraction, review, or publication has finished. Give these stages distinct states so users can understand what is ready and what remains unresolved.

22.12.2 · Nonfunctional requirements

Preserve accepted inputs and required workflow progress under the chosen failures. Results must identify their exact source and pipeline versions. Protect tenant data and set processing-age objectives for supported work.

Do not silently turn uncertain extraction into verified fact. Human decisions need durable actor and rationale information. Search has its own publication delay and access obligations. A complete result is usable, traceable, and permitted output, rather than a worker merely setting one finished flag.

22.12.3 · Explicit assumptions and workload numbers

Assume one million files/day averaging 2 MB: raw input is 2 TB/day. The arrival average is about 11.6 files/s; a tenfold assumed peak is about 116/s.

At an illustrative five CPU-seconds per file, daily extraction work is five million CPU-seconds, or about 1,389 CPU-hours. Classification, I/O, retries, and human wait are additional work. Actual model behaviour and file distribution need measurement. These quantities teach resource accounting and do not assert an extraction benchmark or accuracy rate.

22.12.4 · Naive initial architecture

A prototype can save a file and extract it during the upload request. Large or difficult inputs then extend the request, process loss can erase progress, and human review has no natural place to wait.

Separate resumable byte ingestion from durable workflow stages. Keep immutable source versions and identified outputs. Retries and corrections can then preserve the evidence needed to explain earlier results. The improvement addresses a concrete recovery and lifecycle problem before adding workers for capacity.

22.12.5 · Core API contracts

POST /documents creates an upload intent, and completion verifies stored bytes. Status exposes stages; field reads return versioned values and provenance. Review decisions, versioned reprocessing, and tracked deletion have separate operations.

Authorise search and export only over permitted published output. Explain whether deletion is accepted or fully completed, and what a reprocessing request changes. A request for a new pipeline version must not silently reinterpret old output under the same identity.

22.12.6 · Data model and schema

Document identity binds tenant, source version, digest, and state. Upload sessions track bytes; workflows identify input and pipeline versions. Stage attempts carry epochs, artifacts carry kind/version/object references, and fields carry values, confidence, source regions, and extractor identity.

Review records identify decision, actor, and version. Publication and deletion have tracked progress. Keys preserve tenant and version scope. Retain enough provenance to audit a result, with an explicit lifecycle rather than assuming every raw intermediate should remain forever.

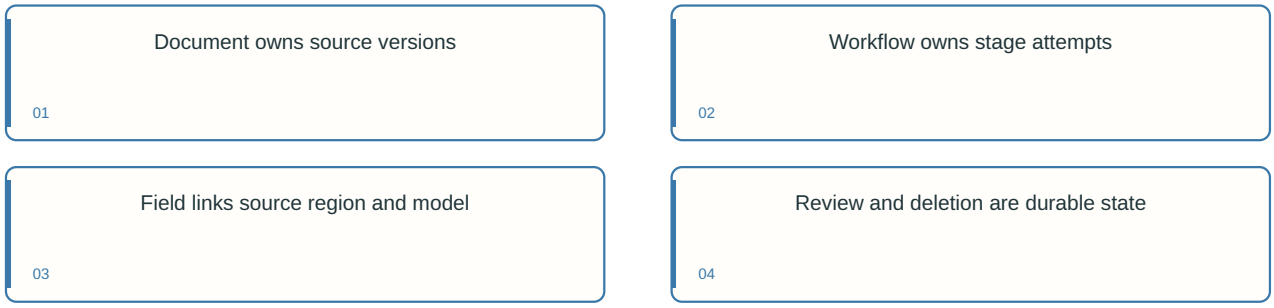


FIG. 22.12.1 **Data ownership and relationships**

Fields and decisions retain tenant, input, transform, and evidence relationships.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.12.7 · Low-level component organization

UploadService and ContentValidator check ingestion. WorkflowCoordinator owns stages. Versioned Classifier and Extractor adapters produce results; ProvenanceRecorder links evidence. ReviewPolicy selects the supported review path, ReviewService records decisions, and Publisher builds permitted projections.

Reconciler repairs stuck or orphaned work. Workers receive limited capabilities and cannot change tenant identity because a message supplied a different field. This division keeps model output generation separate from deciding whether it is valid, reviewed, and ready for others to use.

22.12.8 · High-level system architecture

Use immutable object storage, durable relational workflow metadata, task delivery, bounded isolated compute or model workers, review interfaces, and a search projection. Bulk bytes flow through scoped storage access while metadata controls progress.

A document waiting hours for review should release its compute slot and database transaction. The workflow remains durable without keeping its process alive. Keep source, artifacts, human decisions, and publication generations linked so failure in one component can be investigated and repaired.

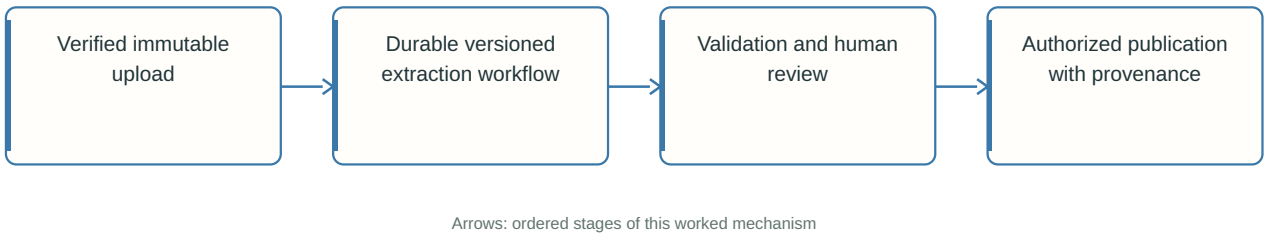


FIG. 22.12.2 **Follow the main operation**

Ingestion, extraction, review, and publication have distinct durable progress.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.12.9 · Data and control flows

Verify upload identity and digest, create a stable workflow, classify, extract, and validate structure and meaning. Save evidence, then either publish or request review. A human decision creates a new validated version and publication intent.

Each stage commits conditionally under its current attempt epoch. Reprocessing creates a new run over an identified input, preserving earlier provenance. Deletion blocks later inappropriate publication. This sequence makes resumed or stale workers safe locally instead of relying on the assumption that only one attempt ever runs.

22.12.10 · Capacity estimates

Thirty days of 2 TB/day raw input requires 60 TB before extra copies. Five million CPU-seconds/day divided by 86,400 gives about 57.9 continuously busy cores. At an assumed 60% target utilisation, that becomes about 96.5 cores before other work and failure margin.

A tenfold arrival peak can exceed this average capacity. Queue age, scaling delay, admission, and peak duration determine whether processing objectives still hold. Include large-file and model-memory distribution rather than treating every input as the average file.

22.12.11 · End-to-end latency estimates

Upload transfer and durable acceptance have one timeline. Automated processing includes queueing, classification, extraction, validation, and publication. Human review is a separate waiting state.

Assume two seconds queued, one second classification, five seconds extraction, and one second validation/publication. The automated path totals nine seconds in this model. These are testable assumptions, not measured model performance. Report the relevant stage and age so a user can distinguish slow computation from a pending human decision.

22.12.12 · Failure scenarios

Uploads can be incomplete, inputs corrupt or hostile, and workers interrupted after writing output but before saving completion. Repeated tasks, stale attempts, invalid model output, concurrent reviews, and missing provenance can each break a different part of the workflow.

Use stable artifact identity and conditional transitions. Quarantine unsupported output with a reason. Check deletion during processing. A completed label with missing, unauthorised, or unexplained output is still a failed business result even when no worker exception was recorded.

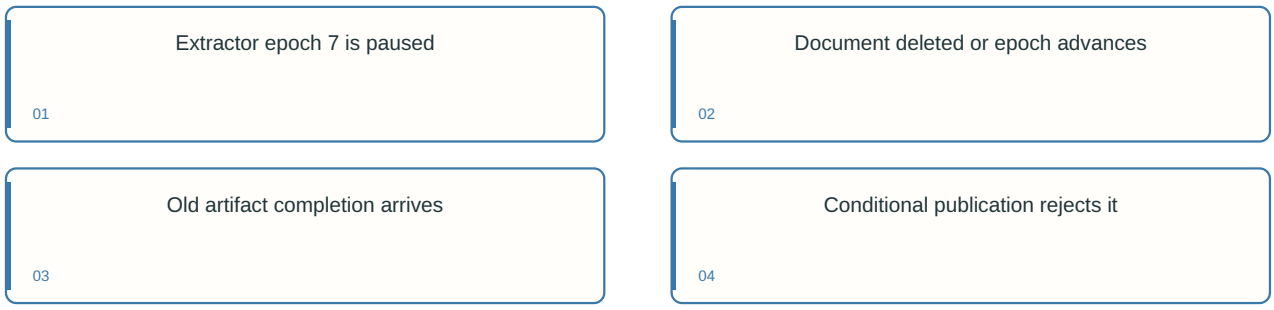


FIG. 22.12.3 **Follow the failure and its correction**

A stale attempt cannot publish after its authority or document has been withdrawn.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.12.13 · Scalability improvements

Move bulk bytes through direct storage and scale independent extraction with bounded workers. Partition metadata by a justified tenant or workflow key while protecting against large files and hot tenants.

Model batching and memory limits need explicit testing. More workers can saturate storage or an external model. Give replay and backfill separate capacity so pipeline upgrades do not starve live inputs or deletion. Scale according to the stage that limits useful output rather than multiplying every worker equally.

22.12.14 · Consistency requirements

Source versions are immutable. Related workflow and review changes need suitable atomic transitions. Artifacts identify their exact input and transform, while repeated stage delivery uses stable output identity and current-epoch completion.

Search may lag publication while respecting access. External model calls can be uncertain or nondeterministic. Retain relevant request and version information, validate results, and state the model's actual reproducibility boundary. Recording a model name alone does not establish that rerunning it will yield the same answer.

22.12.15 · Security model

Enforce tenant scope on objects, tasks, review, search, export, and audit paths. Restrict worker resources and egress, validate supported content, and keep credentials and private data out of routine telemetry.

Treat extracted text as untrusted content, not infrastructure instructions. Reviewers need scoped access and traceable decisions. Deletion covers derived sensitive results as well as raw bytes. A protected upload endpoint is only the first boundary in a pipeline that repeatedly reads and republishes the document's information.

22.12.16 · Deployment approach

Version pipelines and schemas compatibly. Evaluate new models on authorised samples or controlled cohorts and review quality before replacing published output. Retain the previous usable generation until the transition is verified.

Drain workers through durable attempts. Restore exercises must reconnect objects, workflow state, human decisions, and deletion history. Restoring only the metadata database may leave missing artifacts or revive obsolete publication. Test the recovered whole journey before treating it as a usable platform.

22.12.17 · Observability design

Measure accepted bytes, stage waiting, computation, retries, stale-completion rejection, invalid outputs, review age, provenance completeness, publication lag, tenant denials, and deletion progress.

Confidence is not measured accuracy. Use an appropriate labelled evaluation for quality claims. Link source, run, attempt, model, artifact, and reviewer through stable identities in privacy-safe evidence. This

lets an operator find where one document stopped and whether its result is actually usable, rather than merely observe a busy worker fleet.

22.12.18 · Infrastructure cost considerations

Include raw and derived storage, CPU or model calls, retries, review labour, search, transfer, telemetry, and recovery. The assumed five million CPU-seconds/day is a compute input, while review may dominate actual operating cost.

Use explicit unit rates and a stated review fraction when comparing choices. Avoid invented accuracy or throughput numbers. Compare equivalent provenance, access, quality, and recovery needs; dropping review or history may reduce the bill while changing the service's required outcome.

22.12.19 · Alternative architectures and trade-offs

Synchronous processing suits a tiny demonstration but couples upload to model delay and failure. Durable stages add metadata and operating work while supporting recovery and human waits. Batch inference may improve resource use while increasing waiting and memory needs.

Managed workflows or models transfer selected duties but do not automatically provide provenance, validation, tenant isolation, or review policy. Compare the full output and recovery contract. The simplest viable architecture is the one that explains and supports these outcomes for the intended workload.

22.12.20 · Architecture evolution from small scale to large scale

Start with immutable upload and explicit status. Add durable stages and stable artifacts before increasing parallelism. Give provenance and human decisions first-class identities rather than attaching them after extraction.

Then scale workers and storage independently, and introduce authorised search and versioned reprocessing. Regional placement needs a named locality or disaster requirement and a compatible lifecycle design. Every stage should keep prior results explainable and prevent old work from bypassing current access or deletion decisions.

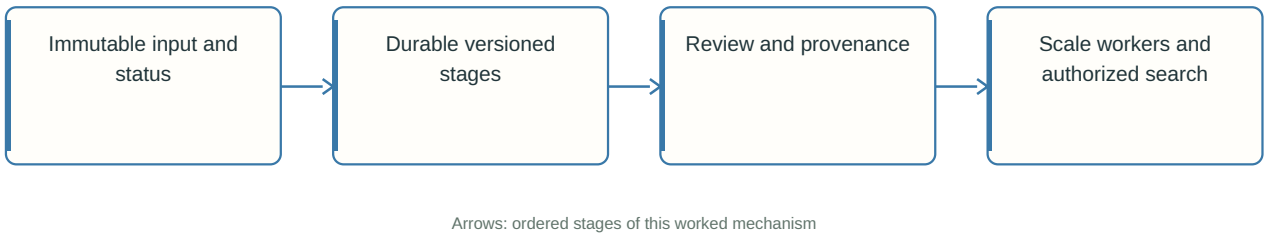


FIG. 22.12.4 **How the design can grow**

Scale stages independently while retaining provenance, access, and deletion behaviour.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.12.21 · Design review identifying weaknesses and improvements

Check whether any stage silently changes its input version, whether a superseded worker can publish after deletion, and whether reprocessing preserves human decisions appropriately.

Test repeated tasks, interruption after artifact write, invalid output, concurrent reviews, stale completion, missing objects, index lag, and full restoration. A few correct extracted fields in one run do not establish recoverable progress or provenance. The review needs evidence for the connected lifecycle, including work that ends in rejection, review, or deletion.

22.12.22 · A short implementation exercise

Implement three stages using immutable input identity, result version, and attempt epoch. Stop after writing an artifact but before recording completion, then replay without duplicating the established output.

Add a review state that releases compute resources while waiting. Finally pause an old worker, delete the document, and submit that worker's completion. Publication must remain blocked. Explain how the persisted condition prevents stale work from reversing deletion rather than relying on worker timing.

A small implementation of the key rule

PSEUDOCODE · ONE PART OF THE DESIGN

```
def complete_stage(run, epoch, artifact):
    require(artifact.input_version == run.input_version)
    validate_output_schema_and_provenance(artifact)
    with transaction():
        current = lock_workflow(run.id)
        require(current.epoch == epoch and not current.deleted)
        insert_artifact_once(run.id, artifact.version, artifact)
        advance_or_request_review(current, artifact)
        insert_next_stage_outbox_if_needed(current)
# Retry reuses identities; old attempts cannot publish new state.
```

Use the implementation exercise to test the rule and the failure described above. You can also run local examples of seat claims, duplicate handling, ledger balance, and workflow epochs in the mechanism collection.

Unit 22 exercises → · Unit 22 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. OpenLineage: core model and documentation ↗

   Official specification documentation · 1.54.0 at access. Dataset, job, and run lineage concepts.

   Access reviewed · 21 October 2026

2. Apache Kafka 4.1: Design ↗

   Official versioned documentation. Partitioning and scoped transactional/processing behavior; external effects are separate.

   Access reviewed · 21 October 2026

3. OWASP: Authorization ↗

   Official project guidance · accessed 21 Oct 2026. Least privilege, deny by default, and per-resource enforcement.

   Access reviewed · 21 October 2026

4. Transactional Outbox — Chris Richardson ↗

   Pattern author's explanation. Publication intent and duplicate delivery.

   Access reviewed · 21 October 2026

THE COMPLETE EDITION

Contents, from objects to systems

All 22 units and 99 chapters are available. Follow prerequisites or use a focused reading path.

Part 1 · System design foundations

01 · Introduction to System Design — READ NOW

- 1.1 · From a program to a system
- 1.2 · Organising code into modules
- 1.3 · From requirements to design choices
- 1.4 · Drawing the system at different levels

02 · Requirements, Estimation, and Engineering Trade-offs — READ NOW

- 2.1 · Turn user journeys into workloads
- 2.2 · Measure the experience
- 2.3 · Estimate the resources
- 2.4 · Queues, bottlenecks & defensible decisions

03 · Networking Foundations for System Design — READ NOW

- 3.1 · Addresses, sockets & transports
- 3.2 · A secure request, one round trip at a time
- 3.3 · Intermediaries & persistent connections
- 3.4 · Discover, route & survive the network

04 · Runtime, Processes, Threads, and Concurrency READ NOW

4.1 · The runtime owns your resources

4.2 · Tasks, threads & event loops

4.3 · Shared memory & coordination

4.4 · Visibility, cancellation & overload

Part 2 · Low-level design & software architecture

05 · Object-Oriented and Modular Software Design READ NOW

5.1 · Model behavior with objects and values

5.2 · Principles under pressure

5.3 · Side effects, domain models & collaboration

5.4 · Design for change, testing & functions

06 · Design Patterns, Dependency Injection, and Object Lifecycles READ NOW

6.1 · Dependencies, inversion & composition

6.2 · Lifetimes, ownership & cleanup

6.3 · Injection, locators & factories

6.4 · Creational patterns

6.5 · Structural patterns

6.6 · Behavioral patterns

6.7 · Enterprise patterns & boundaries

07 · Application Architecture and Domain-Driven Design READ NOW

7.1 · Layers, monoliths & ports

7.2 · Model a bounded context

7.3 · Keep business policy inside

7.4 · Evolution without a distributed monolith

08 · API Design and Communication Contracts READ NOW

8.1 · Model resources and messages

8.2 · Compatibility & idempotency

8.3 · Control access, load & long operations

8.4 · Evolve a public contract

Part 3 · Data, storage & scalability

09 · Data Models and Storage Technologies READ NOW

9.1 · Storage promises & database families

9.2 · Files, objects & analytical layouts

9.3 · Model relationships and ownership

9.4 · Access patterns, retention & operational cost

10 · Database Internals, Transactions, and Concurrency Control READ NOW

10.1 · Pages, indexes & query plans

10.2 · Commit, logging & versions

10.3 · Races, anomalies & isolation

10.4 · Pools, recovery & ticket correctness

11 · Caching and Content Delivery READ NOW

11.1 · Locality and cache placement

11.2 · Copies, freshness & eviction

11.3 · Hot keys, stampedes & negative results

11.4 · Edges, partitions & the price of a cache

12 · Scalability, Load Balancing, and Data Distribution READ NOW

12.1 · Scale compute & route requests

12.2 · Replicate and partition data

12.3 · Lag, proxies & saturated pools

12.4 · Regions, write paths & scale economics

13 · File Systems, Search Infrastructure, and Data Pipelines READ NOW

13.1 · Upload bytes without losing identity

13.2 · Build and rebuild a search index

13.3 · Move data through batch and streams

13.4 · Quality, provenance & replay

Part 4 · Distributed systems & reliability

14 · Distributed Systems Fundamentals READ NOW

14.1 · Partial failure & the meaning of time

14.2 · Consistency as a reader-visible contract

14.3 · Partitions, CAP, PACELC & quorums

14.4 · Coordination, safety & liveness

15 · Replication, Leader Election, and Consensus READ NOW

15.1 · Choose a replication model

15.2 · Raft and replicated state machines

15.3 · Fences, leases & failure detection

15.4 · Conflicts, CRDTs & regional trade-offs

16 · Message Queues, Events, and Distributed Workflows

READ NOW

- 16.1 · Queues, logs & delivery boundaries
- 16.2 · Retries, consumers & event history
- 16.3 · Read models, the outbox & sagas
- 16.4 · Coordinate durable workflows

17 · Reliability, Fault Tolerance, and Disaster Recovery

READ NOW

- 17.1 · Failure domains & redundancy
- 17.2 · Retries, circuit breakers & isolation
- 17.3 · Shed load and degrade deliberately
- 17.4 · Recover, test & learn from failure

Part 5 · Security & production engineering

18 · Security Architecture and Privacy

READ NOW

- 18.1 · Threats, identity & authorization
- 18.2 · Secrets, encryption & trust boundaries
- 18.3 · Tenant isolation, audit & deletion
- 18.4 · Supply chains and security reviews

19 · Observability, Monitoring, and Incident Engineering

READ NOW

- 19.1 · Observe behavior with three signals
- 19.2 · Choose indicators, budgets & alerts
- 19.3 · Profile and investigate production
- 19.4 · Learn from incidents and trace async work

20 · Deployment, Cloud Infrastructure, and Platform Architecture READ NOW

20.1 · Environments, VMs & containers

20.2 · Control planes, scheduling & configuration

20.3 · Releases, flags & schema migrations

20.4 · Platforms, serverless & ownership

21 · Performance, Cost Optimization, and System Validation READ NOW

21.1 · Budgets, bottlenecks & honest benchmarks

21.2 · Capacity, architecture & cost per outcome

21.3 · Efficiency and a useful test portfolio

21.4 · Fitness functions and design validation

Part 6 · Complete system design case studies

22 · End-to-End Design, Architecture Reviews, and Evolution READ NOW

22.1 · URL Shortener

22.2 · Rate Limiter

22.3 · Chat Application

22.4 · Notification Platform

22.5 · Social Media Feed

22.6 · Video Streaming Platform

22.7 · Ride-Hailing System

22.8 · Online Ticket Booking

22.9 · Payment System

22.10 · Distributed Job Scheduler

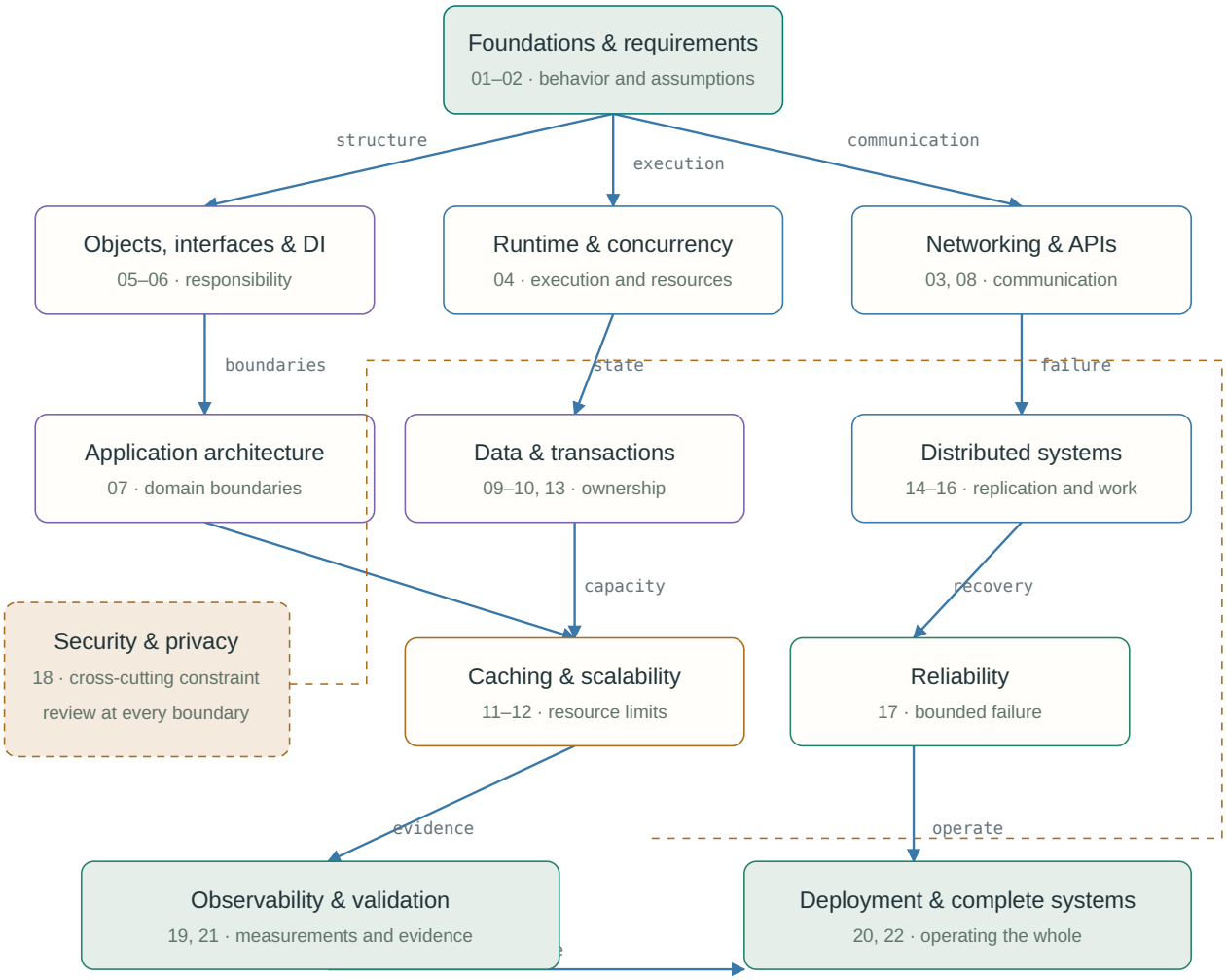
22.11 · Collaborative Document Editor

22.12 · Large-Scale Document Processing Platform

HOW THE IDEAS CONNECT

A map of the territory

The simplified concept spine connects modules, networks, data, distribution, and production engineering.



Simplified concept spine; exact unit and chapter prerequisites appear in Reading paths.

FIG. M.1 **The ideas form a dependency network.**

A simplified prerequisite spine connects low-level structure to storage, networking, distributed behavior, and production engineering. Security constrains the entire design.

Why this view: it shows why a replication or scaling mechanism needs earlier reasoning about state and execution. Author-created illustration.

Solid arrow = conceptual prerequisite · dashed line = cross-cutting constraint. Exact prerequisites are in Reading paths. Author-created illustration.

DEPENDENCY-AWARE ROUTES

Reading paths

Every route below includes the prerequisite closure of its selected units; open any unit for its direct chapter dependencies.

First complete pass

01 → 02 → 03 → 04 → 05 → 06 → 07 → 08 → 09 → 10

Data and scale

01 → 02 → 03 → 04 → 08 → 09 → 10 → 11 → 12 → 13

Distributed correctness

01 → 02 → 03 → 04 → 08 → 09 → 10 → 11 → 12 → 14 → 15 → 16 → 17

Production engineering

01 → 02 → 03 → 04 → 05 → 06 → 07 → 08 → 09 → 10 → 11 → 12 → 14 → 15 → 16 → 17 → 18 → 19 → 20 → 21

Direct prerequisites

Unit	Before this unit
1 · Introduction to System Design	Basic programming
2 · Requirements, Estimation, and Engineering Trade-offs	1
3 · Networking Foundations for System Design	1
4 · Runtime, Processes, Threads, and Concurrency	1
5 · Object-Oriented and Modular Software Design	1
6 · Design Patterns, Dependency Injection, and Object Lifecycles	5
7 · Application Architecture and Domain-Driven Design	5, 6
8 · API Design and Communication Contracts	1, 3
9 · Data Models and Storage Technologies	1
10 · Database Internals, Transactions, and Concurrency Control	4, 9
11 · Caching and Content Delivery	3, 9
12 · Scalability, Load Balancing, and Data Distribution	2, 3, 9, 10, 11
13 · File Systems, Search Infrastructure, and Data Pipelines	8, 9, 10
14 · Distributed Systems Fundamentals	3, 4, 10
15 · Replication, Leader Election, and Consensus	14
16 · Message Queues, Events, and Distributed Workflows	8, 10, 14, 15
17 · Reliability, Fault Tolerance, and Disaster Recovery	2, 12, 15, 16
18 · Security Architecture and Privacy	3, 7, 8, 9
19 · Observability, Monitoring, and Incident Engineering	2, 3, 4, 10

Unit	Before this unit
20 · Deployment, Cloud Infrastructure, and Platform Architecture	3, 7, 17, 18, 19
21 · Performance, Cost Optimization, and System Validation	2, 4, 10, 12, 17, 19, 20
22 · End-to-End Design, Architecture Reviews, and Evolution	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21

The case-study progression

The URL shortener connects estimation, cache freshness, and scaling. Payments connect object boundaries, API identity, transactions, outbox, provider failures, and ledger reconciliation. Tickets connect local concurrency to expiry and external payment. Documents connect immutable input, lineage, durable stages, human review, and publication.

22.1 · URL Shortener — API Design and Communication Contracts, Data Models and Storage Technologies, Caching and Content Delivery, Scalability, Load Balancing, and Data Distribution

22.2 · Rate Limiter — Requirements, Estimation, and Engineering Trade-offs, Runtime, Processes, Threads, and Concurrency, Scalability, Load Balancing, and Data Distribution, Reliability, Fault Tolerance, and Disaster Recovery

22.3 · Chat Application — Networking Foundations for System Design, Data Models and Storage Technologies, Distributed Systems Fundamentals, Message Queues, Events, and Distributed Workflows

22.4 · Notification Platform — Design Patterns, Dependency Injection, and Object Lifecycles, API Design and Communication Contracts, Message Queues, Events, and Distributed Workflows, Reliability, Fault Tolerance, and Disaster Recovery

22.5 · Social Media Feed — Data Models and Storage Technologies, Caching and Content Delivery, Scalability, Load Balancing, and Data Distribution, Message Queues, Events, and Distributed Workflows

22.6 · Video Streaming Platform — Networking Foundations for System Design, Caching and Content Delivery, File Systems, Search Infrastructure, and Data Pipelines, Deployment, Cloud Infrastructure, and Platform Architecture

22.7 · Ride-Hailing System — Data Models and Storage Technologies, Scalability, Load Balancing, and Data Distribution, Distributed Systems Fundamentals, Message Queues, Events, and Distributed Workflows

22.8 · Online Ticket Booking — API Design and Communication Contracts, Database Internals, Transactions, and Concurrency Control, Message Queues, Events, and Distributed Workflows, Security Architecture and Privacy

22.9 · Payment System — API Design and Communication Contracts, Database Internals, Transactions, and Concurrency Control, Message Queues, Events, and Distributed Workflows, Reliability, Fault Tolerance, and Disaster Recovery, Security Architecture and Privacy

22.10 · Distributed Job Scheduler — Runtime, Processes, Threads, and Concurrency, Replication, Leader Election, and Consensus, Message Queues, Events, and Distributed Workflows, Reliability, Fault Tolerance, and Disaster Recovery

22.11 · Collaborative Document Editor — Networking Foundations for System Design, Distributed Systems Fundamentals, Replication, Leader Election, and Consensus, Message Queues, Events, and Distributed Workflows

22.12 · Large-Scale Document Processing Platform — File Systems, Search Infrastructure, and Data Pipelines, Message Queues, Events, and Distributed Workflows, Reliability, Fault Tolerance, and Disaster Recovery, Security Architecture and Privacy, Observability, Monitoring, and Incident Engineering, Deployment, Cloud Infrastructure, and Platform Architecture

ALL 22 UNITS / 242 QUESTIONS

Exercises

Five conceptual questions, three scenarios, one design challenge, one implementation exercise, and one advanced thought experiment per unit.

Unit 1 · Introduction to System Design

CONCEPTUAL / C1

A program adds a task to a dictionary and displays “Saved.” What can it do while running, and what happens when it starts with a new dictionary?

Read the explanation →

CONCEPTUAL / C2

What does the signature create(title) tell a caller? What must the contract explain?

Read the explanation →

CONCEPTUAL / C3

The task service depends on a repository. Does this dependency necessarily make the design difficult to change?

Read the explanation →

CONCEPTUAL / C4

How do a high-level view, a low-level view, and a deployment view help explain the task application?

Read the explanation →

CONCEPTUAL / C5

A service stops accepting requests during recovery but keeps every committed task. Which observation concerns availability, and which concerns durability?

Read the explanation →

SCENARIO / S1

Task 7 is committed, but its success reply is lost. The client waits, then retries. Can it conclude that the first attempt failed?

Read the explanation →

SCENARIO / S2

The web adapter and command-line adapter each enforce an 80-character title limit. Only the web adapter is changed to allow 200 characters. How would you prevent this disagreement?

Read the explanation →

SCENARIO / S3

The team runs two application machines, each with its own unrelated local task database. A user sees a task on one request and cannot find it on the next. What could explain this?

Read the explanation →

DESIGN / D1

A handler adds a task to memory and replies “Saved.” A background writer saves it later. Trace a crash that loses the task, then propose a change that meets the restart requirement.

Read the explanation →

IMPLEMENTATION / I1

Run examples/todo.py. Make a test repository raise a storage error. Then use the real repository to save a task, close it, and read the task through a new connection. What should each check show?

Read the explanation →

THOUGHT EXPERIMENT / T1

The team now requires saved tasks to survive destruction of the only disk. Why is ADR 001 insufficient, and what must the next design address?

Read the explanation →

Unit 2 · Requirements, Estimation, and Engineering Trade-offs

CONCEPTUAL / C1

Opening a feed appears to be one action. Why might it create many requests and writes?

Read the explanation →

CONCEPTUAL / C2

Requests keep arriving faster, but successful completions stay at the same rate. What happens next?

Read the explanation →

CONCEPTUAL / C3

How can a 19.9 ms mean hide a request that took one second?

Read the explanation →

CONCEPTUAL / C4

Which requests belong in an event-based error budget?

Read the explanation →

CONCEPTUAL / C5

When is L = λW useful for estimating work in progress?

Read the explanation →

SCENARIO / S1

A traffic spike ends before new instances can start. What protects the service?

Read the explanation →

SCENARIO / S2

Only 80% of a job can run in parallel. What ideal speedup can four workers give?

Read the explanation →

SCENARIO / S3

Why can requiring fifty parallel calls increase tail latency?

Read the explanation →

DESIGN / D1

A team sizes servers from daily average RPS alone. Identify the missing evidence and propose a safer estimation process.

Read the explanation →

IMPLEMENTATION / I1

Implement a capacity worksheet with explicit user activity, peak, and per-instance inputs.

Read the explanation →

THOUGHT EXPERIMENT / T1

How would simultaneous retries change an estimate based on average user sessions?

Read the explanation →

Unit 3 · Networking Foundations for System Design

CONCEPTUAL / C1

A server listens on port 443. Why might a remote caller still fail to use it?

Read the explanation →

CONCEPTUAL / C2

What does the application receive from TCP, UDP, and QUIC?

Read the explanation →

CONCEPTUAL / C3

Why does changing DNS not move every client immediately?

Read the explanation →

CONCEPTUAL / C4

Which access checks can a gateway perform, and which need domain information?

Read the explanation →

CONCEPTUAL / C5

What work does streaming save compared with polling, and what work does it add?

Read the explanation →

SCENARIO / S1

The server commits a write, but the client times out. What can the client conclude?

Read the explanation →

SCENARIO / S2

Why might a cold HTTPS request take longer than a warm request with the same server work?

Read the explanation →

SCENARIO / S3

A database pool is full while CPU is low. What should you inspect first?

Read the explanation →

DESIGN / D1

A payment client retries every timeout as a new request. Trace a duplicate effect and repair the contract.

Read the explanation →

IMPLEMENTATION / I1

Implement a cold/warm request calculator and check invalid stage times.

Read the explanation →

THOUGHT EXPERIMENT / T1

What changes when a local dependency becomes a cross-region call?

Read the explanation →

Unit 4 · Runtime, Processes, Threads, and Concurrency

CONCEPTUAL / C1

How do processes and threads differ from modules in the source code?

Read the explanation →

CONCEPTUAL / C2

Why can an async function still block other requests?

Read the explanation →

CONCEPTUAL / C3

Can a one-core program be concurrent without executing tasks simultaneously?

Read the explanation →

CONCEPTUAL / C4

Why might atomic fields fail to make a transfer between two balances safe?

Read the explanation →

CONCEPTUAL / C5

Why does an earlier timestamp not prove another thread sees an update?

Read the explanation →

SCENARIO / S1

A and B read counter zero, then each writes one. Where was the increment lost?

Read the explanation →

SCENARIO / S2

One callback computes for two seconds on a single event loop. What happens to ready tasks?

Read the explanation →

SCENARIO / S3

Jobs arrive at 150/s and complete at 100/s. Does a larger queue fix the problem?

Read the explanation →

DESIGN / D1

A server starts an unbounded background task for each upload. Explain its overload and crash behaviour, then redesign acceptance.

Read the explanation →

IMPLEMENTATION / I1

Reproduce a lost counter update, then build a bounded upload-work model.

Read the explanation →

THOUGHT EXPERIMENT / T1

Who should own accepted upload work after its initiating request is cancelled?

Read the explanation →

Unit 5 · Object-Oriented and Modular Software Design

CONCEPTUAL / C1

When can composition avoid many subclasses?

Read the explanation →

CONCEPTUAL / C2

What must a payment gateway contract explain beyond charge(amount)?

Read the explanation →

CONCEPTUAL / C3

Why does applying SOLID not require an interface for every class?

Read the explanation →

CONCEPTUAL / C4

How is Payment 42 different from Money(USD,100)?

Read the explanation →

CONCEPTUAL / C5

How can pure functions and external payment effects work together?

Read the explanation →

SCENARIO / S1

Why ask an account to reserve an amount rather than overwrite its balance from outside?

Read the explanation →

SCENARIO / S2

A mock confirms every expected method call. What could still be wrong?

Read the explanation →

SCENARIO / S3

A refactor changes provider timeout into decline. Has it preserved behaviour?

Read the explanation →

DESIGN / D1

A payment class mixes fee rules, HTTP parsing, provider calls, and storage. Plan a behaviour-preserving refactor.

Read the explanation →

IMPLEMENTATION / I1

Replace hardcoded payment-provider construction with an injected recording gateway.

Read the explanation →

THOUGHT EXPERIMENT / T1

Which payment rules should remain the same when objects are replaced by pure functions?

Read the explanation →

Unit 6 · Design Patterns, Dependency Injection, and Object Lifecycles

CONCEPTUAL / C1

Can a constructor use DI while the service still depends on provider details?

Read the explanation →

CONCEPTUAL / C2

Is a singleton necessarily one object across every host?

Read the explanation →

CONCEPTUAL / C3

Why must a factory say who owns the resource it returns?

Read the explanation →

CONCEPTUAL / C4

Why can injecting a container leave dependencies hidden?

Read the explanation →

CONCEPTUAL / C5

How can a factory and DI cooperate?

Read the explanation →

SCENARIO / S1

A singleton stores a request-scoped unit of work. What happens after the request ends?

Read the explanation →

SCENARIO / S2

Two callers trigger lazy initialisation together. What rules are needed?

Read the explanation →

SCENARIO / S3

Why can leaking one connection per failed request stop healthy requests later?

Read the explanation →

DESIGN / D1

A singleton resolves and stores a request connection from a global container. Find the hidden dependency and lifetime failure.

Read the explanation →

IMPLEMENTATION / I1

Demonstrate transient, scoped, and singleton identities across two request scopes.

Read the explanation →

THOUGHT EXPERIMENT / T1

How should resources be owned when a job outlives its request and user session?

Read the explanation →

Unit 7 · Application Architecture and Domain-Driven Design

CONCEPTUAL / C1

What makes a monolith modular if it has one deployment?

Read the explanation →

CONCEPTUAL / C2

Why can dependency arrows point differently from runtime calls?

Read the explanation →

CONCEPTUAL / C3

Why can catalog Product and fulfilment Item be different valid models?

Read the explanation →

CONCEPTUAL / C4

How should an aggregate boundary follow the rule it protects?

Read the explanation →

CONCEPTUAL / C5

What should an architecture fitness check preserve?

Read the explanation →

SCENARIO / S1

An order commits, then the process dies before publishing its event. What is missing?

Read the explanation →

SCENARIO / S2

Five services share writable tables and always release together. What independence is missing?

Read the explanation →

SCENARIO / S3

When would extracting a module into a service be justified?

Read the explanation →

DESIGN / D1

Several services share tables and require simultaneous releases. Review whether the deployment split is useful.

Read the explanation →

IMPLEMENTATION / I1

Implement shopping modules and one meaningful dependency-boundary check.

Read the explanation →

THOUGHT EXPERIMENT / T1

What evidence would justify extracting payments into a separate service?

Read the explanation →

Unit 8 · API Design and Communication Contracts

CONCEPTUAL / C1

Why give a refund its own identity and status?

Read the explanation →

CONCEPTUAL / C2

What does a keyset cursor improve, and what does it leave unresolved?

Read the explanation →

CONCEPTUAL / C3

What makes an idempotency key more than a string in a request?

Read the explanation →

CONCEPTUAL / C4

Why does signing in not grant access to GET /payments/42?

Read the explanation →

CONCEPTUAL / C5

How can removing an old event decoder break recovery after a successful release?

Read the explanation →

SCENARIO / S1

Does 201 Created for a pending payment mean money was captured?

Read the explanation →

SCENARIO / S2

Four layers each allow three attempts. How much downstream work can one request cause?

Read the explanation →

SCENARIO / S3

A webhook effect commits but its reply is lost. What should redelivery do?

Read the explanation →

DESIGN / D1

A payment API returns a Boolean and provides no operation status. Redesign it for an uncertain provider reply.

Read the explanation →

IMPLEMENTATION / I1

Implement keyset continuation and a durable operation-key model for synthetic payments.

Read the explanation →

THOUGHT EXPERIMENT / T1

Can finite storage support a permanent promise to recognise every old idempotency key?

Read the explanation →

Unit 9 · Data Models and Storage Technologies

CONCEPTUAL / C1

What must be decided before choosing a storage family?

Read the explanation →

CONCEPTUAL / C2

Why is a similar embedding result not proof of truth or permission?

Read the explanation →

CONCEPTUAL / C3

When is storing an address twice correct?

Read the explanation →

CONCEPTUAL / C4

Why is data ownership broader than creating a table?

Read the explanation →

CONCEPTUAL / C5

What work appears when a second storage engine is added?

Read the explanation →

SCENARIO / S1

Why do more cold partitions not solve one very busy conversation partition?

Read the explanation →

SCENARIO / S2

Search still holds a deleted private document. What must the system do?

Read the explanation →

SCENARIO / S3

What prevents an older backup from restoring deleted content into live service?

Read the explanation →

DESIGN / D1

A search engine holds the only copy of private document text. Review ownership, recovery, and deletion.

Read the explanation →

IMPLEMENTATION / I1

Model profiles, financial entries, chat history, events, and PDFs from their required operations.

Read the explanation →

THOUGHT EXPERIMENT / T1

How should restoration from a historical backup respect later deletion decisions?

Read the explanation →

Unit 10 · Database Internals, Transactions, and Concurrency Control

CONCEPTUAL / C1

Why might range and equality queries need different indexes?

Read the explanation →

CONCEPTUAL / C2

Why does an ACID transaction not automatically preserve every business rule?

Read the explanation →

CONCEPTUAL / C3

Why is MVCC alone insufficient to claim serializability?

Read the explanation →

CONCEPTUAL / C4

Why state the engine and statement behaviour when discussing isolation?

Read the explanation →

CONCEPTUAL / C5

How do serializability and linearizability differ?

Read the explanation →

SCENARIO / S1

Two doctors each see the other on call and both go off duty. What rule was missed?

Read the explanation →

SCENARIO / S2

A conditional seat update changes zero rows. What does that mean?

Read the explanation →

SCENARIO / S3

A backup job succeeded. What recovery evidence is still needed?

Read the explanation →

DESIGN / D1

Two doctors can each go off call after seeing the other available. Propose a transaction design preserving at least one on call.

Read the explanation →

IMPLEMENTATION / I1

Run the conditional last-seat claim with two connections to one store.

Read the explanation →

THOUGHT EXPERIMENT / T1

What changes when one local transaction is divided across independent databases?

Read the explanation →

Unit 11 · Caching and Content Delivery

CONCEPTUAL / C1

Why does a working set matter more than the full history size for caching?

Read the explanation →

CONCEPTUAL / C2

How can cache-aside reinsert an old value after invalidation?

Read the explanation →

CONCEPTUAL / C3

Why can a one-minute TTL serve data older than one minute?

Read the explanation →

CONCEPTUAL / C4

For A,B,C,A,D at capacity three, what do ideal LRU and FIFO evict?

Read the explanation →

CONCEPTUAL / C5

When might removing a cache improve the design?

Read the explanation →

SCENARIO / S1

A cache restart sends misses to the origin. How can that load be controlled?

Read the explanation →

SCENARIO / S2

A product is created just after “not found” is cached. What should the policy explain?

Read the explanation →

SCENARIO / S3

Refresh keeps failing while stale data is served. What limit is needed?

Read the explanation →

DESIGN / D1

A cache-aside reader repopulates old data after the writer invalidates it. Define and repair the actual freshness requirement.

Read the explanation →

IMPLEMENTATION / I1

Run ideal LRU, then reproduce stale cache-aside population after invalidation.

Read the explanation →

THOUGHT EXPERIMENT / T1

When should a product stop serving rather than return a stale cached result?

Read the explanation →

Unit 12 · Scalability, Load Balancing, and Data Distribution

CONCEPTUAL / C1

Why can adding application servers fail to improve throughput?

Read the explanation →

CONCEPTUAL / C2

What becomes harder when a dataset is sharded?

Read the explanation →

CONCEPTUAL / C3

What does stable hashing change, and what does it not balance?

Read the explanation →

CONCEPTUAL / C4

Why distinguish received and applied replica progress?

Read the explanation →

CONCEPTUAL / C5

Why compare scaling costs under the same failure objectives?

Read the explanation →

SCENARIO / S1

One partition receives 70% of writes. What should guide the fix?

Read the explanation →

SCENARIO / S2

How can scaling application replicas make a database outage worse?

Read the explanation →

SCENARIO / S3

Two replicas share one host. Which failure do they not survive?

Read the explanation →

DESIGN / D1

Scaling application replicas increases database timeouts. Design an investigation and correction.

Read the explanation →

IMPLEMENTATION / I1

Compare rendezvous owners before and after adding D, then introduce a hot key.

Read the explanation →

THOUGHT EXPERIMENT / T1

Can more partitions scale one globally hot exclusive ownership decision?

Read the explanation →

Unit 13 · File Systems, Search Infrastructure, and Data Pipelines

CONCEPTUAL / C1

Why do acknowledged upload parts not prove the completed file is correct?

Read the explanation →

CONCEPTUAL / C2

Why does a content hash not authorise a download?

Read the explanation →

CONCEPTUAL / C3

What does an inverted index record for “blue car” and “blue bike”?

Read the explanation →

CONCEPTUAL / C4

What can go wrong between a snapshot and its CDC stream?

Read the explanation →

CONCEPTUAL / C5

What provenance makes an extracted field explainable?

Read the explanation →

SCENARIO / S1

Updates and deletes continue during reindexing. What must happen before serving switches?

Read the explanation →

SCENARIO / S2

An event arrives three minutes late. Which time policy matters?

Read the explanation →

SCENARIO / S3

Why is deleting only the PDF insufficient?

Read the explanation →

DESIGN / D1

A reindex copies records while updates and deletions continue. Design the handoff to a new serving generation.

Read the explanation →

IMPLEMENTATION / I1

Implement source-version and attempt checks, then plan a safe index-generation switch.

Read the explanation →

THOUGHT EXPERIMENT / T1

How would you investigate an extracted field after both source and model have changed?

Read the explanation →

Unit 14 · Distributed Systems Fundamentals

CONCEPTUAL / C1

Why is a provider charge with a lost reply a partial failure?

Read the explanation →

CONCEPTUAL / C2

What does Lamport ordering establish?

Read the explanation →

CONCEPTUAL / C3

Which real-time rule must a linearizable register preserve?

Read the explanation →

CONCEPTUAL / C4

Why is CAP not a permanent choice of any two attractive qualities?

Read the explanation →

CONCEPTUAL / C5

Can a system preserve safety while being useless?

Read the explanation →

SCENARIO / S1

A session sees version ten, then version nine. What guarantee failed?

Read the explanation →

SCENARIO / S2

Why does R=2,W=2,N=3 not prove linearizability?

Read the explanation →

SCENARIO / S3

An old leader resumes after a replacement takes over. Where must stale effects be stopped?

Read the explanation →

DESIGN / D1

During a partition, two sides both try to allocate the same exclusive resource. State an acceptable serving policy.

Read the explanation →

IMPLEMENTATION / I1

Evaluate Lamport updates, vector relationships, and fixed-quorum intersection.

Read the explanation →

THOUGHT EXPERIMENT / T1

Which operations may continue during a partition while exclusive allocation must wait?

Read the explanation →

Unit 15 · Replication, Leader Election, and Consensus

CONCEPTUAL / C1

What must “synchronous replication” say about acknowledgment?

Read the explanation →

CONCEPTUAL / C2

Why must replicated command application be deterministic?

Read the explanation →

CONCEPTUAL / C3

Why does Raft require more than counting two acknowledgments?

Read the explanation →

CONCEPTUAL / C4

Why is an abrupt switch to a disjoint voting group unsafe?

Read the explanation →

CONCEPTUAL / C5

Why can convergent replicas still violate a stock rule?

Read the explanation →

SCENARIO / S1

Worker 7 resumes after worker 8 owns the attempt. What enforcement is needed?

Read the explanation →

SCENARIO / S2

A pause creates a false failure suspicion. What must remain correct?

Read the explanation →

SCENARIO / S3

What must happen before a repaired old primary accepts writes again?

Read the explanation →

DESIGN / D1

A paused worker returns after its lease expired and a new owner started. Design stale-effect protection.

Read the explanation →

IMPLEMENTATION / I1

Implement a receiver that rejects stale attempt epochs.

Read the explanation →

THOUGHT EXPERIMENT / T1

What must a repaired region establish before it accepts authoritative writes again?

Read the explanation →

Unit 16 · Message Queues, Events, and Distributed Workflows

CONCEPTUAL / C1

Why cannot an absent ephemeral subscriber replay missed history automatically?

Read the explanation →

CONCEPTUAL / C2

How does acknowledgment placement change crash behaviour?

Read the explanation →

CONCEPTUAL / C3

What can an exactly-once stream transaction actually cover?

Read the explanation →

CONCEPTUAL / C4

Why can an outbox relay publish the same event twice?

Read the explanation →

CONCEPTUAL / C5

Why is saga compensation not always an undo?

Read the explanation →

SCENARIO / S1

An effect commits and the message is delivered again. How can one local update be preserved?

Read the explanation →

SCENARIO / S2

A worker keeps running after its lease expires. What protects later effects?

Read the explanation →

SCENARIO / S3

Why not resend an uncertain payment as a new operation?

Read the explanation →

DESIGN / D1

An order commits but its event is never published. Design durable publication and repeated-delivery handling.

Read the explanation →

IMPLEMENTATION / I1

Crash the outbox relay after publication, then process the duplicate with a local inbox transaction.

Read the explanation →

THOUGHT EXPERIMENT / T1

How could one broker transaction be exactly-once while two emails are sent?

Read the explanation →

Unit 17 · Reliability, Fault Tolerance, and Disaster Recovery

CONCEPTUAL / C1

Why are two replicas not necessarily enough surviving capacity?

Read the explanation →

CONCEPTUAL / C2

Why include pool acquisition in the overall deadline?

Read the explanation →

CONCEPTUAL / C3

What does a circuit breaker help, and what remains unresolved?

Read the explanation →

CONCEPTUAL / C4

How can separate worker pools still share a failure bottleneck?

Read the explanation →

CONCEPTUAL / C5

How do RTO and RPO differ?

Read the explanation →

SCENARIO / S1

How should an end-to-end design limit a retry storm?

Read the explanation →

SCENARIO / S2

Can a default cached balance replace an unavailable ledger?

Read the explanation →

SCENARIO / S3

Why not send traffic straight back to a repaired primary?

Read the explanation →

DESIGN / D1

A provider slowdown holds every connection and triggers layered retries. Break the cascade without inventing business success.

Read the explanation →

IMPLEMENTATION / I1

Calculate nested retry amplification and design a bounded provider-outage policy.

Read the explanation →

THOUGHT EXPERIMENT / T1

How can a disaster drill restore infrastructure yet fail the business recovery requirement?

Read the explanation →

Unit 18 · Security Architecture and Privacy

CONCEPTUAL / C1

Why is an internal network address not proof of a caller's identity?

Read the explanation →

CONCEPTUAL / C2

How do OAuth 2.0 and OpenID Connect differ in purpose?

Read the explanation →

CONCEPTUAL / C3

Why use purpose-built password hashing rather than a fast hash?

Read the explanation →

CONCEPTUAL / C4

Which paths need tenant isolation beyond the database query?

Read the explanation →

CONCEPTUAL / C5

Does an artifact's valid signature prove its code is safe?

Read the explanation →

SCENARIO / S1

A tenant A administrator guesses a tenant B document ID. What denies access?

Read the explanation →

SCENARIO / S2

A backup survives but its only decryption key is destroyed. What went wrong?

Read the explanation →

SCENARIO / S3

Membership is revoked after an export is queued. Where must that policy be enforced?

Read the explanation →

DESIGN / D1

A private export is authorised at submission but its worker trusts the old tenant field forever. Review the complete journey.

Read the explanation →

IMPLEMENTATION / I1

Test the synthetic tenant predicate, then trace all data paths using that decision.

Read the explanation →

THOUGHT EXPERIMENT / T1

What should revocation mean for an issued object capability and a queued export?

Read the explanation →

Unit 19 · Observability, Monitoring, and Incident Engineering

CONCEPTUAL / C1

How do logs, metrics, and traces complement one another?

Read the explanation →

CONCEPTUAL / C2

Why is a trace ID not an access credential?

Read the explanation →

CONCEPTUAL / C3

Why can low CPU coexist with severe request delay?

Read the explanation →

CONCEPTUAL / C4

What is tenfold budget burn for a 99.9% success target?

Read the explanation →

CONCEPTUAL / C5

Why avoid unique request IDs as ordinary metric labels?

Read the explanation →

SCENARIO / S1

A five-second request waits 4.6 seconds for a connection. Where should investigation focus?

Read the explanation →

SCENARIO / S2

Serving recovers but charges remain uncertain. Why is recovery incomplete?

Read the explanation →

SCENARIO / S3

HTTP 202 is fast while jobs wait for hours. Which indicators are missing?

Read the explanation →

DESIGN / D1

HTTP acceptance stays fast while background jobs stop finishing. Redesign the service indicators and alert.

Read the explanation →

IMPLEMENTATION / I1

Build the five-second trace model and an asynchronous completion indicator.

Read the explanation →

THOUGHT EXPERIMENT / T1

What evidence would support a latency explanation beyond correlation with a release?

Read the explanation →

Unit 20 · Deployment, Cloud Infrastructure, and Platform Architecture

CONCEPTUAL / C1

What important boundary differs between a VM and a typical container?

Read the explanation →

CONCEPTUAL / C2

How can a control-plane outage differ from a serving outage?

Read the explanation →

CONCEPTUAL / C3

Why must a canary receive representative traffic?

Read the explanation →

CONCEPTUAL / C4

Why can an old executable fail as a rollback?

Read the explanation →

CONCEPTUAL / C5

Which duties remain with a team using a managed database?

Read the explanation →

SCENARIO / S1

Scale-down kills a worker holding an unrecorded job. What was missing?

Read the explanation →

SCENARIO / S2

A feature is disabled after writing a new data format. What risk remains?

Read the explanation →

SCENARIO / S3

Why do two successfully scheduled replicas not necessarily form a correct shared service?

Read the explanation →

DESIGN / D1

A blue-green release shares one database and removes a column needed by the old version. Review its rollback plan.

Read the explanation →

IMPLEMENTATION / I1

Plan and test an expand-and-contract release with old and new readers.

Read the explanation →

THOUGHT EXPERIMENT / T1

How can rollback restore old code while leaving incompatible data or effects?

Read the explanation →

Unit 21 · Performance, Cost Optimization, and System Validation

CONCEPTUAL / C1

What must accompany a reusable benchmark result?

Read the explanation →

CONCEPTUAL / C2

How can coordinated omission hide waiting?

Read the explanation →

CONCEPTUAL / C3

Why is a short measured throughput maximum not a safe production target?

Read the explanation →

CONCEPTUAL / C4

Why can cost per attempt hide increasing cost per useful outcome?

Read the explanation →

CONCEPTUAL / C5

What should an architecture fitness function protect?

Read the explanation →

SCENARIO / S1

An index fixes the slow query. Why might sharding have been a poorer first step?

Read the explanation →

SCENARIO / S2

What does killing a process only before work begins fail to test?

Read the explanation →

SCENARIO / S3

Source and destination counts match but tenant owners differ. Why did validation miss it?

Read the explanation →

DESIGN / D1

A faster and cheaper benchmark configuration omits failures and disables authorisation. Can it replace the original design?

Read the explanation →

IMPLEMENTATION / I1

Compare the three fictional architectures against equal requirements and costs.

Read the explanation →

THOUGHT EXPERIMENT / T1

When could a more expensive architecture be simpler to operate overall?

Read the explanation →

Unit 22 · End-to-End Design, Architecture Reviews, and Evolution

CONCEPTUAL / C1

Why can a click count and a seat claim use different consistency rules?

Read the explanation →

CONCEPTUAL / C2

What does a persisted chat receipt establish, and what does it not establish?

Read the explanation →

CONCEPTUAL / C3

Why can a scheduler lease not guarantee one external execution?

Read the explanation →

CONCEPTUAL / C4

Why is equal final text insufficient to validate a collaborative editor?

Read the explanation →

CONCEPTUAL / C5

What should complete mean for a document-processing workflow?

Read the explanation →

SCENARIO / S1

Payment succeeds after a seat hold expires. What may the service safely do?

Read the explanation →

SCENARIO / S2

The provider charges, the process stops, and the client retries. How is another logical charge avoided?

Read the explanation →

SCENARIO / S3

An old extraction worker finishes after document deletion. What stops publication?

Read the explanation →

DESIGN / D1

Review a short-link service at 100,000 redirects/s after its cache becomes unavailable.

Read the explanation →

IMPLEMENTATION / I1

Choose one case and implement its failure exercise with a checked invariant.

Read the explanation →

THOUGHT EXPERIMENT / T1

How would you choose between another region, tested restores, and stronger reconciliation?

Read the explanation →

ALL 22 UNITS / EXPLAINED ANSWERS

Solutions & reasoning

Each explanation connects the outcome to assumptions, mechanism, and failure. A later or linked chapter provides the full surrounding context.

Unit 1 · Introduction to System Design

UNIT 01 / C1

A program adds a task to a dictionary and displays “Saved.” What can it do while running, and what happens when it starts with a new dictionary?

While the program runs, it can list and update the task in that dictionary. After restart, a new empty dictionary contains none of the previous tasks. To keep the task across a restart, the program must save it in persistent storage and wait for the save to finish before reporting success.

← Return to question

UNIT 01 / C2

What does the signature create(title) tell a caller? What must the contract explain?

The signature names the operation and its input; a typed signature can also describe the result type. The contract explains the behaviour: which titles are valid, what changes when the call succeeds, and which errors may occur. In our persistent application, it also says that creation returns successfully only after commit. Callers need these rules to use the operation correctly.

← Return to question

UNIT 01 / C3

The task service depends on a repository. Does this dependency necessarily make the design difficult to change?

No. The service needs a way to save tasks. A repository contract lets it request that work without knowing the SQL or file layout. A column rename can then be handled in the repository. The useful question is how much one module must know about another and whether a change to those details spreads through the application.

← Return to question

UNIT 01 / C4

How do a high-level view, a low-level view, and a deployment view help explain the task application?

The high-level view shows the main parts, such as the application and database. The low-level view explains the service methods, task records, and repository interface inside the application. The deployment view places the running application and database file on machines. These views describe the same system, so their decisions must agree. A separate repository box in a module diagram does not imply a separate server.

← Return to question

UNIT 01 / C5

A service stops accepting requests during recovery but keeps every committed task. Which observation concerns availability, and which concerns durability?

The period when users cannot perform task operations concerns availability. The fact that committed tasks remain concerns durability under that recovery scenario. A system can preserve data while being temporarily unavailable. Conversely, it can keep replying while losing tasks that were stored only in memory.

← Return to question

UNIT 01 / S1

Task 7 is committed, but its success reply is lost. The client waits, then retries. Can it conclude that the first attempt failed?

No. In this sequence, task 7 exists even though the client has not received the reply. A fresh create request may add task 8. To recognise the retry, the client needs to reuse an operation identifier, and the server must keep that identifier with the committed result. The design must also specify how it checks that the repeated request matches and how long it remembers the result. Chapter 8.2 develops these rules.

← Return to question

UNIT 01 / S2

The web adapter and command-line adapter each enforce an 80-character title limit. Only the web adapter is changed to allow 200 characters. How would you prevent this disagreement?

Move the shared title rule into the task service and make both adapters call it. Each adapter still checks its own input format, such as whether a web request contains valid JSON. Check that the database can store the longer title and update any matching storage constraint. Editing both copied rules fixes today's mismatch but leaves the same risk for the next change.

← Return to question

UNIT 01 / S3

The team runs two application machines, each with its own unrelated local task database. A user sees a task on one request and cannot find it on the next. What could explain this?

The two requests may have reached different machines. A task saved in one local database is absent from the other. The design now needs a way for the applications to use shared or coordinated data. Adding a load balancer distributes requests; it does not copy or merge the task databases.

← Return to question

UNIT 01 / D1

A handler adds a task to memory and replies “Saved.” A background writer saves it later. Trace a crash that loses the task, then propose a change that meets the restart requirement.

The failing sequence is: add to memory, reply “Saved,” crash before the background write commits. After restart, memory is empty and there is no saved task to reload. Change the order so that success follows a durable commit, and report a write error if the save fails. A local database is enough for our stated single-machine example. A background queue would need its own durable-save and acknowledgment rules before it could support the same promise.

← Return to question

UNIT 01 / I1

Run examples/todo.py. Make a test repository raise a storage error. Then use the real repository to save a task, close it, and read the task through a new connection. What should each check show?

When insert raises a storage error, create must pass that error to its caller rather than return a successful Task. With the real SQLite repository, the task should still be present after closing and reopening the same database. Also submit an invalid title and check that no row is added. The first check covers error handling, the second persistence across reopen, and the third validation. An in-memory substitute cannot establish the persistence result.

← Return to question

UNIT 01 / T1

The team now requires saved tasks to survive destruction of the only disk. Why is ADR 001 insufficient, and what must the next design address?

ADR 001 assumes that committed storage remains intact. Destroying its only disk removes the copy it relies on. The new design needs recoverable data on independent storage and a rule for when it may report success. If copying happens only later, a task reported as saved can be lost before the copy is made. Specify which failures the copies must survive, how much loss is acceptable, and how recovery will work. Then test those cases. Simply counting replicas does not answer these questions.

← Return to question

Unit 2 · Requirements, Estimation, and Engineering Trade-offs

UNIT 02 / C1

Opening a feed appears to be one action. Why might it create many requests and writes?

The browser can fetch images and send analytics in addition to its feed request. Publishing can also create references for many followers. Count the logical action separately from the physical work at each boundary, or storage and transfer estimates will miss that amplification.

← Return to question

UNIT 02 / C2

Requests keep arriving faster, but successful completions stay at the same rate. What happens next?

Once a limiting resource cannot complete work faster, additional requests wait or are rejected. Compare offered traffic, useful completions, errors, and queue growth. A steady completion rate can coexist with a rapidly worsening user wait; it does not establish that increasing demand is being handled.

← Return to question

UNIT 02 / C3

How can a 19.9 ms mean hide a request that took one second?

In the example, 99 requests take 10 ms and one takes 1,000 ms. Their total is 1,990 ms, so the mean is 19.9 ms. The calculation is correct, but it compresses the distribution. Inspect tail observations and percentiles to understand the slow user's experience.

← Return to question

UNIT 02 / C4

Which requests belong in an event-based error budget?

Define the eligible population and good outcome before calculating the budget. At 99.9% success over one million eligible events, 1,000 bad events are allowed. Excluding the failures the objective was meant to track would make the number misleading. A time-based objective uses a different denominator.

← Return to question

UNIT 02 / C5

When is L = λW useful for estimating work in progress?

Use stable long-run averages within the same boundary and consistent units. At 500 operations/s and 0.04 seconds mean residence, L is twenty operations. Include relevant waiting in residence time. The relation does not predict p99 or show that a queue with sustained excess arrivals will become stable.

← Return to question

UNIT 02 / S1

A traffic spike ends before new instances can start. What protects the service?

Existing headroom and admission controls must handle the interval before scaling takes effect. Bound waiting and protect downstream capacity. Measure the spike's duration as well as height, and account for retries. Adding capacity after the demand has passed cannot undo the delay and overload users already experienced.

← Return to question

UNIT 02 / S2

Only 80% of a job can run in parallel. What ideal speedup can four workers give?

The sequential fraction takes 0.2 of the old time; the parallel portion ideally takes 0.8/4 = 0.2. Total time is 0.4, giving 2.5× speedup. Coordination can reduce the actual gain further. Four workers cannot eliminate the fixed sequential part.

← Return to question

UNIT 02 / S3

Why can requiring fifty parallel calls increase tail latency?

The request waits for its slowest required result. If each call independently has a 1% slow probability, at least one is slow with probability $1-0.99^{50}$, about 39.5%. Independence is an assumption; shared dependencies can correlate delays. Parallel initiation does not remove the wait for the last result.

← Return to question

UNIT 02 / D1

A team sizes servers from daily average RPS alone. Identify the missing evidence and propose a safer estimation process.

Add operation mix, peak duration, storage and transfer work, tested per-instance capacity, utilisation, and surviving failure capacity. Keep each assumption explicit. An estimate becomes a test workload; it is not itself proof that the whole architecture meets latency or recovery needs.

← Return to question

UNIT 02 / I1

Implement a capacity worksheet with explicit user activity, peak, and per-instance inputs.

The default calculation should produce about 5,787.04 average RPS and 28,935.19 peak RPS. At an assumed 600 RPS capacity and 60% target, round up to 81 active instances and 82 for one-instance loss. Validate input ranges and keep database, network, and failure-domain feasibility separate from this arithmetic.

← Return to question

UNIT 02 / T1

How would simultaneous retries change an estimate based on average user sessions?

Retries add attempts without adding new user intentions. Model their timing, maximum amplification, and the failing dependency separately from normal sessions. A correlated retry wave can exceed a peak factor based on independent arrivals. Bound attempts and admission, then test useful completion and recovery under that burst.

← Return to question

Unit 3 · Networking Foundations for System Design

UNIT 03 / C1

A server listens on port 443. Why might a remote caller still fail to use it?

Binding, routing, firewall rules, and the return path can prevent communication even when a process listens. If communication succeeds, the caller still needs the required identity and permissions. Reachability and authorisation answer different questions, so checking the port establishes neither the whole network path nor access rights.

← Return to question

UNIT 03 / C2

What does the application receive from TCP, UDP, and QUIC?

TCP provides an ordered reliable byte stream, so the application defines its own message framing. UDP provides datagrams without that stream contract. QUIC implements secure streams over UDP with its own transport rules. Choose using required behaviour and deployment support rather than assuming a particular label always means faster delivery.

← Return to question

UNIT 03 / C3

Why does changing DNS not move every client immediately?

Clients may keep a cached answer until refresh, including negative caching behaviour. Existing connections may continue without another lookup at all. The authoritative record can therefore change before actual traffic does. Check resolver caching and connection lifetimes together when evaluating failover.

← Return to question

UNIT 03 / C4

Which access checks can a gateway perform, and which need domain information?

A gateway can validate appropriate boundary credentials and apply coarse admission or routing. Whether a caller owns payment 42 requires trustworthy resource and membership information. Keep that domain rule enforced where it is known; an authenticated request at the edge is not automatically authorised for every object.

← Return to question

UNIT 03 / C5

What work does streaming save compared with polling, and what work does it add?

Streaming can avoid repeated empty polls. It keeps connections and buffers alive and needs heartbeat, reconnect, and recovery rules. A client returning after disconnection may still need retained history or a fresh snapshot. The live transport alone does not preserve every required business update.

← Return to question

UNIT 03 / S1

The server commits a write, but the client times out. What can the client conclude?

It knows that it did not receive a timely reply, not that the write failed. The committed effect can exist. Recover by the operation's stable identity or status contract rather than blindly creating another effect. Bound retries and keep uncertainty visible until there is evidence of the outcome.

← Return to question

UNIT 03 / S2

Why might a cold HTTPS request take longer than a warm request with the same server work?

The cold path can add DNS, transport, and security setup. In the illustrative model those stages add 20+40+40 = 100 ms before the same 55 ms request/work stage. Reuse removes only steps actually reused. Protocol, caching, and overlap can change this assumed path.

← Return to question

UNIT 03 / S3

A database pool is full while CPU is low. What should you inspect first?

Measure connection-acquisition waits, hold times, lock waits, and idle transactions. Callers may keep connections while waiting on I/O or external services. More connections can increase downstream contention. Find what holds the existing pool before treating the symptom as a need for more CPU or a larger pool.

← Return to question

UNIT 03 / D1

A payment client retries every timeout as a new request. Trace a duplicate effect and repair the contract.

Let the provider commit, lose the reply, then receive the new request. Use stable operation identity and a supported status or retry path. Keep total waiting bounded, while treating timeout as unknown outcome rather than evidence of nonexecution.

← Return to question

UNIT 03 / I1

Implement a cold/warm request calculator and check invalid stage times.

Use 20+40+40+55 = 155 ms for the assumed cold path. A reusable setup can leave the same 55 ms request/work stage. Remove only stages actually reused, reject negative inputs, and label this as an arithmetic model rather than a protocol benchmark.

← Return to question

UNIT 03 / T1

What changes when a local dependency becomes a cross-region call?

The call adds travel, connection, timeout, trust, and partial-failure behaviour. A local exception and an unknown remote outcome are different. Revisit deadlines, operation identity, encoding, credentials, and fallback, measuring the whole path. Preserve the business meaning while changing the communication arrangement.

← Return to question

Unit 4 · Runtime, Processes, Threads, and Concurrency

UNIT 04 / C1

How do processes and threads differ from modules in the source code?

A process owns runtime resources and an address space; threads execute within it and commonly share its heap. Modules organise code. Several modules can share the same process and stop together in a crash. Source separation alone does not create independent resource or failure boundaries.

← Return to question

UNIT 04 / C2

Why can an async function still block other requests?

It can call a blocking library operation or perform long computation on the event-loop thread. Other ready tasks then wait for that thread. Inspect the driver and actual execution path. Async syntax does not by itself change underlying calls into nonblocking operations or supply additional CPU cores.

← Return to question

UNIT 04 / C3

Can a one-core program be concurrent without executing tasks simultaneously?

Yes. Task A can wait while the core executes B, so both are in progress over overlapping periods. That is concurrency. Parallelism requires simultaneous execution resources. Interleaving can still create races on one core, because a task can be interrupted between the read and write of a compound update.

← Return to question

UNIT 04 / C4

Why might atomic fields fail to make a transfer between two balances safe?

Each field operation may be indivisible while the relationship between the two values is temporarily or permanently wrong. The invariant spans both balances. Protect the combined operation with the appropriate atomic boundary or coordination, rather than assuming several separate atomic assignments form one transaction.

← Return to question

UNIT 04 / C5

Why does an earlier timestamp not prove another thread sees an update?

Visibility follows the language memory model and its synchronisation rules. A correctly used lock, channel, or suitable atomic publication can establish the required happens-before relationship. Wall-clock order alone supplies no such relationship. Use the actual runtime's guarantees rather than an intuition about elapsed time.

← Return to question

UNIT 04 / S1

A and B read counter zero, then each writes one. Where was the increment lost?

Both calculate from the same earlier value. B's write replaces A's result instead of adding to it. Protect the entire read–modify–write operation. Locking only the final assignment, or using separate locks for each caller, leaves the shared invariant unprotected.

← Return to question

UNIT 04 / S2

One callback computes for two seconds on a single event loop. What happens to ready tasks?

They cannot execute on that loop until the callback yields or finishes. Completed I/O may make tasks ready without giving them CPU time. Keep loop turns bounded or move appropriate computation to a bounded execution resource. More queued async tasks do not make the long callback run in parallel.

← Return to question

UNIT 04 / S3

Jobs arrive at 150/s and complete at 100/s. Does a larger queue fix the problem?

It stores fifty more waiting jobs each second until its limit is reached. More space delays rejection but does not change the service rate. Reduce admitted work or add useful capacity, bound queue age, and return clear overload outcomes before waiting becomes useless or resources are exhausted.

← Return to question

UNIT 04 / D1

A server starts an unbounded background task for each upload. Explain its overload and crash behaviour, then redesign acceptance.

Queued buffers and open resources can grow without bound, and process loss destroys unrecorded accepted work. Bound active and waiting jobs separately, persist required identity and input references, and define rejection or waiting. Choose CPU parallelism and I/O concurrency according to measured service demand.

← Return to question

UNIT 04 / I1

Reproduce a lost counter update, then build a bounded upload-work model.

Show A and B both reading zero and each writing one. Protect their whole shared read–modify–write step. Bound active workers and waiting references separately. Model I/O waits differently from CPU service demand, and identify what accepted-work state must persist if a process-local queue disappears.

← Return to question

UNIT 04 / T1

Who should own accepted upload work after its initiating request is cancelled?

If cancellation only ends the connection but the business still requires processing, a durable job or workflow must retain identity, input, and progress. Cancel and release request-owned resources without discarding that obligation. If the work itself should stop, persist the chosen cancellation and block inappropriate later completion.

← Return to question

Unit 5 · Object-Oriented and Modular Software Design

UNIT 05 / C1

When can composition avoid many subclasses?

A payment service can combine a gateway and a fraud policy independently instead of creating one subclass for each combination. The collaborators need clear contracts and wiring. Inheritance remains appropriate when specialisation preserves the caller's expected behaviour. Similar fields alone do not establish that relationship.

← Return to question

UNIT 05 / C2

What must a payment gateway contract explain beyond charge(amount)?

Specify currency, amount units, operation identity, possible states, and error meanings. In particular, a timeout can leave an attempted charge uncertain. Callers need that distinction to recover safely. Returning an undifferentiated Boolean hides information needed to distinguish decline, approved effect, and lost response.

← Return to question

UNIT 05 / C3

Why does applying SOLID not require an interface for every class?

The principles help protect responsibilities, substitution, and useful dependency boundaries. An extra interface is valuable when it hides meaningful variation or knowledge. If it only forwards every call, it can add navigation and maintenance without improving behaviour. Evaluate a concrete change or test need rather than maximising interface count.

← Return to question

UNIT 05 / C4

How is Payment 42 different from Money(USD,100)?

Payment 42 retains its identity as its status changes, so it is an entity. Equal Money values can represent the same amount and currency without separate identity, making Money a value object. Both need valid rules; combining different currencies requires an explicit conversion policy rather than arbitrary arithmetic.

← Return to question

UNIT 05 / C5

How can pure functions and external payment effects work together?

A pure decision function takes explicit state and command data and returns decisions or requested actions. A separate interpreter performs storage and provider calls. This makes decisions testable, while the effect path still needs durability, authorisation, duplicate handling, and reconciliation. Pure local calculation does not make a remote charge atomic.

← Return to question

UNIT 05 / S1

Why ask an account to reserve an amount rather than overwrite its balance from outside?

The account owner can keep the rule and coordinated change together. External read/compare/write logic duplicates policy and can race with other writers. Reporting a balance is still a valid query. The owner must enforce the update at the actual shared storage boundary; merely naming a method reserve does not supply atomicity.

← Return to question

UNIT 05 / S2

A mock confirms every expected method call. What could still be wrong?

The final payment state, stored transaction, encoding, or timeout recovery can still violate the contract. Prefer tests of required outcomes and invariants, and integration evidence for real boundaries. A mock can make a specific path easy to exercise but cannot prove the behaviour of the component it replaced.

← Return to question

UNIT 05 / S3

A refactor changes provider timeout into decline. Has it preserved behaviour?

No. Decline says the operation was rejected, while timeout can mean the effect happened but its reply was lost. Callers may retry differently and create duplicates. A structural refactor should preserve this meaning, with tests covering approved, declined, uncertain, and storage-failure outcomes.

← Return to question

UNIT 05 / D1

A payment class mixes fee rules, HTTP parsing, provider calls, and storage. Plan a behaviour-preserving refactor.

Capture approval, decline, uncertainty, and storage-error outcomes. Separate transport, rule calculation, provider mapping, and persistence in small steps. Keep contracts explicit and test outcomes. Structural improvement must preserve amounts and recovery meaning rather than merely move code into more classes.

← Return to question

UNIT 05 / I1

Replace hardcoded payment-provider construction with an injected recording gateway.

Exercise approved, declined, and uncertain results without real charges. Preserve currency and minor units and check resulting business state. A fake helps test orchestration; real encoding, persistence, and provider recovery need separate evidence. Do not turn an uncertain timeout into a decline to simplify the test.

← Return to question

UNIT 05 / T1

Which payment rules should remain the same when objects are replaced by pure functions?

Currency, units, allowed transitions, operation identity, and uncertainty remain business rules regardless of representation. Pure decision functions can make them easier to test. A separate effect interpreter still needs authorisation, transactions, provider identity, and reconciliation. Refactoring should preserve observed outcomes, not merely move methods into functions.

← Return to question

Unit 6 · Design Patterns, Dependency Injection, and Object Lifecycles

UNIT 06 / C1

Can a constructor use DI while the service still depends on provider details?

Yes. Passing a concrete SDK is dependency injection, but the service may still consume its request types and exceptions. Dependency inversion concerns which interface policy relies on. An application-owned contract can separate the provider meaning; construction syntax alone cannot establish that separation.

← Return to question

UNIT 06 / C2

Is a singleton necessarily one object across every host?

No. The lifetime is scoped to the specified container or application arrangement. Each process may have its own singleton. Sharing one object also does not establish safe concurrent use. State its ownership, lifetime, and allowed callers instead of inferring global uniqueness from the name.

← Return to question

UNIT 06 / C3

Why must a factory say who owns the resource it returns?

A returned connection may be new and caller-owned, shared, or managed by a scope. Closing a shared object can break other users; never closing an owned object can leak capacity. Creation and borrowing are distinct from disposal responsibility, so include that responsibility in the contract.

← Return to question

UNIT 06 / C4

Why can injecting a container leave dependencies hidden?

Application methods can still resolve arbitrary services at runtime instead of naming what they require. This behaves like a service locator. Passing focused contracts makes needs visible at construction or call time. Keep broad resolution near a deliberate integration or composition boundary.

← Return to question

UNIT 06 / C5

How can a factory and DI cooperate?

A service can receive a narrow sender factory, then ask it for the appropriate channel during an operation. Injection supplies the factory; the factory performs selection or creation. Define the returned sender's lifetime so the caller knows whether it owns disposal or merely borrows a shared instance.

← Return to question

UNIT 06 / S1

A singleton stores a request-scoped unit of work. What happens after the request ends?

The unit of work may be disposed while the singleton still holds and uses it. This captive dependency breaks the intended lifetime and can retain request context. Supply a currently valid per-operation dependency or appropriate scope/factory rather than hiding the problem with a global lookup.

← Return to question

UNIT 06 / S2

Two callers trigger lazy initialisation together. What rules are needed?

Coordinate creation and safe publication so they do not use partially initialised or unintended duplicate state. Decide how failure is reported and whether later callers retry. A supported lazy primitive or lock can help. Essential resources may be clearer to initialise before advertising readiness.

← Return to question

UNIT 06 / S3

Why can leaking one connection per failed request stop healthy requests later?

Each leak reduces the pool's usable capacity. Eventually new requests cannot borrow a connection even if their own code would succeed. Cleanup must cover failure and cancellation as well as success. Timely release of external resources cannot be assumed from memory garbage collection alone.

← Return to question

UNIT 06 / D1

A singleton resolves and stores a request connection from a global container. Find the hidden dependency and lifetime failure.

The lookup conceals what the singleton needs, and the stored resource may expire at request end. Supply an appropriate per-operation dependency or narrow factory with explicit ownership. Do not repair the graph merely by extending every connection lifetime indefinitely.

← Return to question

UNIT 06 / I1

Demonstrate transient, scoped, and singleton identities across two request scopes.

A scoped object should be A,A in scope one and B,B in scope two. Transients should differ; the same process singleton should remain S. Add an owned disposable resource and check scope cleanup. Demonstrate why retaining it in a singleton breaks the intended lifetime.

← Return to question

UNIT 06 / T1

How should resources be owned when a job outlives its request and user session?

Give the job an explicit lifetime and scope, with durable identifiers rather than captured request objects. Create or borrow resources under a documented owner and close them at the appropriate boundary. Define whether current membership must be rechecked; a long-lived job must not silently inherit expired user authority.

← Return to question

Unit 7 · Application Architecture and Domain-Driven Design

UNIT 07 / C1

What makes a monolith modular if it has one deployment?

Its modules have clear responsibilities, controlled interfaces, and data ownership. Orders should use a payment contract rather than alter private payment tables. One deployment keeps calls local but does not prevent useful boundaries. Enforce important dependencies through review or appropriate checks.

← Return to question

UNIT 07 / C2

Why can dependency arrows point differently from runtime calls?

A service can depend on an application-owned repository interface, while a database adapter implements it. At runtime, calls execute the supplied adapter. The source arrow describes knowledge or implementation relationships, not necessarily the physical direction of a message or a separate network service.

← Return to question

UNIT 07 / C3

Why can catalog Product and fulfilment Item be different valid models?

They describe different business work. Catalog may care about descriptions and categories, while fulfilment tracks units and locations. A bounded context identifies where each meaning applies. Translate between them deliberately rather than forcing both into one schema that obscures their rules.

← Return to question

UNIT 07 / C4

How should an aggregate boundary follow the rule it protects?

Group state that must change atomically under a root controlling those changes. An order's line values and total may need one consistency boundary. Making the entire shop one aggregate adds contention. Cross-aggregate work needs explicit coordination rather than an assumption that object references create a shared transaction.

← Return to question

UNIT 07 / C5

What should an architecture fitness check preserve?

A meaningful required property, such as allowed dependencies, migration compatibility, or tenant isolation. It should detect erosion of that reasoned boundary without freezing incidental class names. Update the check when the architecture decision legitimately changes, and keep evidence within the check's actual scope.

← Return to question

UNIT 07 / S1

An order commits, then the process dies before publishing its event. What is missing?

Publication intent existed only in memory or a separate unsafe step. An outbox can commit that intent with the order so a relay can resume later. The relay may publish again after losing its own progress record, so consumers still need stable event identity and duplicate handling.

← Return to question

UNIT 07 / S2

Five services share writable tables and always release together. What independence is missing?

They cannot change their contracts or stored representation independently and can remain tightly coupled operationally. The network split adds failure and diagnosis work without delivering much independent deployment. Clarify ownership and boundaries, or consider a modular consolidation where appropriate.

← Return to question

UNIT 07 / S3

When would extracting a module into a service be justified?

Look for a stable capability with a concrete scaling, fault-isolation, or team-ownership need, plus the ability to operate it. Compare the benefit with network failures, versioning, data coordination, security, and on-call costs. A rapidly changing model alone is not evidence that more process boundaries will help.

← Return to question

UNIT 07 / D1

Several services share tables and require simultaneous releases. Review whether the deployment split is useful.

Map who owns decisions and data, what changes together, and what fails together. A modular consolidation may be simpler, or revised contracts and ownership may justify separation. Compare independent operation with the actual network, migration, and incident costs.

← Return to question

UNIT 07 / I1

Implement shopping modules and one meaningful dependency-boundary check.

Let orders use a payment contract without importing private payment storage or writing its tables. Check that boundary, then replace the provider adapter while preserving the order rule. Avoid a test tied to every internal class name; protect the dependency whose violation would spread provider details or bypass ownership.

← Return to question

UNIT 07 / T1

What evidence would justify extracting payments into a separate service?

Look for stable capability ownership, different scaling or isolation needs, and an operating team prepared for network and version failures. Compare those benefits with added data coordination and recovery. Preserve existing payment invariants and histories through migration. A separate deployment is useful only when the required independence can actually be exercised.

← Return to question

Unit 8 · API Design and Communication Contracts

UNIT 08 / C1

Why give a refund its own identity and status?

A refund can be requested later, authorised separately, and complete or fail independently of the original payment. Its own identity supports amount checks, retries, and investigation. Treating it as an untracked toggle on paid loses the history and uncertainty needed for recovery.

← Return to question

UNIT 08 / C2

What does a keyset cursor improve, and what does it leave unresolved?

It continues after a stable ordered key rather than skipping a shifting row count, avoiding some duplicates and omissions from insertions before an offset. It also needs validation and query/tenant scope. Changing records can still affect results; the cursor does not automatically provide one consistent snapshot.

← Return to question

UNIT 08 / C3

What makes an idempotency key more than a string in a request?

Durable state binds owner, key, payload fingerprint, progress, and result with suitable uniqueness and atomicity. Identical retries recover the same operation; different content conflicts. Retention and provider recovery rules matter too. A separate unprotected lookup and insert can race or forget an external effect.

← Return to question

UNIT 08 / C4

Why does signing in not grant access to GET /payments/42?

Authentication identifies the caller. Authorisation must still check this caller's permitted action on payment 42 using trusted membership and resource scope. Guess-resistant IDs and gateway authentication do not establish ownership. Apply the rule on every relevant query and effect path.

← Return to question

UNIT 08 / C5

How can removing an old event decoder break recovery after a successful release?

Historical logs and backups can still contain that version. A restoring or delayed consumer needs to interpret it even when current traffic uses only the newest shape. Keep supported historical samples and transformations, including meanings, defaults, and unknown values, for the required replay horizon.

← Return to question

UNIT 08 / S1

Does 201 Created for a pending payment mean money was captured?

No. It can mean the payment resource was created successfully while its financial operation remains pending. The body and operation contract describe that later state. HTTP response semantics do not make a provider effect part of the local resource transaction.

← Return to question

UNIT 08 / S2

Four layers each allow three attempts. How much downstream work can one request cause?

Up to $3^4 = 81$ attempts in the nested worst case. Early success can reduce actual work, but failure can trigger amplification precisely when capacity is scarce. Use an overall retry budget and deadline, and safe operation identity for effects.

← Return to question

UNIT 08 / S3

A webhook effect commits but its reply is lost. What should redelivery do?

Recognise the stable event identity and avoid repeating the established local effect. Persist recognition and effect together when that is the protected boundary. An unrelated external call needs its own protocol. The sender may legitimately retry because it did not observe acknowledgment.

← Return to question

UNIT 08 / D1

A payment API returns a Boolean and provides no operation status. Redesign it for an uncertain provider reply.

Create a stable intent identity, explicit pending/approved/declined/uncertain semantics, validated money units, and scoped repeat-key handling. Offer status recovery. Define compatibility and permission as well as field shape so clients know what repeating the request means.

← Return to question

UNIT 08 / I1

Implement keyset continuation and a durable operation-key model for synthetic payments.

Use a stable ordered pair for continuation and test an insertion before a page boundary. Bind actor, key, fingerprint, progress, and result under appropriate atomicity. Identical retries return the old operation; changed payloads conflict. A dictionary lookup followed by an unrelated insert does not protect concurrent or restarted callers.

← Return to question

UNIT 08 / T1

Can finite storage support a permanent promise to recognise every old idempotency key?

Not for an unbounded set of arbitrary distinct operations unless another constraint supplies a bound or persistent evidence elsewhere. Define key retention, expiry, reuse, and client expectations. A permanent business identity may require a durable record rather than an expiring cache. Make the promise fit the information the system can retain.

← Return to question

Unit 9 · Data Models and Storage Technologies

UNIT 09 / C1

What must be decided before choosing a storage family?

State the important reads and changes, accepted-result boundary, invariants, retention, and supported failures. Identify the authoritative copy and recovery procedure. A product category does not establish configured isolation, durable acknowledgment, or usable backups, so verify the selected implementation against those needs.

← Return to question

UNIT 09 / C2

Why is a similar embedding result not proof of truth or permission?

An embedding is a derived numerical representation used to find candidates. Approximate similarity can miss items or retrieve imperfect matches. Preserve source versions, verify required evidence, and enforce access. A vector score describes its retrieval relationship, not factual correctness or authorisation.

← Return to question

UNIT 09 / C3

When is storing an address twice correct?

A profile can hold the current address while a completed order retains the address actually used at purchase. Those records represent different facts. A maintained cache of the current address has another meaning and needs an update policy. State whether repetition preserves history or tracks a changing source.

← Return to question

UNIT 09 / C4

Why is data ownership broader than creating a table?

The owner defines allowed changes and access contracts and is responsible for schema evolution, retention, incidents, and recovery. If every service writes the table directly, they share release and transaction assumptions despite separate repositories. Ownership needs an enforcement mechanism and operating responsibility.

← Return to question

UNIT 09 / C5

What work appears when a second storage engine is added?

Backup and restoration, access policy, schema changes, monitoring, expertise, and reconciliation now cross another boundary. If it is a derived search store, also manage lag and rebuilding. Compare the new capability with that ownership cost rather than assuming a specialised engine is automatically a net improvement.

← Return to question

UNIT 09 / S1

Why do more cold partitions not solve one very busy conversation partition?

The hot conversation's requests still reach its owner. Extra unrelated capacity does not divide that key's work. Measure whether the pressure comes from size, reads, writes, or ordering, then choose supported splitting, read distribution, or a changed data model that preserves the required semantics.

← Return to question

UNIT 09 / S2

Search still holds a deleted private document. What must the system do?

Enforce the required access and deletion policy when serving, even while asynchronous cleanup catches up. Carry source identity and version so old updates cannot recreate an invalid visible record. Track physical cleanup and derived-copy lifecycle rather than treating a successful delete in the source as completion everywhere.

← Return to question

UNIT 09 / S3

What prevents an older backup from restoring deleted content into live service?

Recovery must apply the relevant deletion history and lifecycle rules before exposing restored copies. Track raw and derived data, including indexes and replay paths. A narrowly retained marker can help prevent resurrection, but has its own minimisation and retention needs.

← Return to question

UNIT 09 / D1

A search engine holds the only copy of private document text. Review ownership, recovery, and deletion.

Keep a recoverable source and exact identity/version relationships for derived entries. Enforce access while indexing lags, and track deletion across copies and restores. If search truly owns the authoritative data, it needs the corresponding complete durability and recovery contract rather than assumptions borrowed from a projection.

← Return to question

UNIT 09 / I1

Model profiles, financial entries, chat history, events, and PDFs from their required operations.

List access patterns, rules, identity, retention, and recoverability before choosing stores. Several needs may share an engine. Keep immutable bytes distinguishable from metadata and derived search results. Explain how each copy is rebuilt or recovered, rather than assigning one product family per data type without a reason.

← Return to question

UNIT 09 / T1

How should restoration from a historical backup respect later deletion decisions?

Restore into a controlled path, apply relevant deletion decisions, and reconcile raw and derived copies before serving. Rebuild indexes from the permitted current state rather than blindly expose an older snapshot. Keep deletion evidence narrow and purpose-limited. The backup and live-data lifecycle need a combined recovery procedure.

← Return to question

Unit 10 · Database Internals, Transactions, and Concurrency Control

UNIT 10 / C1

Why might range and equality queries need different indexes?

An ordered tree can locate a range and continue through neighbouring keys. A hash structure targets equality under its implementation and does not supply ordered traversal. Compare predicates, output size, locality, write maintenance, and engine support rather than choosing only from an abstract lookup complexity.

← Return to question

UNIT 10 / C2

Why does an ACID transaction not automatically preserve every business rule?

The application and constraints must express those rules. The engine can atomically commit a wrongly calculated transfer. Isolation and durability also depend on the selected contract. ACID supplies mechanisms and properties, not knowledge of what the business intended the amount or relationship to mean.

← Return to question

UNIT 10 / C3

Why is MVCC alone insufficient to claim serializability?

Versions can give readers a stable snapshot while concurrent transactions change different rows and violate a shared condition. The two-doctor write-skew example illustrates it. Check the isolation and conflict-detection rules of the engine, not merely whether it retains old row versions.

← Return to question

UNIT 10 / C4

Why state the engine and statement behaviour when discussing isolation?

Identically named levels can provide different visibility and conflict behaviour. For example, PostgreSQL 18 ordinary Read Committed uses statement snapshots, while its stronger levels have other rules. Handle supported retryable failures for the whole transaction using fresh assumptions and bounded attempts.

← Return to question

UNIT 10 / C5

How do serializability and linearizability differ?

Serializability makes transactions equivalent to some serial order. Linearizability constrains operations by real-time precedence. Strict serializability combines transaction ordering with real-time requirements. Neither automatically includes an unrelated provider call outside the database's transaction boundary.

← Return to question

UNIT 10 / S1

Two doctors each see the other on call and both go off duty. What rule was missed?

The invariant spans both rows: at least one doctor must remain on call. Separate row updates can each look acceptable under the old snapshot while jointly violating it. Use suitable serialization, locking, or a shared conflict point that protects the complete rule.

← Return to question

UNIT 10 / S2

A conditional seat update changes zero rows. What does that mean?

The expected version or availability condition no longer matched. Treat it as conflict, re-read, and reconsider the business action. Do not report a reservation merely because the statement executed without a transport error. Success depends on the guarded state transition actually occurring.

← Return to question

UNIT 10 / S3

A backup job succeeded. What recovery evidence is still needed?

Restore it in a usable environment with the required keys, schema, credentials, and logs. Check application invariants and measure the full recovery time. File creation or checksum success alone does not establish that the service can recover its required state and start serving correctly.

← Return to question

UNIT 10 / D1

Two doctors can each go off call after seeing the other available. Propose a transaction design preserving at least one on call.

Protect the multi-row predicate using suitable serialization, locking, or a shared conflict point. Individual row versions alone may not conflict. Handle whole-transaction retries with fresh reads, and keep externally repeated effects outside an unsafe retry body.

← Return to question

UNIT 10 / I1

Run the conditional last-seat claim with two connections to one store.

Check that one claimant succeeds and another receives conflict. Identify the transaction protecting seat ownership, then test rollback or interruption at the relevant boundary. Keep payment and expiry as separate workflow concerns: a successful local claim does not atomically perform an external charge.

← Return to question

UNIT 10 / T1

What changes when one local transaction is divided across independent databases?

A former all-or-nothing invariant may no longer share one commit boundary. Identify which related changes require agreement, whether a distributed transaction is justified, or whether a workflow with explicit intermediate states and compensation fits. Local success at each store does not guarantee the combined business outcome.

← Return to question

Unit 11 · Caching and Content Delivery

UNIT 11 / C1

Why does a working set matter more than the full history size for caching?

Reuse concerns the data active in the chosen interval. A small cache can serve a much larger history if most requests reuse a small portion. If the active set exceeds it, eviction and reload can churn. Include bytes, object size, scan traffic, and admission behaviour as well as entry count.

← Return to question

UNIT 11 / C2

How can cache-aside reinsert an old value after invalidation?

A reader fetches version one, then pauses. A writer commits version two and invalidates the key. The reader resumes and inserts its old value. Ordering a source write before cache deletion alone does not prevent this history; stronger freshness needs suitable version or coordination rules.

← Return to question

UNIT 11 / C3

Why can a one-minute TTL serve data older than one minute?

TTL starts when the cache inserts the value. If that value already lagged the source by five minutes, another minute of residence can make it six minutes old. Expiry and source freshness are different measurements. Choose the policy according to the operation's tolerated staleness.

← Return to question

UNIT 11 / C4

For A,B,C,A,D at capacity three, what do ideal LRU and FIFO evict?

Ideal LRU refreshes A's recency, leaving B oldest when D arrives, so it evicts B. FIFO preserves insertion order and evicts A. This fixture explains their difference; real engines can approximate policies and use byte limits or separate expiration rules.

← Return to question

UNIT 11 / C5

When might removing a cache improve the design?

When reuse saves less than lookup, encoding, memory, and maintenance cost, or when a simpler source improvement meets the requirement with fewer freshness risks. Test the empty and unavailable paths. A warm cache can hide an origin-capacity problem, while a wrongly scoped cache can create a data leak.

← Return to question

UNIT 11 / S1

A cache restart sends misses to the origin. How can that load be controlled?

Combine same-key loading where useful, bound refresh workers, apply origin admission, and rebuild gradually. Negative caching and expiry jitter address particular patterns. Do not let every new instance independently scan the whole source. Protect recovery capacity as well as normal cache-hit traffic.

← Return to question

UNIT 11 / S2

A product is created just after “not found” is cached. What should the policy explain?

How long the negative entry may hide the new product and whether creation invalidates it. Distinguish genuine absence from temporary source failure and authorisation denial. Keys and lifetimes must prevent one tenant's response from misleading another.

← Return to question

UNIT 11 / S3

Refresh keeps failing while stale data is served. What limit is needed?

Define fresh, stale-permitted, and unusable ages or another explicit degradation bound. Serving must stop or change once the allowed stale condition ends. Some descriptions tolerate old values, but correctness-critical operations need their authoritative checks. Do not silently extend stale use forever.

← Return to question

UNIT 11 / D1

A cache-aside reader repopulates old data after the writer invalidates it. Define and repair the actual freshness requirement.

Trace old read, new commit, invalidation, delayed old insertion. If that stale result is forbidden, use a supported version or coordination protocol. If bounded staleness is allowed, state its limit and validate authoritative state for critical actions. TTL is not a universal repair.

← Return to question

UNIT 11 / I1

Run ideal LRU, then reproduce stale cache-aside population after invalidation.

A,B,C,A,D at capacity three evicts B. Next pause a reader with version one, commit version two and invalidate, then resume old population. Show the stale result. Apply a justified version or coordination rule if the freshness requirement forbids it; shortening TTL alone does not remove the race.

← Return to question

UNIT 11 / T1

When should a product stop serving rather than return a stale cached result?

Stop when the operation requires current authority or the permitted stale interval has ended. An old description may be allowed, while allocating scarce stock or deciding current access may not be. State which actions degrade and which wait or reject, with visible status and a recovery path.

← Return to question

Unit 12 · Scalability, Load Balancing, and Data Distribution

UNIT 12 / C1

Why can adding application servers fail to improve throughput?

They can send more work to the same saturated database, disk, lock, or provider. Identify the constrained path through waits and useful completions first. Extra independent compute helps when compute is the limit and downstream capacity remains available; it is not a universal response to a slow service.

← Return to question

UNIT 12 / C2

What becomes harder when a dataset is sharded?

Routing, ownership movement, cross-shard queries, uniqueness, and transactions can require additional protocols. A tenant key may preserve useful locality and leave one large tenant hot. Choose the key and recovery procedure from the actual access and invariant needs rather than from total row count alone.

← Return to question

UNIT 12 / C3

What does stable hashing change, and what does it not balance?

It limits some reassignment when owners change. In rendezvous hashing, adding D moves only keys newly won by D in the model. It does not transfer bytes, synchronise membership, or divide one popular key's traffic. Key distribution and request popularity are separate measurements.

← Return to question

UNIT 12 / C4

Why distinguish received and applied replica progress?

A replica can receive log bytes without yet applying them to the state it serves. Required persistence is another milestone. These positions help explain stale reads and decide safe promotion. “The replica has the update” is too vague unless it names the stage relevant to the claim.

← Return to question

UNIT 12 / C5

Why compare scaling costs under the same failure objectives?

A cheaper healthy-state arrangement may have no capacity after one node fails or may lose more acknowledged data. That is a different service. Include surviving capacity, copies, coordination, transfer, monitoring, and operating work when comparing equal useful outcomes.

← Return to question

UNIT 12 / S1

One partition receives 70% of writes. What should guide the fix?

Find whether its pressure comes from many divisible keys, one hot entity, large records, or a serialised invariant. Splitting a range can help divisible work; more cold partitions cannot split a single key automatically. Choose a change that preserves query and ownership semantics while changing the measured pressure.

← Return to question

UNIT 12 / S2

How can scaling application replicas make a database outage worse?

Each replica may create another pool, multiplying total connections and competing work. During retries, more instances can add attempted demand without useful capacity. Bound aggregate admission and investigate connection hold time and contention before increasing every pool or instance count.

← Return to question

UNIT 12 / S3

Two replicas share one host. Which failure do they not survive?

That host's loss can stop both. They may tolerate some individual process failures while sharing disk, power, configuration, or credentials. Map independent failure domains and actual takeover capacity instead of treating two process boxes as two fully independent copies.

← Return to question

UNIT 12 / D1

Scaling application replicas increases database timeouts. Design an investigation and correction.

Measure aggregate connections, hold and lock times, offered work, useful completion, and queues. Bound admission and preserve recovery capacity. Improve the constrained query or resource where evidence supports it. More upstream capacity can be harmful when it only amplifies demand on the same owner.

← Return to question

UNIT 12 / I1

Compare rendezvous owners before and after adding D, then introduce a hot key.

In the provided model, each changed assignment moves to D. Count key distribution, then send most requests to one key and measure traffic distribution separately. Adding cold owners does not divide that key. State what safe serving or ownership change would address its actual pressure.

← Return to question

UNIT 12 / T1

Can more partitions scale one globally hot exclusive ownership decision?

Cold partitions do not divide the authority required for one exclusive key. Read copies or batching can help suitable work, while changing the invariant or allocation model needs a valid protocol. Measure the hot owner and explain how any proposed partition changes its actual work without permitting conflicting winners.

← Return to question

Unit 13 · File Systems, Search Infrastructure, and Data Pipelines

UNIT 13 / C1

Why do acknowledged upload parts not prove the completed file is correct?

The parts still need correct assembly, identity, and completion verification. Repeated parts or a changed client source can produce another result. Check final size and integrity under the storage protocol. A provider-specific entity tag is not universally a checksum of the final content.

← Return to question

UNIT 13 / C2

Why does a content hash not authorise a download?

The digest identifies bytes under the algorithm's assumptions. It says nothing about the requester's permission to those bytes or their safety. Bind object access to ownership and policy, and scope deduplication carefully where exposing existence could leak information across tenants.

← Return to question

UNIT 13 / C3

What does an inverted index record for “blue car” and “blue bike”?

With documents D1 and D2, the blue posting list contains both IDs, while car contains D1 and bike contains D2. Tokenisation affects the recorded terms. Versions and permission rules are additional metadata needed to keep this derived lookup usable and safe.

← Return to question

UNIT 13 / C4

What can go wrong between a snapshot and its CDC stream?

Changes made while the snapshot is copied can be missed or applied incorrectly unless the handoff uses a defined consistent position. Preserve identities, ordering information, and schema support, and handle repetition. Transporting changes does not automatically give every destination query the source's transaction view.

← Return to question

UNIT 13 / C5

What provenance makes an extracted field explainable?

Name the exact file version and source region, transform or extractor version, relevant parameters, and review decision. The references should locate actual evidence. Confidence alone is not provenance or accuracy. Retain the required evidence under an appropriate privacy and lifecycle policy.

← Return to question

UNIT 13 / S1

Updates and deletes continue during reindexing. What must happen before serving switches?

Apply changes from a defined snapshot position through a validated catch-up boundary, including removals. Check identities, versions, permissions, and useful output before selecting the new generation. Keep a compatible rollback path. Matching document counts alone cannot establish readiness.

← Return to question

UNIT 13 / S2

An event arrives three minutes late. Which time policy matters?

Distinguish its source event time from handling time. The window needs an allowed-lateness and correction rule, with clear provisional or final results. A watermark estimates progress under assumptions; it does not prove no older event can ever arrive.

← Return to question

UNIT 13 / S3

Why is deleting only the PDF insufficient?

Extracted text, thumbnails, indexes, logs, and recoverable copies can retain its information. Track the required deletion work across those paths and prevent old workers or replay from republishing it. Recovery must also apply the relevant deletion history before exposing older restored data.

← Return to question

UNIT 13 / D1

A reindex copies records while updates and deletions continue. Design the handoff to a new serving generation.

Choose a defined snapshot/change position, capture concurrent changes including deletes, validate versions and access, and switch through an explicit pointer or epoch. Keep rollback compatible. Compare meaning and identities, not just counts, and retain appropriate rebuild evidence.

← Return to question

UNIT 13 / I1

Implement source-version and attempt checks, then plan a safe index-generation switch.

Reject old epochs and wrong inputs with Workflow. Preserve transform and source identity. Build the new representation from a defined position, include later updates and deletes, validate, then switch. Counts alone cannot show correct versions, content, or access scope.

← Return to question

UNIT 13 / T1

How would you investigate an extracted field after both source and model have changed?

Use stable references to the exact source version, region, transform/model version, parameters, and review evidence. Keep the disputed output version identifiable. Re-running a changed or nondeterministic model is not proof of the old result. Retain appropriate evidence under the lifecycle policy and distinguish reproducibility limits from missing provenance.

← Return to question

Unit 14 · Distributed Systems Fundamentals

UNIT 14 / C1

Why is a provider charge with a lost reply a partial failure?

The external action happened while the local caller's observation did not complete. A local rollback cannot undo it. Record uncertainty and reconcile using a stable operation reference. Treating the timeout as a definite decline can lead to another charge.

← Return to question

UNIT 14 / C2

What does Lamport ordering establish?

Causally ordered events receive increasing Lamport values under the algorithm. A lower value alone does not prove that event caused the higher-valued event. Vector clocks can identify concurrency under their participant model, but neither clock supplies a business conflict resolution or consensus decision by itself.

← Return to question

UNIT 14 / C3

Which real-time rule must a linearizable register preserve?

A completed write must precede a read that starts afterward in the allowed atomic history. Without an intervening write, that read cannot return an overwritten earlier value. Overlapping operations may have several legal orders. A timeout can still correspond to an effect whose result the client did not receive.

← Return to question

UNIT 14 / C4

Why is CAP not a permanent choice of any two attractive qualities?

It concerns defined atomic-consistency and availability properties in the presence of partitions. During lost communication, decide which operations wait, reject, or offer an explicitly weaker result. Ordinary nonpartitioned coordination and latency need further analysis. The formal terms should not be equated loosely with product adjectives.

← Return to question

UNIT 14 / C5

Can a system preserve safety while being useless?

Yes. Rejecting every booking avoids selling a seat twice but makes no accepted valid booking progress. State both the forbidden outcome and the progress required under explicit environmental assumptions. Recovery must restore useful progress without violating exclusive ownership.

← Return to question

UNIT 14 / S1

A session sees version ten, then version nine. What guarantee failed?

Its reads moved backward, violating monotonic reads if that guarantee was promised. Session progress may need routing or version tracking, including after failover or device changes. A sticky instance alone does not establish the behaviour for every transition.

← Return to question

UNIT 14 / S2

Why does R=2,W=2,N=3 not prove linearizability?

The sets intersect, so a read contacts at least one member of the write set. The protocol still needs version interpretation, concurrent-write ordering, durable evidence, membership, and read rules. A set relationship is one ingredient, not a complete legal-history argument.

← Return to question

UNIT 14 / S3

An old leader resumes after a replacement takes over. Where must stale effects be stopped?

At the resource that accepts them, using a suitable authority protocol such as atomically checked fencing epochs. A local timer cannot stop the old process. Test pauses and delayed messages; a clean process kill hides the resumed-owner history.

← Return to question

UNIT 14 / D1

During a partition, two sides both try to allocate the same exclusive resource. State an acceptable serving policy.

Identify the authority protocol and which side can establish a valid decision. Reject or wait where authority is unavailable; permit only explicitly safe weaker operations. A timeout does not stop the old actor. Recovery needs evidence and fencing rather than merging two exclusive winners.

← Return to question

UNIT 14 / I1

Evaluate Lamport updates, vector relationships, and fixed-quorum intersection.

lamport_receive(5,9) gives 10. Vectors [1,0] and [0,1] are concurrent; [1,0] precedes [1,2] under this comparison. With fixed N=3,R=2,W=2, sets intersect. Explain the additional ordering, persistence, and membership rules needed for any larger consistency claim.

← Return to question

UNIT 14 / T1

Which operations may continue during a partition while exclusive allocation must wait?

Operations with explicitly permitted stale or independent results may continue. Exclusive allocation needs valid authority and may have to wait or reject when agreement is unavailable. State per-operation observations rather than weaken the entire system silently. Recovery must reconcile progress without introducing another owner.

← Return to question

Unit 15 · Replication, Leader Election, and Consensus

UNIT 15 / C1

What must “synchronous replication” say about acknowledgment?

Specify the required replicas and whether their replies mean receipt, durable storage, or application. Those milestones support different recovery and read guarantees. Waiting adds latency and can block during communication failure. Test promotion under the same acknowledgment contract the application promises.

← Return to question

UNIT 15 / C2

Why must replicated command application be deterministic?

Replicas applying the same ordered commands must obtain equivalent state. Independently choosing time or random IDs can make results differ. Supply such decisions consistently in the agreed input or use the protocol's supported mechanism. External side effects remain separately identified and protected.

← Return to question

UNIT 15 / C3

Why does Raft require more than counting two acknowledgments?

Votes, log eligibility, durable state, terms, and commit rules work together. Direct majority-based commitment is restricted to current-term entries; earlier history is protected through those rules. An arbitrary older entry on two nodes is not a replacement safety argument. Reads and membership also need their specified mechanisms.

← Return to question

UNIT 15 / C4

Why is an abrupt switch to a disjoint voting group unsafe?

Old and new majorities could independently accept incompatible histories with no shared evidence. Use the supported membership protocol, such as Raft's joint-consensus transition, and catch new members up appropriately. Preserve identities and test interruption while the configuration changes.

← Return to question

UNIT 15 / C5

Why can convergent replicas still violate a stock rule?

A merge can deterministically include all updates while their combined effect takes stock below zero. Convergence describes agreement under the type's rules; it does not enforce arbitrary cross-update invariants. Choose semantics that fit the domain or introduce the coordination needed for exclusive allocation.

← Return to question

UNIT 15 / S1

Worker 7 resumes after worker 8 owns the attempt. What enforcement is needed?

The protected receiver must atomically compare authority and reject the stale epoch before applying its effect. Creating tokens only in the coordinator does not protect an external store that ignores them. Local workflow checks cover local progress, not every remote operation automatically.

← Return to question

UNIT 15 / S2

A pause creates a false failure suspicion. What must remain correct?

The authority and safety protocol must prevent conflicting accepted decisions even when detection is wrong. Detection tuning affects speed and false suspicion, while election, persistence, and fencing determine which effects are valid. A missing heartbeat cannot be treated as proof that the old process no longer runs.

← Return to question

UNIT 15 / S3

What must happen before a repaired old primary accepts writes again?

It must rejoin under current authority and reconcile or rebuild its history as required. Fence stale writes and account for replica lag and the acknowledged-data policy. Test failback as well as promotion; restoring traffic to an old owner prematurely can reintroduce conflict after the apparent outage ended.

← Return to question

UNIT 15 / D1

A paused worker returns after its lease expired and a new owner started. Design stale-effect protection.

Use ordered ownership tokens checked atomically where protected effects are accepted. Verify owner identity on release as well. Distinguish earlier accepted effects from later stale actions. The lease detects an interval; receiver enforcement prevents the resumed old owner from writing under obsolete authority.

← Return to question

UNIT 15 / I1

Implement a receiver that rejects stale attempt epochs.

After epoch 8 owns the local workflow, a completion at epoch 7 must fail. Perform comparison and update atomically in a persistent version. Then identify every external effect receiver that also needs authority enforcement; a client-side lease flag cannot stop a paused process from resuming elsewhere.

← Return to question

UNIT 15 / T1

What must a repaired region establish before it accepts authoritative writes again?

It needs current authority, compatible committed history, required durable copies, valid routing, and sufficient capacity. An old leader may require rebuilding and fencing. Apply the supported rejoin protocol and reconcile uncertain external effects. A repaired machine being reachable is not evidence that its old state is safe to resume.

← Return to question

Unit 16 · Message Queues, Events, and Distributed Workflows

UNIT 16 / C1

Why cannot an absent ephemeral subscriber replay missed history automatically?

Its subscription may retain nothing while it is disconnected. Recovery requires retained records and a way to track progress, such as a log position under the chosen system. Check the actual broker policy rather than assuming every pub/sub product provides durable replay.

← Return to question

UNIT 16 / C2

How does acknowledgment placement change crash behaviour?

Acknowledging before a required effect can lose work after a crash. Applying the effect first can repeat work if acknowledgment is lost. Protect repetition at the relevant boundary, such as an inbox and local effect in one transaction. Neither position alone covers an unrelated external provider.

← Return to question

UNIT 16 / C3

What can an exactly-once stream transaction actually cover?

The records and progress atomically coupled by its supported protocol and fault assumptions. An arbitrary email server is outside that boundary unless another mechanism covers it. Name the effect and identity scope. Repeated execution can still produce one local relevant result through idempotency without execution starting only once.

← Return to question

UNIT 16 / C4

Why can an outbox relay publish the same event twice?

It can publish successfully, then crash before saving that it sent the event. Recovery sees the pending intent and publishes again. Stable event identity and consumer duplicate handling address this gap. The original database transaction records the obligation; it does not include the entire network delivery.

← Return to question

UNIT 16 / C5

Why is saga compensation not always an undo?

A refund or reservation release is another action that may fail, happen later, or have different fees and consequences from the original. Persist compensation progress and reconcile uncertainty. Some effects cannot be perfectly reversed, so the business must define an acceptable recovery outcome.

← Return to question

UNIT 16 / S1

An effect commits and the message is delivered again. How can one local update be preserved?

Store the unique event identity and update in the same local transaction. A duplicate then observes the established identity and skips repeating the effect. Retain it for the supported replay horizon. Substituting an external call for the local update requires its own protocol.

← Return to question

UNIT 16 / S2

A worker keeps running after its lease expires. What protects later effects?

A current-epoch check can reject stale local completion, while the actual effect receiver needs appropriate fencing or duplicate identity. The expired lease itself cannot stop execution. Distinguish effects already accepted before replacement from later stale attempts, and reconcile what remains uncertain.

← Return to question

UNIT 16 / S3

Why not resend an uncertain payment as a new operation?

The provider may already have charged. Query by stable operation identity and repair local progress, or use the provider's supported safe retry. A new identity can produce another financial effect. Reconciliation is part of the recovery protocol rather than evidence that the original intent never ran.

← Return to question

UNIT 16 / D1

An order commits but its event is never published. Design durable publication and repeated-delivery handling.

Commit the order and outbox intent together. Retry relay progress with a stable event ID, accepting that publish-before-mark can repeat. A consumer protects its local effect with the event identity in one transaction. Other external effects need their own identity and reconciliation contract.

← Return to question

UNIT 16 / I1

Crash the outbox relay after publication, then process the duplicate with a local inbox transaction.

The fixture should show two deliveries and one counter change. Event identity and the local effect must commit together. Replace the counter conceptually with email and identify the missing external protocol; local duplicate recognition alone cannot establish one external send across the failure gap.

← Return to question

UNIT 16 / T1

How could one broker transaction be exactly-once while two emails are sent?

The broker may atomically couple consumed progress and produced records while email lies outside that transaction. A consumer can send, lose its completion reply, and send again on recovery. Use the provider contract, stable identity where supported, and reconciliation; do not extend the broker guarantee to an unrelated effect.

← Return to question

Unit 17 · Reliability, Fault Tolerance, and Disaster Recovery

UNIT 17 / C1

Why are two replicas not necessarily enough surviving capacity?

Their independence and spare resources matter. Two instances each at 70% cannot absorb the other's full traffic after one fails. Plan headroom or a permitted reduced service, with valid state and tested traffic movement. Shared configuration or storage can also fail both together.

← Return to question

UNIT 17 / C2

Why include pool acquisition in the overall deadline?

A request spends useful lifetime waiting before it starts its database work. Giving each stage a fresh full timeout can let it run long after the caller's deadline. Propagate remaining time and clean up appropriately. A timeout still does not prove that a remote effect never occurred.

← Return to question

UNIT 17 / C3

What does a circuit breaker help, and what remains unresolved?

It can stop repeated futile calls and resource waits while allowing bounded recovery probes. It does not repair the dependency or decide a valid business fallback. Define failures and probe limits. Returning a fabricated payment approval would preserve neither correctness nor a useful recovery contract.

← Return to question

UNIT 17 / C4

How can separate worker pools still share a failure bottleneck?

They may exhaust the same database, memory, network, or credential dependency. Partition the resources through which harm can spread, and measure each budget. Separate names or threads do not establish independent downstream capacity. Excess separation also has idle-capacity costs.

← Return to question

UNIT 17 / C5

How do RTO and RPO differ?

RTO concerns how soon required service should recover. RPO concerns acceptable data-loss exposure in time. Restoring at 12:20 after a noon incident and recovering data only through 11:55 gives different twenty-minute and five-minute measurements. Test the full recovery procedure against both objectives.

← Return to question

UNIT 17 / S1

How should an end-to-end design limit a retry storm?

Use one overall attempt budget and deadline, bounded admission, safe operation identity, and resource isolation. Backoff with jitter spreads timing but does not reduce allowed attempts by itself. Track useful intents and completions separately from attempts, and leave capacity available for recovery.

← Return to question

UNIT 17 / S2

Can a default cached balance replace an unavailable ledger?

Only a contract explicitly permitting that particular observation could justify a display. It cannot invent authority to approve a financial action. A help page and a ledger lookup have different meanings. Define freshness, duration, permissions, and allowed degraded operations instead of treating any returned value as a safe fallback.

← Return to question

UNIT 17 / S3

Why not send traffic straight back to a repaired primary?

It may have stale or incompatible history and no current authority. Rejoin, fence, reconcile, and verify capacity and routing according to the protocol. Existing connections and cached addresses can complicate the transition. Failback needs testing just as failover does.

← Return to question

UNIT 17 / D1

A provider slowdown holds every connection and triggers layered retries. Break the cascade without inventing business success.

Use overall deadlines, bounded admission and retry budgets, resource isolation, and limited recovery probes. Preserve a valid pending or degraded response where allowed. Track unresolved effects, and reserve capacity for reconciliation and recovery. Test the fallback path for its own overload risk.

← Return to question

UNIT 17 / I1

Calculate nested retry amplification and design a bounded provider-outage policy.

Three attempts at four layers permit up to 81 downstream attempts. Introduce an overall budget, deadlines, admission, and appropriate isolation. Choose a valid pending or failure response instead of fabricated payment approval. Check recovery and remaining uncertain operations after the provider becomes reachable.

← Return to question

UNIT 17 / T1

How can a disaster drill restore infrastructure yet fail the business recovery requirement?

Machines can restart while the recovered data is too old, encrypted backups are unreadable, active authority conflicts, or accepted jobs are missing. Check the complete business journey and its RTO/RPO, including keys, state, identity, validation, and useful traffic return. A drill proves only the scenarios it actually exercises.

← Return to question

Unit 18 · Security Architecture and Privacy

UNIT 18 / C1

Why is an internal network address not proof of a caller's identity?

Network location establishes a route or reachability, not necessarily the actor's identity or permission. A compromised internal component can still send harmful requests. Validate the relevant credentials and resource policy at explicit boundaries, including trusted forwarding configuration rather than arbitrary supplied headers.

← Return to question

UNIT 18 / C2

How do OAuth 2.0 and OpenID Connect differ in purpose?

OAuth concerns delegated access authorisation. OpenID Connect adds an identity layer with defined assertions and validation. An arbitrary access token is not automatically a login assertion. Use the intended flow and reviewed validation for issuer, audience, token type, and required context.

← Return to question

UNIT 18 / C3

Why use purpose-built password hashing rather than a fast hash?

Offline attackers can try guesses cheaply against fast hashes. Password hashing adds salts and deliberately expensive work under a suitable algorithm and parameters. Use maintained libraries and current guidance, with migration metadata and capacity checks. Recovery and online verification also need separate abuse protection.

← Return to question

UNIT 18 / C4

Which paths need tenant isolation beyond the database query?

Caches, task messages, worker reads and writes, object capabilities, search, exports, telemetry, and administrative tools can all carry private data. Preserve trusted scope and current resource relationships throughout. A correct query cannot compensate for a public object link or a cache key that omits tenant identity.

← Return to question

UNIT 18 / C5

Does an artifact's valid signature prove its code is safe?

It proves the checked signing relationship and integrity under the scheme's assumptions. The signer or build inputs can still be compromised, or the code can be vulnerable. Control dependencies, build privileges, provenance, deployment access, and revocation alongside signature verification.

← Return to question

UNIT 18 / S1

A tenant A administrator guesses a tenant B document ID. What denies access?

Resource-specific authorisation using trustworthy membership and ownership must reject it on every relevant path. The tenant-admin role is scoped, not global. Difficult identifiers and a successfully authenticated session do not supply the missing permission.

← Return to question

UNIT 18 / S2

A backup survives but its only decryption key is destroyed. What went wrong?

Recovery depended on a key that did not survive the protected failure. Backup testing must include key versions, credentials, schema, and the actual application restore path. Preserved ciphertext alone does not establish usable recoverable data.

← Return to question

UNIT 18 / S3

Membership is revoked after an export is queued. Where must that policy be enforced?

At each later boundary required by the chosen revocation contract, including execution and publication or download. The original request's valid permission may no longer suffice. Give queued work trusted context but recheck current state where needed, and track denied or cancelled progress durably.

← Return to question

UNIT 18 / D1

A private export is authorised at submission but its worker trusts the old tenant field forever. Review the complete journey.

Carry trusted identity and resource scope, apply the chosen current revocation rule, limit worker capabilities, and protect the output/download lifetime. Trace caches, search, and audit too. Test membership removal between enqueue, execution, publication, and download under the stated policy.

← Return to question

UNIT 18 / I1

Test the synthetic tenant predicate, then trace all data paths using that decision.

permit_demo should allow the intended same-tenant action and deny tenant A access to B. It is not credential validation. Test scope in caches, jobs, objects, exports, and search too. A valid predicate cannot protect a boundary that never calls it or receives untrusted identity context.

← Return to question

UNIT 18 / T1

What should revocation mean for an issued object capability and a queued export?

Specify both boundaries. An already issued capability may remain valid until expiry unless the delivery system supports revocation. A queued export may need a current policy check before execution or publication. Keep users informed about the chosen lifetime and ensure workers cannot bypass it using an old submission snapshot.

← Return to question

Unit 19 · Observability, Monitoring, and Incident Engineering

UNIT 19 / C1

How do logs, metrics, and traces complement one another?

Metrics describe populations and trends, traces connect one execution's work and waits, and logs record events with detailed context. Use stable identities to relate appropriate evidence. Sampling, dropped records, and privacy limits affect all three, so no single signal automatically tells the complete story.

← Return to question

UNIT 19 / C2

Why is a trace ID not an access credential?

It labels diagnostic context, which may originate from an untrusted caller and does not prove identity or resource permission. Validate or replace context according to policy. Authentication and authorisation must use their intended evidence, while trace propagation helps investigate the resulting operation.

← Return to question

UNIT 19 / C3

Why can low CPU coexist with severe request delay?

Requests can be waiting for connections, locks, storage, or a slow remote call. RED can show rising durations while USE identifies saturation of the constrained resource. Inspect the path before buying CPU; idle compute does not free an occupied connection pool.

← Return to question

UNIT 19 / C4

What is tenfold budget burn for a 99.9% success target?

The allowed bad-event rate is 0.1%. An observed 1% bad-event rate spends budget ten times as fast for the defined eligible population. Keep the measurement window and denominator clear, and segment critical work where a large overall count could hide its harm.

← Return to question

UNIT 19 / C5

Why avoid unique request IDs as ordinary metric labels?

Each new value can create another time series, making cardinality and storage grow without a practical bound. Use bounded dimensions for metrics and put detailed operation identities in appropriate logs or traces. Retention and sampling should match the diagnostic purpose.

← Return to question

UNIT 19 / S1

A five-second request waits 4.6 seconds for a connection. Where should investigation focus?

Pool wait accounts for 92% of that request's time. Examine hold times, locks, queueing, and population evidence before assuming query execution or CPU is the problem. One trace suggests a mechanism to test; it does not explain why the pool is full by itself.

← Return to question

UNIT 19 / S2

Serving recovers but charges remain uncertain. Why is recovery incomplete?

Users can reach the API while business outcomes still disagree with the provider or lack postings. Track those operations through reconciliation and safe final states. Closing an infrastructure alert does not establish that every accepted financial intent is resolved.

← Return to question

UNIT 19 / S3

HTTP 202 is fast while jobs wait for hours. Which indicators are missing?

Measure submission-to-start and submission-to-required-outcome age, stage queues, attempts, and final useful completion. Follow stable operation identity beyond the HTTP trace. Acceptance latency describes only one boundary and can remain excellent while the promised asynchronous service stops progressing.

← Return to question

UNIT 19 / D1

HTTP acceptance stays fast while background jobs stop finishing. Redesign the service indicators and alert.

Define eligible accepted jobs, required terminal outcomes, and age limits. Track stage wait, attempts, stuck work, and useful completion through stable identity. Alert on actionable oldest-age or completion harm. A healthy endpoint or enqueue rate covers only the beginning of the journey.

← Return to question

UNIT 19 / I1

Build the five-second trace model and an asynchronous completion indicator.

The fixture has 4.6 seconds of pool waiting, 100 ms query, and 300 ms elsewhere. Pool wait is 92% of elapsed time. Investigate its cause using population and resource evidence. For jobs, measure submission to required outcome, not just HTTP acceptance. Keep detailed identifiers out of unbounded metric dimensions.

← Return to question

UNIT 19 / T1

What evidence would support a latency explanation beyond correlation with a release?

Locate the extra wait or execution on affected paths, compare normal and slow populations, inspect resource saturation and changed behaviour, and test a focused intervention. A release marker is a clue. Evidence is stronger when the mechanism predicts the symptom and the correction changes that mechanism under comparable conditions.

← Return to question

Unit 20 · Deployment, Cloud Infrastructure, and Platform Architecture

UNIT 20 / C1

What important boundary differs between a VM and a typical container?

A VM supplies a guest operating system on virtualised hardware; containers commonly share a host kernel while isolating processes. Both need resource, identity, network, storage, and image management. Check actual isolation and shared failure domains rather than treating packaging as a complete security or reliability guarantee.

← Return to question

UNIT 20 / C2

How can a control-plane outage differ from a serving outage?

Existing workloads may keep serving while new placement or configuration changes cannot proceed. Replacement may need a dependency that current requests do not use. Draw control and data paths separately and define cached-configuration validity. Avoid unnecessary fresh control decisions in each request.

← Return to question

UNIT 20 / C3

Why must a canary receive representative traffic?

A candidate exposed only to cheap reads can appear healthy while failing the expensive checkout path. Select cohorts and indicators that exercise the release risks, with stop conditions and sufficient observations. A small exposure is useful only when it tests behaviour relevant to the broader rollout.

← Return to question

UNIT 20 / C4

Why can an old executable fail as a rollback?

New code may have written data or a schema that the old reader cannot understand. Expand-and-contract preserves an overlap period before obsolete state is removed. Identify when contraction ends safe rollback and what forward repair remains. Reversing traffic does not reverse every stored or external effect.

← Return to question

UNIT 20 / C5

Which duties remain with a team using a managed database?

Schema, access, retention, application transaction use, acknowledgment choices, and validation of recovery still require owners. Provider tasks vary by contract. Test maintenance, throttling, failover, and credential changes instead of assuming “managed” covers every application correctness need.

← Return to question

UNIT 20 / S1

Scale-down kills a worker holding an unrecorded job. What was missing?

Durable accepted-work identity and a safe drain or transfer protocol. Stop new claims, persist progress, and let another valid owner recover as required. Killing a process cannot preserve work that existed only in its memory, regardless of how well the platform schedules replacements.

← Return to question

UNIT 20 / S2

A feature is disabled after writing a new data format. What risk remains?

Old code and clients may still read the new state. A flag changes exposure, not stored compatibility or past effects. Keep migration and reader support explicit, including what rollback or forward repair is supported after the new data exists.

← Return to question

UNIT 20 / S3

Why do two successfully scheduled replicas not necessarily form a correct shared service?

Each may use an unrelated local database or incompatible configuration. Placement establishes running instances, not common state ownership. Define the shared or coordinated data model, identity, and failure behaviour before distributing requests across them.

← Return to question

UNIT 20 / D1

A blue-green release shares one database and removes a column needed by the old version. Review its rollback plan.

Parallel app environments do not make shared state reversible. Use compatible expansion and overlap-safe readers, backfill and verify, then retire old readers before contraction. Mark the end of safe rollback and the forward repair path. Test representative mixed-version traffic.

← Return to question

UNIT 20 / I1

Plan and test an expand-and-contract release with old and new readers.

Add compatible state, deploy overlap-safe readers, backfill with bounded progress, and verify. Retire old readers before removing obsolete schema. Test rollback before that point and identify what requires forward repair afterward. Expose representative traffic to a canary, including important expensive paths.

← Return to question

UNIT 20 / T1

How can rollback restore old code while leaving incompatible data or effects?

Old code can return while new schemas, values, provider calls, and published messages remain. Maintain compatibility during overlap and identify the point where contraction ends safe rollback. Some outcomes need forward repair or compensation. Restoring an artifact cannot reverse every state transition it previously performed.

← Return to question

Unit 21 · Performance, Cost Optimization, and System Validation

UNIT 21 / C1

What must accompany a reusable benchmark result?

Workload, data distribution, hardware or capacity, versions, warmup, concurrency, duration, success definition, variability, and relevant failures. A function microbenchmark cannot establish full application throughput. Preserve enough setup to reproduce and interpret the claim rather than presenting one number without its conditions.

← Return to question

UNIT 21 / C2

How can coordinated omission hide waiting?

A generator waits for a slow reply before issuing later work that independent real arrivals would already have submitted. Some queueing and delay disappear from the observed population. Choose a generator model matching the workload and record offered, completed, failed, and abandoned work.

← Return to question

UNIT 21 / C3

Why is a short measured throughput maximum not a safe production target?

Real operation includes variance, maintenance, background work, and required failure capacity. Near saturation, extra arrivals sharply worsen waiting or rejection. Locate the useful knee and reserve margin against the actual objective rather than running indefinitely at one short benchmark's ceiling.

← Return to question

UNIT 21 / C4

Why can cost per attempt hide increasing cost per useful outcome?

Retries and failures can enlarge the attempt denominator without producing additional completed orders or documents. Define successful business outcomes and the accounting period, while retaining technical breakdowns. Compare candidates with equivalent correctness and service objectives.

← Return to question

UNIT 21 / C5

What should an architecture fitness function protect?

A meaningful required property such as module dependencies, tenant isolation, compatibility, or tested recovery. It should permit harmless implementation changes and be updated when a valid architecture decision changes the requirement. Its pass establishes only its covered evidence, not universal correctness.

← Return to question

UNIT 21 / S1

An index fixes the slow query. Why might sharding have been a poorer first step?

Sharding changes ownership, routing, transactions, and migration while the avoidable scan may remain. A targeted implementation fix can address the measured mechanism with less risk. Architectural change is justified when the underlying limit actually requires it, not because it sounds more scalable.

← Return to question

UNIT 21 / S2

What does killing a process only before work begins fail to test?

It misses gaps such as commit before reply, output write before completion record, or resumed work after lease expiry. Inject faults at the boundary in question and check both serving and required state. Successful restart alone does not establish that accepted work survived.

← Return to question

UNIT 21 / S3

Source and destination counts match but tenant owners differ. Why did validation miss it?

A count compares quantity, not identity or meaning. Compare keys, versions, relationships, ownership, and business invariants as appropriate. State whether the check examines all records or samples. Migration, replay, and recovery need evidence that the right data reached the right scope.

← Return to question

UNIT 21 / D1

A faster and cheaper benchmark configuration omits failures and disables authorisation. Can it replace the original design?

It tests a different workload and service contract. Restore mandatory correctness and access checks, include failed and waiting work, disclose setup, and compare useful outcomes under the original requirements. A favourable number cannot compensate for removing the properties the service must provide.

← Return to question

UNIT 21 / I1

Compare the three fictional architectures against equal requirements and costs.

cost_options totals 371, 551, and 1,402 monthly units. Keep these inputs separate from measured capacity and reliability. Use the constructed latency and independent-component examples only within their assumptions, then state which real tests would establish each candidate meets the original workload and service goals.

← Return to question

UNIT 21 / T1

When could a more expensive architecture be simpler to operate overall?

Compare total work for people as well as the invoice. A managed service may reduce patching, backup, or recovery burden while adding charges and leaving application responsibilities. Evaluate the same correctness and service goals with explicit staffing assumptions. Choose the design the team can understand, change, and recover reliably.

← Return to question

Unit 22 · End-to-End Design, Architecture Reviews, and Evolution

UNIT 22 / C1

Why can a click count and a seat claim use different consistency rules?

A click count can often tolerate delayed aggregation. A seat claim needs one valid owner before confirmation. State each operation’s rule and failure behaviour rather than assigning one vague consistency label to the whole application.

← Return to question

UNIT 22 / C2

What does a persisted chat receipt establish, and what does it not establish?

It establishes the accepted history state under the storage contract. It does not prove a device received a frame or a person read it. Track those later acknowledgments separately, with reconnect and deduplication behaviour for missing or repeated frames.

← Return to question

UNIT 22 / C3

Why can a scheduler lease not guarantee one external execution?

An old worker can pause past expiry, then resume beside a replacement. Conditional local completion protects local progress, while the external receiver needs safe identity or enforced fencing. Some effects accepted before authority changed still need reconciliation.

← Return to question

UNIT 22 / C4

Why is equal final text insufficient to validate a collaborative editor?

The history might include unauthorised edits, invalid rich-text structure, wrong Unicode handling, or broken undo semantics. Check the selected protocol's convergence and supported context, then test access, structure, and recovery independently. A matching string proves only that particular final comparison.

← Return to question

UNIT 22 / C5

What should complete mean for a document-processing workflow?

It must name the required usable, permitted output and its exact source, pipeline, and review evidence. A worker finishing extraction may leave publication or review unresolved. Track those stages separately and preserve provenance instead of treating one exception-free attempt as verified truth.

← Return to question

UNIT 22 / S1

Payment succeeds after a seat hold expires. What may the service safely do?

Use the documented recovery policy. It may reacquire the seat only if current ownership permits it, or follow a refund/other supported resolution. It must not issue against another buyer's confirmed seat. Preserve provider identity and reconcile the financial outcome rather than discard it.

← Return to question

UNIT 22 / S2

The provider charges, the process stops, and the client retries. How is another logical charge avoided?

Keep stable intent and provider operation identity. Use the provider’s safe repeat or status protocol and reconcile the existing effect before new submission. Apply verified evidence to one valid local transition and posting set. Local keys alone cannot impose unsupported behaviour on the provider.

← Return to question

UNIT 22 / S3

An old extraction worker finishes after document deletion. What stops publication?

The authoritative completion transition checks current epoch, input, and live deletion state atomically. The publisher and derived paths must respect that state too. Cancelling the worker alone is insufficient because it may already be delayed or running elsewhere.

← Return to question

UNIT 22 / D1

Review a short-link service at 100,000 redirects/s after its cache becomes unavailable.

The warm 90% hit assumption no longer limits source traffic to 10,000 reads/s. Define bounded origin admission or an allowed failure response, test cold capacity, and preserve revocation rules. More cache replicas alone do not establish safe recovery or immediate invalidation.

← Return to question

UNIT 22 / I1

Choose one case and implement its failure exercise with a checked invariant.

Name the exact local boundary and expected outcomes, inject the described interruption, and compare the recovered state. Include repetition or stale work where relevant. Use the mechanism fixtures as examples, while keeping external guarantees within the actual provider or protocol contract.

← Return to question

UNIT 22 / T1

How would you choose between another region, tested restores, and stronger reconciliation?

Start with the most harmful unmet requirement and demonstrated failure. Another region cannot recover missing operation identity, and retries cannot restore a destroyed sole copy. Compare risk reduction, operating work, and the evidence each option can produce. Record the choice and the condition for reconsidering it.

← Return to question

THE REFERENCE DESK

A searchable glossary

Definitions and concept summaries link directly to their complete treatment, including individual patterns.

441 of 441 terms

Abstract Factory	Creates a compatible family of collaborators through one selected construction interface. Read the lesson →
Abstraction	Presenting the operations a caller needs while hiding implementation details; the contract still states the behaviour the caller relies on. Read the lesson →
Abstraction, encapsulation, polymorphism, inheritance	Abstraction presents needed behaviour; encapsulation controls representation; polymorphism permits compatible implementations; inheritance specialises a relationship. Read the lesson →
ACID properties	ACID separates atomicity, invariant preservation, isolation, and conditional durability; application rules still need correct expression. Read the lesson →
Active-active vs active-passive	Active-active and active-passive assign serving roles for a particular function, each needing tested transitions. Read the lesson →
Adapter	Translates an existing interface or representation into a caller's required behavioural contract. Read the lesson →

Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy	Structural patterns organise interface translation, composition, grouping, added behaviour, simplified access, sharing, and mediation to a target. Read the lesson →
ADR	Architecture Decision Record: a short record of a design choice, why it was made, its consequences, and when to reconsider it. Read the lesson →
Aggregates, entities, value objects, domain services	An aggregate groups changes requiring one consistency boundary, with a root controlling its members' transitions. Read the lesson →
Alert design and alert fatigue	Useful alerts identify actionable harm or impending objective loss with an owner and response. Read the lesson →
Amdahl's Law and bottlenecks	Amdahl's Law bounds fixed-work speedup when part of the job remains sequential. Read the lesson →
Anti-corruption layers	An anti-corruption layer translates external meanings so they do not silently redefine the local model. Read the lesson →
API documentation and discoverability	Useful API documentation explains complete success and failure histories beyond machine-readable message shape. Read the lesson →
API fundamentals and interface contracts	An API is a behavioural communication contract, including identity, effects, failures, timing, and supported change. Read the lesson →

API gateways and BFF	Gateways provide shared boundary policy; BFFs adapt interactions to particular client needs. Read the lesson →
API security and common vulnerabilities	API security follows how accepted input is used, including ownership, injection, resource, and destination risks. Read the lesson →
API versioning and compatibility	Compatibility preserves supported meanings and client behaviour across a planned transition. Read the lesson →
Application architecture	The organization and interaction of application responsibilities and their dependency rules. Read the lesson →
Architecture fitness functions	Architecture fitness checks protect required properties without freezing irrelevant implementation details. Read the lesson →
At-most-once and at-least-once processing	At-most-once can lose required effects; at-least-once can repeat them and therefore needs safe effect handling. Read the lesson →
Audit trails	Audit trails record scoped actors, actions, and actual outcomes while protecting integrity and sensitive context. Read the lesson →
Authentication vs authorization	Authentication establishes identity; authorisation grants a particular action on a resource under policy. Read the lesson →

Authentication vs authorization at API boundaries	Authentication identifies a caller; authorisation checks the permitted action on the particular resource. Read the lesson →
Autoscaling	Autoscaling must include useful signals, startup, draining, durable work, downstream limits, and spending bounds. Read the lesson →
Autoscaling metrics	Autoscaling reacts to workload signals with delay, bounds, warmup, and downstream-capacity constraints. Read the lesson →
Availability	Whether a system can provide the required service when needed. A measurement must define which requests and results count. Read the lesson →
Availability vs durability	Availability concerns useful serving; durability concerns committed data surviving the specified failure model. Read the lesson →
Availability, reliability, durability, consistency	Availability, durability, reliability, and consistency concern different serving, persistence, correctness, and observation requirements. Read the lesson →
Average vs median vs p95/p99 latency	Means and percentiles describe different features of a timing distribution; combined percentiles require appropriate underlying data. Read the lesson →
B-tree/B+tree and hash indexing	Ordered trees support ranges and equality; hash indexes target equality under their implementation's rules. Read the lesson →

Backpressure	A mechanism that communicates or enforces downstream capacity limits on upstream work. Read the lesson →
Backpressure and replay	Backpressure and replay need capacity budgets, retained identity, and useful completion indicators. Read the lesson →
Backup, recovery, and durability	Recovery needs tested backups, logs, keys, compatible applications, and an explicit acknowledged-data policy. Read the lesson →
Backups and restore testing	Usable backups require independent storage, keys, compatible state, and tested application restoration. Read the lesson →
Batch vs streaming pipelines	Batch works on bounded input; streaming maintains continuing progress, with different freshness and recovery costs. Read the lesson →
Benchmark methodology and statistical pitfalls	Benchmark validity depends on realistic input, sufficient observations, honest failures, and a generator that represents arrivals correctly. Read the lesson →
Binary/object storage	Large binary objects and mutable workflow metadata need stable identities and an explicit lifecycle handoff. Read the lesson →
Blameless postmortems	Blameless postmortems reconstruct impact and conditions, with specific owned improvements and verification evidence. Read the lesson →

Blocking vs nonblocking I/O	Nonblocking I/O lets a thread perform other ready work during waits, using supported readiness or completion mechanisms. Read the lesson →
Bottleneck identification	Bottleneck diagnosis connects useful outcomes and critical-path waits to constrained resources. Read the lesson →
Boundary	A line that separates the parts being considered from their surroundings. A diagram should explain whether it marks a system, process, machine, or trust boundary. Read the lesson →
Bounded contexts and ubiquitous language	A bounded context defines a model's valid scope; ubiquitous language makes its concepts consistent within that scope. Read the lesson →
Bridge	Uses composition to let two dimensions of variation evolve independently. Read the lesson →
Browser, CDN, application, and database caches	Browser, edge, application, and database caches hold different copies with independent freshness and trust requirements. Read the lesson →
Builder	Collects construction choices and validates them before producing a usable value. Read the lesson →
Bulkheads	Bulkheads partition relevant resources so one workload cannot consume another's entire budget. Read the lesson →
Business logic vs orchestration vs infrastructure	Business logic decides outcomes, orchestration sequences work, and infrastructure performs technical effects. Read the lesson →

Cache consistency and invalidation	Invalidation coordinates copies with source changes; stronger freshness can require versions or additional coordination. Read the lesson →
Cache locality and the working set	Locality and the active working set determine which data can usefully fit and remain in a cache. Read the lesson →
Cache partitioning and replication	Cache partitions and replicas need failure, remapping, fallback, and rebuild-capacity policies. Read the lesson →
Cache stampedes, penetration, and avalanche	Stampedes, absent-key traffic, and broad expiry need controlled loading and origin protection. Read the lesson →
Cache warming and prefetching	Warming and prefetching spend speculative work that should be justified by actual future reuse. Read the lesson →
Cache-aside, read-through, write-through, write-behind	Cache population and write patterns specify ordering and failure gaps; their names alone do not remove races. Read the lesson →
Cancellation and structured concurrency	Structured concurrency gives child work a scope owner; cancellation requires cleanup and cannot automatically undo remote effects. Read the lesson →
CAP	Under a network partition, the theorem rules out simultaneously guaranteeing its formal consistency and availability properties. It is not a general “pick any two” menu. Read the lesson →

CAP theorem, with its actual assumptions	CAP concerns the incompatibility of defined atomic consistency and availability guarantees under its partition assumptions. Read the lesson →
Capacity and growth planning	Capacity planning connects growth scenarios and resource limits to the time and evidence needed for expansion. Read the lesson →
Capacity monitoring	Capacity monitoring tracks meaningful approaching limits, growth uncertainty, and the time available to respond. Read the lesson →
Capacity reservation vs elastic capacity	Reserved and elastic capacity trade commitment, idle risk, startup delay, and commercial terms against measured demand. Read the lesson →
Cascading failures	Cascading failures spread through resource pressure and retries, including potentially overloaded fallback paths. Read the lesson →
CDN and edge networking	A CDN caches permitted representations near users, with keys, freshness, and origin recovery requirements. Read the lesson →
CDN architecture and origin shielding	CDN edges and shields distribute suitable representations under explicit key, capability, freshness, and purge rules. Read the lesson →
Chain of Responsibility	Passes a request through ordered handlers with explicit continuation, rejection, and handling rules. Read the lesson →

Change Data Capture (CDC)	CDC transports committed source changes through a supported handoff and ordering mechanism. Read the lesson →
Chaos engineering	Chaos exercises test explicit hypotheses with bounded scope, observations, stop conditions, and recovery ownership. Read the lesson →
Checksums and content addressing	Checksums detect corruption under algorithm assumptions; content addressing identifies bytes without establishing permission or safety. Read the lesson →
Choosing storage based on access patterns	Storage selection follows actual queries and invariants, including the complete cost of another engine. Read the lesson →
Choreography vs orchestration	Choreography distributes event reactions; orchestration gives workflow direction and progress an explicit owner. Read the lesson →
CI/CD	CI, delivery, and deployment preserve artifact identity and run checks appropriate to actual release risks. Read the lesson →
Circuit breakers	Circuit breakers limit futile calls and probe recovery without repairing dependencies or inventing valid outcomes. Read the lesson →
Clean Architecture	Clean Architecture directs source dependencies toward policy through application-owned abstractions. Read the lesson →

Cloud vs on-premises trade-offs	Cloud/on-premises comparisons need equal workload, ownership, procurement, recovery, and verified cost assumptions. Read the lesson →
Cohesion	How closely the work inside a module belongs together. Read the lesson →
Command	Represents a requested action as explicit data and identity for an appropriate handler. Read the lesson →
Command-query separation	Command-query separation makes state-changing operations distinguishable from reads without requiring separate deployments. Read the lesson →
Component	A part of a system with a defined responsibility. Several components may run inside one process. Read the lesson →
Composite	Lets callers use leaves and groups through a compatible interface, with explicit traversal and partial-failure rules. Read the lesson →
Composition root	The startup code that creates the objects an application needs and passes them to the parts that use them. Read the lesson →
Composition roots and DI containers	A composition root assembles collaborators; a DI container automates selected construction and lifetime rules. Read the lesson →
Composition vs inheritance	Composition combines collaborators, while inheritance requires subclasses to preserve their callers' behavioural expectations. Read the lesson →

Compute, storage, networking, and telemetry costs	Total cost follows compute, requests, data, copies, transfer, telemetry, and operating work under explicit rates. Read the lesson →
Concurrency	Multiple activities in progress with potentially interleaved execution; parallel execution is not required. Read the lesson →
Concurrency vs parallelism	Concurrency permits overlapping progress; parallelism requires simultaneous execution resources. Read the lesson →
Configuration and secret delivery	Configuration and secret delivery need validation, versions, rollout/reload semantics, and a recovery path. Read the lesson →
Conflict resolution and CRDT introduction	CRDTs converge through specified update/merge rules but do not enforce arbitrary business invariants. Read the lesson →
Connection establishment and connection reuse	Reusing a connection can avoid setup work while introducing lifetime, endpoint freshness, and replacement decisions. Read the lesson →
Connection pool saturation	Pool saturation includes waiting and held connections; aggregate demand across application replicas must remain bounded. Read the lesson →
Connection pooling and keepalive	A pool bounds and reuses connections; acquisition waiting and resource hold time belong in capacity and deadline reasoning. Read the lesson →

Consensus and replicated state machines	Replicated state machines apply agreed commands deterministically; external effects remain a separate boundary. Read the lesson →
Consistency model	A contract specifying which observations of shared or replicated state are allowed. Read the lesson →
Consistency models	A consistency model defines permitted observations across operations, sessions, and state copies. Read the lesson →
Consistent hashing and rendezvous hashing	Consistent and rendezvous hashing limit some assignment changes without guaranteeing even traffic or moving the data itself. Read the lesson →
Constructor, property/setter, and method injection	Constructor, setter, and method injection provide dependencies at different points with different validity and lifetime implications. Read the lesson →
Consumer acknowledgements	Consumer acknowledgment advances broker progress, with loss or repetition gaps around separately committed effects. Read the lesson →
Consumer-driven contracts	Consumer-driven contracts protect meaningful assumptions of actual clients without freezing incidental formatting. Read the lesson →
Container (C4)	An application or data store in a C4 diagram. It need not be a Docker or other operating-system container. Read the lesson →

Container fundamentals	Containers package execution while runtime controls, persistent-state placement, identity, and shutdown still need design. Read the lesson →
Contract	The rules for how an operation behaves, including valid inputs, results, state changes, and errors. Read the lesson →
Control plane vs data plane	Control planes decide configuration and placement; data planes serve traffic with different startup and failure dependencies. Read the lesson →
Controllers, services, repositories, adapters	Controllers, services, repositories, and adapters are responsibilities, not mandatory class-name conventions. Read the lesson →
Cost per request or transaction	Cost per useful outcome avoids denominators inflated by failed or repeated attempts. Read the lesson →
Cost, security, operational complexity, and maintainability as constraints	Feasible designs must satisfy mandatory security, cost, and operating constraints before preferences are ranked. Read the lesson →
Cost-performance optimization	Cost-performance optimisation compares feasible equivalent service outcomes and checks downstream effects. Read the lesson →
Coupling	The dependencies between parts of a system: what one part must know about or rely on in another. Read the lesson →

CPU, memory, disk, network, and database bottlenecks	Bottlenecks can be computation or waiting at CPU, memory, storage, network, locks, or data services. Read the lesson →
CPU-bound vs I/O-bound workloads	CPU-bound work is limited by computation, while I/O-bound work spends important time waiting on external operations. Read the lesson →
CQRS	Uses different command and query models where their needs justify extra projection and freshness work. Read the lesson →
Cursor vs offset pagination	Keyset cursors continue after ordered keys; offsets skip positions that can shift as data changes. Read the lesson →
Dashboards	Dashboards organise evidence for decisions with clear units, denominators, windows, gaps, and ownership. Read the lesson →
Data consistency validation	Data consistency checks compare identities, versions, relationships, and invariants beyond equal counts. Read the lesson →
Data lakes and warehouses	Lakes and warehouses organise raw and curated analytics, requiring ownership, catalog, quality, and lifecycle rules. Read the lesson →
Data minimization	Data minimisation retains information justified by a stated purpose and lifecycle. Read the lesson →

Data ownership	Data ownership includes who changes a fact and who maintains its access, schema, retention, and recovery. Read the lesson →
Data provenance and lineage	Provenance identifies origins; lineage links transformations so outputs can be investigated and corrected. Read the lesson →
Data quality, schema validation, and replay	Quality checks validate meaning as well as shape, while replay needs stable versions and safe effect handling. Read the lesson →
Data retention and lifecycle	Retention follows raw and derived copies through expiry, archival, deletion, replay, and recovery. Read the lesson →
Database connection pooling	Database pools reuse bounded connections, with acquisition, hold time, and downstream contention included in deadlines. Read the lesson →
Database proxying	Database proxies add routing or pooling facilities whose session-state and retry contracts need verification. Read the lesson →
Deadlocks, starvation, livelocks	Deadlock blocks progress through dependencies; starvation denies service; livelock keeps work active without useful progress. Read the lesson →
Decision matrices and trade-off analysis	A decision matrix makes weighted judgments explicit; it does not replace correctness evidence or measurements. Read the lesson →

Decorator	Adds behaviour around a collaborator while retaining its interface, with composition order affecting meaning. Read the lesson →
Defense in depth	Defense in depth combines complementary controls rather than repeating one unverified assumption at several layers. Read the lesson →
Delivery guarantees	Delivery guarantees need a stated record or effect boundary and do not automatically compose across external systems. Read the lesson →
Dependency cycles and captive dependencies	Cycles can signal unclear responsibilities; captive dependencies retain collaborators beyond their valid shorter lifetime. Read the lesson →
Dependency injection	Supplying collaborators to an object or function rather than having it locate or construct them internally. Read the lesson →
Dependency Injection (DI)	Dependency Injection supplies collaborators through construction or calls instead of hiding their creation or lookup. Read the lesson →
Dependency inversion principle vs DI	Dependency inversion concerns knowledge direction; DI concerns supplying collaborators. Either term alone does not establish the other. Read the lesson →
Dependency rules and module boundaries	Module dependency rules limit knowledge and bypass paths, using review and useful enforcement where appropriate. Read the lesson →

Dependency, dependency direction, and inversion	Dependency inversion lets important policy rely on suitable abstractions rather than changeable implementation details. Read the lesson →
Deployment topologies and networking	Deployment topology places execution, state, routes, trust, and failure domains on real infrastructure. Read the lesson →
Designing data-intensive applications	Data-intensive design connects versioned transformations with explicit authority, validation, progress, and recovery. Read the lesson →
Development vs staging vs production	Environments have different purposes while relevant protocols, state transitions, and configuration meanings need deliberate compatibility. Read the lesson →
DI vs factory pattern	A factory chooses or creates a collaborator; injection supplies it or the factory, with ownership still specified. Read the lesson →
DI vs service locator	A service locator hides dependencies behind runtime lookup; DI can make them explicit at the appropriate boundary. Read the lesson →
Dirty reads, nonrepeatable reads, phantom reads	Dirty, nonrepeatable, and phantom reads describe different concurrent histories whose occurrence depends on isolation implementation. Read the lesson →
Disposal, cleanup, and resource lifecycles	Cleanup must release owned resources on success, failure, and cancellation under the relevant contract. Read the lesson →

Distributed coordination	Coordination establishes shared decisions under an explicit failure and progress protocol. Read the lesson →
Distributed locks and leases	Locks and leases grant permission with identity and expiry rules, while resumed stale effects may still need fencing. Read the lesson →
Distributed monolith antipattern	A distributed monolith has multiple processes without useful independent change or operation. Read the lesson →
Distributed tracing	Tracing relates spans while sampled executions still need comparison with wider timing and error evidence. Read the lesson →
DNS resolution and DNS caching	DNS resolves names through records and caches; existing answers and connections delay the effect of a record change. Read the lesson →
Document databases	Document storage groups nested data by identity, with explicit size, schema, index, and cross-document rules. Read the lesson →
Domain events	A domain event records a meaningful accepted fact; durable publication needs a separate explicit mechanism. Read the lesson →
Domain models, entities, value objects	Entities retain identity; value objects represent equal values; domain models express meaningful concepts and rules. Read the lesson →

Domain-Driven Design (DDD)	Domain-Driven Design develops a shared business model with domain experts where rules and meanings need clarification. Read the lesson →
DRY, KISS, YAGNI	DRY avoids duplicated knowledge; KISS favours understandable required design; YAGNI defers unsupported speculative capability. Read the lesson →
Durability	Whether committed data survives the specified failures, such as a process restart or loss of a disk. Read the lesson →
Durable job execution	Durable jobs retain identity, input, state, and attempt authority beyond the initiating process. Read the lesson →
Encapsulation	Controlling access to internal data so callers use the operations that protect its rules. Read the lesson →
Encryption in transit and at rest	Transit and storage encryption protect different boundaries under key/access assumptions and do not replace authorisation. Read the lesson →
ER diagrams and relational modeling	ER modelling identifies keys, relationships, cardinalities, constraints, and record lifecycles. Read the lesson →
Error budget	The allowed unreliability implied by a chosen SLO over its measurement window. Read the lesson →
ETL vs ELT	ETL transforms before target loading; ELT transforms after loading, changing raw-data placement and governance work. Read the lesson →

Evaluating a design against its original requirements	Design validation traces original requirements to tests, measurements, reviews, and clearly unresolved assumptions. Read the lesson →
Event loops, tasks, futures, promises	An event loop runs ready work; futures or promises represent later results under the runtime's rules. Read the lesson →
Event sourcing	Event sourcing makes interpretable accepted events authoritative, with snapshot, correction, privacy, and replay obligations. Read the lesson →
Event-driven architecture	Event-driven components react independently, with delivery, ordering, replay, and progress obligations. Read the lesson →
Eventual consistency	A convergence property under the model’s assumptions; it alone gives neither an immediate freshness bound nor a complete conflict policy. Read the lesson →
Eviction: LRU, LFU, FIFO, and alternatives	LRU, LFU, and FIFO rank eviction differently; admission, bytes, expiry, and approximations also affect behaviour. Read the lesson →
Evolutionary architecture	Evolutionary architecture changes through evidence while checks protect important required properties. Read the lesson →
Exactly-once processing	A scoped guarantee about effects or records within a defined system boundary and assumptions. It does not automatically include arbitrary external side effects. Read the lesson →

Exactly-once processing within defined boundaries	Exactly-once guarantees cover only the supported atomic boundary and its identity, retention, and failure assumptions. Read the lesson →
Facade	Offers a focused entry to cooperating subsystem operations without hiding consequential outcomes. Read the lesson →
Factory Method	Varies a creation operation so processing code can use a product without selecting its concrete implementation. Read the lesson →
Factory Method, Abstract Factory, Builder, Prototype, Singleton	Creational patterns organise different construction choices, families, staged assembly, copying, and bounded instance access. Read the lesson →
Failover and failback	Failover and failback transfer traffic or authority while preserving current history, fencing, and capacity. Read the lesson →
Failure detection	Failure detection produces suspicions under timing assumptions; authority safety must tolerate false suspicion. Read the lesson →
Failure domain	Elements exposed to a common cause of failure, such as a process, host, disk, or region. Read the lesson →
Failure domains and fault isolation	Failure domains identify shared causes; isolation limits their spread through resources and dependencies. Read the lesson →

Fallbacks	Fallbacks substitute defined business outcomes, with their own access, duration, capacity, and recovery risks. Read the lesson →
Fault models	A fault model specifies failures and persistence assumptions a protocol claims to tolerate. Read the lesson →
Fault-injection tests	Fault injection targets meaningful interruptions and validates both useful recovery and required state. Read the lesson →
Feature flags	Feature flags control exposure with ownership and safe defaults, but do not reverse stored effects or replace permission. Read the lesson →
Flyweight	Shares suitable immutable intrinsic state while keeping per-use context separate. Read the lesson →
Functional programming alternatives to object-centric design	Pure functions can express decisions, while an explicit effect layer owns durable execution and external calls. Read the lesson →
Gathering and prioritizing requirements	Requirements state necessary outcomes and constraints; prioritisation identifies what matters most and what still needs evidence. Read the lesson →
Golden signals	Golden signals connect latency, traffic, errors, and saturation using meaningful eligible outcomes. Read the lesson →
Graceful degradation	Graceful degradation preserves a permitted subset of behaviour with visible freshness and action limits. Read the lesson →

Graph databases	Graph storage supports relationship traversal whose depth, degree, cost, and access scope need limits. Read the lesson →
Health checking	Health probes support bounded local decisions rather than establishing the complete business service guarantee. Read the lesson →
Health checks and failover	Readiness, liveness, and failover make different decisions and require state and capacity beyond a probe response. Read the lesson →
Hexagonal/Ports and Adapters	Hexagonal architecture uses application ports and adapters to separate core operations from external mechanisms. Read the lesson →
High cohesion and low coupling	Cohesion concerns related work inside a module; coupling concerns dependencies and knowledge between modules. Read the lesson →
HLD	High-Level Design: a view of a system's main parts, where data is kept, and how the parts work together. Read the lesson →
Horizontal scaling economics	Horizontal scaling adds baseline and coordination costs whose usefulness depends on divisible work and shared limits. Read the lesson →
Hot keys	A hot key concentrates work on an owner and needs a remedy suited to its actual read or write pressure. Read the lesson →

Hot partitions and skew	Hot partitions require diagnosis of popularity, size, and ownership pressure rather than more unrelated capacity.	Read the lesson →
How databases store and retrieve records	A record lookup performs physical and visibility work through indexes, pages, memory, and storage.	Read the lesson →
How the internet works from the application perspective	A page request can include name resolution, transport setup, security, server work, resource transfer, and rendering.	Read the lesson →
HTTP/1.1, HTTP/2, HTTP/3	HTTP versions carry common application semantics using different connection and stream mechanisms.	Read the lesson →
Idempotency	Repeated application of an operation has the same relevant effect as one application, under its defined identity and scope.	Read the lesson →
Idempotency and idempotency keys	Idempotency keys bind repeated attempts to one logical operation through scoped durable identity and result rules.	Read the lesson →
Idempotent consumers	Idempotent consumers commit recognition and the protected local effect together, retaining identity for supported replay.	Read the lesson →
Immutability and side-effect management	Immutable values and explicit effects make changes easier to follow, with allocation and execution costs still considered.	Read the lesson →

Immutable infrastructure	Immutable infrastructure replaces defined instances, while runtime state, migrations, and emergency drift remain managed concerns. Read the lesson →
In-process vs distributed caches	In-process caches duplicate local copies; distributed caches add a shared network boundary and its failure behaviour. Read the lesson →
Incident analysis and prevention	Incident analysis links contributing mechanisms to verified corrective changes without substituting blame for explanation. Read the lesson →
Incident response	Incident response coordinates mitigation, decisions, communication, and remaining uncertain business outcomes. Read the lesson →
Index selection and query optimization	Indexes improve selected reads while adding space, maintenance, and write costs. Read the lesson →
Indexing pipelines	Indexing stages need identified inputs, repeat-safe output, validated completion, and an owner for failed progress. Read the lesson →
Infrastructure as Code	Infrastructure as Code makes desired resources reviewable without eliminating state, privileges, drift, or destructive-change risks. Read the lesson →
Infrastructure ownership and managed services	Managed services transfer selected duties while clients retain responsibilities defined by their actual contract. Read the lesson →

Interface	The operations a component offers to its callers. Its contract explains how those operations behave. Read the lesson →
Interfaces and contracts	A contract specifies valid inputs, outcomes, effects, errors, and relevant timing or repetition rules beyond a signature. Read the lesson →
Invariant	A rule that must hold whenever the system is in a valid state, such as unique task identifiers. Read the lesson →
Inversion of Control (IoC)	Inversion of Control delegates some calling or construction decisions to an external coordinator or framework. Read the lesson →
IP addresses, ports, sockets, routing	Addresses, ports, sockets, and routing connect application endpoints through actual network paths. Read the lesson →
Isolation levels and implementation differences	Isolation behaviour depends on engine, version, and statement form; retries require fresh whole-transaction reasoning. Read the lesson →
Iterator	Exposes traversal without revealing collection representation, while retaining ordering and resource-lifetime obligations. Read the lesson →
Key management and rotation	Key management tracks data dependencies through creation, rotation, historical recovery, and destruction. Read the lesson →

Key-value databases	Key-value storage serves keyed operations; other queries and expiry need their own supported structures and policy. Read the lesson →
Kubernetes fundamentals	Kubernetes reconciles workload resources; application transactions, access, backups, and service objectives remain explicit responsibilities. Read the lesson →
L4 vs L7 load balancing	L4 routing uses transport information; L7 routing can inspect application messages where encryption and policy permit it. Read the lesson →
Lamport clocks and vector clocks	Lamport clocks preserve causal order in one direction; vector comparisons can identify concurrency under their model. Read the lesson →
Large-file upload architecture	Direct large-file upload reduces application byte handling while retaining permission, quota, and completion responsibilities. Read the lesson →
Latency	Elapsed time for an operation under a specified measurement boundary. Read the lesson →
Latency caused by network round trips	Each required network exchange adds travel and waiting to the operation's end-to-end path. Read the lesson →
Latency, throughput, QPS, RPS, TPS	Latency is elapsed time per operation; throughput is completed work per interval. Define which queries, requests, or transactions are counted. Read the lesson →

Law of Demeter	The Law of Demeter discourages unnecessary dependence on distant objects' internal structure. Read the lesson →
Layered/N-tier architecture	Layered architecture separates kinds of responsibility; physical tiers are a separate deployment choice. Read the lesson →
Lazy initialization	Lazy initialisation defers work to first use, requiring concurrency, publication, failure, and latency decisions. Read the lesson →
Leader/follower replication	Leader/follower replication orders updates, while receipt, persistence, commitment, and application can mark different progress. Read the lesson →
Leaderless replication	Leaderless protocols reconcile several replicas with implementation-specific version, repair, and membership rules. Read the lesson →
Least privilege	Least privilege limits effective identity capabilities to the work and resources actually needed. Read the lesson →
Lifecycle policies and retention	Lifecycle rules follow originals, intermediates, indexes, logs, and recovery copies through required deletion. Read the lesson →
Linearizability	A correctness condition in which operations appear atomic and respect real-time precedence. Read the lesson →

Little's Law and queueing intuition	Little's Law relates stable long-run averages: work in progress equals rate times residence time within one boundary. Read the lesson →
LLD	Low-Level Design: the internal detail of a component, including its functions, classes, interfaces, and algorithms. Read the lesson →
Load balancers vs API gateways	Balancers select backends; gateways commonly add API boundary policy. Products can combine those responsibilities. Read the lesson →
Load distribution algorithms	Balancing algorithms distribute selections using different load clues whose usefulness depends on request and connection behaviour. Read the lesson →
Load shedding	Load shedding rejects selected work early to preserve useful higher-priority outcomes under overload. Read the lesson →
Load, stress, spike, and soak testing	Load, stress, spike, and soak tests answer different expected-load, limit, abrupt-change, and long-duration questions. Read the lesson →
Locking and optimistic concurrency control	Locks coordinate conflicting access; optimistic checks validate expected state before applying changes. Read the lesson →
Log ordering and durability	A log entry's presence, durability, commitment, and application are distinct claims during replication and recovery. Read the lesson →

Logs, metrics, and traces	Logs record events, metrics aggregate populations, and traces connect execution work and waits. Read the lesson →
Lost updates, write skew, serialization anomalies	Lost updates overwrite work; write skew can violate a multi-row rule even when separate rows are protected. Read the lesson →
LSM trees and write amplification	LSM organisation batches sorted writes and compaction, trading ingestion behaviour against read work and amplification. Read the lesson →
Mediator	Coordinates collaborators through a focused owner instead of many direct relationships. Read the lesson →
Membership changes	Membership changes need a supported transition preserving decision safety between old and new configurations. Read the lesson →
Memento	Captures and restores supported local state without claiming to reverse independent external effects. Read the lesson →
Memory visibility and happens-before relationships	Happens-before describes memory-model ordering and visibility; earlier wall-clock time alone does not establish it. Read the lesson →
Memory, stack, heap, and object lifetimes	Stack, heap, and references have runtime-specific lifetimes; external resources also need explicit timely release. Read the lesson →

Metadata vs binary data

Immutable source versions keep changing annotations and processing attempts traceable to the bytes they used.

Read the lesson →

Modular monoliths

A modular monolith has one release unit with enforced internal contracts and data ownership.

Read the lesson →

Modularity

Organising a system into parts with clear responsibilities and interfaces.

Read the lesson →

Monitoring vs observability

Monitoring follows known signals; observability helps explain behaviour through correlated outputs and instrumentation.

Read the lesson →

Monolith vs microservices

Microservices seek useful independent deployment and operation while adding network, version, and coordination work.

Read the lesson →

Multi-environment configuration

Environment values should be validated and explicit without silently changing business meanings.

Read the lesson →

Multi-leader replication

Multiple write leaders need valid conflict semantics, not just deterministic convergence that discards business meaning.

Read the lesson →

Multi-region replication trade-offs

Regional replication needs authority, acknowledgment geography, conflict, failover, and safe return-to-service rules.

Read the lesson →

Multi-region strategies	Multi-region operation needs write authority, read freshness, loss, latency, and rejoin policies. Read the lesson →
Multi-tenant isolation	Tenant isolation must hold across queries, caches, jobs, objects, search, exports, telemetry, and administration. Read the lesson →
Multi-zone and multi-region deployments	Zone and region placement address different correlations while adding propagation, cost, and recovery dependencies. Read the lesson →
Multipart and resumable uploads	Resumable or multipart upload tracks confirmed pieces and separately verifies the completed object's identity and integrity. Read the lesson →
Mutexes, semaphores, atomics, condition variables	Mutexes, semaphores, atomics, and condition variables coordinate different kinds of access, permits, operations, and waiting. Read the lesson →
MVCC	Multiversion concurrency control: retaining versions to help coordinate concurrent database access. Exact visibility and isolation depend on implementation. Read the lesson →
NAT, firewalls, ingress, egress	NAT changes addressing, firewalls enforce traffic rules, and ingress/egress describe traffic entering or leaving a boundary. Read the lesson →
Negative caching	Negative caching stores a scoped absence or failure meaning for a deliberate lifetime. Read the lesson →

Network partitions, timeouts, packet loss	A timeout ends the caller's wait without establishing whether the remote operation executed. Read the lesson →
Network segmentation	Segmentation restricts network reachability while application identity and resource permissions remain necessary. Read the lesson →
Network uncertainty	Network delay and failure can appear alike; timeouts bound waiting but cannot prove a remote process stopped. Read the lesson →
Nonfunctional requirement	A requirement about the quality or conditions of a system’s behaviour, such as response time or persistence. Read the lesson →
Normalization vs denormalization	Normalisation reduces repeated facts; deliberate copies require a clear historical or maintained-current meaning. Read the lesson →
OAuth 2.0 and OpenID Connect	OAuth delegates access; OpenID Connect adds identity assertions with flow-specific validation. Read the lesson →
Object collaboration	Object collaboration describes requests, results, and ownership between components, including partial-failure handling. Read the lesson →
Object creation vs object ownership	Creating or holding a resource does not determine who owns its disposal. Read the lesson →
Object storage vs block vs file storage	Object, block, and file interfaces expose different byte-access and lifecycle contracts. Read the lesson →

Observability across asynchronous workflows	Asynchronous observability follows accepted work through later attempts, stage waits, and required final outcomes. Read the lesson →
Observer	Notifies subscribed listeners under defined timing, lifetime, ordering, and failure rules. Read the lesson →
OLTP vs OLAP	OLTP serves short operational transactions; OLAP serves analytical scans and aggregation, often with different interference costs. Read the lesson →
On-call and runbooks	On-call needs useful alerts, access, escalation, rehearsed runbooks, and service ownership that reduces recurring toil. Read the lesson →
Onion Architecture	Onion Architecture organises inward policy dependencies; concrete contracts matter more than diagram shape. Read the lesson →
Ordering guarantees	Publication, delivery, execution, and effect order can differ when work proceeds concurrently. Read the lesson →
PACELC as a useful extension	PACELC highlights partition choices and ordinary latency/consistency costs as a design lens, not a complete protocol. Read the lesson →
Pages, records, indexes, and buffer pools	Pages organise records; buffer pools cache pages, while query locality determines which work can be avoided. Read the lesson →

Pagination, filtering, sorting	Pagination bounds query and response work using defined order, filtering, size, and access rules. Read the lesson →
Partial failures	Partial failure permits one participant's effect to succeed while another fails to observe or record it. Read the lesson →
Partitioning and sharding	Sharding distributes partitions among owners, adding routing, query, transaction, and movement requirements. Read the lesson →
Partitions, consumer groups, offsets	Partitions provide ordered lanes; consumer groups and offsets track ownership and progress within those lanes. Read the lesson →
Password hashing	Password hashing uses a purpose-built salted work mechanism with maintained implementations and deployment-appropriate parameters. Read the lesson →
Paxos: conceptual foundations	Paxos preserves chosen decisions through numbered proposals and required prior evidence across intersecting majorities. Read the lesson →
Peak vs average load	Peak load and its duration can differ substantially from an evenly distributed daily average. Read the lesson →
Performance budgets	Performance budgets allocate acceptable end-to-end costs under a workload and remain targets until measured. Read the lesson →

Performance tuning vs architecture changes	Tuning changes implementation costs; architecture changes alter major ownership or placement and need proportionate evidence. Read the lesson →
Physical vs logical clocks	Physical clocks estimate time; logical clocks represent ordering relationships without necessarily measuring seconds. Read the lesson →
Platform engineering basics	Platform engineering offers reusable operating capabilities whose limits and ownership remain visible to developer users. Read the lesson →
Poison messages	Poison messages need diagnosis and controlled quarantine rather than unlimited identical retries. Read the lesson →
Polyglot persistence and its operational costs	Polyglot persistence combines engines for useful capabilities while adding operating and cross-store coordination work. Read the lesson →
Primary/replica architectures	Primary/replica topology needs explicit write authority, acknowledgment, promotion, and stale-owner protection. Read the lesson →
Privacy, retention, and deletion	Privacy lifecycle spans collection, access, correction, retention, deletion, and historical recovery of copies. Read the lesson →
Producers, consumers, brokers	Producers, brokers, and consumers acknowledge different stages that must be named to reason about ownership. Read the lesson →

Profiling	Profiling locates runtime work or waits under measured conditions, which must be related to user outcomes. Read the lesson →
Profiling and benchmarking	Profiles locate runtime work; benchmarks compare controlled conditions with disclosed workloads, versions, variation, and outcomes. Read the lesson →
Program vs process vs thread	A program is code; a process is an executing instance; threads are execution paths sharing process resources under runtime rules. Read the lesson →
Prototype	Creates an object by copying configured state under explicit sharing and ownership rules. Read the lesson →
Proxy	Represents a target interface while controlling access, creation, or location with explicit cost and failure behaviour. Read the lesson →
Public vs internal APIs	Internal APIs still cross trust, version, and failure boundaries; public APIs add explicit audience and support commitments. Read the lesson →
Query and application performance diagnosis	Diagnosis separates waiting from execution and tests proposed bottleneck mechanisms against evidence. Read the lesson →
Query planning and EXPLAIN	A query planner chooses physical execution using estimates; compare plans with actual work where safely supported. Read the lesson →

Queue saturation and throughput ceilings	Saturation limits useful completion; added arrivals become waiting or rejection, requiring operating margin. Read the lesson →
Queueing and overload	Queues absorb temporary mismatch but sustained overload requires admission, capacity, or workload changes. Read the lesson →
Queues vs logs vs pub/sub	Queues distribute work, logs retain ordered history, and pub/sub follows its stated subscription and retention contract. Read the lesson →
Quorum	A set of participants whose responses meet a protocol's decision rule. Intersection alone does not establish all read or write correctness properties. Read the lesson →
Quorum-based systems	A quorum is a permitted decision set within a protocol whose members preserve the required evidence. Read the lesson →
Quorums and read/write trade-offs	Quorum intersection is a fixed-set property; version, durability, concurrency, and membership rules determine its use. Read the lesson →
Race conditions and critical sections	A race can make an outcome depend on interleaving; protect the full critical state transition. Read the lesson →
Raft: terms, elections, log replication, commit	Raft combines term-based elections, log freshness, replication, commitment, and membership rules to preserve history. Read the lesson →

Range partitioning vs hash partitioning	Range placement supports ordered locality; hashing spreads keys, with different scan and hot-key trade-offs. Read the lesson →
Rate limiting	Rate limiting applies a scoped pace and burst rule with an explicit state-failure policy. Read the lesson →
Rate limits and quotas	Rate limits control pace and bursts; quotas cap accumulated use under defined scope and failure rules. Read the lesson →
Read replicas and read/write splitting	Read/write splitting must respect observation and transaction needs, not merely classify by SQL verb. Read the lesson →
Read-heavy vs write-heavy systems	A read/write ratio counts work at a specified boundary; one user action can cause many physical operations. Read the lesson →
Read-your-writes and monotonic reads	Read-your-writes and monotonic reads preserve particular session observations across supported routing and recovery paths. Read the lesson →
Rebalancing	Rebalancing transfers current data and authority while preserving concurrent writes, routing, validation, and recovery. Read the lesson →
Reconciliation after partial failure	Reconciliation compares intended and observed outcomes and repairs partial failure through safe identified operations. Read the lesson →

RED and USE methods	RED focuses on serving rate, errors, and duration; USE examines resource utilisation, saturation, and errors. Read the lesson →
Redis-like data structures	Redis-like data structures offer specific operations; local command atomicity does not automatically cover a larger workflow. Read the lesson →
Redundancy	Redundancy requires independent failure exposure, valid state, usable takeover, and enough surviving capacity. Read the lesson →
Refactoring and recognizing code smells	Refactoring improves structure while preserving the intended observable behaviour. Read the lesson →
Referential integrity	Referential integrity protects valid relationships using mechanisms appropriate to the ownership and transaction boundary. Read the lesson →
Regression testing	Regression tests preserve important behaviour while allowing deliberate contract changes and tolerating measured noise appropriately. Read the lesson →
Relational databases and SQL	Relational storage models facts with keys and constraints; SQL expresses operations whose physical cost needs investigation. Read the lesson →
Replication	Replication maintains copies under explicit acknowledgment, authority, lag, and recovery rules. Read the lesson →

Replication lag	Replica receipt, persistence, and apply progress can differ and support different freshness or recovery claims. Read the lesson →
Repository	Offers application-oriented persistence operations through a contract defining transactions, bounds, errors, and lifetimes. Read the lesson →
Repository, Unit of Work, Specification, CQRS, and dependency boundaries	Repository, Unit of Work, Specification, and CQRS clarify data access, commit ownership, predicates, and distinct read/write models. Read the lesson →
Request scopes, session scopes, and custom scopes	Scopes define resource lifetimes for requests, sessions, or jobs; their meanings need explicit ownership and cleanup. Read the lesson →
Request/response schemas	Schemas describe structure; validation also checks business meaning, size limits, and safe error behaviour. Read the lesson →
Resource exhaustion and backpressure	Backpressure limits upstream work according to available capacity; larger queues do not repair sustained overload. Read the lesson →
Resource modeling	Resource modelling gives business objects and operations stable identities and meaningful lifecycles. Read the lesson →
REST, GraphQL, gRPC, RPC	REST, GraphQL, and RPC styles organise interaction differently; none alone supplies safe retries or compatibility. Read the lesson →

Retries and dead-letter queues	Retry and dead-letter policies need bounds, useful error context, ownership, and a safe repair/replay path. Read the lesson →
Retries and timeouts	Retries spend bounded time and capacity and need safe repetition, particularly for uncertain effects. Read the lesson →
Retries, exponential backoff, jitter	Retries spend bounded capacity and time, with backoff, jitter, classification, and safe effect identity. Read the lesson →
Retry storms	Retry storms amplify failure traffic; timing randomisation alone does not bound total attempts. Read the lesson →
Reverse proxies vs forward proxies	A forward proxy acts for clients; a reverse proxy fronts servers, with explicit forwarding and trust rules. Read the lesson →
Role-based and attribute-based access control	RBAC assigns scoped role permissions; ABAC evaluates trusted attributes, with enforcement and testing required for either. Read the lesson →
Rollbacks and database migrations	Rollback depends on compatible state; expand-and-contract maintains a supported overlap before removing old structures. Read the lesson →
Rolling, blue-green, and canary deployments	Rolling, blue-green, and canary exposure need representative verification, compatible state, and sufficient transition capacity. Read the lesson →

Root-cause investigation	Cause investigation distinguishes triggers from contributing mechanisms and tests explanations beyond correlation. Read the lesson →
Row-oriented vs column-oriented storage	Row and column layouts group physical values differently to support different read and update patterns. Read the lesson →
RPO	Recovery Point Objective: an objective for the amount of data loss measured in time. Read the lesson →
RTO	Recovery Time Objective: an objective for restoring service after disruption. Read the lesson →
RTO and RPO	RTO targets recovery time; RPO targets acceptable data-loss exposure in time. Read the lesson →
Safety vs liveness	Safety excludes bad outcomes; liveness requires progress under specified environmental conditions. Read the lesson →
Saga	A coordinated sequence of local transactions with compensating actions; compensation need not be a perfect inverse. Read the lesson →
Sagas and compensating transactions	Sagas coordinate local steps and supported compensations with durable progress and explicit uncertainty. Read the lesson →
Sampling and telemetry cost	Sampling and retention balance diagnostic questions, privacy, cardinality, and telemetry capacity. Read the lesson →

Scale-out economics	Scale economics includes surviving capacity, coordination, transfer, baseline resources, and operating effort per useful outcome. Read the lesson →
Scaling writes vs scaling reads	Read scaling can reuse copies; write scaling must divide or reduce ownership work while preserving invariants. Read the lesson →
Scheduling and context switching	Scheduling assigns CPU time; context switches and excessive runnable work can add overhead and waiting. Read the lesson →
Scheduling and service discovery	Scheduling selects eligible resources; discovery advertises endpoints whose readiness and shared-state behaviour still need handling. Read the lesson →
Scheduling, leases, and worker coordination	Scheduling separates stable due occurrences from attempts, with atomic creation, time, overlap, and stale-worker rules. Read the lesson →
Schema evolution	Schema evolution must preserve supported interpretation of stored events, historical data, and delayed readers. Read the lesson →
Search engines and inverted indexes	An inverted index maps analysed terms to document postings, with source and access metadata for safe serving. Read the lesson →
Search indexes	Search indexes are retrieval representations with source-version, freshness, access, and rebuild obligations. Read the lesson →

Search ingestion and reindexing	Reindexing builds and validates a new generation while catching up edits and deletes before serving switches. Read the lesson →
Secrets management	Secrets need controlled distribution, ownership, rotation, revocation, and protection from accidental exposure. Read the lesson →
Secure-by-design reviews	Secure reviews follow sensitive operations and changed conditions through every required enforcement boundary. Read the lesson →
Security vs usability trade-offs	Security and usability need a workable safe journey without trading away mandatory isolation or correctness. Read the lesson →
Sequential and causal consistency	Sequential consistency preserves participant program order; causal consistency preserves dependencies while allowing unrelated orders to differ. Read the lesson →
Serializability	A transactional correctness condition requiring equivalence to some serial execution; real-time order is a separate issue. Read the lesson →
Serializability vs linearizability	Serializability concerns transaction equivalence; strict serializability additionally respects real-time constraints. Read the lesson →
Serializable execution and serializability	Serializability concerns serial transaction equivalence; linearizability also concerns operation real-time precedence. Read the lesson →

Serverless and function-oriented architectures	Serverless execution delegates platform work while retaining limits, state, identity, and retry requirements. Read the lesson →
Serverless compute	Serverless delegates selected platform duties while retaining limits, durable state, identity, and safe retries. Read the lesson →
Service discovery	Service discovery maps logical services to endpoints whose readiness and capacity still require handling. Read the lesson →
Sessions, cookies, tokens	Sessions, tokens, and cookies have different state and transport roles with explicit validation and revocation rules. Read the lesson →
Shared memory vs message passing	Shared memory needs access coordination; message passing needs ownership, delivery, and recovery rules. Read the lesson →
Single points of failure	A single point of failure can be any shared dependency defeating the required service or recovery. Read the lesson →
Singleton	Restricts instance access within a specified program or container boundary, without implying global uniqueness or safe mutation. Read the lesson →
SLI / SLO / SLA	Indicator: measured behavior. Objective: target for that behavior. Agreement: an explicit service agreement, potentially with consequences. Read the lesson →

SLI, SLO, SLA, and error budgets	An SLI measures behaviour, an SLO targets it over a window, and an SLA states agreement obligations. Read the lesson →
SLIs, SLOs, error budgets	SLIs measure behaviour; SLOs target it; error-budget burn compares observed shortfall with its allowance. Read the lesson →
SLO-based reliability decisions	SLO evidence guides reliability investment without hiding critical low-volume harm inside a broad denominator. Read the lesson →
SOLID principles, including their limitations	SOLID offers guidelines for responsibilities, useful extension, substitution, focused interfaces, and dependency direction. Read the lesson →
Specification	Names a reusable eligibility or selection predicate whose later state-changing use may need atomic revalidation. Read the lesson →
Split-brain prevention and fencing tokens	Fencing epochs reject stale authority only when receivers enforce them atomically with protected effects. Read the lesson →
Split-brain situations	Split brain creates conflicting authorities; stale actors need prevention or enforcement at the protected effect boundary. Read the lesson →
Stale-while-revalidate	Stale-while-revalidate serves an allowed older value while refresh proceeds, with a limit on unusable age. Read the lesson →

State	Organises behaviour around a lifecycle whose guarded transitions follow entity history. Read the lesson →
Stateless vs stateful services	Stateless instances keep no required per-client conversation only locally, while using appropriate external state owners. Read the lesson →
Status codes and error models	Protocol status and business state can differ: a created resource may still represent pending work. Read the lesson →
Sticky sessions	Sticky sessions preserve affinity while adding load-skew and recovery consequences for locally held state. Read the lesson →
Storage, memory, bandwidth, and CPU estimates	Estimate requests, bytes, memory, CPU, and transfer separately using explicit workload assumptions. Read the lesson →
Strategy	Supplies an interchangeable algorithm or policy behind a stable behavioural contract. Read the lesson →
Strategy, Observer, Command, State, Chain of Responsibility, Template Method, Iterator, Mediator, Memento, Visitor	Behavioural patterns organise algorithms, notifications, intentions, state transitions, traversal, and coordination at chosen extension points. Read the lesson →
Stream processing fundamentals	Stream processing needs keyed state, event/processing time, lateness, corrections, and recoverable progress. Read the lesson →

Streaming APIs and long-running operations	Long-running and streaming APIs need explicit ownership, progress, cancellation, buffering, and recovery contracts. Read the lesson →
Strong consistency and eventual consistency	Eventual convergence needs stated repair assumptions and conflict semantics; “strong” needs a named precise model. Read the lesson →
Strongly consistent metadata services	Consistent metadata services own small control decisions, with explicit consequences when new decisions cannot proceed. Read the lesson →
Structured logging	Structured logs give fields stable meanings, bounded size, safe content, and explicit event boundaries. Read the lesson →
Supply-chain security	Supply-chain controls protect inputs, builds, artifacts, credentials, deployment, and compromised-release response. Read the lesson →
Sustainability and resource efficiency	Resource efficiency removes unnecessary work and transfer while broader sustainability claims require actual supporting evidence. Read the lesson →
Synchronous vs asynchronous communication	Asynchronous submission separates acceptance from completion and needs an owner for the intervening work. Read the lesson →
Synchronous vs asynchronous execution	Asynchronous interfaces separate initiation from completion; their underlying operations can still block. Read the lesson →

Synchronous vs asynchronous replication	Synchronous replication waits for named milestones; asynchronous acknowledgment can precede another durable copy. Read the lesson →
Tail latency analysis	Tail analysis connects timing distributions and representative paths without incorrectly averaging percentile summaries. Read the lesson →
Tail latency and latency budgets	A request needing many results can be delayed by its slowest dependency; budget the complete path. Read the lesson →
TCP vs UDP vs QUIC	TCP supplies a byte stream, UDP datagrams, and QUIC secure streams under their respective transport rules. Read the lesson →
Technical vs business boundaries	Business boundaries group capabilities; technical layers organise mechanisms within or across them. Read the lesson →
Tell, Don't Ask	Tell, Don't Ask favours sending intentions to the owner that can protect the relevant state rule. Read the lesson →
Template Method	Preserves a process skeleton while allowing selected implementation steps to vary. Read the lesson →
Testability and design for change	Testable design gives controlled inputs and visible outcomes; substitutes and real integration tests establish different evidence. Read the lesson →

The fallacies of distributed computing	Distributed-computing fallacies expose hidden assumptions about networks, cost, topology, trust, and administration. Read the lesson →
Thread pools and worker pools	A worker pool bounds active jobs; a bounded queue separately limits waiting jobs and retained payloads. Read the lesson →
Thread safety and immutability	Thread safety states allowed concurrent behaviour; immutable values still need valid publication and surrounding ownership. Read the lesson →
Threat modeling	Threat modelling connects assets, actors, abuse paths, and impact to controls and concrete tests. Read the lesson →
Throughput	Completed work per unit time under a specified workload and success definition. Read the lesson →
Time, clocks, and event ordering	Physical timestamps, elapsed durations, and causal order answer different questions and need suitable clocks or protocols. Read the lesson →
Time-series databases	Time-series data combines measurements, times, and dimensions with explicit cardinality, lateness, and retention handling. Read the lesson →
Timeouts	Timeouts bound waiting; overall deadlines include acquisition and stage waits without proving remote nonexecution. Read the lesson →

TLS and HTTPS	TLS protects and authenticates transport under its trust configuration; application resource authorisation remains necessary. Read the lesson →
Trace IDs, span IDs, context propagation	Trace context links observations across calls or messages and must not be treated as authorisation. Read the lesson →
Trade-off	A choice that accepts one cost or limitation to gain another benefit, judged against the requirements. Read the lesson →
Transaction commit and rollback	Commit accepts a transaction under the engine protocol; rollback abandons uncommitted work and cannot reverse unrelated remote effects. Read the lesson →
Transactional outbox	Recording a state change and publication intent in one local transaction, then relaying the intent; duplicate delivery still requires handling. Read the lesson →
Transactions across multiple operations	Related statements need an explicit common transaction when their invariant requires one all-or-nothing change. Read the lesson →
Transient, scoped, singleton, and framework-specific lifetimes	Transient, scoped, and singleton describe reuse within defined framework boundaries, not universal global identity or safety. Read the lesson →
Trust boundaries	Trust boundaries mark changes in identity, authority, and confidence that require appropriate validation. Read the lesson →

Trust boundary	A point across which identity, privilege, or confidence in input changes. Read the lesson →
TTL and cache expiration	TTL limits intended residence after insertion, not necessarily the source age of the inserted fact. Read the lesson →
Twelve-Factor App principles	Twelve-Factor practices guide repeatable service operation while workload-specific correctness and recovery remain required. Read the lesson →
Unit of Work	Owns related local changes under one commit and cleanup boundary. Read the lesson →
Unit, integration, contract, end-to-end, and performance tests	Test scopes provide different evidence; choose checks by failures they can detect rather than a mandatory count ratio. Read the lesson →
User journeys and workload characteristics	A workload describes the operations users cause, their frequency, payloads, and dependent work. Read the lesson →
Vector indexes: introductory architecture only	Vector indexes retrieve similarity candidates from embeddings; scores establish neither truth nor access permission. Read the lesson →
Vertical vs horizontal scaling	Vertical scaling enlarges a node; horizontal scaling adds nodes, with gains limited by the work that can divide. Read the lesson →
Virtual machines vs containers	VMs virtualise hardware with guest systems; typical containers share a host kernel, with actual isolation and failure scope verified. Read the lesson →

Visitor	Adds operations across a known structure, trading easier new operations for more work when element types change. Read the lesson →
Webhooks and delivery acknowledgments	Webhooks are asynchronous deliveries requiring authenticity, stable event identity, and defined acknowledgment behaviour. Read the lesson →
WebSockets, Server-Sent Events, and polling	Polling repeats requests; SSE streams server events; WebSockets supports two-way messages. Recovery remains an application concern. Read the lesson →
What data storage must guarantee	Storage guarantees state accepted outcomes, observation rules, ownership, retention, and recovery under specified failures. Read the lesson →
What makes a distributed system difficult	Distributed participants observe only parts of an operation, so legal outcomes need explicit message and recovery protocols. Read the lesson →
When caching is harmful	Caching can be harmful when it saves little, weakens scope, hides origin limits, or adds excessive lifecycle work. Read the lesson →
When not to use microservices	Microservice extraction needs concrete benefits and operating capability sufficient to justify its added coordination cost. Read the lesson →
Why caching works	Caching saves repeated work when reuse outweighs access, storage, and maintenance costs under the required semantics. Read the lesson →

Wide-column stores

Wide-column designs commonly combine partition keys with ordered dimensions, requiring bounded partitions and query-driven modelling.

Read the lesson →

Workflow engines

Workflow engines persist coordination under a programming model while external activities still need safe effect contracts.

Read the lesson →

Write-ahead logging

Write-ahead logging preserves recovery information before corresponding data pages, under configured persistence rules.

Read the lesson →

Zero-trust principles

Zero-trust principles avoid implicit permission from location alone and make trust evidence and limits explicit.

Read the lesson →

No terms match that search.

THE VISUAL INDEX

Every diagram has a job

575 original figures: chapter mechanisms, full-design views, and the required unit plates. The separate master map is Figure M.1.

FIG. 1.1.1

The application and storage work together

Include the parts needed to explain the behaviour you are studying.

FIG. 1.1.2

Creating a task changes the stored state

Read the state before and after the operation to see exactly what changed.

FIG. 1.1.3

Save before reporting success

Compare the two versions by restarting the process.

FIG. 1.1.4

Two levels of detail in one design

Check that both views agree about saving a task before reporting success.

FIG. 1.2.1

Shared storage details make changes spread

Follow the arrows to find which parts know about the shared data.

FIG. 1.2.2

Separate responsibilities inside one process

Each kind of work has a clear place in the code.

FIG. 1.2.3

The service uses a repository contract

The dashed arrows show code dependencies. These objects can run in the same process.

FIG. 1.2.4

Each layer works at a different level of detail

Follow one request through the layers to connect the user's action to the work below it.

FIG. 1.3.1

Turn “safe” into a checkable requirement

A specific requirement tells us which test to run.

FIG. 1.3.2

Choose, record, and review a design

Keep the reasons for a decision alongside the decision itself.

FIG. 1.3.3

Check requirements before scoring preferences

An easy-to-run choice is useful only if it meets the requirements.

FIG. 1.3.4

Availability and durability answer different questions

Check both whether requests succeed and whether saved data survives the stated failure.

FIG. 1.4.1

C4: zoom from the system into its code

Select the level that helps your reader answer a particular question.

FIG. 1.4.2

The application and database share one machine

Show physical placement when asking what can fail together.

FIG. 1.4.3

The task is saved, but the reply is lost

Trace the events in order to see why the client can be unsure even when a task exists.

FIG. 1.4.4

Each design change answers a new need

State the requirement that makes each change necessary.

FIG. 2.1.1

Gathering and prioritizing requirements · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.1.2

User journeys and workload characteristics · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.1.3

Read-heavy vs write-heavy systems · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.1.4

State and ownership in the related design

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.2.1

Latency, throughput, QPS, RPS, TPS · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.2.2

A small mean can hide one slow request

Compare calculated magnitudes under the same explicit model; bar length is not a benchmark.

FIG. 2.2.3

Availability, reliability, durability, consistency · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.2.4

SLI, SLO, SLA, and error budgets · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.3.1

Capacity and growth planning · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.3.2

Different workload dimensions have different magnitudes

Compare calculated magnitudes under the same explicit model; bar length is not a benchmark.

FIG. 2.3.3

Peak vs average load · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.3.4

State and ownership in the related design

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.4.1

Little's Law and queueing intuition · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.4.2

Amdahl's Law and bottlenecks · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.4.3

Tail latency and latency budgets · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.4.4

Cost, security, operational complexity, and maintainability as constraints · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.4.5

Decision matrices and trade-off analysis · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.1.1

How the internet works from the application perspective · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.1.2

IP addresses, ports, sockets, routing · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.1.3

TCP vs UDP vs QUIC · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.1.4

State and ownership in the related design

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.2.1

DNS resolution and DNS caching · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.2.2

TLS and HTTPS · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.2.3

HTTP/1.1, HTTP/2, HTTP/3 · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.2.4

Connection establishment and connection reuse · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.2.5

A cold request spends time before server work

Separate stage waits and execution rather than attach one unexplained number to the entire request.

FIG. 3.3.1

Reverse proxies vs forward proxies · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.3.2

Load balancers vs API gateways · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.3.3

L4 vs L7 load balancing · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.3.4

WebSockets, Server-Sent Events, and polling · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.4.1

NAT, firewalls, ingress, egress · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.4.2

Service discovery · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.4.3

CDN and edge networking · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.4.4

Network partitions, timeouts, packet loss · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.4.5

Connection pooling and keepalive · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.1.1

Threads share a heap but have execution-local stacks

Make the internal structure or state relation visible before claiming its behavior.

FIG. 4.1.2

Memory, stack, heap, and object lifetimes · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.1.3

Scheduling and context switching · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.1.4

CPU-bound vs I/O-bound workloads · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.1.5

Synchronous vs asynchronous execution · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.2.1

Blocking vs nonblocking I/O · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.2.2

Concurrency vs parallelism · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.2.3

Event loops, tasks, futures, promises · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.2.4

Thread pools and worker pools · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.3.1

Shared memory vs message passing · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.3.2

Race conditions and critical sections · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.3.3

Mutexes, semaphores, atomics, condition variables · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.3.4

A circular wait prevents both participants progressing

Make the internal structure or state relation visible before claiming its behavior.

FIG. 4.4.1

Memory visibility and happens-before relationships · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.4.2

Thread safety and immutability · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.4.3

Resource exhaustion and backpressure · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.4.4

Cancellation and structured concurrency · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.1.1

Abstraction, encapsulation, polymorphism, inheritance · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.1.2

Composition vs inheritance · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.1.3

Interfaces and contracts · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.1.4

High cohesion and low coupling · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.1.5

SOLID principles, including their limitations · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.2.1

DRY, KISS, YAGNI · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.2.2

Law of Demeter · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.2.3

Tell, Don't Ask · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.2.4

Command-query separation · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.3.1

Immutability and side-effect management · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.3.2

Domain models, entities, value objects · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.3.3

Object collaboration · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.3.4

Testability and design for change · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.4.1

Refactoring and recognizing code smells · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.4.2

Functional programming alternatives to object-centric design · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.4.3

Follow the related complete design

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 5.4.4

State and ownership in the related design

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.1.1

Dependency, dependency direction, and inversion · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.1.2

Inversion of Control (IoC) · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.1.3

Dependency Injection (DI) · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.1.4

Constructor, property/setter, and method injection · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.1.5

Dependency inversion principle vs DI · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.1.6

Composition roots and DI containers · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.2.1

Scope identities make lifetime semantics visible

Make the internal structure or state relation visible before claiming its behavior.

FIG. 6.2.2

Request scopes, session scopes, and custom scopes · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.2.3

Object creation vs object ownership · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.2.4

Disposal, cleanup, and resource lifecycles · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.2.5

Lazy initialization · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.2.6

Dependency cycles and captive dependencies · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.3.1

DI vs service locator · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.3.2

DI vs factory pattern · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.3.3

Follow the related complete design

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.3.4

State and ownership in the related design

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.4.1

Factory Method, Abstract Factory, Builder, Prototype, Singleton · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.4.2

Factory Method · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.4.3

Abstract Factory · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.4.4

Builder · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.4.5

Prototype · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.4.6

Singleton · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.5.1

Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.5.2

Adapter · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.5.3

Bridge · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.5.4

Composite · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.5.5

Decorator · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.5.6

Facade · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.5.7

Flyweight · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.5.8

Proxy · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.6.1

Strategy, Observer, Command, State, Chain of Responsibility, Template Method, Iterator, Mediator, Memento, Visitor · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.6.2

Strategy · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.6.3

Observer · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.6.4

Command · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.6.5

State · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.6.6

Chain of Responsibility · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.6.7

Template Method · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.6.8

Iterator · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.6.9

Mediator · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.6.10

Memento · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.6.11

Visitor · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.7.1

Repository, Unit of Work, Specification, CQRS, and dependency boundaries · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.7.2

Repository · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.7.3

Unit of Work · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.7.4

Specification · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.7.5

CQRS · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 7.1.1

Layered/N-tier architecture · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.1.2

Modular monoliths · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.1.3

Monolith vs microservices · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.1.4

Clean Architecture · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.1.5

Hexagonal/Ports and Adapters · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.1.6

Onion Architecture · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.2.1

Domain-Driven Design (DDD) · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.2.2

Context boundaries protect different business meanings

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 7.2.3

Aggregates, entities, value objects, domain services · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.2.4

Domain events · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.3.1

Dependency rules and module boundaries · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.3.2

Business logic vs orchestration vs infrastructure · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.3.3

Controllers, services, repositories, adapters · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.3.4

Anti-corruption layers · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.3.5

Event-driven architecture · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.4.1

Serverless and function-oriented architectures · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.4.2

Evolutionary architecture · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.4.3

Technical vs business boundaries · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.4.4

Distributed monolith antipattern · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.4.5

When not to use microservices · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.1.1

API fundamentals and interface contracts · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.1.2

REST, GraphQL, gRPC, RPC · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.1.3

Resource modeling · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.1.4

Request/response schemas · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.1.5

Status codes and error models · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.1.6

Pagination, filtering, sorting · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.2.1

Cursor vs offset pagination · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.2.2

API versioning and compatibility · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.2.3

A payment retry follows durable operation identity

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 8.2.4

State and ownership in the related design

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.3.1

Authentication vs authorization at API boundaries · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.3.2

Rate limits and quotas · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.3.3

Retries and timeouts · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.3.4

Streaming APIs and long-running operations · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.3.5

Webhooks and delivery acknowledgments · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.4.1

Schema evolution · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.4.2

Consumer-driven contracts · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.4.3

API gateways and BFF · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.4.4

Public vs internal APIs · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.4.5

API documentation and discoverability · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.1.1

What data storage must guarantee · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.1.2

Relational databases and SQL · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.1.3

Document databases · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.1.4

Key-value databases · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.1.5

Wide-column stores · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.1.6

Graph databases · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.1.7

Time-series databases · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.1.8

Search indexes · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.1.9

Vector indexes: introductory architecture only · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.2.1

Object storage vs block vs file storage · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.2.2

OLTP vs OLAP · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.2.3

Row-oriented vs column-oriented storage · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.2.4

Normalization vs denormalization · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.3.1

An order schema preserves relationships and history

Make the internal structure or state relation visible before claiming its behavior.

FIG. 9.3.2

Referential integrity · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.3.3

Data ownership · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.3.4

Choosing storage based on access patterns · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.4.1

Data retention and lifecycle · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.4.2

Polyglot persistence and its operational costs · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.4.3

Follow the related complete design

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 9.4.4

State and ownership in the related design

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.1.1

How databases store and retrieve records · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.1.2

Pages, records, indexes, and buffer pools · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.1.3

B+tree lookup narrows an ordered key range

Make the internal structure or state relation visible before claiming its behavior.

FIG. 10.1.4

LSM trees and write amplification · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.1.5

Query planning and EXPLAIN · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.1.6

Index selection and query optimization · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.2.1

ACID properties · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.2.2

Transaction commit and rollback · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.2.3

Write-ahead logging · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.2.4

Two snapshots can legitimately see different versions

Make the internal structure or state relation visible before claiming its behavior.

FIG. 10.3.1

Locking and optimistic concurrency control · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.3.2

Dirty reads, nonrepeatable reads, phantom reads · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.3.3

Lost updates, write skew, serialization anomalies · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.3.4

Isolation levels and implementation differences · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.3.5

Serializable execution and serializability · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.4.1

Database connection pooling · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.4.2

Transactions across multiple operations · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.4.3

Backup, recovery, and durability · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.4.4

State and ownership in the related design

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.1.1

Why caching works · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.1.2

Cache locality and the working set · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.1.3

In-process vs distributed caches · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.1.4

Browser, CDN, application, and database caches · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.2.1

Cache-aside, read-through, write-through, write-behind · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.2.2

TTL and cache expiration · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.2.3

A stale cache is a separate state copy

Make the internal structure or state relation visible before claiming its behavior.

FIG. 11.2.4

LRU and FIFO disagree after the same accesses

Make the internal structure or state relation visible before claiming its behavior.

FIG. 11.3.1

Hot keys · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.3.2

Cache stampedes, penetration, and avalanche · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.3.3

Negative caching · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.3.4

Stale-while-revalidate · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.3.5

Cache warming and prefetching · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.4.1

Redis-like data structures · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.4.2

CDN architecture and origin shielding · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.4.3

Cache partitioning and replication · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.4.4

When caching is harmful · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.1.1

Vertical vs horizontal scaling · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.1.2

Stateless vs stateful services · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.1.3

Load distribution algorithms · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.1.4

Health checks and failover · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.1.5

Sticky sessions · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.1.6

Autoscaling metrics · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.2.1

Replication · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.2.2

Partitioning and sharding · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.2.3

Range partitioning vs hash partitioning · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.2.4

A new owner takes only part of the hash space

Make the internal structure or state relation visible before claiming its behavior.

FIG. 12.2.5

Rebalancing · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.2.6

Averages hide a hot partition

Compare calculated magnitudes under the same explicit model; bar length is not a benchmark.

FIG. 12.3.1

Read replicas and read/write splitting · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.3.2

Replication lag · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.3.3

Database proxying · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.3.4

Connection pool saturation · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.3.5

Bottleneck identification · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.4.1

Queueing and overload · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.4.2

Single points of failure · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.4.3

Multi-region strategies · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.4.4

Scaling writes vs scaling reads · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.4.5

Scale-out economics · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.1.1

Binary/object storage · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.1.2

The upload pipeline separates bytes from control

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 13.1.3

Multipart and resumable uploads · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.1.4

Checksums and content addressing · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.1.5

Metadata vs binary data · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.2.1

An inverted index is a term-to-document map

Make the internal structure or state relation visible before claiming its behavior.

FIG. 13.2.2

Indexing pipelines · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.2.3

Search ingestion and reindexing · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.2.4

State and ownership in the related design

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.3.1

Batch vs streaming pipelines · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.3.2

ETL vs ELT · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.3.3

Change Data Capture (CDC) · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.3.4

Data lakes and warehouses · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.3.5

Stream processing fundamentals · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.4.1

Data provenance and lineage · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.4.2

Data quality, schema validation, and replay · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.4.3

Lifecycle policies and retention · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.4.4

Designing data-intensive applications · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.1.1

What makes a distributed system difficult · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.1.2

Partial failures · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.1.3

Network uncertainty · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.1.4

Fault models · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.1.5

Time, clocks, and event ordering · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.1.6

Physical vs logical clocks · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.1.7

Vector comparison exposes concurrent events

Make the internal structure or state relation visible before claiming its behavior.

FIG. 14.2.1

Consistency models · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.2.2

Strong consistency and eventual consistency · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.2.3

Linearizability · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.2.4

Serializability vs linearizability · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.2.5

Sequential and causal consistency · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.2.6

Read-your-writes and monotonic reads · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.3.1

CAP theorem, with its actual assumptions · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.3.2

PACELC as a useful extension · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.3.3

Intersecting quorums still need a protocol

Make the internal structure or state relation visible before claiming its behavior.

FIG. 14.3.4

A partition can leave two live but disconnected sides

Make the internal structure or state relation visible before claiming its behavior.

FIG. 14.4.1

Distributed coordination · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.4.2

The fallacies of distributed computing · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.4.3

Safety vs liveness · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.4.4

State and ownership in the related design

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.1.1

Primary/replica architectures · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.1.2

Leader/follower replication · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.1.3

Multi-leader replication · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.1.4

Leaderless replication · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.1.5

Synchronous vs asynchronous replication · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.1.6

Quorum-based systems · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.2.1

Consensus and replicated state machines · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.2.2

Raft election changes authority under protocol rules

Make the internal structure or state relation visible before claiming its behavior.

FIG. 15.2.3

Paxos: conceptual foundations · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.2.4

Presence in a log is not the same as commitment

Make the internal structure or state relation visible before claiming its behavior.

FIG. 15.2.5

Membership changes · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.3.1

Split-brain prevention and fencing tokens · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.3.2

Failure detection · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.3.3

Strongly consistent metadata services · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.3.4

Distributed locks and leases · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.4.1

Conflict resolution and CRDT introduction · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.4.2

Multi-region replication trade-offs · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.4.3

Follow the related complete design

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 15.4.4

State and ownership in the related design

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.1.1

Synchronous vs asynchronous communication · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.1.2

Queues vs logs vs pub/sub · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.1.3

Producers, consumers, brokers · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.1.4

Partitions, consumer groups, offsets · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.1.5

Delivery guarantees · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.1.6

At-most-once and at-least-once processing · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.1.7

Exactly-once processing within defined boundaries · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.1.8

Ordering guarantees · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.1.9

Consumer acknowledgements · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.2.1

Retries and dead-letter queues · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.2.2

Poison messages · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.2.3

Idempotent consumers · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.2.4

Event sourcing · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.3.1

CQRS · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.3.2

The outbox closes a local publication gap

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 16.3.3

A failed shipment starts tracked compensation

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 16.3.4

State and ownership in the related design

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.4.1

Choreography vs orchestration · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.4.2

Workflow engines · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.4.3

Durable job execution · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.4.4

Scheduling, leases, and worker coordination · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.4.5

Backpressure and replay · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.4.6

Reconciliation after partial failure · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.1.1

Failure domains and fault isolation · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.1.2

Availability vs durability · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.1.3

Redundancy · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.1.4

Active-active vs active-passive · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.2.1

Timeouts · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.2.2

Retries, exponential backoff, jitter · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.2.3

Retry storms · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.2.4

A circuit breaker bounds futile calls

Make the internal structure or state relation visible before claiming its behavior.

FIG. 17.2.5

Bulkheads · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.2.6

Rate limiting · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.3.1

Load shedding · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.3.2

Graceful degradation · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.3.3

Fallbacks · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.3.4

Health checking · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.4.1

Recovery time and recoverable data are different

Make the internal structure or state relation visible before claiming its behavior.

FIG. 17.4.2

Backups and restore testing · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.4.3

Failover and failback · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.4.4

Multi-zone and multi-region deployments · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.4.5

Chaos engineering · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.4.6

SLO-based reliability decisions · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.4.7

Cascading failures · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.4.8

Incident analysis and prevention · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.1.1

Threat modeling · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.1.2

Trust boundaries · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.1.3

Authentication vs authorization · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.1.4

Sessions, cookies, tokens · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.1.5

OAuth 2.0 and OpenID Connect · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.1.6

Role-based and attribute-based access control · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.1.7

Least privilege · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.2.1

Defense in depth · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.2.2

Secrets management · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.2.3

Encryption in transit and at rest · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.2.4

Password hashing · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.2.5

Key management and rotation · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.2.6

Network segmentation · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.2.7

Zero-trust principles · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.3.1

API security and common vulnerabilities · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.3.2

Tenant identity must survive every trust boundary

Make the internal structure or state relation visible before claiming its behavior.

FIG. 18.3.3

Data minimization · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.3.4

Audit trails · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.3.5

Privacy, retention, and deletion · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.4.1

Supply-chain security · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.4.2

Security vs usability trade-offs · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.4.3

Secure-by-design reviews · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.4.4

State and ownership in the related design

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.1.1

Monitoring vs observability · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.1.2

Logs, metrics, and traces · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.1.3

Structured logging · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.1.4

This slow request is dominated by waiting

Separate stage waits and execution rather than attach one unexplained number to the entire request.

FIG. 19.1.5

Trace IDs, span IDs, context propagation · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.1.6

Golden signals · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.2.1

RED and USE methods · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.2.2

SLIs, SLOs, error budgets · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.2.3

Alert design and alert fatigue · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.2.4

Sampling and telemetry cost · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.2.5

Dashboards · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.3.1

Profiling · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.3.2

Query and application performance diagnosis · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.3.3

Incident response · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.3.4

On-call and runbooks · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.3.5

Root-cause investigation · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.4.1

Blameless postmortems · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.4.2

Capacity monitoring · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.4.3

Observability across asynchronous workflows · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.4.4

State and ownership in the related design

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.1.1

Development vs staging vs production · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.1.2

VMs and containers isolate at different layers

Make the internal structure or state relation visible before claiming its behavior.

FIG. 20.1.3

Container fundamentals · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.1.4

Kubernetes fundamentals · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.2.1

Control plane vs data plane · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.2.2

Scheduling and service discovery · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.2.3

Autoscaling · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.2.4

Configuration and secret delivery · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.2.5

Immutable infrastructure · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.3.1

Infrastructure as Code · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.3.2

CI/CD · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.3.3

Canary exposure expands only after verification

Compare calculated magnitudes under the same explicit model; bar length is not a benchmark.

FIG. 20.3.4

Feature flags · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.3.5

Rollbacks and database migrations · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.3.6

Multi-environment configuration · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.4.1

Twelve-Factor App principles · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.4.2

Serverless compute · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.4.3

Cloud vs on-premises trade-offs · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.4.4

Infrastructure ownership and managed services · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.4.5

Deployment topologies and networking · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.4.6

Platform engineering basics · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.1.1

Performance budgets · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.1.2

CPU, memory, disk, network, and database bottlenecks · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.1.3

Profiling and benchmarking · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.1.4

Load, stress, spike, and soak testing · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.1.5

Benchmark methodology and statistical pitfalls · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.1.6

Tail latency analysis · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.2.1

Queue saturation and throughput ceilings · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.2.2

Performance tuning vs architecture changes · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.2.3

Architecture costs are not capacity guarantees

Compare calculated magnitudes under the same explicit model; bar length is not a benchmark.

FIG. 21.2.4

Compute, storage, networking, and telemetry costs · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.2.5

Cost per request or transaction · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.2.6

Capacity reservation vs elastic capacity · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.3.1

Cost-performance optimization · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.3.2

Sustainability and resource efficiency · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.3.3

Unit, integration, contract, end-to-end, and performance tests · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.3.4

Fault-injection tests · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.4.1

Data consistency validation · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.4.2

Architecture fitness functions · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.4.3

Regression testing · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.4.4

Evaluating a design against its original requirements · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.1.1

Data ownership and relationships

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.1.2

Follow the main operation

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.1.3

Follow the failure and its correction

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.1.4

How the design can grow

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.2.1

Data ownership and relationships

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.2.2

Follow the main operation

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.2.3

Follow the failure and its correction

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.2.4

How the design can grow

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.3.1

Data ownership and relationships

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.3.2

Follow the main operation

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.3.3

Follow the failure and its correction

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.3.4

How the design can grow

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.4.1

Data ownership and relationships

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.4.2

Follow the main operation

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.4.3

Follow the failure and its correction

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.4.4

How the design can grow

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.5.1

Data ownership and relationships

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.5.2

Follow the main operation

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.5.3

Follow the failure and its correction

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.5.4

How the design can grow

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.6.1

Data ownership and relationships

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.6.2

Follow the main operation

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.6.3

Follow the failure and its correction

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.6.4

How the design can grow

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.7.1

Data ownership and relationships

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.7.2

Follow the main operation

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.7.3

Follow the failure and its correction

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.7.4

How the design can grow

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.8.1

Data ownership and relationships

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.8.2

Follow the main operation

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.8.3

Follow the failure and its correction

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.8.4

How the design can grow

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.9.1

Data ownership and relationships

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.9.2

Follow the main operation

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.9.3

Follow the failure and its correction

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.9.4

How the design can grow

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.10.1

Data ownership and relationships

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.10.2

Follow the main operation

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.10.3

Follow the failure and its correction

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.10.4

How the design can grow

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.11.1

Data ownership and relationships

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.11.2

Follow the main operation

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.11.3

Follow the failure and its correction

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.11.4

How the design can grow

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.12.1

Data ownership and relationships

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.12.2

Follow the main operation

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.12.3

Follow the failure and its correction

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.12.4

How the design can grow

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 2.0.1

Request-volume timeline

Compare calculated magnitudes under the same explicit model; bar length is not a benchmark.

FIG. 2.0.2

throughput-latency curve

Queueing can dominate near a modeled capacity ceiling; this is calculated theory, not a benchmark.

FIG. 2.0.3

latency waterfall

Separate stage waits and execution rather than attach one unexplained number to the entire request.

FIG. 2.0.4

capacity planning worksheet

Compare the stated dimensions under one explicit contract.

FIG. 3.0.1

Complete HTTP request timeline

Follow explicit message ordering and locate the acknowledgment or effect boundary.

FIG. 3.0.2

TCP/TLS interaction

Follow explicit message ordering and locate the acknowledgment or effect boundary.

FIG. 3.0.3

load-balancing topology

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 3.0.4

WebSocket vs polling communication

Compare the stated dimensions under one explicit contract.

FIG. 4.0.1

Threads share a heap but have execution-local stacks

Make the internal structure or state relation visible before claiming its behavior.

FIG. 4.0.2

scheduling timeline

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 4.0.3

race-condition interleaving

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 4.0.4

A circular wait prevents both participants progressing

Make the internal structure or state relation visible before claiming its behavior.

FIG. 4.0.5

event-loop flow

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 5.0.1

Before/after class diagrams

Compare the stated dimensions under one explicit contract.

FIG. 5.0.2

dependency graph

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 5.0.3

inheritance vs composition

Compare the stated dimensions under one explicit contract.

FIG. 5.0.4

cohesion/coupling illustrations

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.0.1

Object creation timelines

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.0.2

Scope identities make lifetime semantics visible

Make the internal structure or state relation visible before claiming its behavior.

FIG. 6.0.3

UML class relationships

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.0.4

dependency inversion before/after

Compare the stated dimensions under one explicit contract.

FIG. 6.0.5

pattern comparisons

Compare the stated dimensions under one explicit contract.

FIG. 7.0.1

Layered vs Hexagonal

Compare the stated dimensions under one explicit contract.

FIG. 7.0.2

modular monolith vs microservices

Compare the stated dimensions under one explicit contract.

FIG. 7.0.3

dependency arrows

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 7.0.4

bounded context map

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 8.0.1

REST request flow

Follow explicit message ordering and locate the acknowledgment or effect boundary.

FIG. 8.0.2

API gateway topology

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 8.0.3

synchronous vs asynchronous API

Compare the stated dimensions under one explicit contract.

FIG. 8.0.4

payment state-machine diagram

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 9.0.1

Database taxonomy

Compare the stated dimensions under one explicit contract.

FIG. 9.0.2

physical storage representations

Compare the stated dimensions under one explicit contract.

FIG. 9.0.3

normalized vs denormalized schema

Compare the stated dimensions under one explicit contract.

FIG. 9.0.4

An order schema preserves relationships and history

Make the internal structure or state relation visible before claiming its behavior.

FIG. 10.0.1

B+tree lookup narrows an ordered key range

Make the internal structure or state relation visible before claiming its behavior.

FIG. 10.0.2

Two snapshots can legitimately see different versions

Make the internal structure or state relation visible before claiming its behavior.

FIG. 10.0.3

concurrent transaction interleavings

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 10.0.4

lock wait graph

Compare the stated dimensions under one explicit contract.

FIG. 10.0.5

WAL and recovery

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 11.0.1

Cache hit/miss sequence

Follow explicit message ordering and locate the acknowledgment or effect boundary.

FIG. 11.0.2

cache hierarchy

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 11.0.3

LRU and FIFO disagree after the same accesses

Make the internal structure or state relation visible before claiming its behavior.

FIG. 11.0.4

stampede timeline

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 11.0.5

geographic CDN delivery

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 12.0.1

Architecture evolution across four load levels

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 12.0.2

A new owner takes only part of the hash space

Make the internal structure or state relation visible before claiming its behavior.

FIG. 12.0.3

Averages hide a hot partition

Compare calculated magnitudes under the same explicit model; bar length is not a benchmark.

FIG. 12.0.4

load-balancer failover

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 13.0.1

Upload pipeline

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 13.0.2

An inverted index is a term-to-document map

Make the internal structure or state relation visible before claiming its behavior.

FIG. 13.0.3

ingestion DAG

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 13.0.4

batch vs stream timeline

Compare the stated dimensions under one explicit contract.

FIG. 13.0.5

lineage graph

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 14.0.1

A partition can leave two live but disconnected sides

Make the internal structure or state relation visible before claiming its behavior.

FIG. 14.0.2

causality timeline

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 14.0.3

consistency model comparisons

Compare the stated dimensions under one explicit contract.

FIG. 14.0.4

Intersecting quorums still need a protocol

Make the internal structure or state relation visible before claiming its behavior.

FIG. 15.0.1

Raft election changes authority under protocol rules

Make the internal structure or state relation visible before claiming its behavior.

FIG. 15.0.2

election timeline

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 15.0.3

Presence in a log is not the same as commitment

Make the internal structure or state relation visible before claiming its behavior.

FIG. 15.0.4

quorum intersection

Compare the stated dimensions under one explicit contract.

FIG. 15.0.5

network partition recovery

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 16.0.1

Queue lifecycle

Follow explicit message ordering and locate the acknowledgment or effect boundary.

FIG. 16.0.2

consumer partition assignment

Compare the stated dimensions under one explicit contract.

FIG. 16.0.3

Saga happy/failure paths

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 16.0.4

The outbox closes a local publication gap

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 16.0.5

event replay

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 17.0.1

Retry amplification timeline

Compare calculated magnitudes under the same explicit model; bar length is not a benchmark.

FIG. 17.0.2

A circuit breaker bounds futile calls

Make the internal structure or state relation visible before claiming its behavior.

FIG. 17.0.3

bulkhead isolation

Compare the stated dimensions under one explicit contract.

FIG. 17.0.4

failure-domain map

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 17.0.5

Recovery time and recoverable data are different

Make the internal structure or state relation visible before claiming its behavior.

FIG. 18.0.1

Tenant identity must survive every trust boundary

Make the internal structure or state relation visible before claiming its behavior.

FIG. 18.0.2

authentication sequence

Follow explicit message ordering and locate the acknowledgment or effect boundary.

FIG. 18.0.3

authorization decision flow

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 18.0.4

Tenant identity must survive every trust boundary

Make the internal structure or state relation visible before claiming its behavior.

FIG. 19.0.1

This slow request is dominated by waiting

Separate stage waits and execution rather than attach one unexplained number to the entire request.

FIG. 19.0.2

request ID propagation

Follow explicit message ordering and locate the acknowledgment or effect boundary.

FIG. 19.0.3

metrics-to-root-cause workflow

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 19.0.4

error-budget burn chart

Compare calculated magnitudes under the same explicit model; bar length is not a benchmark.

FIG. 20.0.1

VMs and containers isolate at different layers

Make the internal structure or state relation visible before claiming its behavior.

FIG. 20.0.2

Kubernetes cluster

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 20.0.3

CI/CD pipeline

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 20.0.4

blue-green deployment

Compare the stated dimensions under one explicit contract.

FIG. 20.0.5

Canary exposure expands only after verification

Compare calculated magnitudes under the same explicit model; bar length is not a benchmark.

FIG. 21.0.1

Performance bottleneck tree

Each branch is a hypothesis requiring profiling, traces, and workload evidence.

FIG. 21.0.2

throughput-latency curve

Queueing can dominate near a modeled capacity ceiling; this is calculated theory, not a benchmark.

FIG. 21.0.3

Architecture costs are not capacity guarantees

Compare calculated magnitudes under the same explicit model; bar length is not a benchmark.

FIG. 21.0.4

test pyramid

Test scope and feedback cost differ; choose by the failure evidence needed.

FIG. 21.0.5

design decision matrix

Compare the stated dimensions under one explicit contract.

THE FORMULA REFERENCE

Symbols with assumptions

Every indexed model is introduced in the lessons. Inputs and guarantee boundaries accompany the notation.

State transition

```
T(S, c) → (S′, result)
```

Identify valid state changes and preserved invariants. This abstracts away implementation and failures.

Derivation and worked example →

Weighted preference

```
score(a) = Σ wᵢ sᵢ(a) / Σ wᵢ
```

Use nonnegative weights with positive total. Compare only feasible designs. Scores are illustrative judgments, not benchmark results.

Derivation and worked example →

Request rate

```
λ = daily users × requests per user per day / 86,400
```

An average workload estimate. Peak factors must be assumed or measured separately.

Derivation and worked example →

Little's Law

```
L = λ W
```

Long-run stable averages with consistent boundaries and units. It is not a universal p99 prediction.

Derivation and worked example →

Amdahl's Law

```
speedup(N) = 1 / [(1 − p) + p/N]
```

Idealized fixed workload and parallel fraction p; excludes extra coordination overhead unless modeled.

Derivation and worked example →

Storage growth

```
bytes/day = records/day × bytes/record × retained copies
```

Account separately for indexes, compression, retention, metadata, and headroom.

Derivation and worked example →

Error allowance

```
allowed bad events = (1 − target fraction) × eligible events
```

Valid for an event-based SLO; time-based objectives need their own denominator.

Derivation and worked example →

Token refill

```
tokens′ = min(capacity, tokens + refill_rate × elapsed)
```

Admission consumes tokens atomically. Distributed clocks and state ownership need additional design.

Derivation and worked example →

Quorum intersection

```
R + W > N
```

An intersection condition for reads and writes on a fixed replica set. Not by itself a proof of linearizability.

Derivation and worked example →

Cost per outcome

```
cost / successful business outcomes
```

Define accounting period and include capacity, storage, networking, telemetry, and operational work where relevant.

Derivation and worked example →

Ideal M/M/1 mean residence

```
W = S / (1 − rho)
```

S is mean service time and rho is utilization, below one. This single-server model assumes Poisson arrivals and independent exponential service times. W includes waiting and service; it is a mean, so it cannot predict p99.

Derivation and worked example →

Fan-out slow probability

```
P(any slow) = 1 − (1 − p)^N
```

Requires independent equal-probability call outcomes in this model. Shared failures can correlate tails.

Derivation and worked example →

Retry amplification

```
maximum attempts = a^L
```

a allowed attempts at each of L nested independently retrying layers; actual execution can terminate earlier.

Derivation and worked example →

Busy-capacity worksheet

```
instances = ceil(peak RPS / (assumed capacity × utilization target))
```

Capacity is an explicit hypothetical or tested input; other bottlenecks and surviving failure capacity remain separate.

Derivation and worked example →

Error-budget burn

```
burn = observed bad fraction / allowed bad fraction
```

Use the same eligible-event denominator and a positive allowance.

Derivation and worked example →

Cache service-time average

```
E[T] = h × T_hit + (1 − h) × T_miss
```

A simple service-time mixture, excluding queue effects, dependence, and tail inference.

Derivation and worked example →

Backlog growth

```
growth/s = arrival/s − completion/s
```

Sustained positive growth requires rejection, reduction, or more useful capacity; a bigger queue only changes waiting.

Derivation and worked example →

Raw daily storage

```
bytes/day = records/s × bytes/record × 86400
```

Add indexes, history, metadata, replication, retention, compression and migration separately.

Derivation and worked example →

THE PATTERN INDEX

Start with the problem, then the pattern

Individual LLD patterns include their intent, collaborators, miniature implementation, costs, and alternatives.

6.1 · Dependencies, inversion & composition

Dependency Injection | Composition Root | Inversion of Control

6.4 · Creational patterns

Factory Method | Abstract Factory | Builder | Prototype | Singleton

6.5 · Structural patterns

Adapter | Bridge | Composite | Decorator | Facade | Flyweight | Proxy

6.6 · Behavioral patterns

Strategy | Observer | Command | State | Chain of Responsibility | Template Method | Iterator | Mediator | Memento | Visitor

6.7 · Enterprise patterns & boundaries

Repository | Unit of Work | Specification | CQRS

Application Architecture and Domain-Driven Design

Modular Monolith | Layered Architecture | Hexagonal Architecture | Clean Architecture | Anti-corruption Layer

Caching and Content Delivery

Cache-aside | Read-through | Write-through | Write-behind | Stale-while-revalidate

Message Queues, Events, and Distributed Workflows

Transactional Outbox | Saga | Event Sourcing | Orchestration | Choreography

Reliability, Fault Tolerance, and Disaster Recovery

Circuit Breaker | Bulkhead | Retry with Jitter | Load Shedding

Deployment, Cloud Infrastructure, and Platform Architecture

Rolling Deployment | Blue-green Deployment | Canary Release

THE READING ROOM

Source bibliography

Original research and specifications are distinguished from university teaching, implementation documentation, and engineering guidance.

Access and the stated use were reviewed on 21 October 2026. Versioned documentation is pinned where practical. A checked link does not prove a production system correct; worked workloads, costs, and architectures are original illustrative models. Recheck security guidance and implementation versions when applying them.

61 of 61 sources

OFFICIAL DOCUMENTATION · ACCESSED 21 OCT 2026

Atomic Commit in SQLite ↗

Commit and operating-system assumptions

Access reviewed · 21 October 2026

ORIGINAL STANDARD

RFC 9110: HTTP Semantics, June 2022 ↗

HTTP semantics; application commit policy is separate

Access reviewed · 21 October 2026

ORIGINAL RESEARCH · ACM DOI

On the criteria to be used in decomposing systems into modules — D. L. Parnas, 1972 ↗

Module decomposition criteria; metadata and abstract verified

Access reviewed · 21 October 2026

ORIGINAL ENGINEERING ARTICLE

Inversion of Control Containers and the Dependency Injection pattern — Martin Fowler, 2004 ↗

Dependency injection and wiring

Access reviewed · 21 October 2026

ORIGINAL PRACTICE ARTICLE

Documenting Architecture Decisions — Michael Nygard, 2011 ↗

ADR structure and decision history

Access reviewed · 21 October 2026

OFFICIAL GUIDANCE · ACCESSED 21 OCT 2026

AWS Well-Architected: the six pillars ↗

Review lenses, not correctness guarantees

Access reviewed · 21 October 2026

ORIGINAL MODEL DOCUMENTATION

C4: diagrams ↗

Formal diagram levels and supporting views

Access reviewed · 21 October 2026

ORIGINAL MODEL DOCUMENTATION

C4: container abstraction ↗

Meaning of container

Access reviewed · 21 October 2026

ENGINEERING BOOK

Implementing SLOs — The SRE Workbook ↗

Service indicators, objectives, and error budgets; not a source of the atlas workload numbers

Access reviewed · 21 October 2026

ORIGINAL RESEARCH · PUBLISHER RECORD

A Proof for the Queuing Formula: L = lambda W — John D. C. Little, 1961 ↗

Long-run queue accounting; publisher record inspected, full proof access may depend on access rights

Access reviewed · 21 October 2026

ORIGINAL RESEARCH

The Tail at Scale — Dean and Barroso, 2013 ↗

Tail-latency considerations; atlas probabilities and latency examples are author-calculated

Access reviewed · 21 October 2026

ORIGINAL SPECIFICATION

RFC 9000: QUIC Transport, 2021 ↗

QUIC streams and transport behavior; application walkthrough is author-created

Access reviewed · 21 October 2026

ORIGINAL SPECIFICATION

RFC 8446: TLS 1.3, 2018 ↗

Secure transport handshake and protocol scope

Access reviewed · 21 October 2026

UNIVERSITY TEXTBOOK

Operating Systems: Three Easy Pieces ↗

Runtime, concurrency, and persistence

Access reviewed · 21 October 2026

LANGUAGE SPECIFICATION · ACCESSED 21 OCT 2026

The Go Memory Model ↗

One concrete memory-model contract; other languages differ

Access reviewed · 21 October 2026

ENGINEERING TEACHING REFERENCE

The Catalog of Design Patterns — Refactoring Guru ↗

Pattern taxonomy; miniature examples and judgments are original to this atlas

Access reviewed · 21 October 2026

OFFICIAL GUIDANCE · ACCESSED 21 OCT 2026

Architecture styles — Microsoft Architecture Center ↗

Style comparisons; provider-specific recommendations are not mandatory in the atlas

Access reviewed · 21 October 2026

ORIGINAL ENGINEERING EXPLANATION

Bounded Context — Martin Fowler, 2014 ↗

Model boundaries and language

Access reviewed · 21 October 2026

ORIGINAL ENGINEERING ARTICLE

Microservices — James Lewis and Martin Fowler, 2014 ↗

Architecture characteristics and costs; no exact company architecture is asserted here

Access reviewed · 21 October 2026

PROJECT DOCUMENTATION

OWASP API Security Top 10, 2023 edition ↗

Threat taxonomy for Unit 18

Access reviewed · 21 October 2026

ORIGINAL PROTOCOL SECURITY GUIDANCE

RFC 9700: OAuth 2.0 Security Best Current Practice, 2025 ↗

OAuth threat and flow guidance; implementation should use maintained protocol libraries

Access reviewed · 21 October 2026

UNIVERSITY COURSES

Carnegie Mellon Database Group: courses ↗

Database internals reading

Access reviewed · 21 October 2026

OFFICIAL VERSIONED DOCUMENTATION

PostgreSQL 18: Index Types ↗

Supported access structures and their purposes

Access reviewed · 21 October 2026

OFFICIAL VERSIONED DOCUMENTATION

PostgreSQL 18: Transaction Isolation ↗

Specific PostgreSQL isolation behavior; no all-database equivalence claim

Access reviewed · 21 October 2026

OFFICIAL VERSIONED DOCUMENTATION

PostgreSQL 18: Write-Ahead Logging ↗

Logging and recovery ordering; durability remains configuration-dependent

Access reviewed · 21 October 2026

OFFICIAL IMPLEMENTATION DOCUMENTATION · ACCESSED 21 OCT 2026

RocksDB Overview ↗

An LSM-based engine; exact tuning is implementation-specific

Access reviewed · 21 October 2026

ORIGINAL STANDARD

RFC 9111: HTTP Caching, 2022 ↗

HTTP cache semantics; application cache races are separate examples

Access reviewed · 21 October 2026

OFFICIAL DOCUMENTATION · ACCESSED 21 OCT 2026

Redis: Key eviction ↗

Implementation policies can approximate ideal LRU/LFU

Access reviewed · 21 October 2026

OFFICIAL ENGINEERING EXPLANATION

Content delivery networks — Cloudflare Learning Center ↗

CDN role; atlas delivery and traffic numbers are illustrative

Access reviewed · 21 October 2026

OFFICIAL GUIDANCE · ACCESSED 21 OCT 2026

Application best practices — Microsoft Architecture Center ↗

Scaling and application boundary review

Access reviewed · 21 October 2026

ORIGINAL RESEARCH · AUTHOR ORGANIZATION

Dynamo: Amazon’s highly available key-value store, 2007 ↗

Replication, partitioning, and reconciliation ideas; capstones are original designs, not Dynamo deployments

Access reviewed · 21 October 2026

OFFICIAL SPECIFICATION DOCUMENTATION · 1.54.0 AT ACCESS

OpenLineage: core model and documentation ↗

Dataset, job, and run lineage concepts

Access reviewed · 21 October 2026

OFFICIAL DOCUMENTATION · ACCESSED 21 OCT 2026

Elasticsearch: Reindex documents ↗

Implementation reindex controls; full generation-switch protocol in the atlas is an original design

Access reviewed · 21 October 2026

OFFICIAL VERSIONED DOCUMENTATION

Apache Kafka 4.1: Design ↗

Partitioning and scoped transactional/processing behavior; external effects are separate

Access reviewed · 21 October 2026

PRIMARY PRACTITIONER RESEARCH REFERENCE

Jepsen: Consistency models ↗

Formal-history distinctions and references to underlying literature

Access reviewed · 21 October 2026

ORIGINAL RESEARCH

Time, Clocks, and the Ordering of Events in a Distributed System — Lamport, 1978 ↗

Ordering reference for Unit 14

Access reviewed · 21 October 2026

ORIGINAL RESEARCH · UNIVERSITY-HOSTED PAPER

Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services — Gilbert and Lynch, 2002 ↗

CAP assumptions and impossibility; PDF text extraction is imperfect, original page inspected

Access reviewed · 21 October 2026

UNIVERSITY COURSE

MIT 6.5840: Distributed Systems, Spring 2026 ↗

Advanced distributed-systems reading

Access reviewed · 21 October 2026

ORIGINAL RESEARCH

In Search of an Understandable Consensus Algorithm — Ongaro & Ousterhout ↗

Raft reference for Unit 15

Access reviewed · 21 October 2026

ORIGINAL RESEARCH

Paxos Made Simple — Leslie Lamport, 2001 ↗

Conceptual consensus safety; atlas does not present toy code as production Paxos

Access reviewed · 21 October 2026

ORIGINAL RESEARCH · 2017 REVISION

A Conflict-Free Replicated JSON Datatype — Kleppmann and Beresford ↗

One formal CRDT model; the miniature tie-break exercise is not this complete implementation

Access reviewed · 21 October 2026

OFFICIAL ENGINEERING GUIDANCE

Saga distributed transactions pattern — Microsoft ↗

Local transactions and compensation; compensation is not always a perfect inverse

Access reviewed · 21 October 2026

PATTERN AUTHOR'S EXPLANATION

Transactional Outbox — Chris Richardson ↗

Publication intent and duplicate delivery

Access reviewed · 21 October 2026

ENGINEERING BOOK

Handling Overload — Google SRE ↗

Admission, capacity, and overload response

Access reviewed · 21 October 2026

OFFICIAL GUIDANCE · ACCESSED 21 OCT 2026

Disaster recovery planning guide — Google Cloud ↗

Recovery planning, distinct from a measured achievement of an objective

Access reviewed · 21 October 2026

OFFICIAL GUIDANCE · 2024 REVISION AT ACCESS

Reliability Pillar — AWS Well-Architected ↗

Reliability review lenses and operations; provider-neutral teaching uses the concepts

Access reviewed · 21 October 2026

OFFICIAL PROJECT GUIDANCE · ACCESSED 21 OCT 2026

OWASP: Authorization ↗

Least privilege, deny by default, and per-resource enforcement

Access reviewed · 21 October 2026

OFFICIAL PROJECT GUIDANCE · ACCESSED 21 OCT 2026

OWASP: Password Storage ↗

Password-specific hashing and current deployment parameter guidance

Access reviewed · 21 October 2026

ORIGINAL SPECIFICATION · DECEMBER 2023

OpenID Connect Core 1.0, errata set 2 ↗

Identity layer and validation requirements; distinct from API authorization

Access reviewed · 21 October 2026

OFFICIAL DOCUMENTATION · ACCESSED 21 OCT 2026

OpenTelemetry: Signals ↗

Complementary observability signals

Access reviewed · 21 October 2026

OFFICIAL DOCUMENTATION · ACCESSED 21 OCT 2026

OpenTelemetry: Context propagation ↗

Trace context relationships; trace identity does not authorize access

Access reviewed · 21 October 2026

ENGINEERING BOOK

Alerting on SLOs — The SRE Workbook ↗

Budget burn and actionable alerts

Access reviewed · 21 October 2026

ORIGINAL METHODOLOGY

The Twelve-Factor App ↗

Application deployment conventions

Access reviewed · 21 October 2026

OFFICIAL DOCUMENTATION · ACCESSED 21 OCT 2026

Kubernetes components ↗

Platform reference for Unit 20

Access reviewed · 21 October 2026

ENGINEERING BOOK

Canarying Releases — The SRE Workbook ↗

Representative rollout evaluation and stop conditions

Access reviewed · 21 October 2026

ORIGINAL ENGINEERING ARTICLE

The Practical Test Pyramid — Ham Vocke ↗

Test scope and feedback trade-offs, not a mandatory test-count ratio

Access reviewed · 21 October 2026

ORIGINAL PROTOCOL DESCRIPTION · INFORMATIONAL RFC

RFC 2697: A Single Rate Three Color Marker, 1999 ↗

Token-based traffic metering example; atlas admission bucket is a simpler model

Access reviewed · 21 October 2026

ORIGINAL STANDARD

RFC 6455: The WebSocket Protocol, 2011 ↗

Bidirectional transport framing; not durable chat delivery

Access reviewed · 21 October 2026

ORIGINAL PROTOCOL DESCRIPTION · INFORMATIONAL RFC

RFC 8216: HTTP Live Streaming, 2017 ↗

One adaptive streaming protocol; do not confuse Informational RFC with Standards Track

Access reviewed · 21 October 2026

OFFICIAL IMPLEMENTATION DOCUMENTATION · ACCESSED 21 OCT 2026

PostGIS: ST_DWithin ↗

Coordinate-system-aware proximity predicate, not exclusive dispatch assignment

Access reviewed · 21 October 2026

ENGINEERING BOOK

Distributed Periodic Scheduling — Google SRE ↗

Durable scheduling concerns; atlas schedule architecture is an original worked design

Access reviewed · 21 October 2026

No sources match that search.

PRODUCTION AND EVIDENCE

Completion manifest

All units are produced. Coverage, technical review, executable tests, and browser verification are separate kinds of evidence.

COMPLETE FIRST EDITION

22 units · 99 chapters · 390 exact syllabus items · 12 full case studies · 242 questions with separate explained solutions.

Unit	Chapters produced	Concept / case coverage	Questions / answers
1 · Introduction to System Design	4 / 4	13	11 / 11
2 · Requirements, Estimation, and Engineering Trade-offs	4 / 4	15	11 / 11
3 · Networking Foundations for System Design	4 / 4	17	11 / 11
4 · Runtime, Processes, Threads, and Concurrency	4 / 4	17	11 / 11
5 · Object-Oriented and Modular Software Design	4 / 4	15	11 / 11
6 · Design Patterns, Dependency Injection, and Object Lifecycles	7 / 7	18	11 / 11
7 · Application Architecture and Domain-Driven Design	4 / 4	20	11 / 11
8 · API Design and Communication Contracts	4 / 4	19	11 / 11

Unit	Chapters produced	Concept / case coverage	Questions / answers
9 · Data Models and Storage Technologies	4 / 4	19	11 / 11
10 · Database Internals, Transactions, and Concurrency Control	4 / 4	18	11 / 11
11 · Caching and Content Delivery	4 / 4	17	11 / 11
12 · Scalability, Load Balancing, and Data Distribution	4 / 4	22	11 / 11
13 · File Systems, Search Infrastructure, and Data Pipelines	4 / 4	17	11 / 11
14 · Distributed Systems Fundamentals	4 / 4	20	11 / 11
15 · Replication, Leader Election, and Consensus	4 / 4	17	11 / 11
16 · Message Queues, Events, and Distributed Workflows	4 / 4	22	11 / 11
17 · Reliability, Fault Tolerance, and Disaster Recovery	4 / 4	22	11 / 11
18 · Security Architecture and Privacy	4 / 4	22	11 / 11
19 · Observability, Monitoring, and Incident Engineering	4 / 4	19	11 / 11
20 · Deployment, Cloud Infrastructure, and Platform Architecture	4 / 4	21	11 / 11
21 · Performance, Cost Optimization, and System Validation	4 / 4	20	11 / 11

Unit	Chapters produced	Concept / case coverage	Questions / answers
22 · End-to-End Design, Architecture Reviews, and Evolution	12 / 12	12 × 22 sections	11 / 11

What the evidence means

Content checks establish presence, exact syllabus allocation, figures, references, and assessment. Executable examples test bounded arithmetic and local invariants. Browser checks test reading, interactions, layouts, offline behavior, and print assembly. The book does not report illustrative objectives as measured production achievements or toy protocols as reviewed production consensus, security, or financial integrations.

The original request is retained in BRIEF.txt. Additional optional case subjects are suggestions beyond the twelve mandatory worked designs, not omitted required chapters.